

Newton, *Principia*, with its Lean reconstruction interleaved

The 1687 Definitions, Laws and Book I Sections I-II in modern language, each
item followed by the theorems that formalize it

October 2026

Contents

About this rendering	1
How to read this version	2
How the editions differ	2
Definitions	3
Scholium	6
Axioms, or Laws of Motion	9
Scholium	18
Book I: On the Motion of Bodies	20
Section I. On the method of first and last ratios, by whose help what follows is demonstrated	20
Section II. On finding centripetal forces	37
Appendix A. Modules anchored outside Sections I-II	139
Appendix B. The foundation: modules with no source anchor	140

About this rendering

This is a rendering of the first edition (1687) of Newton's *Philosophiæ Naturalis Principia Mathematica*, made from the Newton Project TEI transcriptions kept in this repository: the Definitions and their Scholium (docs/m1/NATP00075.xml), the Axioms or Laws of Motion (NATP00076.xml) and Book I (NATP00077.xml). It covers the Definitions, the Laws with their Corollaries and Scholium, Book I Section I (Lemmas I-XI and the Scholium), and Book I Section II through Proposition IV and its Scholium. Propositions V-X, which close Section II, are not included.

Every item keeps Newton's number, order and concepts. Statements are put into modern mathematical language; proofs follow Newton's steps and add none. Where a mod-

ern formula helps, it follows Newton’s statement under *In modern terms*. Newton’s figures are not reproduced; the letters refer to them, and the descriptions are enough to redraw them. Square brackets mark the few notes added for a modern reader. Long physical or historical explanations are condensed, in Newton’s order.

The changes of the second edition (1713) and the third (1726) follow each item, marked **1713** and **1726**. They come from the Newton Project TEI of the 1713 Definitions, Laws and Book I (NATP00080–NATP00082, in this repository) and of the 1726 Definitions, Laws and Book I (NATP00085–NATP00087, in docs/m4). The three editions were aligned paragraph by paragraph; changes of spelling, punctuation and abbreviation are omitted, and only changes of content or of mathematical language are given. A summary of the pattern follows.

Vocabulary.

Newton	Modern meaning
ratio of equality; <i>ultimately equal</i>	$X/Y \rightarrow 1$ in the stated limit
ultimate ratio of vanishing quantities	$\lim X/Y$ as $X, Y \rightarrow 0$
first ratio of nascent quantities	$\lim X/Y$ as X, Y start from 0
duplicate, triplicate ratio	ratio of squares, of cubes
subduplicate, sesquuplicate ratio	ratio of square roots, of $3/2$ powers
<i>A applied to B</i>	A/B
<i>componendo</i>	summing proportional terms
<i>in consequentia</i>	in the direction of motion

Bold letters are vectors; $\times$ is the planar cross product $\mathbf{a} \times \mathbf{b} = a_1 b_2 - a_2 b_1$.

How to read this version

After each Definition, Law, Lemma, Proposition or Scholium, the theorems of the Lean reconstruction whose catalogued source passages cite that item are printed as complete modules in a smaller monospace face: definitions, docstrings, statements and proofs, verbatim, in import order. The catalogue (research/formal-results.json) assigns each theorem to Newton Project paragraph anchors; a module whose theorems cite several items is printed under the item most of them cite, with a cross-reference under the others. Items that no theorem cites say so. The Lean is a modern reconstruction in rational arithmetic with Lean 4 core only: it supplies no historical premise, and its appearance under an item records that the item motivated it, not that the item is thereby proved. Two appendices print, in full, the modules anchored outside the rendered range and the foundation modules with no source anchor, which are most of the code. Every line of the two libraries appears exactly once; the verification harnesses under research/verification are not included.

How the editions differ

1713, the limiting arguments are tightened. Lemma I now says that the quantities actually come nearer than any given difference before the end of the finite time. Lemma

VI gets a new proof from the continuity of the curvature. The proofs of Lemmas VII-IX enlarge the vanishing figure so that its lines stay finite while the original shrinks.

1713, regularity premises enter the statements. Lemma X, which assumed a “regular” force, now assumes a finite force, either constant or continually increasing or decreasing. Lemma XI is restricted to curves with finite curvature at the point of contact.

1713, the sagitta becomes the measure of force. Lemma XI gains corollaries on sagittas; Proposition I gains six corollaries measuring force by the sagitta of arcs described in equal times; Proposition IV is proved anew through the versed sines of those arcs, and its Scholium compares centripetal force with gravity through the new Corollary IX.

1713, the language of proportion is defined, and an error is corrected. A new Scholium after Lemma X states what “A is as B directly and C inversely” means, and Lemma XI defines the sesquiplicate ratio. Proposition IV’s Corollary 6 is corrected and generalized to any power of the radius.

1713, the Definitions and Laws. Definition V gains a long explanation (the sling, the projectile fired from a mountain, the Moon, and the inverse problems of mathematics). Corollary I of the Laws specifies impulses impressed at one place, and Law III is extended explicitly to attractions, with a new argument for the mutual gravity of the parts of the Earth.

1726, products and quotients give way to compound ratios. Where 1713 said “multiplied into” or “applied to”, 1726 often says “jointly” or “in the ratio compounded of” (Definition VIII’s explanation, Proposition IV’s Corollaries 1 and 2, the reflection argument after Proposition IV). The Scholium to the Laws gains an explanation of Galileo’s results by equal impulses in equal particles of time.

Definitions

Definition I. *The quantity of matter is its measure, arising from its density and its volume jointly.*

In modern terms: $m \propto \rho V$.

Air of double density in double the space is quadruple; likewise snow or powders condensed by compression or melting, and all bodies condensed by any cause. A medium freely pervading the interstices of the parts, if there is one, is disregarded. This quantity is meant below by *body* or *mass*. It is known through each body’s weight, to which it is proportional, as very accurate pendulum experiments show (shown later).

1713. The example adds: in triple the space, sextuple.

No theorem of the Lean reconstruction cites this item. Not encoded. The model has no mass: forces are accelerative (Definition VII). A ‘mass’ parameter appears only in the potential and circular diagnostics (‘Diagnostic/ConstructedHarmonicPotential’, ‘Comparison/CircleCompare’).

Definition II. *The quantity of motion is its measure, arising from the velocity and the quantity of matter jointly.*

In modern terms: $\mathbf{p} = m\mathbf{v}$.

The motion of the whole is the sum of the motions of the parts; a body twice as large with equal velocity has double the motion, and with double velocity quadruple.

No theorem of the Lean reconstruction cites this item. Not encoded; with mass absent, velocity stands in for quantity of motion.

Definition III. *The inherent force of matter (vis insita) is a power of resisting, by which every body, as far as it can, perseveres in its state of rest or of uniform motion in a straight line.*

It is always proportional to the body and differs from the inertia of mass only in the way it is conceived; hence it may be called the *force of inertia*. A body exerts it only when another force impressed on it changes its state. The exertion is, from different points of view, *resistance* (the body resists the impressed force to keep its state) and *impetus* (the body, yielding with difficulty to an obstacle, tries to change the obstacle's state). Common usage gives resistance to bodies at rest and impetus to moving ones; but rest and motion, as commonly conceived, differ only relatively.

No theorem of the Lean reconstruction cites this item. Encoded, not proved: the inertial drift 'pointAdd s.1 (pointScale d s.2)' that opens every cell ('CentralSchedule.cell'), and 'ZeroForce.inertialAt'.

Definition IV. *Impressed force is an action exerted on a body to change its state of rest or of uniform motion in a straight line.*

It consists in the action alone and does not remain in the body afterwards; the body perseveres in any new state by the force of inertia alone. Impressed force has various origins: percussion, pressure, centripetal force.

No theorem of the Lean reconstruction cites this item. Encoded, not proved: the velocity kick 'pointAdd s.2 (pointScale d (a y))' at the arrival vertex of every cell ('CentralSchedule.cell', 'TimeSubdivision.endKick').

Definition V. *Centripetal force is that by which a body is drawn, impelled, or in any way tends toward some point as to a centre.*

1713. *Bodies (plural) are drawn, impelled or tend toward the point from all sides.*

Examples: gravity, toward the centre of the Earth; magnetic force, by which iron seeks a magnet; and the force, whatever it is, by which the planets are continually drawn back from rectilinear motions and made to revolve in curved lines. Its quantity is of three kinds: absolute, accelerative and motive.

1713. The explanation is much expanded. A stone whirled in a sling endeavours to leave the hand, stretches the sling the more strongly the faster it revolves, and flies off when released; the contrary force by which the sling keeps drawing the stone back toward the hand, the centre of the orbit, is called centripetal. The same holds for all bodies moving in orbits: without a contrary force they would go off in straight lines with uniform motion. A projectile without gravity would go off in a straight line into the heavens; gravity bends it continually toward the Earth, more or less according to its gravity and velocity. A lead ball fired horizontally from a mountain top that lands two miles away would, with double or tenfold velocity, go about twice or ten

times as far (air resistance removed); by increasing the velocity, the distance could be increased at will and the curvature of its path decreased, so that it would land ten, thirty or ninety degrees away, or go round the whole Earth before falling, or never fall and go off into the heavens. In the same way the Moon, by gravity or some other force urging it toward the Earth, can be drawn continually from its rectilinear course and bent into its orbit; with too small a force it would not be bent enough, with too great a force too much. It is for mathematicians to find the force that keeps a body exactly in a given orbit with a given velocity, and conversely to find the curved path into which a body leaving a given place with a given velocity is bent by a given force.

1726. “The less its gravity *for its quantity of matter*”, and the last alternative reads simply “or go off into the heavens”.

No theorem of the Lean reconstruction cites this item. Encoded as the predicate ‘CentralSchedule.central’ ($\det p (a \ p) = 0$: the force is parallel to the radius) and the ‘inward’ field of ‘ForceClasses.CentralOracle’ (every sample a nonnegative multiple of $-p$).

Definition VI. *The absolute quantity of a centripetal force is its measure, greater or less according to the efficacy of the cause propagating it from the centre through the surrounding regions.*

For instance, magnetic power is greater in one magnet and less in another.

1713. “according to the size of the magnet or the intensity of its power”.

No theorem of the Lean reconstruction cites this item. Not encoded.

Definition VII. *The accelerative quantity of a centripetal force is its measure proportional to the velocity it generates in a given time.*

In modern terms: at a given place, $a \propto \Delta v$ for a given Δt .

The same magnet acts more strongly at a smaller distance; gravity is greater in valleys and less on high mountain tops (pendulum experiments), and less still at greater distances from the Earth (shown later). At equal distances it is the same on all sides, because all falling bodies, heavy or light, large or small, are equally accelerated once air resistance is removed.

No theorem of the Lean reconstruction cites this item. Encoded: a force is a map ‘Field := Point → Point’ from positions to accelerations (‘CentralSchedule’), or its rational samples with error (‘ForceClasses.Oracle’).

Definition VIII. *The motive quantity of a centripetal force is its measure proportional to the motion it generates in a given time.*

In modern terms: $F \propto \Delta p$ for a given Δt .

Weight is greater in a greater body and less in a smaller; in the same body, greater near the Earth and less in the heavens. This quantity is the centripetency of the whole body, its propensity toward the centre, its weight; it is always known by the equal and contrary force that can prevent the body’s descent.

These quantities may be called briefly *absolute*, *accelerative* and *motive forces*, and referred respectively to the centre, to the place of the body, and to the body. The

motive force belongs to the body as the endeavour of the whole toward the centre, compounded of the endeavours of all its parts; the accelerative force belongs to the place, as an efficacy diffused from the centre through each surrounding place to move the bodies there; the absolute force belongs to the centre, as endowed with some cause without which motive forces are not propagated, whether that cause is a central body or something else that does not appear. The concept is mathematical only; physical causes and seats of forces are not considered here.

Hence accelerative force is to motive force as velocity is to motion: the quantity of motion is velocity times quantity of matter, and motive force is accelerative force times the same quantity of matter.

In modern terms: $F = ma$ (as a proportionality).

1726. Motive force arises from the accelerative force *and* the quantity of matter *jointly*, and weight is as the body and the accelerative gravity *jointly*, where 1687 and 1713 say “multiplied into”.

The sum of the actions of the accelerative force on the single particles of a body is the motive force of the whole. Near the Earth’s surface, where accelerative gravity is the same in all bodies, weight is as the body; where accelerative gravity is smaller, weight decreases in proportion and is always as the body times accelerative gravity. Where accelerative gravity is half as large, a body half or a third as large weighs four or six times less.

Attractions and impulses are called accelerative and motive in the same sense. The words attraction, impulse or propensity toward a centre are used indifferently, the forces being considered mathematically and not physically. The reader should not take them to define a kind or mode of action or a physical cause, nor to attribute forces truly and physically to centres, which are mathematical points.

No theorem of the Lean reconstruction cites this item. Not encoded (no mass).

Scholium

Time, space, place and motion are not defined, being well known to all. But the common understanding conceives them only through their relation to sensible things, which breeds prejudices; to remove these, each is divided into absolute and relative, true and apparent, mathematical and common.

I. *Absolute, true and mathematical time*, of itself and by its nature without relation to anything external, flows equably, and is also called *duration*. *Relative, apparent and common time* is any sensible, external measure of duration by motion, accurate or not, used in place of true time: an hour, a day, a month, a year.

II. *Absolute space*, by its nature without relation to anything external, remains always similar and immovable. *Relative space* is any movable measure or dimension of this space, determined by our senses through its position relative to bodies and commonly taken for immovable space, such as the dimension of a subterranean, aerial or celestial space fixed by its position relative to the Earth. Absolute and relative space are the same in kind and magnitude but do not always remain numerically the same: if the Earth moves, the space of our air, always the same

relative to the Earth, is now one part of absolute space and now another.

- III. *Place* is the part of space a body occupies, absolute or relative according to the space. It is a part of space, not the position of the body or its surrounding surface: equal solids always have equal places, while their surfaces, for dissimilar figures, are mostly unequal; positions, properly speaking, have no quantity and are affections of places rather than places. Since the motion of the whole equals the sum of the motions of the parts, the place of the whole equals the sum of the places of the parts, and is internal and in the whole body.
- IV. *Absolute motion* is the translation of a body from one absolute place to another; *relative motion*, from one relative place to another. In a ship under sail, a body's relative place is the region of the ship it occupies, which moves with the ship; relative rest is remaining in that region; true rest is remaining in the same part of immovable space. If the Earth truly rests, a body resting relative to the ship moves truly with the ship's velocity on the Earth; if the Earth also moves, the true motion arises partly from the Earth's true motion in immovable space and partly from the ship's relative motion; and if the body moves within the ship, from all three. Example: if the Earth's surface there moves truly east with 10010 parts of velocity, the ship west with 10 parts, and a sailor walks east with 1 part, the sailor moves truly east with 10001 parts and relatively to the Earth west with 9 parts.

Absolute time is distinguished from relative time in astronomy by the equation of time: natural days are unequal, and astronomers correct this to measure celestial motions by a truer time. Perhaps no equable motion exists by which time can be measured exactly; all motions can speed up or slow down, but the flow of absolute time cannot change. Duration is the same whether motions are fast, slow or absent. The need for the equation is shown by pendulum clocks and by the eclipses of Jupiter's satellites.

As the order of the parts of time is immutable, so is the order of the parts of space. Times and spaces are, as it were, the places of themselves and of all things: all things are placed in time as to order of succession and in space as to order of position. It is of their essence to be places, and for primary places to move is absurd. These are absolute places, and only translations from them are absolute motions.

These parts of space cannot be seen or distinguished by our senses, so sensible measures are used instead: places are defined by positions and distances from some body regarded as immovable, and motions estimated with respect to those places. In human affairs this serves; in philosophy one must abstract from the senses, since possibly no body truly rests to which places and motions could be referred.

Absolute and relative rest and motion are distinguished by their properties, causes and effects.

Properties. Bodies truly at rest rest among themselves; but since some body far away, among or beyond the fixed stars, may rest absolutely while the positions of bodies in our regions cannot tell us whether any of them keeps a given position to it, true rest cannot be defined from their mutual positions. Parts that keep given positions in their wholes share the wholes' motions: the parts of a revolving body endeavour to recede from the axis, and the impetus of a progressing body arises from the joint impetus of its parts. Hence when surrounding bodies move, bodies at rest relative to them move too, and true motion cannot be defined by translation from the neighbourhood of bodies

regarded as resting; those bodies must truly rest. Likewise, when a place moves, what is in it moves with it; a body moving from a moving place shares its place's motion. Every whole and absolute motion is compounded of the motion of the body from its first place, of that place from its place, and so on, until an immovable place is reached. Whole and absolute motions can therefore be defined only by immovable places, and immovable places are only those that keep given positions to each other from infinity to infinity, which constitute immovable space.

Causes. True motion is neither generated nor changed except by forces impressed on the moving body itself; relative motion can be generated or changed without forces impressed on it, by forces impressed only on the bodies of reference. True motion always changes under forces impressed on the moving body; relative motion need not, if the same forces act on the bodies of reference so as to keep the relative position.

Effects. The forces of receding from the axis of circular motion distinguish absolute from relative motion: in purely relative circular motion they are null, in true motion greater or less according to the quantity of motion. A bucket hung by a long cord is turned until the cord is stiffly twisted, filled with water, and released. At first the surface of the water is flat, as before the motion; as the bucket gradually communicates its motion to the water, the water recedes from the middle and rises at the sides, becoming concave, and rises more as its motion grows, until it revolves in equal times with the bucket and rests relative to it. The rise shows the endeavour to recede from the axis, and by it the true and absolute circular motion of the water is known and measured, quite contrary to the relative motion: at the start, when the relative motion was greatest, there was no such endeavour and no true circular motion.

Relative quantities are therefore not the quantities whose names they bear but their sensible measures. Those who read these words in Scripture as the measured quantities do it violence, and those who confuse true quantities with their relations and common measures corrupt mathematics and philosophy.

To know the true motions of single bodies and distinguish them from apparent ones is very difficult, because the parts of immovable space do not come under the senses. But the matter is not hopeless: arguments are available partly from apparent motions, which are differences of true motions, and partly from forces, which are causes and effects of true motions. Two globes joined by a cord at a given distance and revolving about their common centre of gravity show, by the cord's tension, their endeavour to recede from the axis, from which the quantity of circular motion can be computed. Impressing equal forces on alternate faces to increase or decrease the motion, the change in tension shows which faces must be pushed to increase the motion most, namely the trailing faces, and hence the direction of the motion. In this way the quantity and direction of the circular motion could be found even in an immense vacuum with nothing external and sensible to compare. How true motions are inferred from their causes, effects and apparent differences, and conversely, is taught more fully in what follows; the treatise was composed for this purpose.

No theorem of the Lean reconstruction cites this item. Absolute time is the rational parameter of every schedule and, in the completed layer, the constructed 'BinaryTime' quotient; absolute space is the rational plane 'Point := Fraction × Fraction' ('TimeSubdivision'). Relative motion appears only as the uniformly moving centre of Proposition II, Case 2.

Axioms, or Laws of Motion

Law I. *Every body perseveres in its state of rest or of uniform motion in a straight line, except insofar as it is compelled by impressed forces to change that state.*

In modern terms: with no impressed force, $\mathbf{v}$ is constant.

Projectiles persevere in their motions except as air resistance retards them and gravity impels them downward. A top, whose cohering parts continually draw each other back from rectilinear motions, keeps spinning except as the air retards it. The greater bodies of planets and comets keep their progressive and circular motions longer in less resisting spaces.

Lean reconstruction: 37 theorems in 3 modules cite this item; the modules follow in full, in import order.

Also encoded as the drift in every cell, and stated as ‘RelativeMotion.lawI_uniform’ (printed under Proposition III, which cites it).

NewtonLimitDynamics/Polygon/ZeroForce.lean, 22 theorems, 215 lines

```
import NewtonLimitDynamics.Polygon.PartitionControl

namespace NewtonLimitDynamics.Polygon.ZeroForce

open NewtonLimitDynamics
open TimeSubdivision
open PartitionControl

/-- The zero impressed acceleration used in the finite end-kick recurrence. -/
def zeroPoint : Point := (Fraction.ofInt 0, Fraction.ofInt 0)

/-- The rational-time comparison map for an inertial state. This is built
    from the finite recurrence below; it is not an assumed continuum curve. -/
def inertialAt (p v : Point) (t : Fraction) : Point :=
  pointAdd p (pointScale t v)

private theorem pointEquiv_refl (p : Point) : pointEquiv p p :=
  (Fraction.equiv_refl _, Fraction.equiv_refl _)

private theorem inertial_add_scalar (p v s t : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul s v)) (Fraction.mul t v))
    (Fraction.add p (Fraction.mul (Fraction.add s t) v)) := by
  have hsum : Fraction.equiv (Fraction.add (Fraction.mul s v) (Fraction.mul t v))
    (Fraction.mul (Fraction.add s t) v) := by
    exact Fraction.equiv_trans
      (Fraction.equiv_trans
        (Fraction.add_equiv_right (Fraction.mul t v) (Fraction.mul_comm s v))
        (Fraction.add_equiv_left (Fraction.mul v s) (Fraction.mul_comm t v)))
      (Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_add v s t))
        (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add s t) v)))
    exact Fraction.equiv_trans (Fraction.add_assoc p (Fraction.mul s v) (Fraction.mul t v))
      (Fraction.add_equiv_left p hsum)

/-- Inertial rational-time evolution joins by addition of elapsed times. -/
theorem inertialAt_add (p v : Point) (s t : Fraction) :
  pointEquiv (inertialAt (inertialAt p v s) v t) (inertialAt p v (Fraction.add s t)) := by
  constructor <=> apply inertial_add_scalar

private theorem zero_scale (d : Fraction) : pointEquiv (pointScale d zeroPoint) zeroPoint := by
  constructor <=> unfold pointScale zeroPoint Fraction.equiv Fraction.mul Fraction.ofInt <=> dsimp <=>
    ↪ simp

private theorem pointAdd_zero (p : Point) : pointEquiv (pointAdd p zeroPoint) p := by
  constructor <=> unfold pointAdd zeroPoint Fraction.equiv Fraction.add Fraction.ofInt <=> dsimp <=> simp
```

```

/-- One actual zero-force cell is exactly a drift at its stated Fraction time,
and leaves velocity unchanged. -/
theorem endKick_zero (d : Fraction) (state : Point × Point) :
  pointEquiv (endKick d state zeroPoint).1 (inertialAt state.1 state.2 d) ∧
  pointEquiv (endKick d state zeroPoint).2 state.2 := by
  constructor
  · exact pointEquiv_refl _
  · change pointEquiv (pointAdd state.2 (pointScale d zeroPoint)) state.2
    exact pointEquiv_trans (pointAdd_congr (pointEquiv_refl _) (zero_scale d)) (pointAdd_zero state.2)

/-- Replacing a rational time by an equivalent fraction leaves its inertial
position unchanged. -/
theorem inertialAt_time_congr (p v : Point) {s t : Fraction} (h : Fraction.equiv s t) :
  pointEquiv (inertialAt p v s) (inertialAt p v t) := by
  apply pointAdd_congr (pointEquiv_refl _)
  constructor
  · exact Fraction.equiv_trans (Fraction.mul_comm s v.1)
    (Fraction.equiv_trans (Fraction.mul_equiv_left v.1 h) (Fraction.equiv_symm (Fraction.mul_comm t
    ↪ v.1)))
  · exact Fraction.equiv_trans (Fraction.mul_comm s v.2)
    (Fraction.equiv_trans (Fraction.mul_equiv_left v.2 h) (Fraction.equiv_symm (Fraction.mul_comm t
    ↪ v.2)))

private theorem encodedPosition_zero (D : Nat) (hD : 0 < D) (s : PartitionStats) (p v : Point) :
  pointEquiv (encodedPosition D hD s p v zeroPoint)
    (inertialAt p v (duration D s.T hD)) := by
  unfold encodedPosition inertialAt
  exact pointEquiv_trans
    (pointAdd_congr (pointEquiv_refl _)
      (zero_scale (squareDuration D s.A hD)))
    (pointAdd_zero _)

private theorem encodedVelocity_zero (D : Nat) (hD : 0 < D) (s : PartitionStats) (v : Point) :
  pointEquiv (encodedVelocity D hD s v zeroPoint) v := by
  unfold encodedVelocity
  exact pointEquiv_trans
    (pointAdd_congr (pointEquiv_refl _)
      (zero_scale (duration D s.T hD)))
    (pointAdd_zero _)

/-- Every actual finite zero-force schedule reaches the inertial map at its
elapsed rational time and retains its incoming velocity. -/
theorem partitionMotion_zero_force (D : Nat) (hD : 0 < D) (p v : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p v zeroPoint weights).1
    (inertialAt p v (duration D (total weights) hD)) ∧
  pointEquiv (partitionMotion D hD p v zeroPoint weights).2 v := by
  have hformula := partitionMotion_formula D hD p v zeroPoint weights
  constructor
  · exact pointEquiv_trans hformula.1
    (encodedPosition_zero D hD (stats weights) p v)
  · exact pointEquiv_trans hformula.2
    (encodedVelocity_zero D hD (stats weights) v)

/-- Addition of two common-denominator elapsed times represents their summed
numerator. -/
theorem duration_add (D a b : Nat) (hD : 0 < D) :
  Fraction.equiv (Fraction.add (duration D a hD) (duration D b hD))
    (duration D (a + b) hD) := by
  unfold Fraction.equiv Fraction.add duration
  dsimp
  simp only [Int.ofNat_add, Int.mul_add, Int.add_mul]
  ac_rfl

/-- The finite recurrence itself restarts exactly: this is `foldl_append`, not
an assumption about a background curve. -/
theorem partitionMotion_append (D : Nat) (hD : 0 < D) (p v : Point)
  (ws xs : List Nat) :
  partitionMotion D hD p v zeroPoint (ws ++ xs) =
  partitionMotion D hD (partitionMotion D hD p v zeroPoint ws).1

```

```

    (partitionMotion D hD p v zeroPoint ws).2 zeroPoint xs := by
  unfold partitionMotion
  rw [List.foldl_append]

private theorem inertialAt_state_congr {p p' v v' : Point} (hp : pointEquiv p p')
  (hv : pointEquiv v v') (t : Fraction) :
  pointEquiv (inertialAt p v t) (inertialAt p' v' t) :=
  pointAdd_congr hp (pointScale_congr t hv)

/-- Drifting for a rational amount `r` from the actual prefix state agrees
  with the inertial map at elapsed prefix time plus `r`. -/
theorem withinCell_position (D : Nat) (hD : 0 < D) (p v : Point)
  (pre : List Nat) (r : Fraction) :
  pointEquiv (endKick r (partitionMotion D hD p v zeroPoint pre) zeroPoint).1
  (inertialAt p v (Fraction.add (duration D (total pre) hD) r)) := by
  have hp := partitionMotion_zero_force D hD p v pre
  have hkick := endKick_zero r (partitionMotion D hD p v zeroPoint pre)
  exact pointEquiv_trans hkick.1
  (pointEquiv_trans (inertialAt_state_congr hp.1 hp.2 r)
  (inertialAt_add p v (duration D (total pre) hD) r))

/-- The same algebra applies to an in-cell physical drift; the displayed
  inequalities express that `r` lies between the prefix vertex and the next
  cell endpoint and are not used as algebraic premises. -/
theorem withinCell_position_bounded (D w : Nat) (hD : 0 < D) (p v : Point)
  (pre : List Nat) (r : Fraction)
  (_hr0 : Fraction.le (Fraction.ofInt 0) r)
  (_hrCell : Fraction.le r (duration D w hD)) :
  pointEquiv (endKick r (partitionMotion D hD p v zeroPoint pre) zeroPoint).1
  (inertialAt p v (Fraction.add (duration D (total pre) hD) r)) :=
  withinCell_position D hD p v pre r

/-- Rest is the zero-velocity specialization of the actual finite recurrence. -/
theorem partitionMotion_rest (D : Nat) (hD : 0 < D) (p : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p zeroPoint zeroPoint weights).1 p ∧
  pointEquiv (partitionMotion D hD p zeroPoint zeroPoint weights).2 zeroPoint := by
  have h := partitionMotion_zero_force D hD p zeroPoint weights
  constructor
  · exact pointEquiv_trans h.1 (by
    unfold inertialAt
    exact pointEquiv_trans
    (pointAdd_congr (pointEquiv_refl _) (zero_scale (duration D (total weights) hD)))
    (pointAdd_zero _))
  · exact h.2

/-- Equal rational elapsed times give equal positions even for schedules with
  different positive common denominators and different partitions. -/
theorem partitionMotion_cross_partition (D E : Nat) (hD : 0 < D) (hE : 0 < E)
  (p v : Point) (ws xs : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total xs) hE)) :
  pointEquiv (partitionMotion D hD p v zeroPoint ws).1
  (partitionMotion E hE p v zeroPoint xs).1 := by
  have hleft := partitionMotion_zero_force D hD p v ws
  have hright := partitionMotion_zero_force E hE p v xs
  exact pointEquiv_trans hleft.1
  (pointEquiv_trans (inertialAt_time_congr p v ht)
  (Fraction.equiv_symm hright.1.1, Fraction.equiv_symm hright.1.2))

/-- Cross-partition agreement includes the unchanged terminal velocity. -/
theorem partitionMotion_cross_partition_state (D E : Nat) (hD : 0 < D) (hE : 0 < E)
  (p v : Point) (ws xs : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total xs) hE)) :
  pointEquiv (partitionMotion D hD p v zeroPoint ws).1
  (partitionMotion E hE p v zeroPoint xs).1 ∧
  pointEquiv (partitionMotion D hD p v zeroPoint ws).2
  (partitionMotion E hE p v zeroPoint xs).2 := by
  constructor
  · exact partitionMotion_cross_partition D E hD hE p v ws xs ht
  · have hleft := partitionMotion_zero_force D hD p v ws
    have hright := partitionMotion_zero_force E hE p v xs

```

```

    exact pointEquiv_trans hleft.2
    (Fraction.equiv_symm hright.2.1, Fraction.equiv_symm hright.2.2)

def scalarZero : Fraction := Fraction.ofInt 0
def scalarOne : Fraction := Fraction.ofInt 1
def scalarTwo : Fraction := Fraction.ofInt 2
def scalarHalf : Fraction := (1, 2, by decide)

def slow (t : Fraction) : Point := (t, scalarZero)
def fast (t : Fraction) : Point := (Fraction.mul scalarTwo t, scalarZero)

/-- Same endpoint with different elapsed times: `slow(1) = fast(1/2)`. -/
theorem slow_fast_equal_endpoint : pointEquiv (slow scalarOne) (fast scalarHalf) := by
  decide

/-- At a common half-time the two velocity choices give different positions. -/
theorem slow_fast_different_half_time : ¬ pointEquiv (slow scalarHalf) (fast scalarHalf) := by
  decide

/-- Collinear samples of these distinct motions close with zero directed area.
  This is a geometric diagnostic only, not fixed-data nonuniqueness. -/
theorem slow_fast_collinear_closedBoundary :
  Fraction.equiv (closedBoundaryTwice zeroPoint (slow scalarHalf) (slow scalarOne)
    (fast scalarHalf)) scalarZero := by
  decide

/-- A concrete two-cell actual schedule has the expected inertial endpoint. -/
theorem slow_two_cell_schedule :
  pointEquiv (partitionMotion 2 (by decide) zeroPoint (slow scalarOne) zeroPoint [1, 1]).1
    (slow scalarOne) := by
  decide

end NewtonLimitDynamics.Polygon.ZeroForce

```

NewtonLimitDynamics/Polygon/InertialControl.lean, 6 theorems, 114 lines

```

import NewtonLimitDynamics.Polygon.ZeroForce

namespace NewtonLimitDynamics.Polygon.InertialControl

open NewtonLimitDynamics
open TimeSubdivision
open ZeroForce

/-- Strict comparison of the absolute value of a represented rational with a
  positive rational. It is cross multiplication, so it tolerates unnormalised
  representatives. -/
def absLt (a bound : Fraction) : Prop :=
  (a.num.natAbs : Int) * bound.den < bound.num * a.den

/-- A common positive integer bound for both velocity numerators. -/
def velocityBound (v : Point) : Nat :=
  v.1.num.natAbs + v.2.num.natAbs + 1

/-- An explicit time radius, for an arbitrary fixed rational velocity. -/
def radius (eps : Fraction) (v : Point) : Fraction :=
  (eps.num, eps.den * (velocityBound v : Int),
    Int.mul_pos eps.den_pos (by
      unfold velocityBound
      exact Int.ofNat_pos.mpr (by omega))))

theorem radius_positive (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) : Fraction.positive (radius eps v) := heps

private theorem scalar_bound (eps h x : Fraction) (K : Nat)
  (heps : Fraction.positive eps) (hK : x.num.natAbs < K)
  (hh : absLt h (eps.num, eps.den * (K : Int),
    Int.mul_pos eps.den_pos (Int.ofNat_pos.mpr (by omega)))) :
  absLt (Fraction.mul h x) eps := by
  have hK' : (x.num.natAbs : Int) ≤ K := Int.ofNat_le.mpr (Nat.le_of_lt hK)
  have hden : 1 ≤ x.den := by

```

```

have := x.den_pos
omega
have hleft : 0 ≤ (h.num.natAbs : Int) * eps.den :=
  Int.mul_nonneg (Int.ofNat_zero_le _) (Int.le_of_lt eps.den_pos)
have hmul := Int.mul_le_mul_of_nonneg_left hK' hleft
have hrigh : 0 ≤ eps.num * h.den :=
  Int.mul_nonneg (Int.le_of_lt heps) (Int.le_of_lt h.den_pos)
have hdenmul := Int.mul_le_mul_of_nonneg_left hden hrigh
have hsmall : (h.num.natAbs : Int) * eps.den * (K : Int) < eps.num * h.den := by
  unfold absLt at hh
  dsimp at hh
  simpa only [Int.mul_assoc] using hh
have hchain : (h.num.natAbs : Int) * (x.num.natAbs : Int) * eps.den <
  eps.num * (h.den * x.den) := by
  calc
    (h.num.natAbs : Int) * (x.num.natAbs : Int) * eps.den
      = ((h.num.natAbs : Int) * eps.den) * (x.num.natAbs : Int) := by ac_rfl
    _ ≤ ((h.num.natAbs : Int) * eps.den) * (K : Int) := hmul
    _ < eps.num * h.den := hsmall
    _ ≤ (eps.num * h.den) * x.den := by simpa using hdenmul
    _ = eps.num * (h.den * x.den) := by ac_rfl
unfold absLt Fraction.mul
dsimp
rw [Int.natAbs_mul]
exact hchain

/-- For each positive rational tolerance, one explicit radius controls both
coordinates of every rational drift at fixed velocity. -/
theorem drift_small (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) (h : Fraction)
  (hh : absLt h (radius eps v)) :
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  have hK1 : v.1.num.natAbs < velocityBound v := by
    unfold velocityBound
    omega
  have hK2 : v.2.num.natAbs < velocityBound v := by
    unfold velocityBound
    omega
  constructor
  · exact scalar_bound eps h v.1 (velocityBound v) heps hK1 hh
  · exact scalar_bound eps h v.2 (velocityBound v) heps hK2 hh

/-- The controlled drift is the increment in the rational inertial map at
every rational base time. This is an equivalence of represented positions,
not an assumed curve or a limit theorem. -/
theorem inertialAt_small_increment (eps : Fraction) (p v : Point)
  (heps : Fraction.positive eps) (t h : Fraction)
  (hh : absLt h (radius eps v)) :
  pointEquiv (inertialAt (inertialAt p v t) v h)
    (inertialAt p v (Fraction.add t h)) ∧
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  exact (inertialAt_add p v t h, drift_small eps v heps h hh)

/-- The same estimate applies to an actual zero-force end-kick cell begun at
any rational inertial time. -/
theorem endKick_zero_small_increment (eps : Fraction) (p v : Point)
  (heps : Fraction.positive eps) (t h : Fraction)
  (hh : absLt h (radius eps v)) :
  pointEquiv (endKick h (inertialAt p v t, v) zeroPoint).1
    (inertialAt p v (Fraction.add t h)) ∧
  absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
  have hcell := (endKick_zero h (inertialAt p v t, v)).1
  have hmap := inertialAt_add p v t h
  exact (⟨Fraction.equiv_trans hcell.1 hmap.1,
    Fraction.equiv_trans hcell.2 hmap.2⟩, drift_small eps v heps h hh)

/-- Quantified small-time form. The witness is `radius eps v`, independent of
the base time and initial position. -/
theorem exists_uniform_inertial_radius (eps : Fraction) (v : Point)
  (heps : Fraction.positive eps) :

```

```

    ∃ delta : Fraction, Fraction.positive delta ∧
    ∀ (p : Point) (t h : Fraction), absLt h delta →
    pointEquiv (inertialAt (inertialAt p v t) v h)
    (inertialAt p v (Fraction.add t h)) ∧
    absLt (pointScale h v).1 eps ∧ absLt (pointScale h v).2 eps := by
    exact (radius eps v, radius_positive eps v heps,
    fun p t h hh => inertialAt_small_increment eps p v heps t h hh)

end NewtonLimitDynamics.Polygon.InertialControl

NewtonLimitDynamics/Polygon/InertialDefect.lean, 9 theorems, 111 lines

import NewtonLimitDynamics.Polygon.ZeroForce

namespace NewtonLimitDynamics.Polygon.InertialDefect

open NewtonLimitDynamics
open TimeSubdivision
open ZeroForce
open PartitionControl

/-- The directed determinant of two rational-time inertial samples. -/
def inertialEdge (p v : Point) (s t : Fraction) : Fraction :=
  det (inertialAt p v s) (inertialAt p v t)

/-- Three samples of the same finite zero-force comparison map have an
    additive directed determinant. The identity is finite Fraction arithmetic. -/
theorem inertialEdge_compose (p v : Point) (s t u : Fraction) :
  Fraction.equiv
  (Fraction.add (inertialEdge p v s t) (inertialEdge p v t u))
  (inertialEdge p v s u) := by
  unfold Fraction.equiv Fraction.add inertialEdge det inertialAt pointAdd pointScale
  Fraction.mul Fraction.add
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Determinant of a degenerate connector. -/
theorem inertialEdge_self (p v : Point) (s : Fraction) :
  Fraction.equiv (inertialEdge p v s s) (Fraction.ofInt 0) := by
  unfold Fraction.equiv inertialEdge det Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Directed determinant along a finite list, including its final connector. -/
def inertialWalk (p v : Point) (a b : Fraction) : List Fraction → Fraction
| [] => inertialEdge p v a b
| t :: ts => Fraction.add (inertialEdge p v a t) (inertialWalk p v t b ts)

/-- Every finite collinear walk telescopes to its end connector. -/
theorem inertialWalk_eq_edge (p v : Point) (a b : Fraction) (times : List Fraction) :
  Fraction.equiv (inertialWalk p v a b times) (inertialEdge p v a b) := by
  induction times generalizing a with
  | nil => exact Fraction.equiv_refl _
  | cons t ts ih =>
    exact Fraction.equiv_trans (Fraction.add_equiv_left _ (ih t))
    (inertialEdge_compose p v a t b)

/-- Any finite closed polygon sampled from an inertial recurrence has zero
    signed doubled determinant sum, for any start, times, and velocity. -/
theorem inertialWalk_closed (p v : Point) (a : Fraction) (times : List Fraction) :
  Fraction.equiv (inertialWalk p v a a times) (Fraction.ofInt 0) := by
  exact Fraction.equiv_trans (inertialWalk_eq_edge p v a a times)
  (inertialEdge_self p v a)

/-- The four-vertex boundary used by the finite scheduling diagnostic is a
    special case of the arbitrary closed walk. -/
theorem inertial_closedBoundaryTwice (p v : Point) (a b c d : Fraction) :

```

```

    Fraction.equiv
      (closedBoundaryTwice (inertialAt p v a) (inertialAt p v b)
        (inertialAt p v c) (inertialAt p v d)) (Fraction.ofInt 0) := by
exact Fraction.equiv_trans
  (Fraction.add_assoc (inertialEdge p v a b) (inertialEdge p v b c)
    (Fraction.add (inertialEdge p v c d) (inertialEdge p v d a)))
  (inertialWalk_closed p v a [b, c, d])

private def fracNeg (a : Fraction) : Fraction := {-a.num, a.den, a.den_pos}

private theorem fracNeg_congr {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv (fracNeg a) (fracNeg b) := by
  unfold Fraction.equiv fracNeg at *
  dsimp at *
  simpa only [Int.neg_mul] using congrArg Neg.neg h

/-- The represented directed determinant respects point equivalence. -/
theorem det_congr {a a' b b' : Point} (ha : pointEquiv a a')
  (hb : pointEquiv b b') : Fraction.equiv (det a b) (det a' b') := by
  unfold det
  exact Fraction.add_equiv (Fraction.mul_equiv ha.1 hb.2) (fracNeg_congr (Fraction.mul_equiv ha.2 hb.1))

theorem closedBoundaryTwice_congr {a a' b b' c c' d d' : Point}
  (ha : pointEquiv a a') (hb : pointEquiv b b')
  (hc : pointEquiv c c') (hd : pointEquiv d d') :
  Fraction.equiv (closedBoundaryTwice a b c d) (closedBoundaryTwice a' b' c' d') := by
  unfold closedBoundaryTwice
  exact Fraction.add_equiv (Fraction.add_equiv (det_congr ha hb) (det_congr hb hc))
    (Fraction.add_equiv (det_congr hc hd) (det_congr hd ha))

/-- Four arbitrary actual zero-force schedules (possibly with different
  partitions) have a vanishing signed determinant boundary. -/
theorem partitionMotion_closedBoundaryTwice (D : Nat) (hD : 0 < D)
  (p v : Point) (w₀ w₁ w₂ w₃ : List Nat) :
  Fraction.equiv
    (closedBoundaryTwice
      (partitionMotion D hD p v zeroPoint w₀).1
      (partitionMotion D hD p v zeroPoint w₁).1
      (partitionMotion D hD p v zeroPoint w₂).1
      (partitionMotion D hD p v zeroPoint w₃).1)
    (Fraction.ofInt 0) := by
exact Fraction.equiv_trans
  (closedBoundaryTwice_congr
    (partitionMotion_zero_force D hD p v w₀).1
    (partitionMotion_zero_force D hD p v w₁).1
    (partitionMotion_zero_force D hD p v w₂).1
    (partitionMotion_zero_force D hD p v w₃).1)
  (inertial_closedBoundaryTwice p v
    (duration D (total w₀) hD) (duration D (total w₁) hD)
    (duration D (total w₂) hD) (duration D (total w₃) hD))

end NewtonLimitDynamics.Polygon.InertialDefect

```

Law II. *The change of motion is proportional to the motive force impressed, and takes place along the straight line in which that force is impressed.*

In modern terms: $\Delta \mathbf{p} \propto \mathbf{J}$, with $\mathbf{J}$ the impressed motive force (an impulse) and $\Delta \mathbf{p}$ parallel to it.

If a force generates some motion, double the force generates double the motion and triple the force triple, whether impressed all at once or gradually and successively. This motion, always directed with the generating force, is added to the body's earlier motion if they conspire, subtracted if contrary, and compounded obliquely if oblique, according to both directions.

No theorem of the Lean reconstruction cites this item. Encoded in impulse form by the kick of

‘CentralSchedule.cell’: the change of velocity is ‘ $d \cdot a(y)$ ’, along the force and proportional to it; and by ‘Finite.EuclideanConstruction.kick’, which displaces the vertex parallel to the radius.

Law III. *To every action there is always an equal and contrary reaction; or, the mutual actions of two bodies on each other are always equal and directed to contrary parts.*

In modern terms: $\mathbf{F}_{AB} = -\mathbf{F}_{BA}$.

Whatever presses or pulls another is pressed or pulled by it as much. If you press a stone with your finger, the finger is pressed by the stone. If a horse pulls a stone tied to a rope, the horse is pulled back equally toward the stone: the rope, stretched both ways, urges each toward the other with the same endeavour to relax. If a body striking another changes its motion, it undergoes the same change of its own motion in the contrary direction. The equal changes are of motions, not velocities (for bodies not otherwise impeded); the changes of velocities, also contrary, are inversely proportional to the bodies.

In modern terms: $m_A \Delta \mathbf{v}_A = -m_B \Delta \mathbf{v}_B$.

1713. The explanation ends: this law also holds in attractions, as will be proved in the next Scholium.

No theorem of the Lean reconstruction cites this item. Not encoded: the model is single-body. Proposition III cancels the second body’s force by Corollary VI, not by Law III.

Corollary I. *A body acted on by two forces jointly describes the diagonal of a parallelogram in the same time in which it would describe the sides by the forces separately.*

Proof. Let the body in a given time be carried by force M alone from A to B , and by force N alone from A to C ; complete the parallelogram $ABDC$. Since N acts along AC , parallel to BD , it does not change the velocity of approach to the line BD generated by M . So the body reaches the line BD in the same time whether N acts or not, and at the end of that time it is somewhere on BD . By the same argument it is somewhere on CD , so it is at their intersection D .

In modern terms: $\overrightarrow{AD} = \overrightarrow{AB} + \overrightarrow{AC}$.

1713. The forces M and N are *impressed at the place A*, the motion from A to B is *uniform*, the invariance of the velocity of approach is *by Law II*, and the proof ends: the body proceeds in a straight line from A to D by Law I. The corollary is thereby stated for impulses given at one place, followed by inertial motion.

No theorem of the Lean reconstruction cites this item. Encoded as ‘Finite.step g p q j = g.kick q (g.extend p q) j’ with the field ‘same_base_parallel’ of ‘EuclideanConstruction’: the displaced vertex lies on the line through the inertial point parallel to the radius, Newton’s parallelogram.

Corollary II. *Hence the composition of a direct force AD out of any oblique forces AB and BD , and conversely the resolution of any direct force AD into any oblique forces AB and BD . This is abundantly confirmed by mechanics.*

Weights A and P hung by threads from unequal spokes OM , ON of a wheel balance when they are inversely as the perpendicular distances OK , OL of their threads from the centre (the property of the balance, the lever and the wheel and axle): resolving the weight A , represented by AD , into AC along the spoke OD , which does nothing

to turn the wheel, and DC perpendicular to it, gives $P : A = DC : DA = OK : OL$ by the similar triangles ADC , DOK . A weight p resting partly on an inclined plane and partly hung by a thread resolves likewise, which gives the tensions of oblique threads, the forces of the wedge between the faces of a split body, of the mallet, and of the screw, which is a wedge driven by a lever. From these follow the forces of all machines built from wheels, drums, pulleys, levers, taut strings and weights, and of the muscles moving the bones of animals.

No theorem of the Lean reconstruction cites this item. Not encoded.

Corollary III. *The quantity of motion, obtained by taking the sum of the motions directed to the same side and the difference of those directed to contrary sides, is not changed by the actions of bodies on each other.*

In modern terms: $\sum_i m_i \mathbf{v}_i$ is invariant under mutual interactions.

Proof. Action and reaction are equal (Law III), so by Law II they produce equal changes of motion toward contrary sides. If the motions are to the same side, whatever is added to the motion of the body fleeing is taken from the body pursuing, so the sum stays the same; if the bodies meet, equal amounts are taken from both, so the difference stays the same.

Example: a sphere A three times as large as a sphere B moves with 2 parts of velocity, and B follows on the same line with 10, so the motions are 6 and 10, total 16. Whatever A gains in the collision, 3, 4 or 5 parts, B loses, leaving A with 9, 10 or 11 and B with 7, 6 or 5; if A gains 9 to 12, B continues with 1, stops, or recoils with 1 or 2. The sum of conspiring motions or the difference of contrary ones is always 16. For non-spherical bodies or oblique collisions, find the plane touching both at the point of contact, resolve each motion (Corollary II) into components perpendicular and parallel to that plane, keep the parallel components, and give the perpendicular components equal and contrary changes so that the sum or difference is unchanged. Collisions also produce rotations about the bodies' own centres, which are not considered here.

No theorem of the Lean reconstruction cites this item. Not encoded (no mass, single body).

Corollary IV. *The common centre of gravity of two or more bodies does not change its state of motion or rest by the actions of the bodies among themselves; therefore the common centre of gravity of all bodies acting on each other (external actions and impediments excluded) either rests or moves uniformly in a straight line.*

In modern terms: with no external forces, $\mathbf{R} = \sum_i m_i \mathbf{r}_i / \sum_i m_i$ satisfies $\ddot{\mathbf{R}} = 0$.

1713. "The common centre of gravity of two or more bodies". The proof treats points moving in one plane and adds that the same holds if their motions are not in one plane. **1726** cites Lemma XXIII and its corollary.

Proof. If two points move uniformly in straight lines and their distance is divided in a given ratio, the dividing point rests or moves uniformly in a straight line (proved later in Lemma XXIII for the plane; the same holds in space). So if any number of bodies move uniformly in straight lines, the common centre of any two rests or moves uniformly, since it divides in a given ratio the line joining their centres; likewise the common centre of these two and any third, and so on to infinity. Hence in a system of bodies free of mutual actions and of all external forces, each moving uniformly in

a straight line, the common centre of all rests or moves uniformly in a straight line. Further, for two bodies acting on each other, their distances from the common centre are inversely as the bodies, so their relative motions of approach to or recession from the centre are equal; the centre is neither advanced nor retarded by equal changes of motion in contrary directions, that is, by the mutual actions. In a system of many bodies, the common centre of any two acting on each other keeps its state, and the common centre of the rest is unaffected by that action; the distance between these two centres is divided by the common centre of all inversely as the total bodies whose centres they are; so the common centre of all keeps its state. All mutual actions are between pairs of bodies or compounded of such, so the common centre of all is never changed by them.

No theorem of the Lean reconstruction cites this item. Not encoded.

Corollary V. *The motions of bodies contained in a given space are the same among themselves whether that space rests or moves uniformly in a straight line without circular motion.*

Proof. The differences of motions toward the same side and the sums of motions toward contrary sides are the same at the start in both cases (by hypothesis), and from them arise the collisions and impulses by which the bodies strike each other; so by Law II the effects are equal in both cases, and the motions among themselves stay equal. A clear experiment confirms it: all motions behave the same in a ship at rest or moving uniformly in a straight line.

In modern terms: the laws are invariant under $\mathbf{r}' = \mathbf{r} - \mathbf{u}t$, $t' = t$, with $\mathbf{u}$ constant.

No theorem of the Lean reconstruction cites this item. Used as 'Converse.moving_centre_equal_areas_centra Proposition II, Case 2 (printed under Proposition II).

Corollary VI. *If bodies move in any way among themselves and are urged along parallel lines by equal accelerative forces, they all continue to move among themselves as if those forces did not act.*

Proof. The forces, acting equally (in proportion to the quantities of the bodies) along parallel lines, move all bodies equally as to velocity (Law II), and so never change their positions and motions among themselves.

No theorem of the Lean reconstruction cites this item. Stated and used as 'Relative-Motion.corVI_relative' (printed under Proposition III, whose dependency edge in 'research/dependencies.json' names it).

Scholium

These principles are received by mathematicians and confirmed by many experiments. By the first two Laws and Corollaries Galileo found that the descent of heavy bodies is in the duplicate ratio of the time and that the motion of projectiles is in a parabola, as experiment agrees except for the small retardation by air. On the same Laws and Corollaries depend the demonstrations about the times of oscillating pendulums, confirmed daily by clocks. From these and the third Law, Wren, Wallis and Huygens independently found the rules of the collision and reflection of two bodies and communicated them to the Royal Society at about the same time, in full agreement. Wren confirmed them by

an experiment with pendulums before the Society, and Mariotte soon devoted a whole book to it. For exact agreement one must allow for the air's resistance and for the elastic force of the colliding bodies. Newton describes how to correct a pendulum-collision experiment for air resistance by measuring the loss over swings, and reports his own trials with bodies of various hardness, including tightly wound wool balls: the bodies separate with a relative velocity in a given ratio to the relative velocity of approach, and the rules hold for soft bodies as well as hard ones.

1713. The description of the pendulum experiment is made exact: the arcs are marked so that $RS = TV$ and $RS : ST = 3 : 2$, and the equal changes of motion are stated for the bodies.

1726. A passage is inserted after the mention of Galileo. Uniform gravity, acting equally in equal particles of time, impresses equal forces on a falling body and generates equal velocities; in the whole time it impresses the whole force and generates the whole velocity, proportional to the time; and the spaces described in proportional times are as the velocities and times jointly, that is, in the duplicate ratio of the times. For a body thrown upward, uniform gravity impresses forces and takes away velocities proportional to the times; the times of ascent to the greatest heights are as the velocities to be taken away, and the heights as the velocities and times jointly, or in the duplicate ratio of the velocities. A body thrown along any straight line compounds the motion from the projection with the motion from gravity: if by projection alone it would describe AB in a given time and by falling alone the height AC , completing the parallelogram $ABDC$ puts it at D at the end of the time, and the curve AED it describes is a parabola touching AB at A whose ordinate BD is as AB^2 .

For attractions, suppose two bodies A and B attract each other and an obstacle is placed between them. If A were drawn toward B more than B toward A , the obstacle would be pressed more by A than by B and would not stay in equilibrium; the stronger pressure would make the system of the two bodies and the obstacle move toward B and, accelerating forever in free space, go off to infinity, which is absurd and contrary to the first Law. So the bodies press the obstacle equally and attract each other equally. Newton tried this with a lodestone and iron floating in separate vessels on still water: neither pushed the other, and they came to rest in equilibrium.

1713. A new paragraph: gravity between the Earth and its parts is mutual. Cut the Earth by any plane EG into parts EGF and EGI . Cut the larger part EGI by a parallel plane HK into $EGKH$ and HKI , with HKI equal to EGF . The middle part $EGKH$ inclines by its own weight toward neither extreme part and rests between them in equilibrium; the extreme part HKI presses with its whole weight on the middle part and urges it toward EGF . So the force with which EGI tends toward EGF equals the weight of HKI , that is, of EGF , and the two parts weigh equally on each other. Otherwise the whole Earth, floating in the free aether, would yield to the greater weight and go off to infinity.

As bodies are equipollent in collision when their velocities are inversely as their inherent forces, so in mechanical instruments agents are equipollent and sustain each other when their velocities, estimated along the forces, are inversely as the forces. So for the

balance, the pulley, trains of wheels, the screw and the wedge. The use of machines lies only in increasing force by diminishing velocity and conversely; this solves, for every suitable instrument, the problem of moving a given weight by a given force. If the action of the agent is estimated by its force and velocity jointly, and the reaction of the resistance by the velocities of its parts and their forces of resisting (from friction, cohesion, weight and acceleration), action and reaction are always equal in every use of instruments. Mechanics is not the subject here; this only shows how widely and how surely the third Law holds.

No theorem of the Lean reconstruction cites this item. Not encoded, except that Galileo's parabola is the parallel-force instance 'ForceClasses.parallelOracle' and 'Polygon/ParallelQuadraticEndpoint'.

Book I: On the Motion of Bodies

Section I. On the method of first and last ratios, by whose help what follows is demonstrated

Lemma I. *Quantities, and also ratios of quantities, which in a given finite time tend constantly to equality, and before the end of that time approach nearer to each other than by any given difference, become ultimately equal.*

In modern terms: if $X(t), Y(t)$ on $[0, T)$ approach each other so that for every $\varepsilon > 0$ there is $t < T$ beyond which $|X - Y| < \varepsilon$, then their ultimate values at T are equal.

Proof. If not, let their ultimate difference be $D > 0$. Then they cannot approach nearer to equality than by the given difference D , contrary to the hypothesis.

1713. The statement reads “in any finite time” and replaces “can approach nearer” by “before the end of that time approach nearer than by any given difference”: the approach is asserted to happen within the time, not merely to be possible. The proof begins “If you deny it, let them be ultimately unequal”.

No theorem of the Lean reconstruction cites this item. Encoded as the limit interface of 'BarrowLib/Common/Quadratic.lean': 'Near' and 'Ultimate' (ultimate equality is approach closer than any given difference), used by 'enclosure_reconstruction', the squeeze; and as the 'Within' and 'Vanishes' predicates of 'CauchyValues' and 'Enclosure'.

Lemma II. *If in any figure $AacE$, bounded by the straight lines Aa, AE and the curve acE , any number of parallelograms $Ab, Bc, Cd, \dots$ are inscribed on equal bases $AB, BC, CD, \dots$ with sides $Bb, Cc, Dd, \dots$ parallel to the side Aa , and the parallelograms $aKbl, bLcm, cMdn, \dots$ are completed; and if then the width of the parallelograms is diminished and their number increased indefinitely: the ultimate ratios which the inscribed figure $AKbLcMdD$, the circumscribed figure $AalbmcndoE$ and the curvilinear figure $AabcdE$ have to one another are ratios of equality.*

In modern terms: for the region under a monotone curve over $[a, b]$, with lower and upper step sums L_n, U_n on n equal subintervals and area $\mathcal{A}$, the ratios $L_n : U_n : \mathcal{A}$ tend to $1 : 1 : 1$. [In Newton's figure the curve is monotone, and the curvilinear figure's area is taken as given.]

Proof. The difference of the inscribed and circumscribed figures is the sum of the parallelograms $Kl + Lm + Mn + Do$, that is (the bases being equal) the rectangle under one base Kb and the sum of the heights Aa , the rectangle $ABla$. Since its width AB is diminished indefinitely, it becomes less than any given rectangle. So by Lemma I the inscribed and circumscribed figures, and still more the curvilinear figure between them, become ultimately equal.

In modern terms: $U_n - L_n = \Delta x (f(a) - f(b)) \rightarrow 0$.

No theorem of the Lean reconstruction cites this item. Not separately encoded; its equal-base step sums appear with Lemma III.

Lemma III. *The same ultimate ratios are also ratios of equality when the widths AB , BC , CD , ... of the parallelograms are unequal and all are diminished indefinitely.*

Proof. Let AF equal the greatest width and complete the parallelogram $FAaf$. It is greater than the difference of the inscribed and circumscribed figures, and as its width AF is diminished indefinitely it becomes less than any given rectangle.

In modern terms: $U - L \leq \max_i \Delta x_i (f(a) - f(b)) \rightarrow 0$ as the mesh tends to 0.

Corollary 1. Hence the ultimate sum of the vanishing parallelograms coincides in every part with the curvilinear figure.

Corollary 2. Still more does the rectilinear figure bounded by the chords ab , bc , cd , ... of the vanishing arcs coincide ultimately with the curvilinear figure.

Corollary 3. So does the rectilinear figure circumscribed by the tangents of the same arcs.

Corollary 4. And therefore these ultimate figures (as to their perimeters acE) are not rectilinear, but curvilinear limits of rectilinear figures.

Lean reconstruction: 6 theorems in 1 module cite this item; the modules follow in full, in import order.

Corollary 4, the passage Proposition I cites for its limit, is the premise named 'PolygonTrajectoryEnclosure' in 'Polygon/PathDefect.lean': assumed by the edition theorems, not derived.

NewtonLimitDynamics/Polygon/Enclosure.lean, 6 theorems, 69 lines

```
import NewtonLimitDynamics.Polygon.Finite
import NewtonLimitDynamics.Common.RationalMagnitudes

namespace NewtonLimitDynamics.Polygon

theorem isum_mono (f g : Nat → Int) (n : Nat) (h : ∀ i, i < n → f i ≤ g i) :
  isum f n ≤ isum g n := by
  induction n with
  | zero => exact Int.le_refl _
  | succ n ih =>
    simp only [isum]
    exact Int.add_le_add (ih (fun i hi => h i (by omega))) (h n (by omega))

theorem isum_mul (f : Nat → Int) (c : Int) (n : Nat) :
  isum (fun i => c * f i) n = c * isum f n := by
  induction n with
  | zero => simp [isum]
  | succ n ih => simp [isum, ih, Int.mul_add]

theorem telescoping (height : Nat → Int) (n : Nat) :
```

```

    isum (fun i => height (i+1) - height i) n = height n - height 0 := by
induction n with
| zero => simp [isum]
| succ n ih => simp only [isum, ih]; omega

/-- Lemmas II/III's finite rectangle estimate, for a monotone patch. Widths
may be unequal; every width must obey the SAME maximum. Multiplication
measures rectangle area. A curve enclosed by these rectangles is an
additional geometric hypothesis, not supplied by this arithmetic result. -/
theorem rectangle_gap_bound (width height : Nat → Int) (maxWidth : Int) (n : Nat)
(hw : ∀ i, i < n → width i ≤ maxWidth)
(hh : ∀ i, i < n → height i ≤ height (i+1)) :
isum (fun i => width i * (height (i+1) - height i)) n ≤
maxWidth * (height n - height 0) := by
have h := isum_mono (fun i => width i * (height (i+1) - height i))
(fun i => maxWidth * (height (i+1) - height i)) n (by
intro i hi
exact Int.mul_le_mul_of_nonneg_right (hw i hi) (by have := hh i hi; omega))
rw [isum_mul, telescoping] at h
exact h

/-- Refinement indexed by positive rational mesh. The budget represents the
maximum-width times total-height estimate. Making that budget small must
be justified for the selected curve; it does not assert a trajectory. -/
def Vanishes (gap : Fraction → Fraction) : Prop :=
∀ epsilon, Fraction.positive epsilon →
Near Fraction.magnitudes (fun mesh => Fraction.lt (gap mesh) epsilon)

theorem enclosed_gap_vanishes (gap budget : Fraction → Fraction)
(hbudget : Vanishes budget)
(henclose : Near Fraction.magnitudes (fun mesh => Fraction.le (gap mesh) (budget mesh))) :
Vanishes gap := by
intro epsilon hepsilon
obtain ⟨d, hd, h⟩ := near_and Fraction.magnitudes _ _ henclose (hbudget epsilon hepsilon)
exact ⟨d, hd, fun mesh hm hmd =>
Fraction.magnitudes.lt_of_le_lt (h mesh hm hmd).1 (h mesh hm hmd).2⟩

/-- Conditional transfer of polygon area ratios to enclosed sector area ratios.
Lower and upper limits are geometric premises. No trajectory existence or
identification with continuous force follows from this type. -/
theorem sector_ratio_reconstruction (sector inner outer : Fraction → Fraction)
(c : Fraction) (hin : Ultimate Fraction.magnitudes inner c)
(hout : Ultimate Fraction.magnitudes outer c)
(henclose : Near Fraction.magnitudes (fun mesh =>
Fraction.le (inner mesh) (sector mesh) ∧ Fraction.le (sector mesh) (outer mesh))) :
Ultimate Fraction.magnitudes sector c :=
enclosure_reconstruction Fraction.magnitudes sector inner outer c hin hout henclose

end NewtonLimitDynamics.Polygon

```

Lemma IV. *If in two figures $AacE$, $PprT$ two series of parallelograms are inscribed (as above), equal in number, and when the widths are diminished indefinitely the ultimate ratios of the parallelograms of one figure to those of the other, each to each, are the same: the two figures are to each other in that same ratio.*

Proof. As the parallelograms are to each other one by one, so (*componendo*) is the sum of all to the sum of all, and so is figure to figure, each figure being to its sum in the ratio of equality (Lemma III).

In modern terms: if $a_i/b_i \rightarrow r$ for all i as the mesh tends to 0, then $\lim \sum a_i / \lim \sum b_i = r$. [The convergence is tacitly uniform in i .]

Corollary. Hence if two quantities of any kind are divided into the same number of parts, and those parts, as their number is increased and their magnitude diminished indefinitely, have a given ratio to each other, first to first, second to second and so on

in order, the wholes are to each other in that given ratio. For if parallelograms are taken in the figures of this Lemma proportional to the parts, the sums of the parts are always as the sums of the parallelograms, and so, in the limit, in the ultimate ratio of parallelogram to parallelogram, that is (by hypothesis) of part to part.

No theorem of the Lean reconstruction cites this item. Not encoded.

Lemma V. *All corresponding sides of similar figures, curvilinear as well as rectilinear, are proportional, and the areas are in the duplicate ratio of the sides.*

In modern terms: under a similarity of ratio λ , lengths scale by λ and areas by λ^2 .

No theorem of the Lean reconstruction cites this item. Not encoded; Proposition IV's limiting route through it is documented as an editorial interpretation, not derived.

Lemma VI. *If any arc AB , given in position, is subtended by its chord AB , and at some point A in the middle of a continuous curvature it is touched by a straight line AD produced both ways; and if then the points A and B approach each other and coincide: the angle BAD contained by the chord and the tangent is diminished indefinitely and ultimately vanishes.*

In modern terms: at a point where the curve has a tangent, the direction of the chord AB tends to the direction of the tangent as $B \rightarrow A$.

Proof. Produce AB to b and AD to d . As A and B coincide, no part AB of the line Ab lies any longer inside the curve, so the line Ab either coincides with the tangent Ad or passes between the tangent and the curve. The latter is contrary to the nature of curvature, so the former holds.

1713. A new proof replaces this one: if the angle does not vanish, the arc AB will contain with the tangent AD an angle equal to a rectilinear angle, and so the curvature at A will not be continuous, contrary to the hypothesis. **1726** names the arc ACB , through an intermediate point C , here and in Lemmas VII and VIII.

No theorem of the Lean reconstruction cites this item. Not encoded; the chord, tangent and arc comparison is not needed by the finite steps of Propositions I-IV.

Lemma VII. *Under the same suppositions, the ultimate ratio of the arc, the chord and the tangent to each other is the ratio of equality.*

Proof. Produce AB and AD to b and d , draw bd parallel to the secant BD , and let the arc Ab be similar to the arc AB . As A and B coincide, the angle dAb vanishes (Lemma VI), so the straight lines Ab , Ad and the arc between them coincide and are equal. Hence AB , AD and the arc AB , always proportional to these, ultimately have the ratio of equality.

In modern terms: $\text{arc } AB / \text{chord } AB \rightarrow 1$ and $AD / \text{chord } AB \rightarrow 1$ as $B \rightarrow A$.

1713. The proof is reorganized as a rescaling: while B approaches A , the lines AB , AD are imagined always produced to *distant* points b , d , so that the enlarged lines Ab , Ad and the arc between them stay *always finite* while the angle dAb vanishes; they coincide and are equal, and the proportional small lines AB , AD and the arc AB *vanish* with ultimate ratio of equality.

Corollary 1. If through B a line BF is drawn parallel to the tangent, meeting any straight line AF through A in F , then BF ultimately has the ratio of equality to the vanishing arc AB , because, completing the parallelogram $AFBD$, it always has that ratio to AD .

Corollary 2. If through B and A more lines BE , BD , AF , AG are drawn cutting the tangent AD and its parallel BF , the ultimate ratio of all the abscissas AD , AE , BF , BG , and of the chord and arc AB , to each other is the ratio of equality.

Corollary 3. Therefore all these lines may be used for one another in any argument about ultimate ratios.

No theorem of the Lean reconstruction cites this item. Not encoded (see Lemma VI).

Lemma VIII. *If the given straight lines AR , BR form with the arc AB , the chord AB and the tangent AD three triangles RAB , RAB , RAD (the first with the arc as a side), and then the points A and B approach each other: the ultimate form of the vanishing triangles is that of similarity, and their ultimate ratio is that of equality.*

Proof. Produce AB , AD , AR to b , d , r ; draw rbd parallel to RD , and the arc Ab similar to the arc AB . As A and B coincide, the angle bAd vanishes, so the three triangles rAb , rAb , rAd coincide and are therefore similar and equal. Hence RAB , RAB , RAD , always similar and proportional to these, become ultimately similar and equal to each other.

1713. The same rescaling: AB , AD , AR are always produced to distant points b , d , r , and the three triangles rAb , rAb , rAd stay *always finite* while they coincide.

Corollary. Hence those triangles may be used for one another in any argument about ultimate ratios.

No theorem of the Lean reconstruction cites this item. Not encoded (see Lemma VI).

Lemma IX. *If a straight line AE and a curve AC , given in position, cut each other at a given angle A , and to that line, at another given angle, ordinates BD , EC are applied meeting the curve in B , C , and then the points B , C approach the point A : the areas of the triangles ADB , AEC will be ultimately to each other in the duplicate ratio of the sides.*

In modern terms: $\text{area}(ADB)/\text{area}(AEC) \rightarrow (AD/AE)^2$ as $B, C \rightarrow A$, the triangles having the curved side.

Proof. On AD produced take Ad , Ae proportional to AD , AE , and erect ordinates db , ec parallel and proportional to DB , EC . Produce AC to c , draw the curve Abc similar to ABC , and the straight line Ag touching both curves at A and cutting the ordinates in F , G , f , g . Then let B and C coincide with A : the angle cAg vanishes, the curvilinear areas Abd , Ace coincide with the rectilinear Afd , Age , and so (Lemma V) are in the duplicate ratio of the sides Ad , Ae . But the areas ABD , ACE are always proportional to these, and the sides AD , AE to these sides. So the areas ABD , ACE are ultimately in the duplicate ratio of the sides AD , AE .

1713. The enlarged figure is again kept finite: while B , C approach A , the

line AD is always produced to distant points d, e , and the points B, C coalesce with A while the length Ae stays fixed.

Lean reconstruction: 1 theorem in 1 module cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Principial687/ConstructedRatio.lean, 1 theorem, 25 lines

```
import NewtonLimitDynamics.Principial687.LemmaX
import NewtonLimitDynamics.Common.RationalMagnitudes

namespace Principial687
open NewtonLimitDynamics NewtonLimitDynamics.Fraction

/-- Explicit contact and area-enclosure obligations for the constructed ratio.
    Areas use half base times height; half the secant slope has limit c.
    Establishing these data from an actual curved diagram remains open. -/
structure ContactEnclosure (s : Fraction → Fraction) (c : Fraction) where
  lowerSlope : Fraction → Fraction
  upperSlope : Fraction → Fraction
  lower_contact : Ultimate magnitudes (fun t => half (lowerSlope t)) c
  upper_contact : Ultimate magnitudes (fun t => half (upperSlope t)) c
  mechanical_enclosure : Near magnitudes (fun t =>
    le (ratio (fun u => triangleArea u (mul (lowerSlope u) u)) t) (ratio s t) ∧
    le (ratio s t) (ratio (fun u => triangleArea u (mul (upperSlope u) u)) t))

theorem constructed_quadratic_bridge (s : Fraction → Fraction) (c : Fraction)
  (hc : positive c) (p : ContactEnclosure s c) :
  DeMotu1684.QuadraticInitialDeflection magnitudes (ratio s) c := by
  refine (hc, enclosure_reconstruction magnitudes _ _ c ?_ ?_ p.mechanical_enclosure)
  · exact constructed_triangle_limit p.lowerSlope c p.lower_contact
  · exact constructed_triangle_limit p.upperSlope c p.upper_contact
end Principial687
```

Lemma X. *The spaces which a body describes, urged by any regular force, are at the very beginning of the motion in the duplicate ratio of the times.*

1713. *The spaces which a body describes, urged by any finite force, whether that force is determinate and immutable or is continually increased or continually decreased, are at the very beginning of the motion in the duplicate ratio of the times.* The 1687 “regular force” becomes a finite force that is constant or monotone; Corollaries 1 and 2 are reworded accordingly (“by any equal forces”, “measured by the distances of the bodies from those places of the similar figures”).

In modern terms: $s(t_1)/s(t_2) \rightarrow (t_1/t_2)^2$ as $t_1, t_2 \rightarrow 0$; that is, $s(t) \sim c t^2$ at the start.

Proof. Represent the times by the lines AD, AE and the velocities generated by the ordinates DB, EC . The spaces described with these velocities are as the areas ABD, ACE described by these ordinates, that is, at the very beginning of the motion (Lemma IX), in the duplicate ratio of the times AD, AE .

In modern terms: $s(t) = \int_0^t v(\tau) d\tau$, the area under the velocity graph, whose initial piece is the curvilinear triangle of Lemma IX.

Corollary 1. Hence the errors of bodies describing similar parts of similar figures in proportional times, generated by equal forces similarly applied to the bodies in those parts, and measured from the places of the figures which the bodies would reach in the same proportional times without those forces, are very nearly as the squares of the times in which they are generated.

Corollary 2. The errors generated by proportional forces similarly applied are as the forces and the squares of the times jointly.

In modern terms: the deviation from the force-free path is $\delta \approx \frac{1}{2}at^2$, so $\delta \propto Ft^2$ for a given body.

1713. Three new corollaries and a Scholium.

Corollary 3. The same holds for any spaces that bodies describe under different forces: at the very beginning of the motion they are as the forces and the squares of the times jointly, $s \propto Ft^2$.

Corollary 4. So the forces are as the spaces described at the very beginning directly and the squares of the times inversely, $F \propto s/t^2$.

Corollary 5. And the squares of the times are as the spaces directly and the forces inversely, $t^2 \propto s/F$.

Scholium. When indeterminate quantities of different kinds are compared, and one is said to be as another directly or inversely, the meaning is that the first increases or decreases in the same ratio as the second or as its reciprocal. When one is said to be as two or more others directly or inversely, it increases or decreases in the ratio compounded of the ratios in which the others, or their reciprocals, increase or decrease. So “ A is as B directly and C directly and D inversely” means that A changes in the same ratio as $B \cdot C/D$, that is, A and BC/D are in a given ratio.

Lean reconstruction: 15 theorems in 5 modules cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Principial687/LemmaX.lean, 2 theorems, 33 lines

```
import NewtonLimitDynamics.DeMotu1684.QuadraticDeflection

namespace Principial687
open NewtonLimitDynamics

/-- Explicit obligations for Newton's area-diagram route (NATP00077 par28).
    Lemma IX geometry and the velocity-area representation remain premises. -/
structure LemmaXPremises {Q : Type} (g : Magnitudes Q)
  (deflectionRatio : Q → Q) (c : Q) where
  areaRatio : Q → Q
  lowerTriangle : Q → Q
  upperTriangle : Q → Q
  velocity_area : ∀ h, deflectionRatio h = areaRatio h
  lower_limit : Ultimate g lowerTriangle c
  upper_limit : Ultimate g upperTriangle c
  enclosure : Near g (fun h => g.le (lowerTriangle h) (areaRatio h) ∧
    g.le (areaRatio h) (upperTriangle h))

theorem lemmaX_reconstruction {Q : Type} (g : Magnitudes Q)
  (ratio : Q → Q) (c : Q) (p : LemmaXPremises g ratio c) :
  Ultimate g ratio c := by
  have heq : ratio = p.areaRatio := funext p.velocity_area
  rw [heq]
  exact enclosure_reconstruction g _ _ c p.lower_limit p.upper_limit p.enclosure

/-- Conditional bridge; not a completed historical discharge of Hypothesis 4. -/
theorem discharges_quadraticPremise_reconstruction {Q : Type}
  (g : Magnitudes Q) (ratio : Q → Q) (c : Q)
  (hc : g.positive c) (p : LemmaXPremises g ratio c) :
  DeMotu1684.QuadraticInitialDeflection g ratio c :=
  (hc, lemmaX_reconstruction g ratio c p)
```

end Principial687

NewtonLimitDynamics/Principial713/LemmaX.lean, 1 theorem, 17 lines

```
import NewtonLimitDynamics.Principial687.LemmaX

namespace Principial713
open NewtonLimitDynamics

/-- Shared geometry of the proof, not an assertion that the editions state
    identical mechanical hypotheses. See NATP00082 par28-34. -/
structure LemmaXPremises {Q : Type} (g : Magnitudes Q)
  (ratio : Q → Q) (c : Q) where
  geometry : Principial687.LemmaXPremises g ratio c

theorem lemmaX_reconstruction {Q : Type} (g : Magnitudes Q)
  (ratio : Q → Q) (c : Q) (p : LemmaXPremises g ratio c) :
  Ultimate g ratio c :=
  Principial687.lemmaX_reconstruction g ratio c p.geometry

end Principial713
```

NewtonLimitDynamics/Principial713/ForceComparison.lean, 2 theorems, 32 lines

```
import NewtonLimitDynamics.Common.RationalMagnitudes

namespace Principial713
open NewtonLimitDynamics NewtonLimitDynamics.Fraction

/-- Division of represented magnitudes requires a positive divisor. -/
def quotient (a b : Fraction) (hb : positive b) : Fraction :=
  (a.num * b.den, a.den * b.num, Int.mul_pos a.den_pos hb)

/-- Exact coefficient model of Corollary 3, NOT an assertion that variable
    force has this finite-time law. k is shared calibration, and f denotes
    accelerative force (or motive force with matched mass). -/
def generated (k f t : Fraction) : Fraction := mul (mul k f) (mul t t)

theorem corollary4_coefficient_reconstruction (k f t : Fraction)
  (hk : positive k) (ht : positive t) :
  equiv (quotient (generated k f t) (mul k (mul t t))
    (positive_mul k _ hk (positive_mul t t ht ht))) f := by
  unfold equiv quotient generated mul
  dsimp
  ac_rfl

/-- Unlike Corollary 4, this rearrangement excludes zero force. -/
theorem corollary5_coefficient_reconstruction (k f t : Fraction)
  (hk : positive k) (hf : positive f) :
  equiv (quotient (generated k f t) (mul k f) (positive_mul k f hk hf))
    (mul t t) := by
  unfold equiv quotient generated mul
  dsimp
  ac_rfl

end Principial713
```

NewtonLimitDynamics/Polygon/MonotoneEnclosure.lean, 6 theorems, 97 lines

```
/-!
Finite form of the enclosure behind Lemma X, under the force clause that 1713
added (NATP00082 par28: *Vi finita, sive Vis illa determinata & immutabilis
sit, sive eadem continuo augetur vel continuo diminuatur*; 1687 NATP00077
par27 has *vi regulari*). One-dimensional impulse construction with unit
time cells and integer magnitudes, starting at rest: drift with the current
velocity, then the impulse `a i` of cell `i`. If every impulse lies between
`lo` and `hi`, the space described lies between the spaces described under the
constant forces `lo` and `hi`. For a monotone force history these are the
initial and final forces. A finite but non-monotone history escapes that
enclosure. Rational cells follow by a common scaling (not formalized); no
```

```

limit, curve, or continuous force is produced.
-/

namespace NewtonLimitDynamics.Polygon.MonotoneEnclosure

/-- Velocity before cell `k`. -/
def vel (a : Nat → Int) : Nat → Int
| 0 => 0
| k + 1 => vel a k + a k

/-- Space described after `n` cells. -/
def pos (a : Nat → Int) : Nat → Int
| 0 => 0
| n + 1 => pos a n + vel a n

/-- `0 + 1 + ... + (n-1)`: the space coefficient of a constant unit force. -/
def tri : Nat → Int
| 0 => 0
| n + 1 => tri n + n

/-- Under a constant force the space described is `force * tri n`. -/
theorem pos_const (c : Int) : (n : Nat) → pos (fun _ => c) n = c * tri n ∧ vel (fun _ => c) n = c * n
| 0 => by simp [pos, vel, tri]
| n + 1 => by
  obtain {hp, hv} := pos_const c n
  simp only [pos, vel, tri, hp, hv, Int.mul_add, Int.mul_one, Int.ofNat_add]
  refine (trivial, ?_)
  rw [Int.ofNat_one, Int.mul_one]

theorem vel_bounds (a : Nat → Int) (lo hi : Int) (n : Nat)
  (h : ∀ i, i < n → lo ≤ a i ∧ a i ≤ hi) :
  ∀ k, k ≤ n → lo * k ≤ vel a k ∧ vel a k ≤ hi * k := by
  intro k
  induction k with
  | zero => intro _; simp [vel]
  | succ k ih =>
    intro hk
    obtain {h1, h2} := ih (Nat.le_of_succ_le hk)
    obtain {h3, h4} := h k (Nat.lt_of_succ_le hk)
    simp only [vel, Int.ofNat_add, Int.mul_add, Int.mul_one]
    constructor <=> omega

/-- Enclosure: impulses between `lo` and `hi` put the space described between
the spaces described under the constant forces `lo` and `hi`. -/
theorem enclosure (a : Nat → Int) (lo hi : Int) (n : Nat)
  (h : ∀ i, i < n → lo ≤ a i ∧ a i ≤ hi) :
  lo * tri n ≤ pos a n ∧ pos a n ≤ hi * tri n := by
  have hv := vel_bounds a lo hi n h
  have key : ∀ m, m ≤ n → lo * tri m ≤ pos a m ∧ pos a m ≤ hi * tri m := by
    intro m
    induction m with
    | zero => intro _; simp [pos, tri]
    | succ m ih =>
      intro hm
      obtain {h1, h2} := ih (Nat.le_of_succ_le hm)
      obtain {h3, h4} := hv m (Nat.le_of_succ_le hm)
      simp only [pos, tri, Int.mul_add]
      constructor <=> omega
  exact key n (Nat.le_refl n)

/-- The 1713 clause: a force that continually increases (monotone history)
gives the enclosure between the initial and final forces. -/
theorem monotone_enclosure (a : Nat → Int) (n : Nat)
  (hmono : ∀ i j, i ≤ j → j ≤ n → a i ≤ a j) :
  a 0 * tri (n + 1) ≤ pos a (n + 1) ∧ pos a (n + 1) ≤ a n * tri (n + 1) :=
  enclosure a (a 0) (a n) (n + 1) (fun i hi =>
    (hmono 0 i (Nat.zero_le i) (Nat.le_of_lt_succ hi),
    hmono i n (Nat.le_of_lt_succ hi) (Nat.le_refl n)))

/-- And a force that continually decreases, symmetrically. -/

```

```

theorem antitone_enclosure (a : Nat → Int) (n : Nat)
  (hanti : ∀ i j, i ≤ j → j ≤ n → a j ≤ a i) :
  a n * tri (n + 1) ≤ pos a (n + 1) ∧ pos a (n + 1) ≤ a 0 * tri (n + 1) :=
  enclosure a (a n) (a 0) (n + 1) (fun i hi =>
    (hanti i n (Nat.le_of_lt_succ hi) (Nat.le_refl n),
    hanti 0 i (Nat.zero_le i) (Nat.le_of_lt_succ hi)))

/-- A finite force that rises and falls (impulses 0, 5, 0) describes space 5,
    outside the enclosure `0 ≤ pos ≤ 0` fixed by its initial and final forces. -/
theorem nonmonotone_escapes :
  pos (fun i => if i = 1 then 5 else 0) 3 = 5 ∧
  (fun i => if i = 1 then (5 : Int) else 0) 0 * tri 3 = 0 ∧
  (fun i => if i = 1 then (5 : Int) else 0) 2 * tri 3 = 0 := by
  decide

end NewtonLimitDynamics.Polygon.MonotoneEnclosure

NewtonLimitDynamics/Contact/AreaCoefficient.lean, 4 theorems, 171 lines

import NewtonLimitDynamics.Contact.FiniteSums

namespace NewtonLimitDynamics.Contact

private theorem zero_of_scaled_bounds (delta C : Int)
  (h : ∀ n : Nat, 0 < n → (n : Int)*delta ≤ C ∧ (n : Int)*(-delta) ≤ C) : delta = 0 := by
  let n := C.natAbs + 1
  have hn : 0 < n := by omega
  obtain {hu, hl} := h n hn
  have habs : C ≤ (C.natAbs : Int) := Int.le_natAbs
  have hnlarge : C < (n : Int) := by dsimp [n]; omega
  have hnp : 0 ≤ (n : Int) := by omega
  by_cases hp : 0 < delta
  · have hd : 1 ≤ delta := by omega
    have hm := Int.mul_le_mul_of_nonneg_left hd hnp
    simp only [Int.mul_one] at hm
    omega
  · by_cases hm : delta < 0
    · have hd : 1 ≤ -delta := by omega
      have hb := Int.mul_le_mul_of_nonneg_left hd hnp
      simp only [Int.mul_one] at hb
      omega
    · omega

def lowerParabola (n : Nat) (hn : 0 < n) : Fraction :=
  ((nsum (fun i => i*i) n : Int), (n*n*n : Nat),
  Int.ofNat_lt.mpr (Nat.mul_pos (Nat.mul_pos hn hn) hn))
def upperParabola (n : Nat) (hn : 0 < n) : Fraction :=
  ((nsum (fun i => i*i) n + n*n : Nat), (n*n*n : Nat),
  Int.ofNat_lt.mpr (Nat.mul_pos (Nat.mul_pos hn hn) hn))

/-- Rectangle enclosure determines the normalized parabolic area as 1/3.
    Only finite sums and rational order are used. The geometric assertion that
    a selected curve's area obeys these rectangle enclosures is the premise. -/
theorem parabolic_area_coefficient (area : Fraction)
  (enclosed : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerParabola n hn) area ∧ Fraction.le area (upperParabola n hn)) :
  Fraction.equiv area (1, 3, by decide) := by
  have heq : 3*area.num-area.den = 0 := by
    apply zero_of_scaled_bounds _ (2*area.den)
    intro n hn
    let N : Int := n
    let S : Int := nsum (fun i => i*i) n
    have hN : 0 < N := Int.ofNat_lt.mpr hn
    have hNN := Int.mul_pos hN hN
    obtain {hl, hu} := enclosed n hn
    change S*area.den ≤ area.num*(N*N*N) at hl
    change area.num*(N*N*N) ≤ (S+N*N)*area.den at hu
    have hi := congrArg (fun x : Nat => (x : Int)) (quadratic_rectangles n)
    simp only [Int.ofNat_add, Int.ofNat_mul] at hi
    have hid := congrArg (fun x : Int => x*area.den) hi
    have ident : 6*(S*area.den)+3*(N*N*area.den) =

```

```

      2*(N*N*area.den)+N*area.den := by
    simp only [Int.add_mul, Int.mul_assoc] using hid
    have hlo := Int.mul_le_mul_of_nonneg_left hl (by decide : (0 : Int) ≤ 6)
    have hhi := Int.mul_le_mul_of_nonneg_left hu (by decide : (0 : Int) ≤ 6)
    simp only [Int.add_mul, Int.mul_add] at hhi
    have wpos := Int.mul_pos hN area.den_pos
    have wle : N*area.den ≤ N*N*area.den := by
      have hge : (1 : Int) ≤ N := by omega
      have ht := Int.mul_le_mul_of_nonneg_right hge (Int.le_of_lt wpos)
      simp only [Int.one_mul, Int.mul_assoc] using ht
    have hupper : 6*(area.num*(N*N*N))-2*(N*N*N*area.den) ≤ 4*(N*N*area.den) := by omega
    have hlower : 2*(N*N*N*area.den)-6*(area.num*(N*N*N)) ≤ 4*(N*N*area.den) := by omega
    have factor : 2*(N*(3*area.num-area.den))*(N*N) =
      6*(area.num*(N*N*N))-2*(N*N*N*area.den) := by
      simp only [Int.mul_sub, Int.sub_mul]
    have six : (6 : Int) = 2*3 := by decide
    rw [six]
    congr 1 <=> ac_rfl
    have factor' : 2*(N*(area.den-3*area.num))*(N*N) =
      2*(N*N*N*area.den)-6*(area.num*(N*N*N)) := by
      simp only [Int.mul_sub, Int.sub_mul]
    have six : (6 : Int) = 2*3 := by decide
    rw [six]
    congr 1 <=> ac_rfl
    have rhs : 4*(N*N*area.den) = (4*area.den)*(N*N) := by ac_rfl
    rw [← factor, rhs] at hupper
    rw [← factor', rhs] at hlower
    have hu' := Int.le_of_mul_le_mul_right hupper hNN
    have hl' := Int.le_of_mul_le_mul_right hlower hNN
    have hnneg : N*(-(3*area.num-area.den)) = N*(area.den-3*area.num) := by
      congr 1
      omega
    change N*(3*area.num-area.den) ≤ 2*area.den ∧ N*(-(3*area.num-area.den)) ≤ 2*area.den
    rw [hnneg]
    omega
  unfold Fraction.equiv
  dsimp
  omega

def lowerLinear (n : Nat) (hn : 0 < n) : Fraction :=
  ((nsum (fun i => i) n : Int), (n*n : Nat),
   Int.ofNat_lt.mpr (Nat.mul_pos hn hn))
def upperLinear (n : Nat) (hn : 0 < n) : Fraction :=
  ((nsum (fun i => i) n + n : Nat), (n*n : Nat),
   Int.ofNat_lt.mpr (Nat.mul_pos hn hn))

/-- Normalized linear velocity area is 1/2, from finite rectangles alone. -/
theorem linear_area_coefficient (area : Fraction)
  (enclosed : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerLinear n hn) area ∧ Fraction.le area (upperLinear n hn)) :
  Fraction.equiv area (1, 2, by decide) := by
  have heq : 2*area.num-area.den = 0 := by
    apply zero_of_scaled_bounds _ area.den
    intro n hn
    let N : Int := n
    let S : Int := nsum (fun i => i) n
    have hN : 0 < N := Int.ofNat_lt.mpr hn
    obtain {hl, hu} := enclosed n hn
    change S*area.den ≤ area.num*(N*N) at hl
    change area.num*(N*N) ≤ (S+N)*area.den at hu
    have hi := congrArg (fun x : Nat => (x : Int)) (linear_rectangles n)
    simp only [Int.ofNat_add, Int.ofNat_mul] at hi
    have hid := congrArg (fun x : Int => x*area.den) hi
    have ident : 2*(S*area.den)+N*area.den = N*N*area.den := by
      simp only [Int.add_mul, Int.mul_assoc] using hid
    have hlo := Int.mul_le_mul_of_nonneg_left hl (by decide : (0 : Int) ≤ 2)
    have hhi := Int.mul_le_mul_of_nonneg_left hu (by decide : (0 : Int) ≤ 2)
    simp only [Int.add_mul, Int.mul_add] at hhi
    have hupper : 2*(area.num*(N*N))-N*N*area.den ≤ N*area.den := by omega
    have hlower : N*N*area.den-2*(area.num*(N*N)) ≤ N*area.den := by omega

```

```

have factor : (N*(2*area.num-area.den))*N = 2*(area.num*(N*N))-N*N*area.den := by
  simp only [Int.mul_sub, Int.sub_mul]
  congr 1 <=> ac_rfl
have factor' : (N*(area.den-2*area.num))*N = N*N*area.den-2*(area.num*(N*N)) := by
  simp only [Int.mul_sub, Int.sub_mul]
  congr 1 <=> ac_rfl
have rhs : N*area.den = area.den*N := by ac_rfl
rw [← factor, rhs] at hupper
rw [← factor', rhs] at hlower
have hu' := Int.le_of_mul_le_mul_right hupper hN
have hl' := Int.le_of_mul_le_mul_right hlower hN
have hnneg : N*(-(2*area.num-area.den)) = N*(area.den-2*area.num) := by
  congr 1
  omega
change N*(2*area.num-area.den) ≤ area.den ∧ N*(-(2*area.num-area.den)) ≤ area.den
rw [hnneg]
exact {hu', hl'}
unfold Fraction.equiv
dsimp
omega

/-- Constant-force example with unit tangential speed and zero initial normal
velocity. Similarity scales normalized rectangle areas. Mechanical
identification with displacement/defect is still separate from this
geometric coefficient theorem. -/
theorem constant_force_area_coefficients (acc time linearArea parabolaArea : Fraction)
  (hlinear : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerLinear n hn) linearArea ∧ Fraction.le linearArea (upperLinear n hn))
  (hparabola : ∀ n : Nat, (hn : 0 < n) →
    Fraction.le (lowerParabola n hn) parabolaArea ∧ Fraction.le parabolaArea (upperParabola n hn)) :
  Fraction.equiv (Fraction.mul (Fraction.mul acc (Fraction.mul time time)) linearArea)
    (Fraction.half (Fraction.mul acc (Fraction.mul time time))) ∧
  Fraction.equiv (Fraction.mul (Fraction.half (Fraction.mul acc (Fraction.mul time (Fraction.mul time
    ↪ time)))) parabolaArea)
    (Fraction.mul (1, 6, by decide) (Fraction.mul acc (Fraction.mul time (Fraction.mul time time)))) :=
  ↪ by
  constructor
  · apply Fraction.equiv_trans (Fraction.mul_equiv_left _ (linear_area_coefficient linearArea hlinear))
    unfold Fraction.equiv Fraction.mul Fraction.half
    dsimp
    simp only [Int.one_mul, Int.mul_one]
    ac_rfl
  · apply Fraction.equiv_trans (Fraction.mul_equiv_left _ (parabolic_area_coefficient parabolaArea
    ↪ hparabola))
    unfold Fraction.equiv Fraction.mul Fraction.half
    dsimp
    simp only [Int.one_mul, Int.mul_one]
    have six : (6 : Int) = 2*3 := by decide
    rw [six]
    ac_rfl
end NewtonLimitDynamics.Contact

```

Lemma XI. *The vanishing subtense of the angle of contact is ultimately in the duplicate ratio of the subtense of the conterminous arc.*

In modern terms: with BD the distance from B on the curve to the tangent at A , measured in a fixed direction, $BD \propto AB^2$ ultimately; for finite curvature κ , $BD \approx \frac{1}{2}\kappa AB^2$.

1713. The statement is restricted: *in all curves having finite curvature at the point of contact.*

Case 1. Let the arc be AB , its tangent AD , the subtense BD of the angle of contact perpendicular to the tangent, and AB the subtense of the arc. Erect AG perpendicular to the subtense AB and BG perpendicular to the tangent AD , meeting in G . Then

let the points D, B, G go to d, b, g , and let I be the intersection of the lines BG, AG ultimately formed when D, B reach A . The distance GI can be made less than any assigned length. By the nature of circles through A, B, G and through A, b, g , $AB^2 = AG \cdot BD$ and $Ab^2 = Ag \cdot bd$, so the ratio $AB^2 : Ab^2$ is compounded of $AG : Ag$ and $BD : bd$. Since GI can be taken less than any assigned length, the ratio $AG : Ag$ can differ from equality by less than any assigned difference, and so $AB^2 : Ab^2$ can differ from $BD : bd$ by less than any assigned difference. By Lemma I the ultimate ratio $AB^2 : Ab^2$ equals the ultimate ratio $BD : bd$.

Case 2. Let BD be inclined to AD at any given angle; the ultimate ratio $BD : bd$ is the same as before, and so the same as $AB^2 : Ab^2$.

Case 3. Even if the angle D is not given, the angles D, d always tend to equality and approach each other nearer than by any assigned difference, so they are ultimately equal (Lemma I), and the lines BD, bd are in the same ratio as before.

1713. Case 3 also covers the line BD converging to a given point, or placed by any other law, provided the angles D, d are placed by a common law.

Corollary 1. Since the tangents AD, Ad , the arcs AB, Ab and their sines BC, bc become ultimately equal to the chords AB, Ab , their squares too are ultimately as the subtenses BD, bd .

1713. Two new corollaries follow, and the old Corollaries 2 and 3 become Corollaries 4 and 5.

Corollary 2. Their squares are also ultimately as the sagittas of the arcs that bisect the chords and converge to a given point, for those sagittas are as the subtenses BD, bd .

Corollary 3. Hence the sagitta is in the duplicate ratio of the time in which a body describes the arc with a given velocity.

Corollary 2. The rectilinear triangles ADB, Adb are ultimately in the triplicate ratio of the sides AD, Ad , and in the sesquiplicate ratio of the sides DB, db , being in the ratio compounded of AD and DB, Ad and db . Likewise the triangles ABC, Abc are ultimately in the triplicate ratio of the sides BC, bc .

1713 (now Corollary 4) adds a definition: the sesquiplicate ratio is the subduplicate of the triplicate, compounded of the simple and the subduplicate ratio, which others call sesquialteral; that is, the ratio of $3/2$ powers.

Corollary 3. Since DB, db are ultimately parallel and in the duplicate ratio of AD, Ad , the ultimate curvilinear areas ADB, Adb are (by the nature of the parabola) two thirds of the rectilinear triangles ADB, Adb , and the segments AB, Ab one third of the same triangles. Hence these areas and segments are in the triplicate ratio both of the tangents AD, Ad and of the chords and arcs AB, Ab .

Lean reconstruction: 13 theorems in 2 modules cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Contact/Bounds.lean, 4 theorems, 64 lines

```
import NewtonLimitDynamics.Common.RationalMagnitudes
import NewtonLimitDynamics.Principia1713.ForceComparison
```



```

namespace NewtonLimitDynamics.Contact
open Fraction

/-- The normal contact subtense, the tangent departure and the defect area
are separate quantities. Identifying any two is an additional configuration
hypothesis. For oblique contact subtenses an angle factor is required. -/
structure Quantities where
  tangentLength : Fraction
  chordLength : Fraction
  normalSubtense : Fraction
  tangentDeparture : Fraction
  defectArea : Fraction

theorem le_quotient_iff (a b c : Fraction) (hc : positive c) :
  le a (Principal713.quotient b c hc) ↔ le (mul a c) b := by
  unfold le Principal713.quotient mul
  dsimp
  have e1 : a.num * (b.den * c.num) = a.num * c.num * b.den := by ac_rfl
  have e2 : b.num * c.den * a.den = b.num * (a.den * c.den) := by ac_rfl
  rw [e1, e2]

/-- Circle identity  $AB^2=AG*BD$  imported from Lemma XI case 1. A uniform
positive lower bound on AG is essential; the identity alone is insufficient.
This proves an inequality, not existence of the osculating configuration. -/
theorem normal_subtense_bound (chord subtense diameter minDiameter : Fraction)
  (hs : positive subtense) (hd : positive minDiameter)
  (hmin : le minDiameter diameter)
  (circle : equiv (mul diameter subtense) (mul chord chord)) :
  le subtense (Principal713.quotient (mul chord chord) minDiameter hd) := by
  apply (le_quotient_iff _ _ hd).mpr
  have hc := (equiv_iff_mutual_le _ _).mp (mul_comm subtense minDiameter)
  have hm := mul_le_mul_positive hmin subtense hs
  have he := (equiv_iff_mutual_le _ _).mp circle
  exact magnitudes.le_trans hc.1 (magnitudes.le_trans hm he.1)

/-- Explicit finite cubic inequality from a rectangle enclosing the defect.
K is a proved/assumed quadratic bound valid on the SAME neighbourhood.
No assertion of contact or enclosure is hidden in power notation. -/
theorem cubic_rectangle_bound (base departure defect K : Fraction)
  (hb : positive base)
  (hquad : le departure (mul K (mul base base)))
  (henclose : le defect (mul departure base)) :
  le defect (mul K (mul base (mul base base))) := by
  have hm := mul_le_mul_positive hquad base hb
  have he : equiv (mul (mul K (mul base base)) base) (mul K (mul base (mul base base))) := by
    unfold equiv mul
  dsimp
  ac_rfl
  exact magnitudes.le_trans henclose (magnitudes.le_trans hm ((equiv_iff_mutual_le _ _).mp he).1)

/-- Coordinate construction for Lemma XI case 1: A=(0,0), B=(x,y),
G=(0,D), with AB perpendicular to BG. The corresponding right-triangle
relation yields  $AB^2=AG*BD$ ; existence of G and its limiting position is
deliberately not inferred. This is a reconstruction of circle geometry. -/
theorem circle_identity_from_perpendicular (x y D : Int)
  (perpendicular : x*x + y*(y-D) = 0) : x*x+y*y = D*y := by
  rw [Int.mul_sub] at perpendicular
  have h : y*D = D*y := Int.mul_comm _ _
  omega

```

```

end NewtonLimitDynamics.Contact

```

NewtonLimitDynamics/Contact/FiniteSums.lean, 9 theorems, 125 lines

```

import NewtonLimitDynamics.Polygon.Enclosure

```

```

namespace NewtonLimitDynamics.Contact

```

```

def nsum (f : Nat → Nat) : Nat → Nat
| 0 => 0
| n+1 => nsum f n + f n

```

```

/-- Lower rectangles for a linear velocity diagram. With n equal cells the
    doubled lower sum differs from  $n^2$  by n, yielding coefficient 1/2. -/
theorem linear_rectangles (n : Nat) : 2 * nsum (fun i => i) n + n = n*n := by
  induction n with
  | zero => decide
  | succ n ih =>
    simp only [nsum, Nat.mul_add, Nat.add_mul, Nat.mul_one, Nat.one_mul]
    omega

/-- Lower rectangles for  $y=x^2$ . Division by  $6n^3$  yields the area coefficient
    1/3 with explicit corrections  $-1/(2n)+1/(6n^2)$ , not an integral theorem. -/
theorem quadratic_rectangles (n : Nat) :
  6 * nsum (fun i => i*i) n + 3*(n*n) = 2*(n*n*n)+n := by
  induction n with
  | zero => decide
  | succ n ih =>
    simp only [nsum, Nat.mul_add, Nat.add_mul, Nat.mul_one, Nat.one_mul]
    omega

theorem upper_lower_gap (n : Nat) :
  nsum (fun i => (i+1)*(i+1)) n = nsum (fun i => i*i) n + n*n := by
  induction n with
  | zero => decide
  | succ n ih => simp only [nsum, ih]

/-- Error of the lower quadratic rectangle sum in units of  $6n^3$  is  $\leq 3n^2$ ;
    after positive division this is  $\leq 1/(2n)$ . -/
theorem quadratic_lower_error (n : Nat) :
  2*(n*n*n)  $\leq$  6*nsum (fun i => i*i) n + 3*(n*n) := by
  have := quadratic_rectangles n
  omega

theorem sum_bound (f : Nat → Nat) (C n : Nat) (hf :  $\forall i, i < n \rightarrow f i \leq C$ ) :
  nsum f n  $\leq$  n*C := by
  induction n with
  | zero => simp [nsum]
  | succ n ih =>
    have h := ih (fun i hi => hf i (by omega))
    have hn := hf n (by omega)
    simp only [nsum, Nat.add_mul, Nat.one_mul]
    omega

/-- Uniform partition error, expressed without division:  $n^2 \sum \text{error}_i \leq C$ .
    All errors use a common unit; the local bound is  $n^3 * \text{error}_i \leq C$  for EVERY
    cell of the fixed interval. C must include the fixed interval's  $T^3$  factor.
    Positivity of n permits cancellation; pointwise local scaling is not enough. -/
theorem uniform_partition_error (error : Nat → Nat) (C n : Nat) (hn :  $0 < n$ )
  (hlocal :  $\forall i, i < n \rightarrow n^3 * \text{error } i \leq C$ ) :
  n*n*nsum error n  $\leq$  C := by
  have hs := sum_bound (fun i => n*n*n*error i) C n hlocal
  have hmul :  $\forall k, \text{nsum } (\text{fun } i \Rightarrow n^3 * \text{error } i) k = n^3 * \text{nsum error } k$  := by
    intro k
    induction k with
    | zero => simp [nsum]
    | succ k ih => simp [nsum, ih, Nat.mul_add]
  rw [hmul] at hs
  have he :  $n^3 * \text{nsum error } n = n * (n^2 * \text{nsum error } n)$  := by ac_rfl
  rw [he] at hs
  exact Nat.le_of_mul_le_mul_left hs hn

theorem nsum_mul (f : Nat → Nat) (k n : Nat) :
  nsum (fun i => f i*k) n = nsum f n*k := by
  induction n with
  | zero => simp [nsum]
  | succ n ih => simp [nsum, ih, Nat.add_mul]

/-- Rational version: errors have a common denominator D within this partition,
    and the fixed uniform coefficient is  $K_{num}/K_{den}$ . D may vary with refinement.
    Hence this is not restricted to integer errors eventually becoming zero. -/

```

```

theorem rational_uniform_partition_error (error : Nat → Nat)
  (Knum Kden D n : Nat) (hk : 0 < Kden) (hd : 0 < D) (hn : 0 < n)
  (hlocal : ∀ i, i < n → n*n*(error i*Kden) ≤ Knum*D) :
  Fraction.le
    ((nsum error n : Int), (D : Int), Int.ofNat_lt.mpr hd)
    ((Knum : Int), (Kden*(n*n) : Nat),
      Int.ofNat_lt.mpr (Nat.mul_pos hk (Nat.mul_pos hn hn))) := by
  have h := uniform_partition_error (fun i => error i*Kden) (Knum*D) n hn hlocal
  rw [nsum_mul] at h
  have he : n*n*(nsum error n*Kden) = nsum error n*(Kden*(n*n)) := by ac_rfl
  rw [he] at h
  unfold Fraction.le
  dsimp
  simpa only [Int.ofNat_mul] using (Int.ofNat_le.mpr h)

/-- The same error sum with a nonuniform coefficient cannot in general obey
    the proposed uniform budget. This boundary is executable without analysis. -/
example : ¬ (2*2*nsum (fun _ => 1) 2 ≤ 1) := by decide

/-- Sequential vanishing appropriate to a finite-sum refinement index. -/
def SeqVanishes (error : Nat → Fraction) : Prop :=
  ∀ epsilon, Fraction.positive epsilon →
  ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (error n) epsilon

/-- An explicit Archimedean argument for rational error budgets C/n.
    This imports no analytic convergence theorem. -/
theorem reciprocal_budget_vanishes (error : Nat → Fraction) (C : Nat)
  (hbound : ∀ n, (hn : 0 < n) → Fraction.le (error n)
    ((C : Int), (n : Int), Int.ofNat_lt.mpr hn)) : SeqVanishes error := by
  intro e he
  refine (C * e.den.natAbs + 1, ?_)
  intro n hn
  have hnpos : 0 < n := by omega
  have hd : (e.den.natAbs : Int) = e.den := Int.natAbs_of_nonneg (Int.le_of_lt e.den_pos)
  have hlarge : (C : Int) * e.den < (n : Int) := by
    have hnats : C * e.den.natAbs < n := by omega
    have hi := Int.ofNat_lt.mpr hnats
    simpa only [Int.ofNat_mul, hd] using hi
  have hen : 1 ≤ e.num := by unfold Fraction.positive at he; omega
  have hm := Int.mul_le_mul_of_nonneg_right hen (Int.le_of_lt (Int.ofNat_lt.mpr hnpos))
  have hbracket : Fraction.lt ((C : Int), (n : Int), Int.ofNat_lt.mpr hnpos) e := by
    unfold Fraction.lt
    dsimp
    simp only [Int.one_mul] at hm
    omega
  exact Fraction.magnitudes.lt_of_le_lt (hbound n hnpos) hbracket
end NewtonLimitDynamics.Contact

```

Scholium

In all this we suppose the angle of contact neither infinitely greater than the angles of contact which circles make with their tangents, nor infinitely less; that is, the curvature at the point A neither infinitely small nor infinitely great, or the interval AI of finite magnitude. For DB can be taken as AD^3 , in which case no circle can be drawn through A between the tangent AD and the curve AB , and the angle of contact is infinitely less than circular ones. By a similar argument, taking DB successively as AD^4 , AD^5 , AD^6 , AD^7 , ... gives a series of angles of contact going on to infinity, each infinitely less than the one before. Taking DB successively as AD^2 , $AD^{3/2}$, $AD^{4/3}$, $AD^{5/4}$, $AD^{6/5}$, $AD^{7/6}$, ... gives another infinite series, the first of the same kind as the circular ones, the second infinitely greater, and each infinitely greater than the one before. Between any two of these angles a series going on to infinity both ways can be inserted, each infinitely greater than the one before: for instance, between the terms AD^2 and AD^3

the series $AD^{13/6}$, $AD^{11/5}$, $AD^{9/4}$, $AD^{7/3}$, $AD^{5/2}$, $AD^{8/3}$, $AD^{11/4}$, $AD^{14/5}$, $AD^{17/6}$, And again between any two angles of this series a new series of intermediate angles can be inserted, differing from each other by infinite intervals. Nature knows no limit.

In modern terms: $DB \propto AD^p$ near A has curvature 0 for $p > 2$, finite nonzero curvature for $p = 2$, and infinite curvature for $1 < p < 2$; the orders p are dense.

What has been shown about curved lines and the surfaces they enclose applies easily to curved surfaces and contents of solids. These lemmas were set down first to avoid the tedium of long proofs by reduction to absurdity in the manner of the ancient geometers. Proofs are shorter by the method of indivisibles; but the hypothesis of indivisibles is harsher, and that method is therefore reckoned less geometrical. So the proofs of what follows are reduced to the ultimate sums and ratios of vanishing quantities and the first ratios of nascent ones, that is, to the limits of sums and ratios, and the proofs of those limits are given first as briefly as possible. This achieves what the method of indivisibles achieves, with principles now proved. Hence in what follows, whenever quantities are considered as made of particles, or small curved lines are used for straight ones, what is meant is not indivisibles but vanishing divisible quantities, not sums and ratios of determinate parts but always limits of sums and ratios, and the force of such proofs always rests on the method of the preceding lemmas.

Objection: vanishing quantities have no ultimate proportion, since before they vanish it is not ultimate, and when they have vanished there is none. By the same argument a body arriving at a place would have no ultimate velocity: before it arrives the velocity is not ultimate, and when it has arrived there is none. The answer is easy. The ultimate velocity is that with which the body moves neither before it reaches the last place and the motion stops, nor after, but at the moment it arrives. Likewise the ultimate ratio of vanishing quantities is the ratio with which they vanish, neither before nor after; the first ratio of nascent quantities is the ratio with which they begin; and the first and last sum is that with which they begin and cease to be (or to increase and decrease). There is a limit which the velocity can reach at the end of the motion but not exceed; this is the ultimate velocity. The same holds for the limit of all beginning and ceasing quantities and proportions. Since this limit is certain and definite, finding it is a truly geometrical problem, and geometrical things may legitimately be used to determine and prove other geometrical things.

Objection: if the ultimate ratios of vanishing quantities are given, their ultimate magnitudes are given too, and so every quantity consists of indivisibles, contrary to what Euclid proved about incommensurables in Book X of the *Elements*. This objection rests on a false hypothesis. The ultimate ratios with which quantities vanish are not ratios of ultimate quantities, but limits which the ratios of quantities decreasing without limit always approach, nearer than by any given difference, never passing them and never reaching them before the quantities are diminished indefinitely. This is clearer with infinitely great quantities: if two quantities with a given difference are increased indefinitely, their ultimate ratio, equality, is given, though no ultimate or greatest quantities of which it is the ratio are given. So in what follows, whenever for ease of imagination quantities are called least, vanishing or ultimate, do not understand quantities of determinate magnitude, but think of them as always to be diminished without limit.

In modern terms: an ultimate ratio is $\lim X/Y$, and Lemma I supplies its defining property.

No theorem of the *Lean* reconstruction cites this item. Not encoded; its distinction between vanishing divisible quantities and indivisibles is the reading the rational construction follows.

Section II. On finding centripetal forces

Proposition I. Theorem I. *The areas which bodies made to move in orbits describe by radii drawn to an immovable centre of forces lie in immovable planes and are proportional to the times.*

In modern terms: for a centripetal force toward a fixed point S , the motion stays in one plane and $\frac{1}{2} \mathbf{r} \times \dot{\mathbf{r}}$ is constant, so swept area is proportional to time.

Proof. Divide the time into equal parts, and in the first part let the body describe the straight line AB by its inherent force. In the second part, if nothing impeded it, it would go straight on to c (Law I), describing Bc equal to AB , so that with radii AS , BS , cS drawn to the centre, the areas ASB and BSc would be equal. But when the body reaches B , let a centripetal force act with a single but great impulse and make the body deviate from the line Bc and go on in the line BC . Draw cC parallel to BS , meeting BC in C ; at the end of the second part of time the body (Corollary I of the Laws) will be at C , in the same plane as the triangle ASB . Join SC ; the triangle SBC equals the triangle SBC , because of the parallels SB , Cc , and so it also equals the triangle SAB . By a similar argument, if the centripetal force acts successively at C , D , E , ..., making the body describe in the single particles of time the single straight lines CD , DE , EF , ..., these lie in the same plane, and the triangle SCD equals the triangle SBC , SDE equals SCD , and SEF equals SDE . So in equal times equal areas are described in an immovable plane, and *componendo*, any sums of areas $SADS$, $SAFS$ are to each other as the times of description. Now let the number of triangles be increased and their width diminished indefinitely: their ultimate perimeter ADF (Corollary 4 of Lemma III) will be a curved line; so the centripetal force, by which the body is perpetually drawn back from the tangent of this curve, will act uninterruptedly, and any areas described, $SADS$, $SAFS$, being always proportional to the times of description, will be proportional to those times in this case too.

In modern terms: with vertices $\mathbf{r}_k$ and impulses toward S , $\mathbf{r}_{k+1} = 2\mathbf{r}_k - \mathbf{r}_{k-1} + \lambda_k \mathbf{r}_k$. Then $\mathbf{r}_k \times \mathbf{r}_{k+1} = \mathbf{r}_{k-1} \times \mathbf{r}_k$, so every triangle $S \mathbf{r}_k \mathbf{r}_{k+1}$ has the same area (Newton: triangles on the same base between parallels, Euclid I.37–38). The final step passes to the limit of the polygons.

Corollary 1. In non-resisting media, if the areas are not proportional to the times, the forces do not tend to the point where the radii meet.

Corollary 2. In all media, if the description of areas is accelerated, the forces do not tend to the point where the radii meet, but decline from it *in consequentia*.

1713. These two corollaries move, expanded, to Proposition II. In their place come six new ones.

Corollary 1. The velocity of a body attracted to an immovable centre is, in non-resisting spaces, inversely as the perpendicular let fall from the centre onto the rectilinear tangent of the orbit; for the velocities at A , B , C , D , E are as the bases AB , BC , CD , DE , EF of equal triangles, and these bases are

inversely as the perpendiculars on them. In modern terms: $v \cdot p$ is constant, with p the distance from S to the tangent.

Corollary 2. If the chords AB , BC of two arcs described successively in equal times by the same body in non-resisting spaces are completed into a parallelogram $ABCU$, its diagonal BU , in the position it has ultimately when the arcs are diminished indefinitely, passes through the centre of forces when produced both ways.

Corollary 3. If the chords AB , BC and DE , EF of arcs described in equal times are completed into parallelograms $ABCU$, $DEFZ$, the forces at B and E are in the ultimate ratio of the diagonals BU , EZ as the arcs are diminished indefinitely; for the motions BC , EF are compounded (Corollary I of the Laws) of Bc , BU and Ef , EZ , and BU , EZ , equal to Cc , Ff , were generated in the proof of this Proposition by the impulses of the centripetal force at B and E , and so are proportional to those impulses.

Corollary 4. The forces by which any bodies in non-resisting spaces are drawn from rectilinear motions and bent into curved orbits are to each other as the sagittas of arcs described in equal times that converge to the centre of forces and bisect the chords, as the arcs are diminished indefinitely; for these sagittas are halves of the diagonals of Corollary 3.

Corollary 5. So these forces are to the force of gravity as these sagittas are to the vertical sagittas of the parabolic arcs that projectiles describe in the same time.

Corollary 6. All the same holds, by Corollary V of the Laws, when the planes in which the bodies move, together with the centres of forces in them, are not at rest but move uniformly in a straight line.

Lean reconstruction: 435 theorems in 24 modules cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Polygon/Finite.lean, 16 theorems, 183 lines

```
import Std

namespace NewtonLimitDynamics.Polygon

/-- Synthetic construction interface. Its two identities are imported Euclidean
geometry: equal triangles on a common altitude (I.38) and same base between
parallels (I.37). They are not the polygon area-law conclusion. Centre S is
fixed in this interface; `area p q` measures twice the oriented area Spq.
`extend p q` continues pq by an equal segment; `kick q x j` translates x
parallel to Sq, by the central impulse j and the common time cell. -/
structure EuclideanConstruction (Point Impulse : Type) where
  area : Point → Point → Int
  extend : Point → Point → Point
  kick : Point → Point → Impulse → Point
  equal_base_altitude : ∀ p q, area q (extend p q) = area p q
  same_base_parallels : ∀ q x j, area q (kick q x j) = area q x

variable {Point Impulse : Type}

def step (g : EuclideanConstruction Point Impulse) (p q : Point) (j : Impulse) :=
  g.kick q (g.extend p q) j

theorem central_step_area (g : EuclideanConstruction Point Impulse)
  (p q : Point) (j : Impulse) : g.area q (step g p q j) = g.area p q := by
```

```

    rw [step, g.same_base_parallel, g.equal_base_altitude]

/-- Successive pairs of vertices, produced by the construction, not supplied
with equal-area proofs. Each step has the same positive time cell dt. -/
def motion (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) : Nat → Point × Point
| 0 => (p, q)
| n+1 => let old := motion g p q impulse n
        (old.2, step g old.1 old.2 (impulse n))

theorem all_cell_areas (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  g.area (motion g p q impulse n).1 (motion g p q impulse n).2 = g.area p q := by
  induction n with
  | zero => rfl
  | succ n ih =>
    simp only [motion]
    rw [central_step_area]
    exact ih

def isum (f : Nat → Int) : Nat → Int
| 0 => 0
| n+1 => isum f n + f n

theorem sum_constant (f : Nat → Int) (c : Int) (hf : ∀ i, f i = c) (n : Nat) :
  isum f n = (n : Int) * c := by
  induction n with
  | zero => simp [isum]
  | succ n ih => simp [isum, ih, hf, Int.add_mul]

def swept (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) : Int :=
  isum (fun i => g.area (motion g p q impulse i).1 (motion g p q impulse i).2) n

theorem swept_eq (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) : swept g p q impulse n = (n : Int) * g.area p q :=
  sum_constant _ _ (all_cell_areas g p q impulse) n

/-- Cross-multiplied area/time ratio. Positive common dt is stated; division by
total times additionally requires positive counts. No curve is produced. -/
theorem equal_time_area_reconstruction (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Int) (_hdt : 0 < dt) (m n : Nat) :
  swept g p q impulse m * ((n : Int) * dt) =
  swept g p q impulse n * ((m : Int) * dt) := by
  rw [swept_eq, swept_eq]
  ac_rfl

/-- Finite sum of natural-number magnitudes. -/
def nsum (f : Nat → Nat) : Nat → Nat
| 0 => 0
| n + 1 => nsum f n + f n

theorem nsum_constant (f : Nat → Nat) (c : Nat) (hf : ∀ i, f i = c) (n : Nat) :
  nsum f n = n * c := by
  induction n with
  | zero => simp [nsum]
  | succ n ih => simp [nsum, ih, hf, Nat.add_mul]

/-- Magnitude of the supplied oriented doubled triangle-area datum. The
interpretation as Euclidean area is part of the construction's supplied
semantics; the two preservation identities alone do not certify it. -/
def unsignedCellArea (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) : Nat :=
  (g.area (motion g p q impulse n).1 (motion g p q impulse n).2).natAbs

theorem all_unsigned_cell_areas (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  congrArg Int.natAbs (all_cell_areas g p q impulse n)

```

```

/-- Sum over a consecutive block of cells, starting at `start`. Revisited
triangles count again: this is not the area of their geometric union. -/
def unsignedBlock (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (start count : Nat) : Nat :=
  nsum (fun i => unsignedCellArea g p q impulse (start + i)) count

theorem unsigned_block_eq (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (start count : Nat) :
  unsignedBlock g p q impulse start count = count * (g.area p q).natAbs :=
  nsum_constant _ _ (fun i => all_unsigned_cell_areas g p q impulse (start + i)) count

/-- Algebraic cross multiplication, valid also for zero counts. A time-ratio
interpretation additionally requires a positive cell and positive counts. -/
theorem unsigned_block_time_cross (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (dt start1 start2 m n : Nat) :
  unsignedBlock g p q impulse start1 m * (n * dt) =
  unsignedBlock g p q impulse start2 n * (m * dt) := by
  rw [unsigned_block_eq, unsigned_block_eq]
  ac_rfl

/-- Positive total times and the finite unsigned-area/time comparison, with
geometric interpretation and force direction still supplied separately. -/
theorem positive_unsigned_area_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start1 start2 m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start1 m * (n * dt) =
  unsignedBlock g p q impulse start2 n * (m * dt) :=
  (Nat.mul_pos hm hdt, Nat.mul_pos hn hdt,
  unsigned_block_time_cross g p q impulse dt start1 start2 m n)

/-- A concrete consistency model in integer coordinates. The synthetic result
above is conditional on the named Euclidean identities, not on coordinates. -/
abbrev LatticePoint := Int × Int

def det (p q : LatticePoint) : Int := p.1 * q.2 - p.2 * q.1

def extend (p q : LatticePoint) : LatticePoint := (2*q.1-p.1, 2*q.2-p.2)
def kick (q x : LatticePoint) (j : Int) : LatticePoint := (x.1+j*q.1, x.2+j*q.2)

theorem extension_identity (p q : LatticePoint) : det q (extend p q) = det p q := by
  simp only [det, extend, Int.mul_sub, Int.mul_assoc]
  have h : q.1 * (2 * q.2) = q.2 * (2 * q.1) := by ac_rfl
  have h1 : q.1 * p.2 = p.2 * q.1 := Int.mul_comm _ _
  have h2 : q.2 * p.1 = p.1 * q.2 := Int.mul_comm _ _
  omega

theorem parallel_identity (q x : LatticePoint) (j : Int) : det q (kick q x j) = det q x := by
  simp only [det, kick, Int.mul_add]
  have h : q.1 * (j * q.2) = q.2 * (j * q.1) := by ac_rfl
  omega

def lattice : EuclideanConstruction LatticePoint Int :=
  (det, extend, kick, extension_identity, parallel_identity)

/-- Checked orientation control: signed and unsigned sums agree here. -/
theorem positive_orientation_unsigned_example :
  swept lattice (1, 0) (1, 1) (fun _ => 0) 3 = 3 ∧
  unsignedBlock lattice (1, 0) (1, 1) (fun _ => 0) 0 3 = 3 := by
  decide

/-- Reversing orientation preserves unsigned magnitude, not the signed sum. -/
theorem negative_orientation_unsigned_example :
  swept lattice (1, 0) (1, -1) (fun _ => 0) 3 = -3 ∧
  unsignedBlock lattice (1, 0) (1, -1) (fun _ => 0) 0 3 = 3 ∧
  swept lattice (1, 0) (1, -1) (fun _ => 0) 3 ≠
  (unsignedBlock lattice (1, 0) (1, -1) (fun _ => 0) 0 3 : Int) := by
  decide

/-- Radial degeneracy, rest, and empty blocks need no area division. -/

```



```

theorem degenerate_unsigned_examples :
  unsignedBlock lattice (1, 0) (2, 0) (fun _ => -1) 2 3 = 0 ∧
  unsignedBlock lattice (1, 0) (1, 0) (fun _ => 0) 2 3 = 0 ∧
  unsignedBlock lattice (1, 0) (1, 1) (fun _ => 0) 4 0 = 0 := by
  decide

/-- Even inward radial impulses can revisit triangles. The unsigned cell sum
counts repeated coverage and therefore cannot identify a sector union. -/
theorem repeated_triangle_coverage_example :
  motion lattice (1, 0) (0, 1) (fun _ => -2) 4 = ((1, 0), (0, 1)) ∧
  unsignedBlock lattice (1, 0) (0, 1) (fun _ => -2) 0 4 = 4 ∧
  unsignedBlock lattice (1, 0) (0, 1) (fun _ => -2) 0 8 = 8 := by
  decide

/-- Zero force is permitted; radial/zero-area configurations need no division. -/
example : swept lattice (1,0) (1,1) (fun _ => 0) 3 = 3 := by decide
example : swept lattice (1,0) (2,0) (fun _ => -1) 3 = 0 := by decide

end NewtonLimitDynamics.Polygon

NewtonLimitDynamics/DeMotu1684/AreaLaw.lean, 6 theorems, 86 lines

import NewtonLimitDynamics.Polygon.PathDefect

/-!
Finite reconstruction of De Motu **Theorem 1**, not a retrospectively numbered
Principia proposition. Witness NATP00089 par8-9 (De motu corporum in gyrum,
https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NATP00089#par9) and
witness NATP00090 par16-17 (De motu sphaericorum corporum in fluidis,
https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NATP00090#par17)
remain separate. Both explicitly construct equal-time central-impulse polygons
and say that equal areas are described in equal times. The arbitrary-block
comparison below is a derived finite reconstruction; these passages do not
contain the printed editions' explicit `componendo` sentence.

The shared kernel assumes two named Euclidean preservation identities and
supplied area semantics. It constructs finite vertices and sums unsigned
doubled triangle areas with multiplicity. It does not enforce inward impulses,
identify a sector union, or prove the final `numero infinita et infinitè parva`
passage to uninterrupted force. NATP00089's changing marginal hypothesis/lemma
labels are not silently resolved using NATP00090 or a later edition.

The principal remaining area is D_mesh BETWEEN the polygon and actual
trajectory over one common time interval, with endpoint connectors stated if
needed. It is not the radius-swept Kepler area in Theorem 1. The defect-control
theorems below name the supplied geometric enclosure and vanishing budget;
neither follows from the finite equal-area theorem and neither creates a curve.
-/

namespace DeMotu1684.AreaLaw

open NewtonLimitDynamics.Polygon
open NewtonLimitDynamics
open NewtonLimitDynamics.Polygon.PathDefect

variable {Point Impulse : Type}

/-- NATP00089 par9: finite equal areas, conditional on the named construction
premises. No limiting conclusion or reconstructed marginal citation. -/
theorem natp00089_finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- NATP00090 par17: separately named witness-local finite equal-area result. -/
theorem natp00090_finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
  all_unsigned_cell_areas g p q impulse n

/-- Derived block comparison from NATP00089's finite equal-area construction.

```

```

    Positive time cell and counts rule out division by zero total times. -/
theorem natp00089_finite_block_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
  positive_unsigned_area_comparison g p q impulse dt hdt start₁ start₂ m n hm hn

/-- Derived comparison for NATP00090 alone; it imports no printed-edition
    limiting lemma or premise from NATP00089's unresolved revision layer. -/
theorem natp00090_finite_block_comparison (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
  positive_unsigned_area_comparison g p q impulse dt hdt start₁ start₂ m n hm hn

/-- NATP00089's polygon-to-motion obligation, as a conditional reconstruction:
    D_mesh is the nonnegative polygon-trajectory region, NOT its Kepler area.
    No numbered limiting lemma is imported into this witness. -/
theorem natp00089_polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
  polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

/-- Separately named NATP00090 obligation with the same explicit geometric
    premises; its final infinitely-small-triangle assertion remains unproved. -/
theorem natp00090_polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
  polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

end DeMotu1684.AreaLaw

```

NewtonLimitDynamics/Principial1687/PropositionI.lean, 3 theorems, 53 lines

```

import NewtonLimitDynamics.Polygon.PathDefect

/-!
1687 Proposition I, finite equal-cell reconstruction only. Source:
NATP00077 par44–45,
https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NATP00077#par45.
The proof explicitly invokes Law I, the laws' Corollary 1, then composes the
equal triangle areas ('componendo'). The construction's two named Euclidean
preservation identities and geometric area semantics are supplied premises.

Unsigned doubled triangle sums count cells with multiplicity. Inward sense,
sector-union interpretation, and the final curve/uninterrupted-force passage
through Lemma III Corollary 4 remain separate. No 1713 premise is imported.

The main defect D_mesh is the nonnegative area BETWEEN polygon and actual
trajectory over the same interval, with endpoint connectors made explicit.
It is not the Kepler sector area in the proposition. The conditional control
below requires that region's enclosure; the finite area law does not supply it.
-/

```

```

namespace Principial1687.PropositionI

open NewtonLimitDynamics.Polygon
open NewtonLimitDynamics
open NewtonLimitDynamics.Polygon.PathDefect

variable {Point Impulse : Type}

theorem finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=

```

```

all_unsigned_cell_areas g p q impulse n

/-- Finite content of the same-edition `componendo` sentence; positive total
times are derived, not left implicit in a cross-multiplied identity. -/
theorem finite_componendo (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
positive_unsigned_area_comparison g p q impulse dt hdt start₁ start₂ m n hm hn

/-- Conditional control of the intervening polygon-trajectory region. The
curve, its relation to the construction, and the enclosure are supplied
geometric obligations; Lemma III Corollary 4 is not silently discharged. -/
theorem polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry

end Principial687.PropositionI

```

NewtonLimitDynamics/Principial713/PropositionI.lean, 3 theorems, 53 lines

```
import NewtonLimitDynamics.Polygon.PathDefect
```

```

/-!
1713 Proposition I, finite equal-cell reconstruction only. Source:
NATP00082 par50-51,
https://www.newtonproject.ox.ac.uk/view/texts/diplomatic/NATP00082#par51.
The proof invokes this edition's Law I and laws' Corollary 1 and explicitly
composes equal triangle areas. The same mathematical kernel is reusable, but
the historical source and theorem namespace remain 1713-local.

```

The two named Euclidean identities and area semantics are supplied. Sums of unsigned doubled triangle areas count repeated coverage. Inward dynamics, sector identification, Lemma III Corollary 4's curve passage, and uninterrupted force are not concluded. No 1687 mechanical or limiting premise is imported.

The main `D_mesh` is the nonnegative area BETWEEN polygon and actual trajectory over one common time interval, including stated endpoint connectors. It is separate from the Kepler sector area. The defect-control theorem names the supplied geometric enclosure; no sector-area identity proves that premise.

```
namespace Principial713.PropositionI
```

```

open NewtonLimitDynamics.Polygon
open NewtonLimitDynamics
open NewtonLimitDynamics.Polygon.PathDefect

```

```
variable {Point Impulse : Type}
```

```

theorem finite_equal_areas (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (n : Nat) :
  unsignedCellArea g p q impulse n = (g.area p q).natAbs :=
all_unsigned_cell_areas g p q impulse n

```

```

/-- This edition's finite `componendo` consequence, before its limiting clause. -/
theorem finite_componendo (g : EuclideanConstruction Point Impulse)
  (p q : Point) (impulse : Nat → Impulse) (dt : Nat) (hdt : 0 < dt)
  (start₁ start₂ m n : Nat) (hm : 0 < m) (hn : 0 < n) :
  0 < m * dt ∧ 0 < n * dt ∧
  unsignedBlock g p q impulse start₁ m * (n * dt) =
  unsignedBlock g p q impulse start₂ n * (m * dt) :=
positive_unsigned_area_comparison g p q impulse dt hdt start₁ start₂ m n hm hn

```

```

/-- This edition's conditional polygon-trajectory defect interface. A realized
trajectory and its geometric enclosure are premises, not conclusions of
the exact finite Kepler triangle law. -/

```

```

theorem polygon_trajectory_defect_control
  (polygonTrajectoryArea budget : Fraction → Fraction) (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea :=
  polygon_trajectory_defect_vanishes polygonTrajectoryArea budget hbudget hgeometry
end Principal1713.PropositionI

```

NewtonLimitDynamics/Polygon/Contact.lean, 21 theorems, 240 lines

```

import NewtonLimitDynamics.Polygon.Finite

namespace NewtonLimitDynamics.Polygon

/-- Data for one finite motion cell. `departure` and `arrival` are kept
    separate: sharing an endpoint in space does not itself say how motion
    passes through that endpoint. This is a modern diagnostic reconstruction
    of the finite polygon stage, not a continuous trajectory. -/
structure FiniteSegment (Point Velocity : Type) where
  first : Point
  last : Point
  departure : Velocity
  arrival : Velocity

variable {Point Velocity Impulse : Type}

/-- The two pieces meet at the same spatial point. -/
def positionContact (left right : FiniteSegment Point Velocity) : Prop :=
  left.last = right.first

/-- A join with no velocity jump. -/
def velocityContact (left right : FiniteSegment Point Velocity) : Prop :=
  positionContact left right ∧ left.arrival = right.departure

/-- A join whose velocity jump is accounted for by the displayed impulse. -/
def impulseContact (advance : Velocity → Impulse → Velocity)
  (left right : FiniteSegment Point Velocity) (j : Impulse) : Prop :=
  positionContact left right ∧ right.departure = advance left.arrival j

theorem velocityContact_position (left right : FiniteSegment Point Velocity)
  (h : velocityContact left right) : positionContact left right :=
  h.1

theorem impulseContact_position (advance : Velocity → Impulse → Velocity)
  (left right : FiniteSegment Point Velocity) (j : Impulse)
  (h : impulseContact advance left right j) : positionContact left right :=
  h.1

/-- Two finite cells with separately recorded incoming and outgoing data at
    their shared instant. -/
structure TwoCellPath (Point Velocity : Type) where
  firstPoint : Point
  middlePoint : Point
  lastPoint : Point
  initialDeparture : Velocity
  middleArrival : Velocity
  middleDeparture : Velocity
  finalArrival : Velocity

def TwoCellPath.left (path : TwoCellPath Point Velocity) : FiniteSegment Point Velocity :=
  (path.firstPoint, path.middlePoint, path.initialDeparture, path.middleArrival)

def TwoCellPath.right (path : TwoCellPath Point Velocity) : FiniteSegment Point Velocity :=
  (path.middlePoint, path.lastPoint, path.middleDeparture, path.finalArrival)

/-- Assemble two cells only after their spatial endpoints have been shown to
    agree. The construction retains, rather than erases, the possible velocity
    jump at the shared instant. -/
def glue (left right : FiniteSegment Point Velocity)
  (_h : positionContact left right) : TwoCellPath Point Velocity :=
  (left.first, right.first, right.last,

```

```

    left.departure, left.arrival, right.departure, right.arrival)

theorem glue_first (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).firstPoint = left.first :=
  rfl

theorem glue_last (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).lastPoint = right.last :=
  rfl

theorem glue_middle_from_left (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).middlePoint = left.last :=
  h.symm

theorem glue_middle_from_right (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) : (glue left right h).middlePoint = right.first :=
  rfl

theorem glued_position_contact (left right : FiniteSegment Point Velocity)
  (h : positionContact left right) :
    positionContact (glue left right h).left (glue left right h).right :=
  rfl

/-- A finite sample records point values at instants, but leaves arrival and
    departure data independent. It therefore represents no claim that a
    continuous trajectory exists. -/
structure SampledPath (Point Velocity : Type) where
  point : Nat → Point
  arriving : Nat → Velocity
  departing : Nat → Velocity

def SampledPath.cell (path : SampledPath Point Velocity) (i : Nat) :
  FiniteSegment Point Velocity :=
  (path.point i, path.point (i+1), path.departing i, path.arriving (i+1))

/-- Restriction is re-indexing of already supplied samples; it constructs no
    new point at an intermediate time. -/
def SampledPath.restrict (path : SampledPath Point Velocity) (offset : Nat) :
  SampledPath Point Velocity :=
  (fun i => path.point (offset+i), fun i => path.arriving (offset+i),
   fun i => path.departing (offset+i))

theorem restriction_cell (path : SampledPath Point Velocity) (offset i : Nat) :
  (path.restrict offset).cell i = path.cell (offset+i) := by
  simp [SampledPath.restrict, SampledPath.cell, Nat.add_assoc]

theorem adjacent_position_contact (path : SampledPath Point Velocity) (i : Nat) :
  positionContact (path.cell i) (path.cell (i+1)) :=
  rfl

theorem adjacent_velocityContact_iff (path : SampledPath Point Velocity) (i : Nat) :
  velocityContact (path.cell i) (path.cell (i+1)) ↔
  path.arriving (i+1) = path.departing (i+1) := by
  simp [velocityContact, positionContact, SampledPath.cell]

theorem adjacent_impulseContact_iff (advance : Velocity → Impulse → Velocity)
  (path : SampledPath Point Velocity) (i : Nat) (j : Impulse) :
  impulseContact advance (path.cell i) (path.cell (i+1)) j ↔
  path.departing (i+1) = advance (path.arriving (i+1)) j := by
  simp [impulseContact, positionContact, SampledPath.cell]

/-- Restarting the finite construction from its kth constructed pair, with the
    shifted impulse sequence, gives the original construction after k+n cells.
    This is a theorem about the existing recursive polygonal motion only. -/
theorem motion_restart (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (k n : Nat) :
  motion g (motion g p q impulse k).1 (motion g p q impulse k).2
  (fun i => impulse (k+i)) n = motion g p q impulse (k+n) := by
  induction n with
  | zero => simp [motion]

```

```

| succ n ih =>
  rw [show k + (n+1) = (k+n)+1 by omega]
  simp only [motion]
  rw [ih]

/-- Consecutive pairs created by `motion` share their middle vertex. -/
theorem motion_adjacent_pair_position_contact
  (g : EuclideanConstruction Point Impulse) (p q : Point)
  (impulse : Nat → Impulse) (n : Nat) :
  (motion g p q impulse n).2 = (motion g p q impulse (n+1)).1 :=
  rfl

/-- Discrete velocity for unit-time lattice cells. -/
def latticeVelocity (p q : LatticePoint) : LatticePoint :=
  (q.1-p.1, q.2-p.2)

def latticeAdd (p q : LatticePoint) : LatticePoint :=
  (p.1+q.1, p.2+q.2)

def latticeScale (j : Int) (q : LatticePoint) : LatticePoint :=
  (j*q.1, j*q.2)

/-- The actual lattice step changes discrete velocity by the radial impulse.
Unit time is built into the use of adjacent vertex differences. -/
theorem lattice_step_velocity_jump (p q : LatticePoint) (j : Int) :
  latticeVelocity q (step lattice p q j) =
    latticeAdd (latticeVelocity p q) (latticeScale j q) := by
  apply Prod.ext <;>
  simp [latticeVelocity, latticeAdd, latticeScale, step, lattice, kick, extend] <;>
  omega

theorem motion_lattice_velocity_jump (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  latticeVelocity (motion lattice p q impulse n).2
    (motion lattice p q impulse (n+1)).2 =
    latticeAdd
      (latticeVelocity (motion lattice p q impulse n).1
        (motion lattice p q impulse n).2)
      (latticeScale (impulse n) (motion lattice p q impulse n).2) := by
  change latticeVelocity (motion lattice p q impulse n).2
    (step lattice (motion lattice p q impulse n).1
      (motion lattice p q impulse n).2 (impulse n)) = _
  exact lattice_step_velocity_jump _ _ _

/-- The nth actual lattice motion cell arrives and departs with its own
finite-difference velocity. The following cell may have a different one. -/
def latticeMotionCell (p q : LatticePoint) (impulse : Nat → Int) (n : Nat) :
  FiniteSegment LatticePoint LatticePoint :=
  let cell := motion lattice p q impulse n
  (cell.1, cell.2, latticeVelocity cell.1 cell.2, latticeVelocity cell.1 cell.2)

theorem latticeMotionCell_adjacent_position_contact (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  positionContact (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1)) :=
  motion_adjacent_pair_position_contact lattice p q impulse n

theorem latticeMotionCell_impulseContact (p q : LatticePoint)
  (impulse : Nat → Int) (n : Nat) :
  impulseContact latticeAdd (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1))
    (latticeScale (impulse n) (motion lattice p q impulse n).2) := by
  constructor
  · exact latticeMotionCell_adjacent_position_contact p q impulse n
  · change latticeVelocity (motion lattice p q impulse (n+1)).1
    (motion lattice p q impulse (n+1)).2 = _
    rw [← motion_adjacent_pair_position_contact lattice p q impulse n]
    exact motion_lattice_velocity_jump p q impulse n

theorem latticeMotionCell_zero_impulse_velocityContact (p q : LatticePoint)

```

```

    (impulse : Nat → Int) (n : Nat) (hzero : impulse n = 0) :
    velocityContact (latticeMotionCell p q impulse n)
    (latticeMotionCell p q impulse (n+1)) := by
  constructor
  · exact latticeMotionCell_adjacent_position_contact p q impulse n
  · change latticeVelocity (motion lattice p q impulse n).1
    (motion lattice p q impulse n).2 =
      latticeVelocity (motion lattice p q impulse (n+1)).1
      (motion lattice p q impulse (n+1)).2
  rw [← motion_adjacent_pair_position_contact lattice p q impulse n]
  rw [motion_lattice_velocity_jump, hzero]
  simp [latticeAdd, latticeScale]

/-- The two displayed inward central-impulse (discrete-force) histories start
with the same vertices. Every polygonal swept-area sum agrees, although
their first constructed next vertex differs. Thus equal areas do not
identify a polygonal path. The histories are different, so this is not
nonuniqueness for a fixed specified force law. -/
def inwardOneRadialImpulse : Nat → Int := fun _ => -1
def inwardTwoRadialImpulse : Nat → Int := fun _ => -2

theorem inward_impulses_same_swept (n : Nat) :
  swept lattice (1, 0) (1, 1) inwardOneRadialImpulse n =
  swept lattice (1, 0) (1, 1) inwardTwoRadialImpulse n := by
  rw [swept_eq, swept_eq]

theorem inward_impulses_distinct_next_vertex :
  (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 ≠
  (motion lattice (1, 0) (1, 1) inwardTwoRadialImpulse 1).2 := by
  decide

theorem equal_swept_area_does_not_identify_next_vertex :
  (∀ n, swept lattice (1, 0) (1, 1) inwardOneRadialImpulse n =
  swept lattice (1, 0) (1, 1) inwardTwoRadialImpulse n) ∧
  (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 ≠
  (motion lattice (1, 0) (1, 1) inwardTwoRadialImpulse 1).2 :=
  (inward_impulses_same_swept, inward_impulses_distinct_next_vertex)

end NewtonLimitDynamics.Polygon

NewtonLimitDynamics/Polygon/RefinementStrip.lean, 8 theorems, 95 lines

import NewtonLimitDynamics.Polygon.Contact

namespace NewtonLimitDynamics.Polygon

/-- Coordinate difference used only for the finite triangle calculation. -/
def latticeSub (p q : LatticePoint) : LatticePoint :=
  (p.1-q.1, p.2-q.2)

/-- The signed doubled area between a coarse edge A-C and a one-cell refined
polygon A-B-C. It is a closed polygon difference, not a swept sector and
does not name an actual limiting curve. -/
def refinementStripTwice (a b c : LatticePoint) : Int :=
  det a b + det b c - det a c

/-- Nonnegative size of the finite signed strip. Taking an absolute value does
not turn cancellation into an enclosure estimate. -/
def refinementDefect (a b c : LatticePoint) : Nat :=
  (refinementStripTwice a b c).natAbs

/-- The closed polygon difference is exactly the doubled area of its triangle.
This is finite determinant algebra for Euclidean triangle decomposition;
it uses no integration, derivatives, or limiting curve theorem. -/
theorem refinementStripTwice_eq_triangle (a b c : LatticePoint) :
  refinementStripTwice a b c = det (latticeSub b a) (latticeSub c a) := by
  simp [refinementStripTwice, det, latticeSub, Int.sub_mul, Int.mul_sub]
  have hab₁ : a.1*b.2 = b.2*a.1 := Int.mul_comm _ _
  have hab₂ : a.2*b.1 = b.1*a.2 := Int.mul_comm _ _
  have haa : a.1*a.2 = a.2*a.1 := Int.mul_comm _ _
  omega

```

```

theorem det_translation (origin a b : LatticePoint) :
  det (latticeAdd origin a) (latticeAdd origin b) =
    det a b + det origin b - det origin a := by
  simp [det, latticeAdd, Int.add_mul, Int.mul_add]
  have hoo : origin.1*origin.2 = origin.2*origin.1 := Int.mul_comm _ _
  have hao : a.1*origin.2 = origin.2*a.1 := Int.mul_comm _ _
  have hao' : a.2*origin.1 = origin.1*a.2 := Int.mul_comm _ _
  omega

/-- Translating both finite polygons leaves their enclosed signed strip area
    unchanged. -/
theorem refinementStripTwice_translation (origin a b c : LatticePoint) :
  refinementStripTwice (latticeAdd origin a) (latticeAdd origin b)
    (latticeAdd origin c) = refinementStripTwice a b c := by
  simp only [refinementStripTwice, det_translation]
  omega

theorem refinementDefect_eq_zero_iff (a b c : LatticePoint) :
  refinementDefect a b c = 0 ↔ refinementStripTwice a b c = 0 := by
  simp [refinementDefect]

/-- Spatial compatibility for a single coarse cell and a two-cell fine polygon.
    Both sides are finite `motion` data. This condition is deliberately only
    endpoint matching; no common force or time-refinement law has been derived. -/
def oneCellMotionRefinementCompatible
  (coarseStart coarseEnd fineStart fineMiddle : LatticePoint)
  (coarseImpulse fineImpulse : Nat → Int) : Prop :=
  (motion lattice coarseStart coarseEnd coarseImpulse 0).1 =
    (motion lattice fineStart fineMiddle fineImpulse 0).1 ∧
  (motion lattice coarseStart coarseEnd coarseImpulse 0).2 =
    (motion lattice fineStart fineMiddle fineImpulse 1).2

theorem oneCellMotionRefinementCompatible_start
  (coarseStart coarseEnd fineStart fineMiddle : LatticePoint)
  (coarseImpulse fineImpulse : Nat → Int)
  (h : oneCellMotionRefinementCompatible coarseStart coarseEnd fineStart fineMiddle
    coarseImpulse fineImpulse) :
  (motion lattice coarseStart coarseEnd coarseImpulse 0).1 =
    (motion lattice fineStart fineMiddle fineImpulse 0).1 :=
  h.1

theorem oneCellMotionRefinementCompatible_end
  (coarseStart coarseEnd fineStart fineMiddle : LatticePoint)
  (coarseImpulse fineImpulse : Nat → Int)
  (h : oneCellMotionRefinementCompatible coarseStart coarseEnd fineStart fineMiddle
    coarseImpulse fineImpulse) :
  (motion lattice coarseStart coarseEnd coarseImpulse 0).2 =
    (motion lattice fineStart fineMiddle fineImpulse 1).2 :=
  h.2

/-- A nonzero local strip from an actual inward fine impulse and a spatially
    compatible coarse cell. This is not nonuniqueness for one fixed force law. -/
theorem inward_oneCell_refinement_compatible :
  oneCellMotionRefinementCompatible (1, 0) (0, 1) (1, 0) (1, 1)
    (fun _ => 0) inwardOneRadialImpulse := by
  constructor
  · rfl
  · decide

theorem inward_oneCell_refinement_defect :
  refinementDefect (1, 0) (1, 1)
    (motion lattice (1, 0) (1, 1) inwardOneRadialImpulse 1).2 = 1 := by
  decide

end NewtonLimitDynamics.Polygon

```

NewtonLimitDynamics/Polygon/PathDefect.lean, 6 theorems, 139 lines

```

import NewtonLimitDynamics.Polygon.RefinementStrip
import NewtonLimitDynamics.Polygon.Enclosure

```



```

/-!
Area between paths, separated from Kepler area. The finite construction below
compares coarse edges A_i A_(i+1) with fine pairs A_i B_i A_(i+1). Its signed
closed-boundary area is the difference of the two signed Kepler sums; its
nonnegative patch budget is the sum of absolute triangle areas and retains
lobes of opposite orientation. These are finite paired polygons, not an actual
trajectory. Common spatial endpoints of each patch are built into the data;
matching mechanical data and time intervals are further obligations.

For an actual trajectory gamma and polygon P_mesh over one common time
interval, the main target is the nonnegative area D_mesh between gamma and
P_mesh, including stated endpoint connectors when needed. A valid geometric
patch decomposition/enclosure must identify D_mesh with or bound it by a
nonnegative budget. `PolygonTrajectoryEnclosure` names that supplied premise
explicitly. It is not a definition of D_mesh as a Kepler-sector difference and
does not construct gamma. No integration or curve existence is imported.
-/

namespace NewtonLimitDynamics.Polygon.PathDefect

open NewtonLimitDynamics

def signedGap (coarse inserted : Nat → LatticePoint) (n : Nat) : Int :=
  isum (fun i => refinementStripTwice (coarse i) (inserted i) (coarse (i + 1))) n

def absolutePatchBudget (coarse inserted : Nat → LatticePoint) (n : Nat) : Nat :=
  nsum (fun i => refinementDefect (coarse i) (inserted i) (coarse (i + 1))) n

/-- Signed doubled area swept about S by the coarse polygon; S is the origin. -/
def coarseKeplerTwice (coarse : Nat → LatticePoint) (n : Nat) : Int :=
  isum (fun i => det (coarse i) (coarse (i + 1))) n

def fineKeplerTwice (coarse inserted : Nat → LatticePoint) (n : Nat) : Int :=
  isum (fun i => det (coarse i) (inserted i) + det (inserted i) (coarse (i + 1))) n

/-- Unsigned Kepler triangle sums, counted with multiplicity. -/
def coarseKeplerUnsigned (coarse : Nat → LatticePoint) (n : Nat) : Nat :=
  nsum (fun i => (det (coarse i) (coarse (i + 1))).natAbs) n

def fineKeplerUnsigned (coarse inserted : Nat → LatticePoint) (n : Nat) : Nat :=
  nsum (fun i => (det (coarse i) (inserted i)).natAbs +
    (det (inserted i) (coarse (i + 1))).natAbs) n

/-- Radial closures cancel in the signed difference. This identity says nothing
about the unsigned area of lobes between the two paths. -/
theorem signed_gap_eq_Kepler_difference (coarse inserted : Nat → LatticePoint) (n : Nat) :
  signedGap coarse inserted n = fineKeplerTwice coarse inserted n -
  coarseKeplerTwice coarse n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    simp only [signedGap, fineKeplerTwice, coarseKeplerTwice, isum,
      refinementStripTwice] at *
    omega

/-- The absolute patch budget dominates the absolute signed difference. The
reverse inequality need not hold because opposite lobes cancel. -/
theorem signed_gap_abs_le_budget (coarse inserted : Nat → LatticePoint) (n : Nat) :
  (signedGap coarse inserted n).natAbs ≤ absolutePatchBudget coarse inserted n := by
  induction n with
  | zero => exact Nat.le_refl _
  | succ n ih =>
    change (signedGap coarse inserted n +
      refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs ≤
      absolutePatchBudget coarse inserted n +
      (refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs
    exact Nat.le_trans (Int.natAbs_add_le _ _) (Nat.add_le_add_right ih _)

/-- A uniform local triangle bound controls the entire finite absolute budget. -/

```

```

theorem absolute_budget_le_count_mul (coarse inserted : Nat → LatticePoint) (n b : Nat)
  (h : ∀ i, i < n → refinementDefect (coarse i) (inserted i) (coarse (i + 1)) ≤ b) :
  absolutePatchBudget coarse inserted n ≤ n * b := by
  induction n with
  | zero => simp [absolutePatchBudget, nsum]
  | succ n ih =>
    have hold := ih (fun i hi => h i (by omega))
    have hlast := h n (by omega)
    change absolutePatchBudget coarse inserted n +
      refinementDefect (coarse n) (inserted n) (coarse (n + 1)) ≤ (n + 1) * b
    rw [Nat.add_mul, Nat.one_mul]
    exact Nat.add_le_add hold hlast

/-- Moving the origin does not change the area budget between these paths. -/
theorem absolute_budget_translation (origin : LatticePoint)
  (coarse inserted : Nat → LatticePoint) (n : Nat) :
  absolutePatchBudget (fun i => latticeAdd origin (coarse i))
    (fun i => latticeAdd origin (inserted i)) n = absolutePatchBudget coarse inserted n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    change absolutePatchBudget (fun i => latticeAdd origin (coarse i))
      (fun i => latticeAdd origin (inserted i)) n +
        (refinementStripTwice (latticeAdd origin (coarse n))
          (latticeAdd origin (inserted n)) (latticeAdd origin (coarse (n + 1))))
      .natAbs =
      absolutePatchBudget coarse inserted n +
        (refinementStripTwice (coarse n) (inserted n) (coarse (n + 1))).natAbs
    rw [ih, refinementStripTwice_translation]

def exampleCoarse : Nat → LatticePoint := fun i => (i, 3)

def exampleInserted : Nat → LatticePoint := fun i =>
  if i = 0 then (1, 4) else (1, 2)

/-- Both paths have the same signed AND unsigned Kepler sums, but a positive
  area budget between them. Two adjacent, opposite-side triangle lobes have
  doubled unsigned area 1 each; their signed contributions cancel. -/
theorem equal_Kepler_areas_positive_path_defect :
  coarseKeplerTwice exampleCoarse 2 = -6 ∧
  fineKeplerTwice exampleCoarse exampleInserted 2 = -6 ∧
  coarseKeplerUnsigned exampleCoarse 2 = 6 ∧
  fineKeplerUnsigned exampleCoarse exampleInserted 2 = 6 ∧
  signedGap exampleCoarse exampleInserted 2 = 0 ∧
  absolutePatchBudget exampleCoarse exampleInserted 2 = 2 := by
  decide

/-- Explicit geometric premise about the nonnegative area BETWEEN an actual
  trajectory and an impulse polygon over the same interval. The caller must
  supply/construct the trajectory and justify this enclosure. A Kepler area
  or a signed sector-area difference is not an admissible substitution. -/
def PolygonTrajectoryEnclosure (polygonTrajectoryArea budget : Fraction → Fraction) : Prop :=
  Near Fraction.magnitudes (fun mesh =>
    Fraction.le (Fraction.ofInt 0) (polygonTrajectoryArea mesh) ∧
    Fraction.le (polygonTrajectoryArea mesh) (budget mesh))

/-- Conditional defect control: a vanishing geometric budget makes the actual
  polygon-trajectory area small. No curve existence or geometric enclosure
  is inferred from the finite Kepler area law. -/
theorem polygon_trajectory_defect_vanishes (polygonTrajectoryArea budget : Fraction → Fraction)
  (hbudget : Vanishes budget)
  (hgeometry : PolygonTrajectoryEnclosure polygonTrajectoryArea budget) :
  Vanishes polygonTrajectoryArea := by
  apply enclosed_gap_vanishes polygonTrajectoryArea budget hbudget
  obtain ⟨d, hd, h⟩ := hgeometry
  exact ⟨d, hd, fun mesh hm hmd => (h mesh hm hmd).2⟩

end NewtonLimitDynamics.Polygon.PathDefect

```

NewtonLimitDynamics/Polygon/TimeSubdivision.lean, 14 theorems, 139 lines

import BarrowLib.Polygon.PointAlgebra

```

namespace NewtonLimitDynamics.Polygon.TimeSubdivision

open NewtonLimitDynamics

def positiveDuration (d : Fraction) : Prop := Fraction.positive d

/-- One finite end-kick cell: drift using the incoming velocity, then change
    velocity by the constant accelerative force times the cell duration. -/
def endKick (d : Fraction) (state : Point × Point) (a : Point) : Point × Point :=
  (pointAdd state.1 (pointScale d state.2), pointAdd state.2 (pointScale d a))

def coarse (h k : Fraction) (p v a : Point) : Point × Point :=
  endKick (Fraction.add h k) (p, v) a

def fine (h k : Fraction) (p v a : Point) : Point × Point :=
  endKick k (endKick h (p, v) a) a

def totalDuration (h k : Fraction) : Fraction := Fraction.add h k

theorem totalDuration_positive (h k : Fraction)
  (hh : positiveDuration h) (hk : positiveDuration k) :
  positiveDuration (totalDuration h k) := by
  unfold positiveDuration totalDuration Fraction.positive Fraction.add
  dsimp
  exact Int.add_pos (Int.mul_pos hh k.den_pos) (Int.mul_pos hk h.den_pos)

private theorem fine_position_scalar (h k p v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul h v))
      (Fraction.mul k (Fraction.add v (Fraction.mul h a))))
    (Fraction.add (Fraction.add p (Fraction.mul (Fraction.add h k) v))
      (Fraction.mul (Fraction.mul h k) a)) := by
  have hdist : Fraction.equiv (Fraction.mul k (Fraction.add v (Fraction.mul h a)))
    (Fraction.add (Fraction.mul k v) (Fraction.mul k (Fraction.mul h a))) :=
    Fraction.mul_add k v (Fraction.mul h a)
  have hkv : Fraction.equiv (Fraction.add (Fraction.mul h v) (Fraction.mul k v))
    (Fraction.mul (Fraction.add h k) v) := by
    exact Fraction.equiv_trans (Fraction.add_equiv (Fraction.mul_comm h v) (Fraction.mul_comm k v))
    (Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_add v h k))
      (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add h k) v)))
  have hka : Fraction.equiv (Fraction.mul k (Fraction.mul h a))
    (Fraction.mul (Fraction.mul h k) a) := by
    exact Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_assoc k h a))
    (Fraction.mul_equiv_right a (Fraction.mul_comm k h))
  exact Fraction.equiv_trans
    (Fraction.equiv_trans (Fraction.add_assoc p (Fraction.mul h v)
      (Fraction.mul k (Fraction.add v (Fraction.mul h a))))
      (Fraction.add_equiv_left p (Fraction.add_equiv (Fraction.equiv_refl (Fraction.mul h v)) hdist)))
    (Fraction.equiv_trans
      (Fraction.add_equiv_left p (Fraction.equiv_symm (Fraction.add_assoc (Fraction.mul h v)
        (Fraction.mul k v) (Fraction.mul k (Fraction.mul h a)))))
      (Fraction.equiv_trans (Fraction.add_equiv_left p (Fraction.add_equiv hkv hka))
        (Fraction.equiv_symm (Fraction.add_assoc p (Fraction.mul (Fraction.add h k) v)
          (Fraction.mul (Fraction.mul h k) a)))))

/-- For the stated end-kick scheduling convention, time subdivision changes
    the final position by `h*k*a`. This is finite Fraction arithmetic, not a
    trajectory-existence theorem or a Newton central-force theorem. -/
theorem fine_position_eq_coarse_plus (h k : Fraction) (p v a : Point) :
  pointEquiv (fine h k p v a).1
    (pointAdd (coarse h k p v a).1 (pointScale (Fraction.mul h k) a)) := by
  constructor <=> apply fine_position_scalar

private theorem fine_velocity_scalar (h k v a : Fraction) :
  Fraction.equiv (Fraction.add (Fraction.add v (Fraction.mul h a)) (Fraction.mul k a))
    (Fraction.add v (Fraction.mul (Fraction.add h k) a)) := by
  exact Fraction.equiv_trans (Fraction.add_assoc v (Fraction.mul h a) (Fraction.mul k a))
    (Fraction.equiv_trans
      (Fraction.add_equiv_left v (Fraction.add_equiv (Fraction.mul_comm h a) (Fraction.mul_comm k a))))

```

```

(Fraction.equiv_trans
  (Fraction.add_equiv_left v (Fraction.equiv_symm (Fraction.mul_add a h k)))
  (Fraction.add_equiv_left v (Fraction.equiv_symm (Fraction.mul_comm (Fraction.add h k) a))))))

theorem fine_velocity_eq_coarse (h k : Fraction) (p v a : Point) :
  pointEquiv (fine h k p v a).2 (coarse h k p v a).2 := by
  constructor <=>
  dsimp [fine, coarse, endKick, pointAdd, pointScale] <=>
  apply fine_velocity_scalar

def half : Fraction := (1, 2, by decide)
def zero : Fraction := Fraction.ofInt 0
def one : Fraction := Fraction.ofInt 1
def quarter : Fraction := (1, 4, by decide)
def negQuarter : Fraction := (-1, 4, by decide)

def exampleP : Point := (zero, zero)
def exampleV : Point := (one, zero)
def exampleA : Point := (zero, one)
def exampleB : Point := (half, zero)
def exampleC : Point := (one, zero)
def exampleD : Point := (one, quarter)

theorem example_half_positive : positiveDuration half := by
  unfold positiveDuration Fraction.positive half
  decide

theorem example_total_positive : positiveDuration (totalDuration half half) :=
  totalDuration_positive half half example_half_positive example_half_positive

theorem example_coarse_position : pointEquiv (coarse half half exampleP exampleV exampleA).1 exampleC :=
  ↪ by
  decide

theorem example_fine_middle : pointEquiv (endKick half (exampleP, exampleV) exampleA).1 exampleB := by
  decide

theorem example_fine_position : pointEquiv (fine half half exampleP exampleV exampleA).1 exampleD := by
  decide

theorem example_endpoints_differ : ¬ pointEquiv exampleD exampleC := by
  intro h
  have := h.2
  change (1 : Int) * 1 = 0 * 4 at this
  omega

theorem example_fine_coarse_endpoints_differ :
  ¬ pointEquiv (fine half half exampleP exampleV exampleA).1
  (coarse half half exampleP exampleV exampleA).1 := by
  intro h
  exact example_endpoints_differ
  (Fraction.equiv_trans (Fraction.equiv_symm example_fine_position.1)
    (Fraction.equiv_trans h.1 example_coarse_position.1),
    Fraction.equiv_trans (Fraction.equiv_symm example_fine_position.2)
    (Fraction.equiv_trans h.2 example_coarse_position.2))

/-- The explicitly directed closing connector from the fine endpoint D to the
    coarse endpoint C. It is bookkeeping for a closed finite boundary. -/
def directedConnector (d c : Point) : Point := pointSub c d

theorem example_connector : pointEquiv (directedConnector exampleD exampleC) (zero, negQuarter) := by
  decide

theorem example_closedBoundaryTwice :
  Fraction.equiv (closedBoundaryTwice exampleP exampleB exampleD exampleC) ((-1, 8, by decide)) := by
  decide

end NewtonLimitDynamics.Polygon.TimeSubdivision

```

NewtonLimitDynamics/Polygon/PartitionControl.lean, 20 theorems, 252 lines

```

import NewtonLimitDynamics.Polygon.TimeSubdivision

namespace NewtonLimitDynamics.Polygon.PartitionControl

open NewtonLimitDynamics
open TimeSubdivision

/-- Integer bookkeeping for a finite common-denominator schedule. `A` is the
    ordered pair sum accumulated before each new weight is inserted. -/
structure PartitionStats where
  T : Nat
  A : Nat
  Q : Nat

def initial : PartitionStats := {0, 0, 0}

/-- The recurrences `T' = T + w`, `A' = A + w*T`, and `Q' = Q + w*w`. -/
def next (s : PartitionStats) (w : Nat) : PartitionStats :=
  {s.T + w, s.A + w * s.T, s.Q + w * w}

def stats (weights : List Nat) : PartitionStats :=
  weights.foldl next initial

def total (weights : List Nat) : Nat := (stats weights).T
def cross (weights : List Nat) : Nat := (stats weights).A
def squares (weights : List Nat) : Nat := (stats weights).Q

theorem next_identity (s : PartitionStats)
  (h : 2 * s.A + s.Q = s.T * s.T) (w : Nat) :
  2 * (next s w).A + (next s w).Q = (next s w).T * (next s w).T := by
  simp only [next]
  simp only [Nat.mul_add, Nat.add_mul]
  have htw : s.T * w = w * s.T := Nat.mul_comm _ _
  omega

theorem stats_identity_from (s : PartitionStats)
  (h : 2 * s.A + s.Q = s.T * s.T) : (weights : List Nat) ->
  2 * (weights.foldl next s).A + (weights.foldl next s).Q =
  (weights.foldl next s).T * (weights.foldl next s).T
| [] => h
| w :: ws => by
  simp only [List.foldl]
  exact stats_identity_from (next s w) (next_identity s h w) ws

/-- The ordered-pair and square decomposition for every finite schedule. -/
theorem stats_identity (weights : List Nat) :
  2 * cross weights + squares weights = total weights * total weights := by
  exact stats_identity_from initial (by decide) weights

theorem next_bound (M : Nat) (s : PartitionStats) (h : s.Q ≤ M * s.T)
  (w : Nat) (hw : w ≤ M) : (next s w).Q ≤ M * (next s w).T := by
  have hww : w * w ≤ M * w := Nat.mul_le_mul_right w hw
  simp only [next, Nat.mul_add]
  omega

theorem stats_bound_from (M : Nat) (s : PartitionStats) (h : s.Q ≤ M * s.T)
  (weights : List Nat) (hw : ∀ w ∈ weights, w ≤ M) :
  (weights.foldl next s).Q ≤ M * (weights.foldl next s).T := by
  induction weights generalizing s with
  | nil => exact h
  | cons w ws ih =>
    simp only [List.mem_cons] at hw
    simp only [List.foldl]
    exact ih (next s w) (next_bound M s h w (hw w (Or.inl rfl)))
      (fun x hx => hw x (Or.inr hx))

/-- A max-cell estimate for the finite square coefficient. -/
theorem stats_bound (M : Nat) (weights : List Nat) (hw : ∀ w ∈ weights, w ≤ M) :
  squares weights ≤ M * total weights := by

```

```

    exact stats_bound_from M initial (by change  $0 \leq M * 0$ ; omega) weights hw

/-- Zero weights are admitted by the recurrence; strict positive mesh cells are
    an additional hypothesis when a nondegenerate partition is required. -/
def positiveWeights (weights : List Nat) : Prop :=  $\forall w \in \text{weights}, 0 < w$ 

private theorem natDen_pos (D : Nat) (hD :  $0 < D$ ) :  $(0 : \text{Int}) < D$  := by omega

/-- The common-denominator duration `w/D`. -/
def duration (D w : Nat) (hD :  $0 < D$ ) : Fraction :=  $\langle w, D, \text{natDen\_pos } D \text{ hD} \rangle$ 

def squareDuration (D w : Nat) (hD :  $0 < D$ ) : Fraction :=  $\langle w, D * D, \text{by exact Int.mul\_pos (natDen\_pos } D \text{ hD) (natDen\_pos } D \text{ hD)} \rangle$ 

theorem duration_positive (D w : Nat) (hD :  $0 < D$ ) (hw :  $0 < w$ ) :
    Fraction.positive (duration D w hD) := by
    change  $0 < (w : \text{Int})$ 
    omega

private theorem scalar_step_position (D A T w : Nat) (hD :  $0 < D$ ) (p v a : Fraction) :
    Fraction.equiv
    (Fraction.add
    (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
    (Fraction.mul (squareDuration D A hD) a))
    (Fraction.mul (duration D w hD)
    (Fraction.add v (Fraction.mul (duration D T hD) a))))
    (Fraction.add
    (Fraction.add p (Fraction.mul (duration D (T + w) hD) v))
    (Fraction.mul (squareDuration D (A + w * T) hD) a)) := by
    unfold Fraction.equiv Fraction.add Fraction.mul duration squareDuration
    dsimp
    simp only [Int.ofNat_add, Int.ofNat_mul, Int.add_mul, Int.mul_add]
    ac_rfl

private theorem scalar_step_velocity (D T w : Nat) (hD :  $0 < D$ ) (v a : Fraction) :
    Fraction.equiv
    (Fraction.add (Fraction.add v (Fraction.mul (duration D T hD) a))
    (Fraction.mul (duration D w hD) a))
    (Fraction.add v (Fraction.mul (duration D (T + w) hD) a)) := by
    unfold Fraction.equiv Fraction.add Fraction.mul duration
    dsimp
    simp only [Int.ofNat_add, Int.add_mul]
    ac_rfl

private theorem endKick_congr (d : Fraction) {x y : Point × Point} (a : Point)
    (h : pointEquiv x.1 y.1  $\wedge$  pointEquiv x.2 y.2) :
    pointEquiv (endKick d x a).1 (endKick d y a).1  $\wedge$ 
    pointEquiv (endKick d x a).2 (endKick d y a).2 := by
    constructor
    · exact pointAdd_congr h.1 (pointScale_congr d h.2)
    · exact pointAdd_congr h.2 (Fraction.equiv_refl _, Fraction.equiv_refl _)

def encodedPosition (D : Nat) (hD :  $0 < D$ ) (s : PartitionStats) (p v a : Point) : Point :=
    pointAdd (pointAdd p (pointScale (duration D s.T hD) v))
    (pointScale (squareDuration D s.A hD) a)

def encodedVelocity (D : Nat) (hD :  $0 < D$ ) (s : PartitionStats) (v a : Point) : Point :=
    pointAdd v (pointScale (duration D s.T hD) a)

def encodedState (D : Nat) (hD :  $0 < D$ ) (s : PartitionStats) (p v a : Point) : Point × Point :=
    (encodedPosition D hD s p v a, encodedVelocity D hD s v a)

/-- The actual finite polygonal schedule, constructed only by repeated
    `TimeSubdivision.endKick` cells of duration `w/D`. -/
def partitionMotion (D : Nat) (hD :  $0 < D$ ) (p v a : Point) (weights : List Nat) : Point × Point :=
    weights.foldl (fun state w => endKick (duration D w hD) state a) (p, v)

private theorem encoded_step (D : Nat) (hD :  $0 < D$ ) (s : PartitionStats) (w : Nat)
    (p v a : Point) :
    pointEquiv (endKick (duration D w hD) (encodedState D hD s p v a) a).1

```

```

    (encodedState D hD (next s w) p v a).1 ∧
    pointEquiv (endKick (duration D w hD) (encodedState D hD s p v a) a).2
    (encodedState D hD (next s w) p v a).2 := by
  constructor <=> constructor
  · exact scalar_step_position D s.A s.T w hD p.1 v.1 a.1
  · exact scalar_step_position D s.A s.T w hD p.2 v.2 a.2
  · exact scalar_step_velocity D s.T w hD v.1 a.1
  · exact scalar_step_velocity D s.T w hD v.2 a.2

private theorem fold_encoded (D : Nat) (hD : 0 < D) (s : PartitionStats)
  (state : Point × Point) (p v a : Point)
  (hstate : pointEquiv state.1 (encodedState D hD s p v a).1 ∧
    pointEquiv state.2 (encodedState D hD s p v a).2) : (weights : List Nat) ->
  pointEquiv (weights.foldl (fun x w => endKick (duration D w hD) x a) state).1
    (encodedState D hD (weights.foldl next s) p v a).1 ∧
  pointEquiv (weights.foldl (fun x w => endKick (duration D w hD) x a) state).2
    (encodedState D hD (weights.foldl next s) p v a).2
| [] => hstate
| w :: ws => by
  simp only [List.foldl]
  have hkick := endKick_congr (duration D w hD) a hstate
  have hencoded := encoded_step D hD s w p v a
  exact fold_encoded D hD (next s w) _ p v a
    (pointEquiv_trans hkick.1 hencoded.1, pointEquiv_trans hkick.2 hencoded.2) ws

private theorem initial_encoded (D : Nat) (hD : 0 < D) (p v a : Point) :
  pointEquiv p (encodedState D hD initial p v a).1 ∧
  pointEquiv v (encodedState D hD initial p v a).2 := by
  constructor <=> constructor <=>
  unfold encodedState encodedPosition encodedVelocity initial pointAdd pointScale duration
  <- squareDuration
  Fraction.equiv Fraction.add Fraction.mul <=> dsimp <=> simp <=> ac_rfl

/-- The finite end-kick schedule has velocity `v + (T/D)a` and position
`p + (T/D)v + (A/D^2)a`, componentwise up to rational representation. -/
theorem partitionMotion_formula (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (partitionMotion D hD p v a weights).1
    (encodedPosition D hD (stats weights) p v a) ∧
  pointEquiv (partitionMotion D hD p v a weights).2
    (encodedVelocity D hD (stats weights) v a) := by
  exact fold_encoded D hD initial (p, v) p v a (initial_encoded D hD p v a) weights

/-- A finite rational comparison map; this is a formula, not an assumed
limiting trajectory. -/
def candidate (p v a : Point) (t : Fraction) : Point :=
  pointAdd (pointAdd p (pointScale t v)) (pointScale (Fraction.half (Fraction.mul t t)) a)

def residualCoefficient (D Q : Nat) (hD : 0 < D) : Fraction :=
  Fraction.half (squareDuration D Q hD)

def meshCoefficient (D M T : Nat) (hD : 0 < D) : Fraction :=
  Fraction.half (squareDuration D (M * T) hD)

private theorem candidate_residual_scalar (D A Q T : Nat) (hD : 0 < D)
  (h : 2 * A + Q = T * T) (p v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
      (Fraction.mul (Fraction.half (Fraction.mul (duration D T hD) (duration D T hD))) a))
    (Fraction.add
      (Fraction.add (Fraction.add p (Fraction.mul (duration D T hD) v))
        (Fraction.mul (squareDuration D A hD) a))
      (Fraction.mul (residualCoefficient D Q hD) a)) := by
  have hi : (2 : Int) * (A : Int) + (Q : Int) = (T : Int) * (T : Int) := by omega
  unfold Fraction.equiv Fraction.add Fraction.mul duration squareDuration residualCoefficient
  simp only [Fraction.half, squareDuration]
  simp only [Int.ofNat_mul, Int.add_mul, Int.mul_add]
  rw [← hi]
  simp only [Int.add_mul, Int.mul_add]
  ac_rfl

```

```

/-- At the finite terminal time, the candidate differs from the constructed
    polygon position by exactly `Q/(2D^2)` times the common acceleration. -/
theorem candidate_residual (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (candidate p v a (duration D (total weights) hD))
    (pointAdd (encodedPosition D hD (stats weights) p v a)
      (pointScale (residualCoefficient D (squares weights) hD) a)) := by
  constructor
  · exact candidate_residual_scalar D (cross weights) (squares weights) (total weights) hD
    (stats_identity weights) p.1 v.1 a.1
  · exact candidate_residual_scalar D (cross weights) (squares weights) (total weights) hD
    (stats_identity weights) p.2 v.2 a.2

/-- The terminal residual stated against the actual recursively constructed
    polygon endpoint. -/
theorem candidate_partitionMotion_residual (D : Nat) (hD : 0 < D)
  (p v a : Point) (weights : List Nat) :
  pointEquiv (candidate p v a (duration D (total weights) hD))
    (pointAdd (partitionMotion D hD p v a weights).1
      (pointScale (residualCoefficient D (squares weights) hD) a)) := by
  have hcandidate := candidate_residual D hD p v a weights
  have hmotion := partitionMotion_formula D hD p v a weights
  exact pointEquiv_trans hcandidate
    (pointAdd_congr (pointEquiv_symm hmotion.1)
      (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem residual_nonnegative (D Q : Nat) (hD : 0 < D) :
  Fraction.le (Fraction.ofInt 0) (residualCoefficient D Q hD) := by
  unfold Fraction.le Fraction.ofInt residualCoefficient Fraction.half squareDuration
  dsimp
  omega

theorem residual_mesh_bound (D M : Nat) (hD : 0 < D) (weights : List Nat)
  (hw : ∀ w ∈ weights, w ≤ M) :
  Fraction.le (residualCoefficient D (squares weights) hD)
    (meshCoefficient D M (total weights) hD) := by
  have hq := stats_bound M weights hw
  have hi : (squares weights : Int) ≤ (M * total weights : Int) := by omega
  unfold Fraction.le residualCoefficient meshCoefficient Fraction.half squareDuration
  dsimp
  have hp : 0 < (2 : Int) * ((D : Int) * (D : Int)) :=
    Int.mul_pos (by decide) (Int.mul_pos (natDen_pos D hD) (natDen_pos D hD))
  apply Int.mul_le_mul_of_nonneg_right hi (Int.le_of_lt hp)

end NewtonLimitDynamics.Polygon.PartitionControl

```

NewtonLimitDynamics/Polygon/PartialCell.lean, 13 theorems, 134 lines

```

import NewtonLimitDynamics.Polygon.ZeroForce

namespace NewtonLimitDynamics.Polygon.PartialCell

open NewtonLimitDynamics
open TimeSubdivision
open PartitionControl
open ZeroForce

/-- The actual state after a finite prefix schedule followed by one partial
    cell of duration `u/D`. It is the end-kick recurrence applied to the
    actual prefix state; no formula for the motion is assumed. -/
def partialState (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) (u : Nat) :
  Point × Point :=
  endKick (duration D u hD) (partitionMotion D hD p v a weights) a

/-- The drift position reached `u/D` into the next cell: the actual prefix
    position moved by the actual prefix velocity. -/
def actualPartialPosition (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) : Point :=
  pointAdd (partitionMotion D hD p v a weights).1
    (pointScale (duration D u hD) (partitionMotion D hD p v a weights).2)

/-- The partial position is the position component of the end-kick step. -/

```



```

theorem partialState_position (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  (partialState D hD p v a weights u).1 = actualPartialPosition D hD p v a weights u := rfl

/-- The terminal kick does not alter the drift position: the position component
  of an end-kick step is independent of the accelerative force applied at its end. -/
theorem endKick_position_kick_free (d : Fraction) (state : Point × Point) (a b : Point) :
  (endKick d state a).1 = (endKick d state b).1 := rfl

/-- The partial state is literally the recurrence on the appended schedule. -/
theorem partialState_append (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  partialState D hD p v a weights u = partitionMotion D hD p v a (weights ++ [u]) := by
  unfold partialState partitionMotion
  rw [List.foldl_append]
  rfl

/-- Appending one weight applies the statistics recurrence once. -/
theorem stats_append (weights : List Nat) (u : Nat) :
  stats (weights ++ [u]) = next (stats weights) u := by
  unfold stats
  rw [List.foldl_append]
  rfl

theorem total_append (weights : List Nat) (u : Nat) :
  total (weights ++ [u]) = total weights + u := by
  unfold total
  rw [stats_append]
  rfl

theorem squares_append (weights : List Nat) (u : Nat) :
  squares (weights ++ [u]) = squares weights + u * u := by
  unfold squares
  rw [stats_append]
  rfl

/-- Exact residual at the sample time `(T+u)/D` inside the next cell: the
  constructed candidate exceeds the actual partial position by
  `((Q+u*u)/(2D^2))*a`. Derived from the appended schedule's statistics. -/
theorem candidate_partial_residual (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (u : Nat) :
  pointEquiv (candidate p v a (duration D (total weights + u) hD))
    (pointAdd (actualPartialPosition D hD p v a weights u)
      (pointScale (residualCoefficient D (squares weights + u * u) hD) a)) := by
  have h := candidate_partitionMotion_residual D hD p v a (weights ++ [u])
  rw [total_append, squares_append, ← partialState_append, partialState_position] at h
  exact h

/-- Within-cell mesh bound on the square statistic: a partial duration inside a
  designated next cell of weight `w` (`0 ≤ u` is automatic in `Nat`). -/
theorem partial_squares_bound (M w u : Nat) (weights : List Nat)
  (hM : ∀ x ∈ weights, x ≤ M) (hw : w ≤ M) (hu : u ≤ w) :
  squares weights + u * u ≤ M * (total weights + u) := by
  have hall : ∀ x ∈ weights ++ [u], x ≤ M := by
    intro x hx
    rcases List.mem_append.mp hx with h | h
    · exact hM x h
    · rw [List.mem_singleton.mp h]
      exact Nat.le_trans hu hw
  have hb := stats_bound M (weights ++ [u]) hall
  rw [squares_append, total_append] at hb
  exact hb

/-- The corresponding Fraction coefficient bound:
  `(Q+u*u)/(2D^2) ≤ M*(T+u)/(2D^2)`. -/
theorem partial_residual_mesh_bound (D M w u : Nat) (hD : 0 < D) (weights : List Nat)
  (hM : ∀ x ∈ weights, x ≤ M) (hw : w ≤ M) (hu : u ≤ w) :
  Fraction.le (residualCoefficient D (squares weights + u * u) hD)
    (meshCoefficient D M (total weights + u) hD) := by
  have hq := partial_squares_bound M w u weights hM hw hu

```

```

have hi : ((squares weights + u * u : Nat) : Int) ≤ ((M * (total weights + u) : Nat) : Int) :=
  Int.ofNat_le.mpr hq
unfold Fraction.le residualCoefficient meshCoefficient Fraction.half squareDuration
dsimp
have hp : 0 < (2 : Int) * ((D : Int) * (D : Int)) :=
  Int.mul_pos (by decide) (Int.mul_pos (by omega) (by omega))
exact Int.mul_le_mul_of_nonneg_right hi (Int.le_of_lt hp)

/-- Boundary `u = w`: the partial position is the actual next vertex. -/
theorem partial_full_cell (D : Nat) (hD : 0 < D) (p v a : Point)
  (weights : List Nat) (w : Nat) :
  actualPartialPosition D hD p v a weights w =
    (partitionMotion D hD p v a (weights ++ [w])).1 := by
  rw [← partialState_append]
  rfl

private theorem zero_drift_scalar (D : Nat) (hD : 0 < D) (x y : Fraction) :
  Fraction.equiv (Fraction.add x (Fraction.mul (duration D 0 hD) y)) x := by
  unfold Fraction.equiv Fraction.add Fraction.mul duration
  dsimp
  simp only [Int.ofNat_zero, Int.zero_mul, Int.add_zero]
  ac_rfl

/-- Boundary `u = 0`: the partial position is the actual prefix vertex. -/
theorem partial_zero (D : Nat) (hD : 0 < D) (p v a : Point) (weights : List Nat) :
  pointEquiv (actualPartialPosition D hD p v a weights 0)
    (partitionMotion D hD p v a weights).1 :=
  (zero_drift_scalar D hD _, zero_drift_scalar D hD _)

/-- Boundary `a = 0`: the actual partial position lies on the inertial map at
  `(T+u)/D`. -/
theorem partial_zero_force (D : Nat) (hD : 0 < D) (p v : Point)
  (weights : List Nat) (u : Nat) :
  pointEquiv (actualPartialPosition D hD p v zeroPoint weights u)
    (inertialAt p v (duration D (total weights + u) hD)) := by
  have h := (partitionMotion_zero_force D hD p v (weights ++ [u])).1
  rw [total_append, ← partialState_append, partialState_position] at h
  exact h

end NewtonLimitDynamics.Polygon.PartialCell

```

NewtonLimitDynamics/Polygon/UniformRefinement.lean, 6 theorems, 98 lines

```

import NewtonLimitDynamics.Polygon.PartitionControl

namespace NewtonLimitDynamics.Polygon.UniformRefinement

open NewtonLimitDynamics
open TimeSubdivision
open PartitionControl

/-- `n` equal unit cells over a common denominator. -/
def unitCells (n : Nat) : List Nat := List.replicate n 1

private theorem unitCells_T_from (s : PartitionStats) : (n : Nat) ->
  ((unitCells n).foldl next s).T = s.T + n
| 0 => rfl
| n + 1 => by
  show ((unitCells n).foldl next (next s 1)).T = s.T + (n + 1)
  rw [unitCells_T_from (next s 1) n]
  simp only [next]
  omega

theorem unitCells_total (n : Nat) : total (unitCells n) = n := by
  unfold total stats
  rw [unitCells_T_from]
  exact Nat.zero_add n

theorem unitCells_le_one (n : Nat) : ∀ w ∈ unitCells n, w ≤ 1 := by
  intro w hw
  rw [List.eq_of_mem_replicate hw]

```

```

exact Nat.le_refl 1

/-- Refining the rational time `N/E` by a factor `K` keeps the same time. -/
theorem refined_time (N E K : Nat) (hE : 0 < E) (hEK : 0 < E * K) :
  Fraction.equiv (duration (E * K) (total (unitCells (N * K))) hEK) (duration E N hE) := by
  rw [unitCells_total]
  unfold Fraction.equiv duration
  dsimp
  simp only [Int.ofNat_mul]
  ac_refl

/-- Nat core of the estimate: `N*K*d ≤ n*2*(E*K)*(E*K)` once `K > N*d`,
  `n ≥ 1` and `E ≥ 1`. -/
private theorem nat_bound (N E n d : Nat) (hE : 0 < E) (hn : 0 < n) :
  N * (N * d + 1) * d ≤ n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
  have hK : N * d ≤ N * d + 1 := Nat.le_succ _
  have h1 : N * (N * d + 1) * d ≤ (N * d + 1) * (N * d + 1) := by
    calc N * (N * d + 1) * d = (N * d) * (N * d + 1) := by ac_refl
    _ ≤ (N * d + 1) * (N * d + 1) := Nat.mul_le_mul_right _ hK
  have hE1 : 1 ≤ E * E := Nat.mul_le_mul hE hE
  have h2 : (N * d + 1) * (N * d + 1) ≤ (E * E) * ((N * d + 1) * (N * d + 1)) := by
    calc (N * d + 1) * (N * d + 1) = 1 * ((N * d + 1) * (N * d + 1)) := (Nat.one_mul _).symm
    _ ≤ (E * E) * ((N * d + 1) * (N * d + 1)) := Nat.mul_le_mul_right _ hE1
  have h3 : (E * E) * ((N * d + 1) * (N * d + 1)) ≤
    n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
    calc (E * E) * ((N * d + 1) * (N * d + 1))
      = 1 * (1 * ((E * (N * d + 1)) * (E * (N * d + 1)))) := by
        simp only [Nat.one_mul]; ac_refl
    _ ≤ n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) :=
      Nat.mul_le_mul hn (Nat.mul_le_mul_right _ (by decide))
  exact Nat.le_trans h1 (Nat.le_trans h2 h3)

/-- For every positive rational tolerance and rational time `N/E`, an explicit
  uniform refinement (factor `K = N*den(ε)+1`, unit cells over `E*K`) makes
  the exact constant-force residual coefficient `Q/(2D²)` at most the
  tolerance. Together with `candidate_partitionMotion_residual`, the actual
  polygon position at `N/E` is within that coefficient (along `a`) of the
  constructed candidate. -/
theorem uniform_refinement_small (N E : Nat) (hE : 0 < E) (eps : Fraction)
  (heps : Fraction.positive eps) :
  ∃ K : Nat, ∃ hEK : 0 < E * K,
    Fraction.equiv (duration (E * K) (total (unitCells (N * K))) hEK) (duration E N hE) ∧
    Fraction.le (residualCoefficient (E * K) (squares (unitCells (N * K))) hEK) eps := by
  have hdpos := eps.den_pos
  have hnpos : 0 < eps.num := heps
  let d := eps.den.toNat
  let n := eps.num.toNat
  have hd : (d : Int) = eps.den := Int.toNat_of_nonneg (Int.le_of_lt hdpos)
  have hn : (n : Int) = eps.num := Int.toNat_of_nonneg (Int.le_of_lt hnpos)
  have hn0 : 0 < n := by omega
  have hEK : 0 < E * (N * d + 1) := Nat.mul_pos hE (Nat.succ_pos _)
  refine {N * d + 1, hEK, refined_time N E _ hE hEK, ?_}
  have hq := stats_bound 1 (unitCells (N * (N * d + 1))) (unitCells_le_one _)
  rw [Nat.one_mul] at hq
  change squares (unitCells (N * (N * d + 1))) ≤ total (unitCells (N * (N * d + 1))) at hq
  rw [unitCells_total] at hq
  have hn1 := nat_bound N E n d hE hn0
  have hchain : squares (unitCells (N * (N * d + 1))) * d ≤
    n * (2 * ((E * (N * d + 1)) * (E * (N * d + 1)))) :=
    Nat.le_trans (Nat.mul_le_mul_right d hq) hn1
  have hint := Int.ofNat_le.mpr hchain
  unfold Fraction.le residualCoefficient Fraction.half squareDuration
  dsimp
  rw [← hd, ← hn]
  simp only [Int.ofNat_mul] at hint
  have h2 : ((2 : Nat) : Int) = 2 := rfl
  rw [h2] at hint
  simp only [Int.ofNat_mul, Int.mul_assoc, Int.mul_comm, Int.mul_left_comm] using hint
end NewtonLimitDynamics.Polygon.UniformRefinement

```

NewtonLimitDynamics/Polygon/PartitionComparison.lean, 8 theorems, 99 lines

```

import NewtonLimitDynamics.Polygon.PartialCell

namespace NewtonLimitDynamics.Polygon.PartitionComparison

open NewtonLimitDynamics
open TimeSubdivision
open PartitionControl

private theorem half_equiv {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv (Fraction.half a) (Fraction.half b) := by
  unfold Fraction.equiv Fraction.half at *
  dsimp
  calc a.num * (2 * b.den) = 2 * (a.num * b.den) := by ac_rfl
  _ = 2 * (b.num * a.den) := by rw [h]
  _ = b.num * (2 * a.den) := by ac_rfl

private theorem candidate_scalar_congr {s t : Fraction} (h : Fraction.equiv s t)
  (p v a : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add p (Fraction.mul s v)) (Fraction.mul (Fraction.half (Fraction.mul s s))
      ↪ a))
    (Fraction.add (Fraction.add p (Fraction.mul t v)) (Fraction.mul (Fraction.half (Fraction.mul t t))
      ↪ a)) :=
  Fraction.equiv_trans
    (Fraction.add_equiv_right _ (Fraction.add_equiv_left p (Fraction.mul_equiv_right v h)))
    (Fraction.add_equiv_left _ (Fraction.mul_equiv_right a (half_equiv (Fraction.mul_equiv h h))))

/-- The constructed candidate depends only on the represented rational time. -/
theorem candidate_time_congr (p v a : Point) {s t : Fraction} (h : Fraction.equiv s t) :
  pointEquiv (candidate p v a s) (candidate p v a t) :=
  (candidate_scalar_congr h p.1 v.1 a.1, candidate_scalar_congr h p.2 v.2 a.2)

/-- Two arbitrary finite schedules, with possibly different common denominators
  and cells, reaching the same rational time: their actual positions, each
  corrected by its own exact residual `(Q/(2D^2))*a`, agree. The candidate
  serves only as the common algebraic comparison term. -/
theorem partition_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  pointEquiv
    (pointAdd (partitionMotion D hD p v a ws).1
      (pointScale (residualCoefficient D (squares ws) hD) a))
    (pointAdd (partitionMotion E hE p v a ws').1
      (pointScale (residualCoefficient E (squares ws') hE) a)) :=
  pointEquiv_trans (pointEquiv_symm (candidate_partitionMotion_residual D hD p v a ws))
    (pointEquiv_trans (candidate_time_congr p v a ht)
      (candidate_partitionMotion_residual E hE p v a ws'))

private theorem velocity_scalar_congr {s t : Fraction} (h : Fraction.equiv s t) (v a : Fraction) :
  Fraction.equiv (Fraction.add v (Fraction.mul s a)) (Fraction.add v (Fraction.mul t a)) :=
  Fraction.add_equiv_left v (Fraction.mul_equiv_right a h)

/-- Two arbitrary schedules reaching equivalent rational times have equivalent
  actual velocities; no correction term is needed. -/
theorem velocity_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat)
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  pointEquiv (partitionMotion D hD p v a ws).2 (partitionMotion E hE p v a ws').2 :=
  pointEquiv_trans (partitionMotion_formula D hD p v a ws).2
    (pointEquiv_trans
      (show pointEquiv (encodedVelocity D hD (stats ws) v a)
        (encodedVelocity E hE (stats ws') v a) from
        (velocity_scalar_congr ht v.1 a.1, velocity_scalar_congr ht v.2 a.2))
      (pointEquiv_symm (partitionMotion_formula E hE p v a ws').2))

/-- Sample times inside cells: two schedules, each followed by a partial final
  drift (`u/D` and `u/E`), reaching equivalent rational times. Their actual
  partial positions agree after each is corrected by its own residual
  `(Q+u*u)/(2D^2))*a`. Derived through the appended schedules. -/

```

```

theorem partial_comparison (D E : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat) (u u' : Nat)
  (ht : Fraction.equiv (duration D (total ws + u) hD) (duration E (total ws' + u') hE)) :
  pointEquiv
    (pointAdd (PartialCell.actualPartialPosition D hD p v a ws u)
      (pointScale (residualCoefficient D (squares ws + u * u) hD) a))
    (pointAdd (PartialCell.actualPartialPosition E hE p v a ws' u')
      (pointScale (residualCoefficient E (squares ws' + u' * u') hE) a)) := by
  have h := partition_comparison D E hD hE p v a (ws ++ [u]) (ws' ++ [u'])
  (by rw [PartialCell.total_append, PartialCell.total_append]; exact ht)
  rw [PartialCell.squares_append, PartialCell.squares_append,
    ← PartialCell.partialState_append, ← PartialCell.partialState_append,
    PartialCell.partialState_position, PartialCell.partialState_position] at h
  exact h

/-- Packaged gap statement: two schedules reaching one rational time differ
only along `a`, through two nonnegative coefficients each bounded by its
own largest-cell coefficient `M*T/(2D^2)` (largest cell times elapsed time,
halved). -/
theorem partition_gap (D E M M' : Nat) (hD : 0 < D) (hE : 0 < E) (p v a : Point)
  (ws ws' : List Nat) (hw : ∀ w ∈ ws, w ≤ M) (hw' : ∀ w ∈ ws', w ≤ M')
  (ht : Fraction.equiv (duration D (total ws) hD) (duration E (total ws') hE)) :
  ∃ c c' : Fraction,
    Fraction.le (Fraction.ofInt 0) c ∧ Fraction.le c (meshCoefficient D M (total ws) hD) ∧
    Fraction.le (Fraction.ofInt 0) c' ∧ Fraction.le c' (meshCoefficient E M' (total ws') hE) ∧
    pointEquiv (pointAdd (partitionMotion D hD p v a ws).1 (pointScale c a))
      (pointAdd (partitionMotion E hE p v a ws').1 (pointScale c' a)) :=
  (λ_, λ_, residual_nonnegative D _ hD, residual_mesh_bound D M hD ws hw,
    residual_nonnegative E _ hE, residual_mesh_bound E M' hE ws' hw',
    partition_comparison D E hD hE p v a ws ws' ht)

end NewtonLimitDynamics.Polygon.PartitionComparison

NewtonLimitDynamics/Polygon/StripArea.lean, 11 theorems, 151 lines

import NewtonLimitDynamics.Polygon.TimeSubdivision

/-!
Order 3 remainder (TASKS.md): the strip-area sum of the constant-force polygon.
For a uniform rational cell `h`, initial velocity `v` and constant accelerative
force `a`, the end-kick recurrence gives velocity `v_n = v + (n·h)·a` and chord
`c_n = h·v_n` (the drift of cell `n`). The two-cell chord triangle
`(p_n, p_{n+1}, p_{n+2})` has doubled area `det c_n c_{n+1}`. Because
`det (w + s·a) a = det w a`, every such triangle has the same doubled area
`h³·det(v, a)`, and the signed total over `k` triangles is
`k·h³·det(v, a)`. The common value can be negative: equality of the triangles
does not equate signed and absolute sums. An absolute sum would instead be
`k·|h³·det(v, a)|`; its formalization and a geometric strip decomposition remain
open. Pure finite Fraction arithmetic; no curve, limit or force law.
-/

namespace NewtonLimitDynamics.Polygon.StripArea

open NewtonLimitDynamics
open TimeSubdivision

/-- Velocity after `n` equal end-kick cells. -/
def velAt (h : Fraction) (v a : Point) : Nat → Point
| 0 => v
| n + 1 => pointAdd (velAt h v a n) (pointScale h a)

/-- Position at the start of cell `n`, drifting with the incoming velocity. -/
def posAt (h : Fraction) (p v a : Point) : Nat → Point
| 0 => p
| n + 1 => pointAdd (posAt h p v a n) (pointScale h (velAt h v a n))

/-- The chord of cell `n`: the drift `h·v_n` of that cell. -/
def chord (h : Fraction) (v a : Point) (n : Nat) : Point := pointScale h (velAt h v a n)

/-- Doubled area of the two-cell chord triangle at cell `n`. -/
def twoCellTwice (h : Fraction) (v a : Point) (n : Nat) : Fraction :=

```

```

det (chord h v a n) (chord h v a (n + 1))

theorem pointSub_add_self_left_equiv (x y : Point) :
  pointEquiv (pointSub (pointAdd x y) x) y := by
  constructor <=>
  simp only [pointSub, pointAdd, pointNeg, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- The drift of a cell is exactly the position difference `p_{n+1} - p_n`. -/
theorem chord_is_position_diff (h : Fraction) (p v a : Point) (n : Nat) :
  pointEquiv (pointSub (posAt h p v a (n + 1)) (posAt h p v a n)) (chord h v a n) := by
  simp only [posAt, chord]
  exact pointSub_add_self_left_equiv (posAt h p v a n) (pointScale h (velAt h v a n))

private theorem det_add_left (q x y : Point) :
  Fraction.equiv (det (pointAdd x y) q) (Fraction.add (det x q) (det y q)) := by
  simp only [det, pointAdd, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_scale_left (d : Fraction) (q x : Point) :
  Fraction.equiv (det (pointScale d x) q) (Fraction.mul d (det x q)) := by
  simp only [det, pointScale, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_scale_right (d : Fraction) (p x : Point) :
  Fraction.equiv (det p (pointScale d x)) (Fraction.mul d (det p x)) := by
  simp only [det, pointScale, Fraction.add, Fraction.mul, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

private theorem det_self (x : Point) : Fraction.equiv (det x x) (Fraction.ofInt 0) := by
  simp only [det, Fraction.add, Fraction.mul, Fraction.ofInt, Fraction.equiv,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega

/-- The key lemma from the hand computation: `det (w + s·a) a = det w a`. -/
theorem det_kick_direction_constant (s : Fraction) (w a : Point) :
  Fraction.equiv (det (pointAdd w (pointScale s a)) a) (det w a) := by
  have h1 := det_add_left a w (pointScale s a)
  have h2 := det_scale_left s a a
  have h3 := det_self a
  have t1 : Fraction.equiv (det (pointAdd w (pointScale s a)) a)
    (Fraction.add (det w a) (Fraction.mul s (det a a))) :=
    Fraction.equiv_trans h1 (Fraction.add_equiv_left (det w a) h2)
  have t2 : Fraction.equiv (Fraction.mul s (det a a)) (Fraction.mul s (Fraction.ofInt 0)) :=
    Fraction.mul_equiv_left s h3
  have t3 : Fraction.equiv (Fraction.mul s (Fraction.ofInt 0)) (Fraction.ofInt 0) :=
    Fraction.mul_zero s
  have t4 : Fraction.equiv (Fraction.add (det w a) (Fraction.ofInt 0)) (det w a) := by
    simp only [Fraction.add, Fraction.ofInt, Fraction.equiv,
      Int.add_mul, Int.mul_add] <=> ac_nf <=> omega
  exact Fraction.equiv_trans t1
    (Fraction.equiv_trans (Fraction.add_equiv_left (det w a) (Fraction.equiv_trans t2 t3)) t4)

/-- Each velocity has the same determinant with `a` as the initial velocity. -/
theorem vel_det_constant (h : Fraction) (v a : Point) (n : Nat) :
  Fraction.equiv (det (velAt h v a n) a) (det v a) := by
  induction n with
  | zero => exact Fraction.equiv_refl _
  | succ n ih =>
    exact Fraction.equiv_trans (det_kick_direction_constant h (velAt h v a n) a) ih

/-- Every two-cell chord triangle has doubled area `h³·det(v, a)`. -/
theorem two_cell_triangle_constant (h : Fraction) (v a : Point) (n : Nat) :
  Fraction.equiv (twoCellTwice h v a n)
    (Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a)))) := by
  have vn := velAt h v a n
  have vn1 : velAt h v a (n + 1) = pointAdd (velAt h v a n) (pointScale h a) := by simp [velAt]
  simp only [twoCellTwice, chord, vn1]
  -- det (h·v_n) (h·v_{n+1}) = h·det v_n (h·v_{n+1})
  have s1 := det_scale_left h (pointScale h (pointAdd (velAt h v a n) (pointScale h a))) (velAt h v a n)

```

```

-- = h · (h · det v_n v_{n+1})
have s2 := Fraction.mul_equiv_left h (det_scale_right h (velAt h v a n) (pointAdd (velAt h v a n)
  ↪ (pointScale h a)))
-- det v_n v_{n+1} = det v_n v_n + h · det v_n a
have s3 := det_add_right (velAt h v a n) (velAt h v a n) (pointScale h a)
have s4 := Fraction.add_equiv (det_self (velAt h v a n)) (det_scale_right h (velAt h v a n) a)
have s5 := Fraction.mul_equiv (Fraction.equiv_refl h) (vel_det_constant h v a n)
have inner : Fraction.equiv (det (velAt h v a n) (pointAdd (velAt h v a n) (pointScale h a)))
  (Fraction.mul h (det v a)) :=
  Fraction.equiv_trans (Fraction.equiv_trans s3 s4)
  (Fraction.equiv_trans
    (Fraction.add_equiv_left (Fraction.ofInt 0) s5)
    (by simp only [Fraction.add, Fraction.ofInt, Fraction.mul, Fraction.equiv,
      Int.add_mul, Int.mul_add] <=> ac_nf <=> omega))
have rhs := Fraction.mul_equiv_left h (Fraction.mul_equiv_left h inner)
exact Fraction.equiv_trans (Fraction.equiv_trans s1 s2) rhs

/-- All two-cell triangles have the same signed doubled area. No absolute-area
operation or unsigned sum is asserted. -/
theorem all_triangles_equal (h : Fraction) (v a : Point) (m n : Nat) :
  Fraction.equiv (twoCellTwice h v a m) (twoCellTwice h v a n) :=
  Fraction.equiv_trans (two_cell_triangle_constant h v a m)
    (Fraction.equiv_symm (two_cell_triangle_constant h v a n))

/-- Running sum of the two-cell doubled areas over the first `k` cells. -/
def stripSum (h : Fraction) (v a : Point) : Nat → Fraction
| 0 => Fraction.ofInt 0
| k + 1 => Fraction.add (stripSum h v a k) (twoCellTwice h v a k)

/-- The signed strip sum over `k` triangles is `k · h³ · det(v, a)`. -/
theorem total_strip_area (h : Fraction) (v a : Point) (k : Nat) :
  Fraction.equiv (stripSum h v a k)
    (Fraction.mul (Fraction.ofInt k) (Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a))))) := by
  induction k with
  | zero =>
    simp only [stripSum, Fraction.mul, Fraction.ofInt, Fraction.equiv] <=> omega
  | succ k ih =>
    let H := Fraction.mul h (Fraction.mul h (Fraction.mul h (det v a)))
    simp only [stripSum]
    exact Fraction.equiv_trans
      (Fraction.add_equiv ih (two_cell_triangle_constant h v a k))
      (by simp only [Fraction.mul, Fraction.add, Fraction.ofInt, Fraction.equiv,
        Int.add_mul, Int.mul_add, Int.ofNat_add, Int.mul_one, Int.one_mul] <=>
        ac_nf <=> omega)
end NewtonLimitDynamics.Polygon.StripArea

```

NewtonLimitDynamics/Polygon/CentralSchedule.lean, 17 theorems, 254 lines

```

import NewtonLimitDynamics.Polygon.TimeSubdivision

/-!
Finite impulse polygons for an arbitrary position-dependent central force,
with arbitrary (possibly unequal) rational time cells. Modern rational
coordinate reconstruction motivated by the finite part of Proposition I
(1687 NATP00077 par45, 1713 NATP00082 par51), which divides time into
*equal* parts and applies each impulse at the vertex reached by inertial
motion. The centre S is the origin. No curve, limit or force regularity is
assumed; the field is an arbitrary function on represented points.
-/

namespace NewtonLimitDynamics.Polygon.CentralSchedule

open NewtonLimitDynamics
open TimeSubdivision

/-- An accelerative force field on represented points. -/
abbrev Field := Point → Point

/-- Central about S: the force at each point is parallel to the radius. -/
def central (a : Field) : Prop :=

```

```

    ∀ p, Fraction.equiv (det p (a p)) (Fraction.ofInt 0)

/-- One cell: inertial drift for duration `d` with the incoming velocity, then
    the impulse of the field at the arrival vertex. -/
def cell (a : Field) (d : Fraction) (s : Point × Point) : Point × Point :=
  (pointAdd s.1 (pointScale d s.2),
   pointAdd s.2 (pointScale d (a (pointAdd s.1 (pointScale d s.2)))))

/-- Doubled areal velocity `det(position, velocity)` about S. -/
def momentum (s : Point × Point) : Fraction := det s.1 s.2

/-- The constructed polygon for a finite list of rational cell durations. -/
def schedule (a : Field) : List Fraction → Point × Point → Point × Point
| [], s => s
| d :: ds, s => schedule a ds (cell a d s)

/-- Sum of doubled triangle areas `S x_i x_{i+1}` over the schedule. -/
def swept (a : Field) : List Fraction → Point × Point → Fraction
| [], _ => Fraction.ofInt 0
| d :: ds, s => Fraction.add (det s.1 (cell a d s).1) (swept a ds (cell a d s))

/-- Total elapsed time of the schedule. -/
def elapsed : List Fraction → Fraction
| [] => Fraction.ofInt 0
| d :: ds => Fraction.add d (elapsed ds)

theorem det_drift (x v : Point) (d : Fraction) :
  Fraction.equiv (det (pointAdd x (pointScale d v)) v) (det x v) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

theorem det_kick_split (x v w : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd v (pointScale d w)))
    (Fraction.add (det x v) (Fraction.mul d (det x w))) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- The doubled triangle swept in one drift is duration times areal velocity. -/
theorem det_cell_area (x v : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd x (pointScale d v))) (Fraction.mul d (det x v)) := by
  unfold Fraction.equiv det pointAdd pointScale Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- A central impulse at `x` leaves `det(x, ·)` unchanged. -/
theorem central_kick (a : Field) (hc : central a) (x v : Point) (d : Fraction) :
  Fraction.equiv (det x (pointAdd v (pointScale d (a x)))) (det x v) :=
  Fraction.equiv_trans (det_kick_split x v (a x) d)
    (Fraction.equiv_trans
      (Fraction.add_equiv_left (det x v)
        (Fraction.equiv_trans (Fraction.mul_equiv_left d (hc x)) (Fraction.mul_zero d)))
      (Fraction.add_zero (det x v)))

/-- Each cell (drift, then central impulse at the arrival vertex) preserves the
    areal velocity exactly, for any cell duration. -/
theorem cell_momentum (a : Field) (hc : central a) (d : Fraction) (s : Point × Point) :
  Fraction.equiv (momentum (cell a d s)) (momentum s) :=
  Fraction.equiv_trans (central_kick a hc _ s.2 d) (det_drift s.1 s.2 d)

theorem schedule_momentum (a : Field) (hc : central a) :
  (ds : List Fraction) → (s : Point × Point) →
  Fraction.equiv (momentum (schedule a ds s)) (momentum s)

```



```

| [], _ => Fraction.equiv_refl _
| d :: ds, s =>
  Fraction.equiv_trans (schedule_momentum a hc ds (cell a d s)) (cell_momentum a hc d s)

/-- Areas proportional to times, for arbitrary unequal rational cells and any
central field: the swept doubled area equals elapsed time times the initial
doubled areal velocity. -/
theorem swept_eq (a : Field) (hc : central a) :
  (ds : List Fraction) → (s : Point × Point) →
  Fraction.equiv (swept a ds s) (Fraction.mul (elapsed ds) (momentum s))
| [], _ => by
  unfold swept elapsed Fraction.equiv Fraction.mul Fraction.ofInt
  simp
| d :: ds, s => by
  have hcell : Fraction.equiv (det s.1 (cell a d s).1) (Fraction.mul d (momentum s)) :=
    det_cell_area s.1 s.2 d
  have hrest : Fraction.equiv (swept a ds (cell a d s))
    (Fraction.mul (elapsed ds) (momentum s)) :=
    Fraction.equiv_trans (swept_eq a hc ds (cell a d s))
      (Fraction.mul_equiv_left (elapsed ds) (cell_momentum a hc d s))
  exact Fraction.equiv_trans
    (Fraction.equiv_trans (Fraction.add_equiv_right _ hcell) (Fraction.add_equiv_left _ hrest))
    (Fraction.equiv_symm (Fraction.add_mul d (elapsed ds) (momentum s)))

/-- Refinement of one cell `h+k` into `h` then `k`, for an arbitrary field:
the fine endpoint exceeds the coarse one by exactly `h*k` times the force at
the intermediate vertex. No regularity of the field is used. -/
theorem refine_position (a : Field) (h k : Fraction) (s : Point × Point) :
  pointEquiv (cell a k (cell a h s)).1
    (pointAdd (cell a (Fraction.add h k) s).1
      (pointScale (Fraction.mul h k) (a (cell a h s).1))) := by
  constructor <=>
  · unfold cell pointAdd pointScale Fraction.equiv Fraction.add Fraction.mul
    dsimp
    simp only [Int.add_mul, Int.mul_add]
    ac_nf

private def fsub (p q : Fraction) : Fraction := Fraction.add p (-q.num, q.den, q.den_pos)

private theorem velocity_scalar (v ay az aX h k : Fraction) :
  Fraction.equiv
    (Fraction.add (Fraction.add v (Fraction.mul h ay)) (Fraction.mul k az))
    (Fraction.add (Fraction.add v (Fraction.mul (Fraction.add h k) aX))
      (Fraction.add (Fraction.mul h (fsub ay aX)) (Fraction.mul k (fsub az aX)))) := by
  unfold fsub Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Velocity mismatch of the same refinement, exactly: the fine velocity equals
the coarse one plus `h*(a y - a X) + k*(a z - a X)`, where `y`, `z` are the
fine vertices and `X` the coarse vertex. It vanishes for a constant field;
otherwise its control is a premise about the field, not a consequence of
the construction. -/
theorem refine_velocity (a : Field) (h k : Fraction) (s : Point × Point) :
  pointEquiv (cell a k (cell a h s)).2
    (pointAdd (cell a (Fraction.add h k) s).2
      (pointAdd
        (pointScale h (pointSub (a (cell a h s).1) (a (cell a (Fraction.add h k) s).1)))
        (pointScale k (pointSub (a (cell a k (cell a h s)).1) (a (cell a (Fraction.add h k) s).1))))) :=
  (velocity_scalar _ _ _ _ _ , velocity_scalar _ _ _ _ _ )

private theorem expand_left (d d' L K : Fraction) :
  Fraction.equiv (Fraction.mul d (Fraction.mul d' (Fraction.add L K)))
    (Fraction.add (Fraction.mul (Fraction.mul d d') L) (Fraction.mul (Fraction.mul d d') K)) := by
  unfold Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

```

```

private theorem expand_right (d d' L : Fraction) :
  Fraction.equiv (Fraction.mul d' (Fraction.mul d L)) (Fraction.mul (Fraction.mul d d') L) := by
  unfold Fraction.equiv Fraction.mul
  dsimp
  ac_nf

private theorem add_cancel_num {P Q : Fraction} (h : Fraction.equiv (Fraction.add P Q) P) :
  Q.num = 0 := by
  unfold Fraction.equiv Fraction.add at h
  dsimp at h
  have hpd := P.den_pos
  have e1 : P.num * Q.den * P.den = P.num * (P.den * Q.den) := by ac_rfl
  have hz : Q.num * P.den * P.den = 0 := by
    rw [Int.add_mul] at h
    omega
  rcases Int.mul_eq_zero.mp hz with h1 | h1
  · rcases Int.mul_eq_zero.mp h1 with h2 | h2
    · exact h2
    · omega
  · omega

/-- Converse for unequal cells (motivated by Proposition II, whose statement
speaks of areas proportional to times): drift `d` from `x` with velocity
`v`, an arbitrary impulse `J` at the arrival vertex `x`, then drift `d'`.
If the two doubled areas are proportional to the nonzero durations, the
impulse is parallel to the radius `Sx`. -/
theorem unequal_cells_converse (x v J : Point) (d d' : Fraction)
  (hd : d.num ≠ 0) (hd' : d'.num ≠ 0)
  (h : Fraction.equiv
    (Fraction.mul d (det (pointAdd x (pointScale d v))
      (pointAdd (pointAdd x (pointScale d v)) (pointScale d' (pointAdd v J)))))
    (Fraction.mul d' (det x (pointAdd x (pointScale d v))))) :
  Fraction.equiv (det (pointAdd x (pointScale d v)) J) (Fraction.ofInt 0) := by
  -- second triangle: d' * (L + K), with L = det x v and K = det x' J
  have hA : Fraction.equiv
    (det (pointAdd x (pointScale d v))
      (pointAdd (pointAdd x (pointScale d v)) (pointScale d' (pointAdd v J))))
    (Fraction.mul d' (Fraction.add (det x v) (det (pointAdd x (pointScale d v)) J))) :=
  Fraction.equiv_trans (det_cell_area _ _ d')
    (Fraction.mul_equiv_left d'
      (Fraction.equiv_trans (det_add_right _ v J)
        (Fraction.add_equiv_right _ (det_drift x v d))))
  have hB := det_cell_area x v d
  have h1 := Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_equiv_left d hA))
    (Fraction.equiv_trans h (Fraction.mul_equiv_left d' hB))
  have h2 := Fraction.equiv_trans (Fraction.equiv_symm (expand_left d d' _))
    (Fraction.equiv_trans h1 (expand_right d d' _))
  have hq := add_cancel_num h2
  have hq' : d.num * d'.num * (det (pointAdd x (pointScale d v)) J).num = 0 := hq
  have hK : (det (pointAdd x (pointScale d v)) J).num = 0 := by
    rcases Int.mul_eq_zero.mp hq' with h3 | h3
    · rcases Int.mul_eq_zero.mp h3 with h4 | h4
      · exact absurd h4 hd
      · exact absurd h4 hd'
    · exact h3
  unfold Fraction.equiv Fraction.ofInt
  dsimp
  rw [hK]
  simp

/-- A varying central field: force proportional to distance, toward S. -/
def harmonic : Field := fun p => pointNeg p

theorem harmonic_central : central harmonic := by
  intro p
  unfold harmonic pointNeg det Fraction.equiv Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.zero_mul]
  ac_nf

```

```

omega

def exampleState : Point × Point := ((one, zero), (zero, one))

/-- In the harmonic example, refining a unit cell into two half cells changes the
terminal velocity as well as the endpoint: a varying force breaks the
exact velocity agreement that holds for constant force. -/
theorem harmonic_refinement_changes_velocity :
  ~ pointEquiv (cell harmonic half (cell harmonic half exampleState)).2
    (cell harmonic (Fraction.add half half) exampleState).2 ∧
  ~ pointEquiv (cell harmonic half (cell harmonic half exampleState)).1
    (cell harmonic (Fraction.add half half) exampleState).1 := by
  decide

/-- Both schedules nevertheless sweep the same doubled area, as `swept_eq`
requires: elapsed time times the initial areal velocity. -/
theorem harmonic_equal_swept :
  Fraction.equiv (swept harmonic [half, half] exampleState)
    (swept harmonic [Fraction.add half half] exampleState) := by
  decide

end NewtonLimitDynamics.Polygon.CentralSchedule

NewtonLimitDynamics/Polygon/HarmonicRefinement.lean, 15 theorems, 133 lines

import NewtonLimitDynamics.Polygon.HarmonicStability

/-!
Finite common-time comparison for the linear central field. The two actual
fine cells have duration `h`; the one actual coarse cell has duration `h+h`.
Their terminal positions need not agree. The directed connector from the fine
endpoint to the coarse endpoint closes a polygon comparison and is not a
mechanical cell. All statements are finite rational arithmetic. In particular,
this file constructs no limiting curve and makes no global area estimate.
Modern reconstruction motivated by the polygon arguments in De Motu
NATP00089 par9 / NATP00090 par17, 1687 NATP00077 par45 and 1713 NATP00082
par51. No historical edge between stages is inferred. The physical time and
inward-force readings require  $h>0$  and  $w>0$ ; the algebra also holds for signed
or zero parameters.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicRefinement

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability

def middle (w h : Fraction) (s : Point × Point) : Point :=
  (cell (linearField w) h s).1

def fine (w h : Fraction) (s : Point × Point) : Point × Point :=
  cell (linearField w) h (cell (linearField w) h s)

def coarse (w h : Fraction) (s : Point × Point) : Point × Point :=
  cell (linearField w) (Fraction.add h h) s

/-- The directed endpoint connector is `z → X`; it is bookkeeping only. -/
def connector (w h : Fraction) (s : Point × Point) : Point :=
  pointSub (coarse w h s).1 (fine w h s).1

/-- The fine endpoint is the coarse endpoint plus ` $-h^2 w y`'. -/
theorem position_mismatch (w h : Fraction) (s : Point × Point) :
  pointEquiv (fine w h s).1
    (pointAdd (coarse w h s).1
      (pointScale (negF (Fraction.mul (Fraction.mul h h) w)) (middle w h s)))) := by
  constructor <=>
  simp only [fine, coarse, middle, cell, linearField, negF, pointAdd,
    pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega$ 
```

```

/-- Fine-minus-coarse velocity:  $h^2 w v + h^3 w^2 y$ . -/
theorem velocity_mismatch (w h : Fraction) (s : Point × Point) :
  pointEquiv (fine w h s).2
    (pointAdd (coarse w h s).2
      (pointAdd (pointScale (Fraction.mul (Fraction.mul h h) w) s.2)
        (pointScale (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
          (Fraction.mul w w)) (middle w h s)))) := by
  constructor <=>
    simp only [fine, coarse, middle, cell, linearField, negF, pointAdd,
      pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
      Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
    ac_nf <=> omega

/-- Signed doubled area of the closed boundary  $x \rightarrow y \rightarrow z \rightarrow X \rightarrow x$ .
This is the area between the finite polygons, not either Kepler swept area. -/
def closedDefect (w h : Fraction) (s : Point × Point) : Fraction :=
  closedBoundaryTwice s.1 (middle w h s) (fine w h s).1 (coarse w h s).1

/-- The local closed boundary reduces to the single triangle  $y, z, X$ . -/
theorem closed_eq_triangle (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (closedDefect w h s)
    (det (pointSub (fine w h s).1 (middle w h s))
      (pointSub (coarse w h s).1 (middle w h s))) := by
  simp only [closedDefect, closedBoundaryTwice, fine, coarse, middle, cell,
    linearField, negF, pointSub, pointNeg, pointAdd, pointScale, det,
    Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- Exact cubic signed doubled gap for the common-time local refinement. -/
theorem closed_defect_cubic (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (closedDefect w h s)
    (negF (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
      (Fraction.mul w (det s.1 s.2)))) := by
  simp only [closedDefect, closedBoundaryTwice, fine, coarse, middle, cell,
    linearField, negF, pointAdd, pointScale, det,
    Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- Nonnegative magnitude of this one closed signed triangle. It does not
account for overlaps or multiple lobes in a global polygon comparison. -/
def absoluteClosedGap (w h : Fraction) (s : Point × Point) : Fraction :=
  (closedDefect w h s).abs

theorem absoluteClosedGap_nonnegative (w h : Fraction) (s : Point × Point) :
  0 ≤ (absoluteClosedGap w h s).num := by
  exact Fraction.abs_num_nonnegative _

/-- Nonnegative local gap as the magnitude of the exact cubic coefficient.
This is one triangle, so no cancellation of distinct lobes is involved. -/
theorem absolute_closed_defect_cubic (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (absoluteClosedGap w h s)
    (Fraction.mul (Fraction.mul (Fraction.mul h h) h)
      (Fraction.mul w (det s.1 s.2))).abs :=
  Fraction.equiv_trans (Fraction.abs_equiv (closed_defect_cubic w h s))
    (Fraction.abs_neg _)

/-- Central-force Kepler sums agree for these two schedules. -/
theorem swept_equal (w h : Fraction) (s : Point × Point) :
  Fraction.equiv (swept (linearField w) [h, h] s)
    (swept (linearField w) [Fraction.add h h] s) := by
  simp only [swept, fine, coarse, middle, cell, linearField, negF,
    pointAdd, pointScale, det, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.mul_one, Int.one_mul, Int.zero_mul, Int.mul_zero] <=>
  ac_nf <=> omega

private def one : Fraction := (1, 1, by decide)

```

```

private def zero : Fraction := (0, 1, by decide)
private def half : Fraction := (1, 2, by decide)
private def sample : Point × Point := ((one, zero), (zero, one))

theorem sample_middle : pointEquiv (middle one half sample) (one, half) := by decide
theorem sample_fine : pointEquiv (fine one half sample).1
  ((3, 4, by decide), (7, 8, by decide)) := by decide
theorem sample_coarse : pointEquiv (coarse one half sample).1 (one, one) := by decide
theorem sample_fine_swept : Fraction.equiv (swept (linearField one) [half, half] sample) one := by decide
theorem sample_coarse_swept : Fraction.equiv (swept (linearField one) [Fraction.add half half] sample)
  ↪ one := by decide
theorem sample_closed_defect : Fraction.equiv (closedDefect one half sample) (-1, 8, by decide) := by
  ↪ decide
theorem sample_absolute_closed_gap :
  Fraction.equiv (absoluteClosedGap one half sample) (1, 8, by decide) := by decide
theorem sample_closed_defect_nonzero :
  ¬ Fraction.equiv (closedDefect one half sample) zero := by decide

end NewtonLimitDynamics.Polygon.HarmonicRefinement

```

NewtonLimitDynamics/Polygon/HarmonicComparison.lean, 18 theorems, 214 lines

```

import BarrowLib.Polygon.StateDistance
import NewtonLimitDynamics.Polygon.PointBounds
import BarrowLib.Polygon.FiniteEstimates
import NewtonLimitDynamics.Polygon.HarmonicStability

/-!
Finite perturbation comparisons for the actual harmonic end-kick cell. The L1
state magnitude is a chosen coordinate diagnostic and requires a unit
calibration before interpreting position and velocity together physically.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicComparison

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open PointBounds

def kappa (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs)
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs))

private theorem zero_le_product (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) :
  Fraction.le (Fraction.ofInt 0) (Fraction.mul a b) := by
  unfold Fraction.le Fraction.ofInt Fraction.mul
  dsimp
  simpa using Int.mul_nonneg ha hb

private theorem one_plus_bound_right (a b c : Fraction)
  (ha : 0 ≤ a.num) (hc : 0 ≤ c.num) :
  Fraction.le (Fraction.add (Fraction.add a b) (Fraction.mul c b))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c) (Fraction.add a b)) := by
  let lhs := Fraction.add (Fraction.add a b) (Fraction.mul c b)
  have h := Fraction.add_le_add_left (zero_le_product c a hc ha) lhs
  have h' : Fraction.le lhs (Fraction.add lhs (Fraction.mul c a)) :=
    Fraction.le_equiv_left (Fraction.equiv_symm (Fraction.add_zero lhs)) h
  apply Fraction.le_equiv_right h'
  simp only [lhs, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

private theorem one_plus_bound_left (a b c : Fraction)
  (hb : 0 ≤ b.num) (hc : 0 ≤ c.num) :
  Fraction.le (Fraction.add (Fraction.add a b) (Fraction.mul c a))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c) (Fraction.add a b)) := by
  let lhs := Fraction.add (Fraction.add a b) (Fraction.mul c a)
  have h := Fraction.add_le_add_left (zero_le_product c b hc hb) lhs

```

```

have h' : Fraction.le lhs (Fraction.add lhs (Fraction.mul c b)) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (Fraction.add_zero lhs)) h
apply Fraction.le_equiv_right h'
simp only [lhs, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
ac_nf

def drift (h : Fraction) (s : Point × Point) : Point × Point :=
  (pointAdd s.1 (pointScale h s.2), s.2)

def kick (w h : Fraction) (s : Point × Point) : Point × Point :=
  (s.1, pointAdd s.2 (pointScale h (linearField w s.1)))

theorem cell_eq_kick_drift (w h : Fraction) (s : Point × Point) :
  cell (linearField w) h s = kick w h (drift h s) := rfl

theorem drift_bound (h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (drift h s))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs) (stateNorm s)) := by
  have h₁ := Fraction.add_le_add_right (pointNorm_add_le s.1 (pointScale h s.2))
    (pointNorm s.2)
  have h₂ : Fraction.equiv
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm (pointScale h s.2)))
      (pointNorm s.2))
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm s.2))
      (Fraction.mul h.abs (pointNorm s.2))) := by
  have hs := pointNorm_scale h s.2
  have hh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.1)) hs
  have hhh := Fraction.add_equiv hh (Fraction.equiv_refl (pointNorm s.2))
  exact Fraction.equiv_trans hhh (by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul]
    simp only [Int.add_mul, Int.mul_add]
    ac_nf)
  have h₃ := Fraction.le_equiv_right h₁ h₂
  exact Fraction.magnitudes.le_trans h₃
    (one_plus_bound_right (pointNorm s.1) (pointNorm s.2) h.abs
      (pointNorm_nonnegative _) (Fraction.abs_num_nonnegative _))

private theorem kick_scale_norm (w h : Fraction) (p : Point) :
  Fraction.equiv (pointNorm (pointScale h (linearField w p)))
    (Fraction.mul (Fraction.mul h.abs w.abs) (pointNorm p)) := by
  simp only [Fraction.equiv, pointNorm, pointScale, linearField, negF,
    Fraction.abs, Fraction.add, Fraction.mul, Int.natAbs_mul,
    Int.natAbs_neg, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem kick_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (kick w h s))
    (Fraction.mul
      (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs))
      (stateNorm s)) := by
  have h₁ := Fraction.add_le_add_left
    (pointNorm_add_le s.2 (pointScale h (linearField w s.1))) (pointNorm s.1)
  have h₂ : Fraction.equiv
    (Fraction.add (pointNorm s.1)
      (Fraction.add (pointNorm s.2)
        (pointNorm (pointScale h (linearField w s.1)))))
    (Fraction.add (Fraction.add (pointNorm s.1) (pointNorm s.2))
      (Fraction.mul (Fraction.mul h.abs w.abs) (pointNorm s.1))) := by
  have hf := kick_scale_norm w h s.1
  have hh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.2)) hf
  have hhh := Fraction.add_equiv (Fraction.equiv_refl (pointNorm s.1)) hh
  exact Fraction.equiv_trans hhh (by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul]
    simp only [Int.add_mul, Int.mul_add]
    ac_nf)
  have h₃ := Fraction.le_equiv_right h₁ h₂
  exact Fraction.magnitudes.le_trans h₃
    (one_plus_bound_left (pointNorm s.1) (pointNorm s.2))

```

```

(Fraction.mul h.abs w.abs) (pointNorm_nonnegative _)
(Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _)))

/-- One actual harmonic end-kick cell amplifies the coordinate L1 state
magnitude by at most  $(1+|h|)(1+|h||w|)$ , including signed and zero data. -/
theorem cell_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (cell (linearField w) h s))
    (Fraction.mul (kappa w h) (stateNorm s)) := by
  rw [cell_eq_kick_drift]
  have hk := kick_bound w h (drift h s)
  have hd := drift_bound h s
  have hc : 0 ≤ (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)).num := by
    have hp := Int.mul_pos h.abs.den_pos w.abs.den_pos
    have hn := Int.mul_nonneg (Fraction.abs_num_nonnegative h)
      (Fraction.abs_num_nonnegative w)
    unfold Fraction.add Fraction.ofInt Fraction.mul
    dsimp
    omega
  have hm := Fraction.mul_le_mul_nonnegative_left hd
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)) hc
  have hchain := Fraction.magnitudes.le_trans hk hm
  apply Fraction.le_equiv_right hchain
  simp only [kappa, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

/-- Subtracting the outputs of two actual cells equals applying the same
linear harmonic cell to their input difference, as rational values. -/
theorem cell_difference (w h : Fraction) (s t : Point × Point) :
  stateEquiv (stateSub (cell (linearField w) h s) (cell (linearField w) h t))
    (cell (linearField w) h (stateSub s t)) := by
  constructor <=> constructor <=>
  simp only [zero, stateSub, pointEquiv, pointSub, pointNeg, cell, linearField,
    negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- A one-step perturbation estimate for two actual cells under the same
linear field and duration. The input difference is a represented state. -/
theorem linearField_comparison_contract (w : Fraction) :
  FiniteEstimates.comparisonContract (linearField w) (linearField w)
    w.abs (Fraction.ofInt 0) := by
  intro p q
  have hs := FiniteEstimates.difference_scale (negF w) p q
  have he := Fraction.equiv_trans hs
  (Fraction.mul_equiv (Fraction.abs_neg w) (Fraction.equiv_refl _))
  exact Fraction.le_of_equiv (Fraction.equiv_trans he
    (Fraction.equiv_symm (Fraction.add_zero _)))

theorem cell_perturbation (w h : Fraction) (s t : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) h s) (cell (linearField w) h t)))
    (Fraction.mul (kappa w h) (stateNorm (stateSub s t))) := by
  have hg := FiniteEstimates.cell_amplification (linearField w) (linearField w)
    h w.abs (Fraction.ofInt 0) s t (Fraction.abs_num_nonnegative w)
    (linearField_comparison_contract w)
  apply Fraction.le_equiv_right hg
  exact Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _)
    (Fraction.mul_zero h.abs)) (Fraction.add_zero _)

private def one : Fraction := {1, 1, by decide}
private def zero : Fraction := {0, 1, by decide}
private def half : Fraction := {1, 2, by decide}
private def sample : Point × Point := ((one, zero), (zero, one))

/-- Zero duration leaves the represented state unchanged in rational value. -/
theorem zero_step (w : Fraction) (s : Point × Point) :
  stateEquiv (cell (linearField w) zero s) s := by
  constructor <=> constructor <=>
  simp only [zero, stateSub, pointEquiv, pointSub, pointNeg, cell, linearField,

```

```

    negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.zero_mul, Int.mul_zero, Int.add_zero] <=>
ac_nf <=> omega

theorem zero_step_norm (w : Fraction) (s : Point × Point) :
  Fraction.equiv (stateNorm (cell (linearField w) zero s)) (stateNorm s) :=
  stateNorm_equiv (zero_step w s)

theorem zero_kappa (w : Fraction) : Fraction.equiv (kappa w zero) one := by
  simp only [kappa, zero, one, Fraction.equiv, Fraction.abs, Fraction.ofInt,
    Fraction.add, Fraction.mul]
  dsimp
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero, Int.mul_one, Int.one_mul]

theorem sample_kappa : Fraction.equiv (kappa one half) (9, 4, by decide) := by decide
theorem sample_initial_norm : Fraction.equiv (stateNorm sample) (Fraction.ofInt 2) := by decide
theorem sample_cell_norm :
  Fraction.equiv (stateNorm (cell (linearField one) half sample)) (11, 4, by decide) := by decide
theorem sample_cell_bound :
  Fraction.le (stateNorm (cell (linearField one) half sample))
    (Fraction.mul (kappa one half) (stateNorm sample)) := by
  unfold Fraction.le
  decide

end NewtonLimitDynamics.Polygon.HarmonicComparison

```

NewtonLimitDynamics/Polygon/HarmonicAccumulation.lean, 31 theorems, 371 lines

```

import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import BarrowLib.Polygon.StateDistance
import NewtonLimitDynamics.Polygon.HarmonicRefinement
import NewtonLimitDynamics.Polygon.HarmonicComparison

/-!
Finite global comparison of actual harmonic end-kick schedules with common
elapsed time. This is a coordinate L1 state budget only. It constructs no
limiting trajectory and makes no claim about the nonnegative region between
polygonal paths or the separate Kepler swept areas. The De Motu, 1687, and
1713 historical stages remain distinct from this modern rational estimate.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicAccumulation

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicRefinement
open HarmonicComparison
open PointBounds

def localFactor (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul (Fraction.mul h.abs h.abs) w.abs)
    (Fraction.add (kappa w h) (Fraction.ofInt 1))

def coarseFactor (w h : Fraction) : Fraction := kappa w (Fraction.add h h)
def fineFactor (w h : Fraction) : Fraction := Fraction.mul (kappa w h) (kappa w h)

private def localA (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul h h) w
private def localC (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul (Fraction.mul h h) h) (Fraction.mul w w)

/-- The actual fine-minus-coarse state displacement, as represented rational
values. The first component is the position error, the second the velocity
error. -/
theorem local_error_identity (w h : Fraction) (s : Point × Point) :
  stateEquiv (stateSub (HarmonicRefinement.fine w h s) (HarmonicRefinement.coarse w h s))
    (pointScale (negF (localA w h)) (middle w h s),

```



```

    pointAdd (pointScale (localA w h) s.2)
    (pointScale (localC w h) (middle w h s))) := by
constructor <=> constructor <=>
  simp only [stateSub, localA, localC, HarmonicRefinement.fine,
    HarmonicRefinement.coarse, middle,
    pointEquiv, pointSub, pointNeg, cell, linearField, negF,
    pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
ac_nf <=> omega

private theorem localA_abs (w h : Fraction) :
  Fraction.equiv (localA w h).abs
  (Fraction.mul (Fraction.mul h.abs h.abs) w.abs) := by
  simp only [localA, Fraction.equiv, Fraction.abs, Fraction.mul,
    Int.natAbs_mul, Int.ofNat_mul]

private theorem neg_localA_abs (w h : Fraction) :
  Fraction.equiv (negF (localA w h)).abs
  (Fraction.mul (Fraction.mul h.abs h.abs) w.abs) :=
  Fraction.equiv_trans (Fraction.abs_neg (localA w h)) (localA_abs w h)

private theorem localC_abs (w h : Fraction) :
  Fraction.equiv (localC w h).abs
  (Fraction.mul (Fraction.mul (Fraction.mul h.abs h.abs) w.abs)
    (Fraction.mul h.abs w.abs)) := by
  simp only [localC, Fraction.equiv, Fraction.abs, Fraction.mul,
    Int.natAbs_mul, Int.ofNat_mul]
ac_nf

private theorem middle_norm_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (pointNorm (middle w h s))
  (Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (point_le_state (drift h s)) (drift_bound h s)

private def amplitude (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul h.abs h.abs) w.abs

private def kickMagnitude (w h : Fraction) : Fraction := Fraction.mul h.abs w.abs

private theorem amplitude_nonnegative (w h : Fraction) :
  0 ≤ (amplitude w h).num :=
  Fraction.nonnegative_mul _ _
  (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
  (Fraction.abs_num_nonnegative _)

private theorem kickMagnitude_nonnegative (w h : Fraction) :
  0 ≤ (kickMagnitude w h).num :=
  Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _)

/-- Triangle and scaling estimate for the explicit local mismatch. -/
theorem local_error_expanded_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.coarse w h s)))
    (Fraction.add
      (Fraction.mul (amplitude w h) (pointNorm (middle w h s)))
      (Fraction.add
        (Fraction.mul (amplitude w h) (pointNorm s.2))
        (Fraction.mul (amplitude w h) (kickMagnitude w h)
          (pointNorm (middle w h s))))) := by
  let y := middle w h s
  let a := pointScale (negF (localA w h)) y
  let b := pointScale (localA w h) s.2
  let c := pointScale (localC w h) y
  have he := stateNorm_equiv (local_error_identity w h s)
  have ht := Fraction.add_le_add_left (pointNorm_add_le b c) (pointNorm a)
  have hp : Fraction.equiv (pointNorm a)
    (Fraction.mul (amplitude w h) (pointNorm y)) :=
  Fraction.equiv_trans (pointNorm_scale _ _)
    (Fraction.mul_equiv (neg_localA_abs w h) (Fraction.equiv_refl _))

```

```

have hv : Fraction.equiv (pointNorm b)
  (Fraction.mul (amplitude w h) (pointNorm s.2)) :=
  Fraction.equiv_trans (pointNorm_scale _ _)
    (Fraction.mul_equiv (localA_abs w h) (Fraction.equiv_refl _))
have hc : Fraction.equiv (pointNorm c)
  (Fraction.mul (Fraction.mul (amplitude w h) (kickMagnitude w h))
    (pointNorm y)) :=
  Fraction.equiv_trans (pointNorm_scale _ _)
    (Fraction.mul_equiv (localC_abs w h) (Fraction.equiv_refl _))
exact Fraction.le_equiv_left he
  (Fraction.le_equiv_right ht (Fraction.add_equiv hp (Fraction.add_equiv hv hc)))

/-- The local defect of two actual half-cells against one full cell is bounded
by  $|h|^2|w|(\kappa+1)$  times the current state magnitude. -/
theorem local_error_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.coarse w h s)))
    (Fraction.mul (localFactor w h) (stateNorm s)) := by
  have h₀ := local_error_expanded_bound w h s
  have hy := middle_norm_bound w h s
  have hv := velocity_le_state s
  have ha := amplitude_nonnegative w h
  have hat := Fraction.nonnegative_mul (amplitude w h) (kickMagnitude w h) ha
    (kickMagnitude_nonnegative w h)
  have h₁ := Fraction.mul_le_mul_nonnegative_left hy (amplitude w h) ha
  have h₂ := Fraction.mul_le_mul_nonnegative_left hv (amplitude w h) ha
  have h₃ := Fraction.mul_le_mul_nonnegative_left hy
    (Fraction.mul (amplitude w h) (kickMagnitude w h)) hat
  have hs := Fraction.add_le_add h₁ (Fraction.add_le_add h₂ h₃)
  have hchain := Fraction.magnitudes.le_trans h₀ hs
  apply Fraction.le_equiv_right hchain
  simp only [localFactor, amplitude, kickMagnitude, kappa, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

/-- The two actual endpoint recurrences, each block of duration  $h+h$ . -/
def coarseAt (w h : Fraction) (s : Point × Point) : Nat → Point × Point
| 0 => s
| n + 1 => HarmonicRefinement.coarse w h (coarseAt w h s n)

def fineAt (w h : Fraction) (s : Point × Point) : Nat → Point × Point
| 0 => s
| n + 1 => HarmonicRefinement.fine w h (fineAt w h s n)

def fineDurations (h : Fraction) : Nat → List Fraction
| 0 => []
| n + 1 => h :: h :: fineDurations h n

private theorem coarseAt_comm (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    coarseAt w h (HarmonicRefinement.coarse w h s) n =
    HarmonicRefinement.coarse w h (coarseAt w h s n)
| 0 => rfl
| n + 1 => by
  simp only [coarseAt]
  rw [coarseAt_comm w h s n]

private theorem fineAt_comm (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    fineAt w h (HarmonicRefinement.fine w h s) n =
    HarmonicRefinement.fine w h (fineAt w h s n)
| 0 => rfl
| n + 1 => by
  simp only [fineAt]
  rw [fineAt_comm w h s n]

/-- The coarse recurrence is the actual list schedule of  $n$  full cells. -/

```

```

theorem coarseAt_schedule (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    schedule (linearField w) (List.replicate n (Fraction.add h h)) s =
      coarseAt w h s n
| 0 => rfl
| n + 1 => by
  simp only [List.replicate_succ, schedule]
  change schedule (linearField w) (List.replicate n (Fraction.add h h))
    (HarmonicRefinement.coarse w h s) = coarseAt w h s (n + 1)
  rw [coarseAt_schedule w h (HarmonicRefinement.coarse w h s) n]
  exact coarseAt_comm w h s n

/-- The fine recurrence is the actual schedule of `2n` half-cells. -/
theorem fineAt_schedule (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    schedule (linearField w) (fineDurations h n) s = fineAt w h s n
| 0 => rfl
| n + 1 => by
  simp only [fineDurations, schedule]
  change schedule (linearField w) (fineDurations h n)
    (HarmonicRefinement.fine w h s) = fineAt w h s (n + 1)
  rw [fineAt_schedule w h (HarmonicRefinement.fine w h s) n]
  exact fineAt_comm w h s n

/-- Both actual lists have the same represented elapsed duration. -/
theorem schedules_common_time (w h : Fraction) :
  (n : Nat) →
    Fraction.equiv
      (elapsed (List.replicate n (Fraction.add h h)))
      (elapsed (fineDurations h n))
| 0 => Fraction.equiv_refl _
| n + 1 => by
  simp only [List.replicate_succ, fineDurations, elapsed]
  exact Fraction.equiv_trans
    (Fraction.add_assoc h h (elapsed (List.replicate n (Fraction.add h h))))
    (Fraction.add_equiv (Fraction.equiv_refl h))
    (Fraction.add_equiv (Fraction.equiv_refl h) (schedules_common_time w h n))

def errorBudget (w h : Fraction) (s : Point × Point) : Nat → Fraction
| 0 => Fraction.ofInt 0
| n + 1 =>
  Fraction.add (Fraction.mul (fineFactor w h) (errorBudget w h s n))
    (Fraction.mul (Fraction.mul (localFactor w h) (fpower (coarseFactor w h) n))
      (stateNorm s))

theorem kappa_nonnegative (w h : Fraction) : 0 ≤ (kappa w h).num := by
  unfold kappa
  apply Fraction.nonnegative_mul
  · exact Fraction.nonnegative_add _ _ (by decide) (Fraction.abs_num_nonnegative _)
  · exact Fraction.nonnegative_add _ _ (by decide) (kickMagnitude_nonnegative w h)

theorem localFactor_nonnegative (w h : Fraction) :
  0 ≤ (localFactor w h).num :=
  Fraction.nonnegative_mul _ _ (amplitude_nonnegative w h)
  (Fraction.nonnegative_add _ _ (kappa_nonnegative w h) (by decide))

theorem fineFactor_nonnegative (w h : Fraction) :
  0 ≤ (fineFactor w h).num :=
  Fraction.nonnegative_mul _ _ (kappa_nonnegative w h) (kappa_nonnegative w h)

private theorem coarseFactor_nonnegative (w h : Fraction) :
  0 ≤ (coarseFactor w h).num := kappa_nonnegative w (Fraction.add h h)

theorem errorBudget_nonnegative (w h : Fraction) (s : Point × Point) :
  (n : Nat) → 0 ≤ (errorBudget w h s n).num
| 0 => by simp [errorBudget, Fraction.ofInt]
| n + 1 =>
  Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (fineFactor_nonnegative w h)
      (errorBudget_nonnegative w h s n))

```

```

(Fraction.nonnegative_mul _ _
 (Fraction.nonnegative_mul _ _ (localFactor_nonnegative w h)
  (fpower_nonnegative _ (coarseFactor_nonnegative w h) n))
 (stateNorm_nonnegative s))

/-- Two actual fine cells carry an input perturbation by at most `kappa^2`. -/
theorem fine_perturbation (w h : Fraction) (s t : Point × Point) :
  Fraction.le
    (stateNorm (stateSub (HarmonicRefinement.fine w h s)
      (HarmonicRefinement.fine w h t)))
    (Fraction.mul (fineFactor w h) (stateNorm (stateSub s t))) := by
  have h1 := cell_perturbation w h (cell (linearField w) h s)
  (cell (linearField w) h t)
  have h2 := cell_perturbation w h s t
  have hm := Fraction.mul_le_mul_nonnegative_left h2 (kappa w h)
  (kappa_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans h1 hm
  apply Fraction.le_equiv_right hc
  simp only [fineFactor, Fraction.equiv, Fraction.mul]
  ac_nf

/-- Every actual coarse state is bounded by `b^n` times the initial state
magnitude, with `b = kappa(w,h+h)`. -/
theorem coarse_norm_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (coarseAt w h s n))
      (Fraction.mul (fpower (coarseFactor w h) n) (stateNorm s))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [coarseAt, fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [Int.one_mul, Int.mul_one]
| n + 1 => by
  have hc := cell_bound w (Fraction.add h h) (coarseAt w h s n)
  have hi := coarse_norm_bound w h s n
  have hm := Fraction.mul_le_mul_nonnegative_left hi (coarseFactor w h)
  (coarseFactor_nonnegative w h)
  have hchain := Fraction.magnitudes.le_trans hc hm
  apply Fraction.le_equiv_right hchain
  simp only [coarseAt, fpower, coarseFactor, Fraction.equiv, Fraction.mul]
  ac_nf

/-- The actual endpoints after `n` common blocks obey the recursively
constructed finite error budget. The recurrence uses the coarse state at each
block for the local defect and propagates the previous actual endpoint error
through two actual fine cells. -/
theorem actual_error_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (stateSub (fineAt w h s n) (coarseAt w h s n)))
      (errorBudget w h s n)
| 0 => Fraction.le_of_equiv (stateSub_self_norm_zero s)
| n + 1 => by
  let F := fineAt w h s n
  let C := coarseAt w h s n
  have ht := stateSub_triangle
    (HarmonicRefinement.fine w h F)
    (HarmonicRefinement.fine w h C)
    (HarmonicRefinement.coarse w h C)
  have hp := fine_perturbation w h F C
  have hl := local_error_bound w h C
  have hraw := Fraction.magnitudes.le_trans ht (Fraction.add_le_add hp hl)
  have hi := actual_error_bound w h s n
  have hc := coarse_norm_bound w h s n
  have hbi := Fraction.mul_le_mul_nonnegative_left hi (fineFactor w h)
  (fineFactor_nonnegative w h)
  have hbc := Fraction.mul_le_mul_nonnegative_left hc (localFactor w h)
  (localFactor_nonnegative w h)
  have hbudget := Fraction.add_le_add hbi hbc
  have hchain := Fraction.magnitudes.le_trans hraw hbudget
  apply Fraction.le_equiv_right hchain
  simp only [errorBudget, Fraction.equiv, Fraction.add, Fraction.mul]

```

```

    simp only [Int.add_mul, Int.mul_add]
    ac_nf

private def one : Fraction := {1, 1, by decide}
private def zero : Fraction := {0, 1, by decide}
private def half : Fraction := {1, 2, by decide}
private def sample : Point × Point := ((one, zero), (zero, one))

private instance (s t : Point × Point) : Decidable (stateEquiv s t) :=
  inferInstanceAs (Decidable (pointEquiv s.1 t.1 ∧ pointEquiv s.2 t.2))

theorem sample_zero_blocks :
  stateEquiv (fineAt one zero sample 1) (coarseAt one zero sample 1) := by decide

theorem sample_zero_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one zero sample 1) (coarseAt one zero sample 1)))
    zero := by decide

theorem sample_zero_budget :
  Fraction.equiv (errorBudget one zero sample 1) zero := by decide

theorem sample_initial_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 0) (coarseAt one half sample 0)))
    zero := by decide

theorem sample_local_factor :
  Fraction.equiv (localFactor one half) {13, 16, by decide} := by decide

/-- Exact one-block state error. Its four component magnitudes are `1/4`,
`1/8`, `1/8`, and `5/16`; their sum is `13/16`. -/
theorem sample_one_block_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 1) (coarseAt one half sample 1)))
    {13, 16, by decide} := by decide

theorem sample_one_block_budget :
  Fraction.equiv (errorBudget one half sample 1) {13, 8, by decide} := by decide

theorem sample_two_block_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one half sample 2) (coarseAt one half sample 2)))
    {173, 256, by decide} := by decide

theorem sample_one_block_distinct :
  ~ stateEquiv (fineAt one half sample 1) (coarseAt one half sample 1) := by decide

end NewtonLimitDynamics.Polygon.HarmonicAccumulation

```

NewtonLimitDynamics/Polygon/HarmonicUniform.lean, 50 theorems, 678 lines

```

import BarrowLib.Polygon.IntegerSchedule
import NewtonLimitDynamics.Polygon.HarmonicAccumulation
import NewtonLimitDynamics.Common.FiniteGrowth

/-!
Mesh-uniform finite bounds for the actual harmonic coarse and fine schedules
under a small total-time condition. These coordinate L1 estimates provide
finite construction support only: no curve, completion, geometric region
between paths, or historical limiting step is constructed here.
The small-time threshold uses the chosen coordinate/unit calibration; it is
not a universal physical time bound.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicUniform

open NewtonLimitDynamics
open NewtonLimitDynamics.FiniteGrowth
open TimeSubdivision
open CentralSchedule

```

```

open HarmonicStability
open HarmonicRefinement
open HarmonicComparison
open HarmonicAccumulation
open PointBounds

private def halfThreshold : Fraction := (1, 2, by decide)
private def denom (w h : Fraction) : Int := h.den * w.den
private def driftIncrement (w h : Fraction) : Int := h.num * w.den
private def kickIncrement (w h : Fraction) : Int := h.num * (w.num.natAbs : Int)

private theorem denom_pos (w h : Fraction) : 0 < denom w h :=
  Int.mul_pos h.den_pos w.den_pos

private theorem driftIncrement_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  0 ≤ driftIncrement w h :=
  Int.mul_nonneg hh (Int.le_of_lt w.den_pos)

private theorem kickIncrement_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  0 ≤ kickIncrement w h :=
  Int.mul_nonneg hh (Int.ofNat_nonneg _)

/-- `n` full cells have factors `(D+2A)/D` and `(D+2B)/D`. -/
def coarseWeights (w h : Fraction) : Nat → List Int
| 0 => []
| n + 1 => (2 * driftIncrement w h) :: (2 * kickIncrement w h) ::
  coarseWeights w h n

/-- `2n` half-cells have four factors per common block. -/
def fineWeights (w h : Fraction) : Nat → List Int
| 0 => []
| n + 1 => driftIncrement w h :: kickIncrement w h ::
  driftIncrement w h :: kickIncrement w h :: fineWeights w h n

private theorem coarseWeights_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) → Nonnegative (coarseWeights w h n)
| 0 => by simp [coarseWeights, Nonnegative]
| n + 1 => by
  simp only [coarseWeights, Nonnegative, List.mem_cons]
  intro a ha
  rcases ha with rfl | rfl | ht
  · exact Int.mul_nonneg (by decide) (driftIncrement_nonnegative w h hh)
  · exact Int.mul_nonneg (by decide) (kickIncrement_nonnegative w h hh)
  · exact coarseWeights_nonnegative w h hh n a ht

private theorem fineWeights_nonnegative (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) → Nonnegative (fineWeights w h n)
| 0 => by simp [fineWeights, Nonnegative]
| n + 1 => by
  simp only [fineWeights, Nonnegative, List.mem_cons]
  intro a ha
  rcases ha with rfl | rfl | rfl | rfl | ht
  · exact driftIncrement_nonnegative w h hh
  · exact kickIncrement_nonnegative w h hh
  · exact driftIncrement_nonnegative w h hh
  · exact kickIncrement_nonnegative w h hh
  · exact fineWeights_nonnegative w h hh n a ht

private theorem common_weight_sum (w h : Fraction) :
  (n : Nat) →
  weightSum (coarseWeights w h n) =
  weightSum (fineWeights w h n) ∧
  weightSum (fineWeights w h n) =
  2 * (n : Int) * (driftIncrement w h + kickIncrement w h)
| 0 => by simp [coarseWeights, fineWeights, weightSum]
| n + 1 => by
  obtain {hc, hf} := common_weight_sum w h n
  simp only [coarseWeights, fineWeights, weightSum, Int.natCast_add,
    Int.natCast_one]
  constructor

```

```

    · rw [hc]
      omega
    · rw [hf]
      simp only [Int.add_mul, Int.mul_add]
      omega

def SmallTime (w h : Fraction) (n : Nat) : Prop :=
  Fraction.le
    (Fraction.mul (totalTime h n) (Fraction.add (Fraction.ofInt 1) w.abs)) halfThreshold

/-- The displayed total time is the elapsed time of the actual coarse list. -/
theorem coarse_elapsed_totalTime (h : Fraction) (n : Nat) :
  Fraction.equiv (elapsed (List.replicate n (Fraction.add h h)))
    (totalTime h n) := by
  induction n with
  | zero =>
    simp only [List.replicate_zero, elapsed, totalTime, Fraction.equiv,
      Fraction.mul, Fraction.ofInt]
    simp
  | succ n ih =>
    simp only [List.replicate_succ, elapsed]
    have ht := Fraction.equiv_trans
      (Fraction.add_comm (Fraction.add h h)
        (elapsed (List.replicate n (Fraction.add h h))))
      (Fraction.add_equiv_right (Fraction.add h h) ih)
    apply Fraction.equiv_trans ht
    simp only [totalTime, Fraction.equiv, Fraction.add, Fraction.mul,
      Fraction.ofInt, Int.natCast_add, Int.natCast_one,
      Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    simp only [show (2 : Int) = 1 + 1 by rfl,
      Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    ac_nf

/-- The fine list reaches the same displayed total time. -/
theorem fine_elapsed_totalTime (h : Fraction) (n : Nat) :
  Fraction.equiv (elapsed (fineDurations h n)) (totalTime h n) :=
  Fraction.equiv_trans
    (Fraction.equiv_symm (schedules_common_time (Fraction.ofInt 0) h n))
    (coarse_elapsed_totalTime h n)

private theorem smallTime_integer (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
  2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤ denom w h := by
  unfold SmallTime totalTime halfThreshold Fraction.le Fraction.mul Fraction.add
  Fraction.ofInt Fraction.abs at hs
  dsimp at hs
  simp only [Int.mul_one, Int.one_mul, Int.add_zero] at hs
  change 2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤
    h.den * w.den
  calc
    2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) =
      2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int)) * 2 := by ac_rfl
    _ ≤ h.den * w.den := hs

/-- The small-time condition is inherited by every earlier actual block. -/
theorem smallTime_prefix (w h : Fraction) (i n : Nat)
  (hh : 0 ≤ h.num) (hi : i ≤ n) (hs : SmallTime w h n) :
  SmallTime w h i := by
  have hn := smallTime_integer w h n hs
  have hcoef : 0 ≤ 4 * h.num * (w.den + (w.num.natAbs : Int)) := by
    exact Int.mul_nonneg
      (Int.mul_nonneg (by decide) hh)
      (Int.add_nonneg (Int.le_of_lt w.den_pos) (Int.ofNat_nonneg _))
  have hcast : (i : Int) ≤ (n : Int) := Int.ofNat_le.mpr hi
  have hm := Int.mul_le_mul_of_nonneg_right hcast hcoef
  have hpref :
    2 * (2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int))) ≤
      2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by
    calc
      _ = (i : Int) * (4 * h.num * (w.den + (w.num.natAbs : Int))) := by

```

```

    simp only [show (4 : Int) = 2 * 2 by rfl]
    ac_rfl
  _ ≤ (n : Int) * (4 * h.num * (w.den + (w.num.natAbs : Int))) := hm
  _ := by
    simp only [show (4 : Int) = 2 * 2 by rfl]
    ac_rfl
unfold SmallTime totalTime halfThreshold Fraction.le Fraction.mul Fraction.add
Fraction.ofInt Fraction.abs
dsimp
simp only [Int.mul_one, Int.one_mul, Int.add_zero]
calc
  2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int)) * 2 =
    2 * (2 * (i : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by ac_rfl
  _ ≤ 2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := hpref
  _ ≤ denom w h := hn

private theorem fineWeights_small (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
  2 * weightSum (fineWeights w h n) ≤ denom w h := by
  have hi := smallTime_integer w h n hs
  have hf := (common_weight_sum w h n).2
  calc
    2 * weightSum (fineWeights w h n) =
      2 * (2 * (n : Int) * h.num * (w.den + (w.num.natAbs : Int))) := by
        rw [hf]
        simp only [driftIncrement, kickIncrement, Int.mul_add]
        ac_nf
    _ ≤ denom w h := hi

private theorem coarseWeights_small (w h : Fraction) (n : Nat)
  (hs : SmallTime w h n) :
  2 * weightSum (coarseWeights w h n) ≤ denom w h := by
  rw [(common_weight_sum w h n).1]
  exact fineWeights_small w h n hs

theorem two_mul (x : Int) : 2 * x = x + x := by omega

private theorem coarse_block_equiv (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.equiv
    (amplification (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h])
    (coarseFactor w h) := by
  have hsum : 0 ≤ (Fraction.add h h).num := by
  unfold Fraction.add
  exact Int.add_nonneg
    (Int.mul_nonneg hh (Int.le_of_lt h.den_pos))
    (Int.mul_nonneg hh (Int.le_of_lt h.den_pos))
  simp only [amplification, factorProduct, denom, driftIncrement,
    kickIncrement, coarseFactor, kappa, Fraction.equiv, Fraction.abs,
    Fraction.add, Fraction.mul, Fraction.ofInt,
    List.length_cons, List.length_nil, Int.pow_succ,
    Int.pow_zero, Int.mul_one, Int.one_mul]
  change 0 ≤ h.num * h.den + h.num * h.den at hsum
  rw [Int.natAbs_of_nonneg hsum]
  simp only [two_mul, Int.add_mul, Int.mul_add]
  ac_nf

private theorem fine_block_equiv (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.equiv
    (amplification (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h])
    (fineFactor w h) := by
  simp only [amplification, factorProduct, denom, driftIncrement,
    kickIncrement, fineFactor, kappa, Fraction.equiv, Fraction.abs,
    Fraction.add, Fraction.mul, Fraction.ofInt,
    List.length_cons, List.length_nil, Int.pow_succ,
    Int.pow_zero, Int.mul_one, Int.one_mul,
    Int.natAbs_of_nonneg hh]
  simp only [Int.add_mul, Int.mul_add]

```



```

ac_nf

private theorem coarse_power_amplification (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) →
    Fraction.equiv
      (amplification (denom w h) (denom_pos w h) (coarseWeights w h n))
      (fpower (coarseFactor w h) n)
  | 0 => amplification_empty _ _
  | n + 1 => by
    have ha := amplification_append (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h] (coarseWeights w h n)
    have hb := coarse_block_equiv w h hh
    have hi := coarse_power_amplification w h hh n
    change Fraction.equiv
      (amplification (denom w h) (denom_pos w h)
        ([2 * driftIncrement w h, 2 * kickIncrement w h] ++ coarseWeights w h n))
      (Fraction.mul (coarseFactor w h) (fpower (coarseFactor w h) n))
    exact Fraction.equiv_trans ha (Fraction.mul_equiv hb hi)

private theorem fine_power_amplification (w h : Fraction) (hh : 0 ≤ h.num) :
  (n : Nat) →
    Fraction.equiv
      (amplification (denom w h) (denom_pos w h) (fineWeights w h n))
      (fpower (fineFactor w h) n)
  | 0 => amplification_empty _ _
  | n + 1 => by
    have ha := amplification_append (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h] (fineWeights w h n)
    have hb := fine_block_equiv w h hh
    have hi := fine_power_amplification w h hh n
    change Fraction.equiv
      (amplification (denom w h) (denom_pos w h)
        ([driftIncrement w h, kickIncrement w h,
          driftIncrement w h, kickIncrement w h] ++ fineWeights w h n))
      (Fraction.mul (fineFactor w h) (fpower (fineFactor w h) n))
    exact Fraction.equiv_trans ha (Fraction.mul_equiv hb hi)

/-- Uniform finite growth of the actual coarse amplification power. -/
theorem coarse_power_le_two (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (fpower (coarseFactor w h) n) (Fraction.ofInt 2) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (coarse_power_amplification w h hh n))
    (uniform_amplification (denom w h) (denom_pos w h) (coarseWeights w h n)
      (coarseWeights_nonnegative w h hh n) (coarseWeights_small w h n hs))

/-- Uniform finite growth of the actual two-cell perturbation power. -/
theorem fine_power_le_two (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (fpower (fineFactor w h) n) (Fraction.ofInt 2) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (fine_power_amplification w h hh n))
    (uniform_amplification (denom w h) (denom_pos w h) (fineWeights w h n)
      (fineWeights_nonnegative w h hh n) (fineWeights_small w h n hs))

/-- Actual coarse states stay inside twice the initial coordinate magnitude. -/
theorem coarse_state_le_two (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (stateNorm (coarseAt w h s n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (coarse_norm_bound w h s n)
    (Fraction.mul_le_mul_nonnegative (coarse_power_le_two w h n hh hs)
      (stateNorm s) (stateNorm_nonnegative s))

private theorem fine_cell_bound (w h : Fraction) (s : Point × Point) :
  Fraction.le (stateNorm (HarmonicRefinement.fine w h s))
    (Fraction.mul (fineFactor w h) (stateNorm s)) := by
  have h₁ := cell_bound w h (cell (linearField w) h s)
  have h₂ := cell_bound w h s
  have hm := Fraction.mul_le_mul_nonnegative_left h₂ (kappa w h)
  (kappa_nonnegative w h)

```

```

have hc := Fraction.magnitudes.le_trans h1 hm
apply Fraction.le_equiv_right hc
simp only [fineFactor, Fraction.equiv, Fraction.mul]
ac_nf

theorem fine_norm_bound (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    Fraction.le (stateNorm (fineAt w h s n))
      (Fraction.mul (fpower (fineFactor w h) n) (stateNorm s))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [fineAt, fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [Int.one_mul, Int.mul_one]
| n + 1 => by
  have hf := fine_cell_bound w h (fineAt w h s n)
  have hi := fine_norm_bound w h s n
  have hm := Fraction.mul_le_mul_nonnegative_left hi (fineFactor w h)
    (fineFactor_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans hf hm
  apply Fraction.le_equiv_right hc
  simp only [fineAt, fpower, Fraction.equiv, Fraction.mul]
  ac_nf

theorem fine_state_le_two (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (stateNorm (fineAt w h s n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) :=
  Fraction.magnitudes.le_trans (fine_norm_bound w h s n)
    (Fraction.mul_le_mul_nonnegative (fine_power_le_two w h n hh hs)
      (stateNorm s) (stateNorm_nonnegative s))

private theorem square_dominates_double (D a : Int) (ha : 0 ≤ a) :
  D * (D + 2 * a) ≤ (D + a) * (D + a) := by
  have hp := Int.mul_nonneg ha ha
  have he : (D + a) * (D + a) = D * (D + 2 * a) + a * a := by
    simp only [two_mul, Int.add_mul, Int.mul_add]
  ac_nf
  omega

private theorem block_product_order (D A B : Int) (hD : 0 < D)
  (hA : 0 ≤ A) (hB : 0 ≤ B) :
  (D + 2 * A) * (D + 2 * B) * (D * D) ≤
  ((D + A) * (D + A)) * ((D + B) * (D + B)) := by
  have p := square_dominates_double D A hA
  have q := square_dominates_double D B hB
  have hp : 0 ≤ D * (D + 2 * A) :=
    Int.mul_nonneg (Int.le_of_lt hD)
      (Int.add_nonneg (Int.le_of_lt hD) (Int.mul_nonneg (by decide) hA))
  have hq : 0 ≤ (D + B) * (D + B) := by
    have hb : 0 ≤ D + B := Int.add_nonneg (Int.le_of_lt hD) hB
    exact Int.mul_nonneg hb hb
  have h1 := Int.mul_le_mul_of_nonneg_right p hq
  have h2 := Int.mul_le_mul_of_nonneg_left q hp
  calc
    (D + 2 * A) * (D + 2 * B) * (D * D) =
      (D * (D + 2 * A)) * (D * (D + 2 * B)) := by ac_rfl
  _ ≤ (D * (D + 2 * A)) * ((D + B) * (D + B)) := h2
  _ ≤ ((D + A) * (D + A)) * ((D + B) * (D + B)) := h1

private theorem block_amplification_order (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.le
    (amplification (denom w h) (denom_pos w h)
      [2 * driftIncrement w h, 2 * kickIncrement w h])
    (amplification (denom w h) (denom_pos w h)
      [driftIncrement w h, kickIncrement w h,
        driftIncrement w h, kickIncrement w h]) := by
  have ho := block_product_order (denom w h) (driftIncrement w h)
    (kickIncrement w h) (denom_pos w h)
    (driftIncrement_nonnegative w h hh) (kickIncrement_nonnegative w h hh)
  unfold Fraction.le amplification

```

```

simp only [factorProduct, List.length_cons, List.length_nil,
  Int.pow_succ, Int.pow_zero, Int.mul_one, Int.one_mul]
have hd : 0 ≤ denom w h * denom w h :=
  Int.mul_nonneg (Int.le_of_lt (denom_pos w h)) (Int.le_of_lt (denom_pos w h))
have hm := Int.mul_le_mul_of_nonneg_right ho hd
calc
  _ =
    ((denom w h + 2 * driftIncrement w h) *
      (denom w h + 2 * kickIncrement w h) *
      (denom w h * denom w h)) * (denom w h * denom w h) := by ac_rfl
  _ ≤ ((denom w h + driftIncrement w h) *
      (denom w h + driftIncrement w h) *
      ((denom w h + kickIncrement w h) *
        (denom w h + kickIncrement w h))) *
      (denom w h * denom w h) := hm
  _ = _ := by ac_rfl

theorem coarseFactor_le_fineFactor (w h : Fraction) (hh : 0 ≤ h.num) :
  Fraction.le (coarseFactor w h) (fineFactor w h) :=
  Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm (coarse_block_equiv w h hh))
      (block_amplification_order w h hh))
    (fine_block_equiv w h hh)

private theorem one_le_one_add (a : Fraction) (ha : 0 ≤ a.num) :
  Fraction.le (Fraction.ofInt 1) (Fraction.add (Fraction.ofInt 1) a) := by
  unfold Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hp := Int.le_of_lt a.den_pos
  omega

theorem one_le_kappa (w h : Fraction) :
  Fraction.le (Fraction.ofInt 1) (kappa w h) := by
  let u := Fraction.add (Fraction.ofInt 1) h.abs
  let v := Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)
  have hu := one_le_one_add h.abs (Fraction.abs_num_nonnegative h)
  have hv := one_le_one_add (Fraction.mul h.abs w.abs)
    (Int.mul_nonneg (Fraction.abs_num_nonnegative h) (Fraction.abs_num_nonnegative w))
  have h1 := Fraction.mul_le_mul_nonnegative hu (Fraction.ofInt 1) (by decide)
  have h2 := Fraction.mul_le_mul_nonnegative_left hv u (by
    unfold u Fraction.add Fraction.ofInt
    dsimp
    have hp := h.abs.den_pos
    have hn := Fraction.abs_num_nonnegative h
    omega)
  have hc := Fraction.magnitudes.le_trans h1 h2
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    decide) hc

private theorem one_le_fineFactor (w h : Fraction) :
  Fraction.le (Fraction.ofInt 1) (fineFactor w h) := by
  have hk := one_le_kappa w h
  have h1 := Fraction.mul_le_mul_nonnegative hk (Fraction.ofInt 1) (by decide)
  have h2 := Fraction.mul_le_mul_nonnegative_left hk (kappa w h)
    (kappa_nonnegative w h)
  have hc := Fraction.magnitudes.le_trans h1 h2
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    decide) hc

private theorem fpower_monotone (a b : Fraction) (ha : 0 ≤ a.num)
  (hb : 0 ≤ b.num) (hab : Fraction.le a b) :
  (n : Nat) → Fraction.le (fpower a n) (fpower b n)
| 0 => Fraction.magnitudes.le_refl _
| n + 1 => by
  have h1 := Fraction.mul_le_mul_nonnegative hab (fpower a n)
    (fpower_nonnegative a ha n)
  have h2 := Fraction.mul_le_mul_nonnegative_left
    (fpower_monotone a b ha hb hab n) b hb

```

```

    exact Fraction.magnitudes.le_trans h1 h2

private theorem coarse_power_le_fine_power (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) :
  Fraction.le (fpower (coarseFactor w h) n) (fpower (fineFactor w h) n) :=
  fpower_monotone _ (kappa_nonnegative w (Fraction.add h h))
  (fineFactor_nonnegative w h) (coarseFactor_le_fineFactor w h hh) n

private def count (n : Nat) : Fraction := Fraction.ofInt (n : Int)

private def budgetCap (w h : Fraction) (s : Point × Point) (n : Nat) : Fraction :=
  Fraction.mul (count n)
  (Fraction.mul (localFactor w h)
    (Fraction.mul (fpower (fineFactor w h) n) (stateNorm s)))

private theorem budget_power_bound (w h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) :
  (n : Nat) → Fraction.le (errorBudget w h s n) (budgetCap w h s n)
| 0 => by
  apply Fraction.le_of_equiv
  simp only [errorBudget, budgetCap, count, fpower, Fraction.equiv,
    Fraction.ofInt, Fraction.mul]
  simp
| n + 1 => by
  let d := localFactor w h
  let r := fineFactor w h
  let M := stateNorm s
  let X := Fraction.mul (Fraction.mul d (fpower r n)) M
  have h1 := budget_power_bound w h s hh n
  have hr := fineFactor_nonnegative w h
  have hd := localFactor_nonnegative w h
  have hpow := fpower_nonnegative r hr n
  have hX : 0 ≤ X.num :=
    Int.mul_nonneg (Int.mul_nonneg hd hpow) (stateNorm_nonnegative s)
  have h1 := Fraction.mul_le_mul_nonnegative_left h1 r hr
  have hp := coarse_power_le_fine_power w h n hh
  have h2a := Fraction.mul_le_mul_nonnegative_left hp d hd
  have h2 := Fraction.mul_le_mul_nonnegative h2a M (stateNorm_nonnegative s)
  have h2b : Fraction.le X
    (Fraction.mul d (Fraction.mul (fpower r (n + 1)) M)) := by
    have hk := Fraction.mul_le_mul_nonnegative (one_le_fineFactor w h) X hX
    have he : Fraction.equiv X (Fraction.mul (Fraction.ofInt 1) X) := by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
      simp
    have hk' := Fraction.le_equiv_left he hk
    apply Fraction.le_equiv_right hk'
    simp only [d, r, M, X, fpower, Fraction.equiv, Fraction.mul]
    ac_nf
  have h2c := Fraction.magnitudes.le_trans h2 h2b
  have hsum := Fraction.add_le_add h1 h2c
  apply Fraction.le_equiv_right hsum
  simp only [d, r, M, errorBudget, budgetCap, count, fpower,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.one_mul, Int.mul_one]
  ac_nf

private theorem budget_two_bound (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le (errorBudget w h s n)
  (Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (count n) (Fraction.mul (localFactor w h) (stateNorm s)))) := by
  have h0 := budget_power_bound w h s hh n
  have hp := fine_power_le_two w h n hh hs
  have h1 := Fraction.mul_le_mul_nonnegative hp (stateNorm s) (stateNorm_nonnegative s)
  have h2 := Fraction.mul_le_mul_nonnegative_left h1 (localFactor w h)
    (localFactor_nonnegative w h)
  have h3 := Fraction.mul_le_mul_nonnegative_left h2 (count n) (by
    unfold count Fraction.ofInt
    exact Int.ofNat_nonneg n)

```

```

have hc := Fraction.magnitudes.le_trans h₀ h₃
apply Fraction.le_equiv_right hc
simp only [budgetCap, count, Fraction.equiv, Fraction.mul, Fraction.ofInt]
ac_nf

theorem one_le_power (a : Fraction) (ha : 0 ≤ a.num)
  (h1 : Fraction.le (Fraction.ofInt 1) a) :
  (n : Nat) → Fraction.le (Fraction.ofInt 1) (fpower a n)
| 0 => Fraction.magnitudes.le_refl _
| n + 1 => by
  have hi := one_le_power a ha h1 n
  have hm := Fraction.mul_le_mul_nonnegative h1 (fpower a n)
    (fpower_nonnegative a ha n)
  have he : Fraction.equiv (fpower a n)
    (Fraction.mul (Fraction.ofInt 1) (fpower a n)) := by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
  exact Fraction.magnitudes.le_trans hi (Fraction.le_equiv_left he hm)

private theorem factor_le_power_succ (a : Fraction) (ha : 0 ≤ a.num)
  (h1 : Fraction.le (Fraction.ofInt 1) a) (n : Nat) :
  Fraction.le a (fpower a (n + 1)) := by
  have hp := one_le_power a ha h1 n
  have hm := Fraction.mul_le_mul_nonnegative_left hp a ha
  exact Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ) hm

private theorem kappa_le_fineFactor (w h : Fraction) :
  Fraction.le (kappa w h) (fineFactor w h) := by
  have hk := one_le_kappa w h
  have hm := Fraction.mul_le_mul_nonnegative_left hk (kappa w h)
    (kappa_nonnegative w h)
  exact Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ) hm

private theorem kappa_le_two_of_positive_blocks (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h (n + 1)) :
  Fraction.le (kappa w h) (Fraction.ofInt 2) := by
  have h₁ := kappa_le_fineFactor w h
  have h₂ := factor_le_power_succ (fineFactor w h)
    (fineFactor_nonnegative w h) (one_le_fineFactor w h) n
  have h₃ := fine_power_le_two w h (n + 1) hh hs
  exact Fraction.magnitudes.le_trans (Fraction.magnitudes.le_trans h₁ h₂) h₃

private def meshAmplitude (w h : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul h.abs h.abs) w.abs

private theorem meshAmplitude_nonnegative (w h : Fraction) :
  0 ≤ (meshAmplitude w h).num :=
  Int.mul_nonneg
  (Int.mul_nonneg (Fraction.abs_num_nonnegative h) (Fraction.abs_num_nonnegative h))
  (Fraction.abs_num_nonnegative w)

private theorem localFactor_le_three_amplitude (w h : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h (n + 1)) :
  Fraction.le (localFactor w h)
  (Fraction.mul (Fraction.ofInt 3) (meshAmplitude w h)) := by
  have hk := kappa_le_two_of_positive_blocks w h n hh hs
  have hplus := Fraction.add_le_add_right hk (Fraction.ofInt 1)
  have hm := Fraction.mul_le_mul_nonnegative hplus (meshAmplitude w h)
    (meshAmplitude_nonnegative w h)
  have hleft : Fraction.equiv (localFactor w h)
    (Fraction.mul (Fraction.add (kappa w h) (Fraction.ofInt 1))
    (meshAmplitude w h)) := by
    unfold localFactor meshAmplitude
    exact Fraction.mul_comm _ _
  have hright : Fraction.equiv
    (Fraction.mul (Fraction.add (Fraction.ofInt 2) (Fraction.ofInt 1))
    (meshAmplitude w h)) := by
    simp

```

```

    (meshAmplitude w h))
  (Fraction.mul (Fraction.ofInt 3) (meshAmplitude w h)) := by
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.mul_one, Int.one_mul]
  ac_nf
  exact Fraction.le_equiv_right (Fraction.le_equiv_left hleft hm) hright

/-- Uniform finite error at common total time `T=2nh`: actual fine and coarse
endpoints differ in coordinate L1 magnitude by at most `3*T*h*|w|*M`.
The hypothesis includes `h≥0` and `T*(1+|w|)≤1/2`. This is an endpoint
estimate, with no limiting curve or intervening-area assertion. -/
theorem actual_uniform_error (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hs : SmallTime w h n) :
  Fraction.le
    (stateNorm (stateSub (fineAt w h s n) (coarseAt w h s n)))
    (Fraction.mul (Fraction.ofInt 3)
      (Fraction.mul (totalTime h n)
        (Fraction.mul h (Fraction.mul w.abs (stateNorm s))))) := by
  cases n with
  | zero =>
    have h₀ := actual_error_bound w h s 0
    apply Fraction.le_equiv_right h₀
    simp only [errorBudget, totalTime, Fraction.equiv,
      Fraction.mul, Fraction.ofInt]
    simp
  | succ m =>
    have h₀ := actual_error_bound w h s (m + 1)
    have h₁ := budget_two_bound w h s (m + 1) hh hs
    have hδ := localFactor_le_three_amplitude w h m hh hs
    have h₂ := Fraction.mul_le_mul_nonnegative hδ (stateNorm s)
      (stateNorm_nonnegative s)
    have h₃ := Fraction.mul_le_mul_nonnegative_left h₂ (count (m + 1)) (by
      unfold count Fraction.ofInt
      exact Int.ofNat_nonneg _)
    have h₄ := Fraction.mul_le_mul_nonnegative_left h₃ (Fraction.ofInt 2) (by decide)
    have hchain := Fraction.magnitudes.le_trans
      (Fraction.magnitudes.le_trans h₀ h₁) h₄
    apply Fraction.le_equiv_right hchain
    simp only [meshAmplitude, count, totalTime, Fraction.equiv,
      Fraction.mul, Fraction.abs, Fraction.ofInt,
      Int.natAbs_of_nonneg hh]
    ac_nf

private def one : Fraction := (1, 1, by decide)
private def zero : Fraction := (0, 1, by decide)
private def eighth : Fraction := (1, 8, by decide)
private def sample : Point × Point := ((one, zero), (zero, one))

theorem sample_small_time : SmallTime one eighth 1 := by
  unfold SmallTime Fraction.le
  decide

theorem sample_total_time : Fraction.equiv (totalTime eighth 1) (1, 4, by decide) := by
  decide

theorem sample_power_bounds :
  Fraction.le (fpower (coarseFactor one eighth) 1) (Fraction.ofInt 2) ^
  Fraction.le (fpower (fineFactor one eighth) 1) (Fraction.ofInt 2) :=
  (coarse_power_le_two one eighth 1 (by decide) sample_small_time,
    fine_power_le_two one eighth 1 (by decide) sample_small_time)

theorem sample_actual_error :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one eighth sample 1) (coarseAt one eighth sample 1)))
    (145, 4096, by decide) := by decide

theorem sample_uniform_rhs :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt 3)
      (Fraction.mul (totalTime eighth 1)

```

```

      (Fraction.mul eighth (Fraction.mul one.abs (stateNorm sample))))
    (3, 16, by decide) := by decide

theorem sample_uniform_error :
  Fraction.le
    (stateNorm (stateSub (fineAt one eighth sample 1) (coarseAt one eighth sample 1)))
    (Fraction.mul (Fraction.ofInt 3)
      (Fraction.mul (totalTime eighth 1)
        (Fraction.mul eighth (Fraction.mul one.abs (stateNorm sample)))))) :=
  actual_uniform_error one eighth sample 1 (by decide) sample_small_time

theorem sample_zero_blocks :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one eighth sample 0) (coarseAt one eighth sample 0)))
    zero := by decide

theorem sample_zero_duration :
  Fraction.equiv
    (stateNorm (stateSub (fineAt one zero sample 1) (coarseAt one zero sample 1)))
    zero := by decide

/-- Without the total-time hypothesis, the claimed factor-two bound fails:
`w=h=n=1` gives `fineFactor^1=16`. -/
theorem false_unrestricted_power :
  ~ Fraction.le (fpower (fineFactor one one) 1) (Fraction.ofInt 2) := by
  unfold Fraction.le
  decide

end NewtonLimitDynamics.Polygon.HarmonicUniform

```

NewtonLimitDynamics/Polygon/HarmonicCover.lean, 36 theorems, 434 lines

```

import NewtonLimitDynamics.Polygon.ConvexCover
import BarrowLib.Polygon.SquareOuterContent
import NewtonLimitDynamics.Polygon.HarmonicUniform

/-!
Explicit finite square covers for matched pieces of the actual harmonic
coarse and fine polygonal paths. The square area is counted with multiplicity
over blocks. This is a geometric covering budget, not the area of the union
or the area between a polygon and a realized continuum trajectory. It is
separate from Kepler's centre-swept area.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicCover

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicRefinement
open HarmonicComparison
open HarmonicAccumulation
open HarmonicUniform
open PointBounds
open ConvexCover

private def two : Fraction := Fraction.ofInt 2
private def four : Fraction := Fraction.ofInt 4

def maxError (w h : Fraction) (s : Point × Point) (n : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul (totalTime h n)
      (Fraction.mul h (Fraction.mul w.abs (stateNorm s))))

def halfDriftBudget (h : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul two (Fraction.mul h (stateNorm s))

def fullDriftBudget (h : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul four (Fraction.mul h (stateNorm s))

```

```

/-- The radius is `4hM + Emax`; this additive form makes the corner estimates
direct. `radius_formula` gives the equivalent compact expression. -/
def radius (w h : Fraction) (s : Point × Point) (n : Nat) : Fraction :=
  Fraction.add (fullDriftBudget h s) (maxError w h s n)

def squareArea (R : Fraction) : Fraction := SquareOuterContent.squareArea R

/-- Sum of `n` square areas, counted with multiplicity. -/
def coverBudget (w h : Fraction) (s : Point × Point) (n : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (n : Int)) (squareArea (radius w h s n))

private theorem totalTime_nonnegative (h : Fraction) (n : Nat) (hh : 0 ≤ h.num) :
  0 ≤ (totalTime h n).num := by
  unfold totalTime Fraction.mul Fraction.ofInt
  exact Int.mul_nonneg (Int.mul_nonneg (by decide) (Int.ofNat_nonneg _)) hh

private theorem maxError_nonnegative (w h : Fraction) (s : Point × Point)
  (n : Nat) (hh : 0 ≤ h.num) : 0 ≤ (maxError w h s n).num :=
  Fraction.nonnegative_mul _ _ (by decide)
  (Fraction.nonnegative_mul _ _ (totalTime_nonnegative h n hh)
    (Fraction.nonnegative_mul _ _ hh
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))))

private theorem halfDriftBudget_nonnegative (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) : 0 ≤ (halfDriftBudget h s).num :=
  Fraction.nonnegative_mul _ _ (by decide) (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

private theorem fullDriftBudget_nonnegative (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) : 0 ≤ (fullDriftBudget h s).num :=
  Fraction.nonnegative_mul _ _ (by decide) (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

theorem radius_nonnegative (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) : 0 ≤ (radius w h s n).num :=
  Fraction.nonnegative_add _ _ (fullDriftBudget_nonnegative h s hh)
  (maxError_nonnegative w h s n hh)

theorem squareArea_nonnegative (R : Fraction) (hR : 0 ≤ R.num) :
  0 ≤ (squareArea R).num := SquareOuterContent.squareArea_nonnegative R hR

theorem coverBudget_nonnegative (w h : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) : 0 ≤ (coverBudget w h s n).num :=
  Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg _)
  (squareArea_nonnegative _ (radius_nonnegative w h s n hh))

private theorem totalTime_le (h : Fraction) (i n : Nat)
  (hh : 0 ≤ h.num) (hin : i ≤ n) :
  Fraction.le (totalTime h i) (totalTime h n) := by
  have hi : (i : Int) ≤ (n : Int) := Int.ofNat_le.mpr hin
  have hcoef : 0 ≤ 2 * h.num * h.den :=
    Int.mul_nonneg (Int.mul_nonneg (by decide) hh) (Int.le_of_lt h.den_pos)
  have hm := Int.mul_le_mul_of_nonneg_right hi hcoef
  unfold Fraction.le totalTime Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.one_mul]
  calc
    2 * (i : Int) * h.num * h.den ≤
      2 * (n : Int) * h.num * h.den := by
        calc
          _ = (i : Int) * (2 * h.num * h.den) := by ac_rfl
          _ ≤ (n : Int) * (2 * h.num * h.den) := hm
          _ = _ := by ac_rfl

/-- Every earlier actual endpoint error is bounded by the final-count cap. -/
private theorem prefix_error_le_max (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le
    (stateNorm (stateSub (fineAt w h s i) (coarseAt w h s i)))
    (maxError w h s n) := by
  have hsmall := smallTime_prefix w h i n hh hin hs

```



```

have he := actual_uniform_error w h s i hh hsmall
have ht := totalTime_le h i n hh hin
have hfactor : 0 ≤ (Fraction.mul h (Fraction.mul w.abs (stateNorm s))).num :=
  Fraction.nonnegative_mul _ _ hh
  (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))
have hm := Fraction.mul_le_mul_nonnegative ht
  (Fraction.mul h (Fraction.mul w.abs (stateNorm s))) hfactor
have hm' := Fraction.mul_le_mul_nonnegative_left hm (Fraction.ofInt 3) (by decide)
exact Fraction.magnitudes.le_trans he hm'

private theorem drift_offset_le_state (d : Fraction) (hd : 0 ≤ d.num)
  (t : Point × Point) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale d t.2)) t.1))
    (Fraction.mul d (stateNorm t)) := by
  have he := pointNorm_equiv (drift_offset d t.1 t.2)
  have hs := pointNorm_scale d t.2
  have hdabs := Fraction.abs_of_nonnegative d hd
  have hmul := Fraction.mul_equiv hdabs (Fraction.equiv_refl (pointNorm t.2))
  have hstart := Fraction.equiv_trans he (Fraction.equiv_trans hs hmul)
  have hv := velocity_le_state t
  have hm := Fraction.mul_le_mul_nonnegative_left hv d hd
  exact Fraction.le_equiv_left hstart hm

private theorem two_le_four (h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) :
  Fraction.le (halfDriftBudget h s) (fullDriftBudget h s) := by
  have ht : Fraction.le two four := by
    unfold Fraction.le two four Fraction.ofInt
    decide
  exact Fraction.mul_le_mul_nonnegative ht (Fraction.mul h (stateNorm s))
  (Fraction.nonnegative_mul _ _ hh (stateNorm_nonnegative s))

private theorem half_drift_le (h : Fraction) (s t : Point × Point)
  (hh : 0 ≤ h.num)
  (ht : Fraction.le (stateNorm t) (Fraction.mul two (stateNorm s))) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale h t.2)) t.1))
    (halfDriftBudget h s) := by
  have h₀ := drift_offset_le_state h hh t
  have h₁ := Fraction.mul_le_mul_nonnegative_left ht h hh
  have hc := Fraction.magnitudes.le_trans h₀ h₁
  apply Fraction.le_equiv_right hc
  simp only [halfDriftBudget, two, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

private theorem full_drift_le (h : Fraction) (s t : Point × Point)
  (hh : 0 ≤ h.num)
  (ht : Fraction.le (stateNorm t) (Fraction.mul two (stateNorm s))) :
  Fraction.le
    (pointNorm (pointSub (pointAdd t.1 (pointScale (Fraction.add h h) t.2)) t.1))
    (fullDriftBudget h s) := by
  have hsum : 0 ≤ (Fraction.add h h).num :=
    Fraction.nonnegative_add h h hh hh
  have h₀ := drift_offset_le_state (Fraction.add h h) hsum t
  have h₁ := Fraction.mul_le_mul_nonnegative_left ht (Fraction.add h h) hsum
  have hc := Fraction.magnitudes.le_trans h₀ h₁
  apply Fraction.le_equiv_right hc
  simp only [fullDriftBudget, two, four, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt]
  simp only [show (4 : Int) = 2 + 2 by rfl,
    Int.add_mul, Int.mul_add]
  ac_nf

def coarseStart (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (coarseAt w h s i).1

def fineStart (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (fineAt w h s i).1

```

```

/-- Position halfway along the actual coarse cell's inertial drift. This
vertex is a comparison subdivision; the coarse schedule has no impulse here. -/
def coarseMid (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (cell (linearField w) h (coarseAt w h s i)).1

def fineMid (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (cell (linearField w) h (fineAt w h s i)).1

def coarseEnd (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (coarseAt w h s (i + 1)).1

def fineEnd (w h : Fraction) (s : Point × Point) (i : Nat) : Point :=
  (fineAt w h s (i + 1)).1

private theorem coarse_mid_le_half (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (coarseMid w h s i) (coarseStart w h s i)))
    (halfDriftBudget h s) :=
  half_drift_le h s (coarseAt w h s i) hh
  (coarse_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem fine_mid_own_le_half (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineMid w h s i) (fineStart w h s i)))
    (halfDriftBudget h s) :=
  half_drift_le h s (fineAt w h s i) hh
  (fine_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem coarse_end_le_full (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (coarseEnd w h s i) (coarseStart w h s i)))
    (fullDriftBudget h s) :=
  full_drift_le h s (coarseAt w h s i) hh
  (coarse_state_le_two w h s i hh (smallTime_prefix w h i n hh hin hs))

private theorem fine_start_error_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineStart w h s i) (coarseStart w h s i)))
    (maxError w h s n) :=
  Fraction.magnitudes.le_trans
    (point_le_state (stateSub (fineAt w h s i) (coarseAt w h s i)))
    (prefix_error_le_max w h s i n hh hin hs)

private theorem fine_mid_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i ≤ n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineMid w h s i) (coarseStart w h s i)))
    (Fraction.add (maxError w h s n) (halfDriftBudget h s)) := by
  have ht := pointSub_triangle (fineMid w h s i) (fineStart w h s i)
    (coarseStart w h s i)
  have h1 := fine_mid_own_le_half w h s i n hh hin hs
  have h2 := fine_start_error_le w h s i n hh hin hs
  have hc := Fraction.magnitudes.le_trans ht (Fraction.add_le_add h1 h2)
  exact Fraction.le_equiv_right hc
  (Fraction.add_comm (halfDriftBudget h s) (maxError w h s n))

private theorem fine_end_le (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) :
  Fraction.le (pointNorm (pointSub (fineEnd w h s i) (coarseStart w h s i)))
    (Fraction.add (maxError w h s n) (fullDriftBudget h s)) := by
  have hnext : i + 1 ≤ n := Nat.succ_le_of_lt hin
  have ht := pointSub_triangle (fineEnd w h s i) (coarseEnd w h s i)
    (coarseStart w h s i)
  have h1 := fine_start_error_le w h s (i + 1) n hh hnext hs
  have h2 := coarse_end_le_full w h s i n hh (Nat.le_of_lt hin) hs

```

```

exact Fraction.magnitudes.le_trans ht (Fraction.add_le_add h₁ h₂)

private theorem zero_le (R : Fraction) (hR : 0 ≤ R.num) :
  Fraction.le (Fraction.ofInt 0) R := by
  unfold Fraction.le Fraction.ofInt
  simpa using hR

/-- Six actual vertices, each measured from the coarse block start, fit in
the same coordinate L1 ball of radius `4hM + Emax`. The midpoint is only a
subdivision of the coarse drift; it receives no impulse. -/
theorem actual_corners_in_ball (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) :
  let x := coarseStart w h s i
  let R := radius w h s n
  Fraction.le (pointNorm (pointSub x x)) R ∧
  Fraction.le (pointNorm (pointSub (coarseMid w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (coarseEnd w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineStart w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineMid w h s i) x)) R ∧
  Fraction.le (pointNorm (pointSub (fineEnd w h s i) x)) R := by
  dsimp
  have hi : i ≤ n := Nat.le_of_lt hin
  have hE := maxError_nonnegative w h s n hh
  have hH := fullDriftBudget_nonnegative h s hh
  have hH2 := two_le_four h s hh
  have hHR : Fraction.le (fullDriftBudget h s) (radius w h s n) :=
    Fraction.le_add_nonnegative _ _ hE
  have hER : Fraction.le (maxError w h s n) (radius w h s n) :=
    Fraction.le_equiv_right (Fraction.le_add_nonnegative _ _ hH)
    (Fraction.add_comm (maxError w h s n) (fullDriftBudget h s))
  have hsum : Fraction.le
    (Fraction.add (maxError w h s n) (halfDriftBudget h s))
    (radius w h s n) :=
    Fraction.le_equiv_right
      (Fraction.add_le_add_left hH2 (maxError w h s n))
      (Fraction.add_comm (maxError w h s n) (fullDriftBudget h s))
  have hlast : Fraction.le
    (Fraction.add (maxError w h s n) (fullDriftBudget h s))
    (radius w h s n) :=
    Fraction.add_comm _ _
  refine (?, ?, ?, ?, ?)
  · exact Fraction.le_equiv_left (pointSub_self_zero _)
    (zero_le _ (radius_nonnegative w h s n hh))
  · exact Fraction.magnitudes.le_trans (coarse_mid_le_half w h s i n hh hi hs)
    (Fraction.magnitudes.le_trans hH2 hHR)
  · exact Fraction.magnitudes.le_trans (coarse_end_le_full w h s i n hh hi hs) hHR
  · exact Fraction.magnitudes.le_trans (fine_start_error_le w h s i n hh hi hs) hER
  · exact Fraction.magnitudes.le_trans (fine_mid_le w h s i n hh hi hs) hsum
  · exact Fraction.le_equiv_right (fine_end_le w h s i n hh hin hs) hlast

/-- The first matched patch compares the two paths over the first half-cell. -/
def firstPatch (w h : Fraction) (s : Point × Point) (i : Nat)
  (theta lambda : Fraction) : Point :=
  matchedPatch theta lambda (coarseStart w h s i) (coarseMid w h s i)
  (fineStart w h s i) (fineMid w h s i)

/-- The second matched patch compares the same second-half physical times. -/
def secondPatch (w h : Fraction) (s : Point × Point) (i : Nat)
  (theta lambda : Fraction) : Point :=
  matchedPatch theta lambda (coarseMid w h s i) (coarseEnd w h s i)
  (fineMid w h s i) (fineEnd w h s i)

theorem firstPatch_square (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart w h s i) (radius w h s n)
    (firstPatch w h s i theta lambda) := by
  obtain (hc0, hc1, _, hf0, hf1, _) := actual_corners_in_ball w h s i n hh hin hs

```

```

exact matchedPatch_square theta lambda ht hl _ _ _ _ _ hc0 hc1 hf0 hf1

theorem secondPatch_square (w h : Fraction) (s : Point × Point)
  (i n : Nat) (hh : 0 ≤ h.num) (hin : i < n)
  (hs : SmallTime w h n) (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart w h s i) (radius w h s n)
    (secondPatch w h s i theta lambda) := by
  obtain (_, hc0, hc1, _, hf0, hf1) := actual_corners_in_ball w h s i n hh hin hs
  exact matchedPatch_square theta lambda ht hl _ _ _ _ _ hc0 hc1 hf0 hf1

def shapeFactor (w h : Fraction) (n : Nat) : Fraction :=
  Fraction.add (Fraction.ofInt 4)
    (Fraction.mul (Fraction.ofInt 3) (Fraction.mul (totalTime h n) w.abs))

/-- Compact radius formula: `R=h*M*(4+3*T*|w|)` -/
theorem radius_formula (w h : Fraction) (s : Point × Point) (n : Nat) :
  Fraction.equiv (radius w h s n)
    (Fraction.mul (Fraction.mul h (stateNorm s)) (shapeFactor w h n)) := by
  simp only [radius, fullDriftBudget, maxError, shapeFactor, four,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

/-- The summed square budget is `2*T*h*M*(4+3*T*|w|)^2`. It counts one
square per coarse block; no union-area or disjointness assertion is used. -/
theorem coverBudget_formula (w h : Fraction) (s : Point × Point) (n : Nat) :
  Fraction.equiv (coverBudget w h s n)
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (totalTime h n)
        (Fraction.mul h
          (Fraction.mul (Fraction.mul (stateNorm s) (stateNorm s))
            (Fraction.mul (shapeFactor w h n) (shapeFactor w h n)))))) := by
  let R := radius w h s n
  let Q := shapeFactor w h n
  let M := stateNorm s
  have hr := radius_formula w h s n
  have hsq := Fraction.mul_equiv hr hr
  have harea := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt 4)) hsq
  have hbudget := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (n : Int))) harea
  apply Fraction.equiv_trans hbudget
  simp only [coverBudget, squareArea, R, Q, M, totalTime,
    Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp only [show (4 : Int) = 2 * 2 by rfl]
  ac_nf

private def one : Fraction := {1, 1, by decide}
private def zero : Fraction := {0, 1, by decide}
private def eighth : Fraction := {1, 8, by decide}
private def quarter : Fraction := {1, 4, by decide}
private def sample : Point × Point := ((one, zero), (zero, one))

theorem sample_small_time : SmallTime one eighth 1 := by
  unfold SmallTime Fraction.le
  decide

theorem sample_radius :
  Fraction.equiv (radius one eighth sample 1) {19, 16, by decide} := by decide

theorem sample_square_area :
  Fraction.equiv (squareArea (radius one eighth sample 1))
    {361, 64, by decide} := by decide

theorem sample_cover_budget :
  Fraction.equiv (coverBudget one eighth sample 1)
    {361, 64, by decide} := by decide

theorem sample_all_corners :
  let x := coarseStart one eighth sample 0
  let R := radius one eighth sample 1

```

```

    Fraction.le (pointNorm (pointSub x x)) R ^
    Fraction.le (pointNorm (pointSub (coarseMid one eighth sample 0) x)) R ^
    Fraction.le (pointNorm (pointSub (coarseEnd one eighth sample 0) x)) R ^
    Fraction.le (pointNorm (pointSub (fineStart one eighth sample 0) x)) R ^
    Fraction.le (pointNorm (pointSub (fineMid one eighth sample 0) x)) R ^
    Fraction.le (pointNorm (pointSub (fineEnd one eighth sample 0) x)) R :=
    actual_corners_in_ball one eighth sample 0 1 (by decide) (by decide) sample_small_time

theorem sample_first_patch_square (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart one eighth sample 0) (radius one eighth sample 1)
    (firstPatch one eighth sample 0 theta lambda) :=
  firstPatch_square one eighth sample 0 1 (by decide) (by decide)
    sample_small_time theta lambda ht hl

theorem sample_second_patch_square (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda) :
  SquareContains (coarseStart one eighth sample 0) (radius one eighth sample 1)
    (secondPatch one eighth sample 0 theta lambda) :=
  secondPatch_square one eighth sample 0 1 (by decide) (by decide)
    sample_small_time theta lambda ht hl

theorem sample_zero_blocks_budget :
  Fraction.equiv (coverBudget one eighth sample 0) zero := by decide

theorem sample_zero_duration_budget :
  Fraction.equiv (coverBudget one zero sample 1) zero := by decide

theorem sample_quarter_ball_too_small :
  ~ Fraction.le
    (pointNorm (pointSub (fineEnd one eighth sample 0)
      (coarseStart one eighth sample 0))) quarter := by
  unfold Fraction.le
  decide

/-- A separate coordinate-square control: the coarse endpoint's vertical
offset is 1/4, so a square of radius 1/8 cannot cover it. The preceding
L1-ball control does not assert failure of a square of radius 1/4. -/
theorem sample_eighth_square_too_small :
  ~ SquareContains (coarseStart one eighth sample 0) eighth
    (coarseEnd one eighth sample 0) := by
  unfold SquareContains Fraction.le
  decide

end NewtonLimitDynamics.Polygon.HarmonicCover

```

NewtonLimitDynamics/Polygon/HarmonicDyadic.lean, 29 theorems, 307 lines

```

import BarrowLib.Polygon.GeometricTail
import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import BarrowLib.Polygon.EndpointCauchyName
import BarrowLib.Polygon.StateDistance
import NewtonLimitDynamics.Polygon.HarmonicUniform

/-!
Actual dyadic harmonic endpoint data at one fixed represented rational time.
The Cauchy estimates are derived from the finite end-kick cells. A Cauchy
name is not a limit point, continuous trajectory, or geometric region between
polygon and trajectory. Kepler swept area remains a separate quantity.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicDyadic

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicRefinement
open HarmonicComparison
open HarmonicAccumulation

```

```

open HarmonicUniform
open PointBounds

/-- `2^j` actual end-kick cells, each of duration `T/2^j`. -/
def endpoint (w T : Fraction) (s : Point × Point) (j : Nat) : Point × Point :=
  schedule (linearField w) (List.replicate (blocks j) (duration T j)) s

theorem totalTime_dyadic (T : Fraction) (j : Nat) :
  Fraction.equiv (totalTime (duration T (j + 1)) (blocks j)) T := by
  simp only [totalTime, duration, blocks, Fraction.equiv, Fraction.mul,
    Fraction.ofInt, Int.pow_succ, Int.natCast_pow]
  ac_nf

private theorem pointScale_congr {a b : Fraction} {p q : Point}
  (ha : Fraction.equiv a b) (hp : pointEquiv p q) :
  pointEquiv (pointScale a p) (pointScale b q) :=
  (Fraction.mul_equiv ha hp.1, Fraction.mul_equiv ha hp.2)

private theorem cell_congr {d e : Fraction} {s t : Point × Point}
  (hd : Fraction.equiv d e) (hs : stateEquiv s t) (w : Fraction) :
  stateEquiv (cell (linearField w) d s) (cell (linearField w) e t) := by
  have hpos := pointAdd_congr hs.1 (pointScale_congr hd hs.2)
  have hfield : pointEquiv (linearField w (cell (linearField w) d s).1)
    (linearField w (cell (linearField w) e t).1) :=
    pointScale_congr (Fraction.equiv_refl _) hpos
  exact (hpos, pointAdd_congr hs.2 (pointScale_congr hd hfield))

theorem schedule_replicate_congr (w d e : Fraction)
  (hd : Fraction.equiv d e) :
  (n : Nat) → (s t : Point × Point) → stateEquiv s t →
  stateEquiv (schedule (linearField w) (List.replicate n d) s)
    (schedule (linearField w) (List.replicate n e) t)
| 0, _, _, hs => hs
| n + 1, _, _, hs =>
  schedule_replicate_congr w d e hd n _ _ (cell_congr hd hs w)

theorem fineDurations_replicate (h : Fraction) :
  (n : Nat) → fineDurations h n = List.replicate (n + n) h
| 0 => rfl
| n + 1 => by
  have ih := fineDurations_replicate h n
  have hn : (n + 1) + (n + 1) = 2 + (n + n) := by omega
  rw [hn]
  have hc : 2 + (n + n) = (n + n) + 2 := by omega
  rw [hc]
  simp only [fineDurations, Nat.add_succ, List.replicate_succ]
  rw [ih]

/-- The coarser dyadic endpoint is value-equivalent to the actual coarse
block schedule at half the next level's duration. -/
theorem endpoint_coarse (w T : Fraction) (s : Point × Point) (j : Nat) :
  stateEquiv (endpoint w T s j)
    (coarseAt w (duration T (j + 1)) s (blocks j)) := by
  have hc := schedule_replicate_congr w (duration T j)
    (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
    (duration_halving T j) (blocks j) s s
  ((Fraction.equiv_refl _, Fraction.equiv_refl _),
  (Fraction.equiv_refl _, Fraction.equiv_refl _))
  simp only [endpoint, coarseAt_schedule] using hc

/-- The next dyadic endpoint is the actual two-half-cell schedule. -/
theorem endpoint_fine (w T : Fraction) (s : Point × Point) (j : Nat) :
  endpoint w T s (j + 1) =
  fineAt w (duration T (j + 1)) s (blocks j) := by
  unfold endpoint
  rw [blocks_succ]
  rw [← fineDurations_replicate]
  exact fineAt_schedule w (duration T (j + 1)) s (blocks j)

theorem elapsed_replicate_congr {d e : Fraction}

```

```

    (hd : Fraction.equiv d e) :
    (n : Nat) →
      Fraction.equiv (elapsed (List.replicate n d)) (elapsed (List.replicate n e))
  | 0 => Fraction.equiv_refl _
  | n + 1 => Fraction.add_equiv hd (elapsed_replicate_congr hd n)

theorem endpoint_elapsed (T : Fraction) (j : Nat) :
  Fraction.equiv (elapsed (List.replicate (blocks j) (duration T j))) T :=
  Fraction.equiv_trans
    (elapsed_replicate_congr (duration_halving T j) (blocks j))
    (Fraction.equiv_trans
      (coarse_elapsed_totalTime (duration T (j + 1)) (blocks j))
      (totalTime_dyadic T j))

theorem endpoint_next_elapsed (T : Fraction) (j : Nat) :
  Fraction.equiv
    (elapsed (List.replicate (blocks (j + 1)) (duration T (j + 1)))) T := by
  rw [blocks_succ, ← fineDurations_replicate]
  exact Fraction.equiv_trans
    (fine_elapsed_totalTime (duration T (j + 1)) (blocks j))
    (totalTime_dyadic T j)

def halfThreshold : Fraction := {1, 2, by decide}

def DyadicSmallTime (w T : Fraction) : Prop :=
  Fraction.le
    (Fraction.mul T (Fraction.add (Fraction.ofInt 1) w.abs)) halfThreshold

theorem dyadic_smallTime (w T : Fraction) (j : Nat)
  (hs : DyadicSmallTime w T) :
  SmallTime w (duration T (j + 1)) (blocks j) := by
  have ht := totalTime_dyadic T j
  have he := Fraction.mul_equiv ht
    (Fraction.equiv_refl (Fraction.add (Fraction.ofInt 1) w.abs))
  exact Fraction.le_equiv_left he hs

def adjacentCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul T
      (Fraction.mul (duration T (j + 1))
        (Fraction.mul w.abs (stateNorm s))))

theorem adjacent_error_le (w T : Fraction) (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (endpoint w T s (j + 1)) (endpoint w T s j)))
    (adjacentCap w T s j) := by
  let h := duration T (j + 1)
  let n := blocks j
  have hf := endpoint_fine w T s j
  have hc := endpoint_coarse w T s j
  have he : stateEquiv (stateSub (endpoint w T s (j + 1)) (endpoint w T s j))
    (stateSub (fineAt w h s n) (coarseAt w h s n)) :=
    stateSub_congr
      (by rw [hf]; exact ⟨(Fraction.equiv_refl _, Fraction.equiv_refl _),
        (Fraction.equiv_refl _, Fraction.equiv_refl _)⟩)
      hc
  have hbound := actual_uniform_error w h s n hT (dyadic_smallTime w T j hs)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hbound
  apply Fraction.le_equiv_right hfirst
  exact Fraction.mul_equiv (Fraction.equiv_refl _)
    (Fraction.mul_equiv (totalTime_dyadic T j) (Fraction.equiv_refl _))

/-- The finite tail coefficient `A=3*T^2*|w|*M`. -/
def coefficient (w T : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul T (Fraction.mul T (Fraction.mul w.abs (stateNorm s))))

def tailCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  let A := coefficient w T s

```

```

(A.num, A.den * (2 : Int) ^ j,
  Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

def doubleTail (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  let A := coefficient w T s
  (2 * A.num, A.den * (2 : Int) ^ j,
    Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

theorem adjacentCap_tail (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (adjacentCap w T s j) (tailCap w T s (j + 1)) := by
  simp only [adjacentCap, tailCap, coefficient, duration, Fraction.equiv,
    Fraction.mul, Fraction.ofInt, Int.pow_succ]
  ac_nf

theorem tail_halving (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (Fraction.add (tailCap w T s (j + 1)) (tailCap w T s (j + 1)))
    (tailCap w T s j) := by
  exact GeometricTail.tail_halving (coefficient w T s) j

theorem tail_double (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (Fraction.add (tailCap w T s j) (tailCap w T s j))
    (doubleTail w T s j) := by
  exact GeometricTail.tail_double (coefficient w T s) j

theorem coefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (coefficient w T s).num :=
  Int.mul_nonneg (by decide)
    (Int.mul_nonneg hT (Int.mul_nonneg hT
      (Int.mul_nonneg (Fraction.abs_num_nonnegative w) (stateNorm_nonnegative s))))

/-- Any finite separation of dyadic levels has error within the tail at its
coarser endpoint. The proof uses actual neighboring schedules. -/
theorem finite_gap_error (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (k j : Nat) → Fraction.le
    (stateNorm (stateSub (endpoint w T s (j + k)))
      (endpoint w T s j)) (tailCap w T s j) := by
  intro k j
  exact GeometricTail.finite_gap (endpoint w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => Fraction.le_equiv_right (adjacent_error_le w T s i hT hs)
      (adjacentCap_tail w T s i)) k j

/-- Both later endpoints are compared to the same earlier actual endpoint. -/
theorem two_sided_error (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le
    (stateNorm (stateSub (endpoint w T s m) (endpoint w T s n)))
    (doubleTail w T s N) := by
  exact GeometricTail.two_sided (endpoint w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => Fraction.le_equiv_right (adjacent_error_le w T s i hT hs)
      (adjacentCap_tail w T s i)) N m n hm hn

/-- A deliberately simple, potentially large explicit precision modulus. -/
def modulus (w T : Fraction) (s : Point × Point) (eps : Fraction) : Nat :=
  (2 * (coefficient w T s).num * eps.den).toNat

theorem doubleTail_lt_tolerance (w T : Fraction) (s : Point × Point)
  (eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail w T s (modulus w T s eps)) eps := by
  exact GeometricTail.doubleTail_lt_tolerance (coefficient w T s) eps
    (coefficient_nonnegative w T s hT) heps

/-- A Cauchy name stores finite rational endpoint approximants and a proved
positive-tolerance condition. It does not supply a limit point. -/
theorem endpoint_cauchy (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :

```



```

    ∀ eps : Fraction, 0 < eps.num →
      ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
        Fraction.lt
          (stateNorm (stateSub (endpoint w T s m) (endpoint w T s n))) eps := by
intro eps heps
refine (modulus w T s eps, ?_)
intro m n hm hn
exact Fraction.magnitudes.lt_of_le_lt
  (two_sided_error w T s hT hs _ m n hm hn)
  (doubleTail_lt_tolerance w T s eps hT heps)

def endpointName (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : EndpointCauchyName where
  approx := endpoint w T s
  cauchy := endpoint_cauchy w T s hT hs

private theorem stateEquiv_refl (s : Point × Point) : stateEquiv s s :=
  ((Fraction.equiv_refl _, Fraction.equiv_refl _),
   (Fraction.equiv_refl _, Fraction.equiv_refl _))

private theorem stateEquiv_trans {a b c : Point × Point}
  (hab : stateEquiv a b) (hbc : stateEquiv b c) : stateEquiv a c :=
  ((Fraction.equiv_trans hab.1.1 hbc.1.1,
    Fraction.equiv_trans hab.1.2 hbc.1.2),
   (Fraction.equiv_trans hab.2.1 hbc.2.1,
    Fraction.equiv_trans hab.2.2 hbc.2.2))

private theorem zero_duration_cell (w d : Fraction) (s : Point × Point)
  (hd : d.num = 0) : stateEquiv (cell (linearField w) d s) s := by
  let z : Fraction := {0, 1, by decide}
  have he : Fraction.equiv d z := by
    unfold Fraction.equiv z
    simp [hd]
  exact stateEquiv_trans (cell_congr he (stateEquiv_refl s) w)
    (zero_step w s)

theorem zero_duration_schedule (w d : Fraction) (hd : d.num = 0) :
  (n : Nat) → (s : Point × Point) →
    stateEquiv (schedule (linearField w) (List.replicate n d) s) s
| 0, s => stateEquiv_refl s
| n + 1, s =>
  stateEquiv_trans
    (zero_duration_schedule w d hd n (cell (linearField w) d s))
    (zero_duration_cell w d s hd)

theorem zero_time_endpoint (w T : Fraction) (s : Point × Point) (j : Nat)
  (hT : T.num = 0) : stateEquiv (endpoint w T s j) s := by
  exact zero_duration_schedule w (duration T j) (by exact hT) (blocks j) s

private def sampleOne : Fraction := {1, 1, by decide}
private def sampleZero : Fraction := {0, 1, by decide}
private def sampleTime : Fraction := {1, 4, by decide}
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))

theorem sample_dyadic_small_time : DyadicSmallTime sampleOne sampleTime := by
  unfold DyadicSmallTime Fraction.le
  decide

theorem sample_adjacent_error :
  Fraction.equiv
    (stateNorm (stateSub (endpoint sampleOne sampleTime sampleState 1)
      (endpoint sampleOne sampleTime sampleState 0)))
    (145, 4096, by decide) := by decide

theorem sample_adjacent_cap :
  Fraction.equiv (adjacentCap sampleOne sampleTime sampleState 0)
    (3, 16, by decide) := by decide

theorem sample_tail_cap :

```

```

    Fraction.equiv (tailCap sampleOne sampleTime sampleState 0)
      (3, 8, by decide) := by decide

end NewtonLimitDynamics.Polygon.HarmonicDyadic

NewtonLimitDynamics/Polygon/HarmonicTimeComparison.lean, 33 theorems, 635 lines

import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import NewtonLimitDynamics.Polygon.HarmonicDyadic

/-!
Finite comparisons of actual harmonic endpoint schedules at two rational times.
The cell counts agree; their durations differ. These estimates concern Cauchy
data only, without a limit point, trajectory, or intervening-area content.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicTimeComparison

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicComparison
open HarmonicAccumulation
open HarmonicUniform
open HarmonicDyadic
open PointBounds

theorem cell_parameter_difference (w sigma tau : Fraction) (s : Point × Point) :
  stateEquiv
    (stateSub (cell (linearField w) tau s) (cell (linearField w) sigma s))
    (pointScale (durationDifference sigma tau) s.2,
     pointScale (negF (Fraction.mul (durationDifference sigma tau) w))
      (pointAdd s.1 (pointScale (Fraction.add sigma tau) s.2))) := by
  constructor <=> constructor <=>
  simp only [durationDifference, stateSub, pointEquiv, pointSub, pointNeg,
    cell, linearField, negF, pointAdd, pointScale, Fraction.equiv,
    Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem cell_parameter_norm_formula (w sigma tau : Fraction) (s : Point × Point) :
  Fraction.equiv
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.add (pointNorm s.2)
        (Fraction.mul w.abs
          (pointNorm (pointAdd s.1
            (pointScale (Fraction.add sigma tau) s.2)))))) := by
  have hs := stateNorm_equiv (cell_parameter_difference w sigma tau s)
  apply Fraction.equiv_trans hs
  simp only [stateNorm, pointNorm, pointScale, negF, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.abs, Int.natAbs_mul,
    Int.natAbs_neg, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem short_sum_point_bound (sigma tau : Fraction) (x v : Point)
  (hσ : 0 ≤ sigma.num) (hτ : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (pointNorm (pointAdd x (pointScale (Fraction.add sigma tau) v)))
    (Fraction.add (pointNorm x) (pointNorm v)) := by
  let q := Fraction.add sigma tau
  have hq : 0 ≤ q.num := Fraction.nonnegative_add sigma tau hσ hτ
  have hs := pointNorm_scale q v
  have hqabs := Fraction.abs_of_nonnegative q hq
  have hscale : Fraction.le (pointNorm (pointScale q v)) (pointNorm v) := by
    have hq' : Fraction.le q.abs (Fraction.ofInt 1) :=

```

```

    Fraction.le_equiv_left hqabs hsum
    have hm := Fraction.mul_le_mul_nonnegative hq' (pointNorm v)
      (pointNorm_nonnegative v)
    apply Fraction.le_equiv_left hs
    apply Fraction.le_equiv_right hm
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
    exact Fraction.magnitudes.le_trans (pointNorm_add_le x (pointScale q v))
      (Fraction.add_le_add_left hscale (pointNorm x))

private theorem scalar_local_bound (a b c : Fraction)
  (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) (hc : 0 ≤ c.num) :
  Fraction.le
    (Fraction.add b (Fraction.mul c (Fraction.add a b)))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1)
      (Fraction.mul (Fraction.ofInt 2) c)) (Fraction.add a b)) := by
  let M := Fraction.add a b
  let cM := Fraction.mul c M
  have hbM : Fraction.le b M := by
    unfold Fraction.le M Fraction.add
    dsimp
    rw [Int.add_mul]
    have hnon := Int.mul_nonneg
      (Int.mul_nonneg ha (Int.le_of_lt b.den_pos)) (Int.le_of_lt b.den_pos)
    have he : b.num * (a.den * b.den) = b.num * a.den * b.den := by ac_rfl
    rw [he]
    omega
  have hfirst := Fraction.add_le_add_right hbM cM
  have hcM : 0 ≤ cM.num := Int.mul_nonneg hc (Fraction.nonnegative_add a b ha hb)
  have hz : Fraction.le (Fraction.ofInt 0) cM := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simpa using hcM
  have hzero : Fraction.equiv (Fraction.add M (Fraction.ofInt 0)) M := by
    unfold Fraction.equiv Fraction.add Fraction.ofInt
    simp only [Int.mul_one, Int.zero_mul, Int.add_zero, Int.mul_zero]
  have hsecond : Fraction.le (Fraction.add M cM)
    (Fraction.add (Fraction.add M cM) cM) := by
    have hh := Fraction.add_le_add_left hz (Fraction.add M cM)
    exact Fraction.le_equiv_left (by
      simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
      simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
        Int.mul_one, Int.one_mul]) hh
  have he : Fraction.equiv (Fraction.add (Fraction.add M cM) cM)
    (Fraction.mul (Fraction.add (Fraction.ofInt 1)
      (Fraction.mul (Fraction.ofInt 2) c)) M) := by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
    simp only [show (2 : Int) = 1 + 1 by rfl]
    simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
    simp only [cM, Fraction.mul]
    ac_nf
  exact Fraction.le_equiv_right (Fraction.magnitudes.le_trans hfirst hsecond) he

/-- The exact duration mismatch of one actual end-kick cell, bounded under a
short nonnegative combined duration. -/
theorem cell_parameter_bound (w sigma tau : Fraction) (s : Point × Point)
  (ho : 0 ≤ sigma.num) (ht : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.mul
        (Fraction.add (Fraction.ofInt 1)
          (Fraction.mul (Fraction.ofInt 2) w.abs)) (stateNorm s))) := by
  let d := durationDifference sigma tau
  have hp := short_sum_point_bound sigma tau s.1 s.2 ho ht hsum
  have hw := Fraction.mul_le_mul_nonnegative_left hp w.abs
    (Fraction.abs_num_nonnegative w)
  have ha := Fraction.add_le_add_left hw (pointNorm s.2)

```

```

have hd := Fraction.mul_le_mul_nonnegative_left ha d.abs
  (Fraction.abs_num_nonnegative d)
have hc := scalar_local_bound (pointNorm s.1) (pointNorm s.2) w.abs
  (pointNorm_nonnegative s.1) (pointNorm_nonnegative s.2)
  (Fraction.abs_num_nonnegative w)
have hdc := Fraction.mul_le_mul_nonnegative_left hc d.abs
  (Fraction.abs_num_nonnegative d)
have hf := cell_parameter_norm_formula w sigma tau s
exact Fraction.le_equiv_right
  (Fraction.magnitudes.le_trans (Fraction.le_equiv_left hf hd) hdc)
  (by simp only [stateNorm]; exact Fraction.equiv_refl _)

def parameterFactor (w : Fraction) : Fraction :=
  Fraction.add (Fraction.ofInt 1)
    (Fraction.mul (Fraction.ofInt 2) w.abs)

def localParameterBudget (w sigma tau : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul (durationDifference sigma tau).abs
    (Fraction.mul (parameterFactor w)
      (Fraction.mul (Fraction.ofInt 2) (stateNorm s)))

def parameterErrorBudget (w hσ hτ : Fraction) (s : Point × Point) : Nat → Fraction
| 0 => Fraction.ofInt 0
| i + 1 =>
  Fraction.add
    (Fraction.mul (coarseFactor w hτ) (parameterErrorBudget w hσ hτ s i))
    (localParameterBudget w (Fraction.add hσ hσ) (Fraction.add hτ hτ) s)

private theorem parameterFactor_nonnegative (w : Fraction) :
  0 ≤ (parameterFactor w).num := by
  unfold parameterFactor
  exact Fraction.nonnegative_add _ _ (by decide)
  (Int.mul_nonneg (by decide) (Fraction.abs_num_nonnegative w))

private theorem localParameterBudget_nonnegative (w sigma tau : Fraction)
  (s : Point × Point) :
  0 ≤ (localParameterBudget w sigma tau s).num := by
  unfold localParameterBudget
  exact Int.mul_nonneg (Fraction.abs_num_nonnegative _)
    (Int.mul_nonneg (parameterFactor_nonnegative w)
      (Int.mul_nonneg (by decide) (stateNorm_nonnegative s)))

private theorem parameterErrorBudget_nonnegative (w hσ hτ : Fraction)
  (s : Point × Point) :
  (i : Nat) → 0 ≤ (parameterErrorBudget w hσ hτ s i).num
| 0 => by simp [parameterErrorBudget, Fraction.ofInt]
| i + 1 =>
  Fraction.nonnegative_add _ _
    (Int.mul_nonneg (kappa_nonnegative w (Fraction.add hτ hτ))
      (parameterErrorBudget_nonnegative w hσ hτ s i))
    (localParameterBudget_nonnegative w _ _ s)

theorem coarse_parameter_step (w sigma tau : Fraction) (a b : Point × Point)
  (hσ : 0 ≤ sigma.num) (hτ : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau a)
      (cell (linearField w) sigma b)))
    (Fraction.add
      (Fraction.mul (kappa w tau) (stateNorm (stateSub a b)))
      (Fraction.mul (durationDifference sigma tau).abs
        (Fraction.mul (parameterFactor w) (stateNorm b)))) := by
  have htri := stateSub_triangle (cell (linearField w) tau a)
    (cell (linearField w) tau b) (cell (linearField w) sigma b)
  have h₁ := cell_perturbation w tau a b
  have h₂ := cell_parameter_bound w sigma tau b hσ hτ hsum
  exact Fraction.magnitudes.le_trans htri (Fraction.add_le_add h₁ h₂)

theorem actual_coarse_parameter_error (w hσ hτ : Fraction) (s : Point × Point)
  (n : Nat) (hho : 0 ≤ hσ.num) (hht : 0 ≤ hτ.num)

```

```

(hsum : Fraction.le
  (Fraction.add (Fraction.add hσ hσ) (Fraction.add hτ hτ))
  (Fraction.ofInt 1)) (hsmall : SmallTime w hσ n) :
(i : Nat) → i ≤ n →
  Fraction.le
    (stateNorm (stateSub (coarseAt w hτ s i) (coarseAt w hσ s i)))
    (parameterErrorBudget w hσ hτ s i)
| 0, _ => Fraction.le_of_equiv (stateSub_self_norm_zero s)
| i + 1, hi => by
  let sigma := Fraction.add hσ hσ
  let tau := Fraction.add hτ hτ
  let a := coarseAt w hτ s i
  let b := coarseAt w hσ s i
  have hi' : i ≤ n := by omega
  have hnon := Fraction.nonnegative_add hσ hσ hho hho
  have htnon := Fraction.nonnegative_add hτ hτ hht hht
  have hstep := coarse_parameter_step w sigma tau a b
  hnon htnon hsum
  have hprev := actual_coarse_parameter_error w hσ hτ s n hho hht
  hsum hsmall i hi'
  have hA := Fraction.mul_le_mul_nonnegative_left hprev
    (coarseFactor w hτ) (kappa_nonnegative w tau)
  have hprefix := coarse_state_le_two w hσ s i hho
    (smallTime_prefix w hσ i n hho hi' hsmall)
  have hB₁ := Fraction.mul_le_mul_nonnegative_left hprefix
    (parameterFactor w) (parameterFactor_nonnegative w)
  have hB₂ := Fraction.mul_le_mul_nonnegative_left hB₁
    (durationDifference sigma tau).abs
    (Fraction.abs_num_nonnegative _)
  have hsum' := Fraction.add_le_add hA hB₂
  have hchain := Fraction.magnitudes.le_trans hstep hsum'
  simp only [coarseAt, HarmonicRefinement.coarse,
    parameterErrorBudget, localParameterBudget] using hchain

def parameterPowerBudget (w hσ hτ : Fraction) (s : Point × Point)
  (i : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (i : Int))
    (Fraction.mul
      (localParameterBudget w (Fraction.add hσ hσ) (Fraction.add hτ hτ) s)
      (fpower (coarseFactor w hτ) i))

theorem parameter_budget_power (w hσ hτ : Fraction) (s : Point × Point) :
  (i : Nat) → Fraction.le (parameterErrorBudget w hσ hτ s i)
    (parameterPowerBudget w hσ hτ s i)
| 0 => Fraction.le_of_equiv (by
  simp only [parameterErrorBudget, parameterPowerBudget, fpower,
    Fraction.equiv, Fraction.ofInt, Fraction.mul]
  simp)
| i + 1 => by
  let d := localParameterBudget w (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  let b := coarseFactor w hτ
  have hd := localParameterBudget_nonnegative w
    (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  have hb := kappa_nonnegative w (Fraction.add hτ hτ)
  have h₁ := Fraction.mul_le_mul_nonnegative_left
    (parameter_budget_power w hσ hτ s i) b hb
  have hD : Fraction.le d (Fraction.mul d (fpower b (i + 1))) := by
    have hpow := one_le_power b hb (one_le_kappa w (Fraction.add hτ hτ))
    (i + 1)
  have hm := Fraction.mul_le_mul_nonnegative_left hpow d hd
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp) hm
  have hsum := Fraction.add_le_add h₁ hD
  apply Fraction.le_equiv_right hsum
  simp only [d, b, parameterErrorBudget, parameterPowerBudget,
    fpower, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.one_mul, Int.mul_one]
  ac_nf

```

```

theorem actual_coarse_parameter_uniform (w hσ hτ : Fraction)
  (s : Point × Point) (n : Nat)
  (hho : 0 ≤ hσ.num) (hht : 0 ≤ hτ.num)
  (hsum : Fraction.le
    (Fraction.add (Fraction.add hσ hσ) (Fraction.add hτ hτ))
    (Fraction.ofInt 1))
  (hsmallσ : SmallTime w hσ n) (hsmallτ : SmallTime w hτ n) :
  Fraction.le
    (stateNorm (stateSub (coarseAt w hτ s n) (coarseAt w hσ s n)))
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (localParameterBudget w (Fraction.add hσ hσ)
        (Fraction.add hτ hτ) s)) := by
  let d := localParameterBudget w (Fraction.add hσ hσ)
    (Fraction.add hτ hτ) s
  have h₁ := actual_coarse_parameter_error w hσ hτ s n hho hht
  hsum hsmallσ n (Nat.le_refl n)
  have h₂ := parameter_budget_power w hσ hτ s n
  have hp := coarse_power_le_two w hτ n hht hsmallτ
  have hd := localParameterBudget_nonnegative w
    (Fraction.add hσ hσ) (Fraction.add hτ hτ) s
  have hm := Fraction.mul_le_mul_nonnegative_left hp d hd
  have hn := Fraction.mul_le_mul_nonnegative_left hm
    (Fraction.ofInt (n : Int)) (Int.ofNat_nonneg n)
  have hchain := Fraction.magnitudes.le_trans
    (Fraction.magnitudes.le_trans h₁ h₂) hn
  apply Fraction.le_equiv_right hchain
  simp only [parameterPowerBudget, d, Fraction.equiv, Fraction.mul,
    Fraction.ofInt]
  ac_nf

private def half : Fraction := (1, 2, by decide)

theorem dyadic_time_le_half (w T : Fraction)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le T half := by
  have hw : 0 ≤ (w.num.natAbs : Int) := Int.ofNat_nonneg _
  have hnon : 0 ≤ 2 * T.num * (w.num.natAbs : Int) :=
    Int.mul_nonneg (Int.mul_nonneg (by decide) hT) hw
  have hraw :
    2 * T.num * (w.den + (w.num.natAbs : Int)) ≤ T.den * w.den := by
    unfold DyadicSmallTime Fraction.le Fraction.mul Fraction.add Fraction.ofInt
    Fraction.abs at hs
    dsimp [HarmonicDyadic.halfThreshold] at hs
    simp only [Int.one_mul, Int.mul_one] at hs
    calc
      2 * T.num * (w.den + (w.num.natAbs : Int)) =
        T.num * (w.den + (w.num.natAbs : Int)) * 2 := by ac_rfl
      _ ≤ T.den * w.den := hs
  have hmul : (2 * T.num) * w.den ≤ T.den * w.den := by
    rw [Int.mul_add] at hraw
    omega
  have hbase := Int.le_of_mul_le_mul_right hmul w.den_pos
  unfold Fraction.le half
  dsimp
  omega

theorem duration_le_time (T : Fraction) (j : Nat) (hT : 0 ≤ T.num) :
  Fraction.le (duration T j) T := by
  have hpow : 1 ≤ (2 : Int) ^ j := by
    have h := two_pow_ge_succ j
    omega
  have hd := Int.mul_nonneg hT (Int.le_of_lt T.den_pos)
  have hm := Int.mul_le_mul_of_nonneg_left hpow hd
  unfold Fraction.le duration
  dsimp
  have he : T.num * (T.den * (2 : Int) ^ j) =
    (T.num * T.den) * (2 : Int) ^ j := by ac_rfl
  rw [he]
  omega

```

```

theorem short_duration_pair (w T U : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.le
    (Fraction.add
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1))))
    (Fraction.ofInt 1) := by
  have hT' := duration_le_time T j hT
  have hU' := duration_le_time U j hU
  have hT'' := Fraction.le_equiv_left
    (Fraction.equiv_symm (duration_halving T j)) hT'
  have hU'' := Fraction.le_equiv_left
    (Fraction.equiv_symm (duration_halving U j)) hU'
  have hhalf := Fraction.add_le_add (dyadic_time_le_half w T hT hsT)
    (dyadic_time_le_half w U hU hsU)
  have hsum := Fraction.magnitudes.le_trans
    (Fraction.add_le_add hT'' hU'') hhalf
  apply Fraction.le_equiv_right hsum
  decide

/-- The common count cancels the per-cell signed duration difference in
rational value, even though the representatives differ. -/
theorem count_duration_difference (T U : Fraction) (j : Nat) :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt (blocks j : Int))
      (durationDifference
        (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
        (Fraction.add (duration U (j + 1)) (duration U (j + 1)))))
    (durationDifference T U) := by
  have hdur : Fraction.equiv
    (durationDifference
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1))))
    (durationDifference (duration T j) (duration U j)) :=
  Fraction.add_equiv
    (Fraction.equiv_symm (duration_halving U j))
    (HarmonicDyadic.neg_equiv
      (Fraction.equiv_symm (duration_halving T j)))
  have hm := Fraction.mul_equiv
    (Fraction.equiv_refl (Fraction.ofInt (blocks j : Int))) hdur
  apply Fraction.equiv_trans hm
  simp only [blocks, duration, durationDifference, negF, Fraction.equiv,
    Fraction.ofInt, Fraction.add, Fraction.mul, Int.natCast_pow]
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.one_mul, Int.mul_one]
  ac_nf

theorem count_abs_duration_difference (T U : Fraction) (j : Nat) :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt (blocks j : Int))
      (durationDifference
        (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
        (Fraction.add (duration U (j + 1)) (duration U (j + 1)))).abs)
    (durationDifference T U).abs := by
  have hs := count_duration_difference T U j
  have habs := Fraction.abs_equiv hs
  have hm := Fraction.abs_mul (Fraction.ofInt (blocks j : Int))
    (durationDifference
      (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
      (Fraction.add (duration U (j + 1)) (duration U (j + 1)))).abs
  have hcount : Fraction.equiv (Fraction.ofInt (blocks j : Int)).abs
    (Fraction.ofInt (blocks j : Int)) :=
  Fraction.abs_of_nonnegative _ (Int.ofNat_nonneg _)
  have hmul := Fraction.mul_equiv hcount
    (Fraction.equiv_refl
      (durationDifference
        (Fraction.add (duration T (j + 1)) (duration T (j + 1)))
        (Fraction.add (duration U (j + 1)) (duration U (j + 1)))).abs)

```

```

exact Fraction.equiv_trans (Fraction.equiv_symm hmul)
  (Fraction.equiv_trans (Fraction.equiv_symm hm) habs)

def timeLipschitz (w : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul (Fraction.ofInt 4)
    (Fraction.mul (parameterFactor w) (stateNorm s))

/-- Uniform rational-time variation of the actual dyadic endpoint schedules.
The same level has the same count and two different cell durations. -/
theorem endpoint_time_bound (w T U : Fraction) (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.le
    (stateNorm (stateSub (endpoint w U s j) (endpoint w T s j)))
    (Fraction.mul (timeLipschitz w s) (durationDifference T U).abs) := by
  let hσ := duration T (j + 1)
  let hτ := duration U (j + 1)
  let n := blocks j
  have honon : 0 ≤ hσ.num := hT
  have hτnon : 0 ≤ hτ.num := hU
  have hbound := actual_coarse_parameter_uniform w hσ hτ s n
    honon hτnon (short_duration_pair w T U j hT hU hsT hsU)
    (dyadic_smallTime w T j hsT) (dyadic_smallTime w U j hsU)
  have he : stateEquiv
    (stateSub (endpoint w U s j) (endpoint w T s j))
    (stateSub (coarseAt w hτ s n) (coarseAt w hσ s n)) :=
    stateSub_congr (endpoint_coarse w U s j) (endpoint_coarse w T s j)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hbound
  let d := (durationDifference
    (Fraction.add hσ hσ) (Fraction.add hτ hτ)).abs
  have hrewrite : Fraction.equiv
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (localParameterBudget w (Fraction.add hσ hσ)
        (Fraction.add hτ hτ) s))
    (Fraction.mul
      (Fraction.mul (Fraction.ofInt 4)
        (Fraction.mul (parameterFactor w) (stateNorm s)))
      (Fraction.mul (Fraction.ofInt (n : Int)) d)) := by
    simp only [localParameterBudget, d, Fraction.equiv, Fraction.mul,
      Fraction.ofInt]
  ac_nf
  have hc := count_abs_duration_difference T U j
  have hsecond := Fraction.mul_equiv
    (Fraction.equiv_refl (timeLipschitz w s)) hc
  exact Fraction.le_equiv_right hfirst
    (Fraction.equiv_trans hrewrite hsecond)

theorem timeLipschitz_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (timeLipschitz w s).num :=
  Int.mul_nonneg (by decide)
    (Int.mul_nonneg (parameterFactor_nonnegative w) (stateNorm_nonnegative s))

private def timeDenominator (w : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (timeLipschitz w s) (Fraction.ofInt 1)

private theorem timeDenominator_positive (w : Fraction) (s : Point × Point) :
  0 < (timeDenominator w s).num := by
  unfold timeDenominator Fraction.add Fraction.ofInt
  dsimp
  have hL := timeLipschitz_nonnegative w s
  have hd := (timeLipschitz w s).den_pos
  omega

/-- Rational delta equal in value to `eps/(L+1)`. The added one makes the
choice positive even when the initial state magnitude is zero. -/
def timeDelta (w : Fraction) (s : Point × Point) (eps : Fraction) : Fraction :=
  (eps.num * (timeDenominator w s).den,
    eps.den * (timeDenominator w s).num,
    Int.mul_pos eps.den_pos (timeDenominator_positive w s))

```



```

theorem timeDelta_positive (w : Fraction) (s : Point × Point) (eps : Fraction)
  (heps : 0 < eps.num) : 0 < (timeDelta w s eps).num :=
  Int.mul_pos heps (timeDenominator w s).den_pos

private theorem mul_lt_mul_positive_left {a b : Fraction}
  (hab : Fraction.lt a b) (c : Fraction) (hc : 0 < c.num) :
  Fraction.lt (Fraction.mul c a) (Fraction.mul c b) := by
  have hm := Int.mul_lt_mul_of_pos_right hab
  (Int.mul_pos hc c.den_pos)
  unfold Fraction.lt Fraction.mul at *
  dsimp at *
  have h1 : c.num * a.num * (c.den * b.den) =
    (a.num * b.den) * (c.num * c.den) := by ac_rfl
  have h2 : c.num * b.num * (c.den * a.den) =
    (b.num * a.den) * (c.num * c.den) := by ac_rfl
  rw [h1, h2]
  exact hm

private theorem timeLipschitz_le_denominator (w : Fraction) (s : Point × Point) :
  Fraction.le (timeLipschitz w s) (timeDenominator w s) := by
  unfold timeDenominator Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hd := (timeLipschitz w s).den_pos
  have hsq : 0 ≤ (timeLipschitz w s).den * (timeLipschitz w s).den :=
    Int.mul_nonneg (Int.le_of_lt hd) (Int.le_of_lt hd)
  simp only [Int.one_mul, Int.mul_one, Int.add_mul]
  omega

private theorem delta_product_equiv (w : Fraction) (s : Point × Point)
  (eps : Fraction) :
  Fraction.equiv
    (Fraction.mul (timeDenominator w s) (timeDelta w s eps)) eps := by
  unfold Fraction.equiv Fraction.mul timeDelta
  dsimp
  ac_nf

theorem parameter_delta_control (w : Fraction) (s : Point × Point)
  (eps d : Fraction) (heps : 0 < eps.num)
  (hd : 0 ≤ d.num) (hdelta : Fraction.lt d (timeDelta w s eps)) :
  Fraction.lt (Fraction.mul (timeLipschitz w s) d) eps := by
  have hweak := Fraction.mul_le_mul_nonnegative
    (timeLipschitz_le_denominator w s) d hd
  have hstrict := mul_lt_mul_positive_left hdelta
    (timeDenominator w s) (timeDenominator_positive w s)
  have htrans := Fraction.magnitudes.lt_of_le_lt hweak hstrict
  have heq := delta_product_equiv w s eps
  exact Fraction.magnitudes.lt_of_lt_le htrans
    ((Fraction.equiv_iff_mutual_le _ _).mp heq).1

def ShortRationalTime (w : Fraction) :=
  {T : Fraction // 0 ≤ T.num ∧ DyadicSmallTime w T}

/-- Each admissible rational time is mapped to its derived endpoint Cauchy
name. This map does not realize a point of a completed state space. -/
def timeName (w : Fraction) (s : Point × Point)
  (T : ShortRationalTime w) : EndpointCauchyName :=
  endpointName w T.val s T.property.1 T.property.2

/-- One explicit delta controls every approximant level at once. -/
theorem timeName_uniform_continuity (w : Fraction) (s : Point × Point)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ delta : Fraction, 0 < delta.num ∧
    ∀ T U : ShortRationalTime w,
      Fraction.lt (durationDifference T.val U.val).abs delta →
        ∀ j : Nat,
          Fraction.lt
            (stateNorm (stateSub ((timeName w s U).approx j)
              ((timeName w s T).approx j))) eps := by
  refine {timeDelta w s eps, timeDelta_positive w s eps heps, ?_}
  intro T U hdelta j

```

```

have hb := endpoint_time_bound w T.val U.val s j
T.property.1 U.property.1 T.property.2 U.property.2
have hd := Fraction.abs_num_nonnegative (durationDifference T.val U.val)
have hstrict := parameter_delta_control w s eps
(durationDifference T.val U.val).abs heps hd hdelta
exact Fraction.magnitudes.lt_of_le_lt hb hstrict

theorem same_time_error_zero (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (stateNorm (stateSub (endpoint w T s j) (endpoint w T s j)))
    (Fraction.ofInt 0) :=
  stateSub_self_norm_zero _

private def zeroFraction : Fraction := (0, 1, by decide)
private def zeroState : Point × Point :=
  ((zeroFraction, zeroFraction), (zeroFraction, zeroFraction))

theorem zero_state_error_zero (w T U : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) (hU : 0 ≤ U.num)
  (hsT : DyadicSmallTime w T) (hsU : DyadicSmallTime w U) :
  Fraction.equiv
    (stateNorm (stateSub (endpoint w U zeroState j)
      (endpoint w T zeroState j))) (Fraction.ofInt 0) := by
  have hb := endpoint_time_bound w T U zeroState j hT hU hsT hsU
  have hz : Fraction.equiv
    (Fraction.mul (timeLipschitz w zeroState)
      (durationDifference T U).abs) (Fraction.ofInt 0) := by
  simp only [timeLipschitz, parameterFactor, zeroState,
    zeroFraction, stateNorm, pointNorm, Fraction.equiv,
    Fraction.abs, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp
  have hle := Fraction.le_equiv_right hb hz
  have hother : Fraction.le (Fraction.ofInt 0)
    (stateNorm (stateSub (endpoint w U zeroState j)
      (endpoint w T zeroState j))) := by
  unfold Fraction.le Fraction.ofInt
  dsimp
  simp only [Int.zero_mul, Int.mul_one]
  exact stateNorm_nonnegative _
  exact (Fraction.equiv_iff_mutual_le _ _).mpr (hle, hother)

private def sampleOne : Fraction := (1, 1, by decide)
private def sampleZero : Fraction := (0, 1, by decide)
private def sampleQuarter : Fraction := (1, 4, by decide)
private def sampleEighth : Fraction := (1, 8, by decide)
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))

theorem sample_short_times :
  DyadicSmallTime sampleOne sampleQuarter ∧
  DyadicSmallTime sampleOne sampleEighth := by
  constructor <=> unfold DyadicSmallTime Fraction.le <=> decide

theorem sample_parameter_error :
  Fraction.equiv
    (stateNorm (stateSub
      (endpoint sampleOne sampleEighth sampleState 0)
      (endpoint sampleOne sampleQuarter sampleState 0)))
    (19, 64, by decide) := by decide

theorem sample_time_lipschitz :
  Fraction.equiv (timeLipschitz sampleOne sampleState)
    (Fraction.ofInt 24) := by decide

theorem sample_time_budget :
  Fraction.equiv
    (Fraction.mul (timeLipschitz sampleOne sampleState)
      (durationDifference sampleQuarter sampleEighth).abs)
    (Fraction.ofInt 3) := by decide

```

```

theorem sample_parameter_bound :
  Fraction.le
    (stateNorm (stateSub
      (endpoint sampleOne sampleEighth sampleState 0)
      (endpoint sampleOne sampleQuarter sampleState 0)))
    (Fraction.mul (timeLipschitz sampleOne sampleState)
      (durationDifference sampleQuarter sampleEighth).abs) :=
  endpoint_time_bound sampleOne sampleQuarter sampleEighth sampleState 0
  (by decide) (by decide) sample_short_times.1 sample_short_times.2

end NewtonLimitDynamics.Polygon.HarmonicTimeComparison

```

NewtonLimitDynamics/Polygon/HarmonicBinaryPrefix.lean, 32 theorems, 530 lines

```

import BarrowLib.Polygon.GeometricTail
import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import NewtonLimitDynamics.Polygon.HarmonicTimeComparison

/-!
Actual intermediate prefixes of one dyadic harmonic polygon family, indexed
by binary addresses. The approximants are finite schedules only. No completed
point, continuum trajectory, or region between polygon and trajectory is
assumed; Kepler swept area remains a distinct quantity.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicComparison
open HarmonicAccumulation
open HarmonicUniform
open HarmonicDyadic
open HarmonicTimeComparison
open PointBounds

def prefixState (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (j : Nat) : Point × Point :=
  schedule (linearField w) (List.replicate (ticks b j) (duration T j)) s

theorem prefix_coarse (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) :
  stateEquiv (prefixState b w T s j)
    (coarseAt w (duration T (j + 1)) s (ticks b j)) := by
  let h := duration T (j + 1)
  have hc := schedule_replicate_congr w (duration T j) (Fraction.add h h)
    (duration_halving T j) (ticks b j) s s
    ({Fraction.equiv_refl _, Fraction.equiv_refl _},
    {Fraction.equiv_refl _, Fraction.equiv_refl _})
  simpa only [prefixState, coarseAt_schedule] using hc

theorem prefix_state_le_two (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le (stateNorm (prefixState b w T s j))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hprefix := smallTime_prefix w h (ticks b j) (blocks j) hT
    (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  have hc := coarse_state_le_two w h s (ticks b j) hT hprefix
  exact Fraction.le_equiv_left
    (stateNorm_equiv (prefix_coarse b w T s j)) hc

theorem elapsed_replicate (d : Fraction) :
  (n : Nat) → Fraction.equiv (elapsed (List.replicate n d))
    (Fraction.mul (Fraction.ofInt (n : Int)) d)
  | 0 => by
    simpa only [elapsed, List.replicate_zero, Fraction.equiv,

```

```

    Fraction.ofInt, Fraction.mul]
  simp
| n + 1 => by
  have ih := elapsed_replicate d n
  have he := Fraction.add_equiv (Fraction.equiv_refl d) ih
  apply Fraction.equiv_trans he
  simp only [elapsed, List.replicate_succ, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, Int.natCast_add]
  simp only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

theorem prefix_elapsed (b : Nat → Bool) (T : Fraction) (j : Nat) :
  Fraction.equiv
    (elapsed (List.replicate (ticks b j) (duration T j)))
    (Fraction.mul (Fraction.ofInt (ticks b j : Int)) (duration T j)) :=
  elapsed_replicate (duration T j) (ticks b j)

theorem prefix_elapsed_le_time (b : Nat → Bool) (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.le (elapsed (List.replicate (ticks b j) (duration T j))) T := by
  let d := duration T j
  have hcount : Fraction.le (Fraction.ofInt (ticks b j : Int))
    (Fraction.ofInt (blocks j : Int)) := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simp only [Int.mul_one]
    exact Int.ofNat_le.mpr (ticks_le_blocks b j)
  have hm := Fraction.mul_le_mul_nonnegative hcount d hT
  have hfull := endpoint_elapsed T j
  have he := elapsed_replicate d (blocks j)
  have hbound := Fraction.le_equiv_left
    (prefix_elapsed b T j) hm
  exact Fraction.le_equiv_right hbound
    (Fraction.equiv_trans (Fraction.equiv_symm he) hfull)

theorem schedule_replicate_step (w h : Fraction) (s : Point × Point) :
  (n : Nat) →
    schedule (linearField w) (List.replicate (n + 1) h) s =
      cell (linearField w) h
        (schedule (linearField w) (List.replicate n h) s)
| 0 => rfl
| n + 1 => by
  simp only [List.replicate_succ, schedule]
  change schedule (linearField w) (List.replicate (n + 1) h)
    (cell (linearField w) h s) =
    cell (linearField w) h
      (schedule (linearField w) (List.replicate n h)
        (cell (linearField w) h s))
  rw [schedule_replicate_step w h (cell (linearField w) h s) n]

theorem prefix_next (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) :
  prefixState b w T s (j + 1) =
  if b j then
    cell (linearField w) (duration T (j + 1))
      (fineAt w (duration T (j + 1)) s (ticks b j))
  else fineAt w (duration T (j + 1)) s (ticks b j) := by
  let h := duration T (j + 1)
  by_cases hb : b j
  · simp only [prefixState, ticks_next]
    simp only [bit, hb, reduceIte]
    change schedule (linearField w)
      (List.replicate ((ticks b j + ticks b j) + 1) h) s = _
    rw [schedule_replicate_step]
    rw [← fineDurations_replicate]
    exact congrArg (cell (linearField w) h)
      (fineAt_schedule w h s (ticks b j))
  · simp only [prefixState, ticks_next]
    simp [bit, hb] at *
    rw [← fineDurations_replicate]

```

```

exact fineAt_schedule w h s (ticks b j)

private theorem scalar_one_bound (a b c : Fraction)
  (ha : 0 ≤ a.num) :
  Fraction.le
    (Fraction.add b (Fraction.mul c (Fraction.add a b)))
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) c)
      (Fraction.add a b)) := by
  let M := Fraction.add a b
  have hbM : Fraction.le b M := by
    unfold Fraction.le M Fraction.add
    dsimp
    rw [Int.add_mul]
    have hnon := Int.mul_nonneg
      (Int.mul_nonneg ha (Int.le_of_lt b.den_pos)) (Int.le_of_lt b.den_pos)
    have he : b.num * (a.den * b.den) = b.num * a.den * b.den := by ac_refl
    rw [he]
    omega
  have hfirst := Fraction.add_le_add_right hbM (Fraction.mul c M)
  apply Fraction.le_equiv_right hfirst
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  simp only [M, Fraction.add]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem cell_parameter_bound_one (w sigma tau : Fraction) (s : Point × Point)
  (ho : 0 ≤ sigma.num) (ht : 0 ≤ tau.num)
  (hsum : Fraction.le (Fraction.add sigma tau) (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub (cell (linearField w) tau s)
      (cell (linearField w) sigma s)))
    (Fraction.mul (durationDifference sigma tau).abs
      (Fraction.mul
        (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm s))) := by
  let d := durationDifference sigma tau
  have hp := short_sum_point_bound sigma tau s.1 s.2 ho ht hsum
  have hw := Fraction.mul_le_mul_nonnegative_left hp w.abs
    (Fraction.abs_num_nonnegative w)
  have ha := Fraction.add_le_add_left hw (pointNorm s.2)
  have hd := Fraction.mul_le_mul_nonnegative_left ha d.abs
    (Fraction.abs_num_nonnegative d)
  have hc := scalar_one_bound (pointNorm s.1) (pointNorm s.2) w.abs
    (pointNorm_nonnegative s.1)
  have hdc := Fraction.mul_le_mul_nonnegative_left hc d.abs
    (Fraction.abs_num_nonnegative d)
  have hf := cell_parameter_norm_formula w sigma tau s
  exact Fraction.le_equiv_right
    (Fraction.magnitudes.le_trans (Fraction.le_equiv_left hf hd) hdc)
    (by simp only [stateNorm]; exact Fraction.equiv_refl _)

private def zero : Fraction := {0, 1, by decide}

theorem cell_increment_bound (w h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) (hsmall : Fraction.le h (Fraction.ofInt 1)) :
  Fraction.le (stateNorm (stateSub (cell (linearField w) h s) s))
    (Fraction.mul h.abs
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm s))) := by
  have hsum : Fraction.le (Fraction.add zero h) (Fraction.ofInt 1) :=
    Fraction.le_equiv_left (by
      simp only [zero, Fraction.equiv, Fraction.add]
      simp) hsmall
  have hb := cell_parameter_bound_one w zero h s (by decide) hh hsum
  have he : stateEquiv
    (stateSub (cell (linearField w) h s) s)
    (stateSub (cell (linearField w) h s) (cell (linearField w) zero s)) :=
    stateSub_congr
      ((Fraction.equiv_refl _, Fraction.equiv_refl _),
        (Fraction.equiv_refl _, Fraction.equiv_refl _))
      ((Fraction.equiv_symm (zero_step w s).1.1,

```

```

      Fraction.equiv_symm (zero_step w s).1.2),
      (Fraction.equiv_symm (zero_step w s).2.1,
       Fraction.equiv_symm (zero_step w s).2.2))
have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hb
apply Fraction.le_equiv_right hfirst
simp only [durationDifference, zero, negF, Fraction.equiv,
  Fraction.abs, Fraction.add, Fraction.mul]
simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
  Int.natAbs_zero, Int.ofNat_zero, Int.mul_one, Int.one_mul,
  Int.neg_zero]

theorem fine_prefix_state_le_two (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (fineAt w (duration T (j + 1)) s (ticks b j)))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hprefix := smallTime_prefix w h (ticks b j) (blocks j) hT
  (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  exact fine_state_le_two w h s (ticks b j) hT hprefix

theorem prefix_totalTime_le (b : Nat → Bool) (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.le (totalTime (duration T (j + 1)) (ticks b j)) T := by
  let h := duration T (j + 1)
  have he : Fraction.equiv (totalTime h (ticks b j))
    (elapsed (List.replicate (ticks b j) (duration T j))) :=
  Fraction.equiv_trans
    (Fraction.equiv_symm (coarse_elapsed_totalTime h (ticks b j)))
    (elapsed_replicate_congr
      (Fraction.equiv_symm (duration_halving T j)) (ticks b j))
  exact Fraction.le_equiv_left he (prefix_elapsed_le_time b T j hT)

theorem fine_optional_increment (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub
      (cell (linearField w) (duration T (j + 1))
        (fineAt w (duration T (j + 1)) s (ticks b j)))
      (fineAt w (duration T (j + 1)) s (ticks b j))))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (duration T (j + 1))
        (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs)
          (stateNorm s)))) := by
  let h := duration T (j + 1)
  let q := fineAt w h s (ticks b j)
  have hle : Fraction.le h (Fraction.ofInt 1) := by
    have h₁ := duration_le_time T (j + 1) hT
    have h₂ := dyadic_time_le_half w T hT hs
    have h₃ : Fraction.le ((1, 2, by decide) : Fraction)
      (Fraction.ofInt 1) := by unfold Fraction.le; decide
    exact Fraction.magnitudes.le_trans h₁
      (Fraction.magnitudes.le_trans h₂ h₃)
  have hb := cell_increment_bound w h q hT hle
  have hq := fine_prefix_state_le_two b w T s j hT hs
  have hm₁ := Fraction.mul_le_mul_nonnegative_left hq
    (Fraction.add (Fraction.ofInt 1) w.abs) (by
      unfold Fraction.add Fraction.ofInt Fraction.abs
      dsimp
      have hw := Int.ofNat_nonneg w.num.natAbs
      have hd := w.den_pos
      omega)
  have hm₂ := Fraction.mul_le_mul_nonnegative_left hm₁ h hT
  have habs := Fraction.abs_of_nonnegative h hT
  have he := Fraction.mul_equiv habs
    (Fraction.equiv_refl
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm q)))
  have hchain := Fraction.magnitudes.le_trans

```

```

    (Fraction.le_equiv_right hb he) hm₂
  apply Fraction.le_equiv_right hchain
  simp only [h]
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

def refinementCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul T
      (Fraction.mul (duration T (j + 1))
        (Fraction.mul w.abs (stateNorm s))))

def optionalCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (duration T (j + 1))
      (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs)
        (stateNorm s)))

theorem prefix_refinement_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub
      (fineAt w (duration T (j + 1)) s (ticks b j))
      (prefixState b w T s j)))
    (refinementCap w T s j) := by
  let h := duration T (j + 1)
  let n := ticks b j
  have hs' := smallTime_prefix w h n (blocks j) hT
  (ticks_le_blocks b j) (dyadic_smallTime w T j hs)
  have hb := actual_uniform_error w h s n hT hs'
  have he : stateEquiv
    (stateSub (fineAt w h s n) (prefixState b w T s j))
    (stateSub (fineAt w h s n) (coarseAt w h s n)) :=
  stateSub_congr
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
     (Fraction.equiv_refl _, Fraction.equiv_refl _))
    (prefix_coarse b w T s j)
  have hfirst := Fraction.le_equiv_left (stateNorm_equiv he) hb
  have htime := prefix_totalTime_le b T j hT
  have hnon : 0 ≤ (Fraction.mul h (Fraction.mul w.abs (stateNorm s))).num :=
  Int.mul_nonneg hT
    (Int.mul_nonneg (Fraction.abs_num_nonnegative w)
      (stateNorm_nonnegative s))
  have hm₁ := Fraction.mul_le_mul_nonnegative htime
    (Fraction.mul h (Fraction.mul w.abs (stateNorm s))) hnon
  have hm₂ := Fraction.mul_le_mul_nonnegative_left hm₁
    (Fraction.ofInt 3) (by decide)
  exact Fraction.magnitudes.le_trans hfirst hm₂

def adjacentCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  Fraction.add (optionalCap w T s j) (refinementCap w T s j)

private theorem optionalCap_nonnegative (w T : Fraction) (s : Point × Point)
  (j : Nat) (hT : 0 ≤ T.num) :
  0 ≤ (optionalCap w T s j).num := by
  unfold optionalCap
  have hfactor : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
  unfold Fraction.add Fraction.ofInt Fraction.abs
  dsimp
  have hw := Int.ofNat_nonneg w.num.natAbs
  have hd := w.den_pos
  omega
  exact Int.mul_nonneg (by decide)
    (Int.mul_nonneg hT
      (Int.mul_nonneg hfactor (stateNorm_nonnegative s)))

private theorem le_add_optional (a c : Fraction) (hc : 0 ≤ c.num) :
  Fraction.le a (Fraction.add c a) := by
  have hz : Fraction.le (Fraction.ofInt 0) c := by

```

```

    unfold Fraction.le Fraction.ofInt
    dsimp
    simp only [Int.zero_mul, Int.mul_one]
    exact hc
  have h := Fraction.add_le_add_left hz a
  have he : Fraction.equiv (Fraction.add a (Fraction.ofInt 0)) a := by
    simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
    simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
      Int.mul_one, Int.one_mul]
  exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm he) h)
    (Fraction.add_comm a c)

theorem adjacent_error_le_add (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + 1))
      (prefixState b w T s j)))
    (adjacentCap w T s j) := by
  have hnext := prefix_next b w T s j
  have href := prefix_refinement_error b w T s j hT hs
  by_cases hb : b j
  · rw [hnext, if_pos hb]
    have htri := stateSub_triangle
      (cell (linearField w) (duration T (j + 1))
        (fineAt w (duration T (j + 1)) s (ticks b j)))
      (fineAt w (duration T (j + 1)) s (ticks b j))
      (prefixState b w T s j)
    exact Fraction.magnitudes.le_trans htri
      (Fraction.add_le_add (fine_optional_increment b w T s j hT hs) href)
  · rw [hnext, if_neg hb]
    exact Fraction.magnitudes.le_trans href
      (le_add_optional _ _ (optionalCap_nonnegative w T s j hT))

/-- `A=T*M*(2*(1+|w|)+3*T*|w|)` for intermediate prefixes. -/
def coefficient (w T : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul T
    (Fraction.mul (stateNorm s)
      (Fraction.add
        (Fraction.mul (Fraction.ofInt 2)
          (Fraction.add (Fraction.ofInt 1) w.abs))
        (Fraction.mul (Fraction.ofInt 3) (Fraction.mul T w.abs))))

def tailCap (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  let A := coefficient w T s
  (A.num, A.den * (2 : Int) ^ j,
    Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

def doubleTail (w T : Fraction) (s : Point × Point) (j : Nat) : Fraction :=
  let A := coefficient w T s
  (2 * A.num, A.den * (2 : Int) ^ j,
    Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

theorem adjacentCap_tail (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (adjacentCap w T s j) (tailCap w T s (j + 1)) := by
  simp only [adjacentCap, optionalCap, refinementCap, tailCap,
    coefficient, duration, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt, Fraction.abs,
    Int.pow_succ]
  simp only [Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf

theorem adjacent_error_le (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + 1))
      (prefixState b w T s j)))
    (tailCap w T s (j + 1)) :=

```



```

Fraction.le_equiv_right (adjacent_error_le_add b w T s j hT hs)
  (adjacentCap_tail w T s j)

theorem coefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (coefficient w T s).num := by
  have hOneW : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
    unfold Fraction.add Fraction.ofInt Fraction.abs
    dsimp
    have hw := Int.ofNat_nonneg w.num.natAbs
    have hd := w.den_pos
    omega
  have h₁ : 0 ≤ (Fraction.mul (Fraction.ofInt 2)
    (Fraction.add (Fraction.ofInt 1) w.abs)).num :=
    Int.mul_nonneg (by decide) hOneW
  have h₂ : 0 ≤ (Fraction.mul (Fraction.ofInt 3)
    (Fraction.mul T w.abs)).num :=
    Int.mul_nonneg (by decide)
      (Int.mul_nonneg hT (Fraction.abs_num_nonnegative w))
  unfold coefficient
  exact Int.mul_nonneg hT
    (Int.mul_nonneg (stateNorm_nonnegative s)
      (Fraction.nonnegative_add _ _ h₁ h₂))

theorem tail_halving (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv
    (Fraction.add (tailCap w T s (j + 1)) (tailCap w T s (j + 1)))
    (tailCap w T s j) := by
  exact GeometricTail.tail_halving (coefficient w T s) j

theorem tail_double (w T : Fraction) (s : Point × Point) (j : Nat) :
  Fraction.equiv (Fraction.add (tailCap w T s j) (tailCap w T s j))
    (doubleTail w T s j) := by
  exact GeometricTail.tail_double (coefficient w T s) j

theorem finite_gap_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (k j : Nat) → Fraction.le
    (stateNorm (stateSub (prefixState b w T s (j + k))
      (prefixState b w T s j))) (tailCap w T s j) := by
  intro k j
  exact GeometricTail.finite_gap (prefixState b w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => adjacent_error_le b w T s i hT hs) k j

theorem two_sided_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le
    (stateNorm (stateSub (prefixState b w T s m) (prefixState b w T s n)))
    (doubleTail w T s N) := by
  exact GeometricTail.two_sided (prefixState b w T s) (coefficient w T s)
    (coefficient_nonnegative w T s hT)
    (fun i => adjacent_error_le b w T s i hT hs) N m n hm hn

def modulus (w T : Fraction) (s : Point × Point) (eps : Fraction) : Nat :=
  (2 * (coefficient w T s).num * eps.den).toNat

theorem doubleTail_lt_tolerance (w T : Fraction) (s : Point × Point)
  (eps : Fraction) (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail w T s (modulus w T s eps)) eps := by
  exact GeometricTail.doubleTail_lt_tolerance (coefficient w T s) eps
    (coefficient_nonnegative w T s hT) heps

theorem prefix_cauchy (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  ∀ eps : Fraction, 0 < eps.num →
    ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
      Fraction.lt
        (stateNorm (stateSub (prefixState b w T s m)
          (prefixState b w T s n))) eps := by

```

```

intro eps heps
refine (modulus w T s eps, ?_)
intro m n hm hn
exact Fraction.magnitudes.lt_of_le_lt
  (two_sided_error b w T s hT hs _ m n hm hn)
  (doubleTail_lt_tolerance w T s eps hT heps)

def prefixName (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : EndpointCauchyName where
  approx := prefixState b w T s
  cauchy := prefix_cauchy b w T s hT hs

theorem all_zero_prefix (w T : Fraction) (s : Point × Point) (j : Nat) :
  prefixState (fun _ => false) w T s j = s := by
  simp [prefixState, all_zero_ticks, schedule]

theorem zero_time_prefix (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : T.num = 0) :
  stateEquiv (prefixState b w T s j) s := by
  exact zero_duration_schedule w (duration T j) (by exact hT) (ticks b j) s

private def sampleOne : Fraction := {1, 1, by decide}
private def sampleZero : Fraction := {0, 1, by decide}
private def sampleQuarter : Fraction := {1, 4, by decide}
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))
private def firstBit (j : Nat) : Bool := j == 0

theorem sample_ticks : ticks firstBit 1 = 1 ∧ ticks firstBit 2 = 2 := by decide

theorem sample_first_error :
  Fraction.equiv
    (stateNorm (stateSub
      (prefixState firstBit sampleOne sampleQuarter sampleState 1)
      (prefixState firstBit sampleOne sampleQuarter sampleState 0)))
    (17, 64, by decide) := by decide

theorem sample_coefficient :
  Fraction.equiv (coefficient sampleOne sampleQuarter sampleState)
    (19, 8, by decide) := by decide

theorem sample_second_error :
  Fraction.equiv
    (stateNorm (stateSub
      (prefixState firstBit sampleOne sampleQuarter sampleState 2)
      (prefixState firstBit sampleOne sampleQuarter sampleState 1)))
    (545, 65536, by decide) := by decide

def sample_prefix_cauchy_name : EndpointCauchyName :=
  prefixName firstBit sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)

end NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

```

NewtonLimitDynamics/Polygon/HarmonicTimeRealization.lean, 30 theorems, 502 lines

```

import NewtonLimitDynamics.Polygon.CauchyValues
import NewtonLimitDynamics.Polygon.BinaryTime
import BarrowLib.Polygon.ScaledTolerance
import BarrowLib.Polygon.BinaryEndpoints
import BarrowLib.Polygon.FiniteSequenceGap

/-!
Actual harmonic state values indexed by the constructed binary-time quotient.
All comparisons use finite same-grid end-kick schedules. The quotient is not
identified with an external real interval; force identification and actual
intervening-region area remain separate.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicTimeRealization

```

```

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicComparison
open HarmonicAccumulation
open HarmonicUniform
open HarmonicDyadic
open HarmonicBinaryPrefix
open HarmonicTimeComparison
open PointBounds
open CauchyValues
open BinaryTime

def countState (w T : Fraction) (s : Point × Point)
  (j n : Nat) : Point × Point :=
  schedule (linearField w) (List.replicate n (duration T j)) s

theorem countState_coarse (w T : Fraction) (s : Point × Point)
  (j n : Nat) :
  stateEquiv (countState w T s j n)
    (coarseAt w (duration T (j + 1)) s n) := by
  let h := duration T (j + 1)
  have hc := schedule_replicate_congr w (duration T j) (Fraction.add h h)
    (duration_halving T j) n s
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
     (Fraction.equiv_refl _, Fraction.equiv_refl _))
  simpa only [countState, coarseAt_schedule] using hc

theorem countState_le_two (w T : Fraction) (s : Point × Point)
  (j n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hn : n ≤ blocks j) :
  Fraction.le (stateNorm (countState w T s j n))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  let h := duration T (j + 1)
  have hprefix := smallTime_prefix w h n (blocks j) hT hn
    (dyadic_smallTime w T j hs)
  exact Fraction.le_equiv_left (stateNorm_equiv (countState_coarse w T s j n))
    (coarse_state_le_two w h s n hT hprefix)

def stateTimeFactor (w : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm s))

theorem countState_step_bound (w T : Fraction) (s : Point × Point)
  (j n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hn : n ≤ blocks j) :
  Fraction.le (distance (countState w T s j (n + 1))
    (countState w T s j n))
    (Fraction.mul (duration T j) (stateTimeFactor w s)) := by
  let h := duration T j
  let q := countState w T s j n
  have hle : Fraction.le h (Fraction.ofInt 1) := by
    have h₁ := duration_le_time T j hT
    have h₂ := dyadic_time_le_half w T hT hs
    have h₃ : Fraction.le ((1, 2, by decide) : Fraction)
      (Fraction.ofInt 1) := by unfold Fraction.le; decide
    exact Fraction.magnitudes.le_trans h₁
      (Fraction.magnitudes.le_trans h₂ h₃)
  have hb := cell_increment_bound w h q hT hle
  have hq := countState_le_two w T s j n hT hs hn
  have hfac : 0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num := by
    unfold Fraction.add Fraction.ofInt Fraction.abs
    dsimp
    have hw := Int.ofNat_nonneg w.num.natAbs
    have hd := w.den_pos
    omega
  have hm₁ := Fraction.mul_le_mul_nonnegative_left hq
    (Fraction.add (Fraction.ofInt 1) w.abs) hfac
  have hm₂ := Fraction.mul_le_mul_nonnegative_left hm₁ h hT

```

```

have habs := Fraction.abs_of_nonnegative h hT
have he := Fraction.mul_equiv habs
  (Fraction.equiv_refl
   (Fraction.mul (Fraction.add (Fraction.ofInt 1) w.abs) (stateNorm q)))
have hchain := Fraction.magnitudes.le_trans
  (Fraction.le_equiv_right hb he) hm₂
have hstep : countState w T s j (n + 1) = cell (linearField w) h q :=
  schedule_replicate_step w h s n
rw [hstep]
change Fraction.le (stateNorm (stateSub (cell (linearField w) h q) q))
  (Fraction.mul h (stateTimeFactor w s))
apply Fraction.le_equiv_right hchain
simp only [stateTimeFactor, Fraction.equiv, Fraction.mul, Fraction.ofInt]
ac_nf

theorem countState_gap (w T : Fraction) (s : Point × Point)
  (j : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  (n k : Nat) → n + k ≤ blocks j →
  Fraction.le (distance (countState w T s j (n + k))
    (countState w T s j n))
    (Fraction.mul (Fraction.ofInt (k : Int))
      (Fraction.mul (duration T j) (stateTimeFactor w s)))
| n, k, hnk => FiniteSequenceGap.finite_gap distance stateSub_self_norm_zero
  stateSub_triangle (countState w T s j) (blocks j)
  (Fraction.mul (duration T j) (stateTimeFactor w s))
  (fun i hi => countState_step_bound w T s j i hT hs (Nat.le_of_lt hi)) n k hnk

theorem stateTimeFactor_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (stateTimeFactor w s).num := by
  unfold stateTimeFactor Fraction.mul Fraction.add Fraction.ofInt Fraction.abs
  dsimp
  have hw := Int.ofNat_nonneg w.num.natAbs
  have hd := w.den_pos
  have hm := stateNorm_nonnegative s
  exact Int.mul_nonneg (by decide)
    (Int.mul_nonneg (by omega) hm)

theorem countState_ordered_bound (w T : Fraction) (s : Point × Point)
  (j n k : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hnk : n + k ≤ blocks j) :
  Fraction.le (distance (countState w T s j (n + k))
    (countState w T s j n))
    (Fraction.mul
      (durationDifference (countTime T j n) (countTime T j (n + k))).abs
      (stateTimeFactor w s)) := by
  have hb := countState_gap w T s j hT hs n k hnk
  apply Fraction.le_equiv_right hb
  have he := countTime_abs_difference T j n k hT
  apply Fraction.equiv_symm
  apply Fraction.equiv_trans
  (Fraction.mul_equiv he (Fraction.equiv_refl _))
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem countState_same_grid_bound (w T : Fraction) (s : Point × Point)
  (j m n : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hm : m ≤ blocks j) (hn : n ≤ blocks j) :
  Fraction.le (distance (countState w T s j m)
    (countState w T s j n))
    (Fraction.mul
      (durationDifference (countTime T j n) (countTime T j m)).abs
      (stateTimeFactor w s)) := by
  rcases Nat.le_total n m with hnm | hmn
  · have he : n + (m - n) = m := by omega
    have hb := countState_ordered_bound w T s j n (m - n) hT hs
    (by simpa only [he] using hb)
    simpa only [he] using hb
  · have he : m + (n - m) = n := by omega
    have hb := countState_ordered_bound w T s j m (n - m) hT hs
    (by simpa only [he] using hn)

```

```

    rw [he] at hb
    have hd := stateSub_norm_symm (countState w T s j m)
      (countState w T s j n)
    have ht := durationDifference_abs_symm (countTime T j m)
      (countTime T j n)
    exact Fraction.le_equiv_right
      (Fraction.le_equiv_left hd hb)
      (Fraction.mul_equiv ht (Fraction.equiv_refl _))

theorem prefix_time_bound (b c : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  Fraction.le (distance (prefixState b w T s j)
    (prefixState c w T s j))
    (Fraction.mul (distance (timeState b T j) (timeState c T j))
      (stateTimeFactor w s)) := by
  have hb := countState_same_grid_bound w T s j (ticks b j) (ticks c j)
  hT hs (ticks_le_blocks b j) (ticks_le_blocks c j)
  apply Fraction.le_equiv_right hb
  exact Fraction.mul_equiv
    (Fraction.equiv_symm (scalarState_distance
      (timeApprox b T j) (timeApprox c T j)))
    (Fraction.equiv_refl _))

theorem address_state_equiv (b c : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T)
  (hbc : AddressEquiv T hT b c) :
  NameEquiv (prefixName b w T s hT hs)
    (prefixName c w T s hT hs) := by
  intro eps heps
  let C := stateTimeFactor w s
  have hC := stateTimeFactor_nonnegative w s
  let delta := factorDelta C eps hC
  obtain (N, hN) := hbc delta (factorDelta_positive C eps hC heps)
  refine (N, ?_)
  intro j hj
  have hb := prefix_time_bound b c w T s j hT hs
  have hsmall := factor_control C eps
    (distance (timeState b T j) (timeState c T j)) hC
    (stateNorm_nonnegative _) (hN j hj)
  exact Fraction.magnitudes.lt_of_le_lt hb hsmall

def gammaValue (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  BinaryTime T hT → Value :=
  Quotient.lift
    (fun b => binaryValue b w T s hT hs)
    (fun b c hbc => Quotient.sound
      (address_state_equiv b c w T s hT hs hbc))

theorem gammaValue_address (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (Quotient.mk _ b) =
  binaryValue b w T s hT hs := rfl

theorem gamma_within (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (x y : BinaryTime T hT) (R : Fraction)
  (_hR : 0 ≤ R.num) (hxy : TimeWithin T hT x y R) :
  Within (gammaValue w T s hT hs x)
    (gammaValue w T s hT hs y)
    (Fraction.mul R (stateTimeFactor w s)) := by
  induction x using Quotient.inductionOn with
  | _ b =>
    induction y using Quotient.inductionOn with
    | _ c =>
      change NameBound (BinaryTime.timeName b T hT)
        (BinaryTime.timeName c T hT) R at hxy

```

```

    change NameBound (prefixName b w T s hT hs)
      (prefixName c w T s hT hs)
      (Fraction.mul R (stateTimeFactor w s))
  apply nameBound_scale (prefixName b w T s hT hs)
    (prefixName c w T s hT hs)
    (BinaryTime.timeName b T hT) (BinaryTime.timeName c T hT)
    (stateTimeFactor w s) R (stateTimeFactor_nonnegative w s)
  · intro j
    exact prefix_time_bound b c w T s j hT hs
  · exact hxy

def timeTolerance (w : Fraction) (s : Point × Point)
  (eps : Fraction) : Fraction :=
  factorDelta (stateTimeFactor w s) eps.half
  (stateTimeFactor_nonnegative w s)

theorem timeTolerance_positive (w : Fraction) (s : Point × Point)
  (eps : Fraction) (heps : 0 < eps.num) :
  0 < (timeTolerance w s eps).num :=
  factorDelta_positive _ _ heps

theorem gamma_uniform_continuity (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (eps : Fraction) (heps : 0 < eps.num)
  (x y : BinaryTime T hT)
  (hxy : TimeWithin T hT x y (timeTolerance w s eps)) :
  Within (gammaValue w T s hT hs x)
    (gammaValue w T s hT hs y) eps.half := by
  have hb := gamma_within w T s hT hs x y
    (timeTolerance w s eps)
  (Int.le_of_lt (timeTolerance_positive w s eps heps)) hxy
  exact within_mono _ _ _
    (factor_delta_weak _ _ (stateTimeFactor_nonnegative w s)
      (by simpa only [Fraction.half] using Int.le_of_lt heps)) hb

theorem duration_factor_eventually_small (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j : Nat, N ≤ j →
    Fraction.lt (Fraction.mul (duration T j) (stateTimeFactor w s))
      eps := by
  let C := stateTimeFactor w s
  let delta := factorDelta C eps (stateTimeFactor_nonnegative w s)
  obtain (N, hN) := duration_eventually_small T delta hT
  (factorDelta_positive C eps (stateTimeFactor_nonnegative w s) heps)
  refine (N, ?_)
  intro j hj
  exact factor_control C eps (duration T j)
    (stateTimeFactor_nonnegative w s) hT (hN j hj)

theorem left_endpoint_value (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (leftTime T hT) = embed s := by
  apply Quotient.sound
  intro eps heps
  refine {0, ?_}
  intro j _
  change Fraction.lt (distance (prefixState leftAddress w T s j) s) eps
  rw [show prefixState leftAddress w T s j = s from
    all_zero_prefix w T s j]
  exact distance_self_lt s eps heps

theorem right_prefix_endpoint_bound (w T : Fraction)
  (s : Point × Point) (j : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) :
  Fraction.le (distance (endpoint w T s j)
    (prefixState rightAddress w T s j))
    (Fraction.mul (duration T j) (stateTimeFactor w s)) := by
  have hcount : ticks rightAddress j + 1 ≤ blocks j :=
  Nat.le_of_eq (right_ticks j)

```

```

have hb := countState_gap w T s j hT hs (ticks rightAddress j) 1 hcount
rw [right_ticks] at hb
change Fraction.le (distance (endpoint w T s j)
  (prefixState rightAddress w T s j)) _ at hb
apply Fraction.le_equiv_right hb
simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt,
  Int.natCast_one,
  Int.one_mul, Int.mul_one]

theorem right_endpoint_value (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaValue w T s hT hs (rightTime T hT) =
  endpointValue w T s hT hs := by
  apply Quotient.sound
  intro eps heps
  obtain (N, hN) := duration_factor_eventually_small w T s hT eps heps
  refine (N, ?_)
  intro j hj
  have hb := right_prefix_endpoint_bound w T s j hT hs
  have hs := stateSub_norm_symm
    (prefixState rightAddress w T s j) (endpoint w T s j)
  exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_equiv_left hs hb) (hN j hj)

def firstAlias (j : Nat) : Bool := decide (j = 0)
def secondAlias (j : Nat) : Bool := decide (j ≠ 0)

theorem alias_ticks (j : Nat) :
  ticks firstAlias (j + 1) = blocks j ∧
  ticks secondAlias (j + 1) + 1 = blocks j := by
  induction j with
  | zero =>
    constructor <=> decide
  | succ j ih =>
    rcases ih with (hfirst, hsecond)
    constructor
    · change 2 * ticks firstAlias (j + 1) + bit firstAlias (j + 1) =
      blocks (j + 1)
      have hb : bit firstAlias (j + 1) = 0 := by
        simp [bit, firstAlias]
      rw [hb, blocks_succ]
      omega
    · change 2 * ticks secondAlias (j + 1) +
      bit secondAlias (j + 1) + 1 = blocks (j + 1)
      have hb : bit secondAlias (j + 1) = 1 := by
        simp [bit, secondAlias]
      rw [hb, blocks_succ]
      omega

theorem alias_time_distance (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.equiv
    (distance (timeState firstAlias T (j + 1))
      (timeState secondAlias T (j + 1)))
    (duration T (j + 1)) := by
  have hcounts : ticks secondAlias (j + 1) + 1 =
    ticks firstAlias (j + 1) := by
    have ha := alias_ticks j
    omega
  have htime := countTime_abs_difference T (j + 1)
    (ticks secondAlias (j + 1)) 1 hT
  rw [hcounts] at htime
  have hd := scalarState_distance
    (timeApprox firstAlias T (j + 1))
    (timeApprox secondAlias T (j + 1))
  apply Fraction.equiv_trans hd
  apply Fraction.equiv_trans htime
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt,
    Int.natCast_one, Int.one_mul, Int.mul_one]

```

```

theorem alias_address_equiv (T : Fraction) (hT : 0 ≤ T.num) :
  AddressEquiv T hT firstAlias secondAlias := by
  intro eps heps
  obtain (N, hN) := duration_eventually_small T eps hT heps
  refine (N + 1, ?_)
  intro j hj
  have hj' : j - 1 + 1 = j := by omega
  have hbound := alias_time_distance T (j - 1) hT
  rw [hj'] at hbound
  exact Fraction.magnitudes.lt_of_le_lt
    ((Fraction.equiv_iff_mutual_le _ _).mp hbound).1
    (hN j (by omega))

theorem alias_time_eq (T : Fraction) (hT : 0 ≤ T.num) :
  (Quotient.mk (addressSetoid T hT) firstAlias : BinaryTime T hT) =
  Quotient.mk (addressSetoid T hT) secondAlias :=
  Quotient.sound (alias_address_equiv T hT)

theorem alias_value_eq (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  binaryValue firstAlias w T s hT hs =
  binaryValue secondAlias w T s hT hs := by
  exact congrArg (gammaValue w T s hT hs) (alias_time_eq T hT)

theorem zero_time_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hzero : T.num = 0) :
  gammaValue w T s hT hs (Quotient.mk _ b) = embed s := by
  apply Quotient.sound
  intro eps heps
  refine (0, ?_)
  intro j _
  have he := (distance_zero_iff_stateEquiv
    (prefixState b w T s j) s).mpr
    (zero_time_prefix b w T s j hzero)
  have hself := stateSub_self_norm_zero s
  have hle : Fraction.le (distance (prefixState b w T s j) s)
    (distance s s) := Fraction.le_of_equiv
    (Fraction.equiv_trans he (Fraction.equiv_symm hself))
  exact Fraction.magnitudes.lt_of_le_lt hle
    (distance_self_lt s eps heps)

theorem zero_state_norm_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hzero : (stateNorm s).num = 0) :
  gammaValue w T s hT hs (Quotient.mk _ b) = embed s := by
  have hC : (stateTimeFactor w s).num = 0 := by
    unfold stateTimeFactor Fraction.mul
    dsimp
    simp only [hzero, Int.mul_zero, Int.zero_mul]
  calc
    gammaValue w T s hT hs (Quotient.mk _ b) =
      gammaValue w T s hT hs (leftTime T hT) := by
        apply Quotient.sound
      intro eps heps
      refine (0, ?_)
      intro j _
      have hb := prefix_time_bound b leftAddress w T s j hT hs
      have hz : (Fraction.mul
        (distance (timeState b T j) (timeState leftAddress T j))
        (stateTimeFactor w s)).num = 0 := by
        unfold Fraction.mul
        dsimp
        simp [hC]
      have hstrict : Fraction.lt
        (Fraction.mul
          (distance (timeState b T j) (timeState leftAddress T j))
          (stateTimeFactor w s)) eps := by
        unfold Fraction.lt
        simp only [hz, Int.zero_mul]

```



```

    exact Int.mul_pos heps
    (Fraction.mul
      (distance (timeState b T j) (timeState leftAddress T j))
      (stateTimeFactor w s)).den_pos
    exact Fraction.magnitudes.lt_of_le_lt hb hstrict
    _ = embed s := left_endpoint_value w T s hT hs

private def sampleOne : Fraction := (1, 1, by decide)
private def sampleZero : Fraction := (0, 1, by decide)
private def sampleQuarter : Fraction := (1, 4, by decide)
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))

theorem sample_alias_counts :
  ticks firstAlias 2 = 2 ∧ ticks secondAlias 2 = 1 := by decide

theorem sample_alias_time_gap :
  Fraction.equiv
    (distance (timeState firstAlias sampleQuarter 2)
      (timeState secondAlias sampleQuarter 2))
    (1, 16, by decide) := by decide

theorem sample_state_time_factor :
  Fraction.equiv (stateTimeFactor sampleOne sampleState)
    (Fraction.ofInt 8) := by decide

theorem sample_alias_bound :
  Fraction.equiv
    (Fraction.mul (distance
      (timeState firstAlias sampleQuarter 2)
      (timeState secondAlias sampleQuarter 2))
      (stateTimeFactor sampleOne sampleState))
    (1, 2, by decide) := by decide

theorem sample_alias_actual_error :
  Fraction.equiv
    (distance
      (prefixState firstAlias sampleOne sampleQuarter sampleState 2)
      (prefixState secondAlias sampleOne sampleQuarter sampleState 2))
    (8927, 65536, by decide) := by decide

theorem sample_right_ne_left :
  gammaValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (rightTime sampleQuarter (by decide)) ≠
  gammaValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (leftTime sampleQuarter (by decide)) := by
  have hneq := CauchyValues.sample_endpoint_value_ne_initial
  change endpointValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide) ≠
    embed sampleState at hneq
  intro h
  apply hneq
  calc
    endpointValue sampleOne sampleQuarter sampleState
      (by decide) (by unfold DyadicSmallTime Fraction.le; decide) =
      gammaValue sampleOne sampleQuarter sampleState
        (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
        (rightTime sampleQuarter (by decide)) :=
      (right_endpoint_value _ _ _ _).symm
    _ = gammaValue sampleOne sampleQuarter sampleState
      (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
      (leftTime sampleQuarter (by decide)) := h
    _ = embed sampleState := left_endpoint_value _ _ _ _

end NewtonLimitDynamics.Polygon.HarmonicTimeRealization

```

NewtonLimitDynamics/Diagnostic/PhaseArea.lean, 9 theorems, 147 lines

```
import NewtonLimitDynamics.Polygon.TimeSubdivision
```

```

/-!
Action diagnostic layer (research/action-arguments Arg004). One-dimensional
phase plane `(q, v)` for the cell of Proposition I's finite construction:
inertial drift (Law I), then the impulse of the force at the arrival point
(Law II, laws' Corollary I). The construction is the same in De Motu
(NATP00089 par9, NATP00090 par17), 1687 (NATP00077 par45) and 1713
(NATP00082 par51). Modern rational reconstruction; no limit is taken.

Each step moves one coordinate by an amount depending only on the other
(a shear), so each line of fixed velocity (drift) or fixed position (impulse)
is translated rigidly: Cavalieri's comparison of equal chords. For affine
fields (constant force; force as distance) the doubled signed area of every
phase triangle is preserved exactly, at every cell duration. The exact
energy-type invariant of the constant-force cell depends on the duration.
-/

namespace NewtonLimitDynamics.Diagnostic.PhaseArea

open NewtonLimitDynamics
open NewtonLimitDynamics.Polygon.TimeSubdivision

/-- A phase point `(position, velocity)`. -/
abbrev Phase := Point

def negF (w : Fraction) : Fraction := {-w.num, w.den, w.den_pos}
def fsub (a b : Fraction) : Fraction := Fraction.add a (negF b)

/-- Law I drift for duration `d`. -/
def drift (d : Fraction) (z : Phase) : Phase := (Fraction.add z.1 (Fraction.mul d z.2), z.2)

/-- Impulse of an arbitrary field `F` at the current position. -/
def kick (F : Fraction → Fraction) (d : Fraction) (z : Phase) : Phase :=
  (z.1, Fraction.add z.2 (Fraction.mul d (F z.1)))

/-- One cell of the construction: drift, then the impulse at the arrival point. -/
def cell (F : Fraction → Fraction) (d : Fraction) (z : Phase) : Phase := kick F d (drift d z)

/-- `n` equal cells. -/
def cells (F : Fraction → Fraction) (d : Fraction) : Nat → Phase → Phase
| 0, z => z
| n + 1, z => cells F d n (cell F d z)

/-- Doubled signed area of a phase triangle. -/
def area2 (z0 z1 z2 : Phase) : Fraction := det (pointSub z1 z0) (pointSub z2 z0)

/-- Cavalieri, drift: two phase points with the same velocity keep their
position difference. Holds for every field. -/
theorem drift_rigid (d q q' v : Fraction) :
  Fraction.equiv (fsub (drift d (q', v)).1 (drift d (q, v)).1) (fsub q' q) := by
  unfold fsub negF drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Cavalieri, impulse: two phase points at the same position keep their
velocity difference, for an arbitrary field `F`. -/
theorem kick_rigid (F : Fraction → Fraction) (d q v v' : Fraction) :
  Fraction.equiv (fsub (kick F d (q, v')).2 (kick F d (q, v)).2) (fsub v' v) := by
  unfold fsub negF kick Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- The affine field `F(q) = c - w*q` : constant force (`w = 0`) or force as
distance toward the origin (`w > 0`). -/
def affine (c w : Fraction) : Fraction → Fraction :=
  fun q => Fraction.add c (Fraction.mul (negF w) q)

```

```

theorem drift_area2 (d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv (area2 (drift d z0) (drift d z1) (drift d z2)) (area2 z0 z1 z2) := by
  unfold area2 det pointSub pointNeg pointAdd drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

theorem kick_area2 (c w d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv (area2 (kick (affine c w) d z0) (kick (affine c w) d z1) (kick (affine c w) d z2))
    (area2 z0 z1 z2) := by
  unfold area2 det pointSub pointNeg pointAdd kick affine negF Fraction.equiv Fraction.add
  Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- One cell preserves the doubled phase area of every triangle, for every
duration. -/
theorem cell_area2 (c w d : Fraction) (z0 z1 z2 : Phase) :
  Fraction.equiv
    (area2 (cell (affine c w) d z0) (cell (affine c w) d z1) (cell (affine c w) d z2))
    (area2 z0 z1 z2) :=
  Fraction.equiv_trans (kick_area2 c w d _ _ _) (drift_area2 d z0 z1 z2)

/-- Every equal-cell schedule, at every mesh, preserves it. -/
theorem cells_area2 (c w d : Fraction) :
  (n : Nat) → (z0 z1 z2 : Phase) →
  Fraction.equiv
    (area2 (cells (affine c w) d n z0) (cells (affine c w) d n z1) (cells (affine c w) d n z2))
    (area2 z0 z1 z2)
  | 0, _, _, _ => Fraction.equiv_refl _
  | n + 1, z0, z1, z2 =>
    Fraction.equiv_trans (cells_area2 c w d n _ _ _) (cell_area2 c w d z0 z1 z2)

/-- The exact energy-type invariant of the constant-force cell,
`v^2 - 2*a*q - d*a*v`, involves the cell duration `d`. -/
def energyD (a d : Fraction) (z : Phase) : Fraction :=
  fsub (fsub (Fraction.mul z.2 z.2) (Fraction.add (Fraction.mul a z.1) (Fraction.mul a z.1)))
    (Fraction.mul d (Fraction.mul a z.2))

theorem cell_energyD (a d : Fraction) (z : Phase) :
  Fraction.equiv (energyD a d (cell (fun _ => a) d z)) (energyD a d z) := by
  unfold energyD fsub negF cell kick drift Fraction.equiv Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

def one : Fraction := Fraction.ofInt 1
def zero : Fraction := Fraction.ofInt 0
def half : Fraction := {1, 2, by decide}

/-- The duration-dependent invariant differs between meshes at the same state
(`a = 1`, state `(0, 1)`): `0` for `d = 1`, `1/2` for `d = 1/2`. -/
theorem energyD_depends_on_mesh :
  ~ Fraction.equiv (energyD one one (zero, one)) (energyD one half (zero, one)) := by
  decide

/-- Refinement example, force as distance (`c = 0`, `w = 1`): one cell of
duration 1 and two cells of duration 1/2 send the phase point `(1, 0)` to
different places, while both keep the unit triangle's doubled area 1. -/
theorem refinement_moves_points_keeps_area :
  ~ pointEquiv (cells (affine zero one) one 1 (one, zero))
    (cells (affine zero one) half 2 (one, zero)) ∧
  Fraction.equiv
    (area2 (cells (affine zero one) one 1 (zero, zero)) (cells (affine zero one) one 1 (one, zero))
      (cells (affine zero one) one 1 (zero, one))) one 1

```

```

    Fraction.equiv
      (area2 (cells (affine zero one) half 2 (zero, zero)) (cells (affine zero one) half 2 (one, zero))
        (cells (affine zero one) half 2 (zero, one))) one := by
    decide
end NewtonLimitDynamics.Diagnostic.PhaseArea

```

Proposition II. Theorem II. *Every body that moves in some curved line and, by a radius drawn to a point either immovable or moving uniformly in a straight line, describes about that point areas proportional to the times, is urged by a centripetal force tending to that same point.*

In modern terms: if $\mathbf{r} \times \dot{\mathbf{r}}$ is constant about S , the force is parallel to $\mathbf{r}$.

Case 1. Every body moving in a curved line is turned from a rectilinear course by some force acting on it (Law I). The force by which the body is turned from its rectilinear course and made to describe, about the immovable point S , the least triangles SAB , SBC , SCD , ... equal in equal times, acts at the place B along a line parallel to cC (Euclid I.40 and Law II), that is, along the line BS ; and at the place C along a line parallel to dD , that is, along the line CS ; and so on. It therefore always acts along lines tending to that immovable point S .

Case 2. By Corollary V of the Laws it makes no difference whether the surface on which the body describes the curvilinear figure rests, or moves uniformly in a straight line together with the body, the figure described and its point S .

1713. The statement specifies a curved line *described in a plane*. Two corollaries are added, rewritten from the old corollaries of Proposition I.

Corollary 1. In non-resisting spaces or media, if the areas are not proportional to the times, the forces do not tend to the point where the radii meet, but decline from it *in consequentia*, toward the direction of motion, if the description of areas is accelerated, and *in antecedentia* if it is retarded.

Corollary 2. In resisting media too, if the description of areas is accelerated, the directions of the forces decline from the point where the radii meet toward the direction of motion.

Lean reconstruction: 12 theorems in 1 module cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Polygon/Converse.lean, 12 theorems, 141 lines

```
import NewtonLimitDynamics.Polygon.Finite
```

```

/-!
Finite converse of the equal-area polygon construction (motivated by 1687
Proposition II, NATP00077 par48–49, and 1713 Proposition II, NATP00082
par58–59). Modern integer-coordinate reconstruction with the centre  $S$  at the
origin. Given three consecutive vertices  $p, q, C$  and the inertial
continuation  $c = \text{extend } p \ q$ , equality of the oriented doubled areas
 $\text{Sq } p$  and  $\text{Sq } C$  makes the deflection  $C - c$  parallel to  $\text{Sq } p$ , and the step is
then a central kick with a rational impulse. Orientation (Euclid's "same
side"), a vertex distinct from  $S$ , and the inward sense are separate premises;
counterexamples show each is needed. No curve, limit or force law is produced.
-/

```

```
namespace NewtonLimitDynamics.Polygon.Converse
```

```
open NewtonLimitDynamics.Polygon
```

```

def sub (x y : LatticePoint) : LatticePoint := (x.1 - y.1, x.2 - y.2)

/-- The deflection from the inertial continuation `c` of `pq` to the actual
    next vertex `C` (Newton's segment `cC`). -/
def deflection (p q C : LatticePoint) : LatticePoint := sub C (extend p q)

theorem det_sub (q x y : LatticePoint) : det q (sub x y) = det q x - det q y := by
  simp only [det, sub, Int.mul_sub]
  omega

/-- The deflection's determinant with the radius `Sq` is the change of oriented
    area. -/
theorem det_deflection (p q C : LatticePoint) :
  det q (deflection p q C) = det q C - det p q := by
  unfold deflection
  rw [det_sub, extension_identity]

/-- Coordinate form of the Euclid I.39/I.40 step: equal oriented areas make the
    deflection `cC` parallel to `Sq`. -/
theorem equal_area_parallel (p q C : LatticePoint) (h : det q C = det p q) :
  det q (deflection p q C) = 0 := by
  rw [det_deflection, h, Int.sub_self]

/-- And conversely a deflection parallel to `Sq` preserves the oriented area. -/
theorem parallel_equal_area (p q C : LatticePoint) (h : det q (deflection p q C) = 0) :
  det q C = det p q := by
  rw [det_deflection] at h
  omega

/-- A vector parallel to a nonzero radius is a rational multiple of it:
    `b*d = a*q` with `b ≠ 0`. -/
theorem parallel_is_multiple (q d : LatticePoint) (hq : q ≠ (0, 0)) (h : det q d = 0) :
  ∃ a b : Int, b ≠ 0 ∧ b * d.1 = a * q.1 ∧ b * d.2 = a * q.2 := by
  unfold det at h
  by_cases h1 : q.1 = 0
  · have h2 : q.2 ≠ 0 := by
      intro h2
      apply hq
      exact Prod.ext h1 h2
    refine (d.2, q.2, h2, _, _)
    · rw [h1, Int.mul_zero]
      rw [h1, Int.zero_mul] at h
      have : q.2 * d.1 = 0 := by omega
      exact this
    · exact Int.mul_comm _ _
  · refine (d.1, q.1, h1, Int.mul_comm _ _, _)
    have : q.1 * d.2 = q.2 * d.1 := by omega
    rw [this, Int.mul_comm]

/-- Finite converse for one step: equal oriented areas and a vertex distinct
    from S give a rational central impulse `a/b` with `b*c = b*c + a*q`. -/
theorem equal_area_central_step (p q C : LatticePoint) (hq : q ≠ (0, 0))
  (h : det q C = det p q) :
  ∃ a b : Int, b ≠ 0 ∧
    b * C.1 = b * (extend p q).1 + a * q.1 ∧ b * C.2 = b * (extend p q).2 + a * q.2 := by
  obtain (a, b, hb, h1, h2) :=
    parallel_is_multiple q (deflection p q C) hq (equal_area_parallel p q C h)
  refine (a, b, hb, _, _)
  · simp only [deflection, sub, Int.mul_sub] at h1
    omega
  · simp only [deflection, sub, Int.mul_sub] at h2
    omega

/-- A finite vertex sequence whose consecutive oriented triangles about S are
    all equal (equal areas in equal time cells) has every deflection parallel to
    its current radius. -/
theorem equal_areas_all_central (v : Nat → LatticePoint)
  (h : ∀ n, det (v (n + 1)) (v (n + 2)) = det (v n) (v (n + 1))) (n : Nat) :
  det (v (n + 1)) (deflection (v n) (v (n + 1)) (v (n + 2))) = 0 :=

```

```

equal_area_parallel _ _ _ (h n)

/-- Orientation is needed: equal unsigned areas (`det q C = -det p q`) admit a
deflection that is not parallel to `Sq`. -/
theorem unsigned_equal_area_not_central :
  det (1, 1) (2, 1) = -det (1, 0) (1, 1) ∧
  det (1, 1) (deflection (1, 0) (1, 1) (2, 1)) ≠ 0 := by
  decide

/-- A vertex at S is degenerate: both areas vanish for every next vertex, so the
area data fix no direction. -/
theorem vertex_at_centre_degenerate (p C : LatticePoint) :
  det (0, 0) C = det p (0, 0) := by
  simp [det]

/-- The sense is not fixed by areas: an outward kick (`+1`) keeps the oriented
area equal, as does the inward kick (`-1`). -/
theorem outward_kick_equal_area :
  det (1, 1) (kick (1, 1) (extend (1, 0) (1, 1)) 1) = det (1, 0) (1, 1) ∧
  det (1, 1) (kick (1, 1) (extend (1, 0) (1, 1)) (-1)) = det (1, 0) (1, 1) := by
  decide

/-- Position of a uniformly translating centre at cell `n`. -/
def centreAt (s0 w : LatticePoint) (n : Nat) : LatticePoint :=
  (s0.1 + (n : Int) * w.1, s0.2 + (n : Int) * w.2)

/-- Finite Case-2 fact (motivated by the laws' Corollary V, cited in 1687 par50
and 1713 par60): inertial continuation commutes with uniform translation of
the reference centre, so relative vertices obey the same continuation. -/
theorem extend_relative (s0 w p q : LatticePoint) (n : Nat) :
  sub (extend p q) (centreAt s0 w (n + 2)) =
  extend (sub p (centreAt s0 w n)) (sub q (centreAt s0 w (n + 1))) := by
  simp only [sub, extend, centreAt]
  apply Prod.ext <=> dsimp <=> simp only [Int.ofNat_add, Int.add_mul] <=> omega

/-- Case 2, finite step: equal oriented areas about a uniformly moving centre
make each deflection parallel to the current radius from that centre. -/
theorem moving_centre_equal_areas_central (s0 w : LatticePoint) (v : Nat → LatticePoint)
  (h : ∀ n, det (sub (v (n + 1)) (centreAt s0 w (n + 1))) (sub (v (n + 2)) (centreAt s0 w (n + 2)))) =
  det (sub (v n) (centreAt s0 w n)) (sub (v (n + 1)) (centreAt s0 w (n + 1))) (n : Nat) :
  det (sub (v (n + 1)) (centreAt s0 w (n + 1)))
  (sub (v (n + 2)) (extend (v n) (v (n + 1)))) = 0 := by
  have hpar := equal_area_parallel (sub (v n) (centreAt s0 w n))
    (sub (v (n + 1)) (centreAt s0 w (n + 1))) (sub (v (n + 2)) (centreAt s0 w (n + 2))) (h n)
  unfold deflection at hpar
  rw [← extend_relative] at hpar
  have hd : sub (sub (v (n + 2)) (centreAt s0 w (n + 2))) (sub (extend (v n) (v (n + 1))) (centreAt s0 w
    (n + 2))) =
    sub (v (n + 2)) (extend (v n) (v (n + 1))) := by
    simp only [sub]
    apply Prod.ext <=> dsimp <=> omega
  rw [hd] at hpar
  exact hpar
end NewtonLimitDynamics.Polygon.Converse

```

Scholium

A body may be urged by a centripetal force compounded of several forces. In that case the meaning of the Proposition is that the force compounded of them all tends to the point S. Further, if some force acts along a line perpendicular to the surface described, it will make the body deviate from the plane of its motion, but it will neither increase nor diminish the quantity of the surface described, and so it may be neglected in the composition of forces.

No theorem of the Lean reconstruction cites this item. Not encoded.

Proposition III. Theorem III. *Every body that, by a radius drawn to the centre of another body moving in any way, describes about that centre areas proportional to the times, is urged by a force compounded of the centripetal force tending to that other body and of all the accelerative force by which that other body is urged.*

In modern terms: if $\mathbf{r}_{12} \times \dot{\mathbf{r}}_{12}$ is constant for the relative position $\mathbf{r}_{12} = \mathbf{r}_1 - \mathbf{r}_2$, then $\mathbf{a}_1 - \mathbf{a}_2$ is parallel to $\mathbf{r}_{12}$.

Proof. By Corollary VI of the Laws, if both bodies are urged along parallel lines by a new force equal and contrary to the force urging the other body, the first body continues to describe the same areas about the other as before. The force urging the other body is now destroyed by its equal and contrary force, so (Law I) that other body either rests or moves uniformly in a straight line, and the first body, urged by the difference of the forces, continues to describe areas proportional to the times about it. So (Theorem II) the difference of the forces tends to that other body as its centre.

1713. The proof names the bodies L and T ; T , left to itself once its force is cancelled, rests or moves uniformly, and L moves under “the difference of the forces, that is, the remaining force”. The argument is unchanged.

Corollary 1. Hence if one body, by a radius drawn to another, describes areas proportional to the times, and from the total force urging the first (simple, or compounded of several forces by Corollary II of the Laws) the whole accelerative force urging the other is subtracted (by the same Corollary), all the remaining force urging the first tends to the other body as centre.

Corollary 2. And if those areas are very nearly proportional to the times, the remaining force tends very nearly to the other body.

Corollary 3. And conversely, if the remaining force tends very nearly to the other body, those areas are very nearly proportional to the times.

Corollary 4. If a body, by a radius drawn to another body, describes areas that are very unequal compared with the times, and that other body rests or moves uniformly in a straight line, the action of the centripetal force tending to that other body is either null or mixed and compounded with very powerful actions of other forces; and the total force compounded of them all, if there are several, is directed to another centre (immovable or moving) about which the description of areas is uniform. The same holds when the other body moves in any way, provided the centripetal force taken is what remains after subtracting the total force acting on that other body.

Lean reconstruction: 17 theorems in 1 module cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Polygon/RelativeMotion.lean, 17 theorems, 272 lines

```
import NewtonLimitDynamics.Polygon.Converse
```

```
/-!
```

```
Finite relative-motion step for Proposition III (1687 NATP00077 par53–54,
1713 NATP00082 par64–65). Both editions prove the proposition the same way:
the laws' Corollary VI adds an equal and contrary parallel acceleration to both
bodies, Law I fixes the resulting motion of the reference body, and
Proposition II then identifies the remaining force as centripetal.
```

```
The reconstruction is the finite integer-coordinate polygon of
`Polygon.Finite`, now for two bodies in one pair recursion. Each body
```

continues inertially and then takes its own deflection for the cell. Three results mirror the three steps of Newton's proof:

- * ``corVI_relative``: a deflection history added to *both* bodies changes neither relative coordinate, at any stage (Corollary VI);
- * ``lawI_uniform``: with no deflection the reference body's vertices are the uniform motion ``Converse.centreAt`` of its initial pair (Law I);
- * ``relative_equal_area_central``: equal relative oriented areas make the *difference* of the two deflections parallel to the relative radius (Proposition II applied to the relative polygon).

``relative_deflection_difference`` is the compositional content of the proposition's statement: the relative polygon's deflection in each cell is the difference of the two bodies' deflections, i.e. the force toward the other body composed with the whole accelerative force of the other body.

Finite only. No curve, no limit, no force law and no mass is produced; the "force" of a cell is its deflection vector. The equal relative areas are a supplied hypothesis, not a consequence of a realized orbit, and the reference body's own force history is arbitrary.
-/

```
namespace NewtonLimitDynamics.Polygon.RelativeMotion

open NewtonLimitDynamics.Polygon
open NewtonLimitDynamics.Polygon.Converse (sub centreAt)

def add (x y : LatticePoint) : LatticePoint := (x.1 + y.1, x.2 + y.2)

theorem add_zero_right (a : LatticePoint) : add a (0, 0) = a := by
  simp [add]

/-- A summand common to both sides cancels in a difference. -/
theorem add_common_cancel (a b c : LatticePoint) : sub (add a c) (add b c) = sub a b := by
  simp only [add, sub]
  apply Prod.ext <=> dsimp <=> omega

theorem add_sub_distrib (a b c d : LatticePoint) :
  sub (add a b) (add c d) = add (sub a c) (sub b d) := by
  simp only [add, sub]
  apply Prod.ext <=> dsimp <=> omega

/-- Inertial continuation commutes with taking the difference of two pairs. -/
theorem extend_sub (a b c d : LatticePoint) :
  sub (extend a b) (extend c d) = extend (sub a c) (sub b d) := by
  simp only [sub, extend]
  apply Prod.ext <=> dsimp <=> omega

theorem det_add (q x y : LatticePoint) : det q (add x y) = det q x + det q y := by
  simp only [det, add, Int.mul_add]
  omega

/-- Two bodies at one stage: the first body occupies the consecutive vertices
`p, q`, the reference body occupies `s, t`. -/
structure PairState where
  p : LatticePoint
  q : LatticePoint
  s : LatticePoint
  t : LatticePoint

/-- Two-body vertex sequence, one pair recursion. `d` is the first body's
deflection history and `e` the reference body's. In cell `n` each body
continues inertially from its current pair and then takes that cell's
deflection at the arrival vertex. -/
def pairMotion (z : PairState) (d e : Nat → LatticePoint) : Nat → PairState
| 0 => z
| n+1 =>
  let o := pairMotion z d e n
  { p := o.q,
    q := add (extend o.p o.q) (d n),
```



```

    s := 0.t,
    t := add (extend 0.s 0.t) (e n) }

/-- Relative coordinates of the first body with respect to the reference
    body. -/
def relP (z : PairState) : LatticePoint := sub z.p z.s

def relQ (z : PairState) : LatticePoint := sub z.q z.t

def relPAT (z : PairState) (d e : Nat → LatticePoint) (n : Nat) : LatticePoint :=
  relP (pairMotion z d e n)

def relQAT (z : PairState) (d e : Nat → LatticePoint) (n : Nat) : LatticePoint :=
  relQ (pairMotion z d e n)

theorem relP_succ (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  relPAT z d e (n + 1) = relQAT z d e n :=
  rfl

/-- The relative polygon's step: inertial continuation of the relative pair plus
    the difference of the two deflections. This is the finite content of
    Proposition III's force composition (1687 par53, 1713 par64): the relative
    motion is generated by the force toward the other body together with the
    whole accelerative force of the other body, taken with the contrary
    sense. -/
theorem relative_deflection_difference (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  relQAT z d e (n + 1) =
    add (extend (relPAT z d e n) (relQAT z d e n)) (sub (d n) (e n)) := by
  simp only [pairMotion, relPAT, relQAT, relP, relQ, add_sub_distrib, extend_sub]

/-- A body's own deflection is exactly the gap between its next vertex and the
    inertial continuation of its current pair. -/
theorem absolute_deflection (z : PairState) (d e : Nat → LatticePoint) (n : Nat) :
  sub (pairMotion z d e (n + 1)).q (extend (pairMotion z d e n).p (pairMotion z d e n).q) =
    d n := by
  simp only [pairMotion, sub, add]
  apply Prod.ext <=> dsimp <=> omega

/-- The laws' Corollary VI, finite form (1687 par54, 1713 par65): a deflection
    history added to both bodies leaves both relative coordinates unchanged
    at every stage. The common history is arbitrary, so this is the polygon
    form of "equal accelerative forces along parallel lines". -/
theorem corVI_relative (z : PairState) (d e h : Nat → LatticePoint) (n : Nat) :
  relPAT z (fun i => add (d i) (h i)) (fun i => add (e i) (h i)) n = relPAT z d e n ∧
  relQAT z (fun i => add (d i) (h i)) (fun i => add (e i) (h i)) n = relQAT z d e n := by
  induction n with
  | zero => exact (rfl, rfl)
  | succ n ih =>
    refine {?, ?}
    · rw [relP_succ, relP_succ, ih.2]
    · rw [relative_deflection_difference, relative_deflection_difference, ih.1, ih.2]
      simp only [add_common_cancel]

/-- Corollary VI is not vacuous: a nonzero common history really does change
    both bodies' absolute deflections while `corVI_relative` keeps the relative
    polygon fixed. First cell of an explicit pair state. -/
def exampleState : PairState := { p := (0, 0), q := (2, 0), s := (0, 3), t := (2, 3) }

theorem common_deflection_example :
  (relQAT exampleState (fun _ => (0, -2)) (fun _ => (0, 0)) 1 =
    relQAT exampleState (fun _ => (0, -1)) (fun _ => (0, 1)) 1) ∧
  (fun _ => (0, -2)) 0 ≠ (fun _ => (0, -1)) 0 ∧
  sub ((fun _ => (0, -2)) 0) ((fun _ => (0, 0)) 0) =
    sub ((fun _ => (0, -1)) 0) ((fun _ => (0, 1)) 0) := by
  decide

/-- Law I, finite form (cited in 1687 par54, 1713 par65): with no deflection the
    reference body's vertices are the uniform motion `centreAt` of its initial
    pair. The first body's deflections are irrelevant to this claim. -/
theorem lawI_uniform (z : PairState) (d e : Nat → LatticePoint) (he : ∀ i, e i = (0, 0))

```

```

(n : Nat) :
(pairMotion z d e n).s = centreAt z.s (sub z.t z.s) n ∧
(pairMotion z d e n).t = centreAt z.s (sub z.t z.s) (n + 1) := by
induction n with
| zero =>
  refine {?, ?_}
  · simp only [pairMotion, centreAt]
    apply Prod.ext <;> dsimp <;> omega
  · simp only [pairMotion, centreAt, sub]
    apply Prod.ext <;> dsimp <;> omega
| succ n ih =>
  refine {?, ?_}
  · rw [show (pairMotion z d e (n + 1)).s = (pairMotion z d e n).t from rfl]
    exact ih.2
  · rw [show (pairMotion z d e (n + 1)).t =
      add (extend (pairMotion z d e n).s (pairMotion z d e n).t) (e n) from rfl]
    rw [he n, add_zero_right, ih.1, ih.2]
    simp only [extend, centreAt, sub]
    apply Prod.ext <;> dsimp <;> simp only [Int.ofNat_add, Int.add_mul] <;> omega

/-- With an inertial reference body, the relative radius from the uniform
`centreAt` is the relative coordinate of the constructed polygon: Newton's
moving centre is the reference body itself. -/
theorem centre_is_reference_body (z : PairState) (d e : Nat → LatticePoint)
(he : ∀ i, e i = (0, 0)) (n : Nat) :
sub (pairMotion z d e n).q (centreAt z.s (sub z.t z.s) (n + 1)) = relQAt z d e n := by
rw [← (lawI_uniform z d e he n).2, relQAt, relQ]

/-- Newton's route for Proposition III: Corollary VI reduces to the uniformly
moving reference body, Law I makes that body the `centreAt` motion, and the
moving-centre converse of Proposition II (`Converse`) then makes the
deflection from the inertial continuation parallel to the relative
radius. -/
theorem propIII_via_moving_centre (z : PairState) (d e : Nat → LatticePoint)
(h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1)) =
det (relPAt z d e n) (relQAt z d e n)) (n : Nat) :
det (relQAt z d e n) (sub (d n) (e n)) = 0 := by
-- Corollary VI: cancel the reference body's deflection in both histories.
let d' := fun i => add (d i) (sub (0, 0) (e i))
let e' := fun i => add (e i) (sub (0, 0) (e i))
have he' : ∀ i, e' i = (0, 0) := by
  intro i
  apply Prod.ext <;> dsimp [e', add, sub] <;> omega
have hdiff : ∀ i, d' i = sub (d i) (e i) := by
  intro i
  apply Prod.ext <;> dsimp [d', add, sub] <;> omega
have hrelative := corVI_relative z d e (fun i => sub (0, 0) (e i))
-- Law I identifies .p with time i and .q with time i+1.
have hp (i : Nat) :
sub (pairMotion z d' e' i).p (centreAt z.s (sub z.t z.s) i) =
relPAt z d e i := by
  rw [← (lawI_uniform z d' e' he' i).1]
  exact (hrelative i).1
have hq (i : Nat) :
sub (pairMotion z d' e' i).q (centreAt z.s (sub z.t z.s) (i + 1)) =
relQAt z d e i :=
(centre_is_reference_body z d' e' he' i).trans (hrelative i).2
-- Proposition II applies to the reduced body's .p vertex sequence.
have hareas : ∀ i,
det (sub (pairMotion z d' e' (i + 1)).p (centreAt z.s (sub z.t z.s) (i + 1)))
(sub (pairMotion z d' e' (i + 2)).p (centreAt z.s (sub z.t z.s) (i + 2))) =
det (sub (pairMotion z d' e' i).p (centreAt z.s (sub z.t z.s) i))
(sub (pairMotion z d' e' (i + 1)).p (centreAt z.s (sub z.t z.s) (i + 1))) := by
  intro i
  simpa only [hp, relP_succ] using h i
have hc := Converse.moving_centre_equal_areas_central z.s (sub z.t z.s)
(fun i => (pairMotion z d' e' i).p) hareas n
change det (sub (pairMotion z d' e' n).q (centreAt z.s (sub z.t z.s) (n + 1)))
(sub (pairMotion z d' e' (n + 1)).q
(extend (pairMotion z d' e' n).p (pairMotion z d' e' n).q)) = 0 at hc

```

```

    rw [hq, absolute_deflection, hdiff] at hc
    exact hc

/-- Regression for the moving-centre indexing: three consecutive true relative
radii have equal oriented areas, while pairing .q with time n instead of
n+1 gives unequal areas, even with an inertial reference body. -/
theorem moving_centre_alignment_example :
  let z : PairState := { p := (0, -1), q := (2, 0), s := (0, 0), t := (1, 0) }
  let d : Nat → LatticePoint := fun n => if n = 0 then (-2, 0) else (0, -2)
  let e : Nat → LatticePoint := fun _ => (0, 0)
  det (relQAt z d e 0) (relQAt z d e 1) =
    det (relQAt z d e 1) (relQAt z d e 2) ∧
  det (sub (pairMotion z d e 0).q (centreAt z.s (sub z.t z.s) 0))
    (sub (pairMotion z d e 1).q (centreAt z.s (sub z.t z.s) 1)) ≠
  det (sub (pairMotion z d e 1).q (centreAt z.s (sub z.t z.s) 1))
    (sub (pairMotion z d e 2).q (centreAt z.s (sub z.t z.s) 2)) := by
  decide

/-- Proposition III, finite step in relative coordinates: equal relative
oriented areas in equal cells make the difference of the two deflections
parallel to the relative radius. -/
theorem relative_equal_area_central (z : PairState) (d e : Nat → LatticePoint)
  (h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1)) =
    det (relPAT z d e n) (relQAt z d e n)) (n : Nat) :
  det (relQAt z d e n) (sub (d n) (e n)) = 0 := by
  have hn := h n
  rw [relative_deflection_difference, det_add, extension_identity] at hn
  omega

/-- With a nonzero relative radius the same step gives an explicit rational
multiple `a/b` of that radius. The inward sense is a separate premise. -/
theorem relative_rational_central (z : PairState) (d e : Nat → LatticePoint)
  (h : ∀ n, det (relQAt z d e n) (relQAt z d e (n + 1)) =
    det (relPAT z d e n) (relQAt z d e n))
  (hq : relQAt z d e 0 ≠ (0, 0)) :
  ∃ a b : Int, b ≠ 0 ∧ b * (sub (d 0) (e 0)).1 = a * (relQAt z d e 0).1 ∧
  b * (sub (d 0) (e 0)).2 = a * (relQAt z d e 0).2 :=
  Converse.parallel_is_multiple (relQAt z d e 0) (sub (d 0) (e 0)) hq
  (relative_equal_area_central z d e h 0)

/-- The parallelism is a statement about the difference: two different pairs
of absolute deflection histories with the same difference give the same
relative conclusion. Concrete first cell. -/
theorem difference_not_pair_example :
  sub ((fun _ : Nat => (0, -2)) 0) ((fun _ : Nat => (0, 0)) 0) =
  sub ((fun _ : Nat => (1, -1)) 0) ((fun _ : Nat => (1, 1)) 0) ∧
  ((fun _ : Nat => (0, -2)) 0, (fun _ : Nat => (0, 0)) 0) ≠
  ((fun _ : Nat => (1, -1)) 0, (fun _ : Nat => (1, 1)) 0) := by
  decide

end NewtonLimitDynamics.Polygon.RelativeMotion

```

Scholium

Since the uniform description of areas indicates the centre to which the force that most affects the body is directed, and the body is retained in its orbit by a force toward that centre, and every circular motion is rightly said to be performed about the centre by whose force the body is drawn back from rectilinear motion and retained in its orbit: why should we not use the uniform description of areas, in what follows, as the index of the centre about which every circular motion in free spaces is performed?

No theorem of the Lean reconstruction cites this item. Not encoded.

Proposition IV. Theorem IV. *The centripetal forces of bodies describing different circles with uniform motion tend to the centres of those circles, and are to each other*

as the squares of the arcs described in the same time applied to the radii of the circles.

In modern terms: for arcs s described in a common time on circles of radius r , $F \propto s^2/r$; that is, $F \propto v^2/r$.

Proof. Let the bodies B, b , revolving in the circumferences of the circles BD, bd , describe the arcs BD, bd in the same time. By their inherent force alone they would describe tangents BC, bc equal to these arcs; so it is the centripetal forces that perpetually draw the bodies back from the tangents to the circumferences, and they are to each other in the first ratio of the nascent spaces CD, cd . They tend to the centres of the circles by Theorem II, because the areas described by the radii are taken proportional to the times. Make the figure tkb similar to the figure DCB . By Lemma V the line CD is to the line kt as the arc BD to the arc bt ; by Lemma XI the nascent line tk is to the nascent line dc as bt^2 to bd^2 ; so *ex aequo* the nascent line DC is to the nascent line dc as $BD \cdot bt$ to bd^2 , or equivalently as $BD \cdot bt/Sb$ to bd^2/Sb , and so (the ratios bt/Sb and BD/SB being equal) as BD^2/SB to bd^2/Sb .

In modern terms: the departure from the tangent over the arc s is the sagitta $CD \approx s^2/(2r)$ (Lemma XI), and the force is as that departure in a given time (Lemma X, Corollary 2).

1713. A new proof replaces this one. The forces tend to the centres of the circles by Proposition II and Corollary 2 of Proposition I, and are to each other as the versed sines of the least arcs described in equal times (Corollary 4 of Proposition I), that is, as the squares of those arcs applied to the diameters of the circles (Lemma VII). Since these arcs are as the arcs described in any equal times, and the diameters as the radii, the forces are as the squares of arcs described in the same time applied to the radii.

Corollary 1. Hence the centripetal forces are as the squares of the velocities applied to the radii of the circles: $F \propto v^2/r$.

Corollary 2. And they are inversely as the squares of the periodic times applied to the radii: $F \propto r/T^2$. That is, in the language of the geometers, these forces are in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely; and also in the ratio compounded of the simple ratio of the radii directly and the duplicate ratio of the periodic times inversely.

Corollary 3. So if the periodic times are equal, both the centripetal forces and the velocities are as the radii, and conversely.

Corollary 4. If the squares of the periodic times are as the radii, the centripetal forces are equal and the velocities are in the subduplicate ratio of the radii, and conversely: $T^2 \propto r \iff F = \text{const}, v \propto r^{1/2}$.

Corollary 5. If the squares of the periodic times are as the squares of the radii, the centripetal forces are inversely as the radii and the velocities equal, and conversely: $T \propto r \iff F \propto 1/r, v = \text{const}$.

Corollary 6. If the squares of the periodic times are as the cubes of the radii, the centripetal forces are inversely as the squares of the radii, and the velocities in the subduplicate ratio of the radii; and conversely: $T^2 \propto r^3 \iff F \propto 1/r^2$. [As printed

in 1687 the velocity clause lacks “inversely”; by Corollary 1 the consistent relation is $v \propto r^{-1/2}$.]

Corollary 7. All the same holds for the times, velocities and forces with which bodies describe similar parts of any similar figures having similarly placed centres, by applying the preceding proof to those cases.

1713. The corollaries are rewritten and extended to nine.

Corollary 1. Therefore, since those arcs are as the velocities of the bodies, the forces are as the squares of the velocities applied to the radii, or, as the geometers say, in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely.

Corollary 2. And since the periodic times are in the ratio compounded of the radii directly and the velocities inversely, the forces are inversely as the squares of the periodic times applied to the radii, that is, in the ratio compounded of the radii directly and the duplicate ratio of the periodic times inversely: $F \propto r/T^2$.

Corollary 3. So if the periodic times are equal, and therefore the velocities are as the radii, the forces are also as the radii, and conversely.

Corollary 4. If both the periodic times and the velocities are in the subduplicate ratio of the radii, the forces are equal, and conversely.

Corollary 5. If the periodic times are as the radii, and therefore the velocities are equal, the forces are inversely as the radii, and conversely.

Corollary 6. If the periodic times are in the sesquiplicate ratio of the radii, and therefore the velocities inversely in the subduplicate ratio of the radii, the forces are inversely as the squares of the radii, and conversely: $T \propto r^{3/2} \Leftrightarrow v \propto r^{-1/2} \Leftrightarrow F \propto r^{-2}$. This corrects the 1687 velocity clause.

Corollary 7. And universally, if the periodic time is as any power R^n of the radius R , and therefore the velocity inversely as R^{n-1} , the centripetal force is inversely as R^{2n-1} , and conversely.

Corollary 8. The old Corollary 7, with the addition that it is applied by substituting the uniform description of areas for uniform motion, and the distances of the bodies from the centres for the radii.

Corollary 9. From the same proof it also follows that the arc which a body describes in any time, revolving uniformly in a circle under a given centripetal force, is a mean proportional between the diameter of the circle and the descent of the body falling under the same force in the same time: $s^2 = d \cdot h$.

1726. Corollaries 1 and 2 drop the “applied to the radii” formulation and keep only the compound ratios: in the ratio compounded of the duplicate ratio of the velocities directly and the simple ratio of the radii inversely; and in the ratio compounded of the radii directly and the duplicate ratio of the periodic times inversely.

Lean reconstruction: 15 theorems in 3 modules cite this item; the modules follow in full, in import order.

NewtonLimitDynamics/Polygon/HarmonicStability.lean, 11 theorems, 136 lines

```

import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import NewtonLimitDynamics.Polygon.CentralSchedule

/-!
Stability of the finite impulse construction for the linear central field
`a(p) = -w*p` (force proportional to distance; compare Proposition IV Cor. 3,
equal periods with forces as radii, 1687 NATP00077 par64, 1713 NATP00082 par75).
With equal cells `d`, the construction conserves exactly the quadratic form
`w|x|^2 + w*d*(x*v) + |v|^2`, which equals `w|x + (d/2)v|^2 + (1 - w*d^2/4)|v|^2`
(stated with `d = c + c` to avoid numeric literals).
For `w > 0` and `w*d^2 < 4` this bounds every refined orbit. Modern rational
reconstruction; no ODE theorem, limit or curve is used or produced.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicStability

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule

/-- The central field `a(p) = -w*p`. -/
def linearField (w : Fraction) : Field := fun p => pointScale (negF w) p

/-- The discrete invariant of the equal-cell construction. -/
def invariant (w d : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (Fraction.add (Fraction.mul w (dot s.1 s.1))
    (Fraction.mul (Fraction.mul w d) (dot s.1 s.2))) (dot s.2 s.2)

theorem linearField_central (w : Fraction) : central (linearField w) := by
  intro p
  unfold linearField negF pointScale det Fraction.equiv Fraction.add Fraction.mul Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.zero_mul]
  ac_nf
  omega

/-- One cell of duration `d` conserves the invariant built with the same `d`. -/
theorem cell_invariant (w d : Fraction) (s : Point × Point) :
  Fraction.equiv (invariant w d (cell (linearField w) d s)) (invariant w d s) := by
  unfold invariant dot cell linearField negF pointAdd pointScale Fraction.equiv Fraction.add
  Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Every equal-cell schedule conserves it. -/
theorem schedule_invariant (w d : Fraction) :
  (n : Nat) → (s : Point × Point) →
  Fraction.equiv (invariant w d (schedule (linearField w) (List.replicate n d) s))
    (invariant w d s)
  | 0, _ => Fraction.equiv_refl _
  | n + 1, s =>
    Fraction.equiv_trans (schedule_invariant w d n (cell (linearField w) d s))
      (cell_invariant w d s)

/-- `1 - w*c^2`, the stability margin for cells of duration `d = 2c`. -/
def margin (w c : Fraction) : Fraction :=
  Fraction.add (Fraction.ofInt 1) (negF (Fraction.mul w (Fraction.mul c c)))

/-- Completed square, for cells of duration `d = c + c`: the invariant equals
`w|x + c*v|^2 + (1 - w*c^2)|v|^2`. When `w ≥ 0` and `w*c^2 < 1` (that is,
`w*d^2 < 4`) both coefficients are nonnegative, so every equal-cell orbit
keeps `|v|^2` and `w|x + c*v|^2` bounded by its initial invariant
(`schedule_speed_bound`, `schedule_position_bound`). -/
theorem invariant_square (w c : Fraction) (s : Point × Point) :
  Fraction.equiv (invariant w (Fraction.add c c) s)
    (Fraction.add

```

```

      (Fraction.mul w (dot (pointAdd s.1 (pointScale c s.2)) (pointAdd s.1 (pointScale c s.2))))
      (Fraction.mul (margin w c) (dot s.2 s.2))) := by
  unfold invariant margin dot negF pointAdd pointScale Fraction.equiv Fraction.add Fraction.mul
  Fraction.ofInt
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.mul_one, Int.one_mul]
  ac_nf
  omega

private theorem self_mul_nonneg (a : Int) : 0 ≤ a * a := by
  rcases Int.le_total 0 a with h | h
  · exact Int.mul_nonneg h h
  · have h' : 0 ≤ -a := by omega
    have k := Int.mul_nonneg h' h'
    rwa [Int.neg_mul_neg] at k

theorem dot_self_num_nonneg (p : Point) : 0 ≤ (dot p p).num := by
  unfold dot Fraction.add Fraction.mul
  exact Int.add_nonneg (Int.mul_nonneg (self_mul_nonneg _) (self_mul_nonneg _))
    (Int.mul_nonneg (self_mul_nonneg _) (self_mul_nonneg _))

private theorem le_add_of_num_nonneg (A B : Fraction) (hA : 0 ≤ A.num) :
  Fraction.le B (Fraction.add A B) := by
  unfold Fraction.le Fraction.add
  dsimp
  have h := Int.mul_nonneg (Int.mul_nonneg hA (Int.le_of_lt B.den_pos)) (Int.le_of_lt B.den_pos)
  have e1 : B.num * (A.den * B.den) = B.num * A.den * B.den := by ac_rfl
  rw [Int.add_mul, e1]
  omega

/-- For `w ≥ 0`: `(1 - w*c^2)|v|^2` never exceeds the invariant. -/
theorem speed_bound (w c : Fraction) (hw : 0 ≤ w.num) (s : Point × Point) :
  Fraction.le (Fraction.mul (margin w c) (dot s.2 s.2)) (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right
    (le_add_of_num_nonneg _ _ (Int.mul_nonneg hw (dot_self_num_nonneg _)))
    (Fraction.equiv_symm (invariant_square w c s))

/-- For a nonnegative margin: `w|x + c*v|^2` never exceeds the invariant. -/
theorem position_bound (w c : Fraction) (hm : 0 ≤ (margin w c).num) (s : Point × Point) :
  Fraction.le (Fraction.mul w (dot (pointAdd s.1 (pointScale c s.2)) (pointAdd s.1 (pointScale c s.2))))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right
    (Fraction.le_add_nonnegative _ _ (Int.mul_nonneg hm (dot_self_num_nonneg _)))
    (Fraction.equiv_symm (invariant_square w c s))

/-- Mesh-uniform discrete stability: along every equal-cell schedule of cell
    duration `2c`, `(1 - w*c^2)|v_n|^2` stays below the initial invariant. -/
theorem schedule_speed_bound (w c : Fraction) (hw : 0 ≤ w.num) (n : Nat) (s : Point × Point) :
  Fraction.le
    (Fraction.mul (margin w c)
      (dot (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2
        (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right (speed_bound w c hw _) (schedule_invariant w (Fraction.add c c) n s)

theorem schedule_position_bound (w c : Fraction) (hm : 0 ≤ (margin w c).num) (n : Nat)
  (s : Point × Point) :
  Fraction.le
    (Fraction.mul w
      (dot (pointAdd (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).1
        (pointScale c (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))
        (pointAdd (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).1
        (pointScale c (schedule (linearField w) (List.replicate n (Fraction.add c c)) s).2))))
    (invariant w (Fraction.add c c) s) :=
  Fraction.le_equiv_right (position_bound w c hm _) (schedule_invariant w (Fraction.add c c) n s)

end NewtonLimitDynamics.Polygon.HarmonicStability

```

NewtonLimitDynamics/Comparison/CircleCompare.lean, 1 theorem, 83 lines

import NewtonLimitDynamics.Common.RationalMagnitudes

```

/-!
Proposition IV, finite circular comparison (TASKS.md order 7). Both editions
compare the centripetal forces of bodies describing circles in equal times by
the squares of the simultaneous arcs divided by the radii,  $F \propto \text{arc}^2 / (r \cdot t^2)$ .
The exact finite circle fact behind every route is the sagitta-chord relation
 $s \cdot (2r - s) = (c/2)^2$  for a chord  $c$  with sagitta  $s$  in a circle of radius
 $r$ ; the edition-local routes differ only in *which limiting lemma* turns that
finite relation into  $\text{arc}^2 / (r \cdot t^2)$ :

* 1687: Proposition II (force as sagitta over  $t^2$ ), Lemma V (similar figures,
duplicate ratio of sides) and Lemma XI (the contact subtense is ultimately
quadratic in the chord).
* 1713: Proposition II, Proposition I Corollaries 2 and 4 (force by equal-time
sagittae and its ratio form) and Lemma VII (arc, chord and tangent ultimately
equal).

The finite comparison below uses no limit: for two circle chords traversed in
the same time, the cross-multiplied force ratio is the sagitta ratio, and the
sagitta-chord relation rewrites a sagitta as  $(c^2/4)/(2r - s)$  when the
denominator is positive. The two routes are recorded separately below as
limiting obligations; no conditional route theorem is implemented here.
Modern rational reconstruction; no mass, no realized orbit.
-/

namespace NewtonLimitDynamics.Comparison.CircleCompare

open NewtonLimitDynamics
open Fraction

def negate (a : Fraction) : Fraction := (-a.num, a.den, a.den_pos)
def sub (a b : Fraction) : Fraction := add a (negate b)
def twice (t : Fraction) : Fraction := mul (ofInt 2) t
def sq (t : Fraction) : Fraction := mul t t
def quarterOf (c : Fraction) : Fraction := (mul c c).half.half

/-- Division of represented magnitudes by a positive divisor. -/
def quotient (a b : Fraction) (hb : positive b) : Fraction :=
  (a.num * b.den, a.den * b.num, Int.mul_pos a.den_pos hb)

/-- A chord  $c$  with positive sagitta  $s$  in a circle of radius  $r$ , exactly
 $s \cdot (2r - s) = (c/2)^2$  (intersecting chords / right triangle). Finite. -/
structure CircleChord (r : Fraction) where
  c : Fraction
  s : Fraction
  sag_pos : positive s
  sagitta_chord : equiv (mul s (sub (twice r) s)) (quarterOf c)

/-- Centripetal force read off as sagitta over the square of the time
(Proposition II / Lemma X route, finite form). -/
def forceBySagitta {r : Fraction} {w : CircleChord r} (t : Fraction) (ht : positive t) : Fraction :=
  quotient w.s (mul t t) (positive_mul t t ht ht)

/-- Equal-time forces, cross-multiplied, are proportional to the sagittae. -/
theorem force_ratio_is_sagitta_ratio {r r' : Fraction} {w : CircleChord r} {w' : CircleChord r'}
  (t : Fraction) (ht : positive t) :
  equiv (mul (forceBySagitta w t ht) w'.s) (mul (forceBySagitta w' t ht) w.s) := by
  unfold forceBySagitta quotient mul equiv
  dsimp
  ac_nf
  try omega

/-! The edition routes are the *limiting* steps that turn the finite
sagitta-chord relation into  $\text{arc}^2 / (r \cdot t^2)$ , and are deliberately not derived
here (that would be the limit). They are recorded separately so the two
editions are never merged:

* `route_1687`: Proposition II gives force as sagitta over  $t^2$ 
(`forceBySagitta`); Lemma V supplies the duplicate ratio of similar figures
to pass from chords to arcs; Lemma XI (1687 par31, contact subtense

```


ultimately quadratic) replaces the finite $s = (c^2/4)/(2r - s)$ by $s = c^2/(8r)$ for the full chord c , and with arcs $\approx$ chords the comparison becomes arc^2/r (the common factor cancels in the ratio).
 * `route\_1713`: Proposition I Corollary 2 (par53) and 4 (par55) give force by equal-time sagittae and its ratio form; Lemma VII (par18, arc-chord-tangent ultimately equal) performs the same replacement.

Both routes are `editorial\_interpretation` at the limiting step; the finite comparison above – the exact sagitta-chord relation carried by `CircleChord` and `force\_ratio\_is\_sagitta\_ratio` – is the shared checked core. -/

end NewtonLimitDynamics.Comparison.CircleCompare

NewtonLimitDynamics/Diagnostic/InverseCubeAreal.lean, 3 theorems, 63 lines

```

/-!
Action diagnostic layer (research/action-arguments/Arg002). Modern
reconstruction over natural-number magnitudes of two uniform circles compared
by Proposition IV Cor. 1 (force as squared velocity over radius, stated as a
cross-multiplied proportion). No historical proof uses these theorems.
-/

namespace NewtonLimitDynamics.Diagnostic.InverseCubeAreal

/-- Cor. 1 as a proportion between two circles:  $F_1 : F_2 = v_1^2/R_1 : v_2^2/R_2$ . -/
def corOneProportion (F1 F2 R1 R2 v1 v2 : Nat) : Prop :=
  F1 * (v2 * v2) * R1 = F2 * (v1 * v1) * R2

/-- Inverse-cube comparison of the two forces:  $F_1 R_1^3 = F_2 R_2^3$ . -/
def inverseCube (F1 F2 R1 R2 : Nat) : Prop :=
  F1 * (R1 * R1 * R1) = F2 * (R2 * R2 * R2)

/-- Equal squared areal-velocity numerators  $(R v)^2$ . -/
def equalAreal (R1 R2 v1 v2 : Nat) : Prop :=
  (R1 * v1) * (R1 * v1) = (R2 * v2) * (R2 * v2)

/-- Under an inverse-cube comparison, circles related by Cor. 1 have the same
areal velocity. -/
theorem inverseCube_equalAreal (F1 F2 R1 R2 v1 v2 : Nat) (hF : 0 < F2) (hR : 0 < R2)
  (h1 : corOneProportion F1 F2 R1 R2 v1 v2) (h3 : inverseCube F1 F2 R1 R2) :
  equalAreal R1 R2 v1 v2 := by
  unfold corOneProportion at h1
  unfold inverseCube at h3
  unfold equalAreal
  apply Nat.eq_of_mul_eq_mul_left (Nat.mul_pos hF hR)
  calc F2 * R2 * ((R1 * v1) * (R1 * v1))
    = (F2 * (v1 * v1) * R2) * (R1 * R1) := by ac_rfl
    _ = (F1 * (v2 * v2) * R1) * (R1 * R1) := by rw [h1]
    _ = (F1 * (R1 * R1 * R1)) * (v2 * v2) := by ac_rfl
    _ = (F2 * (R2 * R2 * R2)) * (v2 * v2) := by rw [h3]
    _ = F2 * R2 * ((R2 * v2) * (R2 * v2)) := by ac_rfl

/-- Conversely, a common areal velocity with Cor. 1 forces the inverse-cube
comparison. -/
theorem equalAreal_inverseCube (F1 F2 R1 R2 v1 v2 : Nat) (hv : 0 < v2)
  (h1 : corOneProportion F1 F2 R1 R2 v1 v2) (hA : equalAreal R1 R2 v1 v2) :
  inverseCube F1 F2 R1 R2 := by
  unfold corOneProportion at h1
  unfold equalAreal at hA
  unfold inverseCube
  apply Nat.eq_of_mul_eq_mul_left (Nat.mul_pos hv hv)
  calc v2 * v2 * (F1 * (R1 * R1 * R1))
    = (F1 * (v2 * v2) * R1) * (R1 * R1) := by ac_rfl
    _ = (F2 * (v1 * v1) * R2) * (R1 * R1) := by rw [h1]
    _ = F2 * R2 * ((R1 * v1) * (R1 * v1)) := by ac_rfl
    _ = F2 * R2 * ((R2 * v2) * (R2 * v2)) := by rw [hA]
    _ = v2 * v2 * (F2 * (R2 * R2 * R2)) := by ac_rfl

/-- Contrast: under an inverse-square comparison ( $F_1 R_1^2 = F_2 R_2^2$ , 1713
Cor. 6), radii 1 and 4 with speeds 2 and 1 satisfy Cor. 1 but have
unequal areal velocities 2 and 4. -/

```

```

theorem inverseSquare_areal_varies :
  corOneProportion 16 1 1 4 2 1 ∧ 16 * (1 * 1) = 1 * (4 * 4) ∧
  ¬ equalAreal 1 4 2 1 := by
  unfold corOneProportion equalAreal
  decide
end NewtonLimitDynamics.Diagnostic.InverseCubeAreal

```

Scholium

The case of Corollary 6 holds in the celestial bodies (as Wren, Hooke and Halley also gathered separately), and so what concerns a centripetal force decreasing in the duplicate ratio of the distances from the centres is set out more fully in what follows.

The preceding proof also gives the proportion of a centripetal force to any known force, such as gravity. In the time the body traverses the arc BC the force impels it through the space CD , which at the very beginning of the motion equals the square of the arc BD applied to the diameter of the circle; and every body urged by the same force continued always in the same direction describes spaces in the duplicate ratio of the times. So the force, in the time the revolving body describes any given arc, would make the same body, going straight on, describe a space equal to the square of that arc applied to the diameter of the circle; it is therefore to gravity as that space is to the space a heavy body describes falling in the same time. By propositions of this kind Huygens, in his excellent treatise *De Horologio oscillatorio*, compared the force of gravity with the centrifugal forces of revolving bodies.

In modern terms: $CD \approx s^2/d$ with $d = 2r$, so the force is to gravity as s^2/d is to the distance $\frac{1}{2}gt^2$ fallen in the same time.

1713. This paragraph is replaced. By the preceding Proposition and its corollaries one also obtains the proportion of a centripetal force to any known force, such as gravity: if a body revolves by its gravity in a circle concentric with the Earth, that gravity is its centripetal force, and from the descent of heavy bodies both the time of one revolution and the arc described in any given time are given, by Corollary 9. Huygens compared gravity with the centrifugal forces of revolving bodies by propositions of this kind.

The preceding can also be proved this way. In any circle inscribe a polygon of any number of sides. If a body moving along the sides with a given velocity is reflected from the circle at each vertex, the force with which it strikes the circle at each reflection is as its velocity; so the sum of the forces in a given time is as that velocity and the number of reflections jointly, that is (the polygon being given in kind) as the length described in that time and the same length applied to the radius, that is, as the square of that length applied to the radius; and so, if the polygon, its sides diminished indefinitely, coincides with the circle, as the square of the arc described in the given time applied to the radius. This is the force with which the body presses on the circle, and equal to it is the contrary force with which the circle continually repels the body toward the centre.

1713. “This is the *centrifugal force* with which the body presses on the circle.” **1726** replaces “the same length applied to the radius” by “the same length increased or diminished in the ratio of that length to the radius of the circle”.

In modern terms: on a regular n -gon the body makes about $vt n/(2\pi r)$ reflections in the time t , each of impulse about $mv \cdot 2\pi/n$, so the total impulse is about $mv^2 t/r$. For a given t the length described is as v , so this is as $(vt)^2/r$, Newton's square of the length applied to the radius.

No theorem of the Lean reconstruction cites this item. Not encoded; the polygon-reflection argument has no counterpart.

Appendix A. Modules anchored outside Sections I-II

1 module with 4 theorems cite passages outside the rendered range (Proposition VI and its later-edition counterparts).

NewtonLimitDynamics/Comparison/Routes.lean, 4 theorems, 52 lines

```
import NewtonLimitDynamics.Common.RationalMagnitudes

namespace NewtonLimitDynamics.Comparison
open Fraction

def negate (a : Fraction) : Fraction := (-a.num, a.den, a.den_pos)
def twice (t : Fraction) : Fraction := mul (ofInt 2) t

theorem twice_positive (t : Fraction) (ht : positive t) : positive (twice t) :=
  positive_mul (ofInt 2) t (by unfold positive ofInt; decide) ht

/-- Constant accelerative force, zero initial normal velocity. The tangential
coordinate does not enter the normal sagitta. Its area interpretation is
a separate mechanical/geometric premise. -/
def parabolaHeight (acc time : Fraction) : Fraction := half (mul acc (mul time time))
def midpointHeight (y1 y2 : Fraction) : Fraction := half (add y1 y2)

theorem symmetric_parabola_sagitta (acc time : Fraction) :
  equiv (midpointHeight (parabolaHeight acc (negate time)) (parabolaHeight acc time))
    (parabolaHeight acc time) := by
  unfold equiv midpointHeight parabolaHeight negate half add mul
  dsimp
  simp only [Int.neg_mul_neg, Int.add_mul, Int.mul_add]
  have htwo : ∀ x : Int, 2*x = x+x := by intro x; omega
  simp only [htwo, Int.mul_add, Int.add_mul]
  ac_rfl

/-- Calibrated force comparison on matched spatial quantities: the generated
displacement uses one-sided duration t, the symmetric sagitta uses the
full arc duration 2t. Thus the calibration factors are 2 and 8 respectively.
For general curves, matching their limiting coefficients remains an input;
this theorem does not assert equality of arbitrary finite displacements. -/
theorem generated_sagitta_commute (displacement time : Fraction) (ht : positive time) :
  equiv
    (mul (ofInt 8) (deflectionRatio displacement (twice time) (twice_positive time ht)))
    (mul (ofInt 2) (deflectionRatio displacement time ht)) := by
  simp only [equiv, deflectionRatio, twice, mul, ofInt]
  simp only [Int.one_mul, Int.mul_one]
  have eight : (8 : Int) = 2 * (2 * 2) := by decide
  rw [eight]
  ac_rfl

/-- The finite constant-force model gives its acceleration by the generated
route, independently of the prior commutation identity. -/
theorem constant_force_generated (acc time : Fraction) (ht : positive time) :
  equiv (mul (ofInt 2) (deflectionRatio (parabolaHeight acc time) time ht)) acc := by
  unfold equiv deflectionRatio parabolaHeight half mul ofInt
  dsimp
  simp only [Int.one_mul, Int.mul_one]
  ac_rfl
```

```
end NewtonLimitDynamics.Comparison
```

Appendix B. The foundation: modules with no source anchor

90 modules with 1002 theorems have no Newton anchor. They build the rational arithmetic, point algebra, finite estimates, Cauchy names and quotient values, binary time, square covers and the lifting of operations to completed values on which the anchored proofs stand. In full, in import order, followed by the two library root files.

BarrowLib/Common/Quadratic.lean, 4 theorems, 72 lines

```
/- No mathlib. Magnitudes and ratios are abstract: this file supplies logical
bookkeeping for shrinking intervals and ordered enclosures, not calculus. -/
namespace NewtonLimitDynamics

structure Magnitudes (Q : Type) where
  positive : Q → Prop
  lt : Q → Q → Prop
  le : Q → Q → Prop
  /-- Inhabitation and comparison brackets prevent vacuous limits. -/
  inhabited_positive : ∃ d, positive d
  lt_irrefl : ∀ a, ¬ lt a a
  le_refl : ∀ a, le a a
  le_trans : ∀ {a b c}, le a b → le b c → le a c
  lt_implies_le : ∀ {a b}, lt a b → le a b
  surrounds : ∀ c, ∃ a b, lt a c ∧ lt c b
  lt_of_lt_le : ∀ {a b c}, lt a b → le b c → lt a c
  lt_of_le_lt : ∀ {a b c}, le a b → lt b c → lt a c
  /-- No smallest positive interval: neighborhoods must not be vacuous. -/
  shrink : ∀ d, positive d → ∃ h, positive h ∧ lt h d
  /-- Two permitted interval bounds admit a common smaller positive bound. -/
  refine : ∀ a b, positive a → positive b →
    ∃ d, positive d ∧ ∀ h, positive h → lt h d → lt h a ∧ lt h b

variable {Q : Type}

def Near (g : Magnitudes Q) (P : Q → Prop) : Prop :=
  ∃ d, g.positive d ∧ ∀ h, g.positive h → g.lt h d → P h

theorem near_and (g : Magnitudes Q) (P R : Q → Prop)
  (hp : Near g P) (hr : Near g R) : Near g (fun h => P h ∧ R h) := by
  obtain ⟨a, ha, hpa⟩ := hp
  obtain ⟨b, hb, hrb⟩ := hr
  obtain ⟨d, hd, hdab⟩ := g.refine a b ha hb
  refine ⟨d, hd, ?_⟩
  intro h hh hhd
  obtain ⟨hha, hhb⟩ := hdab h hh hhd
  exact ⟨hpa h hh hha, hrb h hh hhb⟩

/- Ultimate equality expressed by eventual enclosure between arbitrary
surrounding ratios. This is an encoding, not a quoted Newton definition. -/
def Ultimate (g : Magnitudes Q) (ratio : Q → Q) (c : Q) : Prop :=
  ∀ a b, g.lt a c → g.lt c b →
    Near g (fun h => g.lt a (ratio h) ∧ g.lt (ratio h) b)

/- Logical squeeze justified by order and unlimited common refinement.
Lower and upper geometric limits must be supplied, not assumed proved. -/
theorem enclosure_reconstruction (g : Magnitudes Q)
  (ratio_lower_upper : Q → Q) (c : Q)
  (hl : Ultimate g lower c) (hu : Ultimate g upper c)
  (hb : Near g (fun h => g.le (lower h) (ratio h) ∧
    g.le (ratio h) (upper h))) : Ultimate g ratio c := by
  intro a b hac hcb
```

```

obtain (d, hd, hx) := near_and g _ _ (near_and g _ _
  (hl a b hac hcb) (hu a b hac hcb)) hb
refine (d, hd, ?_)
intro h hh hhd
obtain ((hal, _), (_, hub)), (h1r, hru)) := hx h hh hhd
exact (g.lt_of_lt_le hal h1r, g.lt_of_le_lt hru hub)

/-- Every eventual assertion has an actual positive witness. -/
theorem near_has_witness (g : Magnitudes Q) (P : Q → Prop)
  (h : Near g P) : ∃ x, g.positive x ∧ P x := by
  obtain (d, hd, hp) := h
  obtain (x, hx, hxd) := g.shrink d hd
  exact (x, hx, hp x hx hxd)

theorem not_near_false (g : Magnitudes Q) : ¬ Near g (fun _ => False) := by
  intro h
  obtain (_, _, hf) := near_has_witness g _ h
  exact hf

end NewtonLimitDynamics

```

BarrowLib/Common/RationalMagnitudes.lean, 57 theorems, 490 lines

```

import BarrowLib.Common.Quadratic
import Std

namespace NewtonLimitDynamics
/-- Signed rational representatives, deliberately unnormalised. Equality of
  magnitudes is cross multiplication, not equality of representations. -/
structure Fraction where
  num : Int
  den : Int
  den_pos : 0 < den

namespace Fraction

def lt (a b : Fraction) := a.num * b.den < b.num * a.den
def le (a b : Fraction) := a.num * b.den ≤ b.num * a.den
def equiv (a b : Fraction) := a.num * b.den = b.num * a.den
def positive (a : Fraction) := 0 < a.num
def ofInt (n : Int) : Fraction := (n, 1, by decide)
def add (a b : Fraction) : Fraction :=
  (a.num * b.den + b.num * a.den, a.den * b.den, Int.mul_pos a.den_pos b.den_pos)
def mul (a b : Fraction) : Fraction :=
  (a.num * b.num, a.den * b.den, Int.mul_pos a.den_pos b.den_pos)
def half (a : Fraction) : Fraction := (a.num, 2 * a.den, Int.mul_pos (by decide) a.den_pos)

/-- Unsigned scalar magnitude. This arithmetic operation does not identify a
  geometric region between paths or erase the need to sum separate lobes. -/
def abs (a : Fraction) : Fraction := (a.num.natAbs, a.den, a.den_pos)

theorem abs_num_nonnegative (a : Fraction) : 0 ≤ a.abs.num := Int.ofNat_nonneg _

/-- Every signed rational value is at most its unsigned magnitude. -/
theorem le_abs (a : Fraction) : le a a.abs := by
  unfold le abs
  dsimp
  exact Int.mul_le_mul_of_nonneg_right (Int.le_natAbs (a:=a.num))
    (Int.le_of_lt a.den_pos)

/-- Unsigned magnitude respects equality of rational values, not just equality
  of their unnormalised representations. -/
theorem abs_equiv {a b : Fraction} (h : equiv a b) : equiv a.abs b.abs := by
  have hn := congrArg Int.natAbs h
  have hi := congrArg (fun n : Nat => (n : Int)) hn
  simp only [Int.natAbs_mul, Int.ofNat_mul,
    Int.natAbs_of_nonneg (Int.le_of_lt a.den_pos),
    Int.natAbs_of_nonneg (Int.le_of_lt b.den_pos)] at hi
  exact hi

```

```

theorem abs_mul (a b : Fraction) : equiv (mul a b).abs (mul a.abs b.abs) := by
  unfold equiv abs mul
  dsimp
  rw [Int.natAbs_mul, Int.ofNat_mul]

theorem abs_of_nonnegative (a : Fraction) (ha : 0 ≤ a.num) : equiv a.abs a := by
  unfold equiv abs
  dsimp
  rw [Int.natAbs_of_nonneg ha]

/-- Nonnegative fractions retain their actual representative under abs. -/
theorem abs_eq_of_nonnegative (a : Fraction) (ha : 0 ≤ a.num) : a.abs = a := by
  cases a
  simp only [Fraction.abs, Int.natAbs_of_nonneg ha]

theorem abs_neg (a : Fraction) : equiv ((-a.num, a.den, a.den_pos) : Fraction).abs a.abs := by
  unfold equiv abs
  dsimp
  rw [Int.natAbs_neg]

/-- Rational triangle estimate for finite perturbation accounting. -/
theorem abs_add_le (a b : Fraction) : le (add a b).abs (add a.abs b.abs) := by
  have hn := Int.natAbs_add_le (a.num * b.den) (b.num * a.den)
  have hi := Int.ofNat_le.mpr hn
  simp only [Int.ofNat_add, Int.natAbs_mul, Int.ofNat_mul,
    Int.natAbs_of_nonneg (Int.le_of_lt a.den_pos),
    Int.natAbs_of_nonneg (Int.le_of_lt b.den_pos)] at hi
  unfold le abs add
  dsimp
  exact Int.mul_le_mul_of_nonneg_right hi
    (Int.le_of_lt (Int.mul_pos a.den_pos b.den_pos))

private theorem transfer {a b c : Fraction} (h : le a b) (k : le b c) : le a c := by
  have h' := Int.mul_le_mul_of_nonneg_right h (Int.le_of_lt c.den_pos)
  have k' := Int.mul_le_mul_of_nonneg_right k (Int.le_of_lt a.den_pos)
  unfold le at *
  have mid : b.num * a.den * c.den = b.num * c.den * a.den := by ac_rfl
  rw [mid] at h'
  have z := Int.le_trans h' k'
  have l : a.num * b.den * c.den = (a.num * c.den) * b.den := by ac_rfl
  have r : c.num * b.den * a.den = (c.num * a.den) * b.den := by ac_rfl
  rw [l, r] at z
  exact Int.le_of_mul_le_mul_right z b.den_pos

private theorem mixed {a b c : Fraction} (h : lt a b) (k : le b c) : lt a c := by
  have h' := Int.mul_lt_mul_of_pos_right h c.den_pos
  have k' := Int.mul_le_mul_of_nonneg_right k (Int.le_of_lt a.den_pos)
  unfold lt le at *
  have mid : b.num * a.den * c.den = b.num * c.den * a.den := by ac_rfl
  rw [mid] at h'
  have z := Int.lt_of_lt_of_le h' k'
  have l : a.num * b.den * c.den = (a.num * c.den) * b.den := by ac_rfl
  have r : c.num * b.den * a.den = (c.num * a.den) * b.den := by ac_rfl
  rw [l, r] at z
  exact Int.lt_of_mul_lt_mul_right z (Int.le_of_lt b.den_pos)

private theorem mixed' {a b c : Fraction} (h : le a b) (k : lt b c) : lt a c := by
  have h' := Int.mul_le_mul_of_nonneg_right h (Int.le_of_lt c.den_pos)
  have k' := Int.mul_lt_mul_of_pos_right k a.den_pos
  unfold lt le at *
  have mid : b.num * a.den * c.den = b.num * c.den * a.den := by ac_rfl
  rw [mid] at h'
  have z := Int.lt_of_le_of_lt h' k'
  have l : a.num * b.den * c.den = (a.num * c.den) * b.den := by ac_rfl
  have r : c.num * b.den * a.den = (c.num * a.den) * b.den := by ac_rfl
  rw [l, r] at z
  exact Int.lt_of_mul_lt_mul_right z (Int.le_of_lt b.den_pos)

theorem half_add_self (a : Fraction) : equiv (add a.half a.half) a := by
  simp only [equiv, add, half]

```

```

simp only [show (2 : Int)=1+1 by rfl, Int.add_mul, Int.mul_add]
ac_nf

theorem half_lt (a : Fraction) (h : positive a) : lt a.half a := by
  have hp := Int.mul_pos h a.den_pos
  unfold lt half
  dsimp
  have e : a.num * (2 * a.den) = 2 * (a.num * a.den) := by ac_rfl
  rw [e]
  omega

def magnitudes : Magnitudes Fraction where
  positive := positive
  lt := lt
  le := le
  inhabited_positive := ⟨ofInt 1, by unfold positive ofInt; decide⟩
  lt_irrefl := fun _ => Int.lt_irrefl _
  le_refl := fun _ => Int.le_refl _
  le_trans := transfer
  lt_implies_le := Int.le_of_lt
  surrounds := by
    intro c
    refine ⟨(c.num - 1, c.den, c.den_pos), (c.num + 1, c.den, c.den_pos), ?_, ?_⟩
    · unfold lt
      dsimp
      rw [Int.sub_mul, Int.one_mul]
      have := c.den_pos
      omega
    · unfold lt
      dsimp
      rw [Int.add_mul, Int.one_mul]
      have := c.den_pos
      omega
  lt_of_lt_le := mixed
  lt_of_le_lt := mixed'
  shrink := fun d hd => ⟨d.half, hd, half_lt d hd⟩
  refine := by
    intro a b ha hb
    by_cases h : le a b
    · exact ⟨a, ha, fun _ _ hx => ⟨hx, mixed hx h⟩⟩
    · have hba : le b a := by unfold le at *; omega
      exact ⟨b, hb, fun _ _ hx => ⟨mixed hx hba, hx⟩⟩

theorem equiv_iff_mutual_le (a b : Fraction) : equiv a b ↔ le a b ∧ le b a := by
  unfold equiv le
  omega

theorem equiv_refl (a : Fraction) : equiv a a := rfl

theorem equiv_symm {a b : Fraction} (h : equiv a b) : equiv b a := h.symm

theorem equiv_trans {a b c : Fraction} (h : equiv a b) (k : equiv b c) : equiv a c := by
  obtain ⟨hab, hba⟩ := (equiv_iff_mutual_le a b).mp h
  obtain ⟨hbc, hcb⟩ := (equiv_iff_mutual_le b c).mp k
  exact (equiv_iff_mutual_le a c).mpr (transfer hab hbc, transfer hcb hba)

theorem le_of_equiv {a b : Fraction} (h : equiv a b) : le a b :=
  ((equiv_iff_mutual_le a b).mp h).1

theorem le_equiv_right {a b c : Fraction} (h : le a b) (k : equiv b c) : le a c :=
  transfer h (le_of_equiv k)

theorem le_equiv_left {a b c : Fraction} (h : equiv a b) (k : le b c) : le a c :=
  transfer (le_of_equiv h) k

/-- Addition preserves weak order: arithmetic and comparisons are compatible. -/
theorem add_le_add_right {a b : Fraction} (h : le a b) (c : Fraction) :
  le (add a c) (add b c) := by
  have hp := Int.mul_pos c.den_pos c.den_pos
  have hm := Int.mul_le_mul_of_nonneg_right h (Int.le_of_lt hp)

```

```

  unfold le add at *
  dsimp
  simp only [Int.add_mul]
  have e1 : a.num * c.den * (b.den * c.den) = (a.num * b.den) * (c.den * c.den) := by ac_rfl
  have e2 : b.num * c.den * (a.den * c.den) = (b.num * a.den) * (c.den * c.den) := by ac_rfl
  have e3 : c.num * a.den * (b.den * c.den) = c.num * b.den * (a.den * c.den) := by ac_rfl
  rw [e1, e2, e3]
  omega

theorem mul_le_mul_positive {a b : Fraction} (h : le a b) (c : Fraction)
  (hc : positive c) : le (mul a c) (mul b c) := by
  have hp := Int.mul_pos hc c.den_pos
  have hm := Int.mul_le_mul_of_nonneg_right h (Int.le_of_lt hp)
  unfold le mul at *
  dsimp
  have e1 : a.num * c.num * (b.den * c.den) = (a.num * b.den) * (c.num * c.den) := by ac_rfl
  have e2 : b.num * c.num * (a.den * c.den) = (b.num * a.den) * (c.num * c.den) := by ac_rfl
  rw [e1, e2]
  exact hm

/-- Cancel a positive rational factor in a weak inequality. -/
theorem mul_le_cancel_positive_right {a b : Fraction} (c : Fraction)
  (hc : positive c) (h : le (mul a c) (mul b c)) : le a b := by
  have hp := Int.mul_pos hc c.den_pos
  unfold le mul at h
  have e1 : a.num * c.num * (b.den * c.den) = (a.num * b.den) * (c.num * c.den) := by ac_rfl
  have e2 : b.num * c.num * (a.den * c.den) = (b.num * a.den) * (c.num * c.den) := by ac_rfl
  rw [e1, e2] at h
  exact Int.le_of_mul_le_mul_right h hp

theorem add_comm (a b : Fraction) : equiv (add a b) (add b a) := by
  unfold equiv add
  dsimp
  simp only [Int.mul_comm, Int.add_comm]

theorem add_le_add_left {a b : Fraction} (h : le a b) (c : Fraction) :
  le (add c a) (add c b) :=
  le_equiv_left (add_comm c a)
  (le_equiv_right (add_le_add_right h c) (add_comm b c))

theorem add_le_add {a b c d : Fraction} (h : le a b) (k : le c d) :
  le (add a c) (add b d) :=
  transfer (add_le_add_right h c) (add_le_add_left k b)

/-- Weak-order multiplication also permits a zero factor. -/
theorem mul_le_mul_nonnegative {a b : Fraction} (h : le a b) (c : Fraction)
  (hc : 0 ≤ c.num) : le (mul a c) (mul b c) := by
  have hp := Int.mul_nonneg hc (Int.le_of_lt c.den_pos)
  have hm := Int.mul_le_mul_of_nonneg_right h hp
  unfold le mul at *
  dsimp
  have e1 : a.num * c.num * (b.den * c.den) = (a.num * b.den) * (c.num * c.den) := by ac_rfl
  have e2 : b.num * c.num * (a.den * c.den) = (b.num * a.den) * (c.num * c.den) := by ac_rfl
  rw [e1, e2]
  exact hm

theorem mul_comm (a b : Fraction) : equiv (mul a b) (mul b a) := by
  unfold equiv mul
  dsimp
  ac_rfl

theorem mul_le_mul_nonnegative_left {a b : Fraction} (h : le a b) (c : Fraction)
  (hc : 0 ≤ c.num) : le (mul c a) (mul c b) :=
  le_equiv_left (mul_comm c a)
  (le_equiv_right (mul_le_mul_nonnegative h c hc) (mul_comm b c))

/-- Cancellation can make the triangle estimate strict; absolute sums cannot
  be replaced by the absolute value of the total signed sum. -/
theorem abs_add_strict_example :
  lt (add ((1, 2, by decide) : Fraction) (-1, 2, by decide)).abs

```



```

      (add ((1, 2, by decide) : Fraction).abs ((-1, 2, by decide) : Fraction).abs) := by
    unfold lt add abs
    decide

/-- The actual ratio s/t^2; time has to be positive before division is built. -/
def deflectionRatio (s t : Fraction) (ht : positive t) : Fraction :=
  (s.num * (t.den * t.den), s.den * (t.num * t.num),
   Int.mul_pos s.den_pos (Int.mul_pos ht ht))

/-- For a triangle of height k*t, twice its area is k*t^2. No geometric
    existence is inferred from this algebraic cancellation. -/
theorem square_ratio (k t : Fraction) (ht : positive t) :
  equiv (deflectionRatio (mul k (mul t t)) t ht) k := by
  unfold equiv deflectionRatio mul
  dsimp
  ac_rfl

theorem add_assoc (a b c : Fraction) : equiv (add (add a b) c) (add a (add b c)) := by
  unfold equiv add
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  ac_rfl

theorem mul_assoc (a b c : Fraction) : equiv (mul (mul a b) c) (mul a (mul b c)) := by
  unfold equiv mul
  dsimp
  ac_rfl

theorem mul_add (a b c : Fraction) : equiv (mul a (add b c)) (add (mul a b) (mul a c)) := by
  unfold equiv mul add
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  ac_rfl

theorem positive_mul (a b : Fraction) (ha : positive a) (hb : positive b) :
  positive (mul a b) := Int.mul_pos ha hb

theorem positive_iff_zero_lt (a : Fraction) : positive a ↔ lt (ofInt 0) a := by
  unfold positive lt_ofInt
  simp

instance (a : Fraction) : Decidable (positive a) := inferInstanceAs (Decidable (0 < a.num))

def ratio (s : Fraction → Fraction) (t : Fraction) : Fraction :=
  if ht : positive t then deflectionRatio (s t) t ht else ofInt 0

/-- Transfer under pointwise equality of represented ratios on positive times. -/
theorem ultimate_congr (f k : Fraction → Fraction) (c : Fraction)
  (he : ∀ t, positive t → equiv (f t) (k t))
  (hk : Ultimate magnitudes k c) : Ultimate magnitudes f c := by
  intro a b hac hcb
  obtain ⟨d, hd, hp⟩ := hk a b hac hcb
  refine ⟨d, hd, ?_⟩
  intro t ht htd
  obtain ⟨hal, hub⟩ := hp t ht htd
  obtain ⟨hfk, hkf⟩ := (equiv_iff_mutual_le _ _).mp (he t ht)
  exact ⟨mixed hal hkf, mixed' hfk hub⟩

/-- Normalised doubled triangle area has the secant-slope limit. The slope
    limit is the remaining contact premise; a triangle limit is not a field. -/
theorem triangle_normalized_limit (slope : Fraction → Fraction) (c : Fraction)
  (hs : Ultimate magnitudes slope c) :
  Ultimate magnitudes (ratio (fun t => mul (slope t) (mul t t))) c := by
  apply ultimate_congr _ slope c
  · intro t ht
    simp only [ratio, dif_pos ht]
    exact square_ratio (slope t) t ht
  · exact hs

/-- Euclidean triangle area from base and corresponding perpendicular height. -/

```

```

def triangleArea (base height : Fraction) : Fraction := half (mul base height)

theorem triangle_area_ratio (slope time : Fraction) (ht : positive time) :
  equiv (deflectionRatio (triangleArea time (mul slope time)) time ht) (half slope) := by
  unfold equiv deflectionRatio triangleArea half mul
  dsimp
  ac_rfl

/-- Actual half-base-times-height triangles, with the contact limit displayed
    as the limit of half the secant slope. -/
theorem constructed_triangle_limit (slope : Fraction → Fraction) (c : Fraction)
  (hs : Ultimate magnitudes (fun t => half (slope t)) c) :
  Ultimate magnitudes (ratio (fun t => triangleArea t (mul (slope t) t))) c := by
  apply ultimate_congr _ (fun t => half (slope t)) c
  · intro t ht
    simp only [ratio, dif_pos ht]
    exact triangle_area_ratio (slope t) t ht
  · exact hs

/-- Arithmetic respects equality of represented magnitudes. -/
theorem mul_equiv_left (k : Fraction) {a b : Fraction} (h : equiv a b) :
  equiv (mul k a) (mul k b) := by
  unfold equiv mul at *
  dsimp
  calc
    k.num*a.num*(k.den*b.den) = (k.num*k.den)*(a.num*b.den) := by ac_rfl
    _ = (k.num*k.den)*(b.num*a.den) := by rw [h]
    _ = k.num*b.num*(k.den*a.den) := by ac_rfl

theorem add_equiv_right (k : Fraction) {a b : Fraction} (h : equiv a b) :
  equiv (add a k) (add b k) := by
  obtain (hab, hba) := (equiv_iff_mutual_le a b).mp h
  exact (equiv_iff_mutual_le _ _).mpr (add_le_add_right hab k, add_le_add_right hba k)

-- Congruence, distributivity and sign lemmas shared by the Polygon modules.

theorem add_equiv {a b c d : Fraction} (h : equiv a b)
  (k : equiv c d) :
  equiv (add a c) (add b d) :=
  equiv_trans (add_equiv_right c h)
  (equiv_trans (add_comm b c)
  (equiv_trans (add_equiv_right b k) (add_comm d b)))

theorem mul_equiv {a b c d : Fraction} (h : equiv a b)
  (k : equiv c d) :
  equiv (mul a c) (mul b d) :=
  equiv_trans (mul_comm a c)
  (equiv_trans (mul_equiv_left c h)
  (equiv_trans (mul_comm c b) (mul_equiv_left b k)))

theorem add_equiv_left (c : Fraction) {a b : Fraction} (h : equiv a b) :
  equiv (add c a) (add c b) := by
  exact equiv_trans (add_comm c a)
  (equiv_trans (add_equiv_right c h) (add_comm b c))

theorem mul_equiv_right (c : Fraction) {a b : Fraction} (h : equiv a b) :
  equiv (mul a c) (mul b c) := by
  exact equiv_trans (mul_comm a c)
  (equiv_trans (mul_equiv_left c h) (mul_comm c b))

theorem le_add_nonnegative (a b : Fraction) (hb : 0 ≤ b.num) :
  le a (add a b) := by
  unfold le add
  dsimp
  have hp := Int.mul_nonneg (Int.mul_nonneg hb (Int.le_of_lt a.den_pos))
  (Int.le_of_lt a.den_pos)
  simp only [Int.add_mul]
  have he : a.num * (a.den * b.den) = a.num * b.den * a.den := by ac_rfl
  rw [he]
  omega

```

```

theorem nonnegative_add (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) :
  0 ≤ (add a b).num := by
  unfold add
  exact Int.add_nonneg
    (Int.mul_nonneg ha (Int.le_of_lt b.den_pos))
    (Int.mul_nonneg hb (Int.le_of_lt a.den_pos))

theorem nonnegative_mul (a b : Fraction) (ha : 0 ≤ a.num) (hb : 0 ≤ b.num) :
  0 ≤ (mul a b).num := Int.mul_nonneg ha hb

theorem nonnegative_equiv {a b : Fraction} (he : equiv a b)
  (hb : 0 ≤ b.num) : 0 ≤ a.num := by
  by_cases ha : 0 ≤ a.num
  · exact ha
  · have hn : a.num < 0 := by omega
    have hm := Int.mul_lt_mul_of_pos_right hn b.den_pos
    have hp := Int.mul_nonneg hb (Int.le_of_lt a.den_pos)
    simp only [Int.zero_mul] at hm
    unfold equiv at he
    omega

theorem mul_zero (d : Fraction) :
  equiv (mul d (ofInt 0)) (ofInt 0) := by
  unfold equiv mul ofInt
  simp

theorem add_zero (c : Fraction) :
  equiv (add c (ofInt 0)) c := by
  unfold equiv add ofInt
  simp only [Int.mul_one, Int.zero_mul, Int.add_zero]
  try ac_rfl

theorem add_mul (a b c : Fraction) :
  equiv (mul (add a b) c)
    (add (mul a c) (mul b c)) := by
  unfold equiv add mul
  dsimp
  simp only [Int.add_mul, Int.mul_add]
  try ac_nf

theorem le_add_cancel_left (z a b : Fraction)
  (h : le (add z a) (add z b)) :
  le a b := by
  unfold le add at h
  unfold le
  dsimp at h
  have hL : (z.num * a.den + a.num * z.den) * (z.den * b.den) =
    z.num * a.den * z.den * b.den +
    (a.num * b.den) * (z.den * z.den) := by
    rw [Int.add_mul]
    ac_nf
  have hR : (z.num * b.den + b.num * z.den) * (z.den * a.den) =
    z.num * a.den * z.den * b.den +
    (b.num * a.den) * (z.den * z.den) := by
    rw [Int.add_mul]
    ac_nf
  rw [hL, hR] at h
  have hm : (a.num * b.den) * (z.den * z.den) ≤
    (b.num * a.den) * (z.den * z.den) := by omega
  exact Int.le_of_mul_le_mul_right hm (Int.mul_pos z.den_pos z.den_pos)

theorem add_lt_add_left {a b : Fraction}
  (hab : lt a b) (c : Fraction) :
  lt (add c a) (add c b) := by
  have hp : 0 < c.den * c.den := Int.mul_pos c.den_pos c.den_pos
  have hm := Int.mul_lt_mul_of_pos_right hab hp
  unfold lt add at *
  dsimp at *
  simp only [Int.add_mul, Int.mul_add] at *

```

```

have he1 : c.num * a.den * (c.den * b.den) =
  c.num * b.den * (c.den * a.den) := by ac_refl
have he2 : a.num * c.den * (c.den * b.den) =
  (a.num * b.den) * (c.den * c.den) := by ac_refl
have he3 : b.num * c.den * (c.den * a.den) =
  (b.num * a.den) * (c.den * c.den) := by ac_refl
rw [he1, he2, he3]
omega

theorem add_lt_add_right {a b : Fraction}
  (hab : lt a b) (c : Fraction) : lt (add a c) (add b c) :=
  magnitudes.lt_of_le_lt (le_of_equiv (add_comm a c))
  (magnitudes.lt_of_lt_le (add_lt_add_left hab c)
    (le_of_equiv (add_comm c b)))

theorem add_lt_add {a b c d : Fraction} (hab : lt a b) (hcd : lt c d) :
  lt (add a c) (add b d) :=
  magnitudes.lt_of_lt_le (add_lt_add_right hab c)
  (magnitudes.lt_implies_le (add_lt_add_left hcd b))

end Fraction
end NewtonLimitDynamics

```

BarrowLib/Common/FiniteGrowth.lean, 19 theorems, 173 lines

```

import BarrowLib.Common.RationalMagnitudes

/-!
Elementary finite growth estimates with a common positive denominator.
These rational arithmetic bounds supply no completion, limiting trajectory,
geometric area or historical analytic theorem.
-!

namespace NewtonLimitDynamics.FiniteGrowth

def weightSum : List Int → Int
| [] => 0
| a :: xs => a + weightSum xs

def factorProduct (D : Int) : List Int → Int
| [] => 1
| a :: xs => (D + a) * factorProduct D xs

def Nonnegative (xs : List Int) : Prop := ∀ a ∈ xs, 0 ≤ a

theorem weightSum_nonnegative (xs : List Int) (hx : Nonnegative xs) :
  0 ≤ weightSum xs := by
  induction xs with
  | nil => exact Int.le_refl 0
  | cons a xs ih =>
    exact Int.add_nonneg (hx a (by simp))
    (ih (fun b hb => hx b (by simp [hb])))

theorem factorProduct_nonnegative (D : Int) (hD : 0 ≤ D)
  (xs : List Int) (hx : Nonnegative xs) : 0 ≤ factorProduct D xs := by
  induction xs with
  | nil => simp [factorProduct]
  | cons a xs ih =>
    exact Int.mul_nonneg (Int.add_nonneg hD (hx a (by simp)))
    (ih (fun b hb => hx b (by simp [hb])))

theorem cofactor_step (D a S : Int) (ha : 0 ≤ a) (hS : 0 ≤ S) :
  (D + a) * (D - (a + S)) ≤ D * (D - S) := by
  have hp : 0 ≤ a * a + a * S :=
    Int.add_nonneg (Int.mul_nonneg ha ha) (Int.mul_nonneg ha hS)
  have he : (D + a) * (D - (a + S)) + (a * a + a * S) =
    D * (D - S) := by
    simp only [Int.mul_sub, Int.sub_mul, Int.add_mul, Int.mul_add]
    ac_nf <=> omega
  omega

```

```

/-- Finite product/cofactor estimate. It even holds when the cofactor is
negative; the useful uniform conclusion additionally bounds the total. -/
theorem cofactor_bound (D : Int) (hD : 0 < D)
  (xs : List Int) (hx : Nonnegative xs) :
    factorProduct D xs * (D - weightSum xs) ≤ D ^ (xs.length + 1) := by
  induction xs with
  | nil => simp [factorProduct, weightSum, Int.pow_succ]
  | cons a xs ih =>
    have ha := hx a (by simp)
    have ht : Nonnegative xs := fun b hb => hx b (by simp [hb])
    have hP := factorProduct_nonnegative D (Int.le_of_lt hD) xs ht
    have hs := weightSum_nonnegative xs ht
    have step := Int.mul_le_mul_of_nonneg_left (cofactor_step D a (weightSum xs) ha hs) hP
    have next := Int.mul_le_mul_of_nonneg_left (ih ht) (Int.le_of_lt hD)
    simp only [factorProduct, weightSum, List.length_cons]
    rw [Int.pow_succ']
    calc
      (D + a) * factorProduct D xs * (D - (a + weightSum xs)) =
        factorProduct D xs * ((D + a) * (D - (a + weightSum xs))) := by ac_rfl
      _ ≤ factorProduct D xs * (D * (D - weightSum xs)) := step
      _ = D * (factorProduct D xs * (D - weightSum xs)) := by ac_rfl
      _ ≤ D * D ^ (xs.length + 1) := next

/-- Uniform finite growth at small total increment, including empty lists. -/
theorem uniform_product_bound (D : Int) (hD : 0 < D)
  (xs : List Int) (hx : Nonnegative xs) (hsmall : 2 * weightSum xs ≤ D) :
    factorProduct D xs ≤ 2 * D ^ xs.length := by
  have hP := factorProduct_nonnegative D (Int.le_of_lt hD) xs hx
  have hco := cofactor_bound D hD xs hx
  have hs : D ≤ 2 * (D - weightSum xs) := by omega
  have hm := Int.mul_le_mul_of_nonneg_left hs hP
  have hb := Int.mul_le_mul_of_nonneg_left hco (by decide : (0 : Int) ≤ 2)
  have hchain : factorProduct D xs * D ≤ (2 * D ^ xs.length) * D := by
    calc
      factorProduct D xs * D ≤ factorProduct D xs * (2 * (D - weightSum xs)) := hm
      _ = 2 * (factorProduct D xs * (D - weightSum xs)) := by ac_rfl
      _ ≤ 2 * D ^ (xs.length + 1) := hb
      _ = (2 * D ^ xs.length) * D := by rw [Int.pow_succ, Int.mul_assoc]
  exact Int.le_of_mul_le_mul_right hchain hD

def factor (D : Int) (hD : 0 < D) (a : Int) : Fraction := (D + a, D, hD)

def amplification (D : Int) (hD : 0 < D) (xs : List Int) : Fraction :=
  (factorProduct D xs, D ^ xs.length, Int.pow_pos hD)

theorem factor_eq_one_add (D : Int) (hD : 0 < D) (a : Int) :
  Fraction.equiv (factor D hD a)
    (Fraction.add (Fraction.ofInt 1) (a, D, hD)) := by
  simp only [factor, Fraction.equiv, Fraction.add, Fraction.ofInt,
    Int.one_mul, Int.mul_one]

theorem amplification_empty (D : Int) (hD : 0 < D) :
  Fraction.equiv (amplification D hD []) (Fraction.ofInt 1) := by
  simp [amplification, factorProduct, Fraction.equiv, Fraction.ofInt]

theorem amplification_cons (D : Int) (hD : 0 < D) (a : Int) (xs : List Int) :
  Fraction.equiv (amplification D hD (a :: xs))
    (Fraction.mul (factor D hD a) (amplification D hD xs)) := by
  simp only [amplification, factor, factorProduct, List.length_cons,
    Fraction.equiv, Fraction.mul, Int.pow_succ']

theorem amplification_nonnegative (D : Int) (hD : 0 < D)
  (xs : List Int) (hx : Nonnegative xs) :
  0 ≤ (amplification D hD xs).num :=
  factorProduct_nonnegative D (Int.le_of_lt hD) xs hx

theorem uniform_amplification (D : Int) (hD : 0 < D)
  (xs : List Int) (hx : Nonnegative xs) (hsmall : 2 * weightSum xs ≤ D) :
  Fraction.le (amplification D hD xs) (Fraction.ofInt 2) := by
  simpa only [amplification, Fraction.le, Fraction.ofInt, Int.mul_one] using

```

```

uniform_product_bound D hD xs hx hsmall

theorem weightSum_append (xs ys : List Int) :
  weightSum (xs ++ ys) = weightSum xs + weightSum ys := by
  induction xs with
  | nil => simp [weightSum]
  | cons a xs ih => simp only [List.cons_append, weightSum, ih, Int.add_assoc]

theorem factorProduct_append (D : Int) (xs ys : List Int) :
  factorProduct D (xs ++ ys) = factorProduct D xs * factorProduct D ys := by
  induction xs with
  | nil => simp [factorProduct]
  | cons a xs ih => simp only [List.cons_append, factorProduct, ih, Int.mul_assoc]

theorem denominator_power_add (D : Int) (m n : Nat) :
  D ^ (m + n) = D ^ m * D ^ n := by
  induction n with
  | zero => simp
  | succ n ih => simp only [← Nat.add_assoc, Int.pow_succ, ih, Int.mul_assoc]

theorem amplification_append (D : Int) (hD : 0 < D) (xs ys : List Int) :
  Fraction.equiv (amplification D hD (xs ++ ys))
    (Fraction.mul (amplification D hD xs) (amplification D hD ys)) := by
  simp only [amplification, factorProduct_append, List.length_append,
    denominator_power_add, Fraction.equiv, Fraction.mul]

theorem weightSum_replicate (a : Int) (n : Nat) :
  weightSum (List.replicate n a) = (n : Int) * a := by
  induction n with
  | zero => simp [weightSum]
  | succ n ih =>
    simp only [List.replicate_succ, weightSum, ih, Int.natCast_add,
      Int.natCast_one, Int.add_mul, Int.one_mul]
    omega

theorem factorProduct_replicate (D a : Int) (n : Nat) :
  factorProduct D (List.replicate n a) = (D + a) ^ n := by
  induction n with
  | zero => simp [factorProduct]
  | succ n ih => simp only [List.replicate_succ, factorProduct, ih, Int.pow_succ']

theorem zero_increments (D : Int) (hD : 0 < D) (n : Nat) :
  Fraction.equiv (amplification D hD (List.replicate n 0)) (Fraction.ofInt 1) := by
  simp only [amplification, factorProduct_replicate, List.length_replicate,
    Fraction.equiv, Fraction.ofInt, Int.add_zero, Int.mul_one, Int.one_mul]

theorem boundary_sample :
  Fraction.equiv (amplification 4 (by decide) [1, 1]) (25, 16, by decide) := by
  unfold Fraction.equiv
  decide

theorem missing_smallness_counterexample :
  ¬ Fraction.le (amplification 1 (by decide) [1, 1]) (Fraction.ofInt 2) := by
  unfold Fraction.le amplification factorProduct
  decide

end NewtonLimitDynamics.FiniteGrowth

```

BarrowLib/Polygon/PointAlgebra.lean, 9 theorems, 106 lines

```
import BarrowLib.Common.RationalMagnitudes
```

```
namespace NewtonLimitDynamics.Polygon.TimeSubdivision
```

```
open NewtonLimitDynamics
```

```

/-- A two-dimensional coordinate point whose components are signed rational
representatives. -/
abbrev Point := Fraction × Fraction

```

```
def pointAdd (p q : Point) : Point := (Fraction.add p.1 q.1, Fraction.add p.2 q.2)
```

```

def pointScale (d : Fraction) (p : Point) : Point :=
  (Fraction.mul d p.1, Fraction.mul d p.2)
def pointNeg (p : Point) : Point :=
  ((-p.1.num, p.1.den, p.1.den_pos), (-p.2.num, p.2.den, p.2.den_pos))
def pointSub (p q : Point) : Point := pointAdd p (pointNeg q)

def pointEquiv (p q : Point) : Prop :=
  Fraction.equiv p.1 q.1 ∧ Fraction.equiv p.2 q.2

instance (a b : Fraction) : Decidable (Fraction.equiv a b) :=
  if h : a.num * b.den = b.num * a.den then isTrue h else isFalse h
instance (p q : Point) : Decidable (pointEquiv p q) :=
  if h1 : Fraction.equiv p.1 q.1 then
    if h2 : Fraction.equiv p.2 q.2 then isTrue (h1, h2)
    else isFalse (fun h => h2 h.2)
  else isFalse (fun h => h1 h.1)

-- Point equivalence lemmas shared by the Polygon modules.

theorem pointEquiv_symm {p q : Point} (h : pointEquiv p q) : pointEquiv q p :=
  (Fraction.equiv_symm h.1, Fraction.equiv_symm h.2)

theorem pointEquiv_trans {p q r : Point} (h : pointEquiv p q) (k : pointEquiv q r) :
  pointEquiv p r :=
  (Fraction.equiv_trans h.1 k.1, Fraction.equiv_trans h.2 k.2)

theorem pointAdd_congr {p p' q q' : Point}
  (hp : pointEquiv p p') (hq : pointEquiv q q') :
  pointEquiv (pointAdd p q) (pointAdd p' q') := by
  constructor
  · exact Fraction.equiv_trans (Fraction.add_equiv_right q.1 hp.1)
    (Fraction.equiv_trans (Fraction.add_comm p'.1 q.1)
      (Fraction.equiv_trans (Fraction.add_equiv_right p'.1 hq.1)
        (Fraction.add_comm q'.1 p'.1)))
  · exact Fraction.equiv_trans (Fraction.add_equiv_right q.2 hp.2)
    (Fraction.equiv_trans (Fraction.add_comm p'.2 q.2)
      (Fraction.equiv_trans (Fraction.add_equiv_right p'.2 hq.2)
        (Fraction.add_comm q'.2 p'.2)))

theorem pointScale_congr (d : Fraction) {p q : Point} (h : pointEquiv p q) :
  pointEquiv (pointScale d p) (pointScale d q) :=
  (Fraction.mul_equiv_left d h.1, Fraction.mul_equiv_left d h.2)

/-- Scaling respects rational value equivalence in both arguments. -/
theorem pointScale_ratio_congr {d e : Fraction} {p q : Point}
  (hde : Fraction.equiv d e) (hpq : pointEquiv p q) :
  pointEquiv (pointScale d p) (pointScale e q) :=
  (Fraction.mul_equiv hde hpq.1, Fraction.mul_equiv hde hpq.2)

theorem pointNeg_congr {p q : Point} (hpq : pointEquiv p q) :
  pointEquiv (pointNeg p) (pointNeg q) := by
  constructor
  · change -p.1.num * q.1.den = -q.1.num * p.1.den
    simp only [Int.neg_mul] using congrArg Neg.neg hpq.1
  · change -p.2.num * q.2.den = -q.2.num * p.2.den
    simp only [Int.neg_mul] using congrArg Neg.neg hpq.2

theorem pointSub_congr {p p' q q' : Point}
  (hpp' : pointEquiv p p') (hqq' : pointEquiv q q') :
  pointEquiv (pointSub p q) (pointSub p' q') :=
  pointAdd_congr hpp' (pointNeg_congr hqq')

theorem pointSub_zero (p : Point) :
  pointEquiv (pointSub p (Fraction.ofInt 0, Fraction.ofInt 0)) p := by
  constructor <=> simp only [pointSub, pointAdd, pointNeg, Fraction.equiv, Fraction.add,
    Fraction.ofInt, Int.neg_zero, Int.zero_mul, Int.mul_one, Int.add_zero]

def det (p q : Point) : Fraction :=
  Fraction.add (Fraction.mul p.1 q.2) ((- (Fraction.mul p.2 q.1).num,

```

```

(Fraction.mul p.2 q.1).den, (Fraction.mul p.2 q.1).den_pos))

theorem det_add_right (x v w : Point) :
  Fraction.equiv (det x (pointAdd v w)) (Fraction.add (det x v) (det x w)) := by
  unfold Fraction.equiv det pointAdd Fraction.add Fraction.mul
  dsimp
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

/-- Signed doubled determinant sum for p -> B -> D -> C -> p. -/
def closedBoundaryTwice (p b d c : Point) : Fraction :=
  Fraction.add (Fraction.add (det p b) (det b d))
  (Fraction.add (det d c) (det c p))

end NewtonLimitDynamics.Polygon.TimeSubdivision

namespace NewtonLimitDynamics.Polygon.HarmonicStability
open NewtonLimitDynamics
open TimeSubdivision
def dot (p q : Point) : Fraction :=
  Fraction.add (Fraction.mul p.1 q.1) (Fraction.mul p.2 q.2)
end NewtonLimitDynamics.Polygon.HarmonicStability

BarrowLib/Polygon/PointBounds.lean, 11 theorems, 93 lines

import BarrowLib.Polygon.PointAlgebra

/-!
Coordinate L1 magnitudes for finite rational polygon data. These diagnostics
depend on the chosen coordinates; they are neither Euclidean area nor a
physical position/velocity norm without a calibration of their units.
-/

namespace NewtonLimitDynamics.Polygon.PointBounds
open NewtonLimitDynamics
open TimeSubdivision

def pointNorm (p : Point) : Fraction := Fraction.add p.1.abs p.2.abs
def stateNorm (s : Point × Point) : Fraction :=
  Fraction.add (pointNorm s.1) (pointNorm s.2)

def stateEquiv (s t : Point × Point) : Prop :=
  pointEquiv s.1 t.1 ∧ pointEquiv s.2 t.2

theorem pointNorm_nonnegative (p : Point) : 0 ≤ (pointNorm p).num := by
  unfold pointNorm Fraction.add
  exact Int.add_nonneg
    (Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Int.le_of_lt p.2.abs.den_pos))
    (Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Int.le_of_lt p.1.abs.den_pos))

theorem stateNorm_nonnegative (s : Point × Point) : 0 ≤ (stateNorm s).num := by
  unfold stateNorm Fraction.add
  exact Int.add_nonneg
    (Int.mul_nonneg (pointNorm_nonnegative _) (Int.le_of_lt (pointNorm s.2).den_pos))
    (Int.mul_nonneg (pointNorm_nonnegative _) (Int.le_of_lt (pointNorm s.1).den_pos))

theorem pointNorm_equiv {p q : Point} (h : pointEquiv p q) :
  Fraction.equiv (pointNorm p) (pointNorm q) :=
  Fraction.add_equiv (Fraction.abs_equiv h.1) (Fraction.abs_equiv h.2)

theorem pointNorm_neg (p : Point) :
  Fraction.equiv (pointNorm (pointNeg p)) (pointNorm p) :=
  Fraction.add_equiv (Fraction.abs_neg p.1) (Fraction.abs_neg p.2)

theorem stateNorm_equiv {s t : Point × Point} (h : stateEquiv s t) :
  Fraction.equiv (stateNorm s) (stateNorm t) :=
  Fraction.add_equiv (pointNorm_equiv h.1) (pointNorm_equiv h.2)

```



```

theorem pointNorm_add_le (p q : Point) :
  Fraction.le (pointNorm (pointAdd p q))
    (Fraction.add (pointNorm p) (pointNorm q)) := by
  have h := Fraction.add_le_add (Fraction.abs_add_le p.1 q.1)
    (Fraction.abs_add_le p.2 q.2)
  apply Fraction.le_equiv_right h
  simp only [Fraction.equiv, Fraction.add, pointNorm]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem pointNorm_scale (d : Fraction) (p : Point) :
  Fraction.equiv (pointNorm (pointScale d p)) (Fraction.mul d.abs (pointNorm p)) := by
  simp only [Fraction.equiv, pointNorm, pointScale, Fraction.abs, Fraction.add,
    Fraction.mul, Int.natAbs_mul, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem stateNorm_add_le (s t : Point × Point) :
  Fraction.le (stateNorm (pointAdd s.1 t.1, pointAdd s.2 t.2))
    (Fraction.add (stateNorm s) (stateNorm t)) := by
  have h := Fraction.add_le_add (pointNorm_add_le s.1 t.1)
    (pointNorm_add_le s.2 t.2)
  apply Fraction.le_equiv_right h
  simp only [Fraction.equiv, Fraction.add, stateNorm]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem stateNorm_scale (d : Fraction) (s : Point × Point) :
  Fraction.equiv
    (stateNorm (pointScale d s.1, pointScale d s.2))
    (Fraction.mul d.abs (stateNorm s)) := by
  simp only [Fraction.equiv, stateNorm, pointNorm, pointScale, Fraction.abs,
    Fraction.add, Fraction.mul, Int.natAbs_mul, Int.ofNat_mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

-- Coordinate magnitudes of the two state components.

theorem point_le_state (s : Point × Point) :
  Fraction.le (pointNorm s.1) (stateNorm s) :=
  Fraction.le_add_nonnegative _ _ (pointNorm_nonnegative s.2)

theorem velocity_le_state (s : Point × Point) :
  Fraction.le (pointNorm s.2) (stateNorm s) :=
  Fraction.le_equiv_right
    (Fraction.le_add_nonnegative _ _ (pointNorm_nonnegative s.1))
    (Fraction.add_comm (pointNorm s.2) (pointNorm s.1))

end NewtonLimitDynamics.Polygon.PointBounds

```

BarrowLib/Polygon/FiniteEstimates.lean, 46 theorems, 597 lines

```

import BarrowLib.Polygon.PointBounds
import BarrowLib.Polygon.ConvexCover

/--! Finite coordinate estimates for an arbitrary rational point map. The
coordinate L1 magnitude is a chosen algebraic gauge; no trajectory or force
law is asserted. -/

namespace NewtonLimitDynamics.Polygon.FiniteEstimates

open NewtonLimitDynamics
open TimeSubdivision
open PointBounds

def pointDistance (p q : Point) : Fraction := pointNorm (pointSub p q)

def stateDistance (s t : Point × Point) : Fraction :=
  Fraction.add (pointDistance s.1 t.1) (pointDistance s.2 t.2)

```

```

theorem pointNorm_le_distance_add (p q : Point) :
  Fraction.le (pointNorm p) (Fraction.add (pointDistance p q) (pointNorm q)) := by
  have he : pointEquiv p (pointAdd (pointSub p q) q) := by
    constructor <=> simp only [pointEquiv, pointSub, pointNeg, pointAdd, Fraction.equiv,
      Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=> ac_nf <=> omega
  exact Fraction.le_equiv_left (pointNorm_equiv he) (pointNorm_add_le _ _)

/-- Reuse the generic point-subtraction triangle estimate. -/
theorem pointDistance_triangle (p q r : Point) :
  Fraction.le (pointDistance p r)
    (Fraction.add (pointDistance p q) (pointDistance q r)) :=
  ConvexCover.pointSub_triangle p q r

theorem pointDistance_symm (p q : Point) :
  Fraction.equiv (pointDistance p q) (pointDistance q p) := by
  have he : pointEquiv (pointSub p q) (pointNeg (pointSub q p)) := by
    constructor <=>
      simp only [pointEquiv, pointSub, pointNeg, pointAdd, Fraction.equiv,
        Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
        ac_nf <=> omega
  exact Fraction.equiv_trans (pointNorm_equiv he) (pointNorm_neg (pointSub q p))

theorem comparisonContract_reverse (a b : Point → Point) (L E : Fraction)
  (hc : ∀ p q, Fraction.le (pointDistance (a p) (b q))
    (Fraction.add (Fraction.mul L (pointDistance p q)) E)) :
  ∀ p q, Fraction.le (pointDistance (b p) (a q))
    (Fraction.add (Fraction.mul L (pointDistance p q)) E) := by
  intro p q
  exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (pointDistance_symm _ _) (hc q p))
    (Fraction.add_equiv (Fraction.mul_equiv_left L (pointDistance_symm q p))
      (Fraction.equiv_refl E))

/-- Triangle inequality for the coordinate distance of actual states. -/
theorem stateDistance_triangle (s t u : Point × Point) :
  Fraction.le (stateDistance s u)
    (Fraction.add (stateDistance s t) (stateDistance t u)) := by
  have hp := pointDistance_triangle s.1 t.1 u.1
  have hv := pointDistance_triangle s.2 t.2 u.2
  exact Fraction.le_equiv_right (Fraction.add_le_add hp hv) (by
    simp only [stateDistance, pointDistance, Fraction.equiv, Fraction.add,
      Int.add_mul, Int.mul_add]
    ac_nf)

theorem stateDistance_symm (s t : Point × Point) :
  Fraction.equiv (stateDistance s t) (stateDistance t s) :=
  Fraction.add_equiv (pointDistance_symm s.1 t.1) (pointDistance_symm s.2 t.2)

theorem pointDistance_self_zero (p : Point) :
  Fraction.equiv (pointDistance p p) (Fraction.ofInt 0) :=
  ConvexCover.pointSub_self_zero p

theorem stateDistance_self_zero (s : Point × Point) :
  Fraction.equiv (stateDistance s s) (Fraction.ofInt 0) :=
  Fraction.equiv_trans
    (Fraction.add_equiv (pointDistance_self_zero s.1) (pointDistance_self_zero s.2))
    (by simp [Fraction.equiv, Fraction.add, Fraction.ofInt])

theorem pointDistance_equiv {p p' q q' : Point}
  (hp : pointEquiv p p') (hq : pointEquiv q q') :
  Fraction.equiv (pointDistance p q) (pointDistance p' q') :=
  pointNorm_equiv (pointSub_congr hp hq)

/-- Drift to the sampled arrival point, then update velocity. -/
def cell (a : Point → Point) (h : Fraction) (s : Point × Point) : Point × Point :=
  let y := pointAdd s.1 (pointScale h s.2)
  (y, pointAdd s.2 (pointScale h (a y)))

/-- This cross-map contract permits additive sampling error `E`; neither
sampled map has to be exactly Lipschitz on its own. -/

```

```

def comparisonContract (a b : Point → Point) (L E : Fraction) : Prop :=
  ∀ p q, Fraction.le (pointDistance (a p) (b q))
    (Fraction.add (Fraction.mul L (pointDistance p q)) E)

private theorem point_difference_add (p q r t : Point) :
  pointEquiv (pointSub (pointAdd p r) (pointAdd q t))
    (pointAdd (pointSub p q) (pointSub r t)) := by
  constructor <=>
  simp only [pointEquiv, pointSub, pointNeg, pointAdd, Fraction.equiv,
    Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

private theorem point_difference_scale (h : Fraction) (p q : Point) :
  pointEquiv (pointSub (pointScale h p) (pointScale h q))
    (pointScale h (pointSub p q)) := by
  constructor <=>
  simp only [pointEquiv, pointSub, pointNeg, pointScale, pointAdd,
    Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add,
    Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem difference_add_bound (p q r t : Point) :
  Fraction.le (pointDistance (pointAdd p r) (pointAdd q t))
    (Fraction.add (pointDistance p q) (pointDistance r t)) := by
  exact Fraction.le_equiv_left
    (pointNorm_equiv (point_difference_add p q r t))
    (pointNorm_add_le (pointSub p q) (pointSub r t))

theorem pointDistance_neg (p q : Point) :
  Fraction.equiv (pointDistance (pointNeg p) (pointNeg q)) (pointDistance p q) := by
  have he : pointEquiv (pointSub (pointNeg p) (pointNeg q)) (pointNeg (pointSub p q)) := by
  constructor <=>
  simp only [pointEquiv, pointSub, pointNeg, pointAdd, Fraction.equiv, Fraction.add,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.neg_neg] <=> ac_nf <=> omega
  exact Fraction.equiv_trans (pointNorm_equiv he) (pointNorm_neg (pointSub p q))

theorem difference_sub_bound (p q r t : Point) :
  Fraction.le (pointDistance (pointSub p r) (pointSub q t))
    (Fraction.add (pointDistance p q) (pointDistance r t)) :=
  Fraction.le_equiv_right (difference_add_bound p q (pointNeg r) (pointNeg t))
    (Fraction.add_equiv (Fraction.equiv_refl _) (pointDistance_neg r t))

theorem difference_scale (h : Fraction) (p q : Point) :
  Fraction.equiv (pointDistance (pointScale h p) (pointScale h q))
    (Fraction.mul h.abs (pointDistance p q)) :=
  Fraction.equiv_trans (pointNorm_equiv (point_difference_scale h p q))
    (pointNorm_scale h (pointSub p q))

def amplification (L h : Fraction) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) h.abs)
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs L))

/-- The excess in this coefficient bound is a sum of nonnegative products. -/
theorem component_amplification (P V H L E : Fraction)
  (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hH : 0 ≤ H.num) (hL : 0 ≤ L.num) :
  Fraction.le
    (Fraction.add (Fraction.add P (Fraction.mul H V))
      (Fraction.add V (Fraction.mul H
        (Fraction.add (Fraction.mul L (Fraction.add P (Fraction.mul H V))) E))))
    (Fraction.add
      (Fraction.mul (Fraction.mul (Fraction.add (Fraction.ofInt 1) H)
        (Fraction.add (Fraction.ofInt 1) (Fraction.mul H L))) (Fraction.add P V))
      (Fraction.mul H E)) := by
  let A := Fraction.add (Fraction.add P (Fraction.mul H V))
    (Fraction.add V (Fraction.mul H
      (Fraction.add (Fraction.mul L (Fraction.add P (Fraction.mul H V))) E)))
  let S := Fraction.add (Fraction.mul H P)
    (Fraction.add (Fraction.mul (Fraction.mul H L) V)
      (Fraction.mul (Fraction.mul (Fraction.mul H H) L) P))
  have hS : 0 ≤ S.num := Fraction.nonnegative_add _ _

```

```

(Fraction.nonnegative_mul _ _ hH hP)
(Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hH hL) hV)
  (Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hH hH) hL) hP))
apply Fraction.le_equiv_right (Fraction.le_add_nonnegative A S hS)
simp only [A, S, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
  Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
ac_nf

/-- Position growth of one triangular cell. -/
theorem cell_position_growth (a : Point → Point) (h : Fraction)
  (s : Point × Point) :
  Fraction.le (pointNorm (cell a h s).1)
    (Fraction.add (pointNorm s.1)
      (Fraction.mul h.abs (pointNorm s.2))) := by
exact Fraction.le_equiv_right (pointNorm_add_le s.1 (pointScale h s.2))
  (Fraction.add_equiv (Fraction.equiv_refl _)
    (pointNorm_scale h s.2))

/-- Velocity growth under a bound at the sampled arrival point. -/
theorem cell_velocity_growth (a : Point → Point) (h B : Fraction)
  (s : Point × Point)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) :
  Fraction.le (pointNorm (cell a h s).2)
    (Fraction.add (pointNorm s.2) (Fraction.mul h.abs B)) := by
have ha := pointNorm_add_le s.2 (pointScale h (a (cell a h s).1))
have hs := pointNorm_scale h (a (cell a h s).1)
have hm := Fraction.mul_le_mul_nonnegative_left hB h.abs
  (Fraction.abs_num_nonnegative h)
exact Fraction.magnitudes.le_trans
  (Fraction.le_equiv_right ha (Fraction.add_equiv (Fraction.equiv_refl _) hs))
  (Fraction.add_le_add_left hm (pointNorm s.2))

/-- A bounded acceleration gives finite one-cell state growth. -/
theorem cell_state_growth (a : Point → Point) (h B : Fraction)
  (s : Point × Point)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) :
  Fraction.le (stateNorm (cell a h s))
    (Fraction.add
      (Fraction.add (pointNorm s.1) (Fraction.mul h.abs (pointNorm s.2)))
      (Fraction.add (pointNorm s.2) (Fraction.mul h.abs B))) :=
Fraction.add_le_add (cell_position_growth a h s)
  (cell_velocity_growth a h B s hB)

/-- Position perturbation keeps the arrival-point displacement visible. -/
theorem cell_position_perturbation (a b : Point → Point) (h : Fraction)
  (s t : Point × Point) :
  Fraction.le (pointDistance (cell a h s).1 (cell b h t).1)
    (Fraction.add (pointDistance s.1 t.1)
      (Fraction.mul h.abs (pointDistance s.2 t.2))) := by
have hd := difference_add_bound s.1 t.1 (pointScale h s.2) (pointScale h t.2)
exact Fraction.le_equiv_right hd
  (Fraction.add_equiv (Fraction.equiv_refl _)
    (difference_scale h s.2 t.2))

/-- Velocity perturbation for arbitrary sample maps. The term `|h|E`
cannot be dropped when the two maps differ at the same point. -/
theorem cell_velocity_perturbation_at (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point)
  (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (cell a h s).1 (cell b h t).1)) E)) :
  Fraction.le (pointDistance (cell a h s).2 (cell b h t).2)
    (Fraction.add (pointDistance s.2 t.2)
      (Fraction.mul h.abs
        (Fraction.add (Fraction.mul L
          (pointDistance (cell a h s).1 (cell b h t).1)) E))) := by
have hd := difference_add_bound s.2 t.2
  (pointScale h (a (cell a h s).1))

```

```

    (pointScale h (b (cell b h t).1))
  have hs := difference_scale h (a (cell a h s).1) (b (cell b h t).1)
  have hm := Fraction.mul_le_mul_nonnegative_left hc h.abs
    (Fraction.abs_num_nonnegative h)
  exact Fraction.magnitudes.le_trans
    (Fraction.le_equiv_right hd (Fraction.add_equiv (Fraction.equiv_refl _) hs))
    (Fraction.add_le_add_left hm (pointDistance s.2 t.2))

theorem cell_velocity_perturbation (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point) (hc : comparisonContract a b L E) :
  Fraction.le (pointDistance (cell a h s).2 (cell b h t).2)
    (Fraction.add (pointDistance s.2 t.2)
      (Fraction.mul h.abs
        (Fraction.add (Fraction.mul L
          (pointDistance (cell a h s).1 (cell b h t).1)) E))) :=
  cell_velocity_perturbation_at a b h L E s t
  (hc (cell a h s).1 (cell b h t).1)

/-- The actual one-cell state perturbation with separate position and
velocity terms and the additive `|h|E` contribution. -/
theorem cell_state_perturbation_at (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point)
  (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (cell a h s).1 (cell b h t).1)) E)) :
  Fraction.le (stateDistance (cell a h s) (cell b h t))
    (Fraction.add
      (Fraction.add (pointDistance s.1 t.1)
        (Fraction.mul h.abs (pointDistance s.2 t.2)))
      (Fraction.add (pointDistance s.2 t.2)
        (Fraction.mul h.abs
          (Fraction.add (Fraction.mul L
            (pointDistance (cell a h s).1 (cell b h t).1)) E)))) :=
  Fraction.add_le_add (cell_position_perturbation a b h s t)
    (cell_velocity_perturbation_at a b h L E s t hc)

theorem cell_state_perturbation (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point) (hc : comparisonContract a b L E) :
  Fraction.le (stateDistance (cell a h s) (cell b h t))
    (Fraction.add
      (Fraction.add (pointDistance s.1 t.1)
        (Fraction.mul h.abs (pointDistance s.2 t.2)))
      (Fraction.add (pointDistance s.2 t.2)
        (Fraction.mul h.abs
          (Fraction.add (Fraction.mul L
            (pointDistance (cell a h s).1 (cell b h t).1)) E)))) :=
  cell_state_perturbation_at a b h L E s t
  (hc (cell a h s).1 (cell b h t).1)

/-- Closed one-cell recurrence when `L` is nonnegative. This does not
assert a uniform mesh bound or any limiting trajectory. -/
theorem cell_state_perturbation_closed_at (a b : Point → Point)
  (h L E : Fraction) (s t : Point × Point)
  (hL : 0 ≤ L.num)
  (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (cell a h s).1 (cell b h t).1)) E)) :
  Fraction.le (stateDistance (cell a h s) (cell b h t))
    (Fraction.add
      (Fraction.add (pointDistance s.1 t.1)
        (Fraction.mul h.abs (pointDistance s.2 t.2)))
      (Fraction.add (pointDistance s.2 t.2)
        (Fraction.mul h.abs
          (Fraction.add (Fraction.mul L
            (Fraction.add (pointDistance s.1 t.1)
              (Fraction.mul h.abs (pointDistance s.2 t.2)))) E)))) := by
  have hp := cell_position_perturbation a b h s t
  have hm := Fraction.mul_le_mul_nonnegative_left hp hL
  have he := Fraction.add_le_add_right hm E
  have hh := Fraction.mul_le_mul_nonnegative_left he h.abs

```

```

(Fraction.abs_num_nonnegative h)
have hv := Fraction.add_le_add_left hh (pointDistance s.2 t.2)
exact Fraction.magnitudes.le_trans (cell_state_perturbation_at a b h L E s t hc)
(Fraction.add_le_add_left hv
(Fraction.add (pointDistance s.1 t.1)
(Fraction.mul h.abs (pointDistance s.2 t.2))))

theorem cell_state_perturbation_closed (a b : Point → Point)
(h L E : Fraction) (s t : Point × Point)
(hL : 0 ≤ L.num) (hc : comparisonContract a b L E) :
Fraction.le (stateDistance (cell a h s) (cell b h t))
(Fraction.add
(Fraction.add (pointDistance s.1 t.1)
(Fraction.mul h.abs (pointDistance s.2 t.2)))
(Fraction.add (pointDistance s.2 t.2)
(Fraction.mul h.abs
(Fraction.add (Fraction.mul L
(Fraction.add (pointDistance s.1 t.1)
(Fraction.mul h.abs (pointDistance s.2 t.2)))) E)))) :=
cell_state_perturbation_closed_at a b h L E s t hL
(hc (cell a h s).1 (cell b h t).1)

/-- Finite Lipschitz amplification retains the oracle discrepancy explicitly. -/
theorem cell_amplification_at (a b : Point → Point) (h L E : Fraction)
(s t : Point × Point) (hL : 0 ≤ L.num)
(hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1))
(Fraction.add (Fraction.mul L
(pointDistance (cell a h s).1 (cell b h t).1)) E)) :
Fraction.le (stateDistance (cell a h s) (cell b h t))
(Fraction.add (Fraction.mul (amplification L h) (stateDistance s t))
(Fraction.mul h.abs E)) :=
Fraction.magnitudes.le_trans (cell_state_perturbation_closed_at a b h L E s t hL hc)
(component_amplification _ _ _ (pointNorm_nonnegative _)
(pointNorm_nonnegative _) (Fraction.abs_num_nonnegative _) hL)

theorem cell_amplification (a b : Point → Point) (h L E : Fraction)
(s t : Point × Point) (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) :
Fraction.le (stateDistance (cell a h s) (cell b h t))
(Fraction.add (Fraction.mul (amplification L h) (stateDistance s t))
(Fraction.mul h.abs E)) :=
cell_amplification_at a b h L E s t hL
(hc (cell a h s).1 (cell b h t).1)

/-- The first and second half cells use the same sampled point map. -/
def twoHalf (a : Point → Point) (h : Fraction) (s : Point × Point) :
Point × Point := cell a h (cell a h s)

def oneFull (a : Point → Point) (h : Fraction) (s : Point × Point) :
Point × Point := cell a (Fraction.add h h) s

/-- Exact position defect of one full versus two half cells. It is the
first half-cell sample times `h^2`, with no curve or adjacent-error premise. -/
theorem twoHalf_position_identity (a : Point → Point) (h : Fraction)
(s : Point × Point) :
pointEquiv (twoHalf a h s).1
(pointAdd (oneFull a h s).1
(pointScale (Fraction.mul h h) (a (cell a h s).1))) := by
constructor <=>
simp only [twoHalf, oneFull, cell, pointEquiv, pointAdd, pointScale,
Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add] <=>
ac_nf <=> omega

private theorem twoHalf_position_difference (a : Point → Point) (h : Fraction)
(s : Point × Point) :
pointEquiv (pointSub (twoHalf a h s).1 (oneFull a h s).1)
(pointScale (Fraction.mul h h) (a (cell a h s).1)) := by
constructor <=>
simp only [twoHalf, oneFull, cell, pointEquiv, pointSub, pointNeg,
pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>

```

```

ac_nf <=> omega

/-- Local position error from a bound on the first sampled value. -/
theorem twoHalf_position_error (a : Point → Point) (h B : Fraction)
  (s : Point × Point)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B) :
  Fraction.le (pointDistance (twoHalf a h s).1 (oneFull a h s).1)
    (Fraction.mul (Fraction.mul h h).abs B) := by
  have he := pointNorm_equiv (twoHalf_position_difference a h s)
  have hs := pointNorm_scale (Fraction.mul h h) (a (cell a h s).1)
  have hm := Fraction.mul_le_mul_nonnegative_left hB (Fraction.mul h h).abs
    (Fraction.abs_num_nonnegative _)
  exact Fraction.magnitudes.le_trans
    (Fraction.le_of_equiv (Fraction.equiv_trans he hs)) hm

private theorem first_to_full_difference (a : Point → Point) (h : Fraction)
  (s : Point × Point) :
  pointEquiv (pointSub (cell a h s).1 (oneFull a h s).1)
    (pointScale h (pointNeg s.2)) := by
  constructor <=>
  simp only [oneFull, cell, pointEquiv, pointSub, pointNeg, pointAdd,
    pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- The first half arrival and full arrival are separated by exactly one
drift of the initial velocity, measured in coordinate L1 magnitude. -/
theorem first_to_full_distance (a : Point → Point) (h : Fraction)
  (s : Point × Point) :
  Fraction.equiv (pointDistance (cell a h s).1 (oneFull a h s).1)
    (Fraction.mul h.abs (pointNorm s.2)) := by
  exact Fraction.equiv_trans
    (pointNorm_equiv (first_to_full_difference a h s))
    (Fraction.equiv_trans (pointNorm_scale h (pointNeg s.2))
      (Fraction.mul_equiv_left h.abs (pointNorm_neg s.2)))

/-- The exact velocity defect is the sum of two differences from the
full-cell sample. All three sample locations come from actual cells. -/
private theorem twoHalf_velocity_difference (a : Point → Point) (h : Fraction)
  (s : Point × Point) :
  pointEquiv (pointSub (twoHalf a h s).2 (oneFull a h s).2)
    (pointScale h
      (pointAdd
        (pointSub (a (cell a h s).1) (a (oneFull a h s).1))
        (pointSub (a (twoHalf a h s).1) (a (oneFull a h s).1)))) := by
  constructor <=>
  simp only [twoHalf, oneFull, cell, pointEquiv, pointSub, pointNeg,
    pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- Two bounds at the actual sample locations suffice; no Lipschitz
coefficient is needed for this local identity and magnitude estimate. -/
theorem twoHalf_velocity_sample_error (a : Point → Point) (h E1 E2 : Fraction)
  (s : Point × Point)
  (h1 : Fraction.le (pointDistance (a (cell a h s).1) (a (oneFull a h s).1)) E1)
  (h2 : Fraction.le (pointDistance (a (twoHalf a h s).1) (a (oneFull a h s).1)) E2) :
  Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2)
    (Fraction.mul h.abs (Fraction.add E1 E2)) := by
  let r1 := pointSub (a (cell a h s).1) (a (oneFull a h s).1)
  let r2 := pointSub (a (twoHalf a h s).1) (a (oneFull a h s).1)
  have he := pointNorm_equiv (twoHalf_velocity_difference a h s)
  have hs := pointNorm_scale h (pointAdd r1 r2)
  have ht := pointNorm_add_le r1 r2
  have hsum := Fraction.add_le_add h1 h2
  have hbound := Fraction.magnitudes.le_trans ht hsum
  have hm := Fraction.mul_le_mul_nonnegative_left hbound h.abs
    (Fraction.abs_num_nonnegative h)
  exact Fraction.magnitudes.le_trans
    (Fraction.le_of_equiv (Fraction.equiv_trans he hs)) hm

```

```

/-- Local velocity error for a sampled map with Lipschitz coefficient `L`
and additive discrepancy `E`. Rounded samples may have `E > 0`. -/
theorem twoHalf_velocity_error (a : Point → Point) (h L E : Fraction)
  (s : Point × Point) (hc : comparisonContract a a L E) :
  Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2)
    (Fraction.mul h.abs
      (Fraction.add
        (Fraction.add (Fraction.mul L
          (pointDistance (cell a h s).1 (oneFull a h s).1)) E)
          (Fraction.add (Fraction.mul L
            (pointDistance (twoHalf a h s).1 (oneFull a h s).1)) E))) :=
  twoHalf_velocity_sample_error a h _ _ s
  (hc (cell a h s).1 (oneFull a h s).1)
  (hc (twoHalf a h s).1 (oneFull a h s).1)

/-- A fully explicit local velocity bound follows once the first sampled
value is bounded by `B` and the comparison coefficient is nonnegative. -/
theorem twoHalf_velocity_error_closed_at (a : Point → Point)
  (h L E B : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B)
  (h1 : Fraction.le
    (pointDistance (a (cell a h s).1) (a (oneFull a h s).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (cell a h s).1 (oneFull a h s).1)) E))
  (h2 : Fraction.le
    (pointDistance (a (twoHalf a h s).1) (a (oneFull a h s).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (twoHalf a h s).1 (oneFull a h s).1)) E)) :
  Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2)
    (Fraction.mul h.abs
      (Fraction.add
        (Fraction.add (Fraction.mul L
          (Fraction.mul h.abs (pointNorm s.2))) E)
          (Fraction.add (Fraction.mul L
            (Fraction.mul (Fraction.mul h h).abs B)) E))) := by
  have hd1 : Fraction.le
    (pointDistance (cell a h s).1 (oneFull a h s).1)
    (Fraction.mul h.abs (pointNorm s.2)) :=
    Fraction.le_of_equiv (first_to_full_distance a h s)
  have hd2 := twoHalf_position_error a h B s hB
  have hm1 := Fraction.mul_le_mul_nonnegative_left hd1 L hL
  have hm2 := Fraction.mul_le_mul_nonnegative_left hd2 L hL
  have he1 := Fraction.add_le_add_right hm1 E
  have he2 := Fraction.add_le_add_right hm2 E
  have hsum := Fraction.add_le_add he1 he2
  have htimes := Fraction.mul_le_mul_nonnegative_left hsum h.abs
    (Fraction.abs_num_nonnegative h)
  exact Fraction.magnitudes.le_trans
    (twoHalf_velocity_sample_error a h _ _ s h1 h2)
    htimes

theorem twoHalf_velocity_error_closed (a : Point → Point)
  (h L E B : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B)
  (hc : comparisonContract a a L E) :
  Fraction.le (pointDistance (twoHalf a h s).2 (oneFull a h s).2)
    (Fraction.mul h.abs
      (Fraction.add
        (Fraction.add (Fraction.mul L
          (Fraction.mul h.abs (pointNorm s.2))) E)
          (Fraction.add (Fraction.mul L
            (Fraction.mul (Fraction.mul h h).abs B)) E))) :=
  twoHalf_velocity_error_closed_at a h L E B s hL hB
  (hc (cell a h s).1 (oneFull a h s).1)
  (hc (twoHalf a h s).1 (oneFull a h s).1)

/-- Combined local state error; the two sampled arrival displacements remain

```



```

explicit so subsequent confinement estimates can be applied separately. -/
theorem twoHalf_state_error (a : Point → Point) (h L E B : Fraction)
  (s : Point × Point)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B)
  (hc : comparisonContract a a L E) :
  Fraction.le (stateDistance (twoHalf a h s) (oneFull a h s))
    (Fraction.add
      (Fraction.mul (Fraction.mul h h).abs B)
      (Fraction.mul h.abs
        (Fraction.add
          (Fraction.add (Fraction.mul L
            (pointDistance (cell a h s).1 (oneFull a h s).1)) E)
          (Fraction.add (Fraction.mul L
            (pointDistance (twoHalf a h s).1 (oneFull a h s).1)) E)))) :=
  Fraction.add_le_add (twoHalf_position_error a h B s hB)
    (twoHalf_velocity_error a h L E s hc)

theorem twoHalf_state_error_closed (a : Point → Point)
  (h L E B : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num)
  (hB : Fraction.le (pointNorm (a (cell a h s).1)) B)
  (hc : comparisonContract a a L E) :
  Fraction.le (stateDistance (twoHalf a h s) (oneFull a h s))
    (Fraction.add
      (Fraction.mul (Fraction.mul h h).abs B)
      (Fraction.mul h.abs
        (Fraction.add
          (Fraction.add (Fraction.mul L
            (Fraction.mul h.abs (pointNorm s.2))) E)
          (Fraction.add (Fraction.mul L
            (Fraction.mul (Fraction.mul h h).abs B)) E)))) :=
  Fraction.add_le_add (twoHalf_position_error a h B s hB)
    (twoHalf_velocity_error_closed a h L E B s hL hB hc)

/-- Any represented zero duration leaves state values unchanged, for every map. -/
theorem zero_duration_cell (a : Point → Point) (h : Fraction)
  (hh : h.num = 0) (s : Point × Point) : stateEquiv (cell a h s) s := by
  constructor <=> constructor <=>
    simp only [cell, stateEquiv, pointEquiv, pointAdd, pointScale, Fraction.equiv,
      Fraction.add, Fraction.mul, hh, Int.zero_mul, Int.mul_zero, Int.add_zero] <=> ac_nf

private def controlZero : Fraction := Fraction.ofInt 0
private def controlHalf : Fraction := {1, 2, by decide}
private def controlEighth : Fraction := {1, 8, by decide}
private def controlState : Point × Point :=
  ((controlZero, controlZero), (controlZero, controlZero))
private def controlA : Point → Point := fun _ => (controlZero, {-1, 1, by decide})
private def controlB : Point → Point := fun _ => (controlZero, {-2, 1, by decide})
private def controllinear : Point → Point := pointNeg
private def controlStart : Point × Point :=
  ((Fraction.ofInt 1, controlZero), (controlZero, Fraction.ofInt 1))

/-- Exact rational arithmetic pin for the pure map `p ↦ -p` at `h=1/8`. -/
theorem linear_sample_control :
  stateEquiv (cell controllinear controlEighth controlStart)
    ((Fraction.ofInt 1, controlEighth),
      ({-1, 8, by decide}, {63, 64, by decide})) := by
  unfold stateEquiv
  decide

/-- Constant downward sample, zero initial state, and a half cell. -/
theorem constant_sample_control :
  stateEquiv (cell controlA controlHalf controlState)
    ((controlZero, controlZero),
      (controlZero, {-1, 2, by decide})) := by
  unfold stateEquiv
  decide

/-- Two unequal constant maps from the same zero state differ after a half
cell. This exact rational control detects omission of the sampling term. -/

```

```

theorem unequal_sample_control :
  Fraction.equiv
    (stateDistance (cell controlA controlHalf controlState)
      (cell controlB controlHalf controlState)) controlHalf := by
  decide

theorem unequal_sample_control_nonzero :
  ¬ Fraction.le
    (stateDistance (cell controlA controlHalf controlState)
      (cell controlB controlHalf controlState)) controlZero := by
  unfold Fraction.le
  decide

/-- Zero duration covers arbitrary maps and states. -/
theorem zero_duration_control (a : Point → Point) (s : Point × Point) :
  stateEquiv (cell a controlZero s) s :=
  zero_duration_cell a controlZero rfl s

```

end NewtonLimitDynamics.Polygon.FiniteEstimates

BarrowLib/Polygon/FiniteAccumulation.lean, 34 theorems, 619 lines

```

import BarrowLib.Polygon.FiniteFactorProducts
import BarrowLib.Polygon.FiniteEstimates
import BarrowLib.Common.FiniteGrowth

/-!
Finite accumulation for actual coarse cells and pairs of half cells. The point
map is arbitrary; applications to Newtonian central fields impose centrality
separately. All estimates are coordinate L1 estimates on rational states.
-/

namespace NewtonLimitDynamics.Polygon.FiniteAccumulation

open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open FiniteEstimates

/-- Iterates of the actual full-cell and two-half-cell maps. -/
def coarseAt (a : Point → Point) (h : Fraction) (s : Point × Point) : Nat → Point × Point
| 0 => s
| n + 1 => oneFull a h (coarseAt a h s n)

def fineAt (a : Point → Point) (h : Fraction) (s : Point × Point) : Nat → Point × Point
| 0 => s
| n + 1 => twoHalf a h (fineAt a h s n)

/-- One application of the two-half-cell perturbation bound. -/
def twoStepBudget (h L E D : Fraction) : Fraction :=
  Fraction.add
    (Fraction.mul (amplification L h)
      (Fraction.add (Fraction.mul (amplification L h) D)
        (Fraction.mul h.abs E)))
    (Fraction.mul h.abs E)

/-- The actual local mismatch, using the bounded first-half sample and the
coarse starting velocity. No adjacent-error hypothesis is hidden here. -/
def localBudget (h L E B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add
    (Fraction.mul (Fraction.mul h h).abs B)
    (Fraction.mul h.abs
      (Fraction.add
        (Fraction.add (Fraction.mul L
          (Fraction.mul h.abs (pointNorm s.2))) E)
        (Fraction.add (Fraction.mul L
          (Fraction.mul (Fraction.mul h h).abs B)) E)))

/-- Replace the actual coarse velocity in the local source by a uniform
bound `V`. This is a separate confinement premise for the iterated result. -/
def uniformLocalBudget (h L E B V : Fraction) : Fraction :=

```

```

Fraction.add
  (Fraction.mul (Fraction.mul h h).abs B)
  (Fraction.mul h.abs
    (Fraction.add
      (Fraction.add (Fraction.mul L (Fraction.mul h.abs V)) E)
      (Fraction.add (Fraction.mul L
        (Fraction.mul (Fraction.mul h h).abs B)) E)))

theorem localBudget_le_uniform (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hV : Fraction.le (pointNorm s.2) V) :
  Fraction.le (localBudget h L E B s) (uniformLocalBudget h L E B V) := by
  have h₁ := Fraction.mul_le_mul_nonnegative_left hV h.abs
  (Fraction.abs_num_nonnegative h)
  have h₂ := Fraction.mul_le_mul_nonnegative_left h₁ L hL
  have h₃ := Fraction.add_le_add_right h₂ E
  have h₄ := Fraction.add_le_add_right h₃
    (Fraction.add (Fraction.mul L
      (Fraction.mul (Fraction.mul h h).abs B)) E)
  have h₅ := Fraction.mul_le_mul_nonnegative_left h₄ h.abs
  (Fraction.abs_num_nonnegative h)
  exact Fraction.add_le_add_left h₅
    (Fraction.mul (Fraction.mul h h).abs B)

/-- The constant source per block includes both half-cell sampling errors. -/
def uniformBlockSource (h L E B V : Fraction) : Fraction :=
  Fraction.add
    (Fraction.add
      (Fraction.mul (amplification L h) (Fraction.mul h.abs E))
      (Fraction.mul h.abs E))
    (uniformLocalBudget h L E B V)

/-- The same recurrence with a constant source, once every actual coarse
state has velocity bounded by `V`. -/
def constantErrorBudget (h L E B V : Fraction) : Nat → Fraction
| 0 => Fraction.ofInt 0
| n + 1 => Fraction.add
  (Fraction.mul (amplification L h)
    (Fraction.mul (amplification L h)
      (constantErrorBudget h L E B V n)))
  (uniformBlockSource h L E B V)

def blockFactor (h L : Fraction) : Fraction :=
  Fraction.mul (amplification L h) (amplification L h)

def factorPower (r : Fraction) : Nat → Fraction
| 0 => Fraction.ofInt 1
| n + 1 => Fraction.mul r (factorPower r n)

def count (n : Nat) : Fraction := Fraction.ofInt (n : Int)

theorem amplification_nonnegative (h L : Fraction) (hL : 0 ≤ L.num) :
  0 ≤ (amplification L h).num :=
  Fraction.nonnegative_mul _ _
  (Fraction.nonnegative_add _ _ (by decide) (Fraction.abs_num_nonnegative h))
  (Fraction.nonnegative_add _ _ (by decide)
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
      hL))

private theorem one_le_one_add (a : Fraction) (ha : 0 ≤ a.num) :
  Fraction.le (Fraction.ofInt 1) (Fraction.add (Fraction.ofInt 1) a) := by
  unfold Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hp := Int.le_of_lt a.den_pos
  omega

theorem one_le_amplification (h L : Fraction) (hL : 0 ≤ L.num) :
  Fraction.le (Fraction.ofInt 1) (amplification L h) := by
  let u := Fraction.add (Fraction.ofInt 1) h.abs
  let v := Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs L)
  have hu := one_le_one_add h.abs (Fraction.abs_num_nonnegative h)

```

```

have hv := one_le_one_add (Fraction.mul h.abs L)
  (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) hL)
have h1 := Fraction.mul_le_mul_nonnegative hu (Fraction.ofInt 1) (by decide)
have h2 := Fraction.mul_le_mul_nonnegative_left hv u (by
  unfold u Fraction.add Fraction.ofInt
  dsimp
  have hp := h.abs.den_pos
  have hn := Fraction.abs_num_nonnegative h
  omega)
have hc := Fraction.magnitudes.le_trans h1 h2
apply Fraction.le_equiv_left (by
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  decide) hc

theorem blockFactor_nonnegative (h L : Fraction) (hL : 0 ≤ L.num) :
  0 ≤ (blockFactor h L).num :=
  Fraction.nonnegative_mul _ _
    (amplification_nonnegative h L hL) (amplification_nonnegative h L hL)

theorem one_le_blockFactor (h L : Fraction) (hL : 0 ≤ L.num) :
  Fraction.le (Fraction.ofInt 1) (blockFactor h L) := by
  have hk := one_le_amplification h L hL
  have h1 := Fraction.mul_le_mul_nonnegative hk (Fraction.ofInt 1) (by decide)
  have h2 := Fraction.mul_le_mul_nonnegative_left hk (amplification L h)
    (amplification_nonnegative h L hL)
  have hc := Fraction.magnitudes.le_trans h1 h2
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    decide) hc

theorem factorPower_nonnegative (r : Fraction) (hr : 0 ≤ r.num) :
  (n : Nat) → 0 ≤ (factorPower r n).num
  | 0 => by simp [factorPower, Fraction.ofInt]
  | n + 1 => Fraction.nonnegative_mul _ _ hr (factorPower_nonnegative r hr n)

theorem one_le_factorPower (r : Fraction) (hr : 0 ≤ r.num)
  (h1 : Fraction.le (Fraction.ofInt 1) r) :
  (n : Nat) → Fraction.le (Fraction.ofInt 1) (factorPower r n)
  | 0 => Fraction.magnitudes.le_refl _
  | n + 1 => by
    have hi := one_le_factorPower r hr h1 n
    have hm := Fraction.mul_le_mul_nonnegative h1 (factorPower r n)
      (factorPower_nonnegative r hr n)
    have he : Fraction.equiv (factorPower r n)
      (Fraction.mul (Fraction.ofInt 1) (factorPower r n)) := by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
      simp
    exact Fraction.magnitudes.le_trans hi (Fraction.le_equiv_left he hm)

theorem uniformBlockSource_nonnegative (h L E B V : Fraction)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) :
  0 ≤ (uniformBlockSource h L E B V).num := by
  unfold uniformBlockSource uniformLocalBudget
  apply Fraction.nonnegative_add
  · apply Fraction.nonnegative_add
    · exact Fraction.nonnegative_mul _ _ (amplification_nonnegative h L hL)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) hE)
    · exact Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) hE
  · apply Fraction.nonnegative_add
    · exact Fraction.nonnegative_mul _ _
      (Fraction.abs_num_nonnegative (Fraction.mul h h)) hB
    · apply Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
      apply Fraction.nonnegative_add
      · apply Fraction.nonnegative_add
        · exact Fraction.nonnegative_mul _ _ hL
          (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) hV)
        · exact hE
    · apply Fraction.nonnegative_add
      · exact Fraction.nonnegative_mul _ _ hL

```

```

        (Fraction.nonnegative_mul _ _
         (Fraction.abs_num_nonnegative (Fraction.mul h h)) hB)
    · exact hE

/-- Propagation through two actual half cells with distinct sample maps.
Each application contributes its own `hE` term. -/
theorem cross_twoHalf_perturbation (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point) (hL : 0 ≤ L.num)
  (hc : comparisonContract a b L E) :
  Fraction.le (stateDistance (twoHalf a h s) (twoHalf b h t))
    (twoStepBudget h L E (stateDistance s t)) := by
  have h₁ := cell_amplification a b h L E (cell a h s) (cell b h t) hL hc
  have h₂ := cell_amplification a b h L E s t hL hc
  have hm := Fraction.mul_le_mul_nonnegative_left h₂ (amplification L h)
    (amplification_nonnegative h L hL)
  have hs := Fraction.add_le_add_right hm (Fraction.mul h.abs E)
  exact Fraction.magnitudes.le_trans h₁ hs

/-- The same-map public theorem is a cross-map instance. -/
theorem twoHalf_perturbation (a : Point → Point) (h L E : Fraction)
  (s t : Point × Point) (hL : 0 ≤ L.num)
  (hc : comparisonContract a a L E) :
  Fraction.le (stateDistance (twoHalf a h s) (twoHalf a h t))
    (twoStepBudget h L E (stateDistance s t)) :=
  cross_twoHalf_perturbation a a h L E s t hL hc

/-- One actual cross-map block. The local defect uses the coarse map `b`,
while propagation of the distinct fine map `a` uses the cross contract. -/
theorem cross_block_error (a b : Point → Point) (h L E B : Fraction)
  (s t : Point × Point) (hL : 0 ≤ L.num)
  (hcross : comparisonContract a b L E)
  (hlocal : comparisonContract b b L E)
  (hB : Fraction.le (pointNorm (b (cell b h t).1)) B) :
  Fraction.le (stateDistance (twoHalf a h s) (oneFull b h t))
    (Fraction.add (twoStepBudget h L E (stateDistance s t))
      (localBudget h L E B t)) := by
  have ht := stateDistance_triangle (twoHalf a h s) (twoHalf b h t)
    (oneFull b h t)
  have hp := cross_twoHalf_perturbation a b h L E s t hL hcross
  have hl := twoHalf_state_error_closed b h L E B t hL hB hlocal
  exact Fraction.magnitudes.le_trans ht (Fraction.add_le_add hp hl)

theorem block_error (a : Point → Point) (h L E B : Fraction)
  (s t : Point × Point) (hL : 0 ≤ L.num)
  (hc : comparisonContract a a L E)
  (hB : Fraction.le (pointNorm (a (cell a h t).1)) B) :
  Fraction.le (stateDistance (twoHalf a h s) (oneFull a h t))
    (Fraction.add (twoStepBudget h L E (stateDistance s t))
      (localBudget h L E B t)) :=
  cross_block_error a a h L E B s t hL hc hc hB

/-- The budget is a finite rational recurrence whose local source is
calculated at each actual coarse state. -/
def errorBudget (a : Point → Point) (h L E B : Fraction)
  (s : Point × Point) : Nat → Fraction
| 0 => Fraction.ofInt 0
| n + 1 => Fraction.add
  (twoStepBudget h L E (errorBudget a h L E B s n))
  (localBudget h L E B (coarseAt a h s n))

/-- Finite accumulation of distinct fine/coarse sample maps. The local
source is calculated at actual coarse states of `b`; the cross contract
propagates the fine-versus-coarse separation. -/
theorem cross_actual_error_le_budget (a b : Point → Point) (h L E B : Fraction)
  (s : Point × Point) (hL : 0 ≤ L.num)
  (hcross : comparisonContract a b L E)
  (hlocal : comparisonContract b b L E) :
  (n : Nat) →
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B) →

```

```

    Fraction.le (stateDistance (fineAt a h s n) (coarseAt b h s n))
      (errorBudget b h L E B s n)
| 0, _ => Fraction.le_of_equiv (stateDistance_self_zero s)
| n + 1, hB => by
  have hs := cross_block_error a b h L E B (fineAt a h s n)
    (coarseAt b h s n) hL hcross hlocal (hB n (Nat.lt_succ_self n))
  have hi := cross_actual_error_le_budget a b h L E B s hL hcross hlocal n
    (fun k hk => hB k (Nat.lt_trans hk (Nat.lt_succ_self n)))
  have hm := Fraction.mul_le_mul_nonnegative_left hi (amplification L h)
    (amplification_nonnegative h L hL)
  have ha := Fraction.add_le_add_right hm (Fraction.mul h.abs E)
  have hm₂ := Fraction.mul_le_mul_nonnegative_left ha (amplification L h)
    (amplification_nonnegative h L hL)
  have hb := Fraction.add_le_add_right hm₂ (Fraction.mul h.abs E)
  have hf := Fraction.add_le_add_right hb
    (localBudget h L E B (coarseAt b h s n))
  exact Fraction.magnitudes.le_trans hs hf

/-- Same-map compatibility instance of the cross-map recurrence. -/
theorem actual_error_le_budget (a : Point → Point) (h L E B : Fraction)
  (s : Point × Point) (hL : 0 ≤ L.num)
  (hc : comparisonContract a a L E) :
  (n : Nat) →
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (a (cell a h (coarseAt a h s k)).1)) B) →
    Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n))
      (errorBudget a h L E B s n) :=
  cross_actual_error_le_budget a a h L E B s hL hc hc

theorem errorBudget_le_constant (a : Point → Point)
  (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) :
  (n : Nat) →
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V) →
    Fraction.le (errorBudget a h L E B s n)
      (constantErrorBudget h L E B V n)
| 0, _ => Fraction.magnitudes.le_refl _
| n + 1, hV => by
  have hi := errorBudget_le_constant a h L E B V s hL n
    (fun k hk => hV k (Nat.lt_trans hk (Nat.lt_succ_self n)))
  have hK := amplification_nonnegative h L hL
  have h₁ := Fraction.mul_le_mul_nonnegative_left hi (amplification L h) hK
  have h₂ := Fraction.add_le_add_right h₁ (Fraction.mul h.abs E)
  have h₃ := Fraction.mul_le_mul_nonnegative_left h₂ (amplification L h) hK
  have h₄ := Fraction.add_le_add_right h₃ (Fraction.mul h.abs E)
  have h₅ := localBudget_le_uniform h L E B V (coarseAt a h s n) hL
    (hV n (Nat.lt_succ_self n))
  have hs := Fraction.add_le_add h₄ h₅
  apply Fraction.le_equiv_right hs
  simp only [errorBudget, twoStepBudget, constantErrorBudget,
    uniformBlockSource, Fraction.equiv, Fraction.add, Fraction.mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

/-- Finite-prefix coarse velocity confinement converts the cross-map error
into the same constant rational recurrence. -/
theorem cross_actual_error_le_constant_budget (a b : Point → Point)
  (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num)
  (hcross : comparisonContract a b L E)
  (hlocal : comparisonContract b b L E)
  (n : Nat)
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt b h s n))
    (constantErrorBudget h L E B V n) :=
  Fraction.magnitudes.le_trans
    (cross_actual_error_le_budget a b h L E B s hL hcross hlocal n hB)
    (errorBudget_le_constant b h L E B V s hL n hV)

```

```

theorem actual_error_le_constant_budget (a : Point → Point)
  (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num)
  (hc : comparisonContract a a L E)
  (n : Nat)
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (a (cell a h (coarseAt a h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n))
    (constantErrorBudget h L E B V n) :=
  cross_actual_error_le_constant_budget a a h L E B V s hL hc hc n hB hV

/-- A constant-source recurrence is bounded by its finite amplification
power. This is pure finite arithmetic and assumes no limit object. -/
theorem constant_budget_power_bound (h L E B V : Fraction)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) :
  (n : Nat) →
    Fraction.le (constantErrorBudget h L E B V n)
      (Fraction.mul (count n)
        (Fraction.mul (uniformBlockSource h L E B V)
          (factorPower (blockFactor h L) n)))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [constantErrorBudget, count, factorPower, Fraction.equiv,
    Fraction.ofInt, Fraction.mul]
  simp
| n + 1 => by
  let r := blockFactor h L
  let C := uniformBlockSource h L E B V
  have hi := constant_budget_power_bound h L E B V hL hE hB hV n
  have hr := blockFactor_nonnegative h L hL
  have hC := uniformBlockSource_nonnegative h L E B V hL hE hB hV
  have h1 := Fraction.mul_le_mul_nonnegative_left hi r hr
  have h1' : Fraction.le
    (Fraction.mul r (constantErrorBudget h L E B V n))
    (Fraction.mul (count n)
      (Fraction.mul C (factorPower r (n + 1)))) := by
    apply Fraction.le_equiv_right h1
    simp only [r, C, blockFactor, factorPower, Fraction.equiv, Fraction.mul]
    ac_nf
  have hp := one_le_factorPower r hr (one_le_blockFactor h L hL) (n + 1)
  have h2a := Fraction.mul_le_mul_nonnegative hp C hC
  have h2 : Fraction.le C (Fraction.mul C (factorPower r (n + 1))) :=
    Fraction.le_equiv_left (by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
      simp) (Fraction.le_equiv_right h2a (Fraction.mul_comm _ _))
  have hs := Fraction.add_le_add h1' h2
  have hleft : Fraction.equiv
    (constantErrorBudget h L E B V (n + 1))
    (Fraction.add (Fraction.mul r (constantErrorBudget h L E B V n)) C) := by
    simp only [constantErrorBudget, C, r, blockFactor,
      Fraction.equiv, Fraction.add, Fraction.mul]
    ac_nf
  apply Fraction.le_equiv_right (Fraction.le_equiv_left hleft hs)
  simp only [count, C, r, blockFactor,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one]
  simp only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

/-- Cross-map mesh-uniform form conditional on the finite power bound. -/
theorem cross_actual_error_le_two_count_source (a b : Point → Point)
  (h L E B V : Fraction) (s : Point × Point) (n : Nat)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  (hcross : comparisonContract a b L E)
  (hlocal : comparisonContract b b L E)
  (hB : ∀ k, k < n → Fraction.le

```

```

    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V)
  (hpower : Fraction.le (factorPower (blockFactor h L) n)
    (Fraction.ofInt 2)) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt b h s n))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (count n) (uniformBlockSource h L E B V))) := by
  have h₀ := cross_actual_error_le_constant_budget a b h L E B V s hL
    hcross hlocal n hB hV
  have h₁ := constant_budget_power_bound h L E B V hL hE hBnonneg hVnonneg n
  have h₂ := Fraction.mul_le_mul_nonnegative_left hpower
    (uniformBlockSource h L E B V)
    (uniformBlockSource_nonnegative h L E B V hL hE hBnonneg hVnonneg)
  have h₃ := Fraction.mul_le_mul_nonnegative_left h₂ (count n) (by
    unfold count Fraction.ofInt
    exact Int.ofNat_nonneg n)
  have hc' := Fraction.magnitudes.le_trans h₀
    (Fraction.magnitudes.le_trans h₁ h₃)
  apply Fraction.le_equiv_right hc'
  simp only [count, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem actual_error_le_two_count_source (a : Point → Point)
  (h L E B V : Fraction) (s : Point × Point) (n : Nat)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  (hc : comparisonContract a a L E)
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (a (cell a h (coarseAt a h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V)
  (hpower : Fraction.le (factorPower (blockFactor h L) n)
    (Fraction.ofInt 2)) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (count n) (uniformBlockSource h L E B V))) :=
  cross_actual_error_le_two_count_source a a h L E B V s n hL hE hBnonneg
    hVnonneg hc hc hB hV hpower

/-- The represented elapsed duration of `n` full cells, each of size `2h`. -/
def totalTime (h : Fraction) (n : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (2 * (n : Int))) h

def SmallWindow (h L : Fraction) (n : Nat) : Prop :=
  Fraction.le
    (Fraction.mul (totalTime h n) (Fraction.add (Fraction.ofInt 1) L))
    (1, 2, by decide)

/-- Compatibility of the retained finite-power name with the shared power. -/
theorem factorPower_fpower (r : Fraction) :
  (n : Nat) → Fraction.equiv (factorPower r n) (HarmonicAccumulation.fpower r n)
  | 0 => Fraction.equiv_refl _
  | n+1 => Fraction.mul_equiv (Fraction.equiv_refl r) (factorPower_fpower r n)

/-- The old unit-gauge window is an instance of the shared dimensionless
pair-factor estimate, with two fine cells in each coarse block. -/
theorem blockFactor_power_le_two (h L : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hs : SmallWindow h L n) :
  Fraction.le (factorPower (blockFactor h L) n) (Fraction.ofInt 2) := by
  have hsmall : Fraction.le
    (Fraction.mul (Fraction.ofInt ((2*n : Nat) : Int))
      (Fraction.add h (Fraction.mul h L))) (1,2,by decide) := by
  apply Fraction.le_equiv_left (b := Fraction.mul (totalTime h n)
    (Fraction.add (Fraction.ofInt 1) L)) _ hs
  simp only [totalTime, Fraction.equiv, Fraction.mul, Fraction.add, Fraction.ofInt,
    Int.natCast_mul, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  have hb := FiniteFactorProducts.repeated_pair_le_two h (Fraction.mul h L) (2*n)
  hh (Fraction.nonnegative_mul _ _ hh hL) hsmall
  have hcoeff : Fraction.equiv (amplification L h)
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) h))

```



```

    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h L))) :=
  Fraction.mul_equiv
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.abs_of_nonnegative h hh))
    (Fraction.add_equiv (Fraction.equiv_refl _)
      (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl L)))
  have he := Fraction.equiv_trans (factorPower_fpower (blockFactor h L) n)
    (Fraction.equiv_trans (FiniteFactorProducts.fpower_square (amplification L h) n)
      (FiniteFactorProducts.fpower_congr hcoeff (2*n)))
  exact Fraction.le_equiv_left he hb

/-- Uniform finite cross-map comparison of actual polygon endpoints on a
small window. `B` and `V` refer to the coarse map `b` on the finite prefix;
the cross contract covers counterfactual fine/coarse sample locations. -/
theorem cross_actual_uniform_error (a b : Point → Point) (h L E B V : Fraction)
  (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  (hs : SmallWindow h L n)
  (hcross : comparisonContract a b L E)
  (hlocal : comparisonContract b b L E)
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt b h s n))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (count n) (uniformBlockSource h L E B V))) :=
  cross_actual_error_le_two_count_source a b h L E B V s n hL hE
  hBnonneg hVnonneg hcross hlocal hB hV
  (blockFactor_power_le_two h L n hh hL hs)

/-- Same-map public theorem, retained as a cross-map instance. -/
theorem actual_uniform_error (a : Point → Point) (h L E B V : Fraction)
  (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hBnonneg : 0 ≤ B.num) (hVnonneg : 0 ≤ V.num)
  (hs : SmallWindow h L n)
  (hc : comparisonContract a a L E)
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (a (cell a h (coarseAt a h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt a h s k).2) V) :
  Fraction.le (stateDistance (fineAt a h s n) (coarseAt a h s n))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (count n) (uniformBlockSource h L E B V))) :=
  cross_actual_uniform_error a a h L E B V s n hh hL hE hBnonneg hVnonneg
  hs hc hc hB hV

private def sampleZero : Fraction := Fraction.ofInt 0
private def sampleHalf : Fraction := ⟨1, 2, by decide⟩
private def sampleQuarter : Fraction := ⟨1, 4, by decide⟩
private def sampleState : Point × Point :=
  ((sampleZero, sampleZero), (sampleZero, sampleZero))
private def sampleDown : Point → Point :=
  fun _ => (sampleZero, Fraction.ofInt (-1))
private def sampleDownTwo : Point → Point :=
  fun _ => (sampleZero, Fraction.ofInt (-2))
private def sampleFiveQuarters : Fraction := ⟨5, 4, by decide⟩

/-- Two unequal constant maps satisfy a cross contract with `L=0, E=1`;
the coarse map also satisfies its local contract with the same upper error. -/
theorem cross_constant_contract :
  comparisonContract sampleDown sampleDownTwo sampleZero (Fraction.ofInt 1) := by
  intro p q
  have hz : Fraction.equiv (Fraction.mul sampleZero (pointDistance p q)) sampleZero := by
    simp only [sampleZero, Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
  have hr : Fraction.equiv
    (Fraction.add (Fraction.mul sampleZero (pointDistance p q)) (Fraction.ofInt 1))
    (Fraction.ofInt 1) :=
    Fraction.equiv_trans (Fraction.add_equiv hz (Fraction.equiv_refl _)) (by decide)
  have hl : Fraction.le (pointDistance (sampleDown p) (sampleDownTwo q))

```

```

    (Fraction.ofInt 1) := by
    unfold sampleDown sampleDownTwo pointDistance pointNorm pointSub pointNeg
    pointAdd Fraction.le Fraction.add Fraction.abs Fraction.ofInt sampleZero
    decide
    exact Fraction.le_equiv_right h1 (Fraction.equiv_symm hr)

theorem coarse_constant_contract :
  comparisonContract sampleDownTwo sampleDownTwo sampleZero (Fraction.ofInt 1) := by
  intro p q
  have hz : Fraction.equiv (Fraction.mul sampleZero (pointDistance p q)) sampleZero := by
    simp only [sampleZero, Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
  have hr : Fraction.equiv
    (Fraction.add (Fraction.mul sampleZero (pointDistance p q)) (Fraction.ofInt 1))
    (Fraction.ofInt 1) :=
    Fraction.equiv_trans (Fraction.add_equiv hz (Fraction.equiv_refl _)) (by decide)
  have h1 : Fraction.le (pointDistance (sampleDownTwo p) (sampleDownTwo q))
    (Fraction.ofInt 1) := by
    unfold sampleDownTwo pointDistance pointNorm pointSub pointNeg
    pointAdd Fraction.le Fraction.add Fraction.abs Fraction.ofInt sampleZero
    decide
    exact Fraction.le_equiv_right h1 (Fraction.equiv_symm hr)

/-- Empty schedules agree for every map and step size. -/
theorem zero_block_control (a : Point → Point) (h : Fraction)
  (s : Point × Point) :
  Fraction.equiv (stateDistance (fineAt a h s 0) (coarseAt a h s 0))
  sampleZero := stateDistance_self_zero s

/-- With zero cell duration, the first full/two-half block agrees exactly. -/
theorem zero_duration_block_control :
  Fraction.equiv
    (stateDistance (fineAt sampleDown sampleZero sampleState 1)
    (coarseAt sampleDown sampleZero sampleState 1)) sampleZero := by
  decide

/-- A constant downward map has `L=E=0` but its first refined block still
has a quarter-unit position defect at `h=1/2`. This catches accidental
omission of the `h^2B` local term. -/
theorem constant_block_control :
  Fraction.equiv
    (stateDistance (fineAt sampleDown sampleHalf sampleState 1)
    (coarseAt sampleDown sampleHalf sampleState 1)) sampleQuarter := by
  decide

theorem constant_block_budget_control :
  Fraction.equiv
    (errorBudget sampleDown sampleHalf sampleZero sampleZero
    (Fraction.ofInt 1) sampleState 1) sampleQuarter := by
  decide

/-- Fine map `(0,-1)` versus coarse map `(0,-2)`, both from zero state:
the first block has one unit of velocity discrepancy plus a quarter-unit
position defect. A cross-map estimate cannot erase its `E` source. -/
theorem cross_constant_block_control :
  Fraction.equiv
    (stateDistance (fineAt sampleDown sampleHalf sampleState 1)
    (coarseAt sampleDownTwo sampleHalf sampleState 1))
    sampleFiveQuarters := by
  decide

theorem cross_constant_block_nonzero :
  ¬ Fraction.le
    (stateDistance (fineAt sampleDown sampleHalf sampleState 1)
    (coarseAt sampleDownTwo sampleHalf sampleState 1)) sampleZero := by
  unfold Fraction.le
  decide

/-- If the cross-map `E` source were silently erased, the derived local
coarse defect alone would give `1/2`, below the actual `5/4` error. -/

```

```

theorem cross_constant_requires_sample_error :
  ¬ Fraction.le
    (stateDistance (fineAt sampleDown sampleHalf sampleState 1)
      (coarseAt sampleDownTwo sampleHalf sampleState 1))
    (errorBudget sampleDownTwo sampleHalf sampleZero sampleZero
      (Fraction.ofInt 2) sampleState 1) := by
  unfold Fraction.le
  decide

end NewtonLimitDynamics.Polygon.FiniteAccumulation

BarrowLib/Polygon/BoundedIteration.lean, 12 theorems, 182 lines

import BarrowLib.Polygon.FiniteEstimates

/-!
Finite position and velocity bounds for iteration of an arbitrary triangular
point map with bounded sampled values. No regularity, trajectory, derivative
or ODE result is used. The sampled-value bound must be derived on the region
or supplied explicitly; the iterates themselves are recursively constructed.
-/

namespace NewtonLimitDynamics.Polygon.BoundedIteration

open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates

def run (a : Point → Point) (h : Fraction) (s : Point × Point) : Nat → Point × Point
| 0 => s
| n + 1 => cell a h (run a h s n)

theorem zero_duration_run (a : Point → Point) (h : Fraction) (hh : h.num = 0)
(s : Point × Point) : (n : Nat) → stateEquiv (run a h s n) s
| 0 => ⟨(Fraction.equiv_refl _, Fraction.equiv_refl _),
  (Fraction.equiv_refl _, Fraction.equiv_refl _)⟩
| n+1 => by
  have hstep := zero_duration_cell a h hh (run a h s n)
  have hi := zero_duration_run a h hh s n
  exact ⟨pointEquiv_trans hstep.1 hi.1, pointEquiv_trans hstep.2 hi.2⟩

def time (h : Fraction) (n : Nat) : Fraction := Fraction.mul (Fraction.ofInt (n : Int)) h

theorem run_commute (a : Point → Point) (h : Fraction) (s : Point × Point) (n : Nat) :
  run a h (cell a h s) n = cell a h (run a h s n) := by
  induction n with
  | zero => rfl
  | succ n ih => simp only [run, ih]

def BoundedSamples (a : Point → Point) (h : Fraction) (s : Point × Point)
(B : Fraction) (n : Nat) : Prop :=
  ∀ i : Nat, i < n → Fraction.le (pointNorm (a (run a h s (i+1)).1)) B

theorem time_nonnegative (h : Fraction) (hh : 0 ≤ h.num) (n : Nat) :
  0 ≤ (time h n).num := Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg _) hh

theorem time_monotone (h : Fraction) (hh : 0 ≤ h.num) (i n : Nat) (hin : i ≤ n) :
  Fraction.le (time h i) (time h n) := by
  have hc : Fraction.le (Fraction.ofInt (i : Int)) (Fraction.ofInt (n : Int)) := by
    simp only [Fraction.le, Fraction.ofInt, Int.mul_one] using
      (show (i : Int) ≤ (n : Int) by omega)
  exact Fraction.mul_le_mul_nonnegative hc h hh

theorem velocity_bound (a : Point → Point) (h : Fraction) (s : Point × Point)
(B : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hb : BoundedSamples a h s B n) :
  Fraction.le (pointNorm (run a h s n).2)
    (Fraction.add (pointNorm s.2) (Fraction.mul (time h n) B)) := by
  induction n with
  | zero =>
    apply Fraction.le_of_equiv
    simp only [run, time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.natCast_zero, Int.zero_mul, Int.mul_zero, Int.add_zero, Int.one_mul, Int.mul_one]

```

```

    ac_nf
| succ n ih =>
  have hb0 : BoundedSamples a h s B n := fun i hi => hb i (by omega)
  have hc := cell_velocity_growth a h B (run a h s n) (hb n (by omega))
  have hi := Fraction.add_le_add_right (ih hb0) (Fraction.mul h.abs B)
  have he : Fraction.equiv
    (Fraction.add (Fraction.add (pointNorm s.2) (Fraction.mul (time h n) B))
      (Fraction.mul h.abs B))
    (Fraction.add (pointNorm s.2) (Fraction.mul (time h (n+1)) B)) := by
  apply Fraction.equiv_trans (Fraction.add_equiv
    (Fraction.equiv_refl (Fraction.add (pointNorm s.2) (Fraction.mul (time h n) B)))
    (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl B)))
  simp only [time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.one_mul, Int.mul_one]
  ac_nf
  exact Fraction.le_equiv_right (Fraction.magnitudes.le_trans hc hi) he

def positionCap (h : Fraction) (s : Point × Point) (B : Fraction) (n : Nat) : Fraction :=
  Fraction.add (pointNorm s.1)
    (Fraction.mul (Fraction.mul (time h n) (pointNorm s.2))
      (Fraction.mul (Fraction.mul (time h n) (time h n)) B))

theorem position_cap_step (h P V B : Fraction) (n : Nat)
  (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) :
  Fraction.le
    (Fraction.add (Fraction.add P
      (Fraction.add (Fraction.mul (time h n) V)
        (Fraction.mul (Fraction.mul (time h n) (time h n)) B)))
      (Fraction.mul h (Fraction.add V (Fraction.mul (time h n) B))))
    (Fraction.add P
      (Fraction.add (Fraction.mul (time h (n+1)) V)
        (Fraction.mul (Fraction.mul (time h (n+1)) (time h (n+1)) B)))) := by
  let A := Fraction.add (Fraction.add P
    (Fraction.add (Fraction.mul (time h n) V)
      (Fraction.mul (Fraction.mul (time h n) (time h n)) B)))
    (Fraction.mul h (Fraction.add V (Fraction.mul (time h n) B)))
  let E := Fraction.mul (Fraction.mul (time h (n+1)) h) B
  have hE : 0 ≤ E.num := Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (time_nonnegative h hh) hh) hB
  apply Fraction.le_equiv_right (Fraction.le_add_nonnegative A E hE)
  simp only [A, E, time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.one_mul, Int.mul_one]
  ac_nf

theorem position_bound (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat)
  (hb : BoundedSamples a h s B n) :
  Fraction.le (pointNorm (run a h s n).1) (positionCap h s B n) := by
  induction n with
  | zero =>
    apply Fraction.le_of_equiv
    simp only [run, positionCap, time, Fraction.equiv, Fraction.add, Fraction.mul,
      Fraction.ofInt, Int.natCast_zero, Int.zero_mul, Int.mul_zero, Int.add_zero, Int.one_mul,
      Int.mul_one]
    ac_nf
  | succ n ih =>
    have hb0 : BoundedSamples a h s B n := fun i hi => hb i (by omega)
    have hc := cell_position_growth a h (run a h s n)
    have hv := velocity_bound a h s B hh n hb0
    have hv' := Fraction.mul_le_mul_nonnegative_left hv h.abs
      (Fraction.abs_num_nonnegative h)
    have hi := Fraction.add_le_add (ih hb0) hv'
    have he := Fraction.add_equiv (Fraction.equiv_refl (positionCap h s B n))
      (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh)
        (Fraction.equiv_refl (Fraction.add (pointNorm s.2) (Fraction.mul (time h n) B))))
    exact Fraction.magnitudes.le_trans
      (Fraction.le_equiv_right (Fraction.magnitudes.le_trans hc hi) he)
      (position_cap_step h (pointNorm s.1) (pointNorm s.2) B n hh hB)

```

```

def uniformPositionCap (T : Fraction) (s : Point × Point) (B : Fraction) : Fraction :=
  Fraction.add (pointNorm s.1)
    (Fraction.add (Fraction.mul T (pointNorm s.2))
      (Fraction.mul (Fraction.mul T T) B))

/-- A bound depending on the total window, uniformly in the cell count. -/
theorem velocity_bound_at_time (a : Point → Point) (h : Fraction)
  (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num)
  (n : Nat) (hb : BoundedSamples a h s B n) (ht : Fraction.le (time h n) T) :
  Fraction.le (pointNorm (run a h s n).2)
    (Fraction.add (pointNorm s.2) (Fraction.mul T B)) :=
  Fraction.magnitudes.le_trans (velocity_bound a h s B hh n hb)
    (Fraction.add_le_add_left (Fraction.mul_le_mul_nonnegative ht B hB) (pointNorm s.2))

theorem position_bound_at_time (a : Point → Point) (h : Fraction)
  (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num)
  (hT : 0 ≤ T.num) (n : Nat) (hb : BoundedSamples a h s B n)
  (ht : Fraction.le (time h n) T) :
  Fraction.le (pointNorm (run a h s n).1) (uniformPositionCap T s B) := by
  have hsquare := Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative ht (time h n) (time_nonnegative h hh n))
    (Fraction.mul_le_mul_nonnegative_left ht T hT)
  have hvel := Fraction.mul_le_mul_nonnegative ht (pointNorm s.2) (pointNorm_nonnegative _)
  have hforce := Fraction.mul_le_mul_nonnegative hsquare B hB
  exact Fraction.magnitudes.le_trans (position_bound a h s B hh hB n hb)
    (Fraction.add_le_add_left (Fraction.add_le_add hvel hforce) (pointNorm s.1))

theorem state_bound_at_time (a : Point → Point) (h : Fraction)
  (s : Point × Point) (B T : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num)
  (hT : 0 ≤ T.num) (n : Nat) (hb : BoundedSamples a h s B n)
  (ht : Fraction.le (time h n) T) :
  Fraction.le (stateNorm (run a h s n))
    (Fraction.add (uniformPositionCap T s B)
      (Fraction.add (pointNorm s.2) (Fraction.mul T B))) :=
  Fraction.add_le_add (position_bound_at_time a h s B T hh hB hT n hb ht)
    (velocity_bound_at_time a h s B T hh hB n hb ht)

theorem run_add (a : Point → Point) (h : Fraction) (s : Point × Point) (n : Nat) :
  (k : Nat) → run a h s (n+k) = run a h (run a h s n) k
| 0 => rfl
| k+1 => by
  change FiniteEstimates.cell a h (run a h s (n+k)) =
    FiniteEstimates.cell a h (run a h (run a h s n) k)
  rw [run_add a h s n k]

theorem boundedSamples_restart (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B : Fraction) (N n k : Nat) (hnk : n+k ≤ N) (hb : BoundedSamples a h s B N) :
  BoundedSamples a h (run a h s n) B k := by
  intro i hi
  have h := hb (n+i) (by omega)
  rw [show n+i+1=n+(i+1) by omega, run_add] at h
  exact h

end NewtonLimitDynamics.Polygon.BoundedIteration

```

BarrowLib/Polygon/TriangleBounds.lean, 15 theorems, 126 lines

```
import BarrowLib.Polygon.PointBounds
```

```

/-!
Unsigned doubled triangle magnitudes for finite rational path patches.
These estimates concern area between compared paths, separately from their
Kepler swept areas. A sum counts patches with multiplicity; interpreting it
as a union's area still needs the geometric decomposition or enclosure.
No limiting trajectory is supplied or constructed here.
-/

```

```
namespace NewtonLimitDynamics.Polygon.TriangleBounds
```

```
open NewtonLimitDynamics
```

```

open TimeSubdivision
open PointBounds

/-- Determinant magnitude is bounded by the product of coordinate L1
magnitudes. The determinant is a doubled oriented area, not a distance. -/
theorem det_abs_le_product (p q : Point) :
  Fraction.le (det p q).abs (Fraction.mul (pointNorm p) (pointNorm q)) := by
  let a := Fraction.mul p.1 q.2
  let b := Fraction.mul p.2 q.1
  let nb : Fraction := (-b.num, b.den, b.den_pos)
  let cross := Fraction.add (Fraction.mul p.1.abs q.2.abs)
    (Fraction.mul p.2.abs q.1.abs)
  let diagonal := Fraction.add (Fraction.mul p.1.abs q.1.abs)
    (Fraction.mul p.2.abs q.2.abs)
  have h₀ := Fraction.abs_add_le a nb
  have he : Fraction.equiv (Fraction.add a.abs nb.abs) cross :=
    Fraction.add_equiv (Fraction.abs_mul _ _)
      (Fraction.equiv_trans (Fraction.abs_neg b) (Fraction.abs_mul _ _))
  have h₁ : Fraction.le (det p q).abs cross := Fraction.le_equiv_right h₀ he
  have hd : 0 ≤ diagonal.num :=
    Fraction.nonnegative_add _ _
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
  have h₂ := Fraction.le_add_nonnegative cross diagonal hd
  have hchain := Fraction.magnitudes.le_trans h₁ h₂
  apply Fraction.le_equiv_right hchain
  simp only [cross, diagonal, pointNorm, Fraction.equiv, Fraction.add, Fraction.mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

def triangleTwice (p q r : Point) : Fraction :=
  det (pointSub q p) (pointSub r p)

/-- A nonzero areal product and bounded second vector give a radial lower bound.
This is finite determinant arithmetic, without a force or motion premise. -/
theorem radius_lower_of_areal_bound (p v : Point) (r V : Fraction)
  (hV : 0 < V.num) (hv : Fraction.le (pointNorm v) V)
  (ha : Fraction.le (Fraction.mul r V) (det p v).abs) :
  Fraction.le r (pointNorm p) := by
  have hd := Fraction.magnitudes.le_trans (det_abs_le_product p v)
    (Fraction.mul_le_mul_nonnegative_left hv (pointNorm p) (pointNorm_nonnegative p))
  exact Fraction.mul_le_cancel_positive_right V hv (Fraction.magnitudes.le_trans ha hd)

def triangleMagnitude (p q r : Point) : Fraction := (triangleTwice p q r).abs

theorem triangleMagnitude_nonnegative (p q r : Point) :
  0 ≤ (triangleMagnitude p q r).num := Fraction.abs_num_nonnegative _

theorem triangleMagnitude_le_product (p q r : Point) :
  Fraction.le (triangleMagnitude p q r)
    (Fraction.mul (pointNorm (pointSub q p)) (pointNorm (pointSub r p))) :=
  det_abs_le_product _ _

/-- The same triangle can be estimated using its two consecutive edges. -/
theorem triangleTwice_consecutive (p q r : Point) :
  Fraction.equiv (triangleTwice p q r)
    (det (pointSub q p) (pointSub r q)) := by
  simp only [triangleTwice, det, pointSub, pointNeg, pointAdd, Fraction.equiv,
    Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf <;> omega

theorem triangleMagnitude_le_consecutive (p q r : Point) :
  Fraction.le (triangleMagnitude p q r)
    (Fraction.mul (pointNorm (pointSub q p)) (pointNorm (pointSub r q))) :=
  Fraction.le_equiv_left (Fraction.abs_equiv (triangleTwice_consecutive p q r))
    (det_abs_le_product _ _)

theorem triangleTwice_translation (origin p q r : Point) :
  Fraction.equiv
    (triangleTwice (pointSub p origin) (pointSub q origin) (pointSub r origin))

```

```

    (triangleTwice p q r) := by
    simp only [triangleTwice, det, pointSub, pointNeg, pointAdd, Fraction.equiv,
    Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
    ac_nf <=> omega

theorem triangleMagnitude_translation (origin p q r : Point) :
    Fraction.equiv
    (triangleMagnitude (pointSub p origin) (pointSub q origin) (pointSub r origin))
    (triangleMagnitude p q r) :=
    Fraction.abs_equiv (triangleTwice_translation origin p q r)

theorem triangleTwice_swap (p q r : Point) :
    Fraction.equiv (triangleTwice p r q)
    ((-(triangleTwice p q r).num,
    (triangleTwice p q r).den, (triangleTwice p q r).den_pos) : Fraction) := by
    simp only [triangleTwice, det, pointSub, pointNeg, pointAdd, Fraction.equiv,
    Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
    ac_nf <=> omega

theorem triangleMagnitude_swap (p q r : Point) :
    Fraction.equiv (triangleMagnitude p r q) (triangleMagnitude p q r) :=
    Fraction.equiv_trans (Fraction.abs_equiv (triangleTwice_swap p q r))
    (Fraction.abs_neg _)

private def zero : Fraction := Fraction.ofInt 0
private def one : Fraction := Fraction.ofInt 1
private def p : Point := (zero, zero)
private def q : Point := (one, zero)
private def r : Point := (zero, one)

theorem sample_signed_positive : Fraction.equiv (triangleTwice p q r) one := by decide
theorem sample_signed_negative :
    Fraction.equiv (triangleTwice p r q) (Fraction.ofInt (-1)) := by decide
theorem sample_opposite_unsigned_sum :
    Fraction.equiv
    (Fraction.add (triangleMagnitude p q r) (triangleMagnitude p r q))
    (Fraction.ofInt 2) := by decide
theorem sample_opposite_signed_sum :
    Fraction.equiv (Fraction.add (triangleTwice p q r) (triangleTwice p r q)) zero := by decide
theorem sample_unsigned_not_signed :
    ~ Fraction.equiv (triangleMagnitude p r q) (triangleTwice p r q) := by decide

end NewtonLimitDynamics.Polygon.TriangleBounds

```

BarrowLib/Polygon/ConvexCover.lean, 17 theorems, 196 lines

```

import BarrowLib.Polygon.PointBounds

/-!
Finite rational convex enclosures in the chosen coordinate L1 magnitude.
The square is an explicit point set; its area is a covering budget, not an
assertion about the area of a union of patches or a limiting trajectory.
-/

namespace NewtonLimitDynamics.Polygon.ConvexCover

open NewtonLimitDynamics
open TimeSubdivision
open PointBounds

def UnitInterval (a : Fraction) : Prop := 0 ≤ a.num ∧ a.num ≤ a.den

def complement (a : Fraction) : Fraction :=
    (a.den - a.num, a.den, a.den_pos)

theorem complement_nonnegative (a : Fraction) (ha : UnitInterval a) :
    0 ≤ (complement a).num := by
    obtain (_, hupper) := ha
    unfold complement
    dsimp
    omega

```

```

theorem complement_abs (a : Fraction) (ha : UnitInterval a) :
  Fraction.equiv (complement a).abs (complement a) :=
  Fraction.abs_of_nonnegative _ (complement_nonnegative a ha)

theorem interval_abs (a : Fraction) (ha : UnitInterval a) :
  Fraction.equiv a.abs a := Fraction.abs_of_nonnegative _ ha.1

theorem weights_sum_one (a : Fraction) :
  Fraction.equiv (Fraction.add (complement a) a) (Fraction.ofInt 1) := by
  unfold complement Fraction.equiv Fraction.add Fraction.ofInt
  dsimp
  simp only [Int.sub_mul, Int.mul_sub]
  ac_nf
  omega

def lerp (a : Fraction) (p q : Point) : Point :=
  pointAdd (pointScale (complement a) p) (pointScale a q)

theorem pointSub_self_zero (p : Point) :
  Fraction.equiv (pointNorm (pointSub p p)) (Fraction.ofInt 0) := by
  have hz (a : Int) : (a + -a).natAbs = 0 := by omega
  simp only [pointNorm, pointSub, pointNeg, pointAdd, Fraction.equiv,
    Fraction.abs, Fraction.add, Fraction.ofInt,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  simp only [hz, Int.ofNat_zero, Int.zero_mul, Int.mul_zero, Int.add_zero]

theorem pointSub_chain (a b c : Point) :
  pointEquiv (pointSub a c)
    (pointAdd (pointSub a b) (pointSub b c)) := by
  constructor <=>
  simp only [pointSub, pointNeg, pointAdd, Fraction.equiv, Fraction.add,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem pointSub_triangle (a b c : Point) :
  Fraction.le (pointNorm (pointSub a c))
    (Fraction.add (pointNorm (pointSub a b)) (pointNorm (pointSub b c))) :=
  Fraction.le_equiv_left (pointNorm_equiv (pointSub_chain a b c))
    (pointNorm_add_le (pointSub a b) (pointSub b c))

theorem drift_offset (h : Fraction) (x v : Point) :
  pointEquiv (pointSub (pointAdd x (pointScale h v)) x) (pointScale h v) := by
  constructor <=>
  simp only [pointSub, pointNeg, pointAdd, pointScale, Fraction.equiv,
    Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add,
    Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- Translation of a lerp is the same weighted sum of endpoint offsets. -/
theorem lerp_offset (a : Fraction) (anchor p q : Point) :
  pointEquiv (pointSub (lerp a p q) anchor)
    (pointAdd (pointScale (complement a) (pointSub p anchor))
      (pointScale a (pointSub q anchor))) := by
  constructor <=>
  simp only [lerp, complement, pointEquiv, pointSub, pointNeg, pointAdd,
    pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.sub_mul, Int.mul_sub,
    Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- A convex combination of two points in the L1 ball remains in that ball. -/
theorem lerp_ball_bound (a : Fraction) (ha : UnitInterval a)
  (anchor p q : Point) (R : Fraction)
  (hp : Fraction.le (pointNorm (pointSub p anchor)) R)
  (hq : Fraction.le (pointNorm (pointSub q anchor)) R) :
  Fraction.le (pointNorm (pointSub (lerp a p q) anchor)) R := by
  let c := complement a
  let P := pointSub p anchor
  let Q := pointSub q anchor

```



```

have ht := pointNorm_add_le (pointScale c P) (pointScale a Q)
have hs : Fraction.equiv
  (Fraction.add (pointNorm (pointScale c P)) (pointNorm (pointScale a Q)))
  (Fraction.add (Fraction.mul c (pointNorm P)) (Fraction.mul a (pointNorm Q))) :=
  Fraction.add_equiv
  (Fraction.equiv_trans (pointNorm_scale c P)
    (Fraction.mul_equiv (complement_abs a ha) (Fraction.equiv_refl _)))
  (Fraction.equiv_trans (pointNorm_scale a Q)
    (Fraction.mul_equiv (interval_abs a ha) (Fraction.equiv_refl _)))
have h₁ := Fraction.mul_le_mul_nonnegative_left hp c (complement_nonnegative a ha)
have h₂ := Fraction.mul_le_mul_nonnegative_left hq a ha.1
have hc := Fraction.add_le_add h₁ h₂
have he : Fraction.equiv
  (Fraction.add (Fraction.mul c R) (Fraction.mul a R)) R := by
  have hw := weights_sum_one a
  have hm := Fraction.mul_equiv_left R hw
  have hz : Fraction.equiv (Fraction.mul (Fraction.add c a) R)
    (Fraction.add (Fraction.mul c R) (Fraction.mul a R)) := by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul]
    simp only [Int.add_mul, Int.mul_add]
    ac_nf
  have hone : Fraction.equiv (Fraction.mul R (Fraction.ofInt 1)) R := by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
  exact Fraction.equiv_trans (Fraction.equiv_symm hz)
    (Fraction.equiv_trans (Fraction.mul_comm (Fraction.add c a) R)
      (Fraction.equiv_trans hm hone))
have hraw := Fraction.le_equiv_left (pointNorm_equiv (lerp_offset a anchor p q))
  (Fraction.le_equiv_right ht hs)
exact Fraction.le_equiv_right (Fraction.magnitudes.le_trans hraw hc) he

def SquareContains (anchor : Point) (R : Fraction) (p : Point) : Prop :=
  Fraction.le (pointSub p anchor).1.abs R ∧
  Fraction.le (pointSub p anchor).2.abs R

theorem ball_inside_square (anchor p : Point) (R : Fraction)
  (h : Fraction.le (pointNorm (pointSub p anchor)) R) :
  SquareContains anchor R p := by
  constructor
  · exact Fraction.magnitudes.le_trans
    (Fraction.le_add_nonnegative _ _ (Fraction.abs_num_nonnegative _)) h
  · have h₂ := Fraction.le_add_nonnegative (pointSub p anchor).2.abs
    (pointSub p anchor).1.abs (Fraction.abs_num_nonnegative _)
    exact Fraction.magnitudes.le_trans
      (Fraction.le_equiv_right h₂
        (Fraction.add_comm (pointSub p anchor).2.abs (pointSub p anchor).1.abs)) h

/-- A matched quad patch uses one physical-time fraction `theta` on both
polygon edges, then interpolates between those simultaneous positions. -/
def matchedPatch (theta lambda : Fraction) (c0 c1 f0 f1 : Point) : Point :=
  lerp lambda (lerp theta c0 c1) (lerp theta f0 f1)

theorem matchedPatch_ball (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda)
  (anchor c0 c1 f0 f1 : Point) (R : Fraction)
  (hc0 : Fraction.le (pointNorm (pointSub c0 anchor)) R)
  (hc1 : Fraction.le (pointNorm (pointSub c1 anchor)) R)
  (hf0 : Fraction.le (pointNorm (pointSub f0 anchor)) R)
  (hf1 : Fraction.le (pointNorm (pointSub f1 anchor)) R) :
  Fraction.le (pointNorm (pointSub
    (matchedPatch theta lambda c0 c1 f0 f1) anchor)) R :=
  lerp_ball_bound lambda hl anchor _ _ R
  (lerp_ball_bound theta ht anchor c0 c1 R hc0 hc1)
  (lerp_ball_bound theta ht anchor f0 f1 R hf0 hf1)

theorem matchedPatch_square (theta lambda : Fraction)
  (ht : UnitInterval theta) (hl : UnitInterval lambda)
  (anchor c0 c1 f0 f1 : Point) (R : Fraction)
  (hc0 : Fraction.le (pointNorm (pointSub c0 anchor)) R)
  (hc1 : Fraction.le (pointNorm (pointSub c1 anchor)) R)

```

```

    (hf0 : Fraction.le (pointNorm (pointSub f0 anchor)) R)
    (hf1 : Fraction.le (pointNorm (pointSub f1 anchor)) R) :
    SquareContains anchor R (matchedPatch theta lambda c0 c1 f0 f1) :=
    ball_inside_square anchor _ R
    (matchedPatch_ball theta lambda ht hl anchor c0 c1 f0 f1 R hc0 hc1 hf0 hf1)

theorem complement_interval (a : Fraction) (ha : UnitInterval a) :
  UnitInterval (complement a) := by
  obtain (hl,hu) := ha
  unfold UnitInterval complement
  dsimp
  omega

theorem lerp_zero (p q : Point) : pointEquiv (lerp (Fraction.ofInt 0) p q) p := by
  constructor <=>
  simp [lerp, complement, pointAdd, pointScale, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt] <=> ac_nf

theorem lerp_one (p q : Point) : pointEquiv (lerp (Fraction.ofInt 1) p q) q := by
  constructor <=>
  simp [lerp, complement, pointAdd, pointScale, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt] <=> ac_nf

theorem lerp_swap (a : Fraction) (p q : Point) :
  pointEquiv (lerp a p q) (lerp (complement a) q p) := by
  constructor <=>
  simp only [lerp, complement, pointAdd, pointScale, Fraction.equiv, Fraction.add,
    Fraction.mul, Int.add_mul, Int.mul_add, Int.sub_mul, Int.mul_sub] <=>
  ac_nf <=> omega

end NewtonLimitDynamics.Polygon.ConvexCover

BarrowLib/Polygon/DyadicArithmetic.lean, 21 theorems, 207 lines

import BarrowLib.Common.RationalMagnitudes

namespace NewtonLimitDynamics.Polygon.HarmonicStability
open NewtonLimitDynamics
def negF (w : Fraction) : Fraction := {-w.num, w.den, w.den_pos}

end NewtonLimitDynamics.Polygon.HarmonicStability

namespace NewtonLimitDynamics.Polygon.HarmonicDyadic
open NewtonLimitDynamics
def blocks (j : Nat) : Nat := 2 ^ j

def duration (T : Fraction) (j : Nat) : Fraction :=
  (T.num, T.den * (2 : Int) ^ j,
    Int.mul_pos T.den_pos (Int.pow_pos (by decide)))

theorem blocks_succ (j : Nat) : blocks (j + 1) = blocks j + blocks j := by
  unfold blocks
  rw [Nat.pow_succ]
  omega

theorem duration_halving (T : Fraction) (j : Nat) :
  Fraction.equiv (duration T j)
    (Fraction.add (duration T (j + 1)) (duration T (j + 1))) := by
  simp only [duration, Fraction.equiv, Fraction.add, Int.pow_succ]
  simp only [show (2 : Int) = 1 + 1 by rfl, Int.add_mul, Int.mul_add]
  ac_nf

theorem duration_congr {T U : Fraction} (h : Fraction.equiv T U) (j : Nat) :
  Fraction.equiv (duration T j) (duration U j) := by
  have hm := congrArg (fun z : Int => z*(2:Int)^j) h
  simpa only [duration, Fraction.equiv, Int.mul_assoc] using hm

theorem blocks_duration (T : Fraction) (m : Nat) :
  Fraction.equiv (Fraction.mul (Fraction.ofInt (blocks m:Int)) (duration T m)) T := by
  simp only [blocks, duration, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.natCast_pow]
  ac_nf

```

```

theorem two_pow_ge_succ (N : Nat) :
  (N : Int) + 1 ≤ (2 : Int) ^ N := by
  induction N with
  | zero => decide
  | succ n ih =>
    rw [Int.pow_succ]
    have hp : 0 ≤ (2 : Int) ^ n := Int.le_of_lt (Int.pow_pos (by decide))
    omega

theorem neg_equiv {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv (-a.num, a.den, a.den_pos) (-b.num, b.den, b.den_pos) := by
  unfold Fraction.equiv at *
  dsimp
  simp only [Int.neg_mul, h]

end NewtonLimitDynamics.Polygon.HarmonicDyadic

namespace NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix
open HarmonicDyadic
def bit (b : Nat → Bool) (j : Nat) : Nat := if b j then 1 else 0

def ticks (b : Nat → Bool) : Nat → Nat
| 0 => 0
| j + 1 => 2 * ticks b j + bit b j

theorem bit_le_one (b : Nat → Bool) (j : Nat) : bit b j ≤ 1 := by
  unfold bit
  split <=> omega

theorem ticks_lt_blocks (b : Nat → Bool) :
  (j : Nat) → ticks b j < blocks j
| 0 => by simp [ticks, blocks]
| j + 1 => by
  have ih := ticks_lt_blocks b j
  have hb := bit_le_one b j
  rw [ticks, blocks_succ]
  omega

theorem ticks_le_blocks (b : Nat → Bool) (j : Nat) :
  ticks b j ≤ blocks j := Nat.le_of_lt (ticks_lt_blocks b j)

theorem ticks_next (b : Nat → Bool) (j : Nat) :
  ticks b (j + 1) = ticks b j + ticks b j + bit b j := by
  simp only [ticks]
  omega

end NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

namespace NewtonLimitDynamics.Polygon.HarmonicTimeComparison
open NewtonLimitDynamics
open HarmonicStability
def durationDifference (sigma tau : Fraction) : Fraction :=
  Fraction.add tau (negF sigma)

theorem durationDifference_chain (a b c : Fraction) :
  Fraction.equiv (durationDifference a c)
  (Fraction.add (durationDifference a b) (durationDifference b c)) := by
  simp only [durationDifference, negF, Fraction.equiv, Fraction.add,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

theorem add_difference_cancel (a b : Fraction) :
  Fraction.equiv (Fraction.add a (durationDifference a b)) b := by
  simp only [durationDifference, negF, Fraction.equiv, Fraction.add,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

```

```

theorem difference_nonnegative_iff (a b : Fraction) :
  0 ≤ (durationDifference a b).num ↔ Fraction.le a b := by
  simp only [durationDifference, negF, Fraction.add, Fraction.le, Int.neg_mul]
  omega

theorem difference_add_bound (a b H : Fraction)
  (h : Fraction.le (durationDifference a b) H) :
  Fraction.le b (Fraction.add a H) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (add_difference_cancel a b))
  (Fraction.add_le_add_left h a)

theorem difference_congr {a b c d : Fraction}
  (hac : Fraction.equiv a c) (hbd : Fraction.equiv b d) :
  Fraction.equiv (durationDifference a b) (durationDifference c d) :=
  Fraction.add_equiv hbd (HarmonicDyadic.neg_equiv hac)

theorem phase_end_difference (a H t : Fraction) :
  Fraction.equiv (durationDifference (durationDifference a t) H)
  (durationDifference t (Fraction.add a H)) := by
  simp only [durationDifference, negF, Fraction.equiv, Fraction.add,
  Int.add_mul, Int.mul_add, Int.mul_neg, Int.mul_neg]
  ac_nf
  omega

theorem difference_interval_gaps (a e c : Fraction)
  (ha : Fraction.le a e) (hc : Fraction.le e c) :
  Fraction.le (durationDifference a e).abs (durationDifference a c).abs ∧
  Fraction.le (durationDifference e c).abs (durationDifference a c).abs := by
  have hae := (difference_nonnegative_iff a e).mpr ha
  have hec := (difference_nonnegative_iff e c).mpr hc
  have hac := (difference_nonnegative_iff a c).mpr
  (Fraction.magnitudes.le_trans ha hc)
  have he := durationDifference_chain a e c
  have hb1 := Fraction.le_add_nonnegative (durationDifference a e)
  (durationDifference e c) hec
  have hb2 := Fraction.le_equiv_right
  (Fraction.le_add_nonnegative (durationDifference e c)
  (durationDifference a e) hae)
  (Fraction.add_comm _ _)
  have h1 := Fraction.le_equiv_right hb1 (Fraction.equiv_symm he)
  have h2 := Fraction.le_equiv_right hb2 (Fraction.equiv_symm he)
  constructor
  · exact Fraction.le_equiv_right
  (Fraction.le_equiv_left (Fraction.abs_of_nonnegative _ hae) h1)
  (Fraction.equiv_symm (Fraction.abs_of_nonnegative _ hac))
  · exact Fraction.le_equiv_right
  (Fraction.le_equiv_left (Fraction.abs_of_nonnegative _ hec) h2)
  (Fraction.equiv_symm (Fraction.abs_of_nonnegative _ hac))
end NewtonLimitDynamics.Polygon.HarmonicTimeComparison

namespace NewtonLimitDynamics.Polygon.HarmonicTimeRealization
open NewtonLimitDynamics
open HarmonicStability HarmonicDyadic HarmonicTimeComparison

def countTime (T : Fraction) (j n : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (n : Int)) (duration T j)

theorem countTime_difference (T : Fraction) (j n k : Nat) :
  Fraction.equiv
  (durationDifference (countTime T j n) (countTime T j (n + k)))
  (Fraction.mul (Fraction.ofInt (k : Int)) (duration T j)) := by
  simp only [countTime, durationDifference, negF, Fraction.equiv,
  Fraction.add, Fraction.mul, Fraction.ofInt, Int.natCast_add]
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
  Int.one_mul, Int.mul_one]
  ac_nf <=> omega

theorem countTime_abs_difference (T : Fraction) (j n k : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.equiv

```

```

      (durationDifference (countTime T j n) (countTime T j (n + k))).abs
      (Fraction.mul (Fraction.ofInt (k : Int)) (duration T j)) := by
    have hs := Fraction.abs_equiv (countTime_difference T j n k)
    have hnon : 0 ≤ (Fraction.mul (Fraction.ofInt (k : Int)) (duration T j)).num :=
      Int.mul_nonneg (Int.ofNat_nonneg _) hT
    exact Fraction.equiv_trans hs
      (Fraction.abs_of_nonnegative _ hnon)

theorem durationDifference_abs_symm (a b : Fraction) :
  Fraction.equiv (durationDifference a b).abs
    (durationDifference b a).abs := by
  have he : Fraction.equiv (durationDifference a b)
    (negF (durationDifference b a)) := by
    simp only [durationDifference, negF, Fraction.equiv,
      Fraction.add]
    simp only [Int.mul_add, Int.mul_neg, Int.neg_mul, Int.neg_add,
      Int.neg_neg]
    ac_nf <=> omega
  exact Fraction.equiv_trans (Fraction.abs_equiv he)
    (Fraction.abs_neg _)

end NewtonLimitDynamics.Polygon.HarmonicTimeRealization

namespace NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix
theorem all_zero_ticks (j : Nat) : ticks (fun _ => false) j = 0 := by
  induction j with
  | zero => rfl
  | succ j ih => simp [ticks, bit, ih]

end NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

```

BarrowLib/Polygon/FinitePower.lean, 1 theorem, 15 lines

```

import BarrowLib.Common.RationalMagnitudes

namespace NewtonLimitDynamics.Polygon.HarmonicAccumulation
open NewtonLimitDynamics
/-- Finite rational powers, avoiding any completeness premise. -/
def fpower (a : Fraction) : Nat → Fraction
| 0 => Fraction.ofInt 1
| n + 1 => Fraction.mul a (fpower a n)

theorem fpower_nonnegative (a : Fraction) (ha : 0 ≤ a.num) :
  (n : Nat) → 0 ≤ (fpower a n).num
| 0 => by simp [fpower, Fraction.ofInt]
| n + 1 => Fraction.nonnegative_mul _ _ ha (fpower_nonnegative a ha n)

end NewtonLimitDynamics.Polygon.HarmonicAccumulation

```

BarrowLib/Polygon/StateDistance.lean, 6 theorems, 52 lines

```

import BarrowLib.Polygon.FiniteEstimates
import BarrowLib.Polygon.DyadicArithmetic

namespace NewtonLimitDynamics.Polygon.HarmonicComparison
open NewtonLimitDynamics
open TimeSubdivision
def stateSub (s t : Point × Point) : Point × Point :=
  (pointSub s.1 t.1, pointSub s.2 t.2)

end NewtonLimitDynamics.Polygon.HarmonicComparison

namespace NewtonLimitDynamics.Polygon.HarmonicDyadic
open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
private theorem pointNeg_congr {p q : Point} (hp : pointEquiv p q) :
  pointEquiv (pointNeg p) (pointNeg q) :=
  (neg_equiv hp.1, neg_equiv hp.2)

```

```

private theorem pointSub_congr {p p' q q' : Point}
  (hp : pointEquiv p p') (hq : pointEquiv q q') :
  pointEquiv (pointSub p q) (pointSub p' q') :=
  pointAdd_congr hp (pointNeg_congr hq)

theorem stateSub_congr {s s' t t' : Point × Point}
  (hs : stateEquiv s s') (ht : stateEquiv t t') :
  stateEquiv (stateSub s t) (stateSub s' t') :=
  (pointSub_congr hs.1 ht.1, pointSub_congr hs.2 ht.2)

theorem stateSub_norm_symm (a b : Point × Point) :
  Fraction.equiv (stateNorm (stateSub a b)) (stateNorm (stateSub b a)) :=
  FiniteEstimates.stateDistance_symm a b

end NewtonLimitDynamics.Polygon.HarmonicDyadic

namespace NewtonLimitDynamics.Polygon.HarmonicAccumulation
open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
open HarmonicDyadic
open FiniteEstimates
theorem stateSub_triangle (a b c : Point × Point) :
  Fraction.le (stateNorm (stateSub a c))
    (Fraction.add (stateNorm (stateSub a b)) (stateNorm (stateSub b c))) :=
  stateDistance_triangle a b c

theorem stateSub_self_norm_zero (s : Point × Point) :
  Fraction.equiv (stateNorm (stateSub s s)) (Fraction.ofInt 0) := stateDistance_self_zero s

end NewtonLimitDynamics.Polygon.HarmonicAccumulation

```

BarrowLib/Polygon/CauchyValues.lean, 38 theorems, 485 lines

```

import BarrowLib.Polygon.EndpointCauchyName
import BarrowLib.Polygon.GeometricTail

namespace NewtonLimitDynamics.Polygon.CauchyValues
open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
open HarmonicAccumulation
open HarmonicDyadic

def distance (a b : Point × Point) : Fraction := stateNorm (stateSub a b)

/-- Position magnitudes on a Cauchy tail are bounded by the magnitude of one
actual approximant plus one. This is derived, rather than a field of a name. -/
theorem position_bounded_tail (a : EndpointCauchyName) :
  ∃ R : Fraction, 0 ≤ R.num ∧ ∃ N : Nat, ∀ j, N ≤ j →
    Fraction.le (pointNorm (a.approx j).1) R := by
  obtain ⟨N, hN⟩ := a.cauchy (Fraction.ofInt 1) (by decide)
  let R := Fraction.add (Fraction.ofInt 1) (pointNorm (a.approx N).1)
  refine ⟨R, Fraction.nonnegative_add _ _ (by decide) (pointNorm_nonnegative _), N, fun j hj => ?_⟩
  have hp := Fraction.magnitudes.le_trans (point_le_state (stateSub (a.approx j) (a.approx N)))
    (Fraction.magnitudes.lt_implies_le (hN j N hj (Nat.le_refl _)))
  exact Fraction.magnitudes.le_trans
    (FiniteEstimates.pointNorm_le_distance_add (a.approx j).1 (a.approx N).1)
    (Fraction.add_le_add_right hp (pointNorm (a.approx N).1))

/-- A Cauchy name is bounded on a proved tail, so its position magnitude
times a geometric mesh tends to zero. No boundedness field is supplied. -/
theorem mesh_position_product_vanishes (a : EndpointCauchyName)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j, N ≤ j →
    Fraction.lt (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (a.approx j).1)) eps := by
  obtain ⟨R, hR, N, hN⟩ := position_bounded_tail a
  obtain ⟨M, hM⟩ := HarmonicTimeRealization.duration_eventually_small R eps hR heps
  refine ⟨max N M, fun j hj => ?_⟩

```

```

have hc := Fraction.mul_le_mul_nonnegative_left (hN j (by omega)) (duration (Fraction.ofInt 1) j)
  (by change (0 : Int) ≤ 1; omega)
have he : Fraction.equiv (Fraction.mul (duration (Fraction.ofInt 1) j) R) (duration R j) := by
  simp only [duration, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one]
  ac_nf
exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_equiv_right hc he) (hM j (by omega))

private theorem equiv_zero_num (a : Fraction)
  (h : Fraction.equiv a (Fraction.ofInt 0)) : a.num = 0 := by
  unfold Fraction.equiv Fraction.ofInt at h
  simp only [Int.mul_one, Int.zero_mul] using h

private theorem zero_lt_positive (eps : Fraction) (heps : 0 < eps.num) :
  Fraction.lt (Fraction.ofInt 0) eps := by
  unfold Fraction.lt Fraction.ofInt
  dsimp
  simp only [Int.zero_mul, Int.mul_one]
  exact heps

theorem distance_self_lt (a : Point × Point) (eps : Fraction)
  (heps : 0 < eps.num) : Fraction.lt (distance a a) eps := by
  have hz := equiv_zero_num _ (stateSub_self_norm_zero a)
  unfold Fraction.lt distance
  simp only [hz, Int.zero_mul, Int.mul_one]
  exact Int.mul_pos heps (distance a a).den_pos

def NameEquiv (a b : EndpointCauchyName) : Prop :=
  ∀ eps : Fraction, 0 < eps.num →
    ∃ N : Nat, ∀ n : Nat, N ≤ n →
      Fraction.lt (distance (a.approx n) (b.approx n)) eps

theorem nameEquiv_refl (a : EndpointCauchyName) : NameEquiv a a := by
  intro eps heps
  exact (0, fun n _ => distance_self_lt (a.approx n) eps heps)

theorem nameEquiv_symm {a b : EndpointCauchyName}
  (hab : NameEquiv a b) : NameEquiv b a := by
  intro eps heps
  obtain (N, hN) := hab eps heps
  refine (N, ?_)
  intro n hn
  have hs := stateSub_norm_symm (b.approx n) (a.approx n)
  exact Fraction.magnitudes.lt_of_le_lt
    ((Fraction.equiv_iff_mutual_le _ _).mp hs).1 (hN n hn)

private theorem half_add_equiv (eps : Fraction) :
  Fraction.equiv (Fraction.add eps.half eps.half) eps :=
  Fraction.half_add_self eps

private theorem half_lt (eps : Fraction) (heps : 0 < eps.num) :
  Fraction.lt eps.half eps := Fraction.half_lt eps heps

theorem nameEquiv_trans {a b c : EndpointCauchyName}
  (hab : NameEquiv a b) (hbc : NameEquiv b c) : NameEquiv a c := by
  intro eps heps
  let q := eps.half.half
  obtain (N1, h1) := hab q heps
  obtain (N2, h2) := hbc q heps
  refine (N1 + N2, ?_)
  intro n hn
  have hn1 : N1 ≤ n := by omega
  have hn2 : N2 ≤ n := by omega
  have htri := stateSub_triangle (a.approx n) (b.approx n) (c.approx n)
  have hadd := Fraction.add_le_add
    (Fraction.magnitudes.lt_implies_le (h1 n hn1))
    (Fraction.magnitudes.lt_implies_le (h2 n hn2))
  have hbound := Fraction.le_equiv_right
    (Fraction.magnitudes.le_trans htri hadd) (half_add_equiv eps.half)
  exact Fraction.magnitudes.lt_of_le_lt hbound (half_lt eps heps)

```

```

instance : Setoid EndpointCauchyName where
  r := NameEquiv
  iseqv := {nameEquiv_refl, @nameEquiv_symm, @nameEquiv_trans}

def Value := Quotient (inferInstance : Setoid EndpointCauchyName)

def realize (a : EndpointCauchyName) : Value := Quotient.mk _ a

private theorem add_zero_split (a b : Fraction)
  (ha : 0 ≤ a.num) (hb : 0 ≤ b.num)
  (h : Fraction.equiv (Fraction.add a b) (Fraction.ofInt 0)) :
  a.num = 0 ∧ b.num = 0 := by
  have hz := equiv_zero_num _ h
  have h1 : 0 ≤ a.num * b.den :=
    Int.mul_nonneg ha (Int.le_of_lt b.den_pos)
  have h2 : 0 ≤ b.num * a.den :=
    Int.mul_nonneg hb (Int.le_of_lt a.den_pos)
  have hs : a.num * b.den + b.num * a.den = 0 := hz
  have ha' : a.num * b.den = 0 := by omega
  have hb' : b.num * a.den = 0 := by omega
  constructor
  · rcases (Int.mul_eq_zero.mp ha') with hz' | hz'
    · exact hz'
    · have hd := b.den_pos
      omega
  · rcases (Int.mul_eq_zero.mp hb') with hz' | hz'
    · exact hz'
    · have hd := a.den_pos
      omega

private theorem pointNorm_zero_coords (p : Point)
  (h : (pointNorm p).num = 0) : p.1.num = 0 ∧ p.2.num = 0 := by
  have he : Fraction.equiv (pointNorm p) (Fraction.ofInt 0) := by
    unfold Fraction.equiv Fraction.ofInt
    simp [h]
  have hs := add_zero_split p.1.abs p.2.abs
  (Fraction.abs_num_nonnegative p.1)
  (Fraction.abs_num_nonnegative p.2) he
  constructor
  · have hh : p.1.num.natAbs = 0 := Int.ofNat_eq_zero.mp hs.1
    exact Int.natAbs_eq_zero.mp hh
  · have hh : p.2.num.natAbs = 0 := Int.ofNat_eq_zero.mp hs.2
    exact Int.natAbs_eq_zero.mp hh

private theorem fraction_sub_zero_equiv (a b : Fraction)
  (h : (Fraction.add a {-b.num, b.den, b.den_pos}).num = 0) :
  Fraction.equiv a b := by
  unfold Fraction.equiv Fraction.add at *
  dsimp at *
  rw [Int.neg_mul] at h
  omega

theorem distance_zero_iff_stateEquiv (a b : Point × Point) :
  Fraction.equiv (distance a b) (Fraction.ofInt 0) ↔ stateEquiv a b := by
  constructor
  · intro hz
    have hs := add_zero_split (pointNorm (stateSub a b).1)
      (pointNorm (stateSub a b).2)
      (pointNorm_nonnegative _) (pointNorm_nonnegative _) hz
    have hp := pointNorm_zero_coords (stateSub a b).1 hs.1
    have hv := pointNorm_zero_coords (stateSub a b).2 hs.2
    exact {(fraction_sub_zero_equiv a.1.1 b.1.1 hp.1,
      fraction_sub_zero_equiv a.1.2 b.1.2 hp.2),
      (fraction_sub_zero_equiv a.2.1 b.2.1 hv.1,
      fraction_sub_zero_equiv a.2.2 b.2.2 hv.2)}
  · intro hab
    have hs : stateEquiv (stateSub a b) (stateSub b b) :=
      stateSub_congr hab
    ({Fraction.equiv_refl _, Fraction.equiv_refl _},
    {Fraction.equiv_refl _, Fraction.equiv_refl _})

```



```

    exact Fraction.equiv_trans (stateNorm_equiv hs)
      (stateSub_self_norm_zero b)

def constantName (s : Point × Point) : EndpointCauchyName where
  approx := fun _ => s
  cauchy := by
    intro eps heps
    exact {0, fun _ _ _ => distance_self_lt s eps heps}

def embed (s : Point × Point) : Value := realize (constantName s)

theorem nameEquiv_of_levelwise_stateEquiv (a b : EndpointCauchyName)
  (h : ∀ n, stateEquiv (a.approx n) (b.approx n)) : NameEquiv a b := by
  intro eps heps
  refine {0, ?_}
  intro n _
  have hr : stateEquiv (b.approx n) (b.approx n) :=
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
     (Fraction.equiv_refl _, Fraction.equiv_refl _))
  have he : Fraction.equiv (distance (a.approx n) (b.approx n))
    (distance (b.approx n) (b.approx n)) :=
    stateNorm_equiv (stateSub_congr (h n) hr)
  exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv he)
    (distance_self_lt _ eps heps)

theorem constantName_equiv_iff (a b : Point × Point) :
  NameEquiv (constantName a) (constantName b) ↔ stateEquiv a b := by
  constructor
  · intro hab
    let d := distance a b
    have hnon := stateNorm_nonnegative (stateSub a b)
    have hnon' : 0 ≤ d.num := hnon
    by_cases hz : d.num = 0
    · have he : Fraction.equiv d (Fraction.ofInt 0) := by
        unfold Fraction.equiv Fraction.ofInt
        simp [hz]
        exact (distance_zero_iff_stateEquiv a b).mp he
    · have hd : 0 < d.num := by omega
        obtain (N, hN) := hab d.half hd
        have hbad : Fraction.lt d d.half := hN N (Nat.le_refl N)
        have hhalf := Fraction.half_lt d hd
        have hloop := Fraction.magnitudes.lt_of_lt_le hbad
          (Fraction.magnitudes.lt_implies_le hhalf)
        exact False.elim ((Fraction.magnitudes.lt_irrefl d) hloop)
    · intro hab eps heps
      have hz := (distance_zero_iff_stateEquiv a b).mpr hab
      have hnum := equiv_zero_num _ hz
      refine {0, ?_}
      intro n _
      change Fraction.lt (distance a b) eps
      unfold Fraction.lt
      simp only [hnum, Int.zero_mul, Int.mul_one]
      exact Int.mul_pos heps (distance a b).den_pos

theorem embed_eq_iff_stateEquiv (a b : Point × Point) :
  embed a = embed b ↔ stateEquiv a b := by
  constructor
  · intro h
    exact (constantName_equiv_iff a b).mp (Quotient.exact h)
  · intro h
    exact Quotient.sound ((constantName_equiv_iff a b).mpr h)

theorem add_lt_add_left {a b : Fraction}
  (hab : Fraction.lt a b) (c : Fraction) :
  Fraction.lt (Fraction.add c a) (Fraction.add c b) :=
  Fraction.add_lt_add_left hab c

private theorem add_lt_add_right {a b : Fraction}
  (hab : Fraction.lt a b) (c : Fraction) :
  Fraction.lt (Fraction.add a c) (Fraction.add b c) :=

```

```

Fraction.add_lt_add_right hab c

private theorem lt_equiv_left {a b c : Fraction}
  (hab : Fraction.equiv a b) (hbc : Fraction.lt b c) : Fraction.lt a c :=
  Fraction.magnitudes.lt_of_le_lt
    ((Fraction.equiv_iff_mutual_le _ _).mp hab).1 hbc

theorem lt_equiv_right {a b c : Fraction}
  (hab : Fraction.lt a b) (hbc : Fraction.equiv b c) : Fraction.lt a c :=
  Fraction.magnitudes.lt_of_lt_le hab
    ((Fraction.equiv_iff_mutual_le _ _).mp hbc).1

private theorem three_quarters_lt (R eps : Fraction) (heps : 0 < eps.num) :
  Fraction.lt
    (Fraction.add (Fraction.add
      (Fraction.add R eps.half.half) eps.half.half) eps.half.half)
    (Fraction.add R eps) := by
  let q := eps.half.half
  have hq : Fraction.lt q eps.half := Fraction.half_lt eps.half heps
  have h1 := add_lt_add_left hq eps.half
  have h2 : Fraction.lt (Fraction.add q eps.half) eps :=
    lt_equiv_right
      (lt_equiv_left (Fraction.add_comm q eps.half) h1)
      (half_add_equiv eps)
  have h3 := add_lt_add_left h2 R
  have hp : Fraction.equiv (Fraction.add q q) eps.half :=
    half_add_equiv eps.half
  have h1 := Fraction.add_equiv (Fraction.equiv_refl q) hp
  have ho := Fraction.add_equiv (Fraction.equiv_refl R) h1
  have he := Fraction.equiv_trans
    (Fraction.add_assoc (Fraction.add R q) q q)
    (Fraction.equiv_trans (Fraction.add_assoc R q (Fraction.add q q)) ho)
  exact lt_equiv_left he h3

def NameBound (a b : EndpointCauchyName) (R : Fraction) : Prop :=
  ∀ eps : Fraction, 0 < eps.num →
  ∃ N : Nat, ∀ n : Nat, N ≤ n →
    Fraction.lt (distance (a.approx n) (b.approx n))
      (Fraction.add R eps)

theorem nameBound_symm {a b : EndpointCauchyName} {R : Fraction}
  (h : NameBound a b R) : NameBound b a R := by
  intro eps heps
  obtain (N, hN) := h eps heps
  refine (N, ?_)
  intro n hn
  exact lt_equiv_left
    (stateSub_norm_symm (b.approx n) (a.approx n)) (hN n hn)

private theorem distance_three (a a' b b' : Point × Point) :
  Fraction.le (distance a' b')
    (Fraction.add (distance a' a)
      (Fraction.add (distance a b) (distance b b'))) := by
  have h1 := stateSub_triangle a' a b'
  have h2 := stateSub_triangle a b b'
  exact Fraction.magnitudes.le_trans h1
    (Fraction.add_le_add_left h2 (distance a' a))

private theorem three_quarters_reordered (R eps : Fraction)
  (heps : 0 < eps.num) :
  Fraction.lt
    (Fraction.add eps.half.half
      (Fraction.add (Fraction.add R eps.half.half) eps.half.half))
    (Fraction.add R eps) := by
  let q := eps.half.half
  have he : Fraction.equiv
    (Fraction.add q (Fraction.add (Fraction.add R q) q))
    (Fraction.add (Fraction.add (Fraction.add R q) q) q) := by
    simp only [Fraction.equiv, Fraction.add]
    simp only [Int.add_mul, Int.mul_add]

```

```

    ac_nf
    exact lt_equiv_left he (three_quarters_lt R eps heps)

private theorem nameBound_transport {a a' b b' : EndpointCauchyName} {R : Fraction}
  (ha : NameEquiv a a') (hb : NameEquiv b b') :
  NameBound a b R → NameBound a' b' R := by
  intro h eps heps
  let q := eps.half.half
  obtain ⟨N1, h1⟩ := nameEquiv_symm ha q heps
  obtain ⟨N2, h2⟩ := h q heps
  obtain ⟨N3, h3⟩ := hb q heps
  refine ⟨N1 + N2 + N3, ?_⟩
  intro n hn
  have hn1 : N1 ≤ n := by omega
  have hn2 : N2 ≤ n := by omega
  have hn3 : N3 ≤ n := by omega
  have htri := distance_three (a.approx n) (a'.approx n)
    (b.approx n) (b'.approx n)
  have hsum := Fraction.add_le_add
    (Fraction.magnitudes.lt_implies_le (h1 n hn1))
    (Fraction.add_le_add
      (Fraction.magnitudes.lt_implies_le (h2 n hn2))
      (Fraction.magnitudes.lt_implies_le (h3 n hn3)))
  have hbound := Fraction.magnitudes.le_trans htri hsum
  exact Fraction.magnitudes.lt_of_le_lt hbound
    (three_quarters_reordered R eps heps)

theorem nameBound_congr {a a' b b' : EndpointCauchyName} {R : Fraction}
  (ha : NameEquiv a a') (hb : NameEquiv b b') :
  NameBound a b R ↔ NameBound a' b' R :=
  (nameBound_transport ha hb,
   nameBound_transport (nameEquiv_symm ha) (nameEquiv_symm hb))

/-- A closed rational radius bound, defined first on names and lifted only
after representative invariance has been proved. -/
def Within (x y : Value) (R : Fraction) : Prop :=
  Quotient.liftOn2 x y (fun a b => NameBound a b R)
  (fun _ _ _ ha hb => propext (nameBound_congr ha hb))

theorem within_realize (a b : EndpointCauchyName) (R : Fraction) :
  Within (realize a) (realize b) R ↔ NameBound a b R := Iff.rfl

theorem within_symm (x y : Value) (R : Fraction)
  (h : Within x y R) : Within y x R := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b => exact nameBound_symm h

private theorem two_quarters_lt (R S eps : Fraction)
  (heps : 0 < eps.num) :
  Fraction.lt
    (Fraction.add (Fraction.add R eps.half.half)
     (Fraction.add S eps.half.half))
    (Fraction.add (Fraction.add R S) eps) := by
  let q := eps.half.half
  have he1 : Fraction.equiv
    (Fraction.add (Fraction.add R q) (Fraction.add S q))
    (Fraction.add (Fraction.add R S) (Fraction.add q q)) := by
  simp only [Fraction.equiv, Fraction.add]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf
  have he2 := Fraction.add_equiv (Fraction.equiv_refl (Fraction.add R S))
    (half_add_equiv eps.half)
  have hlt := add_lt_add_left (Fraction.half_lt eps heps)
    (Fraction.add R S)
  exact lt_equiv_left (Fraction.equiv_trans he1 he2) hlt

theorem nameBound_triangle {a b c : EndpointCauchyName} {R S : Fraction}
  (hab : NameBound a b R) (hbc : NameBound b c S) :

```

```

    NameBound a c (Fraction.add R S) := by
    intro eps heps
    let q := eps.half.half
    obtain ⟨N1, h1⟩ := hab q heps
    obtain ⟨N2, h2⟩ := hbc q heps
    refine ⟨N1 + N2, ?_⟩
    intro n hn
    have hn1 : N1 ≤ n := by omega
    have hn2 : N2 ≤ n := by omega
    have htri := stateSub_triangle (a.approx n) (b.approx n) (c.approx n)
    have hsum := Fraction.add_le_add
      (Fraction.magnitudes.lt_implies_le (h1 n hn1))
      (Fraction.magnitudes.lt_implies_le (h2 n hn2))
    exact Fraction.magnitudes.lt_of_le_lt
      (Fraction.magnitudes.le_trans htri hsum)
      (two_quarters_lt R S eps heps)

theorem within_triangle (x y z : Value) (R S : Fraction)
  (hxy : Within x y R) (hyz : Within y z S) :
  Within x z (Fraction.add R S) := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b =>
      induction z using Quotient.inductionOn with
      | _ c => exact nameBound_triangle hxy hyz

private theorem lt_self_add_positive (R eps : Fraction)
  (heps : 0 < eps.num) : Fraction.lt R (Fraction.add R eps) := by
  have h := add_lt_add_left (zero_lt_positive eps heps) R
  have he : Fraction.equiv (Fraction.add R (Fraction.ofInt 0)) R := by
    simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
    simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
      Int.mul_one, Int.one_mul]
  exact lt_equiv_left (Fraction.equiv_symm he) h

theorem nameBound_of_eventual_le (a b : EndpointCauchyName) (R : Fraction)
  (N : Nat) (h : ∀ n : Nat, N ≤ n →
    Fraction.le (distance (a.approx n) (b.approx n)) R) :
  NameBound a b R := by
  intro eps heps
  refine ⟨N, ?_⟩
  intro n hn
  exact Fraction.magnitudes.lt_of_le_lt (h n hn)
    (lt_self_add_positive R eps heps)

theorem constant_approximants_converge (a : EndpointCauchyName)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m : Nat, N ≤ m →
    Within (embed (a.approx m)) (realize a) eps := by
  obtain ⟨N, hN⟩ := a.cauchy eps.half heps
  refine ⟨N, ?_⟩
  intro m hm
  intro delta hdelta
  refine ⟨N, ?_⟩
  intro n hn
  have hsmall : Fraction.lt eps.half (Fraction.add eps delta) :=
    Fraction.magnitudes.lt_of_lt_le (Fraction.half_lt eps heps)
      (Fraction.magnitudes.lt_implies_le
        (lt_self_add_positive eps delta hdelta))
  exact Fraction.magnitudes.lt_of_lt_le (hN m n hm hn)
    (Fraction.magnitudes.lt_implies_le hsmall)

theorem nameBound_zero_iff (a b : EndpointCauchyName) :
  NameBound a b (Fraction.ofInt 0) ↔ NameEquiv a b := by
  constructor
  · intro h eps heps
    obtain ⟨N, hN⟩ := h eps heps
    refine ⟨N, ?_⟩
    intro n hn

```

```

    have he : Fraction.equiv (Fraction.add (Fraction.ofInt 0) eps) eps := by
      simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
      simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
        Int.mul_one, Int.one_mul, Int.zero_add]
    exact lt_equiv_right (hN n hn) he
  · intro h eps heps
    obtain (N, hN) := h eps heps
    refine (N, ?_)
    intro n hn
    have he : Fraction.equiv eps (Fraction.add (Fraction.ofInt 0) eps) := by
      simp only [Fraction.equiv, Fraction.add, Fraction.ofInt]
      simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
        Int.mul_one, Int.one_mul, Int.zero_add]
    exact lt_equiv_right (hN n hn) he

theorem within_zero_iff (x y : Value) :
  Within x y (Fraction.ofInt 0) ↔ x = y := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b =>
      exact Iff.trans (nameBound_zero_iff a b)
      (fun h => Quotient.sound h, fun h => Quotient.exact h)

theorem le_add_cancel_left (z a b : Fraction)
  (h : Fraction.le (Fraction.add z a) (Fraction.add z b)) :
  Fraction.le a b := Fraction.le_add_cancel_left z a b h

end NewtonLimitDynamics.Polygon.CauchyValues

```

BarrowLib/Polygon/GeometricTail.lean, 7 theorems, 136 lines

```

import BarrowLib.Polygon.StateDistance
import BarrowLib.Polygon.DyadicArithmetic

/-! Rational geometric-tail estimates for an arbitrary sequence of states.
The coefficient and adjacent-error premise are supplied separately. -/

namespace NewtonLimitDynamics.Polygon.GeometricTail

open NewtonLimitDynamics
open TimeSubdivision
open FiniteEstimates
open HarmonicDyadic

def tailCap (A : Fraction) (j : Nat) : Fraction :=
  (A.num, A.den * (2 : Int) ^ j,
    Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

def doubleTail (A : Fraction) (j : Nat) : Fraction :=
  (2 * A.num, A.den * (2 : Int) ^ j,
    Int.mul_pos A.den_pos (Int.pow_pos (by decide)))

theorem tail_halving (A : Fraction) (j : Nat) :
  Fraction.equiv
    (Fraction.add (tailCap A (j + 1)) (tailCap A (j + 1)))
    (tailCap A j) := by
  simp only [tailCap, Fraction.equiv, Fraction.add, Int.pow_succ]
  simp only [show (2 : Int) = 1 + 1 by rfl, Int.add_mul, Int.mul_add]
  ac_nf

theorem tail_double (A : Fraction) (j : Nat) :
  Fraction.equiv (Fraction.add (tailCap A j) (tailCap A j))
    (doubleTail A j) := by
  simp only [tailCap, doubleTail, Fraction.equiv, Fraction.add]
  simp only [show (2 : Int) = 1 + 1 by rfl, Int.add_mul, Int.mul_add]
  ac_nf

theorem tail_add (A B : Fraction) (n : Nat) :
  Fraction.equiv (Fraction.add (tailCap A n) (tailCap B n))
    (tailCap (Fraction.add A B) n) := by

```

```

simp only [tailCap, Fraction.equiv, Fraction.add, Int.add_mul, Int.mul_add]
ac_nf

theorem finite_gap (x : Nat → Point × Point) (A : Fraction)
  (hA : 0 ≤ A.num)
  (hadj : ∀ j, Fraction.le (stateDistance (x (j + 1)) (x j))
    (tailCap A (j + 1))) :
  (k j : Nat) → Fraction.le (stateDistance (x (j + k)) (x j))
    (tailCap A j)
| 0, j => by
  have hz : Fraction.le (Fraction.ofInt 0) (tailCap A j) := by
    simp only [Fraction.le, Fraction.ofInt, tailCap]
    simp only [Int.zero_mul, Int.mul_one]
    exact hA
  simp only [Nat.add_zero] using
    Fraction.le_equiv_left (stateDistance_self_zero (x j)) hz
| k + 1, j => by
  have htri := stateDistance_triangle
    (x (j + (k + 1))) (x (j + 1)) (x j)
  have hk : Fraction.le
    (stateDistance (x (j + (k + 1))) (x (j + 1)))
    (tailCap A (j + 1)) := by
    simp only [Nat.add_succ, Nat.succ_add, Nat.add_assoc] using
      finite_gap x A hA hadj k (j + 1)
  have hsum := Fraction.add_le_add hk (hadj j)
  exact Fraction.le_equiv_right (Fraction.magnitudes.le_trans htri hsum)
    (tail_halving A j)

theorem two_sided (x : Nat → Point × Point) (A : Fraction)
  (hA : 0 ≤ A.num)
  (hadj : ∀ j, Fraction.le (stateDistance (x (j + 1)) (x j))
    (tailCap A (j + 1)))
  (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le (stateDistance (x m) (x n)) (doubleTail A N) := by
  have hm' : N + (m - N) = m := by omega
  have hn' : N + (n - N) = n := by omega
  have hfirst := finite_gap x A hA hadj (m - N) N
  have hsecond := finite_gap x A hA hadj (n - N) N
  rw [hm'] at hfirst
  rw [hn'] at hsecond
  have hsecond' := Fraction.le_equiv_left (stateDistance_symm (x N) (x n)) hsecond
  have htri := stateDistance_triangle (x m) (x N) (x n)
  exact Fraction.le_equiv_right
    (Fraction.magnitudes.le_trans htri (Fraction.add_le_add hfirst hsecond'))
    (tail_double A N)

def modulus (A eps : Fraction) : Nat := (2 * A.num * eps.den).toNat

theorem doubleTail_lt_tolerance (A eps : Fraction)
  (hA : 0 ≤ A.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail A (modulus A eps)) eps := by
  let N := modulus A eps
  have hL : 0 ≤ 2 * A.num * eps.den :=
    Int.mul_nonneg (Int.mul_nonneg (by decide) hA) (Int.le_of_lt eps.den_pos)
  have hN : (N : Int) = 2 * A.num * eps.den := Int.toNat_of_nonneg hL
  have hpow := two_pow_ge_succ N
  have hp : 0 ≤ (2 : Int) ^ N := Int.le_of_lt (Int.pow_pos (by decide))
  have hfactor : 1 ≤ eps.num * A.den := by
    have hmul := Int.mul_pos heps A.den_pos
    omega
  have hmult := Int.mul_le_mul_of_nonneg_right hfactor hp
  simp only [Int.one_mul] at hmult
  unfold Fraction.lt doubleTail
  dsimp
  change 2 * A.num * eps.den < eps.num * (A.den * (2 : Int) ^ N)
  rw [← Int.mul_assoc]
  omega

end NewtonLimitDynamics.Polygon.GeometricTail

```

```

namespace NewtonLimitDynamics.Polygon.HarmonicTimeRealization
open NewtonLimitDynamics HarmonicDyadic

theorem duration_eventually_small (T eps : Fraction)
  (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j : Nat, N ≤ j →
    Fraction.lt (duration T j) eps := by
  let N := (T.num * eps.den).toNat
  have hL : 0 ≤ T.num * eps.den :=
    Int.mul_nonneg hT (Int.le_of_lt eps.den_pos)
  have hN : (N : Int) = T.num * eps.den := Int.toNat_of_nonneg hL
  refine ⟨N, ?_⟩
  intro j hj
  have hpow := two_pow_ge_succ j
  have hp : 0 ≤ (2 : Int) ^ j := Int.le_of_lt (Int.pow_pos (by decide))
  have hfactor : 1 ≤ eps.num * T.den := by
    have hmul := Int.mul_pos heps T.den_pos
    omega
  have hmult := Int.mul_le_mul_of_nonneg_right hfactor hp
  simp only [Int.one_mul] at hmult
  unfold Fraction.lt duration
  dsimp
  change T.num * eps.den < eps.num * (T.den * (2 : Int) ^ j)
  rw [← Int.mul_assoc]
  omega

end NewtonLimitDynamics.Polygon.HarmonicTimeRealization

```

BarrowLib/Polygon/BinaryTime.lean, 12 theorems, 166 lines

```

import BarrowLib.Polygon.CauchyValues
import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.GeometricTail

/-!
Binary-address time names built from the same dyadic tick counts as the actual
harmonic prefixes. The quotient below is a constructed binary-time domain;
no identification with an external real interval is asserted.
-/

namespace NewtonLimitDynamics.Polygon.BinaryTime

open NewtonLimitDynamics
open TimeSubdivision
open HarmonicStability
open HarmonicDyadic
open HarmonicBinaryPrefix
open HarmonicTimeComparison
open PointBounds
open HarmonicComparison
open HarmonicAccumulation
open CauchyValues

private def zero : Fraction := (0, 1, by decide)

def timeApprox (b : Nat → Bool) (T : Fraction) (j : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (ticks b j : Int)) (duration T j)

def scalarState (t : Fraction) : Point × Point :=
  ((t, zero), (zero, zero))

def timeState (b : Nat → Bool) (T : Fraction) (j : Nat) : Point × Point :=
  scalarState (timeApprox b T j)

theorem scalarState_distance (t u : Fraction) :
  Fraction.equiv (distance (scalarState t) (scalarState u))
    (durationDifference u t).abs := by
  simp only [distance, scalarState, stateNorm, stateSub, pointNorm,
    pointSub, pointAdd, pointNeg, durationDifference, negF,
    Fraction.equiv, Fraction.abs, Fraction.add, Fraction.mul, zero]
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,

```

```

    Int.natAbs_zero, Int.ofNat_zero, Int.neg_zero,
    Int.mul_one, Int.one_mul]

theorem timeState_distance (b c : Nat → Bool) (T : Fraction) (j : Nat) :
  Fraction.equiv (distance (timeState b T j) (timeState c T j))
    (durationDifference (timeApprox c T j) (timeApprox b T j)).abs :=
  scalarState_distance _ _

theorem time_step_difference (b : Nat → Bool) (T : Fraction) (j : Nat) :
  Fraction.equiv
    (durationDifference (timeApprox b T j) (timeApprox b T (j + 1)))
    (Fraction.mul (Fraction.ofInt (bit b j : Int)) (duration T (j + 1))) := by
  simp only [timeApprox, durationDifference, negF, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, duration,
    ticks_next, Int.natCast_add, Int.pow_succ]
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.one_mul, Int.mul_one]
  simp only [show (2 : Int) = 1 + 1 by rfl,
    Int.add_mul, Int.mul_add]
  ac_nf <=> omega

theorem adjacent_time_bound (b : Nat → Bool) (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.le (distance (timeState b T (j + 1)) (timeState b T j))
    (duration T (j + 1)) := by
  let h := duration T (j + 1)
  let k := Fraction.ofInt (bit b j : Int)
  have h₁ := scalarState_distance (timeApprox b T (j + 1))
    (timeApprox b T j)
  have h₂ := Fraction.abs_equiv (time_step_difference b T j)
  have h₃ := Fraction.abs_mul k h
  have hk : 0 ≤ k.num := Int.ofNat_nonneg _
  have hkab := Fraction.abs_of_nonnegative k hk
  have hhab := Fraction.abs_of_nonnegative h hT
  have h₄ := Fraction.mul_equiv hkab hhab
  have he := Fraction.equiv_trans h₁
    (Fraction.equiv_trans h₂ (Fraction.equiv_trans h₃ h₄))
  have hkone : Fraction.le k (Fraction.ofInt 1) := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simp only [Int.mul_one, Int.one_mul]
    exact Int.ofNat_le.mpr (bit_le_one b j)
  have hm := Fraction.mul_le_mul_nonnegative hkone h hT
  have hone : Fraction.equiv (Fraction.mul (Fraction.ofInt 1) h) h := by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
  have hbound := Fraction.le_equiv_right hm hone
  exact Fraction.le_equiv_left he hbound

def doubleTail (T : Fraction) (j : Nat) : Fraction :=
  (2 * T.num, T.den * (2 : Int) ^ j,
  Int.mul_pos T.den_pos (Int.pow_pos (by decide)))

theorem tail_double (T : Fraction) (j : Nat) :
  Fraction.equiv (Fraction.add (duration T j) (duration T j))
    (doubleTail T j) := by
  exact GeometricTail.tail_double T j

theorem finite_gap_time (b : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) :
  (k j : Nat) → Fraction.le
    (distance (timeState b T (j + k)) (timeState b T j))
    (duration T j) := by
  intro k j
  exact GeometricTail.finite_gap (timeState b T) T hT
    (fun i => adjacent_time_bound b T i hT) k j

theorem two_sided_time (b : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (N m n : Nat) (hm : N ≤ m) (hn : N ≤ n) :
  Fraction.le (distance (timeState b T m) (timeState b T n))

```



```

    (doubleTail T N) := by
    exact GeometricTail.two_sided (timeState b T) T hT
    (fun i => adjacent_time_bound b T i hT) N m n hm hn

def modulus (T eps : Fraction) : Nat := (2 * T.num * eps.den).toNat

theorem doubleTail_lt_tolerance (T eps : Fraction)
  (hT : 0 ≤ T.num) (heps : 0 < eps.num) :
  Fraction.lt (doubleTail T (modulus T eps)) eps := by
  exact GeometricTail.doubleTail_lt_tolerance T eps hT heps

theorem time_cauchy (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) :
  ∀ eps : Fraction, 0 < eps.num →
  ∃ N : Nat, ∀ m n : Nat, N ≤ m → N ≤ n →
  Fraction.lt (distance (timeState b T m) (timeState b T n)) eps := by
  intro eps heps
  refine (modulus T eps, ?_)
  intro m n hm hn
  exact Fraction.magnitudes.lt_of_le_lt
    (two_sided_time b T hT _ m n hm hn)
    (doubleTail_lt_tolerance T eps hT heps)

def timeName (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) :
  EndpointCauchyName where
  approx := timeState b T
  cauchy := time_cauchy b T hT

def timeValue (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) : Value :=
  realize (timeName b T hT)

def AddressEquiv (T : Fraction) (hT : 0 ≤ T.num)
  (b c : Nat → Bool) : Prop :=
  NameEquiv (timeName b T hT) (timeName c T hT)

theorem addressEquiv_refl (T : Fraction) (hT : 0 ≤ T.num)
  (b : Nat → Bool) : AddressEquiv T hT b b := nameEquiv_refl _

theorem addressEquiv_symm (T : Fraction) (hT : 0 ≤ T.num)
  {b c : Nat → Bool} (h : AddressEquiv T hT b c) :
  AddressEquiv T hT c b := nameEquiv_symm h

theorem addressEquiv_trans (T : Fraction) (hT : 0 ≤ T.num)
  {a b c : Nat → Bool} (hab : AddressEquiv T hT a b)
  (hbc : AddressEquiv T hT b c) : AddressEquiv T hT a c :=
  nameEquiv_trans hab hbc

def addressSetoid (T : Fraction) (hT : 0 ≤ T.num) : Setoid (Nat → Bool) where
  r := AddressEquiv T hT
  iseqv := (addressEquiv_refl T hT,
    @addressEquiv_symm T hT, @addressEquiv_trans T hT)

def BinaryTime (T : Fraction) (hT : 0 ≤ T.num) :=
  Quotient (addressSetoid T hT)

end NewtonLimitDynamics.Polygon.BinaryTime

```

BarrowLib/Polygon/PositionValues.lean, 22 theorems, 264 lines

```

import BarrowLib.Polygon.CauchyValues

namespace NewtonLimitDynamics.Polygon.PositionValues
open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
open HarmonicAccumulation
open HarmonicDyadic
open CauchyValues

private def zero : Fraction := Fraction.ofInt 0
def zeroPoint : Point := (zero, zero)

```

```

def positionState (s : Point × Point) : Point × Point := (s.1, zeroPoint)
def firstState (s : Point × Point) : Point × Point := ((s.1.1, zero), zeroPoint)
def secondState (s : Point × Point) : Point × Point := ((zero, s.1.2), zeroPoint)

private def xGap (a b : Point × Point) : Fraction :=
  (Fraction.add a.1.1 (-b.1.1.num, b.1.1.den, b.1.1.den_pos)).abs
private def yGap (a b : Point × Point) : Fraction :=
  (Fraction.add a.1.2 (-b.1.2.num, b.1.2.den, b.1.2.den_pos)).abs
private def vxGap (a b : Point × Point) : Fraction :=
  (Fraction.add a.2.1 (-b.2.1.num, b.2.1.den, b.2.1.den_pos)).abs
private def vyGap (a b : Point × Point) : Fraction :=
  (Fraction.add a.2.2 (-b.2.2.num, b.2.2.den, b.2.2.den_pos)).abs

private theorem distance_decompose (a b : Point × Point) :
  distance a b = Fraction.add (Fraction.add (xGap a b) (yGap a b))
    (Fraction.add (vxGap a b) (vyGap a b)) := rfl

private theorem projected_distance (a b : Point × Point) :
  Fraction.equiv (distance (positionState a) (positionState b))
    (Fraction.add (xGap a b) (yGap a b)) := by
  simp only [distance, positionState, stateNorm, stateSub, pointNorm,
    pointSub, pointAdd, pointNeg, zeroPoint, zero,
    Fraction.equiv, Fraction.add, Fraction.abs, Fraction.ofInt,
    xGap, yGap]
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
    Int.natAbs_zero, Int.ofNat_zero, Int.neg_zero,
    Int.mul_one, Int.one_mul]

private theorem first_distance (a b : Point × Point) :
  Fraction.equiv (distance (firstState a) (firstState b)) (xGap a b) := by
  simp only [distance, firstState, stateNorm, stateSub, pointNorm,
    pointSub, pointAdd, pointNeg, zeroPoint, zero,
    Fraction.equiv, Fraction.add, Fraction.abs, Fraction.ofInt, xGap]
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
    Int.natAbs_zero, Int.ofNat_zero, Int.neg_zero,
    Int.mul_one, Int.one_mul]

private theorem second_distance (a b : Point × Point) :
  Fraction.equiv (distance (secondState a) (secondState b)) (yGap a b) := by
  simp only [distance, secondState, stateNorm, stateSub, pointNorm,
    pointSub, pointAdd, pointNeg, zeroPoint, zero,
    Fraction.equiv, Fraction.add, Fraction.abs, Fraction.ofInt, yGap]
  simp only [Int.zero_mul, Int.mul_zero, Int.add_zero,
    Int.natAbs_zero, Int.ofNat_zero, Int.neg_zero,
    Int.mul_one, Int.one_mul]
  ac_nf

theorem position_nonexpansive (a b : Point × Point) :
  Fraction.le (distance (positionState a) (positionState b))
    (distance a b) := by
  apply Fraction.le_equiv_left (projected_distance a b)
  rw [distance_decompose]
  exact Fraction.le_add_nonnegative _ _
    (by
      unfold Fraction.add
      exact Int.add_nonneg
        (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
          (Int.le_of_lt (vyGap a b).den_pos))
        (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
          (Int.le_of_lt (vxGap a b).den_pos)))

theorem first_nonexpansive (a b : Point × Point) :
  Fraction.le (distance (firstState a) (firstState b))
    (distance a b) := by
  apply Fraction.le_equiv_left (first_distance a b)
  have h₁ := Fraction.le_add_nonnegative (xGap a b) (yGap a b)
    (Fraction.abs_num_nonnegative _)
  have h₂ := Fraction.le_add_nonnegative (Fraction.add (xGap a b) (yGap a b))
    (Fraction.add (vxGap a b) (vyGap a b)) (by

```

```

    unfold Fraction.add
    exact Int.add_nonneg
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
        (Int.le_of_lt (vyGap a b).den_pos))
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
        (Int.le_of_lt (vxGap a b).den_pos)))
  rw [distance_decompose]
  exact Fraction.magnitudes.le_trans h1 h2

theorem second_nonexpansive (a b : Point × Point) :
  Fraction.le (distance (secondState a) (secondState b))
    (distance a b) := by
  apply Fraction.le_equiv_left (second_distance a b)
  have h1 := Fraction.le_add_nonnegative (yGap a b) (xGap a b)
    (Fraction.abs_num_nonnegative _)
  have hcomm := Fraction.add_comm (yGap a b) (xGap a b)
  have h2 := Fraction.le_add_nonnegative (Fraction.add (xGap a b) (yGap a b))
    (Fraction.add (vxGap a b) (vyGap a b)) (by
    unfold Fraction.add
    exact Int.add_nonneg
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
        (Int.le_of_lt (vyGap a b).den_pos))
      (Int.mul_nonneg (Fraction.abs_num_nonnegative _)
        (Int.le_of_lt (vxGap a b).den_pos)))
  rw [distance_decompose]
  exact Fraction.magnitudes.le_trans
    (Fraction.le_equiv_right h1 hcomm) h2

def mapName (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b))
  (a : EndpointCauchyName) : EndpointCauchyName where
  approx := fun n => f (a.approx n)
  cauchy := by
    intro eps heps
    obtain ⟨N, hN⟩ := a.cauchy eps heps
    refine ⟨N, ?_⟩
    intro m n hm hn
    exact Fraction.magnitudes.lt_of_le_lt
      (hLip (a.approx m) (a.approx n)) (hN m n hm hn)

theorem mapName_equiv (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b))
  {a b : EndpointCauchyName} (hab : NameEquiv a b) :
  NameEquiv (mapName f hLip a) (mapName f hLip b) := by
  intro eps heps
  obtain ⟨N, hN⟩ := hab eps heps
  refine ⟨N, ?_⟩
  intro n hn
  exact Fraction.magnitudes.lt_of_le_lt
    (hLip (a.approx n) (b.approx n)) (hN n hn)

def mapValue (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b)) :
  Value → Value :=
  Quotient.lift (fun a => realize (mapName f hLip a))
    (fun _ _ h => Quotient.sound (mapName_equiv f hLip h))

theorem mapValue_embed (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b))
  (s : Point × Point) :
  mapValue f hLip (embed s) = embed (f s) := rfl

theorem mapName_bound (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b))
  (a b : EndpointCauchyName) (R : Fraction) (h : NameBound a b R) :
  NameBound (mapName f hLip a) (mapName f hLip b) R := by
  intro eps heps
  obtain ⟨N, hN⟩ := h eps heps
  refine ⟨N, ?_⟩
  intro n hn

```

```

exact Fraction.magnitudes.lt_of_le_lt
  (hLip (a.approx n) (b.approx n)) (hN n hn)

theorem mapValue_within (f : Point × Point → Point × Point)
  (hLip : ∀ a b, Fraction.le (distance (f a) (f b)) (distance a b))
  (x y : Value) (R : Fraction) (h : Within x y R) :
  Within (mapValue f hLip x) (mapValue f hLip y) R := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b => exact mapName_bound f hLip a b R h

def positionValue : Value → Value := mapValue positionState position_nonexpansive
def firstValue : Value → Value := mapValue firstState first_nonexpansive
def secondValue : Value → Value := mapValue secondState second_nonexpansive

theorem positionValue_embed (s : Point × Point) :
  positionValue (embed s) = embed (positionState s) :=
  mapValue_embed positionState position_nonexpansive s

theorem firstValue_embed (s : Point × Point) :
  firstValue (embed s) = embed (firstState s) :=
  mapValue_embed firstState first_nonexpansive s

theorem secondValue_embed (s : Point × Point) :
  secondValue (embed s) = embed (secondState s) :=
  mapValue_embed secondState second_nonexpansive s

theorem positionValue_idempotent (v : Value) :
  positionValue (positionValue v) = positionValue v := by
  induction v using Quotient.inductionOn with
  | _ a => rfl

theorem positionValue_within (x y : Value) (R : Fraction)
  (h : Within x y R) :
  Within (positionValue x) (positionValue y) R :=
  mapValue_within positionState position_nonexpansive x y R h

theorem firstValue_within (x y : Value) (R : Fraction)
  (h : Within x y R) :
  Within (firstValue x) (firstValue y) R :=
  mapValue_within firstState first_nonexpansive x y R h

theorem secondValue_within (x y : Value) (R : Fraction)
  (h : Within x y R) :
  Within (secondValue x) (secondValue y) R :=
  mapValue_within secondState second_nonexpansive x y R h

def PositionValue := {v : Value // positionValue v = v}

def asPosition (v : Value) : PositionValue :=
  (positionValue v, positionValue_idempotent v)

def embedPosition (p : Point) : PositionValue :=
  asPosition (embed (p, zeroPoint))

theorem firstValue_positionValue (v : Value) :
  firstValue (positionValue v) = firstValue v := by
  induction v using Quotient.inductionOn with
  | _ a => rfl

theorem secondValue_positionValue (v : Value) :
  secondValue (positionValue v) = secondValue v := by
  induction v using Quotient.inductionOn with
  | _ a => rfl

def NonnegativeRadius := {R : Fraction // 0 ≤ R.num}

/-- A completed coordinate square: each planar coordinate separately has a
closed rational bound. Its corners can have L1 distance twice the radius. -/

```

```

def CoordinateSquare (centre : Point) (radius : NonnegativeRadius)
  (x : PositionValue) : Prop :=
  Within (firstValue x.val)
    (embed (firstState (centre, zeroPoint))) radius.val ∧
  Within (secondValue x.val)
    (embed (secondState (centre, zeroPoint))) radius.val

theorem square_of_eventual_coordinate_bounds
  (a : EndpointCauchyName) (centre : Point)
  (radius : NonnegativeRadius) (N : Nat)
  (hx : ∀ n : Nat, N ≤ n →
    Fraction.le
      (distance (firstState (a.approx n))
        (firstState (centre, zeroPoint))) radius.val)
  (hy : ∀ n : Nat, N ≤ n →
    Fraction.le
      (distance (secondState (a.approx n))
        (secondState (centre, zeroPoint))) radius.val) :
  CoordinateSquare centre radius (asPosition (realize a)) := by
  constructor
  · change Within (firstValue (positionValue (realize a)))
    (embed (firstState (centre, zeroPoint))) radius.val
  rw [firstValue_positionValue]
  exact nameBound_of_eventual_le _ _ _ N hx
  · change Within (secondValue (positionValue (realize a)))
    (embed (secondState (centre, zeroPoint))) radius.val
  rw [secondValue_positionValue]
  exact nameBound_of_eventual_le _ _ _ N hy

theorem square_of_rational_coordinate_bounds (p centre : Point)
  (radius : NonnegativeRadius)
  (hx : Fraction.le (distance (firstState (p, zeroPoint))
    (firstState (centre, zeroPoint))) radius.val)
  (hy : Fraction.le (distance (secondState (p, zeroPoint))
    (secondState (centre, zeroPoint))) radius.val) :
  CoordinateSquare centre radius (embedPosition p) := by
  exact square_of_eventual_coordinate_bounds
    (constantName (p, zeroPoint)) centre radius 0
    (fun _ => hx) (fun _ => hy)

end NewtonLimitDynamics.Polygon.PositionValues

```

BarrowLib/Polygon/ScaledTolerance.lean, 7 theorems, 140 lines

```

import BarrowLib.Polygon.CauchyValues

/-! Elementary rational tolerances for scaling and monotonicity of generic
closed quotient bounds. The retained namespace is an import-compatible name;
no harmonic parameter or mechanical result is a premise. -/

namespace NewtonLimitDynamics.Polygon.HarmonicTimeRealization
open NewtonLimitDynamics
open HarmonicDyadic CauchyValues

def factorDenominator (C : Fraction) : Fraction :=
  Fraction.add C (Fraction.ofInt 1)

theorem factorDenominator_positive (C : Fraction) (hC : 0 ≤ C.num) :
  0 < (factorDenominator C).num := by
  unfold factorDenominator Fraction.add Fraction.ofInt
  dsimp
  have hd := C.den_pos
  omega

def factorDelta (C eps : Fraction) (hC : 0 ≤ C.num) : Fraction :=
  (eps.num * (factorDenominator C).den,
  eps.den * (factorDenominator C).num,
  Int.mul_pos eps.den_pos
    (factorDenominator_positive C hC))

theorem factorDelta_positive (C eps : Fraction) (hC : 0 ≤ C.num)

```

```

    (heps : 0 < eps.num) : 0 < (factorDelta C eps hC).num :=
Int.mul_pos heps (factorDenominator C).den_pos

theorem factor_control (C eps d : Fraction) (hC : 0 ≤ C.num)
  (hd : 0 ≤ d.num)
  (hdelta : Fraction.lt d (factorDelta C eps hC)) :
  Fraction.lt (Fraction.mul d C) eps := by
have hden : Fraction.le C (factorDenominator C) := by
  unfold factorDenominator Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hp := C.den_pos
  have hsq : 0 ≤ C.den * C.den :=
    Int.mul_nonneg (Int.le_of_lt hp) (Int.le_of_lt hp)
  simp only [Int.one_mul, Int.mul_one, Int.add_mul]
  omega
have hwake := Fraction.mul_le_mul_nonnegative_left hden d hd
have hstrict : Fraction.lt
  (Fraction.mul d (factorDenominator C))
  (Fraction.mul (factorDelta C eps hC) (factorDenominator C)) := by
have hm := Int.mul_lt_mul_of_pos_right hdelta
  (Int.mul_pos (factorDenominator_positive C hC)
    (factorDenominator C).den_pos)
  unfold Fraction.lt Fraction.mul at *
  dsimp at *
  have h1 : d.num * (factorDenominator C).num *
    ((factorDelta C eps hC).den * (factorDenominator C).den) =
    (d.num * (factorDelta C eps hC).den) *
    ((factorDenominator C).num * (factorDenominator C).den) := by ac_rfl
  have h2 : (factorDelta C eps hC).num *
    (factorDenominator C).num *
    (d.den * (factorDenominator C).den) =
    ((factorDelta C eps hC).num * d.den) *
    ((factorDenominator C).num * (factorDenominator C).den) := by ac_rfl
  rw [h1, h2]
  exact hm
have hproduct : Fraction.equiv
  (Fraction.mul (factorDelta C eps hC) (factorDenominator C)) eps := by
  unfold Fraction.equiv Fraction.mul factorDelta
  dsimp
  ac_nf
exact Fraction.magnitudes.lt_of_lt_le
  (Fraction.magnitudes.lt_of_le_lt hwake hstrict)
  ((Fraction.equiv_iff_mutual_le _ _).mp hproduct).1

theorem factor_delta_weak (C eps : Fraction) (hC : 0 ≤ C.num)
  (heps : 0 ≤ eps.num) :
  Fraction.le (Fraction.mul (factorDelta C eps hC) C) eps := by
have hden : Fraction.le C (factorDenominator C) := by
  unfold factorDenominator Fraction.le Fraction.add Fraction.ofInt
  dsimp
  have hp := C.den_pos
  have hsq : 0 ≤ C.den * C.den :=
    Int.mul_nonneg (Int.le_of_lt hp) (Int.le_of_lt hp)
  simp only [Int.one_mul, Int.mul_one, Int.add_mul]
  omega
have hm := Fraction.mul_le_mul_nonnegative_left hden
  (factorDelta C eps hC)
  (by
    unfold factorDelta
    dsimp
    exact Int.mul_nonneg heps
    (Int.le_of_lt (factorDenominator C).den_pos))
  apply Fraction.le_equiv_right hm
  unfold Fraction.equiv Fraction.mul factorDelta
  dsimp
  ac_nf

theorem nameBound_mono (a b : EndpointCauchyName) (R S : Fraction)
  (hRS : Fraction.le R S) (h : NameBound a b R) :
  NameBound a b S := by

```

```

intro eps heps
obtain (N, hN) := h eps heps
refine (N, ?_)
intro n hn
exact Fraction.magnitudes.lt_of_lt_le (hN n hn)
(Fraction.add_le_add_right hRS eps)

theorem within_mono (x y : Value) (R S : Fraction)
(hRS : Fraction.le R S) (h : Within x y R) : Within x y S := by
induction x using Quotient.inductionOn with
| _ a =>
induction y using Quotient.inductionOn with
| _ b => exact nameBound_mono a b R S hRS h

theorem nameBound_scale (a b ta tb : EndpointCauchyName)
(C R : Fraction) (hC : 0 ≤ C.num)
(hlevel : ∀ n : Nat,
Fraction.le (distance (a.approx n) (b.approx n))
(Fraction.mul (distance (ta.approx n) (tb.approx n)) C))
(hnear : NameBound ta tb R) :
NameBound a b (Fraction.mul R C) := by
intro eps heps
let q := eps.half
let delta := factorDelta C q hC
obtain (N, hN) := hnear delta
(factorDelta_positive C q hC heps)
refine (N, ?_)
intro n hn
have hmul := Fraction.mul_le_mul_nonnegative
(Fraction.magnitudes.lt_implies_le (hN n hn)) C hC
have hsum := Fraction.le_equiv_right hmul (Fraction.add_mul R delta C)
have hdelta := factor_delta_weak C q hC
(by simp only [q, Fraction.half] using Int.le_of_lt heps)
have httotal := Fraction.add_le_add_left hdelta (Fraction.mul R C)
have hsmall : Fraction.lt
(Fraction.add (Fraction.mul R C) q)
(Fraction.add (Fraction.mul R C) eps) :=
CauchyValues.add_lt_add_left (Fraction.half_lt eps heps) _
exact Fraction.magnitudes.lt_of_le_lt
(Fraction.magnitudes.le_trans (hlevel n)
(Fraction.magnitudes.le_trans hsum httotal)) hsmall

end NewtonLimitDynamics.Polygon.HarmonicTimeRealization

```

BarrowLib/Polygon/AffineValues.lean, 10 theorems, 168 lines

```

import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.BinaryTime
import BarrowLib.Polygon.ScaledTolerance
import BarrowLib.Polygon.ConvexCover

namespace NewtonLimitDynamics.Polygon.AffineValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds
open HarmonicComparison HarmonicDyadic HarmonicTimeComparison HarmonicTimeRealization
open BinaryTime CauchyValues PositionValues

def zeroPoint : Point := (Fraction.ofInt 0, Fraction.ofInt 0)
def affineState (x v : Point) (t : Fraction) : Point × Point :=
(pointAdd x (pointScale t v), zeroPoint)

theorem affine_zero_phase (x v : Point) (t : Fraction) (ht : t.num = 0) :
stateEquiv (affineState x v t) (x, zeroPoint) := by
constructor
· constructor <=> simp only [affineState, pointAdd, pointScale,
Fraction.equiv, Fraction.add, Fraction.mul, ht, Int.zero_mul,
Int.zero_add] <=> ac_nf
· exact {Fraction.equiv_refl _, Fraction.equiv_refl _}

theorem affine_difference (x v : Point) (t u : Fraction) :
stateEquiv (stateSub (affineState x v t) (affineState x v u))

```

```

    (pointScale (durationDifference u t) v, zeroPoint) := by
  constructor <=> constructor <=>
  simp only [affineState, stateSub, zeroPoint, pointEquiv, pointSub, pointNeg,
    pointAdd, pointScale, durationDifference, HarmonicStability.negF,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem affine_distance (x v : Point) (t u : Fraction) :
  Fraction.equiv (distance (affineState x v t) (affineState x v u))
    (Fraction.mul (durationDifference u t).abs (pointNorm v)) := by
  have he := stateNorm_equiv (affine_difference x v t u)
  have hz : pointNorm zeroPoint = Fraction.ofInt 0 := rfl
  apply Fraction.equiv_trans he
  change Fraction.equiv (Fraction.add (pointNorm (pointScale (durationDifference u t) v))
    (pointNorm zeroPoint)) _
  rw [hz]
  exact Fraction.equiv_trans (Fraction.add_zero _) (pointNorm_scale _ _)

theorem shift_difference (c t u : Fraction) :
  Fraction.equiv
    (durationDifference (durationDifference c u) (durationDifference c t))
    (durationDifference u t) := by
  simp only [durationDifference, HarmonicStability.negF, Fraction.equiv,
    Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf
  omega

def edgeName (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (x v : Point) (m : Nat) : EndpointCauchyName where
  approx := fun j => affineState x v
    (durationDifference (timeApprox b T m) (timeApprox b T (m+j)))
  cauchy := by
    intro eps heps
    let V := pointNorm v
    let delta := factorDelta V eps (pointNorm_nonnegative v)
    have hdelta := factorDelta_positive V eps (pointNorm_nonnegative v) heps
    obtain (N,hN) := (BinaryTime.timeName b T hT).cauchy delta hdelta
    refine (N, ?_)
    intro i j hi hj
    have ht := hN (m+i) (m+j) (by omega) (by omega)
    have hd := scalarState_distance (timeApprox b T (m+i)) (timeApprox b T (m+j))
    have hg := Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv (Fraction.equiv_symm hd)) ht
    have hc := factor_control V eps
      (durationDifference (timeApprox b T (m+j)) (timeApprox b T (m+i))).abs
      (pointNorm_nonnegative v) (Fraction.abs_num_nonnegative _) hg
    have ha := Fraction.equiv_trans
      (affine_distance x v
        (durationDifference (timeApprox b T m) (timeApprox b T (m+i)))
        (durationDifference (timeApprox b T m) (timeApprox b T (m+j))))
      (Fraction.mul_equiv (Fraction.abs_equiv (shift_difference _ _ _))
        (Fraction.equiv_refl (pointNorm v)))
    exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv ha) hc

theorem affine_vertex_distance (x v : Point) (t : Fraction) :
  Fraction.equiv (distance (affineState x v t) (x, zeroPoint))
    (Fraction.mul t.abs (pointNorm v)) := by
  have hp := ConvexCover.drift_offset t x v
  have he := Fraction.add_equiv (pointNorm_equiv hp)
    (ConvexCover.pointSub_self_zero zeroPoint)
  exact Fraction.equiv_trans he
    (Fraction.equiv_trans (Fraction.add_zero _) (pointNorm_scale t v))

/-- Every later binary-time approximant remains within one coarse duration
of its level-m starting time. -/
theorem phase_abs_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m j : Nat) :
  Fraction.le (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).abs
    (duration T m) :=

```



```

Fraction.le_equiv_left
  (Fraction.equiv_symm (scalarState_distance _ _))
  (finite_gap_time b T hT j m)

theorem phase_nonnegative (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) : (j : Nat) →
  0 ≤ (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num
| 0 => by
  simp only [Nat.add_zero, durationDifference, HarmonicStability.negF, Fraction.add,
    Int.neg_mul]
  omega
| j+1 => by
  have hp := phase_nonnegative b T hT m j
  have hs := Fraction.nonnegative_equiv (time_step_difference b T (m+j))
    (Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg _) hT)
  have hc := durationDifference_chain (timeApprox b T m)
    (timeApprox b T (m+j)) (timeApprox b T (m+(j+1)))
  exact Fraction.nonnegative_equiv hc (Fraction.nonnegative_add _ _ hp
    (by simp only [Nat.add_assoc] using hs))

/-- The time phase is between the two boundaries of the actual coarse cell. -/
theorem phase_interval (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m j : Nat) :
  0 ≤ (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num ∧
  Fraction.le (durationDifference (timeApprox b T m) (timeApprox b T (m+j)))
    (duration T m) := by
  have hn := phase_nonnegative b T hT m j
  exact ⟨hn, Fraction.le_equiv_left
    (Fraction.equiv_symm (Fraction.abs_of_nonnegative _ hn))
    (phase_abs_bound b T hT m j)⟩

theorem edge_vertex_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (x v : Point) (m : Nat) :
  Within (realize (edgeName b T hT x v m)) (embed (x, zeroPoint))
    (Fraction.mul (duration T m) (pointNorm v)) := by
  apply nameBound_of_eventual_le _ _ _ 0
  intro j _
  exact Fraction.le_equiv_left (affine_vertex_distance x v _)
    (Fraction.mul_le_mul_nonnegative (phase_abs_bound b T hT m j)
      (pointNorm v) (pointNorm_nonnegative v))

theorem edgeName_same_start (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (x v : Point) (m : Nat)
  (hbase : timeApprox b T m = timeApprox c T m)
  (hTm : AddressEquiv T hT b c) :
  NameEquiv (edgeName b T hT x v m) (edgeName c T hT x v m) := by
  intro eps heps
  let V := pointNorm v
  let delta := factorDelta V eps (pointNorm_nonnegative v)
  obtain ⟨N, hN⟩ := hTm delta
  (factorDelta_positive V eps (pointNorm_nonnegative v) heps)
  refine ⟨N, ?_⟩
  intro j hj
  have ht := hN (m+j) (by omega)
  have hd := timeState_distance b c T (m+j)
  have hg := Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (Fraction.equiv_symm hd)) ht
  have hc := factor_control V eps
    (durationDifference (timeApprox c T (m+j)) (timeApprox b T (m+j))).abs
    (pointNorm_nonnegative v) (Fraction.abs_num_nonnegative _) hg
  have ha : Fraction.equiv
    (distance ((edgeName b T hT x v m).approx j)
      ((edgeName c T hT x v m).approx j))
    (Fraction.mul
      (durationDifference (timeApprox c T (m+j)) (timeApprox b T (m+j))).abs V) := by
  dsimp only [edgeName]
  rw [hbase]
  exact Fraction.equiv_trans (affine_distance x v _)
    (Fraction.mul_equiv (Fraction.abs_equiv (shift_difference _ _ _))
      (Fraction.equiv_refl V))

```

```

    exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv ha) hc
end NewtonLimitDynamics.Polygon.AffineValues

BarrowLib/Polygon/IntegerRefinement.lean, 2 theorems, 35 lines

import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Common.FiniteGrowth

/-! Elementary binary tick arithmetic for a finite address followed by zero bits. -/

namespace NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

open HarmonicDyadic

/-- After a finite binary address, zero bits only double the existing tick count. -/
theorem ticks_zero_tail (b : Nat → Bool) (m : Nat)
  (hz : ∀ j, m ≤ j → b j = false) :
  (j : Nat) → ticks b (m + j) = ticks b m * blocks j
| 0 => by simp [blocks]
| j + 1 => by
  have ih := ticks_zero_tail b m hz j
  have hbit : bit b (m + j) = 0 := by
    simp [bit, hz (m + j) (Nat.le_add_right m j)]
  rw [Nat.add_succ, ticks, hbit, ih, blocks_succ]
  simp only [Nat.mul_add]
  omega

end NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

namespace NewtonLimitDynamics.Polygon.HarmonicDyadic

open NewtonLimitDynamics

/-- Dividing a dyadic duration twice adds the exponents. -/
theorem duration_nested (T : Fraction) (m j : Nat) :
  Fraction.equiv (duration (duration T m) j) (duration T (m + j)) := by
  simp only [duration, Fraction.equiv, FiniteGrowth.denominator_power_add]
  ac_nf

end NewtonLimitDynamics.Polygon.HarmonicDyadic

BarrowLib/Polygon/BinaryCells.lean, 7 theorems, 84 lines

import BarrowLib.Polygon.AffineValues

/-! Rational coarse-cell intervals and the adjacent-cell alternatives for
equivalent binary addresses. These facts contain no mechanical premises. -/

namespace NewtonLimitDynamics.Polygon.BinaryTime
open NewtonLimitDynamics
open HarmonicBinaryPrefix HarmonicDyadic HarmonicTimeComparison

theorem count_duration_monotone (T : Fraction) (m : Nat)
  (hT : 0 ≤ T.num) (i k : Nat) (hik : i ≤ k) :
  Fraction.le (Fraction.mul (Fraction.ofInt (i:Int)) (duration T m))
    (Fraction.mul (Fraction.ofInt (k:Int)) (duration T m)) := by
  apply Fraction.mul_le_mul_nonnegative (c:=duration T m) (hc:=hT)
  simp only [Fraction.le, Fraction.ofInt, Int.mul_one]
  exact Int.ofNat_le.mpr hik

theorem coarse_upper (b : Nat → Bool) (T : Fraction) (m : Nat) :
  Fraction.equiv (Fraction.add (timeApprox b T m) (duration T m))
    (Fraction.mul (Fraction.ofInt ((ticks b m+1:Int))) (duration T m)) := by
  simp only [timeApprox, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add]
  ac_nf

theorem time_interval (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m j : Nat) :
  Fraction.le (timeApprox b T m) (timeApprox b T (m+j)) ∧

```

```

    Fraction.le (timeApprox b T (m+j))
      (Fraction.add (timeApprox b T m) (duration T m)) := by
  have hs := AffineValues.phase_interval b T hT m j
  exact ((difference_nonnegative_iff _).mp hs.1,
    difference_add_bound _ _ hs.2)

theorem coarse_two_step (b : Nat → Bool) (T : Fraction) (m : Nat) :
  Fraction.equiv
    (Fraction.add (Fraction.add (timeApprox b T m) (duration T m)) (duration T m))
    (Fraction.mul (Fraction.ofInt ((ticks b m+2:Int))) (duration T m)) := by
  simp only [timeApprox, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add]
  simp only [show (2:Int)=1+1 by rfl, Int.mul_add, Int.mul_one]
  ac_nf
  simp only [Int.mul_add, Int.mul_one, Int.add_mul]
  ac_nf

theorem separated_cell_time_gap (b c : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (m j : Nat) (hgap : ticks b m + 2 ≤ ticks c m) :
  Fraction.le (duration T m)
    (durationDifference (timeApprox b T (m+j)) (timeApprox c T (m+j))).abs := by
  obtain (_,hb) := time_interval b T hT m j
  obtain (hc,_) := time_interval c T hT m j
  have hbc := count_duration_monotone T m hT (ticks b m+2) (ticks c m) hgap
  have hbig := Fraction.magnitudes.le_trans
    (Fraction.add_le_add_right hb (duration T m))
    (Fraction.magnitudes.le_trans
      (Fraction.le_equiv_left (coarse_two_step b T m) hbc) hc)
  have hd := Fraction.le_add_cancel_left (timeApprox b T (m+j)) (duration T m)
    (durationDifference (timeApprox b T (m+j)) (timeApprox c T (m+j)))
    (Fraction.le_equiv_right hbig
      (Fraction.equiv_symm (add_difference_cancel _)))
  exact Fraction.magnitudes.le_trans hd (Fraction.le_abs _)

theorem address_equiv_no_separated_cells (b c : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (hpos : 0 < T.num) (m : Nat)
  (ht : AddressEquiv T hT b c) : ¬ ticks b m + 2 ≤ ticks c m := by
  intro hc
  obtain (N,hN) := addressEquiv_symm T hT ht (duration T m) hpos
  have hsmall := hN (m+N) (by omega)
  have hbound := separated_cell_time_gap b c T hT m N hc
  have hd := timeState_distance c b T (m+N)
  have hlow := Fraction.le_equiv_right hbound (Fraction.equiv_symm hd)
  exact Fraction.magnitudes.lt_irrefl _
    (Fraction.magnitudes.lt_of_le_lt hlow hsmall)

theorem address_equiv_cell_cases (b c : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (hpos : 0 < T.num) (m : Nat)
  (ht : AddressEquiv T hT b c) :
  ticks b m = ticks c m v ticks b m + 1 = ticks c m v
  ticks c m + 1 = ticks b m := by
  have hnot := address_equiv_no_separated_cells b c T hT hpos m ht
  have hnotrev := address_equiv_no_separated_cells c b T hT hpos m
    (addressEquiv_symm T hT ht)
  omega

end NewtonLimitDynamics.Polygon.BinaryTime

```

BarrowLib/Polygon/AffineBoundary.lean, 1 theorem, 74 lines

```
import BarrowLib.Polygon.BinaryCells
```

```

/-! Affine edges meeting at the same rational vertex assign the same completed
point to equivalent time addresses on their common boundary. -/

```

```

namespace NewtonLimitDynamics.Polygon.AffineValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic
open HarmonicAccumulation HarmonicTimeComparison HarmonicTimeRealization
open BinaryTime CauchyValues

```

```

theorem edge_boundary_names (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (x v y u : Point) (m : Nat)
  (hbase : Fraction.equiv (timeApprox c T m)
    (Fraction.add (timeApprox b T m) (duration T m)))
  (hpoint : pointEquiv y (pointAdd x (pointScale (duration T m) v)))
  (htime : AddressEquiv T hT b c) :
  NameEquiv (edgeName b T hT x v m) (edgeName c T hT y u m) := by
  let V := Fraction.add (pointNorm v) (pointNorm u)
  have hV : 0 ≤ V.num := Fraction.nonnegative_add _ _
    (pointNorm_nonnegative v) (pointNorm_nonnegative u)
  intro eps heps
  obtain (N,hN) := addressEquiv_symm T hT htime (factorDelta V eps hV)
    (factorDelta_positive V eps hV heps)
  refine (N, ?_)
  intro j hj
  let tb := timeApprox b T (m+j)
  let tc := timeApprox c T (m+j)
  let e := timeApprox c T m
  let gap := (durationDifference tb tc).abs
  have htb : Fraction.le tb e := Fraction.le_equiv_right
    (time_interval b T hT m j).2 (Fraction.equiv_symm hbase)
  have htc : Fraction.le e tc := (time_interval c T hT m j).1
  obtain (hg1,hg2) := difference_interval_gaps tb e tc htb htc
  have hstate : stateEquiv (affineState x v (duration T m)) (y,zeroPoint) :=
    {pointEquiv_symm hpoint, {Fraction.equiv_refl _, Fraction.equiv_refl _}}
  have hll : Fraction.equiv
    (distance ((edgeName b T hT x v m).approx j) (y,zeroPoint))
    (Fraction.mul (durationDifference tb e).abs (pointNorm v)) := by
  have he := stateNorm_equiv (stateSub_congr
    (s:=(edgeName b T hT x v m).approx j))
    ({Fraction.equiv_refl _, Fraction.equiv_refl _},
    {Fraction.equiv_refl _, Fraction.equiv_refl _}) hstate)
  apply Fraction.equiv_trans (Fraction.equiv_symm he)
  exact Fraction.equiv_trans
    (stateSub_norm_symm ((edgeName b T hT x v m).approx j)
    (affineState x v (duration T m)))
    (Fraction.equiv_trans (affine_distance x v (duration T m) _)
    (Fraction.mul_equiv (Fraction.abs_equiv
    (Fraction.equiv_trans (phase_end_difference _ _ _)
    (difference_congr (Fraction.equiv_refl tb) (Fraction.equiv_symm hbase))))
    (Fraction.equiv_refl _)))
  have hl := Fraction.le_equiv_left hll
    (Fraction.mul_le_mul_nonnegative hg1 (pointNorm v) (pointNorm_nonnegative v))
  have hrl : Fraction.equiv
    (distance ((edgeName c T hT y u m).approx j) (y,zeroPoint))
    (Fraction.mul (durationDifference e tc).abs (pointNorm u)) :=
    affine_vertex_distance y u _
  have hr := Fraction.le_equiv_left
    (Fraction.equiv_trans (stateSub_norm_symm (y,zeroPoint)
    ((edgeName c T hT y u m).approx j)) hrl)
    (Fraction.mul_le_mul_nonnegative hg2 (pointNorm u) (pointNorm_nonnegative u))
  have htri := Fraction.magnitudes.le_trans
    (stateSub_triangle ((edgeName b T hT x v m).approx j) (y,zeroPoint)
    ((edgeName c T hT y u m).approx j)) (Fraction.add_le_add hl hr)
  have hscaled := Fraction.le_equiv_right htri
    (Fraction.equiv_symm (Fraction.mul_add gap (pointNorm v) (pointNorm u)))
  have htimegap := Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (Fraction.equiv_symm (timeState_distance c b T (m+j))))
    (hN (m+j) (by omega))
  exact Fraction.magnitudes.lt_of_le_lt hscaled
    (factor_control V eps gap hV (Fraction.abs_num_nonnegative _) htimegap)

end NewtonLimitDynamics.Polygon.AffineValues

```

BarrowLib/Polygon/IntegerSchedule.lean, 5 theorems, 83 lines

```
import BarrowLib.Common.RationalMagnitudes
```

```
/-! Elementary rational duration arithmetic for an integer number of cells. -/
```

```
namespace NewtonLimitDynamics.Polygon.IntegerSchedule
```

```

open NewtonLimitDynamics

def integerDuration (h : Fraction) : Nat → Fraction
| 0 => Fraction.ofInt 0
| k + 1 => Fraction.add (integerDuration h k) h

theorem integerDuration_closed (h : Fraction) :
  (k : Nat) → Fraction.equiv (integerDuration h k)
    (Fraction.mul (Fraction.ofInt (k : Int)) h)
| 0 => by
  simp only [integerDuration, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
| k + 1 => by
  have hi := integerDuration_closed h k
  have ha := Fraction.add_equiv hi (Fraction.equiv_refl h)
  apply Fraction.equiv_trans ha
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one]
  simp only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

theorem integerDuration_nonnegative (h : Fraction) (hh : 0 ≤ h.num) :
  (k : Nat) → 0 ≤ (integerDuration h k).num
| 0 => by simp [integerDuration, Fraction.ofInt]
| k + 1 => Fraction.nonnegative_add _ _
  (integerDuration_nonnegative h hh k) hh

theorem integerDuration_le (h : Fraction) (hh : 0 ≤ h.num)
  (i k : Nat) (hik : i ≤ k) :
  Fraction.le (integerDuration h i) (integerDuration h k) := by
  have hc : Fraction.le (Fraction.ofInt (i : Int))
    (Fraction.ofInt (k : Int)) := by
    unfold Fraction.le Fraction.ofInt
    dsimp
    simp only [Int.mul_one] using Int.ofNat_le.mpr hik
  have hm := Fraction.mul_le_mul_nonnegative hc h hh
  exact Fraction.le_equiv_left (integerDuration_closed h i)
    (Fraction.le_equiv_right hm
      (Fraction.equiv_symm (integerDuration_closed h k)))

end NewtonLimitDynamics.Polygon.IntegerSchedule

namespace NewtonLimitDynamics.Polygon.HarmonicUniform
open NewtonLimitDynamics
/-- Actual represented total time `T=2nh` and the smallness hypothesis
`T(1+|w|) ≤ 1/2`. Zero cell count and zero duration are included. -/
def totalTime (h : Fraction) (n : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (2 * (n : Int))) h

end NewtonLimitDynamics.Polygon.HarmonicUniform

namespace NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision
open NewtonLimitDynamics IntegerSchedule HarmonicUniform
def fullTime (d : Fraction) (N : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt (N : Int)) d

theorem half_totalTime (d : Fraction) (N : Nat) :
  Fraction.equiv (totalTime d.half N) (fullTime d N) := by
  simp only [totalTime, fullTime, Fraction.half, Fraction.equiv,
    Fraction.mul, Fraction.ofInt]
  ac_nf

theorem integer_fullTime (h : Fraction) (k N : Nat) :
  Fraction.equiv (fullTime h (k * N))
    (fullTime (integerDuration h k) N) := by
  have hc := integerDuration_closed h k
  have hmul := Fraction.mul_equiv
    (Fraction.equiv_refl (Fraction.ofInt (N : Int))) hc
  have he : Fraction.equiv (fullTime h (k * N))

```

```

    (Fraction.mul (Fraction.ofInt (N : Int))
      (Fraction.mul (Fraction.ofInt (k : Int)) h)) := by
    simp only [fullTime, Fraction.equiv, Fraction.mul, Fraction.ofInt,
      Int.natCast_mul]
    ac_nf
    exact Fraction.equiv_trans he (Fraction.equiv_symm hmul)
end NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision

BarrowLib/Polygon/FiniteRecurrence.lean, 9 theorems, 138 lines

import BarrowLib.Polygon.FiniteAccumulation
import BarrowLib.Polygon.FinitePower

/--! Finite rational recurrence with a constant nonnegative source. -/
namespace NewtonLimitDynamics.Polygon.FiniteRecurrence

open NewtonLimitDynamics
open HarmonicAccumulation

def sourceBudget (r C : Fraction) : Nat → Fraction
| 0 => Fraction.ofInt 0
| n + 1 => Fraction.add (Fraction.mul r (sourceBudget r C n)) C

/-- Compatibility between the two pre-existing public finite-power names. -/
theorem factorPower_fpower (r : Fraction) :
  (n : Nat) → Fraction.equiv (FiniteAccumulation.factorPower r n) (fpower r n)
| n => FiniteAccumulation.factorPower_fpower r n

/-- Reuse the pre-existing generic power lower bound. -/
theorem one_le_fpower (r : Fraction) (hr : 0 ≤ r.num)
  (hone : Fraction.le (Fraction.ofInt 1) r) (n : Nat) :
  Fraction.le (Fraction.ofInt 1) (fpower r n) :=
  Fraction.le_equiv_right (FiniteAccumulation.one_le_factorPower r hr hone n)
  (factorPower_fpower r n)

theorem fpower_add (r : Fraction) (m : Nat) :
  (n : Nat) → Fraction.equiv (fpower r (m + n))
  (Fraction.mul (fpower r m) (fpower r n))
| 0 => by
  simp only [Nat.add_zero, fpower, Fraction.equiv, Fraction.mul,
    Fraction.ofInt]
  simp
| n + 1 => by
  have ih := fpower_add r m n
  have hm := Fraction.mul_equiv (Fraction.equiv_refl r) ih
  apply Fraction.equiv_trans hm
  simp only [Nat.add_succ, fpower, Fraction.equiv, Fraction.mul]
  ac_nf

theorem fpower_congr {r q : Fraction} (hrq : Fraction.equiv r q) :
  (n : Nat) → Fraction.equiv (fpower r n) (fpower q n)
| n => FiniteFactorProducts.fpower_congr hrq n

theorem fpower_integer_blocks (r : Fraction) (k : Nat) :
  (N : Nat) → Fraction.equiv (fpower (fpower r k) N)
  (fpower r (k * N))
| 0 => by simp [fpower, Fraction.equiv, Fraction.ofInt, Fraction.mul]
| N + 1 => by
  have ih := fpower_integer_blocks r k N
  have hm := Fraction.mul_equiv (Fraction.equiv_refl (fpower r k)) ih
  have ha := fpower_add r k (k * N)
  have he : k * (N + 1) = k + k * N := by
    simp only [Nat.mul_add, Nat.mul_one, Nat.add_comm]
    rw [he]
  exact Fraction.equiv_trans hm (Fraction.equiv_symm ha)

theorem fpower_prefix_le (r : Fraction) (hr : 0 ≤ r.num)
  (hone : Fraction.le (Fraction.ofInt 1) r)
  (i N : Nat) (hi : i ≤ N) :

```

```

    Fraction.le (fpower r i) (fpower r N) := by
have htail := one_le_fpower r hr hone (N - i)
have hm := Fraction.mul_le_mul_nonnegative_left htail (fpower r i)
(fpower_nonnegative r hr i)
have hleft : Fraction.equiv
(Fraction.mul (fpower r i) (Fraction.ofInt 1)) (fpower r i) := by
simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
simp
have hright := fpower_add r i (N - i)
have he : i + (N - i) = N := by omega
rw [he] at hright
exact Fraction.le_equiv_right
(Fraction.le_equiv_left (Fraction.equiv_symm hleft) hm)
(Fraction.equiv_symm hright)

```

```

theorem sourceBudget_nonnegative (r C : Fraction)
(hr : 0 ≤ r.num) (hC : 0 ≤ C.num) :
(n : Nat) → 0 ≤ (sourceBudget r C n).num
| 0 => by simp [sourceBudget, Fraction.ofInt]
| n + 1 => Fraction.nonnegative_add _ _
(Fraction.nonnegative_mul _ _ hr
(sourceBudget_nonnegative r C hr hC n)) hC

```

```

theorem sourceBudget_power (r C : Fraction)
(hr : 0 ≤ r.num) (hC : 0 ≤ C.num)
(hone : Fraction.le (Fraction.ofInt 1) r) :
(n : Nat) → Fraction.le (sourceBudget r C n)
(Fraction.mul (Fraction.ofInt (n : Int))
(Fraction.mul C (fpower r n)))
| 0 => by
apply Fraction.le_of_equiv
simp only [sourceBudget, fpower, Fraction.equiv, Fraction.ofInt,
Fraction.mul]
simp
| n + 1 => by
have hi := sourceBudget_power r C hr hC hone n
have hpow := one_le_fpower r hr hone (n + 1)
have h1 := Fraction.mul_le_mul_nonnegative_left hi r hr
have h2 : Fraction.le C
(Fraction.mul C (fpower r (n + 1))) := by
have hm := Fraction.mul_le_mul_nonnegative_left hpow C hC
exact Fraction.le_equiv_left (by
simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
simp) hm
have h2' := Fraction.add_le_add h1 h2
have h3 : Fraction.le
(Fraction.add
(Fraction.mul r (sourceBudget r C n)) C)
(Fraction.add
(Fraction.mul (Fraction.ofInt (n : Int))
(Fraction.mul C (fpower r (n + 1))))
(Fraction.mul C (fpower r (n + 1)))) := by
apply Fraction.le_equiv_right h2'
simp only [fpower, Fraction.equiv, Fraction.mul]
ac_nf
apply Fraction.le_equiv_right h3
simp only [sourceBudget, fpower, Fraction.equiv,
Fraction.add, Fraction.mul, Fraction.ofInt,
Int.natCast_add, Int.natCast_one]
simp only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
ac_nf

```

```

/-- Shared closing step for any actual recurrence with a finite factor ≤2. -/
theorem sourceBudget_two_count (r C : Fraction) (n : Nat)
(hr : 0 ≤ r.num) (hC : 0 ≤ C.num)
(hone : Fraction.le (Fraction.ofInt 1) r)
(hp : Fraction.le (fpower r n) (Fraction.ofInt 2)) :
Fraction.le (sourceBudget r C n)
(Fraction.mul (Fraction.ofInt (2*(n : Int))) C) := by
have hb := sourceBudget_power r C hr hC hone n

```

```

have hm := Fraction.mul_le_mul_nonnegative_left
  (Fraction.mul_le_mul_nonnegative_left hp C hC)
  (Fraction.ofInt (n : Int)) (Int.ofNat_nonneg n)
apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans hb hm)
simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
ac_nf

```

end NewtonLimitDynamics.Polygon.FiniteRecurrence

BarrowLib/Polygon/IntegerTime.lean, 3 theorems, 39 lines

```

import BarrowLib.Polygon.IntegerSchedule
import BarrowLib.Polygon.IntegerRefinement
import BarrowLib.Polygon.BinaryTime
import BarrowLib.Common.FiniteGrowth

/-! Rational binary-time and integer-duration bridges. Retained namespaces
are compatibility names; no harmonic field or mechanical premise occurs. -/

namespace NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision
open NewtonLimitDynamics HarmonicDyadic HarmonicBinaryPrefix BinaryTime IntegerSchedule
theorem dyadic_integer_duration (b : Nat → Bool) (T : Fraction) (m j : Nat) :
  Fraction.equiv (integerDuration (duration T (m+j)) (ticks b m))
    (duration (timeApprox b T m) j) := by
  have he := integerDuration_closed (duration T (m+j)) (ticks b m)
  apply Fraction.equiv_trans he
  simp only [timeApprox, duration, Fraction.equiv, Fraction.mul, Fraction.ofInt,
    FiniteGrowth.denominator_power_add]
  ac_nf

theorem dyadic_fullTime (b : Nat → Bool) (T : Fraction) (m j : Nat) :
  Fraction.equiv
    (fullTime (integerDuration (duration T (m+j)) (ticks b m)) (blocks j))
    (timeApprox b T m) := by
  have hd := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (blocks j:Int)))
    (dyadic_integer_duration b T m j)
  apply Fraction.equiv_trans hd
  simp only [fullTime, duration, blocks, Fraction.equiv, Fraction.mul, Fraction.ofInt,
    Int.natCast_pow]
  ac_nf

end NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision

namespace NewtonLimitDynamics.Polygon.BinaryTime
open NewtonLimitDynamics HarmonicDyadic
theorem timeApprox_nonnegative (b : Nat → Bool) (T : Fraction) (m : Nat)
  (hT : 0 ≤ T.num) : 0 ≤ (timeApprox b T m).num :=
  Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg _) hT

end NewtonLimitDynamics.Polygon.BinaryTime

```

BarrowLib/Polygon/FiniteAddress.lean, 3 theorems, 51 lines

```

import BarrowLib.Polygon.IntegerTime

/-! Explicit terminating binary addresses for every integer numerator below
2^m. This is finite arithmetic, with no external real interval assumed. -/

namespace NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix
open HarmonicDyadic

def finiteAddress : Nat → Nat → Nat → Bool
| 0, _, _ => false
| m+1, k, j => if j=m then decide (k%2=1) else finiteAddress m (k/2) j

theorem finiteAddress_tail : (m k j : Nat) → m ≤ j → finiteAddress m k j = false
| 0, _, _, _ => rfl
| m+1, k, j, hj => by
  have hn : j ≠ m := by omega
  simp only [finiteAddress, hn, if_false]
  exact finiteAddress_tail m (k/2) j (by omega)

```



```

theorem ticks_congr_before (b c : Nat → Bool) : (m : Nat) →
  (∀ i, i < m → b i = c i) → ticks b m = ticks c m
| 0, _ => rfl
| m+1, h => by
  have hp := ticks_congr_before b c m (fun i hi => h i (by omega))
  have hb := h m (by omega)
  simp only [ticks, hp, bit, hb]

theorem finiteAddress_ticks : (m k : Nat) → k < blocks m → ticks (finiteAddress m k) m = k
| 0, k, hk => by
  have hb : blocks 0 = 1 := rfl
  have hz : k = 0 := by rw [hb] at hk; omega
  subst k
  rfl
| m+1, k, hk => by
  have hk' : k < blocks m*2 := by simpa only [blocks, Nat.pow_succ] using hk
  have hd : k/2 < blocks m := (Nat.div_lt_iff_lt_mul (by decide)).mpr hk'
  have hp := ticks_congr_before (finiteAddress (m+1) k) (finiteAddress m (k/2)) m
  (fun i hi => by
    have hn : i ≠ m := by omega
    simp only [finiteAddress, hn, if_false])
  have hbit : bit (finiteAddress (m+1) k) m = k%2 := by
    have hmod := Nat.mod_lt k (by decide : 0 < 2)
    by_cases hm : k%2 = 1
    · simp only [bit, finiteAddress, if_pos rfl, hm, decide_true, if_true]
    · have hz : k%2 = 0 := by omega
      simp [bit, finiteAddress, hz]
  change 2*ticks (finiteAddress (m+1) k) m + bit (finiteAddress (m+1) k) m = k
  rw [hp, finiteAddress_ticks m (k/2) hd, hbit]
  exact Nat.div_add_mod k 2

end NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

```

BarrowLib/Polygon/FiniteFactorProducts.lean, 7 theorems, 96 lines

```

import BarrowLib.Common.FiniteGrowth
import BarrowLib.Polygon.FinitePower

/--! Elementary repeated two-factor growth, with arbitrary rational increments.
The common-denominator proof is shared by calibrated and unit-gauge estimates.
No motion, limit or differential equation is a premise. -/

namespace NewtonLimitDynamics.Polygon.FiniteFactorProducts
open NewtonLimitDynamics
open HarmonicAccumulation

def pairWeights (P Q : Fraction) : Nat → List Int
| 0 => []
| n+1 => [P.num * Q.den, Q.num * P.den] ++ pairWeights P Q n

theorem pairWeights_nonnegative (P Q : Fraction)
  (hP : 0 ≤ P.num) (hQ : 0 ≤ Q.num) :
  (n : Nat) → FiniteGrowth.Nonnegative (pairWeights P Q n)
| 0 => by simp [pairWeights, FiniteGrowth.Nonnegative]
| n+1 => by
  intro a ha
  simp only [pairWeights, List.mem_append, List.mem_cons, List.mem_nil_iff,
    or_false] at ha
  rcases ha with (rfl | rfl) | ha
  · exact Int.mul_nonneg hP (Int.le_of_lt Q.den_pos)
  · exact Int.mul_nonneg hQ (Int.le_of_lt P.den_pos)
  · exact pairWeights_nonnegative P Q hP hQ n a ha

theorem pairWeights_sum (P Q : Fraction) :
  (n : Nat) → FiniteGrowth.weightSum (pairWeights P Q n) =
  (n : Int) * (P.num * Q.den + Q.num * P.den)
| 0 => by simp [pairWeights, FiniteGrowth.weightSum]
| n+1 => by
  simp only [pairWeights, FiniteGrowth.weightSum_append, FiniteGrowth.weightSum,
    pairWeights_sum P Q n, Int.natCast_add, Int.natCast_one,

```

```

      Int.add_mul, Int.one_mul]
    omega

theorem pair_product (P Q : Fraction) :
  Fraction.equiv
    (FiniteGrowth.amplification (P.den * Q.den)
      (Int.mul_pos P.den_pos Q.den_pos) [P.num * Q.den, Q.num * P.den])
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) P)
      (Fraction.add (Fraction.ofInt 1) Q)) := by
  simp only [FiniteGrowth.amplification, FiniteGrowth.factorProduct,
    List.length_cons, List.length_nil, Fraction.equiv, Fraction.mul, Fraction.add,
    Fraction.ofInt, Int.pow_succ, Int.pow_zero, Int.one_mul, Int.mul_one]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem repeated_pair_product (P Q : Fraction) :
  (n : Nat) → Fraction.equiv
    (FiniteGrowth.amplification (P.den * Q.den)
      (Int.mul_pos P.den_pos Q.den_pos) (pairWeights P Q n))
    (fpower (Fraction.mul (Fraction.add (Fraction.ofInt 1) P)
      (Fraction.add (Fraction.ofInt 1) Q)) n)
  | 0 => FiniteGrowth.amplification_empty _ _
  | n+1 => Fraction.equiv_trans
    (FiniteGrowth.amplification_append
      [P.num * Q.den, Q.num * P.den] (pairWeights P Q n))
    (Fraction.mul_equiv (pair_product P Q) (repeated_pair_product P Q n))

/-- The sum of increments, rather than a dimensioned expression `1+L`,
controls the shared finite product. -/
theorem repeated_pair_le_two (P Q : Fraction) (n : Nat)
  (hP : 0 ≤ P.num) (hQ : 0 ≤ Q.num)
  (hs : Fraction.le
    (Fraction.mul (Fraction.ofInt (n : Int)) (Fraction.add P Q))
    (1, 2, by decide)) :
  Fraction.le (fpower (Fraction.mul (Fraction.add (Fraction.ofInt 1) P)
    (Fraction.add (Fraction.ofInt 1) Q)) n) (Fraction.ofInt 2) := by
  have hsmall : 2 * FiniteGrowth.weightSum (pairWeights P Q n) ≤ P.den * Q.den := by
  rw [pairWeights_sum]
  unfold Fraction.le Fraction.mul Fraction.add Fraction.ofInt at hs
  dsimp at hs
  simp only [Int.one_mul, Int.mul_one] at hs
  simpa only [Int.mul_comm, Int.mul_left_comm, Int.mul_assoc] using hs
  exact Fraction.le_equiv_left (Fraction.equiv_symm (repeated_pair_product P Q n))
    (FiniteGrowth.uniform_amplification _ _ (pairWeights P Q n)
      (pairWeights_nonnegative P Q hP hQ n) hsmall)

theorem fpower_congr {r q : Fraction} (hrq : Fraction.equiv r q) :
  (n : Nat) → Fraction.equiv (fpower r n) (fpower q n)
  | 0 => Fraction.equiv_refl _
  | n+1 => Fraction.mul_equiv hrq (fpower_congr hrq n)

theorem fpower_square (r : Fraction) :
  (n : Nat) → Fraction.equiv (fpower (Fraction.mul r r) n) (fpower r (2*n))
  | 0 => by simp [fpower, Fraction.equiv, Fraction.ofInt]
  | n+1 => by
    simp only [Nat.mul_succ, Nat.add_succ, Nat.add_zero, fpower]
    exact Fraction.equiv_trans
      (Fraction.mul_equiv (Fraction.equiv_refl _) (fpower_square r n))
      (Fraction.mul_assoc r r (fpower r (2*n)))

end NewtonLimitDynamics.Polygon.FiniteFactorProducts

```

BarrowLib/Polygon/TimeCalibration.lean, 41 theorems, 628 lines

```

import BarrowLib.Polygon.FiniteEstimates
import BarrowLib.Polygon.ScaledTolerance
import BarrowLib.Polygon.FiniteFactorProducts
import BarrowLib.Polygon.FiniteRecurrence
import BarrowLib.Polygon.BoundedIteration

```

/-! Calibrated coordinate gauges and finite triangular-map estimates.

A positive rational time calibration is free data. Rescaling the time unit preserves the weighted gauge, dimensionless growth and sampling budgets. Cauchy convergence is independent of this choice. This modern elementary infrastructure assumes no motion, force law, potential or universal scale. -/

```

namespace NewtonLimitDynamics.Polygon.TimeCalibration
open NewtonLimitDynamics TimeSubdivision PointBounds FiniteEstimates

def inverse (tau : Fraction) (ht : 0 < tau.num) : Fraction :=
  (tau.den, tau.num, ht)

theorem inverse_positive (tau : Fraction) (ht : 0 < tau.num) :
  0 < (inverse tau ht).num := tau.den_pos

theorem inverse_product (tau : Fraction) (ht : 0 < tau.num) :
  Fraction.equiv (Fraction.mul tau (inverse tau ht)) (Fraction.ofInt 1) := by
  simp only [inverse, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem inverse_congr {tau rho : Fraction} (ht : 0 < tau.num) (hr : 0 < rho.num)
  (h : Fraction.equiv tau rho) : Fraction.equiv (inverse tau ht) (inverse rho hr) := by
  simpa only [inverse, Fraction.equiv, Int.mul_comm] using h.symm

def norm (tau : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (pointNorm s.1) (Fraction.mul tau (pointNorm s.2))

theorem norm_nonnegative (tau : Fraction) (s : Point × Point)
  (ht : 0 ≤ tau.num) : 0 ≤ (norm tau s).num :=
  Fraction.nonnegative_add _ _ (pointNorm_nonnegative _)
  (Fraction.nonnegative_mul _ _ ht (pointNorm_nonnegative _))

theorem position_le_norm (tau : Fraction) (s : Point × Point)
  (ht : 0 ≤ tau.num) : Fraction.le (pointNorm s.1) (norm tau s) :=
  Fraction.le_add_nonnegative _ _
  (Fraction.nonnegative_mul _ _ ht (pointNorm_nonnegative _))

theorem velocity_le_norm_inverse (tau : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) :
  Fraction.le (pointNorm s.2) (Fraction.mul (norm tau s) (inverse tau ht)) := by
  have hv := Fraction.le_equiv_right
    (Fraction.le_add_nonnegative (Fraction.mul tau (pointNorm s.2))
      (pointNorm s.1) (pointNorm_nonnegative _)) (Fraction.add_comm _ _)
  exact Fraction.le_equiv_left (by
    simp only [inverse, Fraction.equiv, Fraction.mul, Fraction.ofInt]
    ac_nf) (Fraction.mul_le_mul_nonnegative hv (inverse tau ht))
    (Int.le_of_lt (inverse_positive tau ht)))

def distance (tau : Fraction) (s t : Point × Point) : Fraction :=
  Fraction.add (pointDistance s.1 t.1) (Fraction.mul tau (pointDistance s.2 t.2))

/-- The actual drift and kick increments, in the chosen calibrated gauge. -/
theorem cell_increment (tau h : Fraction) (a : Point → Point) (s : Point × Point) :
  Fraction.equiv (distance tau (cell a h s) s)
  (Fraction.mul h.abs
    (Fraction.add (pointNorm s.2) (Fraction.mul tau (pointNorm (a (cell a h s).1)))))) := by
  have hp := Fraction.equiv_trans
    (pointNorm_equiv (ConvexCover.drift_offset h s.1 s.2)) (pointNorm_scale h s.2)
  have hv := Fraction.equiv_trans
    (pointNorm_equiv (ConvexCover.drift_offset h s.2 (a (cell a h s).1)))
    (pointNorm_scale h (a (cell a h s).1))
  exact Fraction.equiv_trans
    (Fraction.add_equiv hp (Fraction.mul_equiv_left tau hv)) (by
      simp only [Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
      ac_nf)

theorem cell_increment_bound (tau h B V : Fraction) (a : Point → Point)
  (s : Point × Point) (ht : 0 ≤ tau.num)
  (hv : Fraction.le (pointNorm s.2) V)
  (ha : Fraction.le (pointNorm (a (cell a h s).1)) B) :
  Fraction.le (distance tau (cell a h s) s)

```

```

    (Fraction.mul h.abs (Fraction.add V (Fraction.mul tau B))) :=
    Fraction.le_equiv_left (cell_increment tau h a s)
    (Fraction.mul_le_mul_nonnegative_left
    (Fraction.add_le_add hv (Fraction.mul_le_mul_nonnegative_left ha tau ht))
    h.abs (Fraction.abs_num_nonnegative h))

theorem distance_self_zero (tau : Fraction) (s : Point × Point) :
  Fraction.equiv (distance tau s s) (Fraction.ofInt 0) :=
  Fraction.equiv_trans
  (Fraction.add_equiv (pointDistance_self_zero s.1)
  (Fraction.mul_equiv (Fraction.equiv_refl tau) (pointDistance_self_zero s.2)))
  (by simp [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt])

theorem distance_symm (tau : Fraction) (s t : Point × Point) :
  Fraction.equiv (distance tau s t) (distance tau t s) :=
  Fraction.add_equiv (pointDistance_symm s.1 t.1)
  (Fraction.mul_equiv_left tau (pointDistance_symm s.2 t.2))

theorem distance_triangle (tau : Fraction) (ht : 0 < tau.num)
  (s t u : Point × Point) :
  Fraction.le (distance tau s u)
  (Fraction.add (distance tau s t) (distance tau t u)) := by
  have hsum := Fraction.add_le_add (pointDistance_triangle s.1 t.1 u.1)
  (Fraction.mul_le_mul_nonnegative_left (pointDistance_triangle s.2 t.2 u.2)
  tau (Int.le_of_lt ht))
  apply Fraction.le_equiv_right hsum
  simp only [distance, Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

def amplification (tau h L : Fraction) (ht : 0 < tau.num) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs (inverse tau ht)))
  (Fraction.add (Fraction.ofInt 1) (Fraction.mul (Fraction.mul h.abs tau) L))

def rate (tau L : Fraction) (ht : 0 < tau.num) : Fraction :=
  Fraction.add (inverse tau ht) (Fraction.mul tau L)

def rescaleState (c : Fraction) (hc : 0 < c.num) (s : Point × Point) : Point × Point :=
  (s.1, pointScale (inverse c hc) s.2)

def rescaleConstant (c L : Fraction) (hc : 0 < c.num) : Fraction :=
  Fraction.mul (Fraction.mul (inverse c hc) (inverse c hc)) L

theorem norm_rescale (c tau : Fraction) (s : Point × Point)
  (hc : 0 < c.num) :
  Fraction.equiv (norm (Fraction.mul c tau) (rescaleState c hc s)) (norm tau s) := by
  have hn := Fraction.equiv_trans (pointNorm_scale (inverse c hc) s.2)
  (Fraction.mul_equiv (Fraction.abs_of_nonnegative _ (Int.le_of_lt c.den_pos))
  (Fraction.equiv_refl _))
  apply Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _)
  (Fraction.mul_equiv (Fraction.equiv_refl _) hn))
  simp only [norm, rescaleState, inverse, Fraction.equiv, Fraction.add, Fraction.mul,
  Int.add_mul, Int.mul_add]
  ac_nf

theorem amplification_rescale (c tau h L : Fraction)
  (hc : 0 < c.num) (ht : 0 < tau.num) :
  Fraction.equiv
  (amplification (Fraction.mul c tau) (Fraction.mul c h)
  (rescaleConstant c L hc) (Int.mul_pos hc ht))
  (amplification tau h L ht) := by
  have hab := Fraction.equiv_trans (Fraction.abs_mul c h)
  (Fraction.mul_equiv (Fraction.abs_of_nonnegative c (Int.le_of_lt hc))
  (Fraction.equiv_refl _))
  unfold amplification
  apply Fraction.mul_equiv
  · apply Fraction.add_equiv (Fraction.equiv_refl _)
    apply Fraction.equiv_trans (Fraction.mul_equiv hab (Fraction.equiv_refl _))
    simp only [rescaleConstant, inverse, Fraction.equiv, Fraction.mul]
    ac_nf
  · apply Fraction.add_equiv (Fraction.equiv_refl _)

```

```

    apply Fraction.equiv_trans (Fraction.mul_equiv
      (Fraction.mul_equiv hab (Fraction.equiv_refl _)) (Fraction.equiv_refl _))
    simp only [rescaleConstant, inverse, Fraction.equiv, Fraction.mul]
    ac_nf

theorem window_rescale (c tau T L : Fraction)
  (hc : 0 < c.num) (ht : 0 < tau.num) :
  Fraction.equiv
    (Fraction.mul (Fraction.mul c T)
      (rate (Fraction.mul c tau) (rescaleConstant c L hc) (Int.mul_pos hc ht)))
    (Fraction.mul T (rate tau L ht)) := by
  simp only [rate, rescaleConstant, inverse, Fraction.equiv, Fraction.add,
    Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

set_option maxHeartbeats 500000 in
theorem component_amplification (tau P V H L E : Fraction)
  (ht : 0 < tau.num) (hP : 0 ≤ P.num) (hV : 0 ≤ V.num)
  (hH : 0 ≤ H.num) (hL : 0 ≤ L.num) :
  Fraction.le
    (Fraction.add (Fraction.add P (Fraction.mul H V))
      (Fraction.mul tau (Fraction.add V (Fraction.mul H
        (Fraction.add (Fraction.mul L (Fraction.add P (Fraction.mul H V))) E))))))
    (Fraction.add
      (Fraction.mul (amplification tau H L ht)
        (Fraction.add P (Fraction.mul tau V)))
      (Fraction.mul (Fraction.mul tau H) E)) := by
  let A := Fraction.add (Fraction.add P (Fraction.mul H V))
    (Fraction.mul tau (Fraction.add V (Fraction.mul H
      (Fraction.add (Fraction.mul L (Fraction.add P (Fraction.mul H V))) E))))
  let S := Fraction.add (Fraction.mul (Fraction.mul H (inverse tau ht)) P)
    (Fraction.add
      (Fraction.mul (Fraction.mul (Fraction.mul tau tau) H) L) V)
    (Fraction.mul (Fraction.mul (Fraction.mul H H) L) P))
  have hS : 0 ≤ S.num := Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _
      (Fraction.nonnegative_mul _ _ hH (Int.le_of_lt tau.den_pos)) hP)
    (Fraction.nonnegative_add _ _
      (Fraction.nonnegative_mul _ _
        (Fraction.nonnegative_mul _ _
          (Fraction.nonnegative_mul _ _
            (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Int.le_of_lt ht)) hH) hL) hV)
          (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hH hH) hL) hP))
  have hbase := Fraction.le_add_nonnegative A S hS
  let K := Fraction.mul (Fraction.mul (Fraction.ofInt 1) (Fraction.mul H (inverse tau ht)))
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul (Fraction.mul H tau) L))
  have he : Fraction.equiv (Fraction.add A S)
    (Fraction.add (Fraction.mul K (Fraction.add P (Fraction.mul tau V)))
      (Fraction.mul (Fraction.mul tau H) E)) := by
  simp only [A, S, K, inverse, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.add_mul, Int.mul_add, Int.mul_one, Int.one_mul]
  ac_nf
  have hk : Fraction.equiv K (amplification tau H L ht) :=
    Fraction.mul_equiv
      (Fraction.add_equiv (Fraction.equiv_refl _)
        (Fraction.mul_equiv (Fraction.equiv_symm (Fraction.abs_of_nonnegative H hH))
          (Fraction.equiv_refl _)))
      (Fraction.add_equiv (Fraction.equiv_refl _)
        (Fraction.mul_equiv
          (Fraction.mul_equiv (Fraction.equiv_symm (Fraction.abs_of_nonnegative H hH))
            (Fraction.equiv_refl _)) (Fraction.equiv_refl _)))
  exact Fraction.le_equiv_right hbase
    (Fraction.equiv_trans he (Fraction.add_equiv
      (Fraction.mul_equiv hk (Fraction.equiv_refl _)) (Fraction.equiv_refl _)))

theorem cell_amplification_at (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E : Fraction) (s t : Point × Point)
  (hL : 0 ≤ L.num)
  (hc : Fraction.le (pointDistance (a (cell a h s).1) (b (cell b h t).1)))

```

```

    (Fraction.add (Fraction.mul L
      (pointDistance (cell a h s).1 (cell b h t).1)) E)) :
  Fraction.le (distance tau (cell a h s) (cell b h t))
  (Fraction.add (Fraction.mul (amplification tau h L ht) (distance tau s t))
    (Fraction.mul (Fraction.mul tau h.abs) E)) := by
have hp := cell_position_perturbation a b h s t
have hv := cell_velocity_perturbation_at a b h L E s t hc
have ha := Fraction.add_le_add_left
  (Fraction.mul_le_mul_nonnegative_left
    (Fraction.add_le_add_right
      (Fraction.mul_le_mul_nonnegative_left hp L hL) E)
    h.abs (Fraction.abs_num_nonnegative h)) (pointDistance s.2 t.2)
have hvc := Fraction.magnitudes.le_trans hv ha
have hsum := Fraction.add_le_add hp
  (Fraction.mul_le_mul_nonnegative_left hvc tau (Int.le_of_lt ht))
have hcomp := component_amplification tau (pointDistance s.1 t.1)
  (pointDistance s.2 t.2) h.abs L E ht
  (pointNorm_nonnegative _) (pointNorm_nonnegative _)
  (Fraction.abs_num_nonnegative h) hL
have hh : h.abs.abs = h.abs := by simp only [Fraction.abs, Int.natAbs_ofNat]
have hbound := Fraction.magnitudes.le_trans hsum hcomp
simp only [distance, amplification, hh] using hbound

theorem cell_amplification (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E : Fraction) (s t : Point × Point)
  (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) :
  Fraction.le (distance tau (cell a h s) (cell b h t))
  (Fraction.add (Fraction.mul (amplification tau h L ht) (distance tau s t))
    (Fraction.mul (Fraction.mul tau h.abs) E)) :=
  cell_amplification_at tau ht a b h L E s t hL
  (hc (cell a h s).1 (cell b h t).1)

theorem sampling_term_rescale (c tau h E : Fraction) (hc : 0 < c.num) :
  Fraction.equiv
    (Fraction.mul (Fraction.mul (Fraction.mul c tau) (Fraction.mul c h).abs)
      (rescaleConstant c E hc))
    (Fraction.mul (Fraction.mul tau h.abs) E) := by
have ha := Fraction.equiv_trans (Fraction.abs_mul c h)
  (Fraction.mul_equiv (Fraction.abs_of_nonnegative c (Int.le_of_lt hc))
    (Fraction.equiv_refl _))
apply Fraction.equiv_trans (Fraction.mul_equiv
  (Fraction.mul_equiv (Fraction.equiv_refl _) ha) (Fraction.equiv_refl _))
simp only [rescaleConstant, inverse, Fraction.equiv, Fraction.mul]
ac_nf

theorem distance_rescale (c tau : Fraction) (s t : Point × Point) (hc : 0 < c.num) :
  Fraction.equiv
    (distance (Fraction.mul c tau) (rescaleState c hc s) (rescaleState c hc t))
    (distance tau s t) := by
have hv := Fraction.equiv_trans (difference_scale (inverse c hc) s.2 t.2)
  (Fraction.mul_equiv (Fraction.abs_of_nonnegative _ (Int.le_of_lt c.den_pos))
    (Fraction.equiv_refl _))
apply Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _)
  (Fraction.mul_equiv (Fraction.equiv_refl _) hv))
simp only [distance, rescaleState, inverse, Fraction.equiv, Fraction.add, Fraction.mul,
  Int.add_mul, Int.mul_add]
ac_nf

theorem norm_unit_calibration (s : Point × Point) :
  Fraction.equiv (norm (Fraction.ofInt 1) s) (stateNorm s) := by
simp only [norm, stateNorm, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
  Int.mul_one, Int.one_mul]

/-- The calibrated distance is a finite rescaling of the ordinary coordinate gauge. -/
theorem distance_le_uncalibrated (tau : Fraction) (ht : 0 < tau.num)
  (s t : Point × Point) :
  Fraction.le (distance tau s t)
  (Fraction.mul (Fraction.add (Fraction.ofInt 1) tau) (stateDistance s t)) := by
let C := Fraction.add (Fraction.ofInt 1) tau
have hone : Fraction.le (Fraction.ofInt 1) C :=

```

```

    Fraction.le_add_nonnegative _ _ (Int.le_of_lt ht)
  have htau : Fraction.le tau C := Fraction.le_equiv_right
    (Fraction.le_add_nonnegative tau (Fraction.ofInt 1) (by decide))
    (Fraction.add_comm tau (Fraction.ofInt 1))
  have hp := Fraction.mul_le_mul_nonnegative hone (pointDistance s.1 t.1)
    (pointNorm_nonnegative _)
  have hv := Fraction.mul_le_mul_nonnegative htau (pointDistance s.2 t.2)
    (pointNorm_nonnegative _)
  have hsum := Fraction.add_le_add hp hv
  have hl : Fraction.equiv
    (Fraction.add (Fraction.mul (Fraction.ofInt 1) (pointDistance s.1 t.1))
      (Fraction.mul tau (pointDistance s.2 t.2))) (distance tau s t) := by
    simp only [distance, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.one_mul, Int.mul_one]
  have hr : Fraction.equiv
    (Fraction.add (Fraction.mul C (pointDistance s.1 t.1))
      (Fraction.mul C (pointDistance s.2 t.2)))
    (Fraction.mul C (stateDistance s t)) :=
    Fraction.equiv_symm (Fraction.mul_add C _ _)
  exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm hl) hsum) hr

theorem uncalibrated_le_distance (tau : Fraction) (ht : 0 < tau.num)
  (s t : Point × Point) :
  Fraction.le (stateDistance s t)
    (Fraction.mul (Fraction.add (Fraction.ofInt 1) (inverse tau ht))
      (distance tau s t)) := by
  let C := Fraction.add (Fraction.ofInt 1) (inverse tau ht)
  have hone : Fraction.le (Fraction.ofInt 1) C :=
    Fraction.le_add_nonnegative _ _ (Int.le_of_lt tau.den_pos)
  have hct : Fraction.equiv (Fraction.mul C tau)
    (Fraction.add tau (Fraction.ofInt 1)) := by
    simp only [C, inverse, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  have hvt : Fraction.le (Fraction.ofInt 1) (Fraction.mul C tau) :=
    Fraction.le_equiv_right (Fraction.le_add_nonnegative _ tau (Int.le_of_lt ht))
    (Fraction.equiv_symm (Fraction.equiv_trans hct (Fraction.add_comm _ _)))
  have hp := Fraction.mul_le_mul_nonnegative hone (pointDistance s.1 t.1)
    (pointNorm_nonnegative _)
  have hv := Fraction.mul_le_mul_nonnegative hvt (pointDistance s.2 t.2)
    (pointNorm_nonnegative _)
  have hsum := Fraction.add_le_add hp hv
  have hl : Fraction.equiv
    (Fraction.add (Fraction.mul (Fraction.ofInt 1) (pointDistance s.1 t.1))
      (Fraction.mul (Fraction.ofInt 1) (pointDistance s.2 t.2)))
    (stateDistance s t) := by
    simp only [stateDistance, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.one_mul, Int.mul_one]
  have hr : Fraction.equiv
    (Fraction.add (Fraction.mul C (pointDistance s.1 t.1))
      (Fraction.mul (Fraction.mul C tau) (pointDistance s.2 t.2)))
    (Fraction.mul C (distance tau s t)) := by
    simp only [distance, Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf
  exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (Fraction.equiv_symm hl) hsum) hr

def CalibratedCauchy (tau : Fraction) (a : Nat → Point × Point) : Prop :=
  ∀ eps : Fraction, 0 < eps.num → ∃ N : Nat, ∀ m n : Nat,
    N ≤ m → N ≤ n → Fraction.lt (distance tau (a m) (a n)) eps

theorem calibrated_tolerance (tau eps : Fraction) (ht : 0 < tau.num)
  (s t : Point × Point)
  (hd : Fraction.lt (stateDistance s t)
    (HarmonicTimeRealization.factorDelta (Fraction.add (Fraction.ofInt 1) tau) eps
      (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt ht)))) :
  Fraction.lt (distance tau s t) eps := by
  let C := Fraction.add (Fraction.ofInt 1) tau
  have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt ht)

```

```

have hs := HarmonicTimeRealization.factor_control C eps (stateDistance s t) hC
  (Fraction.nonnegative_add _ _ (pointNorm_nonnegative _) (pointNorm_nonnegative _)) hd
exact Fraction.magnitudes.lt_of_le_lt (distance_le_uncalibrated tau ht s t)
  (Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (Fraction.equiv_symm (Fraction.mul_comm (stateDistance s t) C))) hs)

theorem unit_tolerance (tau eps : Fraction) (ht : 0 < tau.num)
  (s t : Point × Point)
  (hd : Fraction.lt (distance tau s t)
    (HarmonicTimeRealization.factorDelta
      (Fraction.add (Fraction.ofInt 1) (inverse tau ht)) eps
      (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)))) :
  Fraction.lt (stateDistance s t) eps := by
let C := Fraction.add (Fraction.ofInt 1) (inverse tau ht)
have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)
have hs := HarmonicTimeRealization.factor_control C eps (distance tau s t) hC
  (Fraction.nonnegative_add _ _ (pointNorm_nonnegative _)
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (pointNorm_nonnegative _))) hd
exact Fraction.magnitudes.lt_of_le_lt (uncalibrated_le_distance tau ht s t)
  (Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (Fraction.equiv_symm (Fraction.mul_comm (distance tau s t) C))) hs)

theorem distance_unit_calibration (s t : Point × Point) :
  Fraction.equiv (distance (Fraction.ofInt 1) s t) (stateDistance s t) := by
simp only [distance, stateDistance, Fraction.equiv, Fraction.add, Fraction.mul,
  Fraction.ofInt, Int.one_mul, Int.mul_one]

theorem cauchy_calibration_iff (tau : Fraction) (ht : 0 < tau.num)
  (a : Nat → Point × Point) :
  CalibratedCauchy tau a ↔ CalibratedCauchy (Fraction.ofInt 1) a := by
constructor
· intro ha eps heps
  let C := Fraction.add (Fraction.ofInt 1) (inverse tau ht)
  have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide)
    (Int.le_of_lt tau.den_pos)
  obtain (N, hN) := ha (HarmonicTimeRealization.factorDelta C eps hC)
    (HarmonicTimeRealization.factorDelta_positive C eps hC heps)
  exact (N, fun m n hm hn => Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (distance_unit_calibration (a m) (a n)))
    (unit_tolerance tau eps ht (a m) (a n) (hN m n hm hn)))
· intro ha eps heps
  let C := Fraction.add (Fraction.ofInt 1) tau
  have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt ht)
  obtain (N, hN) := ha (HarmonicTimeRealization.factorDelta C eps hC)
    (HarmonicTimeRealization.factorDelta_positive C eps hC heps)
  exact (N, fun m n hm hn => calibrated_tolerance tau eps ht (a m) (a n)
    (Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv (Fraction.equiv_symm (distance_unit_calibration (a m) (a n))))
      (hN m n hm hn)))

def CalibratedEquivalent (tau : Fraction) (a b : Nat → Point × Point) : Prop :=
  ∀ eps : Fraction, 0 < eps.num → ∃ N : Nat, ∀ n : Nat,
    N ≤ n → Fraction.lt (distance tau (a n) (b n)) eps

/-- The actual completed value relation is unchanged by a fixed calibration. -/
theorem nameEquiv_calibration_iff (tau : Fraction) (ht : 0 < tau.num)
  (a b : HarmonicDyadic.EndpointCauchyName) :
  CalibratedEquivalent tau a.approx b.approx ↔ CauchyValues.NameEquiv a b := by
constructor
· intro ha eps heps
  let C := Fraction.add (Fraction.ofInt 1) (inverse tau ht)
  have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide)
    (Int.le_of_lt tau.den_pos)
  obtain (N, hN) := ha (HarmonicTimeRealization.factorDelta C eps hC)
    (HarmonicTimeRealization.factorDelta_positive C eps hC heps)
  exact (N, fun n hn => unit_tolerance tau eps ht (a.approx n) (b.approx n) (hN n hn))
· intro ha eps heps
  let C := Fraction.add (Fraction.ofInt 1) tau
  have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt ht)
  obtain (N, hN) := ha (HarmonicTimeRealization.factorDelta C eps hC)

```



```

    (HarmonicTimeRealization.factorDelta_positive C eps hC heps)
  exact (N, fun n hn => calibrated_tolerance tau eps ht
    (a.approx n) (b.approx n) (hN n hn))

theorem amplification_nonnegative (tau h L : Fraction)
  (ht : 0 < tau.num) (hL : 0 ≤ L.num) :
  0 ≤ (amplification tau h L ht).num :=
  Fraction.nonnegative_mul
    (Fraction.nonnegative_add _ _ (by decide)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
        (Int.le_of_lt tau.den_pos)))
    (Fraction.nonnegative_add _ _ (by decide)
      (Fraction.nonnegative_mul _ _
        (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) (Int.le_of_lt ht)) hL))

theorem one_le_amplification (tau h L : Fraction)
  (ht : 0 < tau.num) (hL : 0 ≤ L.num) :
  Fraction.le (Fraction.ofInt 1) (amplification tau h L ht) := by
  have hp := Fraction.le_add_nonnegative (Fraction.ofInt 1)
    (Fraction.mul h.abs (inverse tau ht))
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
      (Int.le_of_lt tau.den_pos))
  have hq := Fraction.le_add_nonnegative (Fraction.ofInt 1)
    (Fraction.mul (Fraction.mul h.abs tau) L)
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) (Int.le_of_lt ht)) hL
  have hfirst := Fraction.mul_le_mul_nonnegative_left hq
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs (inverse tau ht)))
    (Fraction.nonnegative_add _ _ (by decide)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
        (Int.le_of_lt tau.den_pos)))
  have hp' : Fraction.le (Fraction.ofInt 1)
    (Fraction.mul
      (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs (inverse tau ht)))
      (Fraction.ofInt 1)) := Fraction.le_equiv_right hp (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one])
  exact Fraction.magnitudes.le_trans hp' hfirst

def Window (tau h L : Fraction) (ht : 0 < tau.num) (n : Nat) : Prop :=
  Fraction.le (Fraction.mul (Fraction.ofInt (n : Int))
    (Fraction.mul h.abs (rate tau L ht))) (1, 2, by decide)

theorem rate_nonnegative (tau L : Fraction) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) : 0 ≤ (rate tau L ht).num :=
  Fraction.nonnegative_add _ _ (Int.le_of_lt tau.den_pos)
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) hL)

theorem window_mono (tau h L : Fraction) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) (n N : Nat) (hn : n ≤ N) (hs : Window tau h L ht N) :
  Window tau h L ht n := by
  have hc : Fraction.le (Fraction.ofInt (n : Int)) (Fraction.ofInt (N : Int)) := by
    simp only [Fraction.le, Fraction.ofInt, Int.mul_one] using
      (show (n : Int) ≤ (N : Int) by omega)
  exact Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative hc (Fraction.mul h.abs (rate tau L ht))
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
        (rate_nonnegative tau L ht hL))) hs

theorem window_of_elapsed (tau h T L : Fraction) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) (n : Nat)
  (he : Fraction.le (Fraction.mul (Fraction.ofInt (n : Int)) h.abs) T)
  (hs : Fraction.le (Fraction.mul T (rate tau L ht)) (1, 2, by decide)) :
  Window tau h L ht n := by
  have hm := Fraction.mul_le_mul_nonnegative he (rate tau L ht)
    (rate_nonnegative tau L ht hL)
  exact Fraction.le_equiv_left
    (Fraction.equiv_symm (Fraction.mul_assoc (Fraction.ofInt (n : Int)) h.abs (rate tau L ht)))
    (Fraction.magnitudes.le_trans hm hs)

/-- The factors are h/tau and tau*h*L; their product is h²*L. -/

```

```

theorem amplification_expansion (tau h L : Fraction) (ht : 0 < tau.num) :
  Fraction.equiv (amplification tau h L ht)
    (Fraction.add (Fraction.ofInt 1)
      (Fraction.add (Fraction.mul h.abs (rate tau L ht))
        (Fraction.mul (Fraction.mul h.abs h.abs) L))) := by
  simp only [amplification, rate, inverse, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

theorem amplification_power_le_two (tau h L : Fraction)
  (ht : 0 < tau.num) (hL : 0 ≤ L.num) (n : Nat)
  (hs : Window tau h L ht n) :
  Fraction.le (HarmonicAccumulation.fpower (amplification tau h L ht) n)
    (Fraction.ofInt 2) := by
  apply FiniteFactorProducts.repeated_pair_le_two
    (Fraction.mul h.abs (inverse tau ht))
    (Fraction.mul (Fraction.mul h.abs tau) L) n
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
      (Int.le_of_lt tau.den_pos))
    (Fraction.nonnegative_mul _ _
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) (Int.le_of_lt ht)) hL)
  apply Fraction.le_equiv_left (b := Fraction.mul (Fraction.ofInt (n : Int))
    (Fraction.mul h.abs (rate tau L ht))) _ hs
  simp only [rate, Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

/-- Actual sampled schedules, starting at the same state, accumulate the
calibrated sample discrepancy without assuming a motion error estimate. -/
theorem run_distance_le_source (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hc : comparisonContract a b L E) :
  (n : Nat) → Fraction.le
    (distance tau (BoundedIteration.run a h s n) (BoundedIteration.run b h s n))
    (FiniteRecurrence.sourceBudget (amplification tau h L ht)
      (Fraction.mul (Fraction.mul tau h.abs) E) n)
  | 0 => by
    apply Fraction.le_of_equiv
    simp only [distance, BoundedIteration.run, FiniteRecurrence.sourceBudget]
    exact Fraction.equiv_trans
      (Fraction.add_equiv (pointDistance_self_zero s.1)
        (Fraction.mul_equiv (Fraction.equiv_refl tau) (pointDistance_self_zero s.2)))
      (by simp [FiniteRecurrence.sourceBudget, Fraction.equiv, Fraction.add,
        Fraction.mul, Fraction.ofInt])
  | n+1 => by
    have hstep := cell_amplification tau ht a b h L E
      (BoundedIteration.run a h s n) (BoundedIteration.run b h s n) hL hc
    have hnext := Fraction.add_le_add_right
      (Fraction.mul_le_mul_nonnegative_left
        (run_distance_le_source tau ht a b h L E s hL hc n)
        (amplification tau h L ht) (amplification_nonnegative tau h L ht hL))
      (Fraction.mul (Fraction.mul tau h.abs) E)
    exact Fraction.magnitudes.le_trans hstep hnext

theorem run_uniform_discrepancy (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hc : comparisonContract a b L E)
  (n : Nat) (hs : Window tau h L ht n) :
  Fraction.le
    (distance tau (BoundedIteration.run a h s n) (BoundedIteration.run b h s n))
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (Fraction.mul (Fraction.mul tau h.abs) E)) := by
  let K := amplification tau h L ht
  let S := Fraction.mul (Fraction.mul tau h.abs) E
  have hK := amplification_nonnegative tau h L ht hL
  have hS : 0 ≤ S.num := Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Fraction.abs_num_nonnegative h)) hE
  have hp := amplification_power_le_two tau h L ht hL n hs
  exact Fraction.magnitudes.le_trans
    (run_distance_le_source tau ht a b h L E s hL hc n)
    (FiniteRecurrence.sourceBudget_two_count K S n hK hS)

```

```

(one_le_amplification tau h L ht hL) hp)

/-- Time-unit rescaling preserves the actual-family Cauchy condition. -/
theorem cauchy_rescale (c tau : Fraction) (hc : 0 < c.num)
  (a : Nat → Point × Point) :
  CalibratedCauchy (Fraction.mul c tau) (fun n => rescaleState c hc (a n)) ↔
  CalibratedCauchy tau a := by
  constructor
  · intro ha eps heps
    obtain ⟨N,hN⟩ := ha eps heps
    refine ⟨N,fun m n hm hn => ?_⟩
    exact Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv (Fraction.equiv_symm (distance_rescale c tau (a m) (a n) hc)))
      (hN m n hm hn)
  · intro ha eps heps
    obtain ⟨N,hN⟩ := ha eps heps
    refine ⟨N,fun m n hm hn => ?_⟩
    exact Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv (distance_rescale c tau (a m) (a n) hc))
      (hN m n hm hn)

theorem window_rescale_iff (c tau h L : Fraction)
  (hc : 0 < c.num) (ht : 0 < tau.num) (n : Nat) :
  Window (Fraction.mul c tau) (Fraction.mul c h) (rescaleConstant c L hc)
  (Int.mul_pos hc ht) n ↔ Window tau h L ht n := by
  have ha := Fraction.equiv_trans (Fraction.abs_mul c h)
    (Fraction.mul_equiv (Fraction.abs_of_nonnegative c (Int.le_of_lt hc))
    (Fraction.equiv_refl h.abs))
  have hw := Fraction.equiv_trans
    (Fraction.mul_equiv ha (Fraction.equiv_refl _))
    (window_rescale c tau h.abs L hc ht)
  have he := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (n : Int))) hw
  constructor
  · intro hs
    exact Fraction.le_equiv_left (Fraction.equiv_symm he) hs
  · intro hs
    exact Fraction.le_equiv_left he hs

private def two : Fraction := Fraction.ofInt 2
private def three : Fraction := Fraction.ofInt 3
private def half : Fraction := (1,2,by decide)
private def quarter : Fraction := (1,4,by decide)
private def sampleState : Point × Point :=
  ((Fraction.ofInt 1,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 1))

theorem rescaling_control :
  Fraction.equiv (norm two sampleState)
    (norm (Fraction.mul three two) (rescaleState three (by decide) sampleState)) ∧
  Fraction.equiv (amplification two quarter (Fraction.ofInt 1) (by decide))
    (amplification (Fraction.mul three two) (Fraction.mul three quarter)
    (rescaleConstant three (Fraction.ofInt 1) (by decide)) (by decide)) := by decide

theorem uncalibrated_norm_changes_control :
  ¬ Fraction.equiv (stateNorm sampleState)
    (stateNorm (rescaleState two (by decide) sampleState)) := by decide

theorem zero_lipschitz_control :
  Fraction.equiv (amplification two half (Fraction.ofInt 0) (by decide))
    (5,4,by decide) := by decide

end NewtonLimitDynamics.Polygon.TimeCalibration

```

BarrowLib/Polygon/CalibratedGrowth.lean, 9 theorems, 215 lines

```
import BarrowLib.Polygon.TimeCalibration
```

```

/-! Finite calibrated bounds from linear growth of an arbitrary rational map.
No arrival bound or confinement premise is supplied. The recurrence uses the
existing finite amplification and source-budget proofs, rather than an ODE or
an integral inequality. -/

```

```

namespace NewtonLimitDynamics.Polygon.CalibratedGrowth
open NewtonLimitDynamics TimeSubdivision PointBounds FiniteEstimates
open TimeCalibration HarmonicAccumulation

def Growth (a : Point → Point) (L E : Fraction) : Prop :=
  ∀ p, Fraction.le (pointNorm (a p)) (Fraction.add (Fraction.mul L (pointNorm p)) E)

theorem cell_norm_bound_at (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hg : Fraction.le (pointNorm (a (cell a h s).1))
    (Fraction.add (Fraction.mul L (pointNorm (cell a h s).1)) E)) :
  Fraction.le (norm tau (cell a h s))
    (Fraction.add (Fraction.mul (amplification tau h L ht) (norm tau s))
      (Fraction.mul (Fraction.mul tau h.abs) E)) := by
  have hp := cell_position_growth a h s
  have ha := Fraction.magnitudes.le_trans hg
  (Fraction.add_le_add_right (Fraction.mul_le_mul_nonnegative_left hp L hL) E)
  have hv := cell_velocity_growth a h s
  have hb := Fraction.add_le_add hp
  (Fraction.mul_le_mul_nonnegative_left hv tau (Int.le_of_lt ht))
  have hc := component_amplification tau (pointNorm s.1) (pointNorm s.2) h.abs L E
  ht (pointNorm_nonnegative _) (pointNorm_nonnegative _)
  (Fraction.abs_num_nonnegative h) hL
  simp only [norm, TimeCalibration.amplification,
    Fraction.abs_eq_of_nonnegative h.abs (Fraction.abs_num_nonnegative h)] using
    (Fraction.magnitudes.le_trans hb hc)

theorem cell_norm_bound (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hg : Growth a L E) :
  Fraction.le (norm tau (cell a h s))
    (Fraction.add (Fraction.mul (amplification tau h L ht) (norm tau s))
      (Fraction.mul (Fraction.mul tau h.abs) E)) :=
  cell_norm_bound_at tau ht a h L E s hL (hg _)

theorem run_norm_budget_at (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) :
  (n : Nat) →
  (hg : ∀ k, k < n → Fraction.le (pointNorm (a (BoundedIteration.run a h s (k+1)).1))
    (Fraction.add (Fraction.mul L (pointNorm (BoundedIteration.run a h s (k+1)).1)) E)) →
  Fraction.le (norm tau (BoundedIteration.run a h s n))
    (Fraction.add (Fraction.mul (fpower (amplification tau h L ht) n) (norm tau s))
      (FiniteRecurrence.sourceBudget (amplification tau h L ht)
        (Fraction.mul (Fraction.mul tau h.abs) E) n))
  | 0, _ => by
    apply Fraction.le_of_equiv
    simp only [BoundedIteration.run, fpower, FiniteRecurrence.sourceBudget,
      Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.zero_mul, Int.mul_zero, Int.one_mul, Int.mul_one, Int.add_zero]
  | n+1, hg => by
    have hc := cell_norm_bound_at tau ht a h L E (BoundedIteration.run a h s n) hL (hg n (by omega))
    have hi := Fraction.add_le_add_right
      (Fraction.mul_le_mul_nonnegative_left
        (run_norm_budget_at tau ht a h L E s hL n (fun k hk => hg k (by omega)))
        (amplification tau h L ht) (amplification_nonnegative tau h L ht hL))
      (Fraction.mul (Fraction.mul tau h.abs) E)
    apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans hc hi)
    simp only [FiniteRecurrence.sourceBudget, fpower, Fraction.equiv, Fraction.add,
      Fraction.mul, Int.add_mul, Int.mul_add]
    ac_nf

theorem run_norm_budget (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hg : Growth a L E) :
  (n : Nat) → Fraction.le (norm tau (BoundedIteration.run a h s n))
    (Fraction.add (Fraction.mul (fpower (amplification tau h L ht) n) (norm tau s))
      (FiniteRecurrence.sourceBudget (amplification tau h L ht)

```

```

      (Fraction.mul (Fraction.mul tau h.abs) E) n)):=
  fun n => run_norm_budget_at tau ht a h L E s hL n (fun _ => hg _)

def cap (tau T E : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul (Fraction.ofInt 2)
    (Fraction.add (norm tau s) (Fraction.mul (Fraction.mul tau T) E))

/-- The derived length cap is invariant under a positive time-unit change. -/
theorem cap_rescale (c tau T E : Fraction) (s : Point × Point) (hc : 0 < c.num) :
  Fraction.equiv
    (cap (Fraction.mul c tau) (Fraction.mul c T) (rescaleConstant c E hc)
      (rescaleState c hc s)) (cap tau T E s) := by
  apply Fraction.mul_equiv (Fraction.equiv_refl _)
  apply Fraction.add_equiv (norm_rescale c tau s hc)
  simp only [rescaleConstant, inverse, Fraction.equiv, Fraction.mul]
  ac_nf

theorem cap_nonnegative (tau T E : Fraction) (s : Point × Point)
  (ht : 0 ≤ tau.num) (hT : 0 ≤ T.num) (hE : 0 ≤ E.num) :
  0 ≤ (cap tau T E s).num :=
  Fraction.nonnegative_mul _ _ (by decide)
    (Fraction.nonnegative_add _ _ (norm_nonnegative tau s ht)
      (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ ht hT) hE))

/-- An outer ball large enough for the finite drift-and-kick budget. -/
def driftCap (tau T E : Fraction) (s : Point × Point) (ht : 0 < tau.num) : Fraction :=
  let M := cap tau T E s
  Fraction.add M (Fraction.mul T (Fraction.mul M (inverse tau ht)))

/-- The calibrated window closes the ball budget before any force sample.
Only arithmetic and initial-state magnitudes enter this inequality. -/
theorem driftCap_budget (tau T L E : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) (hT : 0 ≤ T.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hw : Fraction.le (Fraction.mul T (rate tau L ht)) (1,2,by decide)) :
  Fraction.le
    (Fraction.add (pointNorm s.1)
      (Fraction.mul T (Fraction.add (pointNorm s.2)
        (Fraction.mul T (Fraction.add (Fraction.mul L (driftCap tau T E s ht)) E))))))
    (driftCap tau T E s ht) := by
  let x := Fraction.mul T (inverse tau ht)
  let y := Fraction.mul (Fraction.mul T tau) L
  have hx0 : 0 ≤ x.num := Fraction.nonnegative_mul _ _ hT (Int.le_of_lt tau.den_pos)
  have hy0 : 0 ≤ y.num := Fraction.nonnegative_mul _ _ hL
    (Fraction.nonnegative_mul _ _ hT (Int.le_of_lt ht))
  have hsum : Fraction.le (Fraction.add x y) (1,2,by decide) := by
    apply Fraction.le_equiv_left (b := Fraction.mul T (rate tau L ht)) _ hw
    simp only [x,y,rate,Fraction.equiv,Fraction.add,Fraction.mul,Int.add_mul,Int.mul_add]
    ac_nf
  have hx := Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative x y hy0) hsum
  have hy := Fraction.magnitudes.le_trans
    (Fraction.le_equiv_right (Fraction.le_add_nonnegative y x hx0) (Fraction.add_comm y x)) hsum
  have hxy := Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative hx y hy0)
    (Fraction.mul_le_mul_nonnegative_left hy (1,2,by decide) (by decide))
  have hquad : Fraction.le (Fraction.mul (Fraction.mul T T) L) (1,2,by decide) := by
    apply Fraction.le_equiv_left (b := Fraction.mul x y)
    · simp only [x,y,inverse,Fraction.equiv,Fraction.mul]
      ac_nf
    · exact Fraction.magnitudes.le_trans hxy
      (show Fraction.le (Fraction.mul (1,2,by decide) (1,2,by decide)) (1,2,by decide) from by
        change (1:Int)*2 ≤ 1*4
        decide)
  let R := driftCap tau T E s ht
  let F := Fraction.add (pointNorm s.1)
    (Fraction.add (Fraction.mul T (pointNorm s.2)) (Fraction.mul (Fraction.mul T T) E))
  let extra := Fraction.add (Fraction.mul tau (pointNorm s.2))
    (Fraction.add (Fraction.mul (Fraction.mul T (pointNorm s.1)) (inverse tau ht))
      (Fraction.mul (Fraction.mul tau T) E))
  have hextra : 0 ≤ extra.num := Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (pointNorm_nonnegative _))

```

```

(Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ hT (pointNorm_nonnegative _))
    (Int.le_of_lt tau.den_pos)))
(Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) hT) hE))
have hM := cap_nonnegative tau T E s (Int.le_of_lt ht) hT hE
have hR : 0 ≤ R.num := Fraction.nonnegative_add _ _ hM
(Fraction.nonnegative_mul _ _ hT (Fraction.nonnegative_mul _ _ hM (Int.le_of_lt tau.den_pos)))
have hF : Fraction.le F R.half := by
  apply Fraction.le_equiv_right (Fraction.le_add_nonnegative F extra hextra)
  simp only [F,extra,R,driftCap,cap,norm,inverse,Fraction.equiv,Fraction.half,
    Fraction.add,Fraction.mul,Fraction.ofInt,Int.add_mul,Int.mul_add]
  simp only [show (2:Int)=1+1 by rfl,Int.add_mul,Int.mul_add,Int.one_mul,Int.mul_one]
  ac_nf
have hQ : Fraction.le (Fraction.mul (Fraction.mul (Fraction.mul T T) L) R) R.half := by
  apply Fraction.le_equiv_right (Fraction.mul_le_mul_nonnegative hquad R hR)
  simp only [Fraction.equiv,Fraction.mul,Fraction.half]
  ac_nf
apply Fraction.le_equiv_left (b := Fraction.add F (Fraction.mul (Fraction.mul (Fraction.mul T T) L) R))
· simp only [F,R,Fraction.equiv,Fraction.add,Fraction.mul,Int.add_mul,Int.mul_add]
  ac_nf
· exact Fraction.le_equiv_right (Fraction.add_le_add hF hQ) (Fraction.half_add_self R)

/-- Uniform actual-state bound on a calibrated window, for every finite mesh. -/
theorem run_norm_le_cap_at (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h T L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (n : Nat)
  (hg : ∀ k, k < n → Fraction.le (pointNorm (a (BoundedIteration.run a h s (k+1)).1))
    (Fraction.add (Fraction.mul L (pointNorm (BoundedIteration.run a h s (k+1)).1)) E)) (he :
    → Fraction.le (Fraction.mul (Fraction.ofInt (n : Int)) h.abs) T)
  (hw : Fraction.le (Fraction.mul T (rate tau L ht)) (1,2,by decide)) :
  Fraction.le (norm tau (BoundedIteration.run a h s n)) (cap tau T E s) := by
have hp := amplification_power_le_two tau h L ht hL n
(window_of_elapsed tau h T L ht hL n he hw)
have hK := amplification_nonnegative tau h L ht hL
have hC : 0 ≤ (Fraction.mul (Fraction.mul tau h.abs) E).num :=
  Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Fraction.abs_num_nonnegative h)) hE
have hs := FiniteRecurrence.sourceBudget_two_count _ _ n hK hC
(one_le_amplification tau h L ht hL) hp
have hb := Fraction.add_le_add
  (Fraction.mul_le_mul_nonnegative hp (norm tau s) (norm_nonnegative _ _ (Int.le_of_lt ht))) hs
have heq : Fraction.equiv
  (Fraction.mul (Fraction.ofInt (2*(n:Int))) (Fraction.mul (Fraction.mul tau h.abs) E))
  (Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (Fraction.mul tau (Fraction.mul (Fraction.ofInt (n:Int)) h.abs)) E)) := by
  simp only [Fraction.equiv,Fraction.mul,Fraction.ofInt,Int.mul_one,Int.one_mul]
  ac_nf
have hsource := Fraction.le_equiv_left heq
  (Fraction.mul_le_mul_nonnegative_left
    (Fraction.mul_le_mul_nonnegative
      (Fraction.mul_le_mul_nonnegative_left he tau (Int.le_of_lt ht)) E hE)
    (Fraction.ofInt 2) (by decide))
apply Fraction.magnitudes.le_trans (run_norm_budget_at tau ht a h L E s hL n hg)
apply Fraction.magnitudes.le_trans hb
apply Fraction.le_equiv_right (Fraction.add_le_add_left hsource
  (Fraction.mul (Fraction.ofInt 2) (norm tau s)))
exact Fraction.equiv_symm (Fraction.mul_add _ _ _)

theorem run_norm_le_cap (tau : Fraction) (ht : 0 < tau.num)
  (a : Point → Point) (h T L E : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hg : Growth a L E)
  (n : Nat) (he : Fraction.le (Fraction.mul (Fraction.ofInt (n : Int)) h.abs) T)
  (hw : Fraction.le (Fraction.mul T (rate tau L ht)) (1,2,by decide)) :
  Fraction.le (norm tau (BoundedIteration.run a h s n)) (cap tau T E s) :=
  run_norm_le_cap_at tau ht a h T L E s hL hE n (fun _ => hg _) he hw

end NewtonLimitDynamics.Polygon.CalibratedGrowth

```

BarrowLib/Polygon/EquivalentDuration.lean, 12 theorems, 217 lines

```

import BarrowLib.Polygon.ConvexCover
import BarrowLib.Polygon.TimeCalibration

/-! Actual triangular cells at value-equivalent rational durations. A sampled
map may distinguish point representatives; comparison retains its additive E. -/

namespace NewtonLimitDynamics.Polygon.EquivalentDuration

open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates ConvexCover

theorem pointDistance_zero_of_equiv {p q : Point} (hpq : pointEquiv p q) :
  Fraction.equiv (pointDistance p q) (Fraction.ofInt 0) := by
  have he : pointEquiv (pointSub p q) (pointSub p p) :=
    pointSub_congr (Fraction.equiv_refl _, Fraction.equiv_refl _)
    (pointEquiv_symm hpq)
  exact Fraction.equiv_trans (pointNorm_equiv he) (pointSub_self_zero p)

theorem cell_position_equiv {d e : Fraction} (a : Point → Point)
  (s : Point × Point) (hde : Fraction.equiv d e) :
  pointEquiv (cell a d s).1 (cell a e s).1 :=
  pointAdd_congr (Fraction.equiv_refl _, Fraction.equiv_refl _)
  (pointScale_ratio_congr hde
    (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem cell_position_perturbation (a b : Point → Point) (d e : Fraction)
  (s t : Point × Point) (hde : Fraction.equiv d e) :
  Fraction.le (pointDistance (cell a d s).1 (cell b e t).1)
    (Fraction.add (pointDistance s.1 t.1)
      (Fraction.mul d.abs (pointDistance s.2 t.2))) := by
  have hpos := cell_position_equiv b t (Fraction.equiv_symm hde)
  exact Fraction.le_equiv_left
    (pointDistance_equiv (Fraction.equiv_refl _, Fraction.equiv_refl _) hpos)
    (FiniteEstimates.cell_position_perturbation a b d s t)

theorem cell_velocity_perturbation_at (a b : Point → Point) (d e L E : Fraction)
  (s t : Point × Point) (hde : Fraction.equiv d e)
  (hc : Fraction.le (pointDistance (a (cell a d s).1) (b (cell b e t).1))
    (Fraction.add (Fraction.mul L
      (pointDistance (cell a d s).1 (cell b e t).1)) E)) :
  Fraction.le (pointDistance (cell a d s).2 (cell b e t).2)
    (Fraction.add (pointDistance s.2 t.2)
      (Fraction.mul d.abs
        (Fraction.add (Fraction.mul L
          (pointDistance (cell a d s).1 (cell b e t).1)) E))) := by
  let A := a (cell a d s).1
  let B := b (cell b e t).1
  have hs : pointEquiv (pointScale e B) (pointScale d B) :=
    pointScale_ratio_congr (Fraction.equiv_symm hde)
    (Fraction.equiv_refl _, Fraction.equiv_refl _)
  have hv : pointEquiv (cell b e t).2 (pointAdd t.2 (pointScale d B)) :=
    pointAdd_congr (Fraction.equiv_refl _, Fraction.equiv_refl _) hs
  have hb := difference_add_bound s.2 t.2 (pointScale d A) (pointScale d B)
  have hm := Fraction.mul_le_mul_nonnegative_left
    hc d.abs (Fraction.abs_num_nonnegative d)
  have hscaled := Fraction.le_equiv_right hb
    (Fraction.add_equiv (Fraction.equiv_refl _) (difference_scale d A B))
  exact Fraction.le_equiv_left
    (pointDistance_equiv (Fraction.equiv_refl _, Fraction.equiv_refl _) hv)
    (Fraction.magnitudes.le_trans hscaled (Fraction.add_le_add_left hm _))

theorem cell_velocity_perturbation (a b : Point → Point) (d e L E : Fraction)
  (s t : Point × Point) (hde : Fraction.equiv d e)
  (hc : comparisonContract a b L E) :
  Fraction.le (pointDistance (cell a d s).2 (cell b e t).2)
    (Fraction.add (pointDistance s.2 t.2)
      (Fraction.mul d.abs
        (Fraction.add (Fraction.mul L
          (pointDistance (cell a d s).1 (cell b e t).1)) E))) :=

```

```

cell_velocity_perturbation_at a b d e L E s t hde
(hc (cell a d s).1 (cell b e t).1)

/-- Both forces remain evaluated at the actual arrivals of their own duration. -/
theorem cell_amplification_at (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (d e L E : Fraction) (s t : Point × Point)
(hde : Fraction.equiv d e) (hL : 0 ≤ L.num)
(hc : Fraction.le (pointDistance (a (cell a d s).1) (b (cell b e t).1))
(Fraction.add (Fraction.mul L
(pointDistance (cell a d s).1 (cell b e t).1)) E)) :
Fraction.le (TimeCalibration.distance tau (cell a d s) (cell b e t))
(Fraction.add
(Fraction.mul (TimeCalibration.amplification tau d L ht)
(TimeCalibration.distance tau s t))
(Fraction.mul (Fraction.mul tau d.abs) E)) := by
have hp := cell_position_perturbation a b d e s t hde
have hv := cell_velocity_perturbation_at a b d e L E s t hde hc
have hvc := Fraction.magnitudes.le_trans hv
(Fraction.add_le_add_left
(Fraction.mul_le_mul_nonnegative_left
(Fraction.add_le_add_right
(Fraction.mul_le_mul_nonnegative_left hp L hL) E)
d.abs (Fraction.abs_num_nonnegative d)) (pointDistance s.2 t.2))
have hsum := Fraction.add_le_add hp
(Fraction.mul_le_mul_nonnegative_left hvc tau (Int.le_of_lt ht))
have hcomp := TimeCalibration.component_amplification tau (pointDistance s.1 t.1)
(pointDistance s.2 t.2) d.abs L E ht
(pointNorm_nonnegative _) (pointNorm_nonnegative _) (Fraction.abs_num_nonnegative d) hL
have hh : d.abs.abs = d.abs := by simp only [Fraction.abs, Int.natAbs_ofNat]
simpa only [TimeCalibration.distance, TimeCalibration.amplification, hh] using
Fraction.magnitudes.le_trans hsum hcomp

theorem cell_amplification (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (d e L E : Fraction) (s t : Point × Point)
(hde : Fraction.equiv d e) (hL : 0 ≤ L.num)
(hc : comparisonContract a b L E) :
Fraction.le (TimeCalibration.distance tau (cell a d s) (cell b e t))
(Fraction.add
(Fraction.mul (TimeCalibration.amplification tau d L ht)
(TimeCalibration.distance tau s t))
(Fraction.mul (Fraction.mul tau d.abs) E)) :=
cell_amplification_at tau ht a b d e L E s t hde hL
(hc (cell a d s).1 (cell b e t).1)

theorem run_distance_le_source_at (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (d e L E : Fraction) (s : Point × Point)
(hde : Fraction.equiv d e) (hL : 0 ≤ L.num) :
(n : Nat) →
(hc : ∀ k, k < n → Fraction.le
(pointDistance
(a (cell a d (BoundedIteration.run a d s k)).1)
(b (cell b e (BoundedIteration.run b e s k)).1))
(Fraction.add (Fraction.mul L
(pointDistance
(cell a d (BoundedIteration.run a d s k)).1
(cell b e (BoundedIteration.run b e s k)).1)) E)) →
Fraction.le
(TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
(FiniteRecurrence.sourceBudget (TimeCalibration.amplification tau d L ht)
(Fraction.mul (Fraction.mul tau d.abs) E) n)
| 0, _ => Fraction.le_of_equiv (TimeCalibration.distance_self_zero tau s)
| n+1, hc => by
have hstep := cell_amplification_at tau ht a b d e L E
(BoundedIteration.run a d s n) (BoundedIteration.run b e s n) hde hL
(hc n (by omega))
have hnext := Fraction.add_le_add_right
(Fraction.mul_le_mul_nonnegative_left
(run_distance_le_source_at tau ht a b d e L E s hde hL n
(fun k hk => hc k (by omega)))
(TimeCalibration.amplification tau d L ht)

```



```

      (TimeCalibration.amplification_nonnegative tau d L ht hL))
      (Fraction.mul (Fraction.mul tau d.abs) E)
      exact Fraction.magnitudes.le_trans hstep hnext

theorem run_distance_le_source (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (d e L E : Fraction) (s : Point × Point)
  (hde : Fraction.equiv d e) (hL : 0 ≤ L.num)
  (hc : comparisonContract a b L E) :
  (n : Nat) → Fraction.le
    (TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
    (FiniteRecurrence.sourceBudget (TimeCalibration.amplification tau d L ht)
      (Fraction.mul (Fraction.mul tau d.abs) E) n) :=
  fun n => run_distance_le_source_at tau ht a b d e L E s hde hL n
  (fun k _ => hc
    (cell a d (BoundedIteration.run a d s k)).1
    (cell b e (BoundedIteration.run b e s k)).1)

theorem run_uniform_error_at (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (d e L E : Fraction) (s : Point × Point)
  (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (n : Nat)
  (hs : TimeCalibration.Window tau d L ht n)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance
      (a (cell a d (BoundedIteration.run a d s k)).1)
      (b (cell b e (BoundedIteration.run b e s k)).1))
    (Fraction.add (Fraction.mul L
      (pointDistance
        (cell a d (BoundedIteration.run a d s k)).1
        (cell b e (BoundedIteration.run b e s k)).1)) E)) :
    Fraction.le
      (TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
      (Fraction.mul (Fraction.ofInt (2*(n : Int)))
        (Fraction.mul (Fraction.mul tau d.abs) E)) := by
  let K := TimeCalibration.amplification tau d L ht
  let S := Fraction.mul (Fraction.mul tau d.abs) E
  have hK := TimeCalibration.amplification_nonnegative tau d L ht hL
  have hS : 0 ≤ S.num := Fraction.nonnegative_mul
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Fraction.abs_num_nonnegative d)) hE
  have hp := TimeCalibration.amplification_power_le_two tau d L ht hL n hs
  exact Fraction.magnitudes.le_trans
    (run_distance_le_source_at tau ht a b d e L E s hde hL n hc)
    (FiniteRecurrence.sourceBudget_two_count K S n hK hS
      (TimeCalibration.one_le_amplification tau d L ht hL) hp)

theorem run_uniform_error (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (d e L E : Fraction) (s : Point × Point)
  (hde : Fraction.equiv d e) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hc : comparisonContract a b L E) (n : Nat)
  (hs : TimeCalibration.Window tau d L ht n) :
  Fraction.le
    (TimeCalibration.distance tau (BoundedIteration.run a d s n) (BoundedIteration.run b e s n))
    (Fraction.mul (Fraction.ofInt (2*(n : Int)))
      (Fraction.mul (Fraction.mul tau d.abs) E)) :=
  run_uniform_error_at tau ht a b d e L E s hde hL hE n hs
  (fun k _ => hc
    (cell a d (BoundedIteration.run a d s k)).1
    (cell b e (BoundedIteration.run b e s k)).1)

private def representativeMap (p : Point) : Point :=
  (Fraction.ofInt 0, if p.l.den = 1 then Fraction.ofInt 1 else Fraction.ofInt 2)
private def rest : Point × Point :=
  ((Fraction.ofInt 0, Fraction.ofInt 0), (Fraction.ofInt 0, Fraction.ofInt 0))

/-- Pure map control: equal duration values do not imply equal sample outputs. -/
theorem representative_sensitive_control :
  Fraction.equiv (Fraction.ofInt 1) (⟨2,2,by decide⟩ : Fraction) ∧
  Fraction.equiv
    (TimeCalibration.distance (Fraction.ofInt 2)
      (cell representativeMap (Fraction.ofInt 1) rest))

```

```

      (cell representativeMap (2,2,by decide) rest)) (Fraction.ofInt 2) ^
    - stateEquiv
      (cell representativeMap (Fraction.ofInt 1) rest)
      (cell representativeMap (2,2,by decide) rest) := by
    unfold stateEquiv
    decide

end NewtonLimitDynamics.Polygon.EquivalentDuration

```

BarrowLib/Polygon/CalibratedRefinement.lean, 13 theorems, 314 lines

```

import BarrowLib.Polygon.EquivalentDuration

/ -! Calibrated actual coarse/two-half-cell accumulation. All maps are rational
point maps; sample/arrival bounds are explicit finite premises. - /

namespace NewtonLimitDynamics.Polygon.CalibratedRefinement
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates FiniteAccumulation

-- The local defect is calculated on the actual coarse map.
def localSource (tau h L E B V : Fraction) : Fraction :=
  Fraction.add (Fraction.mul (Fraction.mul h h).abs B)
    (Fraction.mul tau (Fraction.mul h.abs
      (Fraction.add (Fraction.mul L (Fraction.mul h.abs V)) E)
      (Fraction.add (Fraction.mul L (Fraction.mul (Fraction.mul h h).abs B)) E))))

def blockSource (tau h L E B V : Fraction) (ht : 0 < tau.num) : Fraction :=
  Fraction.add
    (Fraction.add
      (Fraction.mul (TimeCalibration.amplification tau h L ht)
        (Fraction.mul (Fraction.mul tau h.abs) E))
      (Fraction.mul (Fraction.mul tau h.abs) E))
    (localSource tau h L E B V)

def blockFactor (tau h L : Fraction) (ht : 0 < tau.num) : Fraction :=
  Fraction.mul (TimeCalibration.amplification tau h L ht)
    (TimeCalibration.amplification tau h L ht)

theorem localSource_nonnegative (tau h L E B V : Fraction)
  (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) : 0 ≤ (localSource tau h L E B V).num :=
  Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) hB)
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
        (Fraction.nonnegative_add _ _
          (Fraction.nonnegative_add _ _
            (Fraction.nonnegative_mul _ _ hL
              (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h) hV)) hE)
            (Fraction.nonnegative_add _ _
              (Fraction.nonnegative_mul _ _ hL
                (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) hB)) hE))))))

theorem blockSource_nonnegative (tau h L E B V : Fraction)
  (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hB : 0 ≤ B.num) (hV : 0 ≤ V.num) : 0 ≤ (blockSource tau h L E B V ht).num := by
  have hS : 0 ≤ (Fraction.mul (Fraction.mul tau h.abs) E).num :=
  Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Fraction.abs_num_nonnegative h)) hE
  exact Fraction.nonnegative_add _ _
    (Fraction.nonnegative_add _ _
      (Fraction.nonnegative_mul _ _ (TimeCalibration.amplification_nonnegative tau h L ht hL) hS) hS)
    (localSource_nonnegative tau h L E B V ht hL hE hB hV)

theorem twoHalf_local_error_at (tau : Fraction) (ht : 0 < tau.num)
  (b : Point → Point) (h L E B V : Fraction) (t : Point × Point)
  (hL : 0 ≤ L.num)
  (hfirst : Fraction.le
    (pointDistance (b (cell b h t).1) (b (oneFull b h t).1)))

```

```

(Fraction.add (Fraction.mul L
  (pointDistance (cell b h t).1 (oneFull b h t).1)) E))
(hsecond : Fraction.le
  (pointDistance (b (twoHalf b h t).1) (b (oneFull b h t).1))
  (Fraction.add (Fraction.mul L
    (pointDistance (twoHalf b h t).1 (oneFull b h t).1)) E))
(hB : Fraction.le (pointNorm (b (cell b h t).1)) B)
(hV : Fraction.le (pointNorm t.2) V) :
Fraction.le (TimeCalibration.distance tau (twoHalf b h t) (oneFull b h t))
  (localSource tau h L E B V) := by
have hp := twoHalf_position_error b h B t hB
have hv := twoHalf_velocity_error_closed_at b h L E B t hL hB hfirst hsecond
have hvc := Fraction.add_le_add_right
  (Fraction.mul_le_mul_nonnegative_left
    (Fraction.mul_le_mul_nonnegative_left hv h.abs (Fraction.abs_num_nonnegative h)) L hL) E
have hvel := Fraction.mul_le_mul_nonnegative_left
  (Fraction.add_le_add_right hvc
    (Fraction.add (Fraction.mul L (Fraction.mul (Fraction.mul h h).abs B)) E))
  h.abs (Fraction.abs_num_nonnegative h)
exact Fraction.add_le_add hp
  (Fraction.mul_le_mul_nonnegative_left (Fraction.magnitudes.le_trans hv hvel)
    tau (Int.le_of_lt ht))

theorem twoHalf_local_error (tau : Fraction) (ht : 0 < tau.num)
  (b : Point → Point) (h L E B V : Fraction) (t : Point × Point)
  (hL : 0 ≤ L.num) (hlocal : comparisonContract b b L E)
  (hB : Fraction.le (pointNorm (b (cell b h t).1)) B)
  (hV : Fraction.le (pointNorm t.2) V) :
Fraction.le (TimeCalibration.distance tau (twoHalf b h t) (oneFull b h t))
  (localSource tau h L E B V) :=
twoHalf_local_error_at tau ht b h L E B V t hL
  (hlocal (cell b h t).1 (oneFull b h t).1)
  (hlocal (twoHalf b h t).1 (oneFull b h t).1) hB hV

/-- Only the four comparisons made by the actual paired block are needed. -/
structure BlockComparisons (a b : Point → Point) (h L E : Fraction)
  (s t : Point × Point) : Prop where
firstCross : Fraction.le
  (pointDistance (a (cell a h s).1) (b (cell b h t).1))
  (Fraction.add (Fraction.mul L
    (pointDistance (cell a h s).1 (cell b h t).1)) E)
secondCross : Fraction.le
  (pointDistance (a (twoHalf a h s).1) (b (twoHalf b h t).1))
  (Fraction.add (Fraction.mul L
    (pointDistance (twoHalf a h s).1 (twoHalf b h t).1)) E)
firstLocal : Fraction.le
  (pointDistance (b (cell b h t).1) (b (oneFull b h t).1))
  (Fraction.add (Fraction.mul L
    (pointDistance (cell b h t).1 (oneFull b h t).1)) E)
secondLocal : Fraction.le
  (pointDistance (b (twoHalf b h t).1) (b (oneFull b h t).1))
  (Fraction.add (Fraction.mul L
    (pointDistance (twoHalf b h t).1 (oneFull b h t).1)) E)

/-- Actual first and second fine-cell samples each contribute their own E. -/
theorem cross_block_error_at (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E B V : Fraction) (s t : Point × Point)
  (hL : 0 ≤ L.num) (hc : BlockComparisons a b h L E s t)
  (hB : Fraction.le (pointNorm (b (cell b h t).1)) B)
  (hV : Fraction.le (pointNorm t.2) V) :
Fraction.le (TimeCalibration.distance tau (twoHalf a h s) (oneFull b h t))
  (Fraction.add
    (Fraction.mul (blockFactor tau h L ht) (TimeCalibration.distance tau s t))
    (blockSource tau h L E B V ht)) := by
let K := TimeCalibration.amplification tau h L ht
let S := Fraction.mul (Fraction.mul tau h.abs) E
have hK := TimeCalibration.amplification_nonnegative tau h L ht hL
have h1 := TimeCalibration.cell_amplification_at tau ht a b h L E s t hL hc.firstCross
have h2 := TimeCalibration.cell_amplification_at tau ht a b h L E
  (cell a h s) (cell b h t) hL hc.secondCross

```

```

have hprop := Fraction.magnitudes.le_trans h2
(Fraction.add_le_add_right (Fraction.mul_le_mul_nonnegative_left h1 K hK) S)
have h1 := twoHalf_local_error_at tau ht b h L E B V t hL
hc.firstLocal hc.secondLocal hB hV
have htri := TimeCalibration.distance_triangle tau ht
(twoHalf a h s) (twoHalf b h t) (oneFull b h t)
apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans htri (Fraction.add_le_add hprop h1))
simp only [K,S,blockFactor,blockSource,Fraction.equiv,Fraction.add,Fraction.mul,
Int.add_mul,Int.mul_add]
ac_nf

theorem cross_block_error (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (h L E B V : Fraction) (s t : Point × Point)
(hL : 0 ≤ L.num) (hcross : comparisonContract a b L E)
(hlocal : comparisonContract b b L E)
(hB : Fraction.le (pointNorm (b (cell b h t).1)) B)
(hV : Fraction.le (pointNorm t.2) V) :
Fraction.le (TimeCalibration.distance tau (twoHalf a h s) (oneFull b h t))
(Fraction.add
(Fraction.mul (blockFactor tau h L ht) (TimeCalibration.distance tau s t))
(blockSource tau h L E B V ht)) :=
cross_block_error_at tau ht a b h L E B V s t hL
(hcross (cell a h s).1 (cell b h t).1,
hcross (twoHalf a h s).1 (twoHalf b h t).1,
hlocal (cell b h t).1 (oneFull b h t).1,
hlocal (twoHalf b h t).1 (oneFull b h t).1) hB hV

theorem actual_error_le_source_at (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (h L E B V : Fraction) (s : Point × Point)
(hL : 0 ≤ L.num) :
(n : Nat) →
(hc : ∀ k, k < n → BlockComparisons a b h L E
(fineAt a h s k) (coarseAt b h s k)) →
(hB : ∀ k, k < n → Fraction.le
(pointNorm (b (cell b h (coarseAt b h s k)).1)) B) →
(hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) →
Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
(FiniteRecurrence.sourceBudget (blockFactor tau h L ht)
(blockSource tau h L E B V ht) n)
| 0, _, _, _ => Fraction.le_of_equiv (TimeCalibration.distance_self_zero tau s)
| n+1, hc, hB, hV => by
have hs := cross_block_error_at tau ht a b h L E B V
(fineAt a h s n) (coarseAt b h s n) hL (hc n (by omega))
(hB n (by omega)) (hV n (by omega))
have hi := actual_error_le_source_at tau ht a b h L E B V s hL n
(fun k hk => hc k (by omega))
(fun k hk => hB k (by omega)) (fun k hk => hV k (by omega))
have hK := TimeCalibration.amplification_nonnegative tau h L ht hL
exact Fraction.magnitudes.le_trans hs
(Fraction.add_le_add_right
(Fraction.mul_le_mul_nonnegative_left hi (blockFactor tau h L ht)
(Fraction.nonnegative_mul _ _ hK hK)) (blockSource tau h L E B V ht))

theorem actual_error_le_source (tau : Fraction) (ht : 0 < tau.num)
(a b : Point → Point) (h L E B V : Fraction) (s : Point × Point)
(hL : 0 ≤ L.num) (hcross : comparisonContract a b L E)
(hlocal : comparisonContract b b L E) :
(n : Nat) →
(hB : ∀ k, k < n → Fraction.le
(pointNorm (b (cell b h (coarseAt b h s k)).1)) B) →
(hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) →
Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
(FiniteRecurrence.sourceBudget (blockFactor tau h L ht)
(blockSource tau h L E B V ht) n) :=
fun n hB hV => actual_error_le_source_at tau ht a b h L E B V s hL n
(fun k _ =>
(hcross (cell a h (fineAt a h s k)).1 (cell b h (coarseAt b h s k)).1,
hcross (twoHalf a h (fineAt a h s k)).1
(twoHalf b h (coarseAt b h s k)).1,
hlocal (cell b h (coarseAt b h s k)).1

```

```

      (oneFull b h (coarseAt b h s k)).1,
      hlocal (twoHalf b h (coarseAt b h s k)).1
      (oneFull b h (coarseAt b h s k)).1)) hB hV

theorem actual_uniform_error_at (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB0 : 0 ≤ B.num) (hV0 : 0 ≤ V.num)
  (n : Nat) (hs : TimeCalibration.Window tau h L ht (2*n))
  (hc : ∀ k, k < n → BlockComparisons a b h L E
    (fineAt a h s k) (coarseAt b h s k))
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
    (Fraction.mul (Fraction.ofInt (2*(n : Int))) (blockSource tau h L E B V ht)) := by
  let K := TimeCalibration.amplification tau h L ht
  let R := blockFactor tau h L ht
  let S := blockSource tau h L E B V ht
  have hK := TimeCalibration.amplification_nonnegative tau h L ht hL
  have hR : 0 ≤ R.num := Fraction.nonnegative_mul _ _ hK hK
  have hS := blockSource_nonnegative tau h L E B V ht hL hE hB0 hV0
  have hone : Fraction.le (Fraction.ofInt 1) R := by
    have hb := TimeCalibration.one_le_amplification tau h L ht hL
    have hm := Fraction.mul_le_mul_nonnegative_left hb K hK
    have hl : Fraction.le (Fraction.ofInt 1) (Fraction.mul K (Fraction.ofInt 1)) :=
      Fraction.le_equiv_right hb (by
        simp only [K, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one])
    exact Fraction.magnitudes.le_trans hl hm
  have hp := Fraction.le_equiv_left (FiniteFactorProducts.fpower_square K n)
    (TimeCalibration.amplification_power_le_two tau h L ht hL (2*n) hs)
  exact Fraction.magnitudes.le_trans
    (actual_error_le_source_at tau ht a b h L E B V s hL n hc hB hV)
    (FiniteRecurrence.sourceBudget_two_count R S n hR hS hone hp)

theorem actual_uniform_error (tau : Fraction) (ht : 0 < tau.num)
  (a b : Point → Point) (h L E B V : Fraction) (s : Point × Point)
  (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hB0 : 0 ≤ B.num) (hV0 : 0 ≤ V.num)
  (hcross : comparisonContract a b L E) (hlocal : comparisonContract b b L E)
  (n : Nat) (hs : TimeCalibration.Window tau h L ht (2*n))
  (hB : ∀ k, k < n → Fraction.le
    (pointNorm (b (cell b h (coarseAt b h s k)).1)) B)
  (hV : ∀ k, k < n → Fraction.le (pointNorm (coarseAt b h s k).2) V) :
  Fraction.le (TimeCalibration.distance tau (fineAt a h s n) (coarseAt b h s n))
    (Fraction.mul (Fraction.ofInt (2*(n : Int))) (blockSource tau h L E B V ht)) :=
  actual_uniform_error_at tau ht a b h L E B V s hL hE hB0 hV0 n hs
  (fun k _ =>
    {hcross (cell a h (fineAt a h s k)).1 (cell b h (coarseAt b h s k)).1,
      hcross (twoHalf a h (fineAt a h s k)).1
        (twoHalf b h (coarseAt b h s k)).1,
      hlocal (cell b h (coarseAt b h s k)).1
        (oneFull b h (coarseAt b h s k)).1,
      hlocal (twoHalf b h (coarseAt b h s k)).1
        (oneFull b h (coarseAt b h s k)).1)) hB hV

def consistencyCoefficient (tau T L B V : Fraction) : Fraction :=
  Fraction.add B (Fraction.mul (Fraction.mul tau L) (Fraction.add V (Fraction.mul T B)))

/-- An actual block source is 0(h²)+0(hE), with a mesh-independent coefficient. -/
theorem blockSource_bound (tau : Fraction) (ht : 0 < tau.num)
  (h T L E B V : Fraction) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num)
  (hB : 0 ≤ B.num) (hHT : Fraction.le h.abs T)
  (hK : Fraction.le (TimeCalibration.amplification tau h L ht) (Fraction.ofInt 2)) :
  Fraction.le (blockSource tau h L E B V ht)
    (Fraction.add
      (Fraction.mul (Fraction.mul h.abs h.abs) (consistencyCoefficient tau T L B V))
      (Fraction.mul (Fraction.ofInt 5) (Fraction.mul (Fraction.mul tau h.abs) E))) := by
  let H := h.abs
  let S := Fraction.mul (Fraction.mul tau H) E
  have hS : 0 ≤ S.num := Fraction.nonnegative_mul
    (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) (Fraction.abs_num_nonnegative h)) hE

```

```

have hprop := Fraction.add_le_add_right
  (Fraction.mul_le_mul_nonnegative hK S hS) S
have hsq := Fraction.abs_mul h h
have hb := Fraction.le_equiv_left (Fraction.mul_equiv hsq (Fraction.equiv_refl B))
  (Fraction.mul_le_mul_nonnegative
    (Fraction.mul_le_mul_nonnegative_left hHT H (Fraction.abs_num_nonnegative h)) B hB)
have hinner := Fraction.add_le_add_left
  (Fraction.add_le_add_right (Fraction.mul_le_mul_nonnegative_left hb L hL) E)
  (Fraction.add (Fraction.mul L (Fraction.mul H V)) E)
have hvel := Fraction.mul_le_mul_nonnegative_left
  (Fraction.mul_le_mul_nonnegative_left hinner H (Fraction.abs_num_nonnegative h))
  tau (Int.le_of_lt ht)
have hlocal := Fraction.add_le_add_left hvel (Fraction.mul (Fraction.mul h h).abs B)
have hsum := Fraction.add_le_add hprop hlocal
let W := Fraction.mul tau (Fraction.mul H
  (Fraction.add (Fraction.add (Fraction.mul L (Fraction.mul H V)) E)
    (Fraction.add (Fraction.mul L (Fraction.mul (Fraction.mul H T) B)) E)))
have he : Fraction.equiv
  (Fraction.add (Fraction.add (Fraction.mul (Fraction.ofInt 2) S) S)
    (Fraction.add (Fraction.mul (Fraction.mul h h).abs B) W))
  (Fraction.add (Fraction.add (Fraction.mul (Fraction.ofInt 2) S) S)
    (Fraction.add (Fraction.mul (Fraction.mul H H) B) W)) :=
  Fraction.add_equiv (Fraction.equiv_refl _)
  (Fraction.add_equiv (Fraction.mul_equiv hsq (Fraction.equiv_refl B))
    (Fraction.equiv_refl W))
apply Fraction.le_equiv_right (Fraction.le_equiv_right hsum he)
simp only [S,W,H,consistencyCoefficient,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.ofInt,
  show (5 : Int) = 2+1+1+1 by rfl,
  Int.add_mul,Int.mul_add,Int.one_mul,Int.mul_one]
ac_nf

```

```

theorem coarseAt_eq_run (a : Point → Point) (h : Fraction) (s : Point × Point) :
  (n : Nat) → coarseAt a h s n = BoundedIteration.run a (Fraction.add h h) s n
| 0 => rfl
| n+1 => congrArg (oneFull a h) (coarseAt_eq_run a h s n)

```

```

theorem fineAt_eq_run (a : Point → Point) (h : Fraction) (s : Point × Point) :
  (n : Nat) → fineAt a h s n = BoundedIteration.run a h s (2*n)
| 0 => rfl
| n+1 => by
  simp only [Nat.mul_succ,Nat.add_succ,Nat.add_zero,BoundedIteration.run,fineAt,twoHalf]
  rw [fineAt_eq_run a h s n]

```

end NewtonLimitDynamics.Polygon.CalibratedRefinement

BarrowLib/Polygon/ConvexValues.lean, 18 theorems, 270 lines

```

import BarrowLib.Polygon.ConvexCover
import BarrowLib.Polygon.PositionValues

namespace NewtonLimitDynamics.Polygon.ConvexValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds ConvexCover FiniteEstimates CauchyValues
open HarmonicDyadic

/-- A fixed rational interpolation is Lipschitz in both endpoints. -/
theorem lerp_distance (a : Fraction) (ha : UnitInterval a)
  (p q p' q' : Point) :
  Fraction.le (pointDistance (lerp a p q) (lerp a p' q'))
    (Fraction.add (pointDistance p p') (pointDistance q q')) := by
have he : pointEquiv (pointSub (lerp a p q) (lerp a p' q'))
  (pointAdd (pointScale (complement a) (pointSub p p'))
    (pointScale a (pointSub q q'))) := by
  constructor <=>
  simp only [pointEquiv,lerp,pointSub,pointNeg,pointAdd,pointScale,
    Fraction.equiv,Fraction.add,Fraction.mul,Int.add_mul,Int.mul_add,
    Int.neg_mul,Int.mul_neg] <=> ac_nf <=> omega
have hca : Fraction.le (complement a) (Fraction.ofInt 1) := by
  simp only [complement,Fraction.le,Fraction.ofInt,Int.mul_one,Int.one_mul]
have hnonneg := ha.1
omega

```

```

have haa : Fraction.le a (Fraction.ofInt 1) := by
  simp only [Fraction.le, Fraction.ofInt, Int.mul_one, Int.one_mul] using ha.2
have hc := Fraction.mul_le_mul_nonnegative hca (pointDistance p p')
  (pointNorm_nonnegative _)
have hd := Fraction.mul_le_mul_nonnegative haa (pointDistance q q')
  (pointNorm_nonnegative _)
have hs := Fraction.add_equiv
  (Fraction.equiv_trans (pointNorm_scale (complement a) (pointSub p p'))
    (Fraction.mul_equiv (complement_abs a ha) (Fraction.equiv_refl _)))
  (Fraction.equiv_trans (pointNorm_scale a (pointSub q q'))
    (Fraction.mul_equiv (interval_abs a ha) (Fraction.equiv_refl _)))
have hb := Fraction.le_equiv_right
  (pointNorm_add_le (pointScale (complement a) (pointSub p p'))
    (pointScale a (pointSub q q'))) hs
have hlast := Fraction.add_le_add hc hd
have hone : Fraction.equiv
  (Fraction.add (Fraction.mul (Fraction.ofInt 1) (pointDistance p p'))
    (Fraction.mul (Fraction.ofInt 1) (pointDistance q q')))
  (Fraction.add (pointDistance p p') (pointDistance q q')) := by
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.one_mul, Int.mul_one]
exact Fraction.magnitudes.le_trans (Fraction.le_equiv_left (pointNorm_equiv he) hb)
  (Fraction.le_equiv_right hlast hone)

def convexState (a : Fraction) (s t : Point × Point) : Point × Point :=
  (lerp a s.1 t.1, PositionValues.zeroPoint)

theorem convexState_distance (a : Fraction) (ha : UnitInterval a)
  (s t s' t' : Point × Point) :
  Fraction.le (distance (convexState a s t) (convexState a s' t'))
    (Fraction.add (distance s s') (distance t t')) := by
  have he : Fraction.equiv (distance (convexState a s t) (convexState a s' t'))
    (pointDistance (lerp a s.1 t.1) (lerp a s'.1 t'.1)) :=
    Fraction.equiv_trans
      (Fraction.add_equiv (Fraction.equiv_refl _)
        (pointDistance_self_zero PositionValues.zeroPoint)) (Fraction.add_zero _)
  have hs : Fraction.le (pointDistance s.1 s'.1) (distance s s') :=
    Fraction.le_add_nonnegative _ _ (pointNorm_nonnegative _)
  have ht : Fraction.le (pointDistance t.1 t'.1) (distance t t') :=
    Fraction.le_add_nonnegative _ _ (pointNorm_nonnegative _)
  exact Fraction.le_equiv_left he
    (Fraction.magnitudes.le_trans (lerp_distance a ha _ _ _) (Fraction.add_le_add hs ht))

def convexName (a : Fraction) (ha : UnitInterval a)
  (s t : EndpointCauchyName) : EndpointCauchyName where
  approx := fun n => convexState a (s.approx n) (t.approx n)
  cauchy := by
    intro eps heps
    have hh := (show 0 < eps.half.num from heps)
    obtain ⟨N, hN⟩ := s.cauchy eps.half hh
    obtain ⟨M, hM⟩ := t.cauchy eps.half hh
    refine ⟨max N M, ?_⟩
    intro m n hm hn
    have hs := hN m n (Nat.le_trans (Nat.le_max_left _ _) hm)
      (Nat.le_trans (Nat.le_max_left _ _) hn)
    have ht := hM m n (Nat.le_trans (Nat.le_max_right _ _) hm)
      (Nat.le_trans (Nat.le_max_right _ _) hn)
    have hsum := Fraction.add_lt_add hs ht
    exact Fraction.magnitudes.lt_of_le_lt (convexState_distance a ha _ _ _)
      (Fraction.magnitudes.lt_of_lt_le hsum (Fraction.le_of_equiv (Fraction.half_add_self eps)))

theorem convexName_equiv (a : Fraction) (ha : UnitInterval a)
  (s t s' t' : EndpointCauchyName) (hs : NameEquiv s s') (ht : NameEquiv t t') :
  NameEquiv (convexName a ha s t) (convexName a ha s' t') := by
  intro eps heps
  have hh := (show 0 < eps.half.num from heps)
  obtain ⟨N, hN⟩ := hs eps.half hh
  obtain ⟨M, hM⟩ := ht eps.half hh
  refine ⟨max N M, ?_⟩
  intro n hn

```

```

have hsum := Fraction.add_lt_add
  (hN n (Nat.le_trans (Nat.le_max_left _ _) hn))
  (hM n (Nat.le_trans (Nat.le_max_right _ _) hn))
exact Fraction.magnitudes.lt_of_le_lt (convexState_distance a ha _ _ _)
  (Fraction.magnitudes.lt_of_lt_le hsum (Fraction.le_of_equiv (Fraction.half_add_self eps)))

def convexValue (a : Fraction) (ha : UnitInterval a) (x y : Value) : Value :=
  Quotient.liftOn₂ x y (fun s t => realize (convexName a ha s t))
  (fun s t s' t' hs ht => Quotient.sound (convexName_equiv a ha s t s' t' hs ht))

open PositionValues

theorem convexState_anchor_bound (a : Fraction) (ha : UnitInterval a)
  (s t : Point × Point) (centre : Point) (R : Fraction)
  (hs : Fraction.le (distance s (centre, zeroPoint)) R)
  (ht : Fraction.le (distance t (centre, zeroPoint)) R) :
  Fraction.le (distance (convexState a s t) (centre, zeroPoint)) R := by
have hp : Fraction.le (pointNorm (pointSub s.1 centre)) R :=
  Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative _ _
    (pointNorm_nonnegative _)) hs
have hq : Fraction.le (pointNorm (pointSub t.1 centre)) R :=
  Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative _ _
    (pointNorm_nonnegative _)) ht
have he : Fraction.equiv (distance (convexState a s t) (centre, zeroPoint))
  (pointNorm (pointSub (lerp a s.1 t.1) centre)) :=
  Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _
    (pointSub_self_zero zeroPoint)) (Fraction.add_zero _))
exact Fraction.le_equiv_left he (lerp_ball_bound a ha centre _ _ R hp hq)

theorem convexName_ball (a : Fraction) (ha : UnitInterval a)
  (s t : EndpointCauchyName) (centre : Point) (R : Fraction)
  (hs : NameBound s (constantName (centre, zeroPoint)) R)
  (ht : NameBound t (constantName (centre, zeroPoint)) R) :
  NameBound (convexName a ha s t) (constantName (centre, zeroPoint)) R := by
intro eps heps
obtain ⟨N, hN⟩ := hs eps.half heps
obtain ⟨M, hM⟩ := ht eps.half heps
refine ⟨max N M, ?⟩
intro n hn
have hp := Fraction.magnitudes.lt_implies_le
  (hN n (Nat.le_trans (Nat.le_max_left _ _) hn))
have hq := Fraction.magnitudes.lt_implies_le
  (hM n (Nat.le_trans (Nat.le_max_right _ _) hn))
exact Fraction.magnitudes.lt_of_le_lt (convexState_anchor_bound a ha _ _ _ hp hq)
  (CauchyValues.add_lt_add_left (Fraction.half_lt eps heps) R)

theorem convexValue_ball (a : Fraction) (ha : UnitInterval a)
  (x y : Value) (centre : Point) (R : Fraction)
  (hx : Within x (embed (centre, zeroPoint)) R)
  (hy : Within y (embed (centre, zeroPoint)) R) :
  Within (convexValue a ha x y) (embed (centre, zeroPoint)) R := by
induction x using Quotient.inductionOn with
| _ s =>
  induction y using Quotient.inductionOn with
| _ t => exact convexName_ball a ha s t centre R hx hy

theorem convexValue_position (a : Fraction) (ha : UnitInterval a) (x y : Value) :
  positionValue (convexValue a ha x y) = convexValue a ha x y := by
induction x using Quotient.inductionOn with
| _ s =>
  induction y using Quotient.inductionOn with
| _ t => rfl

def convexPosition (a : Fraction) (ha : UnitInterval a)
  (x y : PositionValue) : PositionValue :=
  (convexValue a ha x.val y.val, convexValue_position a ha x.val y.val)

theorem first_convex_state (a : Fraction) (s t : Point × Point) :
  stateEquiv (firstState (convexState a s t))
  (convexState a (firstState s) (firstState t)) := by

```



```

constructor
· constructor
· exact Fraction.equiv_refl _
· change Fraction.equiv (Fraction.ofInt 0)
  (Fraction.add (Fraction.mul (complement a) (Fraction.ofInt 0))
  (Fraction.mul a (Fraction.ofInt 0)))
exact Fraction.equiv_symm (Fraction.equiv_trans
  (Fraction.add_equiv (Fraction.mul_zero _) (Fraction.mul_zero _))
  (Fraction.add_zero _))
· exact (Fraction.equiv_refl _, Fraction.equiv_refl _)

theorem second_convex_state (a : Fraction) (s t : Point × Point) :
  stateEquiv (secondState (convexState a s t))
  (convexState a (secondState s) (secondState t)) := by
  constructor
  · constructor
  · change Fraction.equiv (Fraction.ofInt 0)
    (Fraction.add (Fraction.mul (complement a) (Fraction.ofInt 0))
    (Fraction.mul a (Fraction.ofInt 0)))
  exact Fraction.equiv_symm (Fraction.equiv_trans
    (Fraction.add_equiv (Fraction.mul_zero _) (Fraction.mul_zero _))
    (Fraction.add_zero _))
  · exact Fraction.equiv_refl _
  · exact (Fraction.equiv_refl _, Fraction.equiv_refl _)

theorem firstValue_convex (a : Fraction) (ha : UnitInterval a) (x y : Value) :
  firstValue (convexValue a ha x y) =
  convexValue a ha (firstValue x) (firstValue y) := by
  induction x using Quotient.inductionOn with
  | _ s =>
    induction y using Quotient.inductionOn with
    | _ t =>
      apply Quotient.sound
      exact nameEquiv_of_levelwise_stateEquiv _ _ (fun n => first_convex_state a _ _)

theorem secondValue_convex (a : Fraction) (ha : UnitInterval a) (x y : Value) :
  secondValue (convexValue a ha x y) =
  convexValue a ha (secondValue x) (secondValue y) := by
  induction x using Quotient.inductionOn with
  | _ s =>
    induction y using Quotient.inductionOn with
    | _ t =>
      apply Quotient.sound
      exact nameEquiv_of_levelwise_stateEquiv _ _ (fun n => second_convex_state a _ _)

theorem convexPosition_square (a : Fraction) (ha : UnitInterval a)
  (x y : PositionValue) (centre : Point) (R : NonnegativeRadius)
  (hx : CoordinateSquare centre R x) (hy : CoordinateSquare centre R y) :
  CoordinateSquare centre R (convexPosition a ha x y) := by
  constructor
  · change Within (firstValue (convexValue a ha x.val y.val)) _ _
    rw [firstValue_convex]
    exact convexValue_ball a ha _ _ (firstState (centre, zeroPoint)).1 R.val hx.1 hy.1
  · change Within (secondValue (convexValue a ha x.val y.val)) _ _
    rw [secondValue_convex]
    exact convexValue_ball a ha _ _ (secondState (centre, zeroPoint)).1 R.val hx.2 hy.2

theorem convexValue_zero (ha : UnitInterval (Fraction.ofInt 0)) (x y : Value) :
  convexValue (Fraction.ofInt 0) ha x y = positionValue x := by
  induction x using Quotient.inductionOn with
  | _ s =>
    induction y using Quotient.inductionOn with
    | _ t =>
      apply Quotient.sound
      exact nameEquiv_of_levelwise_stateEquiv _ _ (fun n =>
        (lerp_zero (s.approx n).1 (t.approx n).1,
        (Fraction.equiv_refl _, Fraction.equiv_refl _)))

theorem convexValue_one (ha : UnitInterval (Fraction.ofInt 1)) (x y : Value) :
  convexValue (Fraction.ofInt 1) ha x y = positionValue y := by

```

```

induction x using Quotient.inductionOn with
| _ s =>
  induction y using Quotient.inductionOn with
  | _ t =>
    apply Quotient.sound
    exact nameEquiv_of_levelwise_stateEquiv _ _ (fun n =>
      (lerp_one (s.approx n).1 (t.approx n).1,
        (Fraction.equiv_refl _, Fraction.equiv_refl _)))

theorem convexPosition_zero (ha : UnitInterval (Fraction.ofInt 0))
  (x y : PositionValue) : convexPosition (Fraction.ofInt 0) ha x y = x := by
  apply Subtype.ext
  exact (convexValue_zero ha x.val y.val).trans x.property

theorem convexPosition_one (ha : UnitInterval (Fraction.ofInt 1))
  (x y : PositionValue) : convexPosition (Fraction.ofInt 1) ha x y = y := by
  apply Subtype.ext
  exact (convexValue_one ha x.val y.val).trans y.property

theorem convexValue_swap (a : Fraction) (ha : UnitInterval a) (x y : Value) :
  convexValue a ha x y = convexValue (complement a) (complement_interval a ha) y x := by
  induction x using Quotient.inductionOn with
  | _ s =>
    induction y using Quotient.inductionOn with
    | _ t =>
      apply Quotient.sound
      exact nameEquiv_of_levelwise_stateEquiv _ _ (fun n =>
        (lerp_swap a (s.approx n).1 (t.approx n).1,
          (Fraction.equiv_refl _, Fraction.equiv_refl _)))

theorem convexPosition_swap (a : Fraction) (ha : UnitInterval a)
  (x y : PositionValue) :
  convexPosition a ha x y = convexPosition (complement a) (complement_interval a ha) y x :=
  Subtype.ext (convexValue_swap a ha x.val y.val)

```

end NewtonLimitDynamics.Polygon.ConvexValues

BarrowLib/Polygon/CompletionGeometry.lean, 10 theorems, 130 lines

```

import BarrowLib.Polygon.ConvexValues
import BarrowLib.Polygon.ScaledTolerance
import BarrowLib.Common.RationalExhaustion

/-- Elementary closure and closed coordinate enclosures in the explicit Cauchy
plane. Closure uses all positive rational tolerances; no external topology,
measure, integral or curve is a premise. -/
namespace NewtonLimitDynamics.Polygon.CompletionGeometry
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic CauchyValues PositionValues
open HarmonicTimeRealization

/-- Closed bounds can be exhausted from arbitrarily small rational enlargements. -/
theorem within_of_thickenings (x y : Value) (R : Fraction)
  (h : ∀ eps : Fraction, 0 < eps.num → Within x y (Fraction.add R eps)) :
  Within x y R := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b =>
      intro eps heps
      obtain (N,hN) := h eps.half heps eps.half heps
      refine (N,fun n hn => ?_)
      exact lt_equiv_right (hN n hn) (Fraction.equiv_trans
        (Fraction.add_assoc R eps.half eps.half)
        (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.half_add_self eps)))

/-- Rational closed distance bounds are exact after completion. -/
theorem within_embedded_iff (a b : Point × Point) (R : Fraction) :
  Within (embed a) (embed b) R ↔ Fraction.le (distance a b) R := by
  constructor
  · intro h

```

```

    apply Fraction.le_of_enlargements
    intro eps heps
    obtain (N,hN) := h eps heps
    exact Fraction.magnitudes.lt_implies_le (hN N (Nat.le_refl _))
  · intro h
    exact nameBound_of_eventual_le _ _ R 0 (fun _ _ => h)

/-- The coordinate length of a position is its distance from the origin. -/
theorem position_distance_zero (s : Point × Point) :
  Fraction.equiv (distance (positionState s) (zeroPoint,zeroPoint)) (pointNorm s.1) := by
  change Fraction.equiv
    (distance (s.1,(Fraction.ofInt 0,Fraction.ofInt 0))
      ((Fraction.ofInt 0,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 0)))
    (pointNorm s.1)
  simp only [distance,HarmonicComparison.stateSub,pointSub,
    pointAdd,pointNeg,stateNorm,pointNorm,Fraction.equiv,Fraction.add,Fraction.abs,Fraction.ofInt,
    Int.zero_mul,Int.mul_zero,Int.add_zero,Int.natAbs_zero,Int.ofNat_zero,Int.neg_zero,
    Int.mul_one,Int.one_mul]

/-- A finite coordinate band is closed under Cauchy realization. The inner
bound excludes every smaller closed ball, without adding a completed norm. -/
theorem position_band_realize (a : EndpointCauchyName) (r R : Fraction)
  (h : ∀ n, Fraction.le r (pointNorm (a.approx n).1) ∧
    Fraction.le (pointNorm (a.approx n).1) R) :
  Within (positionValue (realize a)) (embed (zeroPoint,zeroPoint)) R ∧
    ∀ D, Within (positionValue (realize a)) (embed (zeroPoint,zeroPoint)) D →
    Fraction.le r D := by
  constructor
  · exact nameBound_of_eventual_le
    (mapName positionState position_nonexpansive a) (constantName (zeroPoint,zeroPoint)) R 0
    (fun n _ => Fraction.le_equiv_left (position_distance_zero _) (h n).2)
  · intro D hD
    apply Fraction.le_of_enlargements
    intro eps heps
    obtain (N,hN) := hD eps heps
    have hD := Fraction.le_equiv_left (Fraction.equiv_symm (position_distance_zero (a.approx N)))
      (Fraction.magnitudes.lt_implies_le (hN N (Nat.le_refl _)))
    exact Fraction.magnitudes.le_trans (h N).1 hD

def Closure (A : PositionValue → Prop) (x : PositionValue) : Prop :=
  ∀ eps : Fraction, 0 < eps.num →
    ∃ y : PositionValue, A y ∧ Within x.val y.val eps

theorem closure_contains (A : PositionValue → Prop) (x : PositionValue)
  (hx : A x) : Closure A x := by
  intro eps heps
  refine (x,hx,?)
  exact within_mono _ _ _ _
    (Fraction.magnitudes.lt_implies_le ((Fraction.positive_iff_zero_lt eps).mp heps))
    ((within_zero_iff _ _).mpr rfl)

theorem closure_mono (A B : PositionValue → Prop)
  (h : ∀ x, A x → B x) (x : PositionValue) (hx : Closure A x) :
  Closure B x := by
  intro eps heps
  obtain (y,hy,hxy) := hx eps heps
  exact (y,h y hy,hxy)

theorem closure_idempotent (A : PositionValue → Prop) (x : PositionValue) :
  Closure (Closure A) x ↔ Closure A x := by
  constructor
  · intro hx eps heps
    obtain (y,hy,hxy) := hx eps.half heps
    obtain (z,hz,hzy) := hy eps.half heps
    exact (z,hz,within_mono _ _ _ _
      (Fraction.le_of_equiv (Fraction.half_add_self eps))
      (within_triangle _ _ _ _ hxy hzy))
  · intro hx
    exact closure_contains (Closure A) x hx

```

```

theorem closure_image_bound (A : PositionValue → Prop) (x : PositionValue)
  (hx : Closure A x) (f : Value → Value)
  (hf : ∀ u v R, Within u v R → Within (f u) (f v) R)
  (centre : Value) (R : Fraction)
  (hA : ∀ y, A y → Within (f y.val) centre R) :
  Within (f x.val) centre R := by
  apply within_of_thickenings
  intro eps heps
  obtain ⟨y, hy, hxy⟩ := hx eps heps
  exact within_mono _ _ _ (Fraction.le_of_equiv (Fraction.add_comm eps R))
    (within_triangle _ _ _ _ (hf _ _ _ hxy) (hA y hy))

theorem closure_square (A : PositionValue → Prop) (x : PositionValue)
  (hx : Closure A x) (centre : Point) (R : NonnegativeRadius)
  (hA : ∀ y, A y → CoordinateSquare centre R y) :
  CoordinateSquare centre R x := by
  constructor
  · exact closure_image_bound A x hx firstValue firstValue_within _ R.val
    (fun y hy => (hA y hy).1)
  · exact closure_image_bound A x hx secondValue secondValue_within _ R.val
    (fun y hy => (hA y hy).2)

theorem square_of_ball_bound (x : PositionValue) (centre : Point)
  (R : NonnegativeRadius) (h : Within x.val (embed (centre, zeroPoint)) R.val) :
  CoordinateSquare centre R x :=
  (firstValue_within _ _ _ h, secondValue_within _ _ _ h)

end NewtonLimitDynamics.Polygon.CompletionGeometry

```

BarrowLib/Polygon/SquareOuterContent.lean, 12 theorems, 144 lines

```

import BarrowLib.Polygon.CompletionGeometry
import BarrowLib.Common.RationalExhaustion

/-! Nonnegative elementary outer content in the completed coordinate plane.
A cover is a finite family of closed rational squares. Its budget counts
squares with multiplicity; the region is a point set, so overlapping or crossing
lobes have no signed cancellation. Content is the closed lower cut of the
infimum of all covering budgets, rather than an arbitrary chosen cover budget. -/
namespace NewtonLimitDynamics.Polygon.SquareOuterContent
open NewtonLimitDynamics
open TimeSubdivision CauchyValues PositionValues

structure Square where
  centre : Point
  radius : NonnegativeRadius

def Square.Contains (q : Square) (x : PositionValue) : Prop :=
  CoordinateSquare q.centre q.radius x

def squareArea (R : Fraction) : Fraction :=
  Fraction.mul (Fraction.ofInt 4) (Fraction.mul R R)

def Square.area (q : Square) : Fraction := squareArea q.radius.val

theorem squareArea_nonnegative (R : Fraction) (hR : 0 ≤ R.num) :
  0 ≤ (squareArea R).num :=
  Fraction.nonnegative_mul _ _ (by decide) (Fraction.nonnegative_mul _ _ hR hR)

def sumBudget (squares : Nat → Square) : Nat → Fraction
| 0 => Fraction.ofInt 0
| n+1 => Fraction.add (sumBudget squares n) (squares n).area

theorem sumBudget_nonnegative (squares : Nat → Square) :
  ∀ n, 0 ≤ (sumBudget squares n).num
| 0 => by simp [sumBudget, Fraction.ofInt]
| n+1 => Fraction.nonnegative_add _ _ (sumBudget_nonnegative squares n)
  (squareArea_nonnegative _ (squares n).radius.property)

structure Cover (A : PositionValue → Prop) where
  count : Nat

```

```

squares : Nat → Square
covers : ∀ x, A x → ∃ k, k < count ∧ (squares k).Contains x

def Cover.budget {A : PositionValue → Prop} (c : Cover A) : Fraction :=
  sumBudget c.squares c.count

theorem Cover.budget_nonnegative {A : PositionValue → Prop} (c : Cover A) :
  0 ≤ c.budget.num := sumBudget_nonnegative c.squares c.count

def Cover.restrict {A B : PositionValue → Prop} (c : Cover B)
  (h : ∀ x, A x → B x) : Cover A :=
  (c.count, c.squares, fun x hx => c.covers x (h x hx))

theorem uniform_budget (centres : Nat → Point) (R : NonnegativeRadius) :
  ∀ n, Fraction.equiv (sumBudget (fun k => (centres k, R)) n)
    (Fraction.mul (Fraction.ofInt (n : Int)) (squareArea R.val))
| 0 => by simp [sumBudget, Fraction.equiv, Fraction.mul, Fraction.ofInt]
| n+1 => by
  have he := Fraction.add_equiv (uniform_budget centres R n)
    (Fraction.equiv_refl (squareArea R.val))
  apply Fraction.equiv_trans he
  simp only [Square.area, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

/-- Exact lower-cut representation of the infimum over all finite square covers.
If no cover exists this is an unbounded cut; bounded regions below supply a
constructed cover and therefore a finite upper bound. -/
def LowerContent (A : PositionValue → Prop) (q : Fraction) : Prop :=
  ∀ c : Cover A, Fraction.le q c.budget

theorem content_zero_lower (A : PositionValue → Prop) :
  LowerContent A (Fraction.ofInt 0) := by
  intro c
  simp only [Fraction.le, Fraction.ofInt, Int.zero_mul, Int.mul_one] using c.budget_nonnegative

theorem content_downward (A : PositionValue → Prop) (p q : Fraction)
  (hpq : Fraction.le p q) (hq : LowerContent A q) : LowerContent A p := by
  intro c
  exact Fraction.magnitudes.le_trans hpq (hq c)

/-- The infimum cut is closed under exhaustion from its lower bounds. -/
theorem content_closed (A : PositionValue → Prop) (q : Fraction)
  (h : ∀ eps : Fraction, 0 < eps.num →
    ∃ p, LowerContent A p ∧ Fraction.le q (Fraction.add p eps)) :
  LowerContent A q := by
  intro c
  apply Fraction.le_of_enlargements
  intro eps heps
  obtain (p, hp, hqp) := h eps heps
  exact Fraction.magnitudes.le_trans hqp (Fraction.add_le_add_right (hp c) eps)

theorem content_cover_bound (A : PositionValue → Prop) (c : Cover A)
  (q : Fraction) (hq : LowerContent A q) : Fraction.le q c.budget := hq c

theorem content_mono (A B : PositionValue → Prop)
  (hAB : ∀ x, A x → B x) (q : Fraction) (hq : LowerContent A q) :
  LowerContent B q := by
  intro c
  exact hq (c.restrict hAB)

theorem content_union_includes (A B : PositionValue → Prop) (q : Fraction)
  (h : LowerContent A q ∨ LowerContent B q) :
  LowerContent (fun x => A x ∨ B x) q := by
  cases h with
| inl h => exact content_mono A _ (fun _ hx => 0r.inl hx) q h
| inr h => exact content_mono B _ (fun _ hx => 0r.inr hx) q h

/-- Empty regions have content zero; this also checks the zero-square budget. -/
def emptyCover : Cover (fun _ => False) where

```

```

count := 0
squares := fun _ => (zeroPoint, (Fraction.ofInt 0, by decide))
covers := fun _ h => False.elim h

theorem empty_content (q : Fraction) :
  LowerContent (fun _ => False) q ↔ Fraction.le q (Fraction.ofInt 0) := by
  constructor
  · intro h
    exact h emptyCover
  · intro h
    exact content_downward _ q _ h (content_zero_lower _)

/-- Both singleton and zero-radius controls use actual completed-square membership. -/
def singletonCover (p : Point) : Cover (fun x => x = embedPosition p) where
  count := 1
  squares := fun _ => (p, (Fraction.ofInt 0, by decide))
  covers := by
    intro x hx
    subst x
    refine (0, by decide, ?_)
    constructor <=> exact (within_zero_iff _ _).mpr rfl

theorem singleton_content (p : Point) (q : Fraction) :
  LowerContent (fun x => x = embedPosition p) q ↔ Fraction.le q (Fraction.ofInt 0) := by
  constructor
  · intro h
    have hb := h (singletonCover p)
    exact Fraction.le_equiv_right hb (by
      simp [Cover.budget, singletonCover, sumBudget, Square.area, squareArea,
        Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt])
  · intro h
    exact content_downward _ q _ h (content_zero_lower _)

end NewtonLimitDynamics.Polygon.SquareOuterContent

```

BarrowLib/Common/RationalExhaustion.lean, 1 theorem, 33 lines

```

import BarrowLib.Common.RationalMagnitudes

/--! Rational exhaustion of a closed order bound, proved by the explicit
half-gap witness. No completeness, real order or calculus theorem is imported. -/
namespace NewtonLimitDynamics.Fraction

theorem le_of_enlargements (a b : Fraction)
  (h : ∀ eps : Fraction, 0 < eps.num → le a (add b eps)) : le a b := by
  apply Classical.byContradiction
  intro hn
  have hd : 0 < a.num * b.den - b.num * a.den := by
    unfold le at hn
    omega
  let eps : Fraction := (a.num * b.den - b.num * a.den,
    2 * a.den * b.den, Int.mul_pos (Int.mul_pos (by decide) a.den_pos) b.den_pos)
  have hh := h eps hd
  change a.num * (b.den * (2 * a.den * b.den)) ≤
    (b.num * (2 * a.den * b.den) +
      (a.num * b.den - b.num * a.den) * b.den) * a.den at hh
  have hp : 0 < (a.num * b.den - b.num * a.den) * (a.den * b.den) :=
    Int.mul_pos hd (Int.mul_pos a.den_pos b.den_pos)
  have he : a.num * (b.den * (2 * a.den * b.den)) =
    (b.num * (2 * a.den * b.den) +
      (a.num * b.den - b.num * a.den) * b.den) * a.den +
    (a.num * b.den - b.num * a.den) * (a.den * b.den) := by
    simp only [show (2 : Int) = 1+1 by rfl, Int.sub_mul, Int.add_mul, Int.mul_sub,
      Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  omega
  rw [he] at hh
  omega

end NewtonLimitDynamics.Fraction

```

BarrowLib/Polygon/RationalIntervals.lean, 6 theorems, 58 lines

```

import BarrowLib.Polygon.DyadicArithmetic

/-! Elementary rational interval bisection and gap identities. -/
namespace NewtonLimitDynamics.Polygon.RationalIntervals
open NewtonLimitDynamics
open HarmonicTimeComparison

theorem half_equiv {a b : Fraction} (h : Fraction.equiv a b) :
  Fraction.equiv a.half b.half := by
  change a.num*(2*b.den) = b.num*(2*a.den)
  calc
    _ = 2*(a.num*b.den) := by ac_rfl
    _ = 2*(b.num*a.den) := congrArg (fun n : Int => 2*n) h
    _ = _ := by ac_rfl

theorem half_le {a b : Fraction} (h : Fraction.le a b) :
  Fraction.le a.half b.half := by
  change a.num*(2*b.den) ≤ b.num*(2*a.den)
  calc
    _ = 2*(a.num*b.den) := by ac_rfl
    _ ≤ 2*(b.num*a.den) := Int.mul_le_mul_of_nonneg_left h (by decide)
    _ = _ := by ac_rfl

theorem half_double (a : Fraction) :
  Fraction.equiv (Fraction.add a a).half a := by
  simp only [Fraction.equiv, Fraction.half, Fraction.add]
  simp only [show (2 : Int) = 1+1 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

def midpoint (a b : Fraction) : Fraction := (Fraction.add a b).half

theorem midpoint_between (a b : Fraction) (h : Fraction.le a b) :
  Fraction.le a (midpoint a b) ∧ Fraction.le (midpoint a b) b := by
  constructor
  · exact Fraction.le_equiv_left (Fraction.equiv_symm (half_double a))
    (half_le (Fraction.add_le_add_left h a))
  · exact Fraction.le_equiv_right (half_le (Fraction.add_le_add_right h b))
    (half_double b)

theorem midpoint_lower_gap (a b : Fraction) :
  Fraction.equiv (durationDifference a (midpoint a b))
    (durationDifference a b).half := by
  simp only [midpoint, durationDifference, HarmonicStability.negF, Fraction.equiv,
    Fraction.half, Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  simp only [show (2 : Int) = 1+1 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  omega

theorem midpoint_upper_gap (a b : Fraction) :
  Fraction.equiv (durationDifference (midpoint a b) b)
    (durationDifference a b).half := by
  simp only [midpoint, durationDifference, HarmonicStability.negF, Fraction.equiv,
    Fraction.half, Fraction.add, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  simp only [show (2 : Int) = 1+1 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  omega

end NewtonLimitDynamics.Polygon.RationalIntervals

```

BarrowLib/Polygon/ScalarOrder.lean, 6 theorems, 72 lines

```

import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.BinaryTime
import BarrowLib.Common.RationalExhaustion

/-! A closed rational lower comparison for the first scalar coordinate of a
constructed Cauchy value. Representative invariance is proved before lifting. -/
namespace NewtonLimitDynamics.Polygon.ScalarOrder
open NewtonLimitDynamics

```

```

open TimeSubdivision PointBounds HarmonicDyadic HarmonicTimeComparison
open HarmonicComparison CauchyValues PositionValues BinaryTime

theorem first_coordinate_gap (a b : Point × Point) :
  Fraction.le (durationDifference b.1.1 a.1.1).abs (distance a b) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (scalarState_distance a.1.1 b.1.1))
  (first_nonexpansive a b)

def NameBelow (q : Fraction) (a : EndpointCauchyName) : Prop :=
  ∀ eps : Fraction, 0 < eps.num → ∃ N, ∀ n, N ≤ n →
  Fraction.le q (Fraction.add (a.approx n).1.1 eps)

theorem nameBelow_transport (q : Fraction) (a b : EndpointCauchyName)
  (hab : NameEquiv a b) (ha : NameBelow q a) : NameBelow q b := by
  intro eps heps
  obtain ⟨N,hN⟩ := ha eps.half heps
  obtain ⟨M,hM⟩ := hab eps.half heps
  refine ⟨max N M,?_⟩
  intro n hn
  have hg := Fraction.magnitudes.le_trans (first_coordinate_gap (a.approx n) (b.approx n))
    (Fraction.magnitudes.lt_implies_le (hM n (Nat.le_trans (Nat.le_max_right _ ) hn)))
  have hc := difference_add_bound (b.approx n).1.1 (a.approx n).1.1 eps.half
    (Fraction.magnitudes.le_trans (Fraction.le_abs _) hg)
  have hq := Fraction.magnitudes.le_trans
    (hN n (Nat.le_trans (Nat.le_max_left _ ) hn)) (Fraction.add_le_add_right hc eps.half)
  exact Fraction.le_equiv_right hq (Fraction.equiv_trans
    (Fraction.add_assoc _ eps.half eps.half)
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.half_add_self eps)))

theorem nameBelow_congr (q : Fraction) (a b : EndpointCauchyName)
  (hab : NameEquiv a b) : NameBelow q a ↔ NameBelow q b :=
  (nameBelow_transport q a b hab, nameBelow_transport q b a (nameEquiv_symm hab))

def Below (q : Fraction) (v : Value) : Prop :=
  Quotient.liftOn v (fun a => NameBelow q a)
  (fun a b h => propext (nameBelow_congr q a b h))

theorem below_realize (q : Fraction) (a : EndpointCauchyName) :
  Below q (realize a) ↔ NameBelow q a := Iff.rfl

theorem below_embed_iff (q : Fraction) (s : Point × Point) :
  Below q (embed s) ↔ Fraction.le q s.1.1 := by
  constructor
  · intro h
    apply Fraction.le_of_enlargements
    intro eps heps
    obtain ⟨N,hN⟩ := h eps heps
    exact hN N (Nat.le_refl _)
  · intro h eps heps
    exact ⟨0,fun _ _ => Fraction.magnitudes.le_trans h
      (Fraction.le_add_nonnegative _ _ (Int.le_of_lt heps))⟩

theorem below_downward (p q : Fraction) (v : Value)
  (hpq : Fraction.le p q) (hq : Below q v) : Below p v := by
  induction v using Quotient.inductionOn with
  | _ a =>
    intro eps heps
    obtain ⟨N,hN⟩ := hq eps heps
    exact ⟨N,fun n hn => Fraction.magnitudes.le_trans hpq (hN n hn)⟩

/-- Values fixed by the first-coordinate projection have no other component. -/
def ScalarValue := {v : Value // firstValue v = v}

end NewtonLimitDynamics.Polygon.ScalarOrder

```

BarrowLib/Polygon/BoundedCuts.lean, 21 theorems, 267 lines

```

import BarrowLib.Polygon.RationalIntervals
import BarrowLib.Polygon.ScalarOrder
import BarrowLib.Polygon.ScaledTolerance

```



```

/-! Cauchy realization of nonnegative bounded closed rational cuts by explicit
interval bisection. No scalar completeness field or desired Cauchy condition is
supplied; the shrinking intervals prove it. -/
namespace NewtonLimitDynamics.Polygon.BoundedCuts
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicTimeComparison
open HarmonicTimeRealization HarmonicComparison CauchyValues PositionValues BinaryTime ScalarOrder
↪ RationalIntervals

structure Cut where
  lower : Fraction → Prop
  zero_lower : lower (Fraction.ofInt 0)
  downward : ∀ p q, Fraction.le p q → lower q → lower p
  closed : ∀ q, (∀ eps : Fraction, 0 < eps.num →
    ∃ p, lower p ∧ Fraction.le q (Fraction.add p eps)) → lower q
  bound : Fraction
  bound_nonnegative : 0 ≤ bound.num
  bounded : ∀ q, lower q → Fraction.le q bound

noncomputable def step (c : Cut) (p : Fraction × Fraction) : Fraction × Fraction := by
  classical
  exact if c.lower (midpoint p.1 p.2) then (midpoint p.1 p.2, p.2)
    else (p.1, midpoint p.1 p.2)

noncomputable def interval (c : Cut) : Nat → Fraction × Fraction
| 0 => (Fraction.ofInt 0, c.bound)
| n+1 => step c (interval c n)

theorem lower_mem (c : Cut) : ∀ n, c.lower (interval c n).1
| 0 => c.zero_lower
| n+1 => by
  classical
  simp only [interval, step]
  split
  · assumption
  · exact lower_mem c n

theorem upper_bound (c : Cut) : ∀ n q, c.lower q → Fraction.le q (interval c n).2
| 0, q, hq => c.bounded q hq
| n+1, q, hq => by
  classical
  simp only [interval, step]
  split
  · exact upper_bound c n q hq
  · rename_i hm
    apply Classical.byContradiction
    intro hn
    change ↪ Fraction.le q (midpoint (interval c n).1 (interval c n).2) at hn
    have hmid : Fraction.le (midpoint (interval c n).1 (interval c n).2) q := by
      unfold Fraction.le at hn ⊢
      omega
    exact hm (c.downward _ _ hmid hq)

theorem interval_order (c : Cut) (n : Nat) :
  Fraction.le (interval c n).1 (interval c n).2 :=
  upper_bound c n _ (lower_mem c n)

theorem half_tail (A : Fraction) (n : Nat) :
  Fraction.equiv (GeometricTail.tailCap A n).half (GeometricTail.tailCap A (n+1)) := by
  simp only [Fraction.equiv, Fraction.half, GeometricTail.tailCap, Int.pow_succ]
  ac_nf

theorem width_cap (c : Cut) : ∀ n,
  Fraction.equiv (durationDifference (interval c n).1 (interval c n).2)
    (GeometricTail.tailCap c.bound n)
| 0 => by
  simp [interval, durationDifference, HarmonicStability.negF, GeometricTail.tailCap,
    Fraction.equiv, Fraction.add, Fraction.ofInt]
| n+1 => by
  classical

```

```

    have hh := Fraction.equiv_trans (half_equiv (width_cap c n)) (half_tail c.bound n)
    simp only [interval, step]
    split
    · exact Fraction.equiv_trans (midpoint_upper_gap _ _) hh
    · exact Fraction.equiv_trans (midpoint_lower_gap _ _) hh

theorem adjacent_bound (c : Cut) (n : Nat) :
  Fraction.le (FiniteEstimates.stateDistance
    (scalarState (interval c (n+1)).1) (scalarState (interval c n).1))
    (GeometricTail.tailCap c.bound (n+1)) := by
  classical
  have hcap : 0 ≤ (GeometricTail.tailCap c.bound (n+1)).num := c.bound_nonnegative
  have hh := Fraction.equiv_trans (midpoint_lower_gap (interval c n).1 (interval c n).2)
    (Fraction.equiv_trans (half_equiv (width_cap c n)) (half_tail c.bound n))
  simp only [interval, step]
  split
  · exact Fraction.le_of_equiv (Fraction.equiv_trans (scalarState_distance _ _))
    (Fraction.equiv_trans (Fraction.abs_equiv hh)
      (Fraction.abs_of_nonnegative _ hcap)))
  · exact Fraction.le_equiv_left (FiniteEstimates.stateDistance_self_zero _)
    (by simpa only [Fraction.le, Fraction.ofInt, Int.zero_mul, Int.mul_one] using hcap)

noncomputable def name (c : Cut) : EndpointCauchyName where
  approx := fun n => scalarState (interval c n).1
  cauchy := by
    intro eps heps
    refine (GeometricTail.modulus c.bound eps, ?_)
    intro m n hm hn
    exact Fraction.magnitudes.lt_of_le_lt
      (GeometricTail.two_sided (fun k => scalarState (interval c k).1) c.bound
        c.bound_nonnegative (adjacent_bound c) (GeometricTail.modulus c.bound eps) m n hm hn)
      (GeometricTail.doubleTail_lt_tolerance c.bound eps c.bound_nonnegative heps)

theorem name_realizes_cut (c : Cut) (q : Fraction) : NameBelow q (name c) ↔ c.lower q := by
  constructor
  · intro h
    apply c.closed q
    intro eps heps
    obtain (N, hN) := h eps heps
    exact ((interval c N).1, lower_mem c N, hN N (Nat.le_refl _))
  · intro hq eps heps
    obtain (N, hN) := duration_eventually_small c.bound eps c.bound_nonnegative heps
    refine (N, ?_)
    intro n hn
    have hwidth := Fraction.le_equiv_left (width_cap c n)
      (Fraction.magnitudes.lt_implies_le (hN n hn))
    have hupper := Fraction.le_equiv_right (upper_bound c n q hq)
      (Fraction.equiv_symm (add_difference_cancel (interval c n).1 (interval c n).2))
    exact Fraction.magnitudes.le_trans hupper
      (Fraction.add_le_add_left hwidth (interval c n).1)

noncomputable def value (c : Cut) : ScalarValue := (realize (name c), by rfl)

theorem value_realizes_cut (c : Cut) (q : Fraction) : Below q (value c).val ↔ c.lower q :=
  name_realizes_cut c q

theorem value_nonnegative (c : Cut) : Below (Fraction.ofInt 0) (value c).val :=
  (value_realizes_cut c _).mpr c.zero_lower

theorem value_bound (c : Cut) (q : Fraction) (hq : Below q (value c).val) :
  Fraction.le q c.bound := c.bounded q ((value_realizes_cut c q).mp hq)

theorem lower_nonnegative (c : Cut) : ∀ n, 0 ≤ (interval c n).1.num
| 0 => by simp [interval, Fraction.ofInt]
| n+1 => by
  classical
  have hlo := lower_nonnegative c n
  have h0 : Fraction.le (Fraction.ofInt 0) (interval c n).1 := by
    simpa only [Fraction.le, Fraction.ofInt, Int.zero_mul, Int.mul_one] using hlo
  have hm : Fraction.le (Fraction.ofInt 0)

```

```

      (midpoint (interval c n).1 (interval c n).2) :=
        Fraction.magnitudes.le_trans h0 (midpoint_between _ _ (interval_order c n)).1
    simp only [interval, step]
    split
    · simp only [Fraction.le, Fraction.ofInt, Int.zero_mul, Int.mul_one] using hm
    · exact hlo

theorem value_within_zero (c : Cut) :
  Within (value c).val (embed (scalarState (Fraction.ofInt 0))) c.bound := by
  apply nameBound_of_eventual_le _ _ _ 0
  intro n _
  have he := Fraction.equiv_trans (scalarState_distance (interval c n).1 (Fraction.ofInt 0))
    (Fraction.equiv_trans (Fraction.abs_equiv (show
      Fraction.equiv (durationDifference (Fraction.ofInt 0) (interval c n).1)
        (interval c n).1 by
        simp [durationDifference, HarmonicStability.negF, Fraction.equiv, Fraction.add, Fraction.ofInt]))
      (Fraction.abs_of_nonnegative _ (lower_nonnegative c n))))
  exact Fraction.le_equiv_left he (c.bounded _ (lower_mem c n))

theorem abs_width_cap (c : Cut) (n : Nat) :
  Fraction.equiv (durationDifference (interval c n).1 (interval c n).2).abs
    (GeometricTail.tailCap c.bound n) :=
  Fraction.equiv_trans (Fraction.abs_equiv (width_cap c n))
    (Fraction.abs_of_nonnegative _ c.bound_nonnegative)

/-- Different initial upper budgets cannot change the completed content. -/
theorem names_gap (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) (n : Nat) :
  Fraction.le (FiniteEstimates.stateDistance
    (scalarState (interval c n).1) (scalarState (interval d n).1))
    (GeometricTail.tailCap (Fraction.add c.bound d.bound) n) := by
  have hcd := upper_bound d n _ ((heq _).mp (lower_mem c n))
  have hdc := upper_bound c n _ ((heq _).mpr (lower_mem d n))
  have hsum := Fraction.le (FiniteEstimates.stateDistance
    (scalarState (interval c n).1) (scalarState (interval d n).1))
    (Fraction.add (GeometricTail.tailCap c.bound n) (GeometricTail.tailCap d.bound n)) := by
  by_cases horder : Fraction.le (interval d n).1 (interval c n).1
  · have hg := (difference_interval_gaps (interval d n).1 (interval c n).1
    (interval d n).2 horder hcd).1
    have hb := Fraction.le_equiv_right hg (abs_width_cap d n)
    have hlast := Fraction.le_equiv_right
      (Fraction.le_add_nonnegative (GeometricTail.tailCap d.bound n)
        (GeometricTail.tailCap c.bound n) c.bound_nonnegative)
      (Fraction.add_comm _ _)
    exact Fraction.le_equiv_left (scalarState_distance _ _) (Fraction.magnitudes.le_trans hb hlast)
  · have hreverse : Fraction.le (interval c n).1 (interval d n).1 := by
    unfold Fraction.le at horder ⊢
    omega
    have hg := (difference_interval_gaps (interval c n).1 (interval d n).1
      (interval c n).2 hreverse hdc).1
    have hb := Fraction.le_equiv_right hg (abs_width_cap c n)
    have hlast := Fraction.le_add_nonnegative (GeometricTail.tailCap c.bound n)
      (GeometricTail.tailCap d.bound n) d.bound_nonnegative
    have hbound := Fraction.le_equiv_left (scalarState_distance _ _)
      (Fraction.magnitudes.le_trans hb hlast)
    exact Fraction.le_equiv_left (FiniteEstimates.stateDistance_symm _ _) hbound
  exact Fraction.le_equiv_right hsum (GeometricTail.tail_add c.bound d.bound n)

theorem name_equiv_of_lower_iff (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) :
  NameEquiv (name c) (name d) := by
  intro eps heps
  obtain (N, hN) := duration_eventually_small (Fraction.add c.bound d.bound) eps
  (Fraction.nonnegative_add _ _ c.bound_nonnegative d.bound_nonnegative) heps
  exact (N, fun n hn => Fraction.magnitudes.lt_of_le_lt (names_gap c d heq n) (hN n hn))

theorem value_eq_of_lower_iff (c d : Cut) (heq : ∀ q, c.lower q ↔ d.lower q) :
  value c = value d := Subtype.ext (Quotient.sound (name_equiv_of_lower_iff c d heq))

theorem value_zero_of_bound_zero (c : Cut) (hz : c.bound.num = 0) :
  (value c).val = embed (scalarState (Fraction.ofInt 0)) := by
  apply (within_zero_iff _ _).mp

```

```

exact HarmonicTimeRealization.within_mono _ _ _
  (Fraction.le_of_equiv (show Fraction.equiv c.bound (Fraction.ofInt 0) by
    simp [Fraction.equiv,Fraction.ofInt,hz])) (value_within_zero c)

theorem value_of_rational_cut (c : Cut) (r : Fraction)
  (hr : ∀ q, c.lower q ⇔ Fraction.le q r) :
  (value c).val = embed (scalarState r) := by
  apply Quotient.sound
  intro eps heps
  obtain ⟨N,hN⟩ := duration_eventually_small c.bound eps c.bound_nonnegative heps
  refine ⟨N,?_⟩
  intro n hn
  have hl := (hr _).mp (lower_mem c n)
  have hu := upper_bound c n r ((hr r).mpr (Fraction.le_of_equiv (Fraction.equiv_refl _)))
  have hg := (difference_interval_gaps (interval c n).1 r (interval c n).2 hl hu).1
  have hb := Fraction.le_equiv_right hg (abs_width_cap c n)
  have hd := Fraction.le_equiv_left (scalarState_distance r (interval c n).1) hb
  exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_equiv_left (FiniteEstimates.stateDistance_symm _ _) hd) (hN n hn)

def rationalCut (r B : Fraction) (hr : 0 ≤ r.num) (hB : 0 ≤ B.num)
  (hrB : Fraction.le r B) : Cut where
  lower := fun q => Fraction.le q r
  zero_lower := by simp only [Fraction.le,Fraction.ofInt,Int.zero_mul,Int.mul_one] using hr
  downward := fun _ _ hpq hq => Fraction.magnitudes.le_trans hpq hq
  closed := by
    intro q h
    apply Fraction.le_of_enlargements
    intro eps heps
    obtain ⟨p,hp,hqp⟩ := h eps heps
    exact Fraction.magnitudes.le_trans hqp (Fraction.add_le_add_right hp eps)
  bound := B
  bound_nonnegative := hB
  bounded := fun _ hq => Fraction.magnitudes.le_trans hq hrB

theorem rationalCut_value (r B : Fraction) (hr : 0 ≤ r.num) (hB : 0 ≤ B.num)
  (hrB : Fraction.le r B) :
  (value (rationalCut r B hr hB hrB)).val = embed (scalarState r) :=
  value_of_rational_cut _ r (fun _ => Iff.rfl)

private def oneThird : Fraction := (1,3,by decide)
private def firstControl : Cut := rationalCut oneThird (Fraction.ofInt 1)
  (by decide) (by decide) (by unfold Fraction.le oneThird Fraction.ofInt; decide)
private def secondControl : Cut := rationalCut oneThird (Fraction.ofInt 2)
  (by decide) (by decide) (by unfold Fraction.le oneThird Fraction.ofInt; decide)

/-- A known nonzero content cut recovers its rational value with different budgets. -/
theorem one_third_control_value : (value firstControl).val = embed (scalarState oneThird) :=
  rationalCut_value _ _ _ _

theorem one_third_different_budgets : value firstControl = value secondControl :=
  value_eq_of_lower_iff _ _ (fun _ => Iff.rfl)

end NewtonLimitDynamics.Polygon.BoundedCuts

```

BarrowLib/Polygon/SquareContentValues.lean, 8 theorems, 68 lines

```

import BarrowLib.Polygon.SquareOuterContent
import BarrowLib.Polygon.BoundedCuts

/--! The elementary all-cover outer-content cut is realized as a Cauchy scalar
by shrinking rational intervals. Different proved initial covers produce the
same value. An ordinary measure or inner-area identification remains separate. -/
namespace NewtonLimitDynamics.Polygon.SquareContentValues
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic CauchyValues PositionValues BinaryTime ScalarOrder
open SquareOuterContent

def contentCut (A : PositionValue → Prop) (c : Cover A) : BoundedCuts.Cut where
  lower := LowerContent A
  zero_lower := content_zero_lower A

```

```

downward := content_downward A
closed := content_closed A
bound := c.budget
bound_nonnegative := c.budget_nonnegative
bounded := fun _ hq => hq c

noncomputable def contentName (A : PositionValue → Prop) (c : Cover A) : EndpointCauchyName :=
  BoundedCuts.name (contentCut A c)

noncomputable def contentValue (A : PositionValue → Prop) (c : Cover A) : ScalarValue :=
  BoundedCuts.value (contentCut A c)

theorem contentValue_lower_cut (A : PositionValue → Prop) (c : Cover A) (q : Fraction) :
  Below q (contentValue A c).val ↔ LowerContent A q :=
  BoundedCuts.value_realizes_cut (contentCut A c) q

theorem contentValue_nonnegative (A : PositionValue → Prop) (c : Cover A) :
  Below (Fraction.ofInt 0) (contentValue A c).val :=
  BoundedCuts.value_nonnegative (contentCut A c)

theorem contentValue_within_zero (A : PositionValue → Prop) (c : Cover A) :
  Within (contentValue A c).val (embed (scalarState (Fraction.ofInt 0))) c.budget :=
  BoundedCuts.value_within_zero (contentCut A c)

theorem contentValue_independent_cover (A : PositionValue → Prop) (c d : Cover A) :
  contentValue A c = contentValue A d :=
  BoundedCuts.value_eq_of_lower_iff (contentCut A c) (contentCut A d) (fun _ => Iff.rfl)

/-- Equal point sets have the same scalar content, even when their proved
initial covers and bounds differ. -/
theorem contentValue_region_congr (A B : PositionValue → Prop)
  (h : ∀ x, A x ↔ B x) (c : Cover A) (d : Cover B) :
  contentValue A c = contentValue B d :=
  BoundedCuts.value_eq_of_lower_iff (contentCut A c) (contentCut B d) (fun q =>
    (content_mono A B (fun x hx => (h x).mp hx) q,
    content_mono B A (fun x hx => (h x).mpr hx) q))

/-- Any actual cover bounds the canonical value, regardless of the cover used
for its bisection construction. -/
theorem contentValue_any_cover_bound (A : PositionValue → Prop) (c d : Cover A) :
  Within (contentValue A c).val (embed (scalarState (Fraction.ofInt 0))) d.budget := by
  rw [contentValue_independent_cover A c d]
  exact contentValue_within_zero A d

theorem empty_value_zero :
  (contentValue (fun _ => False) emptyCover).val = embed (scalarState (Fraction.ofInt 0)) :=
  BoundedCuts.value_zero_of_bound_zero _ rfl

theorem singleton_value_zero (p : Point) :
  (contentValue (fun x => x = embedPosition p) (singletonCover p)).val =
  embed (scalarState (Fraction.ofInt 0)) :=
  BoundedCuts.value_zero_of_bound_zero _ rfl

end NewtonLimitDynamics.Polygon.SquareContentValues

```

BarrowLib/Polygon/FiniteSequenceGap.lean, 1 theorem, 30 lines

```

import BarrowLib.Polygon.FiniteEstimates

/-! A finite telescoping estimate for any rational-valued distance with a
zero diagonal and triangle inequality. No map or limiting premise is used. -/

namespace NewtonLimitDynamics.Polygon.FiniteSequenceGap
open NewtonLimitDynamics
open TimeSubdivision

theorem finite_gap (D : (Point × Point) → (Point × Point) → Fraction)
  (hz : ∀ s, Fraction.equiv (D s s) (Fraction.ofInt 0))
  (htri : ∀ s t u, Fraction.le (D s u) (Fraction.add (D s t) (D t u)))
  (f : Nat → Point × Point) (N : Nat) (C : Fraction)
  (hstep : ∀ i, i < N → Fraction.le (D (f (i+1)) (f i)) C) :

```

```

    (n k : Nat) → n+k ≤ N →
      Fraction.le (D (f (n+k)) (f n)) (Fraction.mul (Fraction.ofInt (k : Int)) C)
| n, 0, _ => Fraction.le_of_equiv (Fraction.equiv_trans (hz (f n)) (by
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.natCast_zero]
  simp))
| n, k+1, hnk => by
  have hp := finite_gap D hz htri f N C hstep n k (by omega)
  have hs := hstep (n+k) (by omega)
  have hb := Fraction.magnitudes.le_trans (htri (f (n+(k+1))) (f (n+k)) (f n))
    (Fraction.add_le_add hs hp)
  apply Fraction.le_equiv_right hb
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
end NewtonLimitDynamics.Polygon.FiniteSequenceGap

```

BarrowLib/Polygon/BinaryEndpoints.lean, 8 theorems, 130 lines

```

import BarrowLib.Polygon.BinaryTime
import BarrowLib.Polygon.GeometricTail

/-! Constructed time coordinates and endpoints, with retained public names.
No motion or harmonic coefficient is a premise. -/
namespace NewtonLimitDynamics.Polygon.HarmonicTimeRealization
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicStability HarmonicDyadic HarmonicBinaryPrefix
open HarmonicTimeComparison HarmonicComparison HarmonicAccumulation CauchyValues BinaryTime

def timeCoordinate (T : Fraction) (hT : 0 ≤ T.num) :
  BinaryTime T hT → Value :=
  Quotient.lift (fun b => BinaryTime.timeValue b T hT)
    (fun _ _ h => Quotient.sound h)

theorem timeCoordinate_injective (T : Fraction) (hT : 0 ≤ T.num) :
  ∀ x y : BinaryTime T hT,
    timeCoordinate T hT x = timeCoordinate T hT y → x = y := by
  intro x y hxy
  induction x using Quotient.inductionOn with
  | _ b =>
    induction y using Quotient.inductionOn with
    | _ c =>
      change BinaryTime.timeValue b T hT =
        BinaryTime.timeValue c T hT at hxy
      have hnames : NameEquiv (BinaryTime.timeName b T hT)
        (BinaryTime.timeName c T hT) := Quotient.exact hxy
      exact Quotient.sound hnames

def TimeWithin (T : Fraction) (hT : 0 ≤ T.num)
  (x y : BinaryTime T hT) (R : Fraction) : Prop :=
  Within (timeCoordinate T hT x) (timeCoordinate T hT y) R

theorem timeWithin_address (b c : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (R : Fraction) :
  TimeWithin T hT (Quotient.mk _ b) (Quotient.mk _ c) R ↔
  NameBound (BinaryTime.timeName b T hT)
    (BinaryTime.timeName c T hT) R := Iff.rfl

def leftAddress : Nat → Bool := fun _ => false
def rightAddress : Nat → Bool := fun _ => true

def leftTime (T : Fraction) (hT : 0 ≤ T.num) : BinaryTime T hT :=
  Quotient.mk _ leftAddress
def rightTime (T : Fraction) (hT : 0 ≤ T.num) : BinaryTime T hT :=
  Quotient.mk _ rightAddress

theorem left_time_state_equiv (T : Fraction) (j : Nat) :
  stateEquiv (timeState leftAddress T j)
    (scalarState (Fraction.ofInt 0)) := by
  have ht : Fraction.equiv (timeApprox leftAddress T j)
    (Fraction.ofInt 0) := by

```

```

    have hz : ticks leftAddress j = 0 :=
      all_zero_ticks j
    simp only [timeApprox, hz, Fraction.equiv, Fraction.mul,
      Fraction.ofInt]
    simp
    exact ((ht, Fraction.equiv_refl _),
      (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem left_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) :
  timeCoordinate T hT (leftTime T hT) =
    embed (scalarState (Fraction.ofInt 0)) := by
  apply Quotient.sound
  intro eps heps
  refine (0, ?_)
  intro j _
  have he := (distance_zero_iff_stateEquiv
    (timeState leftAddress T j)
    (scalarState (Fraction.ofInt 0))).mpr
    (left_time_state_equiv T j)
  have hself := stateSub_self_norm_zero
    (scalarState (Fraction.ofInt 0))
  have hle : Fraction.le
    (distance (timeState leftAddress T j)
      (scalarState (Fraction.ofInt 0)))
    (distance (scalarState (Fraction.ofInt 0))
      (scalarState (Fraction.ofInt 0))) := Fraction.le_of_equiv
    (Fraction.equiv_trans he (Fraction.equiv_symm hself))
  exact Fraction.magnitudes.lt_of_le_lt hle
    (distance_self_lt _ eps heps)

theorem right_ticks (j : Nat) : ticks rightAddress j + 1 = blocks j := by
  induction j with
  | zero => simp [ticks, rightAddress, bit, blocks]
  | succ j ih =>
    rw [ticks, blocks_succ]
    simp only [rightAddress, bit, ite_true]
    omega

theorem right_time_difference (T : Fraction) (j : Nat) :
  Fraction.equiv (durationDifference (timeApprox rightAddress T j) T)
    (duration T j) := by
  have hk : (ticks rightAddress j : Int) + 1 = (2 : Int) ^ j := by
    have hr := right_ticks j
    have hr' := congrArg Int.ofNat hr
    change ((ticks rightAddress j + 1 : Nat) : Int) =
      ((2 ^ j : Nat) : Int) at hr'
    rw [Int.natCast_add, Int.natCast_one, Int.natCast_pow] at hr'
    exact hr'
  simp only [timeApprox, durationDifference, negF, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, duration]
  simp only [Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  rw [← hk]
  simp only [Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf <;> omega

theorem right_time_distance (T : Fraction) (j : Nat)
  (hT : 0 ≤ T.num) :
  Fraction.equiv (distance (timeState rightAddress T j)
    (scalarState T)) (duration T j) := by
  have h1 := scalarState_distance (timeApprox rightAddress T j) T
  have h2 := durationDifference_abs_symm T (timeApprox rightAddress T j)
  have h3 := Fraction.abs_equiv (right_time_difference T j)
  have h4 := Fraction.abs_of_nonnegative (duration T j) hT
  exact Fraction.equiv_trans h1
    (Fraction.equiv_trans h2 (Fraction.equiv_trans h3 h4))

theorem right_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) :
  timeCoordinate T hT (rightTime T hT) = embed (scalarState T) := by
  apply Quotient.sound
  intro eps heps

```

```

obtain (N, hN) := duration_eventually_small T eps hT heps
refine (N, ?_)
intro j hj
exact Fraction.magnitudes.lt_of_le_lt
  ((Fraction.equiv_iff_mutual_le _ _).mp
    (right_time_distance T j hT)).1 (hN j hj)

```

end NewtonLimitDynamics.Polygon.HarmonicTimeRealization

BarrowLib/Polygon/TailValues.lean, 1 theorem, 22 lines

```

import BarrowLib.Polygon.CauchyValues
import BarrowLib.Polygon.GeometricTail

/-! A shared closed-value bound from an actual name's adjacent geometric
estimate. Applications supply their derived finite estimate and coefficient. -/

namespace NewtonLimitDynamics.Polygon.TailValues
open NewtonLimitDynamics
open HarmonicDyadic CauchyValues FiniteEstimates

theorem approximant_bound (a : EndpointCauchyName) (A : Fraction) (hA : 0 ≤ A.num)
  (hadj : ∀ j, Fraction.le (distance (a.approx (j+1)) (a.approx j))
    (GeometricTail.tailCap A (j+1))) (m : Nat) :
  Within (embed (a.approx m)) (realize a) (GeometricTail.tailCap A m) := by
  apply nameBound_of_eventual_le _ _ m
  intro n hn
  have he : m+(n-m)=n := by omega
  have hb := GeometricTail.finite_gap a.approx A hA hadj (n-m) m
  rw [he] at hb
  exact Fraction.le_equiv_left (stateDistance_symm _ _) hb

end NewtonLimitDynamics.Polygon.TailValues

```

BarrowLib/Polygon/KinematicEstimates.lean, 11 theorems, 159 lines

```

import BarrowLib.Polygon.TimeCalibration
import BarrowLib.Polygon.FiniteSequenceGap

/-! Finite drift/kick remainders at actual sampled arrivals. The  $O(t^2)$ 
position remainder and  $O(t)$  velocity change are derived from the recurrence;
no derivative, curve, integral or ODE theorem is a premise. -/

namespace NewtonLimitDynamics.Polygon.KinematicEstimates
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates BoundedIteration

def inertialPosition (h : Fraction) (s : Point × Point) (n : Nat) : Point :=
  pointAdd s.1 (pointScale (time h n) s.2)

theorem inertial_zero (h : Fraction) (s : Point × Point) :
  pointEquiv (inertialPosition h s 0) s.1 := by
  constructor <=>
  simp only [inertialPosition, time, pointEquiv, pointAdd, pointScale, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, Int.natCast_zero, Int.zero_mul,
    Int.mul_zero, Int.add_zero, Int.one_mul, Int.mul_one] <=> ac_nf

theorem inertial_step (h : Fraction) (s : Point × Point) (n : Nat) :
  pointEquiv (inertialPosition h s (n+1))
    (pointAdd (inertialPosition h s n) (pointScale h s.2)) := by
  constructor <=>
  simp only [inertialPosition, time, pointEquiv, pointAdd, pointScale, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, Int.natCast_add, Int.natCast_one,
    Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one] <=> ac_nf

theorem velocity_increment (a : Point → Point) (h : Fraction) (s : Point × Point) :
  Fraction.equiv (pointDistance (cell a h s).2 s.2)
    (Fraction.mul h.abs (pointNorm (a (cell a h s).1))) :=
  Fraction.equiv_trans (pointNorm_equiv (ConvexCover.drift_offset h s.2 (a (cell a h s).1)))
    (pointNorm_scale h (a (cell a h s).1))

```



```

theorem velocity_displacement (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B : Fraction) (hh : 0 ≤ h.num) (n : Nat) (hb : BoundedSamples a h s B n) :
  Fraction.le (pointDistance (run a h s n).2 s.2) (Fraction.mul (time h n) B) := by
  have hs : ∀ i, i < n → Fraction.le (pointDistance (run a h s (i+1)).2 (run a h s i).2)
    (Fraction.mul h B) := by
    intro i hi
    exact Fraction.le_equiv_left (velocity_increment a h (run a h s i))
      (Fraction.le_equiv_right
        (Fraction.mul_le_mul_nonnegative_left (hb i hi) h.abs (Fraction.abs_num_nonnegative h))
        (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl B)))
  have hg := FiniteSequenceGap.finite_gap (fun x y => pointDistance x.2 y.2)
    (fun x => pointDistance_self_zero x.2) (fun x y z => pointDistance_triangle x.2 y.2 z.2)
    (run a h s) n (Fraction.mul h B) hs 0 n (by omega)
  simp only [Nat.zero_add, run] using
    Fraction.le_equiv_right hg (Fraction.equiv_symm (Fraction.mul_assoc _ _ _))

theorem quadratic_step (h B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat) :
  Fraction.le
    (Fraction.add (Fraction.mul (Fraction.mul (time h n) (time h n)) B)
      (Fraction.mul h (Fraction.mul (time h n) B)))
    (Fraction.mul (Fraction.mul (time h (n+1)) (time h (n+1))) B) := by
  let A := Fraction.add (Fraction.mul (Fraction.mul (time h n) (time h n)) B)
    (Fraction.mul h (Fraction.mul (time h n) B))
  let E := Fraction.mul (Fraction.mul (time h (n+1)) h) B
  have hE : 0 ≤ E.num := Fraction.nonnegative_mul
    (Fraction.nonnegative_mul _ _ (time_nonnegative h hh) hh) hB
  apply Fraction.le_equiv_right (Fraction.le_add_nonnegative A E hE)
  simp only [A, E, time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

/-- Position's actual finite remainder from its inertial continuation. -/
theorem position_remainder (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat)
  (hb : BoundedSamples a h s B n) :
  Fraction.le (pointDistance (run a h s n).1 (inertialPosition h s n))
    (Fraction.mul (Fraction.mul (time h n) (time h n)) B) := by
  induction n with
  | zero =>
    have he := inertial_zero h s
    apply Fraction.le_of_equiv
    apply Fraction.equiv_trans (pointDistance_equiv
      (Fraction.equiv_refl _, Fraction.equiv_refl _) he)
    apply Fraction.equiv_trans (pointDistance_self_zero s.1)
    simp [time, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  | succ n ih =>
    have hb0 : BoundedSamples a h s B n := fun i hi => hb i (by omega)
    have hv := velocity_displacement a h s B hh n hb0
    have hs := Fraction.le_equiv_left (difference_scale h (run a h s n).2 s.2)
      (Fraction.le_equiv_right
        (Fraction.mul_le_mul_nonnegative_left hv h.abs (Fraction.abs_num_nonnegative h))
        (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh)
          (Fraction.equiv_refl (Fraction.mul (time h n) B))))
    have ha := difference_add_bound (run a h s n).1 (inertialPosition h s n)
      (pointScale h (run a h s n).2) (pointScale h s.2)
    have hc := Fraction.magnitudes.le_trans ha (Fraction.add_le_add (ih hb0) hs)
    have hd := Fraction.le_equiv_left
      (pointDistance_equiv (p := (run a h s (n+1)).1) (Fraction.equiv_refl _, Fraction.equiv_refl _)
        (inertial_step h s n)) hc
    exact Fraction.magnitudes.le_trans hd (quadratic_step h B hh hB n)

/-- The finite quotient remainder is obtained by dividing the derived remainder. -/
theorem secant_identity (t : Fraction) (ht : 0 < t.num) (p x v : Point) :
  pointEquiv
    (pointSub (pointScale (TimeCalibration.inverse t ht) (pointSub p x)) v)
    (pointScale (TimeCalibration.inverse t ht)
      (pointSub p (pointAdd x (pointScale t v)))) := by
  constructor <=>
  simp only [TimeCalibration.inverse, pointEquiv, pointSub, pointAdd, pointNeg, pointScale,
    Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>

```

```

ac_nf <=> omega

theorem position_secant_bound (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat)
  (ht : 0 < (time h n).num) (hb : BoundedSamples a h s B n) :
  Fraction.le
    (pointDistance (pointScale (TimeCalibration.inverse (time h n) ht)
      (pointSub (run a h s n).1 s.1)) s.2)
    (Fraction.mul (time h n) B) := by
  have he := Fraction.equiv_trans (pointNorm_equiv (secant_identity (time h n) ht)
    (run a h s n).1 s.1 s.2))
    (pointNorm_scale (TimeCalibration.inverse (time h n) ht)
      (pointSub (run a h s n).1 (inertialPosition h s n)))
  have hm := Fraction.mul_le_mul_nonnegative_left (position_remainder a h s B hh hB n hb)
    (TimeCalibration.inverse (time h n) ht).abs (Fraction.abs_num_nonnegative _)
  apply Fraction.le_equiv_right (Fraction.le_equiv_left he hm)
  rw [Fraction.abs_eq_of_nonnegative (TimeCalibration.inverse (time h n) ht)
    (Int.le_of_lt (time h n).den_pos)]
  simp only [TimeCalibration.inverse, Fraction.equiv, Fraction.mul]
  ac_nf

theorem position_secant_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point)
  (B u : Fraction) (hh : 0 ≤ h.num) (hB : 0 ≤ B.num) (n : Nat)
  (ht : 0 < (time h n).num) (hu : 0 < u.num) (he : Fraction.equiv u (time h n))
  (hb : BoundedSamples a h s B n) :
  Fraction.le
    (pointDistance (pointScale (TimeCalibration.inverse u hu)
      (pointSub (run a h s n).1 s.1)) s.2) (Fraction.mul u B) :=
  Fraction.le_equiv_right
    (Fraction.le_equiv_left (pointDistance_equiv
      (pointScale_ratio_congr (TimeCalibration.inverse_congr hu ht he)
        (Fraction.equiv_refl _, Fraction.equiv_refl _))
      (Fraction.equiv_refl _, Fraction.equiv_refl _))
      (position_secant_bound a h s B hh hB n ht hb))
    (Fraction.mul_equiv (Fraction.equiv_symm he) (Fraction.equiv_refl B))

private def controlHalf : Fraction := {1,2,by decide}
private def controlState : Point × Point :=
  ((Fraction.ofInt 0, Fraction.ofInt 0), (Fraction.ofInt 1, Fraction.ofInt 0))
private def controlForce ( _ : Point ) : Point := (Fraction.ofInt 0, Fraction.ofInt (-1))

theorem two_cell_secant_control :
  Fraction.equiv
    (pointDistance (pointScale (TimeCalibration.inverse (time controlHalf 2) (by decide))
      (pointSub (run controlForce controlHalf controlState 2).1 controlState.1)) controlState.2)
    (1,4,by decide) := by decide

theorem zero_bound_rejects_secant_control :
  ¬ Fraction.le
    (pointDistance (pointScale (TimeCalibration.inverse (time controlHalf 2) (by decide))
      (pointSub (run controlForce controlHalf controlState 2).1 controlState.1)) controlState.2)
    (Fraction.ofInt 0) := by unfold Fraction.le; decide

end NewtonLimitDynamics.Polygon.KinematicEstimates

```

BarrowLib/Polygon/SecantValues.lean, 11 theorems, 120 lines

```

import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.ScaledTolerance

/-! Rationally scaled position differences and velocity projections of
constructed Cauchy values. All Cauchy and representative-invariance proofs
precede quotient lifting; no rate-of-change premise is supplied. -/

namespace NewtonLimitDynamics.Polygon.SecantValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic CauchyValues PositionValues
open FiniteEstimates HarmonicTimeRealization

def pointState (p : Point) : Point × Point := (p, zeroPoint)
def velocityState (s : Point × Point) : Point × Point := pointState s.2

```

```

theorem pointState_distance (p q : Point) :
  Fraction.equiv (distance (pointState p) (pointState q)) (pointDistance p q) :=
  Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _))
    (pointDistance_self_zero zeroPoint)) (Fraction.add_zero _)

theorem velocity_nonexpansive (s t : Point × Point) :
  Fraction.le (distance (velocityState s) (velocityState t)) (distance s t) :=
  Fraction.le_equiv_left (pointState_distance s.2 t.2) (velocity_le_state (stateSub s t))

def velocityValue : Value → Value := mapValue velocityState velocity_nonexpansive

theorem velocityValue_within (x y : Value) (R : Fraction) (h : Within x y R) :
  Within (velocityValue x) (velocityValue y) R := mapValue_within _ velocity_nonexpansive x y R h

theorem velocityValue_embed (s : Point × Point) :
  velocityValue (embed s) = embed (velocityState s) := mapValue_embed _ velocity_nonexpansive s

def secantState (q : Fraction) (s t : Point × Point) : Point × Point :=
  pointState (pointScale q (pointSub s.1 t.1))

theorem secant_distance_bound (q : Fraction) (a b c d : Point × Point) :
  Fraction.le (distance (secantState q a b) (secantState q c d))
    (Fraction.mul q.abs (Fraction.add (distance a c) (distance b d))) := by
  have hp := Fraction.magnitudes.le_trans (difference_sub_bound a.1 c.1 b.1 d.1)
    (Fraction.add_le_add (point_le_state (stateSub a c)) (point_le_state (stateSub b d)))
  exact Fraction.le_equiv_left (Fraction.equiv_trans (pointState_distance _))
    (difference_scale q _))
    (Fraction.mul_le_mul_nonnegative_left hp q.abs (Fraction.abs_num_nonnegative q))

theorem two_scaled_small (q eps r s : Fraction) (_heps : 0 < eps.num)
  (hr : 0 ≤ r.num) (hs : 0 ≤ s.num)
  (h1 : Fraction.lt r (factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q)))
  (h2 : Fraction.lt s (factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q))) :
  Fraction.lt (Fraction.mul q.abs (Fraction.add r s)) eps := by
  have ha := factor_control q.abs eps.half r (Fraction.abs_num_nonnegative q) hr h1
  have hb := factor_control q.abs eps.half s (Fraction.abs_num_nonnegative q) hs h2
  have hc := Fraction.add_lt_add ha hb
  have he := Fraction.half_add_self eps
  have hd := lt_equiv_right hc he
  exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (Fraction.equiv_trans (Fraction.mul_comm _)) (Fraction.add_mul _ _)) hd

def secantName (q : Fraction) (a b : EndpointCauchyName) : EndpointCauchyName where
  approx := fun n => secantState q (a.approx n) (b.approx n)
  cauchy := by
    intro eps heps
    let delta := factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q)
    have hd : 0 < delta.num := factorDelta_positive _ _ _ heps
    obtain ⟨N,hN⟩ := a.cauchy delta hd
    obtain ⟨M,hM⟩ := b.cauchy delta hd
    refine ⟨max N M, fun m n hm hn => ?_⟩
    exact Fraction.magnitudes.lt_of_le_lt (secant_distance_bound q _ _ _ _)
      (two_scaled_small q eps _ _ heps (stateNorm_nonnegative _) (stateNorm_nonnegative _)
        (hN m n (by omega) (by omega)) (hM m n (by omega) (by omega))))

theorem secantName_equiv (q : Fraction) (a b a' b' : EndpointCauchyName)
  (ha : NameEquiv a a') (hb : NameEquiv b b') :
  NameEquiv (secantName q a b) (secantName q a' b') := by
  intro eps heps
  let delta := factorDelta q.abs eps.half (Fraction.abs_num_nonnegative q)
  have hd : 0 < delta.num := factorDelta_positive _ _ _ heps
  obtain ⟨N,hN⟩ := ha delta hd
  obtain ⟨M,hM⟩ := hb delta hd
  refine ⟨max N M, fun n hn => ?_⟩
  exact Fraction.magnitudes.lt_of_le_lt (secant_distance_bound q _ _ _ _)
    (two_scaled_small q eps _ _ heps (stateNorm_nonnegative _) (stateNorm_nonnegative _)
      (hN n (by omega) (by omega)) (hM n (by omega) (by omega))))

def secantValue (q : Fraction) (x y : Value) : Value :=

```

```

Quotient.liftOn₂ x y (fun a b => realize (secantName q a b))
  (fun a b a' b' ha hb => Quotient.sound (secantName_equiv q a b a' b' ha hb))

theorem secantValue_realize (q : Fraction) (a b : EndpointCauchyName) :
  secantValue q (realize a) (realize b) = realize (secantName q a b) := rfl

theorem secantValue_embed (q : Fraction) (s t : Point × Point) :
  secantValue q (embed s) (embed t) = embed (secantState q s t) := rfl

/-- Position of the tangent continuation, using the existing completed
difference and velocity operators. -/
def inertialName (h : Fraction) (a : EndpointCauchyName) : EndpointCauchyName :=
  secantName (Fraction.ofInt 1) a
  (secantName (HarmonicStability.negF h) (mapName velocityState velocity_nonexpansive a)
    (constantName (zeroPoint, zeroPoint)))

def inertialValue (h : Fraction) (x : Value) : Value :=
  secantValue (Fraction.ofInt 1) x
  (secantValue (HarmonicStability.negF h) (velocityValue x) (embed (zeroPoint, zeroPoint)))

def shiftedName (a : EndpointCauchyName) (m : Nat) : EndpointCauchyName where
  approx := fun j => a.approx (m+j)
  cauchy := by
    intro eps heps
    obtain ⟨N, hN⟩ := a.cauchy eps heps
    exact ⟨N, fun i j hi hj => hN (m+i) (m+j) (by omega) (by omega)⟩

theorem shiftedName_equiv (a : EndpointCauchyName) (m : Nat) : NameEquiv (shiftedName a m) a := by
  intro eps heps
  obtain ⟨N, hN⟩ := a.cauchy eps heps
  exact ⟨N, fun j hj => hN (m+j) j (by omega) hj⟩

theorem shiftedValue (a : EndpointCauchyName) (m : Nat) : realize (shiftedName a m) = realize a :=
  Quotient.sound (shiftedName_equiv a m)

end NewtonLimitDynamics.Polygon.SecantValues

```

BarrowLib/Polygon/DyadicNodes.lean, 6 theorems, 68 lines

```

import BarrowLib.Polygon.FiniteAddress
import BarrowLib.Polygon.BinaryEndpoints
import BarrowLib.Polygon.SecantValues
import BarrowLib.Polygon.TailValues

/-! Actual dyadic nodes of the constructed binary-time domain, including
the right endpoint. A truncation lies within one cell of its time value. -/

namespace NewtonLimitDynamics.Polygon.DyadicNodes
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicTimeComparison
open HarmonicTimeRealization CauchyValues BinaryTime SecantValues

theorem blocks_add (m j : Nat) : blocks (m+j)=blocks m*blocks j := by
  simp only [blocks, Nat.pow_add]

theorem finiteAddress_later_ticks (m k j : Nat) (hk : k<blocks m) :
  ticks (finiteAddress m k) (m+j)=k*blocks j := by
  rw [ticks_zero_tail _ m (finiteAddress_tail m k), finiteAddress_ticks m k hk]

theorem grid_time (T : Fraction) (m j : Nat) :
  Fraction.equiv (Fraction.mul (Fraction.ofInt (blocks j : Int)) (duration T (m+j)))
    (duration T m) := by
  simp only [blocks, duration, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.natCast_pow,
    FiniteGrowth.denominator_power_add]
  change (2: Int)^j * T.num * (T.den * 2^m) = T.num * (1 * (T.den * (2^m * 2^j)))
  simp only [Int.one_mul]
  ac_rfl

theorem finiteAddress_time (T : Fraction) (m k j : Nat) (hk : k<blocks m) :
  Fraction.equiv (timeApprox (finiteAddress m k) T (m+j)) (countTime T m k) := by
  rw [timeApprox, finiteAddress_later_ticks m k j hk]

```

```

have he := Fraction.mul_equiv_left (Fraction.ofInt (k : Int)) (grid_time T m j)
apply Fraction.equiv_trans _ he
simp only [countTime, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.natCast_mul]
ac_nf

def nodeTime (T : Fraction) (hT : 0 ≤ T.num) (m k : Nat) : BinaryTime T hT :=
  if k=blocks m then rightTime T hT else Quotient.mk _ (finiteAddress m k)

theorem node_time_coordinate (T : Fraction) (hT : 0 ≤ T.num) (m k : Nat) (hk : k≤blocks m) :
  timeCoordinate T hT (nodeTime T hT m k) = embed (scalarState (countTime T m k)) := by
  by_cases he : k=blocks m
  · rw [nodeTime, if_pos he, right_time_coordinate]
    subst k
    apply Quotient.sound
    apply nameEquiv_of_levelwise_stateEquiv
    intro j
    exact ((Fraction.equiv_symm (blocks_duration T m), Fraction.equiv_refl _),
      (Fraction.equiv_refl _, Fraction.equiv_refl _))
  · rw [nodeTime, if_neg he]
    change realize (BinaryTime.timeName (finiteAddress m k) T hT) = _
    rw [← shiftedValue (BinaryTime.timeName (finiteAddress m k) T hT) m]
    apply Quotient.sound
    apply nameEquiv_of_levelwise_stateEquiv
    intro j
    exact ((finiteAddress_time T m k j (by omega), Fraction.equiv_refl _),
      (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem truncation_time_within (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) :
  TimeWithin T hT (nodeTime T hT m (ticks b m)) (Quotient.mk _ b) (duration T m) := by
  change Within (timeCoordinate T hT (nodeTime T hT m (ticks b m)))
    (BinaryTime.timeValue b T hT) (duration T m)
  rw [node_time_coordinate T hT m (ticks b m) (ticks_le_blocks b m)]
  exact TailValues.approximant_bound (BinaryTime.timeName b T hT) T hT
    (fun j => adjacent_time_bound b T j hT) m

```

end NewtonLimitDynamics.Polygon.DyadicNodes

BarrowLib/Polygon/SampledValues.lean, 16 theorems, 236 lines

```
import BarrowLib.Polygon.ScaledTolerance
```

```

/-! Completion of uniformly coherent rational map samples on a named region.
A completed input has a regional Cauchy representative; comparisons use only
certified approximants. Representative independence is proved before choosing
a representative, without a total-map or completed-continuity premise. -/

```

```

namespace NewtonLimitDynamics.Polygon.SampledValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic CauchyValues
open HarmonicTimeRealization

```

```

structure Family where
  sample : Nat → (Point × Point) → (Point × Point)
  region : (Point × Point) → Prop
  coefficient : Fraction
  coefficient_nonnegative : 0 ≤ coefficient.num
  error : Nat → Fraction
  error_nonnegative : ∀ n, 0 ≤ (error n).num
  error_vanishes : ∀ eps : Fraction, 0 < eps.num →
    ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (error n) eps
  ordered_bound : ∀ i j, i ≤ j → ∀ s t, region s → region t →
    Fraction.le (distance (sample i s) (sample j t))
      (Fraction.add (Fraction.mul (distance s t) coefficient) (error i))

```

```

/-- Membership is witnessed by one Cauchy representative. Equivalent names
need not have every approximant in the region, especially at its boundary. -/
def Admissible (region : (Point × Point) → Prop) (x : Value) : Prop :=
  ∃ a : EndpointCauchyName, realize a = x ∧ ∀ n, region (a.approx n)

```

```

theorem admissible_realize (region : (Point × Point) → Prop)
  (a : EndpointCauchyName) (ha : ∀ n, region (a.approx n)) :

```

```

Admissible region (realize a) := {a,rfl,ha}

theorem admissible_embed (region : (Point × Point) → Prop) (s : Point × Point)
  (hs : region s) : Admissible region (embed s) := {constantName s,rfl,fun _ => hs}

theorem admissible_true (x : Value) : Admissible (fun _ => True) x := by
  induction x using Quotient.inductionOn with
  | _ a => exact {a,rfl,fun _ => True.intro}

noncomputable def admissibleName (region : (Point × Point) → Prop)
  (x : Value) (hx : Admissible region x) : EndpointCauchyName := hx.choose

theorem admissibleName_mem (region : (Point × Point) → Prop)
  (x : Value) (hx : Admissible region x) (n : Nat) :
  region ((admissibleName region x hx).approx n) := hx.choose_spec.2 n

theorem admissibleName_realize (region : (Point × Point) → Prop)
  (x : Value) (hx : Admissible region x) :
  realize (admissibleName region x hx) = x := hx.choose_spec.1

theorem scaled_add_small (C eps d e : Fraction) (hC : 0 ≤ C.num)
  (hd : 0 ≤ d.num)
  (hdelta : Fraction.lt d (factorDelta C eps.half hC))
  (he : Fraction.lt e eps.half) :
  Fraction.lt (Fraction.add (Fraction.mul d C) e) eps :=
  lt_equiv_right (Fraction.add_lt_add
    (factor_control C eps.half d hC hd hdelta) he) (Fraction.half_add_self eps)

def sampledName (f : Family) (a : EndpointCauchyName)
  (ha : ∀ n, f.region (a.approx n)) : EndpointCauchyName where
  approx := fun n => f.sample n (a.approx n)
  cauchy := by
    intro eps heps
    let delta := factorDelta f.coefficient eps.half f.coefficient_nonnegative
    obtain ⟨N,hN⟩ := a.cauchy delta (factorDelta_positive _ _ _ heps)
    obtain ⟨M,hM⟩ := f.error_vanishes eps.half heps
    refine ⟨max N M, fun i j hi hj => ?_⟩
    by_cases hij : i ≤ j
    · exact Fraction.magnitudes.lt_of_le_lt (f.ordered_bound i j hij _ _ (ha i) (ha j))
      (scaled_add_small _ eps _ f.coefficient_nonnegative (stateNorm_nonnegative _))
      (hN i j (by omega) (by omega)) (hM i (by omega)))
    · have hji : j ≤ i := by omega
      exact Fraction.magnitudes.lt_of_le_lt
        (Fraction.le_equiv_left (stateSub_norm_symm _ _ ) (f.ordered_bound j i hji _ _ (ha j) (ha i)))
        (scaled_add_small _ eps _ f.coefficient_nonnegative (stateNorm_nonnegative _))
        (hN j i (by omega) (by omega)) (hM j (by omega)))

theorem sampledName_equiv (f : Family) (a b : EndpointCauchyName)
  (ha : ∀ n, f.region (a.approx n)) (hb : ∀ n, f.region (b.approx n))
  (hab : NameEquiv a b) : NameEquiv (sampledName f a ha) (sampledName f b hb) := by
  intro eps heps
  let delta := factorDelta f.coefficient eps.half f.coefficient_nonnegative
  obtain ⟨N,hN⟩ := hab delta (factorDelta_positive _ _ _ heps)
  obtain ⟨M,hM⟩ := f.error_vanishes eps.half heps
  refine ⟨max N M, fun n hn => ?_⟩
  exact Fraction.magnitudes.lt_of_le_lt (f.ordered_bound n n (Nat.le_refl _ _ _ (ha n) (hb n))
    (scaled_add_small _ eps _ f.coefficient_nonnegative (stateNorm_nonnegative _))
    (hN n (by omega)) (hM n (by omega)))

/-- The existing completed operation is restricted to certified inputs. The
name-equivalence theorem proves that the chosen regional representative does
not change its output. No second completed domain or total extension is built. -/
noncomputable def sampledValue (f : Family) (x : Value)
  (hx : Admissible f.region x) : Value :=
  realize (sampledName f (admissibleName f.region x hx) (admissibleName_mem f.region x hx))

theorem sampledValue_realize (f : Family) (a : EndpointCauchyName)
  (ha : ∀ n, f.region (a.approx n)) (hx : Admissible f.region (realize a)) :
  sampledValue f (realize a) hx = realize (sampledName f a ha) := by
  apply Quotient.sound

```

```

exact sampledName_equiv f _ a (admissibleName_mem _ _ hx) ha
  (Quotient.exact (admissibleName_realize _ _ hx))

theorem sampledValue_congr (f : Family) (x y : Value)
  (hx : Admissible f.region x) (hy : Admissible f.region y) (hxy : x=y) :
  sampledValue f x hx = sampledValue f y hy := by
  cases hxy
  rfl

def offsetFamily (f : Family) (m : Nat) : Family where
  region := f.region
  sample := fun j => f.sample (m+j)
  coefficient := f.coefficient
  coefficient_nonnegative := f.coefficient_nonnegative
  error := fun j => f.error (m+j)
  error_nonnegative := fun j => f.error_nonnegative (m+j)
  error_vanishes := by
    intro eps heps
    obtain ⟨N,hN⟩ := f.error_vanishes eps heps
    exact ⟨N,fun j hj => hN (m+j) (by omega)⟩
  ordered_bound := fun i j hij => f.ordered_bound (m+i) (m+j) (by omega)

theorem sampledName_offset_equiv (f : Family) (a : EndpointCauchyName)
  (ha : ∀ n, f.region (a.approx n)) (m : Nat) :
  NameEquiv (sampledName f a ha) (sampledName (offsetFamily f m) a ha) := by
  intro eps heps
  obtain ⟨N,hN⟩ := f.error_vanishes eps heps
  refine ⟨N,fun j hj => ?_⟩
  have hc := f.ordered_bound j (m+j) (by omega) (a.approx j) (a.approx j) (ha j) (ha j)
  have hz : Fraction.equiv
    (Fraction.add (Fraction.mul (distance (a.approx j) (a.approx j)) f.coefficient) (f.error j))
    (f.error j) := by
    apply Fraction.equiv_trans (Fraction.add_equiv
      (Fraction.mul_equiv (HarmonicAccumulation.stateSub_self_norm_zero _) (Fraction.equiv_refl _))
      (Fraction.equiv_refl _))
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.zero_mul, Int.mul_zero, Int.zero_add, Int.mul_one, Int.one_mul]
    ac_nf
  exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_equiv_right hc hz) (hN j hj)

theorem sampledValue_offset (f : Family) (x : Value)
  (hx : Admissible f.region x) (m : Nat) :
  sampledValue (offsetFamily f m) x hx = sampledValue f x hx :=
  Quotient.sound (nameEquiv_symm (sampledName_offset_equiv f _
    (admissibleName_mem _ _ hx) m))

theorem nameBound_of_vanishing_error (a b : EndpointCauchyName) (R : Fraction)
  (e : Nat → Fraction)
  (he : ∀ eps : Fraction, 0 < eps.num → ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (e n) eps)
  (hlevel : ∀ n, Fraction.le (distance (a.approx n) (b.approx n)) (Fraction.add R (e n))) :
  NameBound a b R := by
  intro eps heps
  obtain ⟨N,hN⟩ := he eps heps
  exact ⟨N,fun n hn => Fraction.magnitudes.lt_of_le_lt (hlevel n)
    (CauchyValues.add_lt_add_left (hN n hn) R)⟩

theorem nameBound_scale_error (a b ta tb : EndpointCauchyName)
  (C R : Fraction) (hC : 0 ≤ C.num) (e : Nat → Fraction)
  (he : ∀ eps : Fraction, 0 < eps.num →
    ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (e n) eps)
  (hlevel : ∀ n, Fraction.le (distance (a.approx n) (b.approx n))
    (Fraction.add (Fraction.mul (distance (ta.approx n) (tb.approx n)) C) (e n)))
  (hnear : NameBound ta tb R) : NameBound a b (Fraction.mul R C) := by
  intro eps heps
  let q := eps.half.half
  let delta := factorDelta C q hC
  obtain ⟨N,hN⟩ := hnear delta (factorDelta_positive _ _ _ heps)
  obtain ⟨M,hM⟩ := he q heps
  refine ⟨max N M,fun n hn => ?_⟩
  have hm := Fraction.mul_le_mul_nonnegative

```

```

    (Fraction.magnitudes.lt_implies_le (hN n (by omega))) C hC
  have hs := Fraction.le_equiv_right hm (Fraction.add_mul R delta C)
  have hd := factor_delta_weak C q hC (Int.le_of_lt heps)
  have hb := Fraction.magnitudes.le_trans (hlevel n)
    (Fraction.add_le_add (Fraction.magnitudes.le_trans hs
      (Fraction.add_le_add_left hd (Fraction.mul R C)))
      (Fraction.magnitudes.lt_implies_le (hM n (by omega))))
  have hc : Fraction.equiv
    (Fraction.add (Fraction.add (Fraction.mul R C) q) q)
    (Fraction.add (Fraction.mul R C) eps.half) :=
    Fraction.equiv_trans (Fraction.add_assoc _ _ _)
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.half_add_self eps.half))
  exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_equiv_right hb hc)
    (CauchyValues.add_lt_add_left (Fraction.half_lt eps heps) _)

theorem sampledValue_within (f : Family) (x y : Value)
  (hx : Admissible f.region x) (hy : Admissible f.region y)
  (R : Fraction) (hxy : Within x y R) :
  Within (sampledValue f x hx) (sampledValue f y hy) (Fraction.mul R f.coefficient) := by
  let a := admissibleName f.region x hx
  let b := admissibleName f.region y hy
  have hab : NameBound a b R := by
    change Within (realize a) (realize b) R
    simpa only [a,b,admissibleName_realize] using hxy
  exact nameBound_scale_error _ a b _ R f.coefficient_nonnegative f.error
    f.error_vanishes (fun n => f.ordered_bound n n (Nat.le_refl _)) _ _
    (admissibleName_mem _ _ hx n) (admissibleName_mem _ _ hy n) hab

theorem nameBound_affine (a b ta tb : EndpointCauchyName)
  (C R E : Fraction) (hC : 0 ≤ C.num) (M : Nat)
  (hlevel : ∀ n, M ≤ n → Fraction.le (distance (a.approx n) (b.approx n))
    (Fraction.add (Fraction.mul (distance (ta.approx n) (tb.approx n)) C) E))
  (hnear : NameBound ta tb R) :
  NameBound a b (Fraction.add (Fraction.mul R C) E) := by
  intro eps heps
  let delta := factorDelta C eps.half hC
  obtain (N,hN) := hnear delta (factorDelta_positive _ _ _ heps)
  refine (max N M, fun n hn => ?_)
  have hm := Fraction.mul_le_mul_nonnegative
    (Fraction.magnitudes.lt_implies_le (hN n (by omega))) C hC
  have hs := Fraction.le_equiv_right hm (Fraction.add_mul R delta C)
  have hd := factor_delta_weak C eps.half hC (Int.le_of_lt heps)
  have hb := Fraction.magnitudes.le_trans (hlevel n (by omega))
    (Fraction.add_le_add_right (Fraction.magnitudes.le_trans hs
      (Fraction.add_le_add_left hd (Fraction.mul R C))) E)
  have hc : Fraction.equiv
    (Fraction.add (Fraction.add (Fraction.mul R C) eps.half) E)
    (Fraction.add (Fraction.add (Fraction.mul R C) E) eps.half) := by
    simp only [Fraction.equiv, Fraction.add, Int.add_mul, Int.mul_add]
  ac_nf
  exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_equiv_right hb hc)
    (CauchyValues.add_lt_add_left (Fraction.half_lt eps heps) _)

theorem sampled_approximant_bound (f : Family) (a : EndpointCauchyName)
  (ha : ∀ n, f.region (a.approx n)) (hx : Admissible f.region (realize a))
  (m : Nat) (R : Fraction) (hnear : Within (embed (a.approx m)) (realize a) R) :
  Within (embed (f.sample m (a.approx m))) (sampledValue f (realize a) hx)
    (Fraction.add (Fraction.mul R f.coefficient) (f.error m)) := by
  rw [sampledValue_realize f a ha hx]
  exact nameBound_affine _ _ (constantName (a.approx m)) a _ R _
    f.coefficient_nonnegative m
    (fun n hn => f.ordered_bound m n hn _ _ (ha m) (ha n)) hnear
end NewtonLimitDynamics.Polygon.SampledValues

```

BarrowLib/Polygon/AccelerationEstimates.lean, 13 theorems, 194 lines

```
import BarrowLib.Polygon.KinematicEstimates
```

```

/-! Actual finite velocity remainders relative to the force at a cell's
starting point. Position displacement and force comparison derive the error;

```



```

no derivative, completed motion or force equation is supplied. -/

namespace NewtonLimitDynamics.Polygon.AccelerationEstimates
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates BoundedIteration KinematicEstimates

theorem position_increment (a : Point → Point) (h : Fraction) (s : Point × Point) :
  Fraction.equiv (pointDistance (cell a h s).1 s.1)
    (Fraction.mul h.abs (pointNorm s.2)) :=
  Fraction.equiv_trans (pointNorm_equiv (ConvexCover.drift_offset h s.1 s.2))
    (pointNorm_scale h s.2)

theorem position_displacement (a : Point → Point) (h : Fraction) (s : Point × Point)
  (V : Fraction) (hh : 0 ≤ h.num) (N n : Nat) (hn : n ≤ N)
  (hv : ∀ i, i < N → Fraction.le (pointNorm (run a h s i).2) V) :
  Fraction.le (pointDistance (run a h s n).1 s.1) (Fraction.mul (time h n) V) := by
  have hs : ∀ i, i < N → Fraction.le
    (pointDistance (run a h s (i+1)).1 (run a h s i).1) (Fraction.mul h V) := by
    intro i hi
    exact Fraction.le_equiv_left (position_increment a h (run a h s i))
      (Fraction.le_equiv_right
        (Fraction.mul_le_mul_nonnegative_left (hv i hi) h.abs (Fraction.abs_num_nonnegative h))
        (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl V)))
  have hg := FiniteSequenceGap.finite_gap (fun x y => pointDistance x.1 y.1)
    (fun x => pointDistance_self_zero x.1) (fun x y z => pointDistance_triangle x.1 y.1 z.1)
    (run a h s) N (Fraction.mul h V) hs 0 n (by omega)
  simpa only [Nat.zero_add, run] using
    Fraction.le_equiv_right hg (Fraction.equiv_symm (Fraction.mul_assoc _ _ _))

def predictedVelocity (a : Point → Point) (h : Fraction) (s : Point × Point) (n : Nat) : Point :=
  inertialPosition h (s.2, a s.1) n

theorem velocity_remainder_from_samples (a : Point → Point) (h : Fraction)
  (s : Point × Point) (C : Fraction) (hh : 0 ≤ h.num) (N : Nat)
  (hs : ∀ i, i < N → Fraction.le (pointDistance (a (run a h s (i+1))).1) (a s.1)) C :
  ∀ n, n ≤ N → Fraction.le (pointDistance (run a h s n).2 (predictedVelocity a h s n))
    (Fraction.mul (time h n) C) := by
  intro n hn
  induction n with
  | zero =>
    have he := inertial_zero h (s.2, a s.1)
    apply Fraction.le_of_equiv
    apply Fraction.equiv_trans (pointDistance_equiv
      (Fraction.equiv_refl _, Fraction.equiv_refl _) he)
    apply Fraction.equiv_trans (pointDistance_self_zero s.2)
    simp [time, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  | succ n ih =>
    have hf := Fraction.le_equiv_left (difference_scale h _)
      (Fraction.le_equiv_right
        (Fraction.mul_le_mul_nonnegative_left (hs n (by omega)) h.abs (Fraction.abs_num_nonnegative h))
        (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl C)))
    have ha := difference_add_bound (run a h s n).2 (predictedVelocity a h s n)
      (pointScale h (a (run a h s (n+1))).1) (pointScale h (a s.1))
    have hc := Fraction.magnitudes.le_trans ha (Fraction.add_le_add (ih (by omega)) hf)
    have hd := Fraction.le_equiv_left
      (pointDistance_equiv (p := (run a h s (n+1))).2)
      (Fraction.equiv_refl _, Fraction.equiv_refl _) (inertial_step h (s.2, a s.1) n) hc
    apply Fraction.le_equiv_right hd
    simp only [time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
    ac_nf

def source (L E t V : Fraction) : Fraction :=
  Fraction.add (Fraction.mul L (Fraction.mul t V)) E

theorem force_variation_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (N i : Nat) (hi : i < N)
  (hc : Fraction.le (pointDistance (a (run a h s (i+1))).1) (a s.1))
  (Fraction.add (Fraction.mul L (pointDistance (run a h s (i+1)).1 s.1)) E))

```

```

    (hv : ∀ k, k < N → Fraction.le (pointNorm (run a h s k).2) V) :
    Fraction.le (pointDistance (a (run a h s (i+1)).1) (a s.1))
      (source L E (time h N) V) := by
  have hp := Fraction.magnitudes.le_trans
    (position_displacement a h s V hh N (i+1) (by omega) hv)
    (Fraction.mul_le_mul_nonnegative (time_monotone h hh (i+1) N (by omega)) V hv)
  exact Fraction.magnitudes.le_trans hc
    (Fraction.add_le_add_right (Fraction.mul_le_mul_nonnegative_left hp L hL) E)

theorem force_variation (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (N i : Nat) (hi : i < N)
  (hv : ∀ k, k < N → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (a (run a h s (i+1)).1) (a s.1))
    (source L E (time h N) V) :=
  force_variation_at a h s L E V hh hL hV N i hi (hc _ _) hv

theorem velocity_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (n : Nat)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1)).1) (a s.1))
    (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (run a h s n).2 (predictedVelocity a h s n))
    (Fraction.mul (time h n) (source L E (time h n) V)) :=
  velocity_remainder_from_samples a h s _ hh n
  (fun i hi => force_variation_at a h s L E V hh hL hV n i hi (hc i hi) hv) n (Nat.le_refl _)

theorem velocity_remainder (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat)
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (run a h s n).2 (predictedVelocity a h s n))
    (Fraction.mul (time h n) (source L E (time h n) V)) :=
  velocity_remainder_at a h s L E V hh hL hV n (fun _ => hc _ _) hv

theorem acceleration_secant_bound_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (n : Nat) (ht : 0 < (time h n).num)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1)).1) (a s.1))
    (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance
    (pointScale (TimeCalibration.inverse (time h n) ht) (pointSub (run a h s n).2 s.2))
    (a s.1)) (source L E (time h n) V) := by
  have he := Fraction.equiv_trans (pointNorm_equiv (secant_identity (time h n) ht)
    (run a h s n).2 s.2 (a s.1)))
    (pointNorm_scale (TimeCalibration.inverse (time h n) ht)
    (pointSub (run a h s n).2 (predictedVelocity a h s n)))
  have hm := Fraction.mul_le_mul_nonnegative_left
    (velocity_remainder_at a h s L E V hh hL hV n hc hv)
    (TimeCalibration.inverse (time h n) ht).abs (Fraction.abs_num_nonnegative _)
  apply Fraction.le_equiv_right (Fraction.le_equiv_left he hm)
  rw [Fraction.abs_eq_of_nonnegative (TimeCalibration.inverse (time h n) ht)
    (Int.le_of_lt (time h n).den_pos)]
  simp only [TimeCalibration.inverse, Fraction.equiv, Fraction.mul]
  ac_nf

theorem acceleration_secant_bound (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat) (ht : 0 < (time h n).num)
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance
    (pointScale (TimeCalibration.inverse (time h n) ht) (pointSub (run a h s n).2 s.2))
    (a s.1)) (source L E (time h n) V) :=

```

```

acceleration_secant_bound_at a h s L E V hh hL hV n ht (fun _ _ => hc _ _) hv

theorem acceleration_secant_equivalent_time_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (n : Nat) (ht : 0 < (time h n).num)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1)).1) (a s.1))
    (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E))
  (hu : 0 < u.num) (he : Fraction.equiv u (time h n))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance
    (pointScale (TimeCalibration.inverse u hu) (pointSub (run a h s n).2 s.2))
    (a s.1)) (source L E u V) :=
  Fraction.le_equiv_right
    (Fraction.le_equiv_left (pointDistance_equiv
      (pointScale_ratio_congr (TimeCalibration.inverse_congr hu ht he)
        (Fraction.equiv_refl _, Fraction.equiv_refl _))
      (Fraction.equiv_refl _, Fraction.equiv_refl _))
      (acceleration_secant_bound_at a h s L E V hh hL hV n ht hc hv))
    (Fraction.add_equiv (Fraction.mul_equiv (Fraction.equiv_refl L)
      (Fraction.mul_equiv (Fraction.equiv_symm he) (Fraction.equiv_refl V)))) (Fraction.equiv_refl E))

theorem acceleration_secant_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat) (ht : 0 < (time h n).num)
  (hu : 0 < u.num) (he : Fraction.equiv u (time h n))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance
    (pointScale (TimeCalibration.inverse u hu) (pointSub (run a h s n).2 s.2))
    (a s.1)) (source L E u V) :=
  acceleration_secant_equivalent_time_at a h s L E V u hh hL hV n ht (fun _ _ => hc _ _) hu he hv

private def controlHalf : Fraction := (1,2,by decide)
private def controlState : Point × Point :=
  ((Fraction.ofInt 1, Fraction.ofInt 0), (Fraction.ofInt 0, Fraction.ofInt 0))

theorem two_cell_acceleration_control :
  Fraction.equiv
    (pointDistance (pointScale (TimeCalibration.inverse (time controlHalf 2) (by decide))
      (pointSub (run pointNeg controlHalf controlState 2).2 controlState.2))
      (pointNeg controlState.1)) (1,8,by decide) := by decide

theorem zero_bound_rejects_acceleration_control :
  ¬ Fraction.le
    (pointDistance (pointScale (TimeCalibration.inverse (time controlHalf 2) (by decide))
      (pointSub (run pointNeg controlHalf controlState 2).2 controlState.2))
      (pointNeg controlState.1)) (Fraction.ofInt 0) := by unfold Fraction.le; decide

end NewtonLimitDynamics.Polygon.AccelerationEstimates

```

BarrowLib/Polygon/PolygonValues.lean, 7 theorems, 122 lines

```

import BarrowLib.Polygon.AffineBoundary
import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.BinaryEndpoints

/-! A polygon on the constructed binary-time domain from actual finite
vertices with proved position joining. Alias invariance, zero-time values and
affine vertex bounds are shared independently of the map producing vertices.
There is no supplied curve, motion equation or convergence field. -/

namespace NewtonLimitDynamics.Polygon.PolygonValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic HarmonicBinaryPrefix
open HarmonicAccumulation HarmonicTimeComparison HarmonicTimeRealization BinaryTime
open CauchyValues PositionValues

structure VertexChain (T : Fraction) (m : Nat) where
  state : Nat → Point × Point
  join : ∀ k, pointEquiv (state (k+1)).1

```

```

    (pointAdd (state k).1 (pointScale (duration T m) (state k).2))

def polygonName (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat)
  (v : VertexChain T m) : EndpointCauchyName :=
  AffineValues.edgeName b T hT (v.state (ticks b m)).1 (v.state (ticks b m)).2 m

def polygonPosition (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num) (m : Nat)
  (v : VertexChain T m) : PositionValue := asPosition (realize (polygonName b T hT m v))

theorem polygon_same_cell (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) (v : VertexChain T m) (hcell : ticks b m=ticks c m)
  (hTm : AddressEquiv T hT b c) :
  polygonPosition b T hT m v = polygonPosition c T hT m v := by
  have hb : timeApprox b T m=timeApprox c T m := by simp only [timeApprox,hcell]
  unfold polygonPosition polygonName
  rw [hcell]
  exact congrArg asPosition (Quotient.sound
    (AffineValues.edgeName_same_start b c T hT _ _ m hb hTm))

theorem polygon_adjacent_cells (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) (v : VertexChain T m) (hcell : ticks b m+1=ticks c m)
  (hTm : AddressEquiv T hT b c) :
  polygonPosition b T hT m v = polygonPosition c T hT m v := by
  have hb : Fraction.equiv (timeApprox c T m)
    (Fraction.add (timeApprox b T m) (duration T m)) := by
    have hk := congrArg (fun n : Nat => (n:Int)) hcell
    simp only [Int.natCast_add,Int.natCast_one] at hk
    simp only [hk,timeApprox] using Fraction.equiv_symm (coarse_upper b T m)
  have hx : pointEquiv (v.state (ticks c m)).1
    (pointAdd (v.state (ticks b m)).1 (pointScale (duration T m) (v.state (ticks b m)).2)) := by
    rw [← hcell]
    exact v.join (ticks b m)
  exact congrArg asPosition (Quotient.sound (AffineValues.edge_boundary_names
    b c T hT _ _ _ m hb hx hTm))

theorem zero_window_vertices (T : Fraction) (m : Nat) (v : VertexChain T m)
  (hz : T.num=0) : ∀ k, pointEquiv (v.state k).1 (v.state 0).1 := by
  intro k
  induction k with
  | zero => exact (Fraction.equiv_refl _,Fraction.equiv_refl _)
  | succ k ih =>
    have hj := v.join k
    have hp := AffineValues.affine_zero_phase (v.state k).1 (v.state k).2
      (duration T m) hz
    exact pointEquiv_trans (pointEquiv_trans hj hp.1) ih

theorem polygon_zero_window (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) (v : VertexChain T m) (hz : T.num=0) :
  polygonPosition b T hT m v = asPosition (embed ((v.state 0).1,AffineValues.zeroPoint)) := by
  apply congrArg asPosition
  apply Quotient.sound
  apply nameEquiv_of_levelwise_stateEquiv
  intro j
  have ht : (durationDifference (timeApprox b T m) (timeApprox b T (m+j))).num=0 := by
    simp only [durationDifference,HarmonicStability.negF,timeApprox,duration,
      Fraction.add,Fraction.mul,Fraction.ofInt,hz,Int.mul_zero,Int.zero_mul,Int.neg_zero,Int.zero_add]
  have hp := AffineValues.affine_zero_phase (v.state (ticks b m)).1 (v.state (ticks b m)).2 _ ht
  exact (pointEquiv_trans hp.1 (zero_window_vertices T m v hz _),hp.2)

theorem polygon_address_independent (b c : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) (v : VertexChain T m) (hTm : AddressEquiv T hT b c) :
  polygonPosition b T hT m v = polygonPosition c T hT m v := by
  by_cases hz : T.num=0
  · exact (polygon_zero_window b T hT m v hz).trans (polygon_zero_window c T hT m v hz).symm
  · have hpos : 0<T.num := by omega
    obtain hc | hc | hc := address_equiv_cell_cases b c T hT hpos m hTm
    · exact polygon_same_cell b c T hT m v hc hTm
    · exact polygon_adjacent_cells b c T hT m v hc hTm
    · exact (polygon_adjacent_cells c b T hT m v hc (addressEquiv_symm T hT hTm)).symm

```

```

def polygonMap (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T m) :
  BinaryTime T hT → PositionValue :=
  Quotient.lift (fun b => polygonPosition b T hT m v)
    (fun b c h => polygon_address_independent b c T hT m v h)

theorem polygon_vertex_bound (b : Nat → Bool) (T : Fraction) (hT : 0 ≤ T.num)
  (m : Nat) (v : VertexChain T m) (V : Fraction)
  (hv : Fraction.le (pointNorm (v.state (ticks b m)).2) V) :
  Within (polygonPosition b T hT m v).val
    (positionValue (embed (v.state (ticks b m)))) (Fraction.mul (duration T m) V) := by
  have hr := Fraction.mul_le_mul_nonnegative_left hv (duration T m) hT
  apply within_mono _ _ _ hr
  exact positionValue_within _ _ _ (AffineValues.edge_vertex_bound b T hT _ _ m)

theorem polygonMap_left (T : Fraction) (hT : 0 ≤ T.num) (m : Nat) (v : VertexChain T m) :
  polygonMap T hT m v (leftTime T hT) = embedPosition (v.state 0).1 := by
  apply Subtype.ext
  change positionValue (realize (polygonName leftAddress T hT m v)) =
    positionValue (embed ((v.state 0).1, zeroPoint))
  apply congrArg positionValue
  apply Quotient.sound
  apply nameEquiv_of_levelwise_stateEquiv
  intro j
  have h0 : ticks leftAddress m=0 := all_zero_ticks m
  have h1 : ticks leftAddress (m+j)=0 := all_zero_ticks (m+j)
  have hz : (durationDifference (timeApprox leftAddress T m)
    (timeApprox leftAddress T (m+j))).num=0 := by
    simp [timeApprox, h0, h1, durationDifference, HarmonicStability.negF, Fraction.add, Fraction.mul, Fraction]
    <= .ofInt]
  change stateEquiv (AffineValues.affineState (v.state (ticks leftAddress m)).1
    (v.state (ticks leftAddress m)).2 _)) ((v.state 0).1, zeroPoint)
  rw [h0]
  exact AffineValues.affine_zero_phase _ _ _ hz

```

end NewtonLimitDynamics.Polygon.PolygonValues

BarrowLib/Polygon/MatchedRegion.lean, 7 theorems, 103 lines

```

import BarrowLib.Polygon.SquareOuterContent
import BarrowLib.Polygon.BinaryEndpoints

/-! Unsigned matched regions for two maps on the constructed binary-time
domain. Each dyadic cell closes all simultaneous rational connectors; their
finite union counts overlaps once. Closed square enclosures and geometric
cover budgets are independent of the construction of either map. -/

namespace NewtonLimitDynamics.Polygon.MatchedRegion
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic HarmonicBinaryPrefix BinaryTime CauchyValues PositionValues
open ConvexCover ConvexValues CompletionGeometry SquareOuterContent

def cellPatch (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m k : Nat) (x : PositionValue) : Prop :=
  ∃ b : Nat → Bool, ticks b m=k ∧ ∃ a : Fraction, ∃ ha : UnitInterval a,
    x=convexPosition a ha (p (Quotient.mk _ b)) (g (Quotient.mk _ b))

def Region (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat) (x : PositionValue) : Prop :=
  ∃ k, k<blocks m ∧ Closure (cellPatch T hT p g m k) x

theorem cellPatch_square (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat)
  (centres : Nat → Point) (R : NonnegativeRadius)
  (h : ∀ b : Nat → Bool,
    CoordinateSquare (centres (ticks b m)) R (p (Quotient.mk _ b)) ∧
    CoordinateSquare (centres (ticks b m)) R (g (Quotient.mk _ b)))
  (k : Nat) (x : PositionValue) (hx : cellPatch T hT p g m k x) :
  CoordinateSquare (centres k) R x := by
  obtain (b, hb, a, ha, hx) := hx
  subst x

```

```

obtain {hp,hg} := h b
rw [hb] at hp hg
exact convexPosition_square a ha _ _ _ hp hg

theorem closed_cell_square (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat)
  (centres : Nat → Point) (R : NonnegativeRadius)
  (h : ∀ b : Nat → Bool,
    CoordinateSquare (centres (ticks b m)) R (p (Quotient.mk _ b)) ∧
    CoordinateSquare (centres (ticks b m)) R (g (Quotient.mk _ b)))
  (k : Nat) (x : PositionValue) (hx : Closure (cellPatch T hT p g m k) x) :
  CoordinateSquare (centres k) R x :=
closure_square _ x hx _ _ (fun y hy => cellPatch_square T hT p g m centres R h k y hy)

def actualCover (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat)
  (centres : Nat → Point) (R : NonnegativeRadius)
  (h : ∀ b : Nat → Bool,
    CoordinateSquare (centres (ticks b m)) R (p (Quotient.mk _ b)) ∧
    CoordinateSquare (centres (ticks b m)) R (g (Quotient.mk _ b))) :
  Cover (Region T hT p g m) where
count := blocks m
squares := fun k => (centres k,R)
covers := by
  intro x hx
  obtain {k,hk,hx} := hx
  exact {k,hk,closed_cell_square T hT p g m centres R h k x hx}

theorem connector_in_region (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat)
  (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) :
  Region T hT p g m (convexPosition a ha (p t) (g t)) := by
  induction t using Quotient.inductionOn with
  | _ b =>
    exact {ticks b m,ticks_lt_blocks b m,closure_contains _ _ (b,rfl,a,ha,rfl)}

theorem reversed_connector_in_region (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat)
  (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) :
  Region T hT p g m (convexPosition a ha (g t) (p t)) := by
  rw [convexPosition_swap]
  exact connector_in_region T hT p g m t _ (complement_interval a ha)

theorem polygon_in_region (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat) (t : BinaryTime T hT) :
  Region T hT p g m (p t) := by
  have ha : UnitInterval (Fraction.ofInt 0) := by constructor <=> decide
  have h := connector_in_region T hT p g m t (Fraction.ofInt 0) ha
  rwa [convexPosition_zero] at h

theorem curve_in_region (T : Fraction) (hT : 0 ≤ T.num)
  (p g : BinaryTime T hT → PositionValue) (m : Nat) (t : BinaryTime T hT) :
  Region T hT p g m (g t) := by
  have ha : UnitInterval (Fraction.ofInt 1) := by constructor <=> decide
  have h := connector_in_region T hT p g m t (Fraction.ofInt 1) ha
  rwa [convexPosition_one] at h

/-- With one square of radius C/2^m per cell, the total budget is 4*C^2/2^m.
This is cover arithmetic; membership and the actual enclosure are proved above. -/
theorem uniform_budget_geometric (centres : Nat → Point) (R : NonnegativeRadius)
  (m : Nat) (C : Fraction) (hr : Fraction.equiv R.val (duration C m)) :
  Fraction.equiv (sumBudget (fun k => (centres k,R)) (blocks m))
  (duration (squareArea C) m) := by
  have hc := Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (blocks m : Int)))
  (Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt 4)) (Fraction.mul_equiv hr hr))
  apply Fraction.equiv_trans (uniform_budget centres R (blocks m))
  apply Fraction.equiv_trans hc
  simp only [squareArea,duration,blocks,Fraction.equiv,Fraction.mul,Fraction.ofInt,Int.natCast_pow]
  ac_nf

```

```
end NewtonLimitDynamics.Polygon.MatchedRegion
```

BarrowLib/Polygon/QuadraticEstimates.lean, 13 theorems, 173 lines

```
import BarrowLib.Polygon.AccelerationEstimates
import BarrowLib.Polygon.RationalIntervals

/--! Exact finite quadratic comparison for drift-then-kick iteration. The
constant-map position coefficient is  $t \cdot (t-h)/2$ , rather than  $t^2/2$ . Variable
maps have a derived cubic/error remainder; the remaining half-mesh bias is
explicit. No curve, derivative or integral is a premise. -/

namespace NewtonLimitDynamics.Polygon.QuadraticEstimates
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates BoundedIteration KinematicEstimates
  ↳ AccelerationEstimates
open HarmonicStability

def quadraticPosition (t : Fraction) (s : Point × Point) (a : Point) : Point :=
  pointAdd (pointAdd s.1 (pointScale t s.2)) (pointScale (Fraction.mul t t).half a)

theorem quadratic_time_congr (t u : Fraction) (s : Point × Point) (a : Point)
  (he : Fraction.equiv t u) : pointEquiv (quadraticPosition t s a) (quadraticPosition u s a) :=
  pointAdd_congr (pointAdd_congr (Fraction.equiv_refl _, Fraction.equiv_refl _)
    (pointScale_ratio_congr he (Fraction.equiv_refl _, Fraction.equiv_refl _)))
    (pointScale_ratio_congr (RationalIntervals.half_equiv (Fraction.mul_equiv he he))
      (Fraction.equiv_refl _, Fraction.equiv_refl _))

def discreteWeight (h : Fraction) (n : Nat) : Fraction :=
  (Fraction.mul (time h n) (Fraction.add (time h n) (negF h))).half

def discretePosition (h : Fraction) (s : Point × Point) (a : Point) (n : Nat) : Point :=
  pointAdd (inertialPosition h s n) (pointScale (discreteWeight h n) a)

theorem discrete_zero (h : Fraction) (s : Point × Point) (a : Point) :
  pointEquiv (discretePosition h s a 0) s.1 := by
  constructor <=>
  simp only [discretePosition, discreteWeight, inertialPosition, time, pointEquiv,
    pointAdd, pointScale, negF, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.half,
    Fraction.ofInt, Int.natCast_zero, Int.zero_mul, Int.mul_zero, Int.add_zero] <=> ac_nf

theorem discrete_step (h : Fraction) (s : Point × Point) (a : Point) (n : Nat) :
  pointEquiv (discretePosition h s a (n+1))
    (pointAdd (discretePosition h s a n) (pointScale h (inertialPosition h (s.2, a) n))) := by
  constructor <=>
  simp only [discretePosition, discreteWeight, inertialPosition, time, pointEquiv,
    pointAdd, pointScale, negF, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.half,
    Fraction.ofInt, Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add,
    Int.neg_mul, Int.mul_neg] <=>
  simp only [show (2 : Int)=1+1 by rfl, Int.add_mul, Int.mul_add] <=> ac_nf <=> omega

/-- The actual constant-map run, at every finite count. -/
theorem constant_run_formula (a : Point) (h : Fraction) (s : Point × Point) (n : Nat) :
  stateEquiv (run (fun _ => a) h s n)
    (discretePosition h s a n, inertialPosition h (s.2, a) n) := by
  induction n with
  | zero => exact (pointEquiv_symm (discrete_zero h s a), pointEquiv_symm (inertial_zero h (s.2, a)))
  | succ n ih =>
    constructor
    · exact pointEquiv_trans (pointAdd_congr ih.1 (pointScale_congr h ih.2))
      (pointEquiv_symm (discrete_step h s a n))
    · exact pointEquiv_trans (pointAdd_congr ih.2
      (pointScale_congr h (Fraction.equiv_refl _, Fraction.equiv_refl _)))
      (pointEquiv_symm (inertial_step h (s.2, a) n))

/-- Sum the actual velocity remainders against the exact discrete quadratic
position, using the shared finite quadratic budget step. -/
theorem position_remainder_from_samples (a : Point → Point) (h : Fraction)
  (s : Point × Point) (C : Fraction) (hh : 0 ≤ h.num) (hC : 0 ≤ C.num) (N : Nat)
  (hs : ∀ i, i ≤ N → Fraction.le (pointDistance (a (run a h s (i+1))).1) (a s.1)) C :
  ∀ n, n ≤ N → Fraction.le (pointDistance (run a h s n).1 (discretePosition h s (a s.1) n))
```

```

    (Fraction.mul (Fraction.mul (time h n) (time h n)) C) := by
intro n hn
induction n with
| zero =>
  apply Fraction.le_of_equiv
  apply Fraction.equiv_trans (pointDistance_equiv
    (Fraction.equiv_refl _, Fraction.equiv_refl _) (discrete_zero h s (a s.1)))
  apply Fraction.equiv_trans (pointDistance_self_zero s.1)
  simp [time, Fraction.equiv, Fraction.mul, Fraction.ofInt]
| succ n ih =>
  have hv := velocity_remainder_from_samples a h s C hh N hs n (by omega)
  have hs' := Fraction.le_equiv_left (difference_scale h _)
    (Fraction.le_equiv_right
      (Fraction.mul_le_mul_nonnegative_left hv h.abs (Fraction.abs_num_nonnegative h))
      (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl _)))
  have ha := difference_add_bound (run a h s n).1 (discretePosition h s (a s.1) n)
    (pointScale h (run a h s n).2) (pointScale h (predictedVelocity a h s n))
  have hb := Fraction.magnitudes.le_trans ha (Fraction.add_le_add (ih (by omega)) hs')
  have hc := Fraction.le_equiv_left
    (pointDistance_equiv (p := (run a h s (n+1)).1)
      (Fraction.equiv_refl _, Fraction.equiv_refl _) (discrete_step h s (a s.1) n)) hb
  exact Fraction.magnitudes.le_trans hc (quadratic_step h C hh hC n)

theorem position_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (n : Nat)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1)).1) (a s.1))
    (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1) E))
    (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (run a h s n).1 (discretePosition h s (a s.1) n))
    (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V)) :=
position_remainder_from_samples a h s _ hh
  (Fraction.nonnegative_add _
    (Fraction.nonnegative_mul _ hL (Fraction.nonnegative_mul _ (time_nonnegative h hh n) hV)) hE)
  n (fun i hi => force_variation_at a h s L E V hh hL hV n i hi (hc i hi) hv) n (Nat.le_refl _)

theorem position_remainder (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat)
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (run a h s n).1 (discretePosition h s (a s.1) n))
    (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V)) :=
position_remainder_at a h s L E V hh hL hE hV n (fun _ => hc _ _) hv

theorem discrete_quadratic_offset (h : Fraction) (s : Point × Point) (a : Point) (n : Nat) :
  pointEquiv (pointSub (quadraticPosition (time h n) s a) (discretePosition h s a n))
    (pointScale (Fraction.mul (time h n) h).half a) := by
constructor <=>
  simp only [quadraticPosition, discretePosition, discreteWeight, inertialPosition,
    pointEquiv, pointSub, pointNeg, pointAdd, pointScale, negF, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.half, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

theorem half_mesh_bias (h : Fraction) (s : Point × Point) (a : Point)
  (hh : 0 ≤ h.num) (n : Nat) :
  Fraction.equiv (pointDistance (discretePosition h s a n) (quadraticPosition (time h n) s a))
    (Fraction.mul (Fraction.mul (time h n) h).half (pointNorm a)) := by
have hr : 0 ≤ (Fraction.mul (time h n) h).half.num :=
  Fraction.nonnegative_mul _ (time_nonnegative h hh n) hh
exact Fraction.equiv_trans (pointDistance_symm _)
  (Fraction.equiv_trans (pointNorm_equiv (discrete_quadratic_offset h s a n))
    (Fraction.equiv_trans (pointNorm_scale _)
      (Fraction.mul_equiv (Fraction.abs_of_nonnegative _ hr) (Fraction.equiv_refl _))))

/-- Cubic force-variation remainder plus the explicit half-mesh bias. -/
theorem position_quadratic_remainder_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (n : Nat)

```



```

(hc : ∀ k, k < n → Fraction.le
  (pointDistance (a (run a h s (k+1))).1) (a s.1))
  (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1) E))
(hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
Fraction.le (pointDistance (run a h s n).1 (quadraticPosition (time h n) s (a s.1)))
  (Fraction.add
    (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V))
    (Fraction.mul (Fraction.mul (time h n) h).half (pointNorm (a s.1)))) :=
Fraction.magnitudes.le_trans (pointDistance_triangle _ (discretePosition h s (a s.1) n) _)
  (Fraction.add_le_add (position_remainder_at a h s L E V hh hL hE hV n hc hv)
    (Fraction.le_of_equiv (half_mesh_bias h s (a s.1) hh n)))

theorem position_quadratic_remainder (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat)
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
Fraction.le (pointDistance (run a h s n).1 (quadraticPosition (time h n) s (a s.1)))
  (Fraction.add
    (Fraction.mul (Fraction.mul (time h n) (time h n)) (source L E (time h n) V))
    (Fraction.mul (Fraction.mul (time h n) h).half (pointNorm (a s.1)))) :=
position_quadratic_remainder_at a h s L E V hh hL hE hV n (fun _ _ => hc _ _) hv

private def controlHalf : Fraction := {1,2,by decide}
private def controlState : Point × Point :=
  ((Fraction.ofInt 0, Fraction.ofInt 0), (Fraction.ofInt 1, Fraction.ofInt 0))
private def controlForce : Point := (Fraction.ofInt 0, Fraction.ofInt (-1))

theorem two_cell_half_mesh_control :
  Fraction.equiv (pointDistance (run (fun _ => controlForce) controlHalf controlState 2).1
    (quadraticPosition (time controlHalf 2) controlState controlForce)) {1,4,by decide} := by decide

theorem exact_quadratic_rejects_finite_control :
  ~ pointEquiv (run (fun _ => controlForce) controlHalf controlState 2).1
    (quadraticPosition (time controlHalf 2) controlState controlForce) := by decide

end NewtonLimitDynamics.Polygon.QuadraticEstimates

```

BarrowLib/Polygon/QuadraticSecants.lean, 6 theorems, 112 lines

```

import BarrowLib.Polygon.QuadraticEstimates
import BarrowLib.Polygon.SecantValues

/-! Normalized second-order position departure, composed from the existing
proved secant operators. Actual finite quadratic remainders yield force error
2*(L*t*V+E)+(h/t)*|a(x0)|. The half-mesh bias remains explicit. -/

namespace NewtonLimitDynamics.Polygon.QuadraticSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates BoundedIteration KinematicEstimates
open AccelerationEstimates QuadraticEstimates SecantValues CauchyValues

def secondState (t : Fraction) (ht : 0 < t.num) (s u : Point × Point) : Point × Point :=
  secantState (Fraction.mul (Fraction.ofInt 2) (TimeCalibration.inverse t ht))
    (secantState (TimeCalibration.inverse t ht) u s) (velocityState s)

def secondValue (t : Fraction) (ht : 0 < t.num) (x y : Value) : Value :=
  secantValue (Fraction.mul (Fraction.ofInt 2) (TimeCalibration.inverse t ht))
    (secantValue (TimeCalibration.inverse t ht) y x) (velocityValue x)

theorem second_identity (t : Fraction) (ht : 0 < t.num) (s u : Point × Point) (a : Point) :
  pointEquiv (pointSub (secondState t ht s u).1 a)
    (pointScale (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (TimeCalibration.inverse t ht) (TimeCalibration.inverse t ht)))
      (pointSub u.1 (quadraticPosition t s a))) := by
  constructor <|>
  simp only [secondState, secantState, pointState, velocityState, quadraticPosition,
    TimeCalibration.inverse, pointEquiv, pointSub, pointNeg, pointAdd, pointScale,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.half, Fraction.ofInt,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg, Int.one_mul, Int.mul_one] <|>
  ac_nf <|> omega

```

```

theorem second_state_time_congr (t u : Fraction) (ht : 0 < t.num) (hu : 0 < u.num)
  (he : Fraction.equiv t u) (s v : Point × Point) :
  pointEquiv (secondState t ht s v).1 (secondState u hu s v).1 := by
  have hi := TimeCalibration.inverse_congr ht hu he
  have h1 := pointScale_ratio_congr hi
  (show pointEquiv (pointSub v.1 s.1) (pointSub v.1 s.1) from
    (Fraction.equiv_refl _, Fraction.equiv_refl _))
  have hd := pointSub_congr h1
  (show pointEquiv s.2 s.2 from (Fraction.equiv_refl _, Fraction.equiv_refl _))
  exact pointScale_ratio_congr (Fraction.mul_equiv (Fraction.equiv_refl _) hi) hd

theorem finite_second_bound_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (n : Nat) (ht : 0 < (time h n).num)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1))).1) (a s.1))
  (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (secondState (time h n) ht s (run a h s n)).1 (a s.1))
    (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E (time h n) V))
      (Fraction.mul (Fraction.mul h (TimeCalibration.inverse (time h n) ht)) (pointNorm (a s.1))))) := by
  let q := Fraction.mul (Fraction.ofInt 2)
  (Fraction.mul (TimeCalibration.inverse (time h n) ht) (TimeCalibration.inverse (time h n) ht))
  have hq : 0 ≤ q.num := Fraction.nonnegative_mul _ _ (by decide)
  (Fraction.nonnegative_mul _ _ (Int.le_of_lt (time h n).den_pos) (Int.le_of_lt (time h n).den_pos))
  have he := Fraction.equiv_trans (pointNorm_equiv (second_identity (time h n) ht s (run a h s n) (a
    ↪ s.1)))
    (pointNorm_scale q _)
  have hb := Fraction.mul_le_mul_nonnegative_left
    (position_quadratic_remainder_at a h s L E V hh hL hE hV n hc hv) q hq
  have he' := Fraction.equiv_trans he
    (Fraction.mul_equiv (Fraction.abs_of_nonnegative q hq) (Fraction.equiv_refl _))
  apply Fraction.le_equiv_right (Fraction.le_equiv_left he' hb)
  simp only
  ↪ [q, TimeCalibration.inverse, Fraction.equiv, Fraction.mul, Fraction.add, Fraction.half, Fraction.ofInt,
    Int.add_mul, Int.mul_add]
  ac_nf

theorem finite_second_bound (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat) (ht : 0 < (time h n).num)
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (secondState (time h n) ht s (run a h s n)).1 (a s.1))
    (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E (time h n) V))
      (Fraction.mul h (Fraction.mul h (TimeCalibration.inverse (time h n) ht)) (pointNorm (a s.1))))) :=
  finite_second_bound_at a h s L E V hh hL hE hV n ht (fun _ => hc _) hv

theorem finite_second_equivalent_time_at (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (n : Nat) (ht : 0 < (time h n).num)
  (hc : ∀ k, k < n → Fraction.le
    (pointDistance (a (run a h s (k+1))).1) (a s.1))
  (Fraction.add (Fraction.mul L (pointDistance (run a h s (k+1)).1 s.1)) E)) (hu : 0 < u.num)
  (he : Fraction.equiv u (time h n))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (secondState u hu s (run a h s n)).1 (a s.1))
    (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E u V))
      (Fraction.mul (Fraction.mul h (TimeCalibration.inverse u hu)) (pointNorm (a s.1))))) := by
  apply Fraction.le_equiv_right
    (Fraction.le_equiv_left (pointDistance_equiv (second_state_time_congr u (time h n) hu ht he s _))
      (Fraction.equiv_refl _, Fraction.equiv_refl _))
    (finite_second_bound_at a h s L E V hh hL hE hV n ht hc hv))
  exact Fraction.add_equiv
    (Fraction.mul_equiv (Fraction.equiv_refl _) (Fraction.add_equiv
      (Fraction.mul_equiv (Fraction.equiv_refl L) (Fraction.mul_equiv (Fraction.equiv_symm he)
        (Fraction.equiv_refl V))) (Fraction.equiv_refl E)))
    (Fraction.mul_equiv (Fraction.mul_equiv (Fraction.equiv_refl h)
      (TimeCalibration.inverse_congr ht hu (Fraction.equiv_symm he))) (Fraction.equiv_refl _))

```

```

theorem finite_second_equivalent_time (a : Point → Point) (h : Fraction) (s : Point × Point)
  (L E V u : Fraction) (hh : 0 ≤ h.num) (hL : 0 ≤ L.num) (hE : 0 ≤ E.num) (hV : 0 ≤ V.num)
  (hc : comparisonContract a a L E) (n : Nat) (ht : 0 < (time h n).num) (hu : 0 < u.num)
  (he : Fraction.equiv u (time h n))
  (hv : ∀ k, k < n → Fraction.le (pointNorm (run a h s k).2) V) :
  Fraction.le (pointDistance (secondState u hu s (run a h s n)).1 (a s.1))
    (Fraction.add (Fraction.mul (Fraction.ofInt 2) (source L E u V))
      (Fraction.mul (Fraction.mul h (TimeCalibration.inverse u hu)) (pointNorm (a s.1)))) :=
  finite_second_equivalent_time_at a h s L E V u hh hL hE hV n ht (fun _ _ => hc _ _) hu he hv

end NewtonLimitDynamics.Polygon.QuadraticSecants

```

BarrowLib/Polygon/PairingValues.lean, 11 theorems, 206 lines

```

import BarrowLib.Polygon.SecantValues
import BarrowLib.Polygon.BinaryTime
import BarrowLib.Polygon.TriangleBounds
import BarrowLib.Polygon.SampledValues

/--! Dot products and determinants of completed positions, with one shared
Cauchy and representative-independence proof. Finite bilinear differences
and magnitude bounds are the input; bounded Cauchy tails are derived.
SampledValues covers globally Lipschitz maps, whereas these pairings are
Lipschitz only on bounded sets. No integral or derivative is a primitive. -/

namespace NewtonLimitDynamics.Polygon.PairingValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates HarmonicTimeComparison HarmonicStability
open HarmonicComparison HarmonicAccumulation
open CauchyValues HarmonicDyadic BinaryTime SecantValues HarmonicTimeRealization

/-- The finite identities needed to complete a scalar pairing. -/
structure Form where
  apply : Point → Point → Fraction
  difference_identity : ∀ p q r s,
    Fraction.equiv (durationDifference (apply p q) (apply r s))
      (Fraction.add (apply (pointSub r p) q) (apply r (pointSub s q)))
  magnitude_bound : ∀ p q,
    Fraction.le (apply p q).abs (Fraction.mul (pointNorm p) (pointNorm q))

theorem dot_abs_le_product (p q : Point) :
  Fraction.le (dot p q).abs (Fraction.mul (pointNorm p) (pointNorm q)) := by
  let cross := Fraction.add (Fraction.mul p.1.abs q.2.abs) (Fraction.mul p.2.abs q.1.abs)
  have hb := Fraction.le_equiv_right (Fraction.abs_add_le (Fraction.mul p.1 q.1) (Fraction.mul p.2 q.2))
    (Fraction.add_equiv (Fraction.abs_mul _ _) (Fraction.abs_mul _ _))
  have hc : 0 ≤ cross.num := Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _) (Fraction.abs_num_nonnegative _))
  apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans hb (Fraction.le_add_nonnegative _ cross
    ↪ hc))
  simp only [cross, pointNorm, Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

theorem dot_congr {p p' q q' : Point} (hp : pointEquiv p p') (hq : pointEquiv q q') :
  Fraction.equiv (dot p q) (dot p' q') :=
  Fraction.add_equiv (Fraction.mul_equiv hp.1 hq.1) (Fraction.mul_equiv hp.2 hq.2)

def dotForm : Form where
  apply := dot
  difference_identity := by
    intro p q r s
    simp only [dot, durationDifference, HarmonicStability.negF, pointSub, pointNeg, pointAdd,
      Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
    ac_nf <=> omega
  magnitude_bound := dot_abs_le_product

def detForm : Form where
  apply := det
  difference_identity := by

```

```

intro p q r s
simp only [det, durationDifference, HarmonicStability.negF, pointSub, pointNeg, pointAdd,
  Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
ac_nf <=> omega
magnitude_bound := TriangleBounds.det_abs_le_product

theorem difference_bound (f : Form) (p q r s : Point) :
  Fraction.le (durationDifference (f.apply p q) (f.apply r s)).abs
    (Fraction.add (Fraction.mul (pointDistance p r) (pointNorm q))
      (Fraction.mul (pointNorm r) (pointDistance q s))) := by
  have hb := Fraction.magnitudes.le_trans (Fraction.abs_add_le _ _)
    (Fraction.add_le_add (f.magnitude_bound (pointSub r p) q)
      (f.magnitude_bound r (pointSub s q)))
  exact Fraction.le_equiv_right (Fraction.le_equiv_left (Fraction.abs_equiv (f.difference_identity p q r
    ↪ s)) hb)
    (Fraction.add_equiv
      (Fraction.mul_equiv (pointDistance_symm r p) (Fraction.equiv_refl _))
      (Fraction.mul_equiv (Fraction.equiv_refl _) (pointDistance_symm s q)))

def pairingState (f : Form) (s t : Point × Point) : Point × Point :=
  scalarState (f.apply s.1 t.1)

theorem pairing_state_bound (f : Form) (R : Fraction) (hR : 0 ≤ R.num)
  (s t u v : Point × Point)
  (hu : Fraction.le (pointNorm u.1) R) (ht : Fraction.le (pointNorm t.1) R) :
  Fraction.le (distance (pairingState f s t) (pairingState f u v))
    (Fraction.mul (Fraction.add (distance s u) (distance t v)) R) := by
  have h1 := Fraction.mul_le_mul_nonnegative_left ht (pointDistance s.1 u.1) (pointNorm_nonnegative _)
  have h2 := Fraction.mul_le_mul_nonnegative hu (pointDistance t.1 v.1) (pointNorm_nonnegative _)
  have h3 := Fraction.mul_le_mul_nonnegative (point_le_state (stateSub s u)) R hR
  have h4 := Fraction.mul_le_mul_nonnegative (point_le_state (stateSub t v)) R hR
  have hb := Fraction.magnitudes.le_trans (difference_bound f s.1 t.1 u.1 v.1)
    (Fraction.add_le_add h1 (Fraction.le_equiv_right h2 (Fraction.mul_comm _ _)))
  exact Fraction.le_equiv_left (Fraction.equiv_trans (stateSub_norm_symm _ _)
    (scalarState_distance _ _))
    (Fraction.le_equiv_right (Fraction.magnitudes.le_trans hb (Fraction.add_le_add h3 h4))
      (Fraction.equiv_symm (Fraction.add_mul _ _ _)))

theorem fixed_left_bound (f : Form) (s t u : Point × Point) :
  Fraction.le (distance (pairingState f s t) (pairingState f s u))
    (Fraction.mul (distance t u) (pointNorm s.1)) := by
  have hb := difference_bound f s.1 t.1 s.1 u.1
  have hz : Fraction.equiv
    (Fraction.add (Fraction.mul (pointDistance s.1 s.1) (pointNorm t.1))
      (Fraction.mul (pointNorm s.1) (pointDistance t.1 u.1)))
    (Fraction.mul (pointDistance t.1 u.1) (pointNorm s.1)) := by
  apply Fraction.equiv_trans (Fraction.add_equiv
    (Fraction.mul_equiv (pointDistance_self_zero _) (Fraction.equiv_refl _)) (Fraction.equiv_refl _))
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.zero_mul, Int.mul_zero, Int.add_zero, Int.zero_add, Int.one_mul, Int.mul_one]
  ac_nf
  exact Fraction.le_equiv_left (Fraction.equiv_trans (stateSub_norm_symm _ _) (scalarState_distance _ _))
    (Fraction.magnitudes.le_trans (Fraction.le_equiv_right hb hz)
      (Fraction.mul_le_mul_nonnegative (point_le_state (stateSub t u)) (pointNorm s.1)
        (pointNorm_nonnegative _)))

private theorem pairing_small (f : Form) (R eps : Fraction) (hR : 0 ≤ R.num)
  (s t u v : Point × Point)
  (hu : Fraction.le (pointNorm u.1) R) (ht : Fraction.le (pointNorm t.1) R)
  (hsu : Fraction.lt (distance s u) (factorDelta R eps.half hR))
  (htv : Fraction.lt (distance t v) (factorDelta R eps.half hR)) :
  Fraction.lt (distance (pairingState f s t) (pairingState f u v)) eps := by
  have hx := factor_control R eps.half (distance s u) hR (stateNorm_nonnegative _) hsu
  have hy := factor_control R eps.half (distance t v) hR (stateNorm_nonnegative _) htv
  have hsmall := lt_equiv_right (Fraction.add_lt_add hx hy) (Fraction.half_add_self eps)
  exact Fraction.magnitudes.lt_of_le_lt (pairing_state_bound f R hR s t u v hu ht)
    (Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv (Fraction.add_mul _ _ _)) hsmall)

def pairingName (f : Form) (a b : EndpointCauchyName) : EndpointCauchyName where
  approx := fun j => pairingState f (a.approx j) (b.approx j)

```

```

cauchy := by
  intro eps heps
  obtain ⟨Ra,hRa,Na,hNa⟩ := position_bounded_tail a
  obtain ⟨Rb,hRb,Nb,hNb⟩ := position_bounded_tail b
  let R := Fraction.add Ra Rb
  have hR : 0 ≤ R.num := Fraction.nonnegative_add _ _ hRa hRb
  let delta := factorDelta R eps.half hR
  have hd : 0 < delta.num := factorDelta_positive _ _ _ heps
  obtain ⟨N,hN⟩ := a.cauchy delta hd
  obtain ⟨M,hM⟩ := b.cauchy delta hd
  refine (max (max Na Nb) (max N M), fun i j hi hj => ?_)
  apply pairing_small f R eps hR
  · exact Fraction.magnitudes.le_trans (hNa j (by omega)) (Fraction.le_add_nonnegative Ra Rb hRb)
  · exact Fraction.magnitudes.le_trans (hNb i (by omega))
    (Fraction.le_equiv_right (Fraction.le_add_nonnegative Rb Ra hRa) (Fraction.add_comm _ _))
  · exact hN i j (by omega) (by omega)
  · exact hM i j (by omega) (by omega)

theorem pairingName_equiv (f : Form) (a b a' b' : EndpointCauchyName)
  (ha : NameEquiv a a') (hb : NameEquiv b b') :
  NameEquiv (pairingName f a b) (pairingName f a' b') := by
  intro eps heps
  obtain ⟨Ra,hRa,Na,hNa⟩ := position_bounded_tail a'
  obtain ⟨Rb,hRb,Nb,hNb⟩ := position_bounded_tail b
  let R := Fraction.add Ra Rb
  have hR : 0 ≤ R.num := Fraction.nonnegative_add _ _ hRa hRb
  let delta := factorDelta R eps.half hR
  have hd : 0 < delta.num := factorDelta_positive _ _ _ heps
  obtain ⟨N,hN⟩ := ha delta hd
  obtain ⟨M,hM⟩ := hb delta hd
  refine (max (max Na Nb) (max N M), fun j hj => ?_)
  apply pairing_small f R eps hR
  · exact Fraction.magnitudes.le_trans (hNa j (by omega)) (Fraction.le_add_nonnegative Ra Rb hRb)
  · exact Fraction.magnitudes.le_trans (hNb j (by omega))
    (Fraction.le_equiv_right (Fraction.le_add_nonnegative Rb Ra hRa) (Fraction.add_comm _ _))
  · exact hN j (by omega)
  · exact hM j (by omega)

def pairingValue (f : Form) (x y : Value) : Value :=
  Quotient.liftOn₂ x y (fun a b => realize (pairingName f a b))
  (fun a b a' b' ha hb => Quotient.sound (pairingName_equiv f a b a' b' ha hb))

theorem pairingValue_embed (f : Form) (s t : Point × Point) :
  pairingValue f (embed s) (embed t) = embed (pairingState f s t) := rfl

theorem pairingValue_scalar (f : Form) (x y : Value) :
  PositionValues.firstValue (pairingValue f x y) = pairingValue f x y := by
  induction x using Quotient.inductionOn with
  | _ a =>
    induction y using Quotient.inductionOn with
    | _ b => rfl

theorem scaled_pairing_approximant (f : Form) (c : Fraction) (a b : EndpointCauchyName) (j : Nat) :
  stateEquiv ((secantName c (pairingName f a b))
    (constantName (PositionValues.zeroPoint, PositionValues.zeroPoint))).approx j)
  (scalarState (Fraction.mul c (f.apply (a.approx j).1 (b.approx j).1))) := by
  have hzero : PositionValues.zeroPoint = (Fraction.ofInt 0, Fraction.ofInt 0) := rfl
  have hscalar : ∀ q, scalarState q = ((q, Fraction.ofInt 0), (Fraction.ofInt 0, Fraction.ofInt 0)) := fun _
  ↦ => rfl
  constructor
  · constructor <:>
    simp only [secantName, pairingName, secantState, pointState, pairingState, constantName,
      hscalar, hzero, pointEquiv, pointSub, pointNeg, pointAdd, pointScale, Fraction.equiv, Fraction.add,
      Fraction.mul, Fraction.ofInt, Int.zero_mul, Int.zero_zero, Int.neg_zero, Int.add_zero,
      Int.zero_add, Int.mul_one, Int.one_mul]
  · exact {Fraction.equiv_refl _, Fraction.equiv_refl _}

/-- A proved bound on the fixed input tail transfers a completed bound in
the other input. The client supplies actual name estimates, not a bilinear
continuity premise on completed values. -/

```

```

theorem pairing_name_bound_right (f : Form) (a b c : EndpointCauchyName)
  (R S : Fraction) (hR : 0 ≤ R.num) (M : Nat)
  (ha : ∀ j, M ≤ j → Fraction.le (pointNorm (a.approx j)).1) R)
  (hbc : NameBound b c S) :
  NameBound (pairingName f a b) (pairingName f a c) (Fraction.mul S R) := by
  have hb := SampledValues.nameBound_affine (pairingName f a b) (pairingName f a c)
  b c R S (Fraction.ofInt 0) hR M (fun j hj => by
    have hlevel := Fraction.magnitudes.le_trans (fixed_left_bound f _ _ _)
      (Fraction.mul_le_mul_nonnegative_left (ha j hj) (distance (b.approx j) (c.approx j)))
      (stateNorm_nonnegative _))
    exact Fraction.le_equiv_right hlevel (Fraction.equiv_symm (Fraction.add_zero _))) hbc
  exact nameBound_mono _ _ _ (Fraction.le_of_equiv (Fraction.add_zero _)) hb
end NewtonLimitDynamics.Polygon.PairingValues

```

BarrowLib/Polygon/QuadraticPotentialValues.lean, 6 theorems, 157 lines

```

import BarrowLib.Polygon.PairingValues
import BarrowLib.Polygon.QuadraticSecants
import BarrowLib.Polygon.RationalIntervals

/-! Quadratic scalar values and their normalized tangent-continuation
increments. The polynomial remainder is finite algebra, and its Cauchy
completion uses the shared pairing and secant operators. No potential-force
relation or desired leading asymptotic is supplied as a premise. -/

namespace NewtonLimitDynamics.Polygon.QuadraticPotentialValues
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates HarmonicStability
open HarmonicTimeComparison CauchyValues SecantValues PairingValues BinaryTime HarmonicDyadic
  ↪ PositionValues
open QuadraticSecants

def quadratic (c : Fraction) (p : Point) : Fraction := Fraction.mul c (dot p p)

def quadraticName (c : Fraction) (a : EndpointCauchyName) : EndpointCauchyName :=
  secantName c (pairingName dotForm a a) (constantName
    ↪ (PositionValues.zeroPoint, PositionValues.zeroPoint))

def quadraticValue (c : Fraction) (x : Value) : Value :=
  secantValue c (pairingValue dotForm x x) (embed (PositionValues.zeroPoint, PositionValues.zeroPoint))

theorem quadraticValue_embed (c : Fraction) (s : Point × Point) :
  quadraticValue c (embed s) = embed (scalarState (quadratic c s.1)) := by
  apply Quotient.sound
  exact nameEquiv_of_levelwise_stateEquiv _ _ (fun j =>
    scaled_pairing_approximant dotForm c (constantName s) (constantName s) j)

def normalizedStep (c h : Fraction) (ht : 0 < h.num) (s u : Point × Point) : Fraction :=
  Fraction.mul (Fraction.mul (TimeCalibration.inverse h ht) (TimeCalibration.inverse h ht))
    (durationDifference (quadratic c (pointAdd s.1 (pointScale h s.2))) (quadratic c u.1))

def normalizedStepName (c h : Fraction) (ht : 0 < h.num) (a b : EndpointCauchyName) : EndpointCauchyName
  ↪ :=
  secantName (Fraction.mul (TimeCalibration.inverse h ht) (TimeCalibration.inverse h ht))
    (quadraticName c b) (quadraticName c (inertialName h a))

def normalizedStepValue (c h : Fraction) (ht : 0 < h.num) (x y : Value) : Value :=
  secantValue (Fraction.mul (TimeCalibration.inverse h ht) (TimeCalibration.inverse h ht))
    (quadraticValue c y) (quadraticValue c (inertialValue h x))

theorem normalized_remainder_identity (c h : Fraction) (ht : 0 < h.num)
  (s u : Point × Point) (a : Point) :
  Fraction.equiv (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u))
    (Fraction.mul c (Fraction.add
      (dot (pointSub (QuadraticSecants.secondState h ht s u).1 a) s.1)
      (Fraction.add (Fraction.mul h (dot (QuadraticSecants.secondState h ht s u).1 s.2))
        (Fraction.mul (Fraction.mul h h).half.half
          (dot (QuadraticSecants.secondState h ht s u).1 (QuadraticSecants.secondState h ht s u).1))))))
    ↪ := by

```

```

simp only [normalizedStep,quadratic,QuadraticSecants.secondState,
  secantState,pointState,velocityState,dot,durationDifference,negF,TimeCalibration.inverse,
  pointSub,pointNeg,pointAdd,pointScale,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.half,
  Fraction.ofInt,Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,Int.one_mul,Int.mul_one]
simp only [show (2 : Int)=1+1 by rfl,Int.add_mul,Int.mul_add,Int.one_mul,Int.mul_one]
ac_nf <;> omega

theorem normalized_remainder_bound (c h P V Z D : Fraction) (ht : 0 < h.num)
  (s u : Point × Point) (a : Point)
  (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hZ : 0 ≤ Z.num)
  (hp : Fraction.le (pointNorm s.1) P) (hv : Fraction.le (pointNorm s.2) V)
  (hz : Fraction.le (pointNorm (QuadraticSecants.secondState h ht s u).1) Z)
  (hd : Fraction.le (pointDistance (QuadraticSecants.secondState h ht s u).1 a) D) :
  Fraction.le (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u)).abs
    (Fraction.mul c.abs (Fraction.add (Fraction.mul D P)
      (Fraction.add (Fraction.mul h (Fraction.mul Z V))
        (Fraction.mul (Fraction.mul h h).half.half (Fraction.mul Z Z)))))) := by
  let z := (QuadraticSecants.secondState h ht s u).1
  let q := (Fraction.mul h h).half.half
  have hh : 0 ≤ h.num := Int.le_of_lt ht
  have hq : 0 ≤ q.num := Fraction.nonnegative_mul _ _ hh hh
  have h1 := Fraction.magnitudes.le_trans (dot_abs_le_product (pointSub z a) s.1)
    (Fraction.magnitudes.le_trans
      (Fraction.mul_le_mul_nonnegative_left hp (pointDistance z a) (pointNorm_nonnegative _))
      (Fraction.mul_le_mul_nonnegative hd P hP))
  have h2 := Fraction.magnitudes.le_trans (dot_abs_le_product z s.2)
    (Fraction.magnitudes.le_trans
      (Fraction.mul_le_mul_nonnegative_left hv (pointNorm z) (pointNorm_nonnegative _))
      (Fraction.mul_le_mul_nonnegative hz V hV))
  have h3 := Fraction.magnitudes.le_trans (dot_abs_le_product z z)
    (Fraction.magnitudes.le_trans
      (Fraction.mul_le_mul_nonnegative_left hz (pointNorm z) (pointNorm_nonnegative _))
      (Fraction.mul_le_mul_nonnegative hz Z hZ))
  have h2s := Fraction.mul_le_mul_nonnegative_left h2 h hh
  have h3s := Fraction.mul_le_mul_nonnegative_left h3 q hq
  have h2e := Fraction.equiv_trans (Fraction.abs_mul h (dot z s.2))
    (Fraction.mul_equiv (Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl _))
  have h3e := Fraction.equiv_trans (Fraction.abs_mul q (dot z z))
    (Fraction.mul_equiv (Fraction.abs_of_nonnegative q hq) (Fraction.equiv_refl _))
  have hinner := Fraction.magnitudes.le_trans (Fraction.abs_add_le (dot (pointSub z a) s.1)
    (Fraction.add (Fraction.mul h (dot z s.2)) (Fraction.mul q (dot z z))))
    (Fraction.add_le_add h1 (Fraction.magnitudes.le_trans (Fraction.abs_add_le _ _
      (Fraction.add_le_add (Fraction.le_equiv_left h2e h2s) (Fraction.le_equiv_left h3e h3s))))))
  have hscaled := Fraction.mul_le_mul_nonnegative_left hinner c.abs (Fraction.abs_num_nonnegative c)
  exact Fraction.le_equiv_left (Fraction.equiv_trans (Fraction.abs_equiv
    (normalized_remainder_identity c h ht s u a)) (Fraction.abs_mul _ _)) hscaled

def remainderCoefficient (c T P V Z U : Fraction) : Fraction :=
  Fraction.mul c.abs (Fraction.add (Fraction.mul P U)
    (Fraction.add (Fraction.mul Z V) (Fraction.mul T.half.half (Fraction.mul Z Z))))

def roundingCoefficient (c P A : Fraction) : Fraction :=
  Fraction.mul c.abs (Fraction.mul P A)

theorem remainderCoefficient_nonnegative (c T P V Z U : Fraction)
  (hT : 0 ≤ T.num) (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hZ : 0 ≤ Z.num) (hU : 0 ≤ U.num) :
  0 ≤ (remainderCoefficient c T P V Z U).num :=
  Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative c)
  (Fraction.nonnegative_add _ _ (Fraction.nonnegative_mul _ _ hP hU)
    (Fraction.nonnegative_add _ _ (Fraction.nonnegative_mul _ _ hZ hV)
      (Fraction.nonnegative_mul _ _ hT (Fraction.nonnegative_mul _ _ hZ hZ))))

theorem uniform_frame_bound (c h T P V Z U r A : Fraction) (ht : 0 < h.num)
  (s u : Point × Point) (a : Point)
  (hP : 0 ≤ P.num) (hV : 0 ≤ V.num) (hZ : 0 ≤ Z.num)
  (hwindow : Fraction.le h T)
  (hp : Fraction.le (pointNorm s.1) P) (hv : Fraction.le (pointNorm s.2) V)
  (hz : Fraction.le (pointNorm (QuadraticSecants.secondState h ht s u).1) Z)
  (hd : Fraction.le (pointDistance (QuadraticSecants.secondState h ht s u).1 a)
    (Fraction.add (Fraction.mul h U) (Fraction.mul r A))) :

```

```

    Fraction.le (durationDifference (Fraction.mul c (dot a s.1)) (normalizedStep c h ht s u)).abs
    (Fraction.add (Fraction.mul h (remainderCoefficient c T P V Z U))
      (Fraction.mul r (roundingCoefficient c P A))) := by
have hb := normalized_remainder_bound c h P V Z _ ht s u a hP hV hZ hp hv hz hd
have hsq := Fraction.mul_le_mul_nonnegative_left hwindow h (Int.le_of_lt ht)
have hquarter := RationalIntervals.half_le (RationalIntervals.half_le hsq)
have hlast := Fraction.mul_le_mul_nonnegative hquarter (Fraction.mul Z Z)
  (Fraction.nonnegative_mul _ _ hZ hZ)
have httotal := Fraction.mul_le_mul_nonnegative_left
  (Fraction.add_le_add_left (Fraction.add_le_add_left hlast (Fraction.mul h (Fraction.mul Z V)))
    (Fraction.mul (Fraction.add (Fraction.mul h U) (Fraction.mul r A)) P))
  c.abs (Fraction.abs_num_nonnegative c)
apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans hb httotal)
simp only [remainderCoefficient,roundingCoefficient,Fraction.equiv,Fraction.add,Fraction.mul,
  Fraction.half,Int.add_mul,Int.mul_add]
ac_nf

/-- The constructed scalar increment has the expected finite approximant.
This identity connects completed potential values to the polynomial estimate. -/
theorem normalized_step_approximant (c h : Fraction) (ht : 0 < h.num)
  (a b : EndpointCauchyName) (j : Nat) :
  stateEquiv ((normalizedStepName c h ht a b).approx j)
    (scalarState (normalizedStep c h ht (a.approx j) (b.approx j))) := by
have hzero : zeroPoint=(Fraction.ofInt 0,Fraction.ofInt 0) := rfl
have hscalar : ∀ q, scalarState q=((q,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 0)) := fun _
  ↦ => rfl
constructor
· constructor <=>
  simp only [normalizedStepName,normalizedStep,quadraticName,quadratic,inertialName,
    secantName,secantState,pointState,pairingName,pairingState,dotForm,hscalar,
    mapName,velocityState,constantName,hzero,
    pointEquiv,pointSub,pointNeg,pointAdd,pointScale,dot,durationDifference,negF,
    Fraction.equiv,Fraction.add,Fraction.mul,Fraction.ofInt,
    Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,Int.one_mul,Int.mul_one,
    Int.zero_mul,Int.mul_zero,Int.add_zero,Int.zero_add,Int.neg_zero] <=>
  ac_nf <=> omega
· exact (Fraction.equiv_refl _,Fraction.equiv_refl _)
end NewtonLimitDynamics.Polygon.QuadraticPotentialValues

```

BarrowLib/Polygon/TangentTriangleValues.lean, 2 theorems, 68 lines

```

import BarrowLib.Polygon.PairingValues
import BarrowLib.Polygon.QuadraticSecants

/-! The signed doubled triangle of a completed point, its tangent continuation
and an actual next completed point. Shared pairings and secants construct it;
finite algebra identifies its  $H^3$  normalization with half the determinant of
velocity and normalized second departure. It is not an unsigned lobe or area
of a matched region. -/

namespace NewtonLimitDynamics.Polygon.TangentTriangleValues
open NewtonLimitDynamics
open TimeSubdivision HarmonicStability PointBounds HarmonicDyadic CauchyValues
open PositionValues SecantValues PairingValues BinaryTime QuadraticSecants

def triangleValue (h : Fraction) (x y : Value) : Value :=
  pairingValue detForm (secantValue (Fraction.ofInt 1) (inertialValue h x) x)
    (secantValue (Fraction.ofInt 1) y (inertialValue h x))

def normalizedTriangleValue (h : Fraction) (ht : 0 < h.num) (x y : Value) : Value :=
  let q := TimeCalibration.inverse h ht
  secantValue (Fraction.mul q (Fraction.mul q q)) (triangleValue h x y) (embed (zeroPoint,zeroPoint))

theorem triangleValue_embed (h : Fraction) (s u : Point × Point) :
  triangleValue h (embed s) (embed u) =
    embed (scalarState (TriangleBounds.triangleTwice s.1 (pointAdd s.1 (pointScale h s.2)) u.1)) := by
  apply Quotient.sound
  apply nameEquiv_of_levelwise_stateEquiv
  intro j
  have hzero : zeroPoint=(Fraction.ofInt 0,Fraction.ofInt 0) := rfl

```



```

have hscalar : ∀ q, scalarState q=((q,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 0)) := fun _
  ↪ => rfl
constructor
· constructor <=>
  simp only [triangleValue,inertialValue,inertialName,secantName,secantState,pointState,
    velocityValue,mapValue,mapName,velocityState,pairingName,pairingState,detForm,det,
    TriangleBounds.triangleTwice,constantName,hscalar,hzero,pointEquiv,pointSub,pointNeg,
    pointAdd,pointScale,negF,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.ofInt,
    Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,Int.one_mul,Int.mul_one,
    Int.zero_mul,Int.mul_zero,Int.add_zero,Int.zero_add,Int.neg_zero] <=>
  ac_nf <=> omega
· exact {Fraction.equiv_refl _,Fraction.equiv_refl _}

/-- Exact completed identity for the signed doubled tangent-deflection
triangle. The next point is arbitrary; no curve expansion is assumed. -/
theorem normalized_triangle_identity (h : Fraction) (ht : 0 < h.num) (x y : Value) :
  normalizedTriangleValue h ht x y =
    secantValue (Fraction.ofInt 1).half
      (pairingValue detForm (velocityValue x) (QuadraticSecants.secondValue h ht x y)) (embed
        ↪ (zeroPoint,zeroPoint)) := by
induction x using Quotient.inductionOn with
| _ a =>
  induction y using Quotient.inductionOn with
  | _ b =>
    apply Quotient.sound
    apply nameEquiv_of_levelwise_stateEquiv
    intro j
    have hzero : zeroPoint=(Fraction.ofInt 0,Fraction.ofInt 0) := rfl
    have hscalar : ∀ q, scalarState q=((q,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 0)) := fun
      ↪ _ => rfl
    constructor
    · constructor <=>
      simp only [normalizedTriangleValue,triangleValue,inertialValue,inertialName,QuadraticSecants.s,
        ↪ econdValue,
        velocityValue,mapValue,mapName,velocityState,secantName,secantState,pointState,
        pairingName,pairingState,detForm,det,constantName,hscalar,hzero,TimeCalibration.inverse,
        pointEquiv,pointSub,pointNeg,pointAdd,pointScale,negF,Fraction.equiv,Fraction.add,
        Fraction.mul,Fraction.half,Fraction.ofInt,Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,
        Int.one_mul,Int.mul_one,Int.zero_mul,Int.mul_zero,Int.add_zero,Int.zero_add,Int.neg_zero] <=>
      ac_nf <=> omega
    · exact {Fraction.equiv_refl _,Fraction.equiv_refl _}

end NewtonLimitDynamics.Polygon.TangentTriangleValues

```

NewtonLimitDynamics/Polygon/ForceClasses.lean, 23 theorems, 397 lines

```

import NewtonLimitDynamics.Polygon.PositionValues
import NewtonLimitDynamics.Polygon.InertialDefect
import NewtonLimitDynamics.Polygon.StripArea
import BarrowLib.Polygon.FiniteEstimates
import BarrowLib.Polygon.BoundedIteration
import BarrowLib.Polygon.FiniteAccumulation
import BarrowLib.Polygon.CalibratedRefinement

/-!
Uniform rational approximations to possibly irrational accelerations on an
explicit region. These are contracts on the force data, not assertions that
an impulse polygon converges. Realized sample values are constructed Cauchy
quotients. Historical editions and action diagnostics are not premises here.
-/

namespace NewtonLimitDynamics.Polygon.ForceClasses

open NewtonLimitDynamics
open TimeSubdivision PointBounds CentralSchedule HarmonicStability
open HarmonicComparison HarmonicDyadic CauchyValues

/-- Store a rational acceleration in the position slots of a state name. -/
def accelerationState (p : Point) : Point × Point :=
  PositionValues.positionState (p, p)

```

```

/-- Uniform sampling error is proved for the force oracle, not the motion. -/
structure Oracle where
  region : Point → Prop
  sample : Nat → Field
  error : Nat → Fraction
  error_nonnegative : ∀ n, 0 ≤ (error n).num
  error_vanishes : ∀ eps : Fraction, 0 < eps.num →
    ∃ N : Nat, ∀ n, N ≤ n → Fraction.lt (error n) eps
  coherent : ∀ i j, i ≤ j → ∀ p, region p →
    Fraction.le (distance (accelerationState (sample i p))
      (accelerationState (sample j p))) (error i)

/-- A central oracle also records the inward sense, absent from collinearity. -/
structure CentralOracle extends Oracle where
  inward : ∀ n p, ∃ k : Fraction, 0 ≤ k.num ∧
    pointEquiv (sample n p) (linearField k p)

/-- Inward samples vanish at the centre, even when their magnitude is rounded. -/
theorem sample_origin_zero (o : CentralOracle) (n : Nat) :
  pointEquiv (o.sample n (Fraction.ofInt 0, Fraction.ofInt 0))
    (Fraction.ofInt 0, Fraction.ofInt 0) := by
  obtain ⟨k, _, hk⟩ := o.inward n (Fraction.ofInt 0, Fraction.ofInt 0)
  apply pointEquiv_trans hk
  constructor <=> simp only [linearField, pointScale, Fraction.equiv, Fraction.mul,
    Fraction.ofInt, Int.mul_zero, Int.zero_mul]

/-- Every approximating finite polygon is central, even outside confinement. -/
theorem sample_central (o : CentralOracle) (n : Nat) : central (o.sample n) := by
  intro p
  obtain ⟨k, _, hk⟩ := o.inward n p
  exact Fraction.equiv_trans
    (InertialDefect.det_congr (Fraction.equiv_refl _, Fraction.equiv_refl _) hk)
    (linearField_central k p)

theorem sampled_finite_area_law (o : CentralOracle) (n : Nat)
  (ds : List Fraction) (s : Point × Point) :
  Fraction.equiv (swept (o.sample n) ds s)
    (Fraction.mul (elapsed ds) (momentum s)) :=
  swept_eq (o.sample n) (sample_central o n) ds s

/-- Pointwise force values are constructed from the uniform-error contract. -/
def accelerationName (o : Oracle) (p : Point) (hp : o.region p) :
  EndpointCauchyName where
  approx := fun n => accelerationState (o.sample n p)
  cauchy := by
    intro eps heps
    obtain ⟨N, hN⟩ := o.error_vanishes eps heps
    refine ⟨N, ?_⟩
    intro m n hm hn
    by_cases hmn : m ≤ n
    · exact Fraction.magnitudes.lt_of_le_lt (o.coherent m n hmn p hp) (hN m hm)
    · have hnm : n ≤ m := by omega
      have he := stateSub_norm_symm
        (accelerationState (o.sample m p)) (accelerationState (o.sample n p))
      exact Fraction.magnitudes.lt_of_le_lt
        (Fraction.le_equiv_left he (o.coherent n m hnm p hp)) (hN n hn)

def accelerationValue (o : Oracle) (p : Point) (hp : o.region p) : Value :=
  realize (accelerationName o p hp)

theorem acceleration_approximants_converge (o : Oracle) (p : Point)
  (hp : o.region p) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ n, N ≤ n →
    Within (embed (accelerationState (o.sample n p)))
      (accelerationValue o p hp) eps :=
  constant_approximants_converge (accelerationName o p hp) eps heps

/-- Rounding error remains additive; individual samples need not be Lipschitz. -/
def Lipschitz0n (o : Oracle) (L : Fraction) : Prop :=

```

```

 $0 \leq L.\text{num} \wedge \forall n p q, o.\text{region } p \rightarrow o.\text{region } q \rightarrow$ 
  Fraction.le (pointNorm (pointSub (o.sample n p) (o.sample n q)))
    (Fraction.add (Fraction.mul L (pointNorm (pointSub p q)))
      (Fraction.add (o.error n) (o.error n)))

/-- Centrality at the origin turns comparison into linear growth on the region.
This does not assert a globally bounded force. -/
theorem sample_linear_growth (o : CentralOracle) (L : Fraction)
  (hL : LipschitzOn o.toOracle L)
  (hzero : o.region (Fraction.ofInt 0, Fraction.ofInt 0))
  (n : Nat) (p : Point) (hp : o.region p) :
  Fraction.le (pointNorm (o.sample n p))
    (Fraction.add (Fraction.mul L (pointNorm p)) (Fraction.add (o.error n) (o.error n))) := by
have ha : Fraction.equiv
  (pointNorm (pointSub (o.sample n p) (o.sample n (Fraction.ofInt 0, Fraction.ofInt 0))))
  (pointNorm (o.sample n p)) := pointNorm_equiv (pointEquiv_trans
  (pointSub_congr (Fraction.equiv_refl _, Fraction.equiv_refl _)
    (sample_origin_zero o n)) (pointSub_zero _))
exact Fraction.le_equiv_right
  (Fraction.le_equiv_left (Fraction.equiv_symm ha) (hL.2 n p _ hp hzero))
  (Fraction.add_equiv
    (Fraction.mul_equiv_left L (pointNorm_equiv (pointSub_zero p))) (Fraction.equiv_refl _))

/-- Continuous classes have a uniform modulus on the named confined region. -/
def ContinuousOn (o : Oracle) : Prop :=
   $\forall \text{eps} : \text{Fraction}, 0 < \text{eps}.\text{num} \rightarrow \exists \text{delta} : \text{Fraction}, 0 < \text{delta}.\text{num} \wedge$ 
   $\exists N : \text{Nat}, \forall n p q, N \leq n \rightarrow o.\text{region } p \rightarrow o.\text{region } q \rightarrow$ 
  Fraction.lt (pointNorm (pointSub p q)) delta  $\rightarrow$ 
  Fraction.lt (pointNorm (pointSub (o.sample n p) (o.sample n q))) eps

def BoundedOn (o : Oracle) (B : Fraction) : Prop :=
   $0 \leq B.\text{num} \wedge \forall n p, o.\text{region } p \rightarrow \text{Fraction.le (pointNorm (o.sample n p)) } B$ 

theorem acceleration_distance (p q : Point) :
  Fraction.equiv (distance (accelerationState p) (accelerationState q))
    (FiniteEstimates.pointDistance p q) := by
change Fraction.equiv
  (Fraction.add (FiniteEstimates.pointDistance p q) (Fraction.ofInt 0))
  (FiniteEstimates.pointDistance p q)
exact Fraction.add_zero _

/-- Uniform force coherence also controls successive precisions at the same
rational point. No exact force value is presumed rational. -/
theorem sample_point_error (o : Oracle) (i j : Nat) (hij : i  $\leq$  j)
  (p : Point) (hp : o.region p) :
  Fraction.le (FiniteEstimates.pointDistance (o.sample i p) (o.sample j p))
    (o.error i) :=
  Fraction.le_equiv_left (Fraction.equiv_symm (acceleration_distance _ _))
    (o.coherent i j hij p hp)

/-- Compare only the two supplied regional points. No whole-plane premise
is needed, and coherence is used at the second certified point. -/
theorem samples_comparison_contract (o : Oracle) (L : Fraction)
  (hL : LipschitzOn o L) (i j : Nat) (hij : i  $\leq$  j)
  (p q : Point) (hp : o.region p) (hq : o.region q) :
  Fraction.le (FiniteEstimates.pointDistance (o.sample i p) (o.sample j q))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q))
      (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i))) := by
have ht := FiniteEstimates.pointDistance_triangle (o.sample i p)
  (o.sample i q) (o.sample j q)
have hs := Fraction.add_le_add (hL.2 i p q hp hq)
  (sample_point_error o i j hij q hq)
apply Fraction.le_equiv_right (Fraction.magnitudes.le_trans ht hs)
simp only [Fraction.equiv, Fraction.add, Int.add_mul, Int.mul_add]
ac_nf

/-- Distance is encoded by its square; no rational square root is assumed. -/
def DistanceOnly (o : CentralOracle) : Prop :=
   $\forall n p q, \text{Fraction.equiv (dot p p) (dot q q)} \rightarrow$ 
   $\exists k : \text{Fraction}, 0 \leq k.\text{num} \wedge$ 

```

```

    pointEquiv (o.sample n p) (linearField k p) ∧
    pointEquiv (o.sample n q) (linearField k q)

/-- Class (a)'s force regularity. Potential identification is a separate step. -/
def ClassAForce (o : CentralOracle) : Prop :=
  DistanceOnly o ∧ ∃ L : Fraction, LipschitzOn o.toOracle L

/-- Class (b) permits direction dependence; nonconservativity needs its own test. -/
def ClassBForce (o : CentralOracle) : Prop := ContinuousOn o.toOracle

def ClassCForce (o : CentralOracle) : Prop :=
  DistanceOnly o ∧ ContinuousOn o.toOracle

/-- No regularity premise is used by the finite area law. -/
def ClassDForce (_o : CentralOracle) : Prop := True

/-- An exactly rational law is a zero-error instance of the sampling interface. -/
def exactOracle (a : Field) : Oracle where
  region := fun _ => True
  sample := fun _ => a
  error := fun _ => Fraction.ofInt 0
  error_nonnegative := fun _ => by decide
  error_vanishes := by
    intro eps heps
    exact {0, fun _ => (Fraction.positive_iff_zero_lt eps).mp heps}
  coherent := by
    intro _ _ _ p _
    exact Fraction.le_of_equiv (HarmonicAccumulation.stateSub_self_norm_zero _)

theorem exact_accelerationValue (a : Field) (p : Point) :
  accelerationValue (exactOracle a) p True.intro =
  embed (accelerationState (a p)) := rfl

def harmonicOracle (w : Fraction) (hw : 0 ≤ w.num) : CentralOracle where
  toOracle := exactOracle (linearField w)
  inward := fun _ p => {w, hw, {Fraction.equiv_refl _, Fraction.equiv_refl _}}

theorem harmonic_distance_only (w : Fraction) (hw : 0 ≤ w.num) :
  DistanceOnly (harmonicOracle w hw) := by
  intro _ p q _
  exact {w, hw, {Fraction.equiv_refl _, Fraction.equiv_refl _},
    {Fraction.equiv_refl _, Fraction.equiv_refl _}}

theorem harmonic_comparison_contract (w : Fraction) :
  FiniteEstimates.comparisonContract (linearField w) (linearField w)
  w.abs (Fraction.ofInt 0) := linearField_comparison_contract w

theorem harmonic_lipschitz_on (w : Fraction) (hw : 0 ≤ w.num) :
  LipschitzOn (harmonicOracle w hw).toOracle w.abs := by
  refine {Fraction.abs_num_nonnegative w, ?_}
  intro n p q _
  have h := harmonic_comparison_contract w p q
  apply Fraction.le_equiv_right h
  exact Fraction.add_equiv (Fraction.equiv_refl _)
    (Fraction.equiv_symm (Fraction.add_zero (Fraction.ofInt 0)))

theorem harmonic_class_a_force (w : Fraction) (hw : 0 ≤ w.num) :
  ClassAForce (harmonicOracle w hw) :=
  (harmonic_distance_only w hw, w.abs, harmonic_lipschitz_on w hw)

/-- The permitted centre-at-infinity instance is a uniform parallel field. -/
def parallelOracle (a : Point) : Oracle := exactOracle (fun _ => a)

theorem parallel_cell (a : Point) (h : Fraction) (s : Point × Point) :
  cell ((parallelOracle a).sample 0) h s = endKick h s a := rfl

/-- Multiplication by the fixed force magnitude converts this determinant
into the velocity component perpendicular to the parallel force. -/
theorem parallel_transverse_cell (a : Point) (h : Fraction) (s : Point × Point) :
  Fraction.equiv (det (cell (fun _ => a) h s).2 a) (det s.2 a) :=

```

```

StripArea.det_kick_direction_constant h s.2 a

theorem parallel_transverse_schedule (a : Point) (ds : List Fraction)
  (s : Point × Point) :
  Fraction.equiv (det (schedule (fun _ => a) ds s).2 a) (det s.2 a) := by
  induction ds generalizing s with
  | nil => exact Fraction.equiv_refl _
  | cons h ds ih =>
    exact Fraction.equiv_trans (ih (cell (fun _ => a) h s))
      (parallel_transverse_cell a h s)

/-- Identify the generic forward iterates with the actual finite schedule. -/
theorem run_eq_schedule (a : Field) (h : Fraction) (s : Point × Point) (n : Nat) :
  BoundedIteration.run a h s n = schedule a (List.replicate n h) s := by
  induction n generalizing s with
  | zero => rfl
  | succ n ih =>
    simp only [List.replicate_succ, schedule]
    rw [← ih]
    exact (BoundedIteration.run_commute a h s n).symm

theorem bounded_samples_of_confined (o : Oracle) (j : Nat) (h : Fraction)
  (s : Point × Point) (B : Fraction) (n : Nat) (hB : BoundedOn o B)
  (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run (o.sample j) h s (i+1)).1) :
  BoundedIteration.BoundedSamples (o.sample j) h s B n :=
  fun i hi => hB.2 j _ (hR i hi)

/-- These actual polygon bounds apply also to continuous, non-Lipschitz laws;
confinement is an explicit premise, not a conclusion of continuity. -/
theorem sampled_polygon_velocity_bound (o : Oracle) (j : Nat) (h : Fraction)
  (s : Point × Point) (B : Fraction) (n : Nat) (hh : 0 ≤ h.num)
  (hB : BoundedOn o B)
  (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run (o.sample j) h s (i+1)).1) :
  Fraction.le (pointNorm (schedule (o.sample j) (List.replicate n h) s).2)
    (Fraction.add (pointNorm s.2) (Fraction.mul (BoundedIteration.time h n) B)) := by
  rw [← run_eq_schedule]
  exact BoundedIteration.velocity_bound (o.sample j) h s B hh n
  (bounded_samples_of_confined o j h s B n hB hR)

theorem sampled_polygon_position_bound (o : Oracle) (j : Nat) (h : Fraction)
  (s : Point × Point) (B : Fraction) (n : Nat) (hh : 0 ≤ h.num)
  (hB : BoundedOn o B)
  (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run (o.sample j) h s (i+1)).1) :
  Fraction.le (pointNorm (schedule (o.sample j) (List.replicate n h) s).1)
    (BoundedIteration.positionCap h s B n) := by
  rw [← run_eq_schedule]
  exact BoundedIteration.position_bound (o.sample j) h s B hh hB.1 n
  (bounded_samples_of_confined o j h s B n hB hR)

theorem sampled_polygon_state_bound (o : Oracle) (j : Nat) (h : Fraction)
  (s : Point × Point) (B T : Fraction) (n : Nat) (hh : 0 ≤ h.num)
  (hT : 0 ≤ T.num) (hB : BoundedOn o B)
  (ht : Fraction.le (BoundedIteration.time h n) T)
  (hR : ∀ i : Nat, i < n → o.region (BoundedIteration.run (o.sample j) h s (i+1)).1) :
  Fraction.le (stateNorm (schedule (o.sample j) (List.replicate n h) s))
    (Fraction.add (BoundedIteration.uniformPositionCap T s B)
      (Fraction.add (pointNorm s.2) (Fraction.mul T B))) := by
  rw [← run_eq_schedule]
  exact BoundedIteration.state_bound_at_time (o.sample j) h s B T hh hB.1 hT n
  (bounded_samples_of_confined o j h s B n hB hR) ht

/-- Merely continuous laws give a vanishing local refinement source.
The modulus controls all three actual arrival points. This is a consistency
estimate, distinct from stability, full-family convergence or uniqueness. -/
theorem continuous_local_refinement (o : Oracle) (hC : ContinuousOn o)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ delta : Fraction, 0 < delta.num ∧ ∃ N : Nat,
  ∀ (j : Nat) (h B : Fraction) (s : Point × Point), N ≤ j → BoundedOn o B →
  o.region (FiniteEstimates.cell (o.sample j) h s).1 →
  o.region (FiniteEstimates.oneFull (o.sample j) h s).1 →

```

```

o.region (FiniteEstimates.twoHalf (o.sample j) h s).1 →
Fraction.lt (Fraction.mul h.abs (pointNorm s.2)) delta →
Fraction.lt (Fraction.mul (Fraction.mul h h).abs B) delta →
Fraction.le
  (FiniteEstimates.stateDistance (FiniteEstimates.twoHalf (o.sample j) h s)
    (FiniteEstimates.oneFull (o.sample j) h s))
  (Fraction.add (Fraction.mul (Fraction.mul h h).abs B)
    (Fraction.mul h.abs (Fraction.add eps eps))) := by
obtain (delta, hdelta, N, hN) := hC eps heps
refine (delta, hdelta, N, ?_)
intro j h B s hj hB hfirst hfull hfine hv hf
have hb := hB.2 j _ hfirst
have hd₁ := Fraction.magnitudes.lt_of_le_lt
  (Fraction.le_of_equiv (FiniteEstimates.first_to_full_distance (o.sample j) h s)) hv
have hd₂ := Fraction.magnitudes.lt_of_le_lt
  (FiniteEstimates.twoHalf_position_error (o.sample j) h B s hb) hf
have hs₁ := Fraction.magnitudes.lt_implies_le
  (hN j _ _ hj hfirst hfull hd₁)
have hs₂ := Fraction.magnitudes.lt_implies_le
  (hN j _ _ hj hfine hfull hd₂)
exact Fraction.add_le_add
  (FiniteEstimates.twoHalf_position_error (o.sample j) h B s hb)
  (FiniteEstimates.twoHalf_velocity_sample_error (o.sample j) h eps eps s hs₁ hs₂)

/-- Successive oracle precisions instantiate the actual mesh comparison.
Only the five actual and shadow arrivals need regional certificates. The
central construction derives those certificates from its finite invariant. -/
theorem sampled_uniform_refinement (o : Oracle) (j : Nat)
  (h L B V : Fraction) (s : Point × Point) (n : Nat)
  (hh : 0 ≤ h.num) (hL : LipschitzOn o L)
  (hR : ∀ k, k < n →
    let t := FiniteAccumulation.coarseAt (o.sample j) h s k
    let u := FiniteAccumulation.fineAt (o.sample (j+1)) h s k
    o.region (FiniteEstimates.cell (o.sample (j+1)) h u).1 ∧
    o.region (FiniteEstimates.twoHalf (o.sample (j+1)) h u).1 ∧
    o.region (FiniteEstimates.cell (o.sample j) h t).1 ∧
    o.region (FiniteEstimates.twoHalf (o.sample j) h t).1 ∧
    o.region (FiniteEstimates.oneFull (o.sample j) h t).1)
  (hB : BoundedOn o B) (hV : 0 ≤ V.num)
  (hs : FiniteAccumulation.SmallWindow h L n)
  (hvel : ∀ k, k < n → Fraction.le
    (pointNorm (FiniteAccumulation.coarseAt (o.sample j) h s k).2) V) :
Fraction.le
  (FiniteEstimates.stateDistance (FiniteAccumulation.fineAt (o.sample (j+1)) h s n)
    (FiniteAccumulation.coarseAt (o.sample j) h s n))
  (Fraction.mul (Fraction.ofInt 2)
    (Fraction.mul (FiniteAccumulation.count n)
      (FiniteAccumulation.uniformBlockSource h L
        (Fraction.add (Fraction.add (o.error j) (o.error j)) (o.error j)) B V))) := by
let E := Fraction.add (Fraction.add (o.error j) (o.error j)) (o.error j)
have hE : 0 ≤ E.num := Fraction.nonnegative_add _ _
  (Fraction.nonnegative_add _ _ (o.error_nonnegative j) (o.error_nonnegative j))
  (o.error_nonnegative j)
have hcross : ∀ p q, o.region p → o.region q →
  Fraction.le (FiniteEstimates.pointDistance (o.sample (j+1) p) (o.sample j q))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) E) := by
intro p q hp hq
exact Fraction.le_equiv_right
  (Fraction.le_equiv_left (FiniteEstimates.pointDistance_symm _ _)
    (samples_comparison_contract o L hL j (j+1) (by omega) q p hq hp))
  (Fraction.add_equiv (Fraction.mul_equiv_left L (FiniteEstimates.pointDistance_symm q p))
    (Fraction.equiv_refl _))
have hwindow : TimeCalibration.Window (Fraction.ofInt 1) h L (by decide) (2*n) := by
unfold TimeCalibration.Window
rw [Fraction.abs_eq_of_nonnegative h hh]
apply Fraction.le_equiv_left (b := Fraction.mul (FiniteAccumulation.totalTime h n)
  (Fraction.add (Fraction.ofInt 1) L)) _ hs
simp only [TimeCalibration.rate, TimeCalibration.inverse, FiniteAccumulation.totalTime,
  Fraction.equiv, Fraction.mul, Fraction.add, Fraction.ofInt, Int.natCast_mul,
  Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]

```

```

ac_nf
have hc : ∀ k, k < n → CalibratedRefinement.BlockComparisons
  (o.sample (j+1)) (o.sample j) h L E
  (FiniteAccumulation.fineAt (o.sample (j+1)) h s k)
  (FiniteAccumulation.coarseAt (o.sample j) h s k) := by
  intro k hk
  obtain (hfirst,hsecond,hshadow,hshadow2,hfull) := hR k hk
  exact {hcross _ _ hfirst hshadow,hcross _ _ hsecond hshadow2,
    samples_comparison_contract o L hL j j (by omega) _ _ hshadow hfull,
    samples_comparison_contract o L hL j j (by omega) _ _ hshadow2 hfull}
have hb := CalibratedRefinement.actual_uniform_error_at (Fraction.ofInt 1) (by decide)
  (o.sample (j+1)) (o.sample j) h L E B V s hL.1 hE hB.1 hV n hwindow hc
  (fun k hk => hB.2 j _ (hR k hk).2.2.1) hvel
apply Fraction.le_equiv_right
  (Fraction.le_equiv_left (Fraction.equiv_symm (TimeCalibration.distance_unit_calibration _ _)) hb)
simp only [E,CalibratedRefinement.blockSource,CalibratedRefinement.localSource,
  FiniteAccumulation.count,FiniteAccumulation.uniformBlockSource,
  FiniteAccumulation.uniformLocalBudget,TimeCalibration.amplification,TimeCalibration.inverse,
  FiniteEstimates.amplification,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.ofInt,
  Int.add_mul,Int.mul_add,Int.one_mul,Int.mul_one]
ac_nf

```

end NewtonLimitDynamics.Polygon.ForceClasses

NewtonLimitDynamics/Polygon/CauchyValues.lean, 7 theorems, 130 lines

```

import BarrowLib.Polygon.CauchyValues
import BarrowLib.Polygon.TailValues
import NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

namespace NewtonLimitDynamics.Polygon.CauchyValues
open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
open HarmonicAccumulation
open HarmonicDyadic
open HarmonicTimeComparison
open HarmonicBinaryPrefix

def timeValue (w : Fraction) (s : Point × Point)
  (T : ShortRationalTime w) : Value := realize (timeName w s T)

def binaryValue (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : Value :=
  realize (prefixName b w T s hT hs)

theorem timeValue_bound (w : Fraction) (s : Point × Point)
  (T U : ShortRationalTime w) :
  Within (timeValue w s U) (timeValue w s T)
  (Fraction.mul (timeLipschitz w s)
    (durationDifference T.val U.val).abs) := by
  apply nameBound_of_eventual_le _ _ _ 0
  intro n _
  exact endpoint_time_bound w T.val U.val s n
  T.property.1 U.property.1 T.property.2 U.property.2

theorem binaryValue_prefix_bound (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (m : Nat) :
  Within (embed (prefixState b w T s m))
  (binaryValue b w T s hT hs) (HarmonicBinaryPrefix.tailCap w T s m) := by
  exact TailValues.approximant_bound (prefixName b w T s hT hs)
  (HarmonicBinaryPrefix.coefficient w T s) (HarmonicBinaryPrefix.coefficient_nonnegative w T s hT)
  (fun j => HarmonicBinaryPrefix.adjacent_error_le b w T s j hT hs) m

theorem rational_time_uniform_value_bound (w : Fraction) (s : Point × Point)
  (eps : Fraction) (heps : 0 < eps.num)
  (T U : ShortRationalTime w)
  (hnear : Fraction.lt (durationDifference T.val U.val).abs
    (timeDelta w s eps.half)) :

```

```

    Within (timeValue w s U) (timeValue w s T) eps.half := by
    have hb := parameter_delta_control w s eps.half
    (durationDifference T.val U.val).abs heps
    (Fraction.abs_num_nonnegative _) hnear
    apply nameBound_of_eventual_le _ _ _ 0
    intro n _
    have hlevel := endpoint_time_bound w T.val U.val s n
    T.property.1 U.property.1 T.property.2 U.property.2
    exact Fraction.magnitudes.lt_implies_le
    (Fraction.magnitudes.lt_of_le_lt hlevel hb)

def endpointValue (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) : Value :=
  realize (endpointName w T s hT hs)

private def sampleOne : Fraction := {1, 1, by decide}
private def sampleZero : Fraction := {0, 1, by decide}
private def sampleQuarter : Fraction := {1, 4, by decide}
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))
private def sampleTail : Fraction := {3, 8, by decide}
private def sampleLower : Fraction := {3, 16, by decide}

theorem sample_endpoint_zero_distance :
  Fraction.equiv (distance
    (endpoint sampleOne sampleQuarter sampleState 0) sampleState)
    (9, 16, by decide) := by decide

theorem sample_tail_zero :
  Fraction.equiv (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 0)
    sampleTail := by decide

theorem sample_lower_all_levels (j : Nat) :
  Fraction.le sampleLower
    (distance (endpoint sampleOne sampleQuarter sampleState j) sampleState) := by
  let e₀ := endpoint sampleOne sampleQuarter sampleState 0
  let eⱼ := endpoint sampleOne sampleQuarter sampleState j
  have hs : DyadicSmallTime sampleOne sampleQuarter := by
    unfold DyadicSmallTime Fraction.le
    decide
  have hgap := HarmonicDyadic.finite_gap_error sampleOne sampleQuarter
    sampleState (by decide) hs j 0
  have hgap' : Fraction.le (distance e₀ eⱼ)
    (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 0) :=
    Fraction.le_equiv_left (stateSub_norm_symm e₀ eⱼ)
    (by simpa only [Nat.zero_add] using hgap)
  have htri := stateSub_triangle e₀ eⱼ sampleState
  have hchain := Fraction.magnitudes.le_trans htri
  (Fraction.add_le_add_right hgap' (distance eⱼ sampleState))
  have hL : Fraction.equiv (Fraction.add sampleTail sampleLower)
    (distance e₀ sampleState) :=
    Fraction.equiv_trans (by decide)
    (Fraction.equiv_symm sample_endpoint_zero_distance)
  have hR : Fraction.equiv
    (Fraction.add
      (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 0)
      (distance eⱼ sampleState))
    (Fraction.add sampleTail (distance eⱼ sampleState)) :=
    Fraction.add_equiv sample_tail_zero (Fraction.equiv_refl _)
  have hfull := Fraction.le_equiv_right (Fraction.le_equiv_left hL hchain) hR
  exact le_add_cancel_left sampleTail sampleLower (distance eⱼ sampleState) hfull

theorem sample_endpoint_value_ne_initial :
  endpointValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide) ≠
  embed sampleState := by
  intro heq
  have hnames : NameEquiv
    (endpointName sampleOne sampleQuarter sampleState
      (by decide) (by unfold DyadicSmallTime Fraction.le; decide))

```



```

    (constantName sampleState) := Quotient.exact heq
let eps : Fraction := (3, 32, by decide)
have heps : 0 < eps.num := by decide
obtain (N, hN) := hnames eps heps
have hsmall : Fraction.lt
  (distance (endpoint sampleOne sampleQuarter sampleState N) sampleState)
  eps := hN N (Nat.le_refl N)
have hlower := sample_lower_all_levels N
have hhalf : Fraction.lt eps sampleLower := by unfold Fraction.lt; decide
have hloop := Fraction.magnitudes.lt_of_le_lt hlower hsmall
have hloop' := Fraction.magnitudes.lt_of_lt_le hloop
  (Fraction.magnitudes.lt_implies_le hhalf)
exact (Fraction.magnitudes.lt_irrefl sampleLower) hloop'
end NewtonLimitDynamics.Polygon.CauchyValues

NewtonLimitDynamics/Polygon/PositionValues.lean, 14 theorems, 208 lines

import BarrowLib.Polygon.PositionValues
import NewtonLimitDynamics.Polygon.HarmonicTimeRealization

/-!
Planar positions of the constructed harmonic state values. All magnitudes are
coordinate L1 diagnostics. Closed coordinate squares below are point sets in
this quotient plane, not state-space regions or supplied Euclidean area.
-/

namespace NewtonLimitDynamics.Polygon.PositionValues

open NewtonLimitDynamics
open TimeSubdivision
open PointBounds
open HarmonicComparison
open HarmonicAccumulation
open HarmonicDyadic
open HarmonicBinaryPrefix
open HarmonicTimeRealization
open BinaryTime
open CauchyValues

def gammaPosition (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (t : BinaryTime T hT) : PositionValue :=
  asPosition (gammaValue w T s hT hs t)

theorem gammaPosition_within (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (x y : BinaryTime T hT) (R : Fraction)
  (hR : 0 ≤ R.num) (hxy : TimeWithin T hT x y R) :
  Within (gammaPosition w T s hT hs x).val
    (gammaPosition w T s hT hs y).val
    (Fraction.mul R (stateTimeFactor w s)) :=
  positionValue_within _ _ _
    (gamma_within w T s hT hs x y R hR hxy)

theorem gammaPosition_uniform_continuity (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (eps : Fraction)
  (heps : 0 < eps.num) (x y : BinaryTime T hT)
  (hxy : TimeWithin T hT x y (timeTolerance w s eps)) :
  Within (gammaPosition w T s hT hs x).val
    (gammaPosition w T s hT hs y).val eps.half :=
  positionValue_within _ _ _
    (gamma_uniform_continuity w T s hT hs eps heps x y hxy)

theorem gammaPosition_alias (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaPosition w T s hT hs (Quotient.mk _ firstAlias) =
  gammaPosition w T s hT hs (Quotient.mk _ secondAlias) := by
  exact congrArg (gammaPosition w T s hT hs) (alias_time_eq T hT)

```

```

theorem gammaPosition_left (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaPosition w T s hT hs (leftTime T hT) = embedPosition s.1 := by
  apply Subtype.ext
  change positionValue (gammaValue w T s hT hs (leftTime T hT)) =
    positionValue (embed (s.1, zeroPoint))
  rw [left_endpoint_value w T s hT hs, positionValue_embed]
  rfl

theorem gammaPosition_right (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  gammaPosition w T s hT hs (rightTime T hT) =
    asPosition (endpointValue w T s hT hs) := by
  exact congrArg asPosition (right_endpoint_value w T s hT hs)

theorem gammaPosition_zero_time (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hzero : T.num = 0) :
  gammaPosition w T s hT hs (Quotient.mk _ b) =
    embedPosition s.1 := by
  apply Subtype.ext
  change positionValue (gammaValue w T s hT hs (Quotient.mk _ b)) =
    positionValue (embed (s.1, zeroPoint))
  rw [zero_time_value b w T s hT hs hzero, positionValue_embed]
  rfl

theorem gammaPosition_zero_state_norm (b : Nat → Bool)
  (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hzero : (stateNorm s).num = 0) :
  gammaPosition w T s hT hs (Quotient.mk _ b) =
    embedPosition s.1 := by
  apply Subtype.ext
  change positionValue (gammaValue w T s hT hs (Quotient.mk _ b)) =
    positionValue (embed (s.1, zeroPoint))
  rw [zero_state_norm_value b w T s hT hs hzero, positionValue_embed]
  rfl

private def sampleOne : Fraction := {1, 1, by decide}
private def sampleZero : Fraction := {0, 1, by decide}
private def sampleCentre : Point := (sampleZero, sampleZero)
private def sampleCorner : Point := (sampleOne, sampleOne)
private def sampleRadius : NonnegativeRadius := (sampleOne, by decide)

theorem sample_corner_in_square :
  CoordinateSquare sampleCentre sampleRadius
  (embedPosition sampleCorner) := by
  apply square_of_rational_coordinate_bounds
  · unfold Fraction.le
  · decide
  · unfold Fraction.le
  · decide

theorem sample_corner_L1_distance_two :
  Fraction.equiv (pointNorm (pointSub sampleCorner sampleCentre))
  (Fraction.ofInt 2) := by decide

private def sampleQuarter : Fraction := {1, 4, by decide}
private def sampleState : Point × Point :=
  ((sampleOne, sampleZero), (sampleZero, sampleOne))
private def sampleTail : Fraction := {3, 16, by decide}
private def sampleLower : Fraction := {39, 512, by decide}

theorem sample_position_level_one :
  Fraction.equiv
  (distance (positionState (endpoint sampleOne sampleQuarter sampleState 1))
    (positionState sampleState))
  (135, 512, by decide) := by decide

theorem sample_tail_one :

```

```

    Fraction.equiv (HarmonicDyadic.tailCap
      sampleOne sampleQuarter sampleState 1) sampleTail := by decide

theorem sample_position_lower_all_levels (j : Nat) (hj : 1 ≤ j) :
  Fraction.le sampleLower
    (distance (positionState (endpoint sampleOne sampleQuarter sampleState j))
      (positionState sampleState)) := by
    let e₁ := endpoint sampleOne sampleQuarter sampleState 1
    let eⱼ := endpoint sampleOne sampleQuarter sampleState j
    have hs : DyadicSmallTime sampleOne sampleQuarter := by
      unfold DyadicSmallTime Fraction.le
      decide
    have hj' : 1 + (j - 1) = j := by omega
    have hgap := HarmonicDyadic.finite_gap_error sampleOne sampleQuarter
      sampleState (by decide) hs (j - 1) 1
    rw [hj'] at hgap
    have hgap' : Fraction.le
      (distance (positionState e₁) (positionState eⱼ))
      (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 1) :=
      Fraction.magnitudes.le_trans (position_nonexpansive e₁ eⱼ)
      (Fraction.le_equiv_left (stateSub_norm_symm e₁ eⱼ) hgap)
    have htri := stateSub_triangle
      (positionState e₁) (positionState eⱼ) (positionState sampleState)
    have hchain := Fraction.magnitudes.le_trans htri
      (Fraction.add_le_add_right hgap'
        (distance (positionState eⱼ) (positionState sampleState)))
    have hL : Fraction.equiv (Fraction.add sampleTail sampleLower)
      (distance (positionState e₁) (positionState sampleState)) :=
      Fraction.equiv_trans (by decide)
      (Fraction.equiv_symm sample_position_level_one)
    have hR : Fraction.equiv
      (Fraction.add
        (HarmonicDyadic.tailCap sampleOne sampleQuarter sampleState 1)
        (distance (positionState eⱼ) (positionState sampleState)))
      (Fraction.add sampleTail
        (distance (positionState eⱼ) (positionState sampleState))) :=
      Fraction.add_equiv sample_tail_one (Fraction.equiv_refl _)
    have hfull := Fraction.le_equiv_right
      (Fraction.le_equiv_left hL hchain) hR
    exact le_add_cancel_left sampleTail sampleLower
      (distance (positionState eⱼ) (positionState sampleState)) hfull

theorem sample_endpoint_position_ne_initial :
  positionValue (endpointValue sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)) ≠
  embed (positionState sampleState) := by
  intro heq
  have hnames : NameEquiv
    (mapName positionState position_nonexpansive
      (endpointName sampleOne sampleQuarter sampleState
        (by decide) (by unfold DyadicSmallTime Fraction.le; decide)))
    (constantName (positionState sampleState)) := Quotient.exact heq
  let eps : Fraction := {39, 1024, by decide}
  have heps : 0 < eps.num := by decide
  obtain (N, hN) := hnames eps heps
  have hsmall : Fraction.lt
    (distance
      (positionState (endpoint sampleOne sampleQuarter sampleState (N + 1)))
      (positionState sampleState)) eps := hN (N + 1) (by omega)
  have hlower := sample_position_lower_all_levels (N + 1) (by omega)
  have hhalf : Fraction.lt eps sampleLower := by unfold Fraction.lt; decide
  have hloop := Fraction.magnitudes.lt_of_le_lt hlower hsmall
  have hloop' := Fraction.magnitudes.lt_of_lt_le hloop
    (Fraction.magnitudes.lt_implies_le hhalf)
  exact (Fraction.magnitudes.lt_irrefl sampleLower) hloop'

theorem sample_gammaPosition_right_ne_left :
  gammaPosition sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (rightTime sampleQuarter (by decide)) ≠

```

```

    gammaPosition sampleOne sampleQuarter sampleState
    (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
    (leftTime sampleQuarter (by decide)) := by
intro h
have hneq := sample_endpoint_position_ne_initial
apply hneq
have hright := gammaPosition_right sampleOne sampleQuarter sampleState
  (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
have hleft := gammaPosition_left sampleOne sampleQuarter sampleState
  (by decide) (by unfold DyadicSmallTime Fraction.le; decide)
have hsub := congrArg Subtype.val h
rw [hright, hleft] at hsub
exact hsub

end NewtonLimitDynamics.Polygon.PositionValues

NewtonLimitDynamics/Diagnostic/DeflectionPotential.lean, 6 theorems, 107 lines

import NewtonLimitDynamics.Polygon.ForceClasses
import NewtonLimitDynamics.Polygon.TriangleBounds

/-!
Exact finite deflection-triangle/potential identities. These compare the next
polygon point to its inertial continuation, not a polygon to a completed
curve. The kick at B changes the next position by  $h^2 a(B)$ . This diagnostic
is neither a historical proof nor a universal action-constant argument.
-/

namespace NewtonLimitDynamics.Diagnostic.DeflectionPotential

open NewtonLimitDynamics
open Polygon.TimeSubdivision Polygon.HarmonicStability Polygon.TriangleBounds

def inertial (h : Fraction) (B v : Point) : Point := pointAdd B (pointScale h v)
def deflected (h : Fraction) (B v a : Point) : Point :=
  pointAdd (inertial h B v) (pointScale (Fraction.mul h h) a)

def potentialStep (V : Point → Fraction) (c C : Point) : Fraction :=
  Fraction.add (V C) (negF (V c))

def linearPotential (m g : Fraction) (p : Point) : Fraction :=
  Fraction.mul (Fraction.mul m g) p.2

def quadraticPotential (m w : Fraction) (p : Point) : Fraction :=
  (Fraction.mul (Fraction.mul m w) (dot p p)).half

/-- Signed doubled area; the unsigned triangle uses its absolute value. -/
theorem deflection_triangle (h : Fraction) (B v a : Point) :
  Fraction.equiv (triangleTwice B (inertial h B v) (deflected h B v a))
    (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v a)) := by
  simp only [triangleTwice, inertial, deflected, det, pointSub, pointNeg,
    pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  ac_nf <=> omega

theorem unsigned_deflection_triangle (h : Fraction) (B v a : Point) :
  Fraction.equiv (triangleMagnitude B (inertial h B v) (deflected h B v a))
    (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v a)).abs :=
  Fraction.abs_equiv (deflection_triangle h B v a)

def fallAcceleration (g : Fraction) : Point := (Fraction.ofInt 0, negF g)

/-- The potential at C minus its value at c, exactly for Galilean fall. -/
theorem galilean_potential_step (m g h : Fraction) (B v : Point) :
  Fraction.equiv
    (potentialStep (linearPotential m g) (inertial h B v)
      (deflected h B v (fallAcceleration g)))
    (negF (Fraction.mul (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h))) := by
  simp only [potentialStep, linearPotential, inertial, deflected, fallAcceleration,
    negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,

```

```

    Int.zero_mul, Int.mul_zero, Int.mul_one, Int.one_mul]
ac_nf <;> omega

/-- A division-free exact proportionality. For  $m, g, h, v_x \geq 0$  and  $g > 0$ ,
halving absolute doubled area gives  $\tau = v_x / (2g)$ . -/
theorem galilean_area_potential_cross_relation (m g h : Fraction) (B v : Point) :
  Fraction.equiv
    (Fraction.mul (Fraction.mul m g)
      (triangleTwice B (inertial h B v) (deflected h B v (fallAcceleration g))))
    (Fraction.mul (Fraction.mul h v.1)
      (potentialStep (linearPotential m g) (inertial h B v)
        (deflected h B v (fallAcceleration g)))) := by
  simp only [potentialStep, linearPotential, triangleTwice, inertial, deflected,
    fallAcceleration, det, negF, pointSub, pointNeg, pointAdd, pointScale,
    Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg,
    Int.zero_mul, Int.mul_zero, Int.mul_one, Int.one_mul]
ac_nf <;> omega

/-- Exact harmonic correction terms: the leading potential drop is
 $-m h^2 |a(B)|^2$ , followed by a cubic tangent term and a quartic term. -/
theorem harmonic_potential_step (m w h : Fraction) (B v : Point) :
  Fraction.equiv
    (potentialStep (quadraticPotential m w) (inertial h B v)
      (deflected h B v (linearField w B)))
    (Fraction.add
      (negF (Fraction.mul (Fraction.mul m (Fraction.mul w w))
        (Fraction.mul (Fraction.mul h h) (dot B B))))
      (Fraction.add
        (negF (Fraction.mul (Fraction.mul m (Fraction.mul w w))
          (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (dot B v))))
        (Fraction.mul (Fraction.mul m (Fraction.mul (Fraction.mul w w) w))
          (Fraction.mul (Fraction.mul h h) (Fraction.mul h h)
            (dot B B)).half))) := by
  simp only [potentialStep, quadraticPotential, inertial, deflected, linearField,
    dot, negF, pointAdd, pointScale, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.half, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg]
  simp only [show (2 : Int) = 1 + 1 by rfl, Int.add_mul, Int.mul_add]
ac_nf <;> omega

/-- Dropping the correction terms is false at a finite nonzero step. -/
theorem harmonic_leading_term_not_exact :
  let one := Fraction.ofInt 1
  let zero := Fraction.ofInt 0
  let h : Fraction := (1, 2, by decide)
  let B : Point := (one, zero)
  let v : Point := (zero, one)
  ~ Fraction.equiv
    (potentialStep (quadraticPotential one one) (inertial h B v)
      (deflected h B v (linearField one B)))
    (negF (Fraction.mul h h)) := by
  decide

```

end NewtonLimitDynamics.Diagnostic.DeflectionPotential

NewtonLimitDynamics/Polygon/HarmonicConstructionAgreement.lean, 9 theorems, 113 lines

```

import BarrowLib.Polygon.IntegerRefinement
import NewtonLimitDynamics.Polygon.HarmonicTimeRealization

/-! Agreement of endpoint and global-prefix harmonic names at dyadic times. -/

namespace NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement

open NewtonLimitDynamics
open TimeSubdivision
open HarmonicStability
open HarmonicDyadic
open HarmonicBinaryPrefix
open HarmonicTimeComparison
open HarmonicTimeRealization

```

```

open CauchyValues
open BinaryTime
open PointBounds

private theorem factor_nonnegative (w : Fraction) :
  0 ≤ (Fraction.add (Fraction.ofInt 1) w.abs).num :=
  Fraction.nonnegative_add _ _ (by decide) (Fraction.abs_num_nonnegative w)

/-- A nonnegative dyadic subduration of a short window is short. -/
theorem subduration_small (w T : Fraction) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  DyadicSmallTime w (duration T m) := by
  have h := Fraction.mul_le_mul_nonnegative
    (duration_le_time T m hT) (Fraction.add (Fraction.ofInt 1) w.abs)
    (factor_nonnegative w)
  exact Fraction.magnitudes.le_trans h hs

theorem unit_tail_level (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m j : Nat)
  (htick : ticks b m = 1) (hz : ∀ i, m ≤ i → b i = false) :
  stateEquiv (endpoint w (duration T m) s j)
    (prefixState b w T s (m + j)) := by
  have hcount : ticks b (m + j) = blocks j := by
    rw [ticks_zero_tail b m hz j, htick]
    simp
  have hd := duration_nested T m j
  simpa only [endpoint, prefixState, hcount] using
    schedule_replicate_congr w (duration (duration T m) j)
    (duration T (m + j)) hd (blocks j) s s
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
     (Fraction.equiv_refl _, Fraction.equiv_refl _))

/-- For a one-tick address, the endpoint and prefix Cauchy names agree. -/
theorem unit_tail_names (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (htick : ticks b m = 1) (hz : ∀ i, m ≤ i → b i = false) :
  NameEquiv
    (endpointName w (duration T m) s hT (subduration_small w T m hT hs))
    (prefixName b w T s hT hs) := by
  intro eps heps
  obtain ⟨N, hN⟩ := (prefixName b w T s hT hs).cauchy eps heps
  refine ⟨N, ?_⟩
  intro j hj
  have hshift : N ≤ m + j := by omega
  have he := unit_tail_level b w T s m j htick hz
  have he' : Fraction.equiv
    (distance (endpoint w (duration T m) s j) (prefixState b w T s j))
    (distance (prefixState b w T s (m+j)) (prefixState b w T s j)) :=
    stateNorm_equiv (stateSub_congr he
      ((Fraction.equiv_refl _, Fraction.equiv_refl _),
       (Fraction.equiv_refl _, Fraction.equiv_refl _)))
  exact Fraction.magnitudes.lt_of_le_lt
    ((Fraction.equiv_iff_mutual_le _ _).mp he').1
    (hN (m + j) j hshift hj)

/-- Equality of the two completed state values at reciprocal dyadic times. -/
theorem unit_tail_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (htick : ticks b m = 1) (hz : ∀ i, m ≤ i → b i = false) :
  timeValue w s (duration T m, hT, subduration_small w T m hT hs) =
  gammaValue w T s hT hs (Quotient.mk _ b) := by
  exact Quotient.sound (unit_tail_names b w T s m hT hs htick hz)

theorem full_window_value (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  timeValue w s (T, hT, hs) = gammaValue w T s hT hs (rightTime T hT) :=
  (right_endpoint_value w T s hT hs).symm

```

```

theorem zero_window_value (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (hz : T.num = 0) :
  timeValue w s (T,hT,hs) = gammaValue w T s hT hs (Quotient.mk _ b) := by
  have he : timeValue w s (T,hT,hs) = embed s :=
    Quotient.sound (nameEquiv_of_levelwise_stateEquiv
      (endpointName w T s hT hs) (constantName s)
      (fun j => zero_time_endpoint w T s j hz))
  exact he.trans (zero_time_value b w T s hT hs hz).symm

private def one : Fraction := Fraction.ofInt 1
private def quarter : Fraction := (1,4,by decide)
private def threeSixteenths : Fraction := (3,16,by decide)
private def testState : Point × Point :=
  ((one,Fraction.ofInt 0),(Fraction.ofInt 0,one))
private def threeQuarterAddress (j : Nat) : Bool := j < 2

/-- A finite endpoint schedule at 3T/4 differs from the same-time global
three-cell prefix. Limit agreement cannot be proved by equating these cells. -/
theorem three_tick_finite_control :
  Fraction.equiv
    (distance (endpoint one threeSixteenths testState 0)
      (prefixState threeQuarterAddress one quarter testState 2))
    (426975,16777216,by decide) := by decide

theorem three_tick_finite_identity_false :
  ¬ stateEquiv (endpoint one threeSixteenths testState 0)
    (prefixState threeQuarterAddress one quarter testState 2) := by decide

end NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement

```

NewtonLimitDynamics/Polygon/HarmonicPolygonCurve.lean, 16 theorems, 181 lines

```

import NewtonLimitDynamics.Polygon.PositionValues
import BarrowLib.Polygon.AffineBoundary
import BarrowLib.Polygon.PolygonValues

/-!
Actual within-cell polygon positions and their distance from the constructed
harmonic position value, for every binary address. The affine names use later
times in the same cell. No curve or adjacent-error condition is supplied.
Address independence includes same-cell and shared-boundary aliases.
-/

namespace NewtonLimitDynamics.Polygon.HarmonicPolygonCurve

open NewtonLimitDynamics
open TimeSubdivision PointBounds
open HarmonicDyadic HarmonicBinaryPrefix HarmonicTimeRealization
open BinaryTime CauchyValues PositionValues

def vertices (w T : Fraction) (s : Point × Point) (m : Nat) :
  PolygonValues.VertexChain T m where
  state := countState w T s m
  join := by
    intro k
    change pointEquiv (CentralSchedule.schedule (HarmonicStability.linearField w)
      (List.replicate (k+1) (duration T m)) s).1 _
    rw [schedule_replicate_step]
    exact (Fraction.equiv_refl _, Fraction.equiv_refl _)

def polygonName (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (m : Nat) : EndpointCauchyName :=
  AffineValues.edgeName b T hT (prefixState b w T s m).1
  (prefixState b w T s m).2 m

def polygonPosition (b : Nat → Bool) (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (m : Nat) : PositionValue :=
  asPosition (realize (polygonName b w T s hT m))

def edgeRadius (w T : Fraction) (s : Point × Point) (m : Nat) : Fraction :=
  Fraction.add

```

```

(Fraction.mul (duration T m) (Fraction.mul (Fraction.ofInt 2) (stateNorm s)))
(HarmonicBinaryPrefix.tailCap w T s m)

def edgeCoefficient (w T : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (Fraction.mul T (Fraction.mul (Fraction.ofInt 2) (stateNorm s)))
  (HarmonicBinaryPrefix.coefficient w T s)

theorem edgeCoefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (edgeCoefficient w T s).num :=
  Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _ hT
    (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s)))
  (HarmonicBinaryPrefix.coefficient_nonnegative w T s hT)

theorem edgeRadius_geometric (w T : Fraction) (s : Point × Point) (m : Nat) :
  Fraction.equiv (edgeRadius w T s m) (duration (edgeCoefficient w T s m)) := by
  simp only [edgeRadius, edgeCoefficient, duration, HarmonicBinaryPrefix.tailCap,
    Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

theorem edgeRadius_eventually_small (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → Fraction.lt (edgeRadius w T s m) eps := by
  obtain ⟨N, hN⟩ := duration_eventually_small (edgeCoefficient w T s) eps
  (edgeCoefficient_nonnegative w T s hT) heps
  exact ⟨N, fun m hm => Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (edgeRadius_geometric w T s m)) (hN m hm)⟩

/-- Every approximant lies at a time phase between the cell's boundaries. -/
theorem polygon_phase_interval (b : Nat → Bool) (T : Fraction)
  (hT : 0 ≤ T.num) (m j : Nat) :
  0 ≤ (HarmonicTimeComparison.durationDifference (timeApprox b T m)
    (timeApprox b T (m+j))).num ∧
  Fraction.le (HarmonicTimeComparison.durationDifference (timeApprox b T m)
    (timeApprox b T (m+j))) (duration T m) :=
  AffineValues.phase_interval b T hT m j

theorem polygon_vertex_bound (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (m : Nat) :
  Within (polygonPosition b w T s hT m).val
    (positionValue (embed (prefixState b w T s m)))
    (Fraction.mul (duration T m) (Fraction.mul (Fraction.ofInt 2) (stateNorm s))) := by
  have hv := Fraction.magnitudes.le_trans (velocity_le_state (prefixState b w T s m))
    (prefix_state_le_two b w T s m hT hs)
  exact PolygonValues.polygon_vertex_bound b T hT m (vertices w T s m) _ hv

/-- Whole-cell polygon/curve bound, including every interior binary time.
Its radius is an explicit sum of two geometric mesh terms. -/
theorem whole_edge_bound (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (m : Nat) :
  Within (polygonPosition b w T s hT m).val
    (gammaPosition w T s hT hs (Quotient.mk _ b)).val
    (edgeRadius w T s m) := by
  have hv := polygon_vertex_bound b w T s hT hs m
  have hg := positionValue_within _ _ _
    (binaryValue_prefix_bound b w T s hT hs m)
  exact within_triangle _ _ _ _ hv hg

theorem uniform_whole_edge_convergence (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (polygonPosition b w T s hT m).val
      (gammaPosition w T s hT hs (Quotient.mk _ b)).val eps := by
  obtain ⟨N, hN⟩ := edgeRadius_eventually_small w T s hT eps heps
  exact ⟨N, fun m hm b => within_mono _ _ _
    (Fraction.magnitudes.lt_implies_le (hN m hm))
    (whole_edge_bound b w T s hT hs m)⟩

```



```

theorem polygon_same_cell_address_independent (b c : Nat → Bool)
  (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat)
  (hcell : ticks b m = ticks c m)
  (htime : AddressEquiv T hT b c) :
  polygonPosition b w T s hT m = polygonPosition c w T s hT m := by
  exact PolygonValues.polygon_same_cell b c T hT m (vertices w T s m) hcell htime

theorem polygon_adjacent_address_independent (b c : Nat → Bool)
  (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat)
  (hcell : ticks b m + 1 = ticks c m)
  (htime : AddressEquiv T hT b c) :
  polygonPosition b w T s hT m = polygonPosition c w T s hT m := by
  exact PolygonValues.polygon_adjacent_cells b c T hT m (vertices w T s m) hcell htime

theorem polygon_zero_window (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat) (hz : T.num = 0) :
  polygonPosition b w T s hT m =
  asPosition (embed (s.1, AffineValues.zeroPoint)) := by
  exact PolygonValues.polygon_zero_window b T hT m (vertices w T s m) hz

theorem polygon_address_independent (b c : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (m : Nat)
  (htime : AddressEquiv T hT b c) :
  polygonPosition b w T s hT m = polygonPosition c w T s hT m := by
  exact PolygonValues.polygon_address_independent b c T hT m (vertices w T s m) htime

/-- The actual coarse polygon, lifted to the same constructed time quotient
as the trajectory after proving both kinds of address invariance. -/
def polygonMap (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (m : Nat) : BinaryTime T hT → PositionValue :=
  Quotient.lift (fun b => polygonPosition b w T s hT m)
  (fun b c h => polygon_address_independent b c w T s hT m h)

theorem polygonMap_whole_edge_bound (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (t : BinaryTime T hT) :
  Within (polygonMap w T s hT m t).val (gammaPosition w T s hT hs t).val
  (edgeRadius w T s m) := by
  induction t using Quotient.inductionOn with
  | _ b => exact whole_edge_bound b w T s hT hs m

theorem polygonMap_uniform_convergence (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ t : BinaryTime T hT,
    Within (polygonMap w T s hT m t).val
    (gammaPosition w T s hT hs t).val eps := by
  obtain ⟨N, hN⟩ := uniform_whole_edge_convergence w T s hT hs eps heps
  refine ⟨N, ?_⟩
  intro m hm t
  induction t using Quotient.inductionOn with
  | _ b => exact hN m hm b

/-- The standard terminating/nonterminating half-time addresses occupy
different coarse cells and nevertheless give the same polygon point. -/
theorem half_time_polygon_alias (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (m : Nat) :
  polygonPosition firstAlias w T s hT m =
  polygonPosition secondAlias w T s hT m :=
  polygon_address_independent firstAlias secondAlias w T s hT m
  (alias_address_equiv T hT)

theorem half_time_distinct_coarse_cells :
  ticks firstAlias 1 = 1 ∧ ticks secondAlias 1 = 0 := by decide

theorem polygonMap_left (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (m : Nat) :
  polygonMap w T s hT m (leftTime T hT) = embedPosition s.1 := by
  exact PolygonValues.polygonMap_left T hT m (vertices w T s m)

```

```
end NewtonLimitDynamics.Polygon.HarmonicPolygonCurve
```

NewtonLimitDynamics/Polygon/HarmonicIntegerSubdivision.lean, 37 theorems, 854 lines

```
import BarrowLib.Polygon.BoundedIteration
import BarrowLib.Polygon.IntegerSchedule
import BarrowLib.Polygon.FiniteRecurrence
import NewtonLimitDynamics.Polygon.HarmonicTimeRealization

/-- Finite unequal subdivision identities for the actual harmonic end-kick cell. -/

namespace NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision

open NewtonLimitDynamics
open TimeSubdivision
open CentralSchedule
open HarmonicStability
open HarmonicDyadic
open HarmonicBinaryPrefix
open HarmonicComparison
open PointBounds
open HarmonicAccumulation
open HarmonicUniform
open IntegerSchedule
open FiniteRecurrence

def integerFine (w h : Fraction) (s : Point × Point) (k : Nat) : Point × Point :=
  schedule (linearField w) (List.replicate k h) s

def integerCoarse (w h : Fraction) (s : Point × Point) (k : Nat) : Point × Point :=
  cell (linearField w) (integerDuration h k) s

/-- The position defect caused by replacing one `(a+b)` cell with an `a`
cell followed by a `b` cell. The sampled force is at the actual first arrival. -/
theorem split_position_identity (w a b : Fraction) (s : Point × Point) :
  pointEquiv
    (stateSub (cell (linearField w) b (cell (linearField w) a s))
      (cell (linearField w) (Fraction.add a b) s)).1
    (pointScale (Fraction.mul a b)
      (linearField w (cell (linearField w) a s).1)) := by
  constructor <=>
  simp only [stateSub, cell, linearField, negF, pointEquiv, pointSub,
    pointNeg, pointAdd, pointScale, Fraction.equiv, Fraction.add,
    Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- The velocity defect contains a sampling shift of the initial velocity
and the second kick applied to the position defect. -/
theorem split_velocity_identity (w a b : Fraction) (s : Point × Point) :
  pointEquiv
    (stateSub (cell (linearField w) b (cell (linearField w) a s))
      (cell (linearField w) (Fraction.add a b) s)).2
    (pointAdd
      (pointScale (negF (Fraction.mul a b)) (linearField w s.2))
      (pointScale (Fraction.mul b (Fraction.mul a b))
        (linearField w (linearField w (cell (linearField w) a s).1)))) := by
  constructor <=>
  simp only [stateSub, cell, linearField, negF, pointEquiv, pointSub,
    pointNeg, pointAdd, pointScale, Fraction.equiv, Fraction.add,
    Fraction.mul, Int.add_mul, Int.mul_add, Int.neg_mul, Int.mul_neg] <=>
  ac_nf <=> omega

/-- A local state bound with every force sample still at its actual point.
For `a=i*h` and `b=h`, all three coefficients contain two step factors. -/
theorem split_state_sample_bound (w a b : Fraction) (s : Point × Point) :
  Fraction.le
    (stateNorm (stateSub
      (cell (linearField w) b (cell (linearField w) a s))
      (cell (linearField w) (Fraction.add a b) s)))
    (Fraction.add
```

```

(Fraction.mul (Fraction.mul a b).abs
 (pointNorm (linearField w (cell (linearField w) a s).1)))
(Fraction.add
 (Fraction.mul (Fraction.mul a b).abs
 (pointNorm (linearField w s.2)))
 (Fraction.mul (Fraction.mul b (Fraction.mul a b)).abs
 (pointNorm
 (linearField w
 (linearField w (cell (linearField w) a s).1)))))) := by
have hp := pointNorm_equiv (split_position_identity w a b s)
have hv := pointNorm_equiv (split_velocity_identity w a b s)
have htri := pointNorm_add_le
 (pointScale (negF (Fraction.mul a b)) (linearField w s.2))
 (pointScale (Fraction.mul b (Fraction.mul a b))
 (linearField w (linearField w (cell (linearField w) a s).1)))
have hraw := Fraction.add_le_add_left
 (Fraction.le_equiv_left hv htri)
 (pointNorm (stateSub
 (cell (linearField w) b (cell (linearField w) a s))
 (cell (linearField w) (Fraction.add a b s)).1)
have hepos := Fraction.equiv_trans hp
 (pointNorm_scale (Fraction.mul a b)
 (linearField w (cell (linearField w) a s).1))
have hevel1 := Fraction.equiv_trans
 (pointNorm_scale (negF (Fraction.mul a b)) (linearField w s.2))
 (Fraction.mul_equiv
 (Fraction.abs_neg (Fraction.mul a b)) (Fraction.equiv_refl _))
have hevel2 := pointNorm_scale (Fraction.mul b (Fraction.mul a b))
 (linearField w (linearField w (cell (linearField w) a s).1))
change Fraction.le
 (Fraction.add
 (pointNorm (stateSub
 (cell (linearField w) b (cell (linearField w) a s))
 (cell (linearField w) (Fraction.add a b s)).1)
 (pointNorm (stateSub
 (cell (linearField w) b (cell (linearField w) a s))
 (cell (linearField w) (Fraction.add a b s)).2)) _
exact Fraction.le_equiv_right hraw
 (Fraction.add_equiv hepos (Fraction.add_equiv hevel1 hevel2))

theorem linearField_norm (w : Fraction) (p : Point) :
  Fraction.equiv (pointNorm (linearField w p))
  (Fraction.mul w.abs (pointNorm p)) := by
exact Fraction.equiv_trans (pointNorm_scale (negF w) p)
 (Fraction.mul_equiv (Fraction.abs_neg w) (Fraction.equiv_refl _))

/-- The local defect uses only the initial velocity and the actual first
arrival. Every term contains `a*b`; the last also contains `b`. -/
theorem split_state_harmonic_bound (w a b : Fraction) (s : Point × Point) :
  Fraction.le
  (stateNorm (stateSub
  (cell (linearField w) b (cell (linearField w) a s))
  (cell (linearField w) (Fraction.add a b s))))
  (Fraction.add
  (Fraction.mul (Fraction.mul a b).abs
  (Fraction.mul w.abs
  (pointNorm (cell (linearField w) a s).1)))
  (Fraction.add
  (Fraction.mul (Fraction.mul a b).abs
  (Fraction.mul w.abs (pointNorm s.2)))
  (Fraction.mul (Fraction.mul b (Fraction.mul a b)).abs
  (Fraction.mul w.abs
  (Fraction.mul w.abs
  (pointNorm (cell (linearField w) a s).1)))))) := by
have h := split_state_sample_bound w a b s
have hp := linearField_norm w (cell (linearField w) a s).1
have hv := linearField_norm w s.2
have hpp := Fraction.equiv_trans
 (linearField_norm w (linearField w (cell (linearField w) a s).1))
 (Fraction.mul_equiv (Fraction.equiv_refl _ ) hp)

```

```

exact Fraction.le_equiv_right h
  (Fraction.add_equiv
    (Fraction.mul_equiv (Fraction.equiv_refl _) hp)
    (Fraction.add_equiv
      (Fraction.mul_equiv (Fraction.equiv_refl _) hv)
      (Fraction.mul_equiv (Fraction.equiv_refl _) hpp)))

def uniformSplitBudget (w a b M : Fraction) : Fraction :=
  Fraction.add
    (Fraction.mul (Fraction.mul a b).abs (Fraction.mul w.abs M))
    (Fraction.add
      (Fraction.mul (Fraction.mul a b).abs (Fraction.mul w.abs M))
      (Fraction.mul (Fraction.mul a b).abs
        (Fraction.mul w.abs (Fraction.mul w.abs M))))

/-- If the first arrival and initial velocity lie in one rational L1 ball,
the unequal split has a uniform quadratic-step source. -/
theorem split_state_uniform_bound (w a b M : Fraction) (s : Point × Point)
  (hb : Fraction.le b.abs (Fraction.ofInt 1))
  (hP : Fraction.le (pointNorm (cell (linearField w) a s).1) M)
  (hV : Fraction.le (pointNorm s.2) M) :
  Fraction.le
    (stateNorm (stateSub
      (cell (linearField w) b (cell (linearField w) a s))
      (cell (linearField w) (Fraction.add a b) s)))
    (uniformSplitBudget w a b M) := by
  have hp₁ := Fraction.mul_le_mul_nonnegative_left hP w.abs
  (Fraction.abs_num_nonnegative w)
  have hp := Fraction.mul_le_mul_nonnegative_left hp₁
  (Fraction.mul a b).abs (Fraction.abs_num_nonnegative _)
  have hv₁ := Fraction.mul_le_mul_nonnegative_left hV w.abs
  (Fraction.abs_num_nonnegative w)
  have hv := Fraction.mul_le_mul_nonnegative_left hv₁
  (Fraction.mul a b).abs (Fraction.abs_num_nonnegative _)
  have hpp₁ := Fraction.mul_le_mul_nonnegative_left hp₁ w.abs
  (Fraction.abs_num_nonnegative w)
  have hM : 0 ≤ M.num := by
    have hPnum := pointNorm_nonnegative (cell (linearField w) a s).1
    have hden := (pointNorm (cell (linearField w) a s).1).den_pos
    have hMden := M.den_pos
    unfold Fraction.le at hP
    by_cases hn : 0 ≤ M.num
    · exact hn
    have hnegative : M.num * (pointNorm (cell (linearField w) a s).1).den < 0 :=
      Int.mul_neg_of_neg_of_pos (by omega) hden
    have hpositive : 0 ≤
      (pointNorm (cell (linearField w) a s).1).num * M.den :=
      Int.mul_nonneg hPnum (Int.le_of_lt hMden)
    omega
  have hcoef : Fraction.le (Fraction.mul b (Fraction.mul a b)).abs
    (Fraction.mul a b).abs := by
    have hm := Fraction.mul_le_mul_nonnegative hb (Fraction.mul a b).abs
    (Fraction.abs_num_nonnegative _)
    exact Fraction.le_equiv_left
      (Fraction.abs_mul b (Fraction.mul a b))
      (Fraction.le_equiv_right hm (by
        simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
        simp))
  have hcoef₂ := Fraction.mul_le_mul_nonnegative hcoef
    (Fraction.mul w.abs (Fraction.mul w.abs M))
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
        hm))
  have hpp := Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative_left hpp₁
      (Fraction.mul b (Fraction.mul a b)).abs
      (Fraction.abs_num_nonnegative _)) hcoef₂
  exact Fraction.magnitudes.le_trans
    (split_state_harmonic_bound w a b s)
    (Fraction.add_le_add hp (Fraction.add_le_add hv hpp))

```

```

theorem first_arrival_le_state (w a : Fraction) (s : Point × Point)
  (ha : Fraction.le a.abs (Fraction.ofInt 1)) :
  Fraction.le (pointNorm (cell (linearField w) a s).1) (stateNorm s) := by
  have hp := FiniteEstimates.cell_position_growth (linearField w) a s
  have hm := Fraction.mul_le_mul_nonnegative ha (pointNorm s.2)
  (pointNorm_nonnegative s.2)
  have hunit : Fraction.equiv
    (Fraction.mul (Fraction.ofInt 1) (pointNorm s.2))
    (pointNorm s.2) := by
  simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
  have hv := Fraction.le_equiv_right hm hunit
  have hsum := Fraction.add_le_add_left hv (pointNorm s.1)
  exact Fraction.magnitudes.le_trans hp hsum

/-- An unequal split whose two durations have absolute value at most one
has a defect bounded by its product duration and the initial state ball. -/
theorem split_state_short_bound (w a b : Fraction) (s : Point × Point)
  (ha : Fraction.le a.abs (Fraction.ofInt 1))
  (hb : Fraction.le b.abs (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub
      (cell (linearField w) b (cell (linearField w) a s))
      (cell (linearField w) (Fraction.add a b) s)))
    (uniformSplitBudget w a b (stateNorm s)) :=
  split_state_uniform_bound w a b (stateNorm s) s hb
  (first_arrival_le_state w a s ha) (velocity_le_state s)

def splitFactor (w : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (Fraction.mul w.abs (stateNorm s))
  (Fraction.add (Fraction.mul w.abs (stateNorm s))
    (Fraction.mul w.abs (Fraction.mul w.abs (stateNorm s))))

theorem uniformSplitBudget_factor (w a b : Fraction) (s : Point × Point) :
  Fraction.equiv (uniformSplitBudget w a b (stateNorm s))
  (Fraction.mul (Fraction.mul a b).abs (splitFactor w s)) := by
  simp only [uniformSplitBudget, splitFactor, Fraction.equiv,
    Fraction.add, Fraction.mul]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem splitFactor_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (splitFactor w s).num := by
  unfold splitFactor
  apply Fraction.nonnegative_add
  · exact Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
    (stateNorm_nonnegative s)
  · apply Fraction.nonnegative_add
    · exact Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
      (stateNorm_nonnegative s)
    · exact Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w)
        (stateNorm_nonnegative s))

theorem integer_duration_product_abs (h : Fraction) (k : Nat)
  (hh : 0 ≤ h.num) :
  Fraction.equiv (Fraction.mul (IntegerSchedule.integerDuration h k) h).abs
  (Fraction.mul (Fraction.ofInt (k : Int)) (Fraction.mul h h)) := by
  have h₁ := Fraction.abs_equiv
    (Fraction.mul_equiv (IntegerSchedule.integerDuration_closed h k)
      (Fraction.equiv_refl h))
  have h₂ := Fraction.abs_mul
    (Fraction.mul (Fraction.ofInt (k : Int)) h) h
  have h₃ := Fraction.mul_equiv
    (Fraction.abs_mul (Fraction.ofInt (k : Int)) h)
    (Fraction.abs_of_nonnegative h hh)
  have h₄ := Fraction.mul_equiv
    (Fraction.mul_equiv
      (Fraction.abs_of_nonnegative (Fraction.ofInt (k : Int)) (Int.ofNat_nonneg k))

```

```

    (Fraction.abs_of_nonnegative h hh)) (Fraction.equiv_refl h)
exact Fraction.equiv_trans h1
    (Fraction.equiv_trans h2 (Fraction.equiv_trans h3
    (Fraction.equiv_trans h4 (Fraction.mul_assoc _ _ _))))

def kappaCap (w : Fraction) : Fraction :=
  Fraction.mul (Fraction.ofInt 2)
    (Fraction.add (Fraction.ofInt 1) w.abs)

theorem kappaCap_nonnegative (w : Fraction) : 0 ≤ (kappaCap w).num :=
  Fraction.nonnegative_mul _ _ (by decide)
    (Fraction.nonnegative_add _ _ (by decide)
    (Fraction.abs_num_nonnegative w))

theorem kappa_le_cap (w h : Fraction)
  (hh : Fraction.le h.abs (Fraction.ofInt 1)) :
  Fraction.le (kappa w h) (kappaCap w) := by
  have h2 : Fraction.le
    (Fraction.add (Fraction.ofInt 1) h.abs) (Fraction.ofInt 2) := by
    exact Fraction.le_equiv_right
      (Fraction.add_le_add_left hh (Fraction.ofInt 1)) (by decide)
  have hw1 := Fraction.mul_le_mul_nonnegative hh w.abs
    (Fraction.abs_num_nonnegative w)
  have hw2 := Fraction.add_le_add_left hw1 (Fraction.ofInt 1)
  have hnon : 0 ≤
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)).num :=
    Fraction.nonnegative_add _ _ (by decide)
      (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative h)
      (Fraction.abs_num_nonnegative w))
  have hm1 := Fraction.mul_le_mul_nonnegative h2
    (Fraction.add (Fraction.ofInt 1) (Fraction.mul h.abs w.abs)) hnon
  have hm2 := Fraction.mul_le_mul_nonnegative_left hw2 (Fraction.ofInt 2)
    (by decide)
  exact Fraction.magnitudes.le_trans hm1
    (Fraction.le_equiv_right hm2 (by
      simp only [kappaCap, Fraction.equiv, Fraction.add, Fraction.mul,
      Fraction.ofInt]
      simp only [Int.one_mul, Int.mul_one]))

def integerLocalBudget (w h : Fraction) (s : Point × Point) (k : Nat) : Fraction :=
  Fraction.add
    (Fraction.mul (Fraction.mul (integerDuration h k) h).abs
    (Fraction.mul w.abs (pointNorm (integerCoarse w h s k).1)))
    (Fraction.add
      (Fraction.mul (Fraction.mul (integerDuration h k) h).abs
      (Fraction.mul w.abs (pointNorm s.2)))
      (Fraction.mul (Fraction.mul h (Fraction.mul (integerDuration h k) h)).abs
      (Fraction.mul w.abs
      (Fraction.mul w.abs (pointNorm (integerCoarse w h s k).1)))))

/-- Actual `k`-cell subdivision recurrence. The source is the measured
unequal-cell defect at the coarse first arrival, not a supplied Cauchy bound. -/
theorem integer_error_step (w h : Fraction) (s : Point × Point) (k : Nat) :
  Fraction.le
    (stateNorm (stateSub
      (integerFine w h s (k + 1)) (integerCoarse w h s (k + 1))))
    (Fraction.add
      (Fraction.mul (kappa w h)
      (stateNorm (stateSub
        (integerFine w h s k) (integerCoarse w h s k))))
      (integerLocalBudget w h s k)) := by
  have htri := stateSub_triangle
    (cell (linearField w) h (integerFine w h s k))
    (cell (linearField w) h (integerCoarse w h s k))
    (integerCoarse w h s (k + 1))
  have hp := cell_perturbation w h (integerFine w h s k)
    (integerCoarse w h s k)
  have hl := split_state_harmonic_bound w (integerDuration h k) h s
  have hsum := Fraction.add_le_add hp hl
  have hbound := Fraction.magnitudes.le_trans htri hsum

```

```

simpa only [integerFine, integerCoarse, integerDuration,
  HarmonicBinaryPrefix.schedule_replicate_step, integerLocalBudget] using hbound

theorem integer_error_step_quadratic (w h : Fraction)
  (s : Point × Point) (k : Nat)
  (hh : 0 ≤ h.num)
  (ha : Fraction.le (integerDuration h k).abs (Fraction.ofInt 1))
  (hb : Fraction.le h.abs (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub
      (integerFine w h s (k + 1)) (integerCoarse w h s (k + 1))))
    (Fraction.add
      (Fraction.mul (kappa w h)
        (stateNorm (stateSub
          (integerFine w h s k) (integerCoarse w h s k))))
      (Fraction.mul
        (Fraction.mul (Fraction.ofInt (k : Int)) (Fraction.mul h h))
        (splitFactor w s))) := by
have htri := stateSub_triangle
  (cell (linearField w) h (integerFine w h s k))
  (cell (linearField w) h (integerCoarse w h s k))
  (integerCoarse w h s (k + 1))
have hp := cell_perturbation w h (integerFine w h s k)
  (integerCoarse w h s k)
have hl := split_state_short_bound w (integerDuration h k) h s ha hb
have he := Fraction.equiv_trans
  (uniformSplitBudget_factor w (integerDuration h k) h s)
  (Fraction.mul_equiv (integer_duration_product_abs h k hh)
    (Fraction.equiv_refl _))
have hlocal := Fraction.le_equiv_right hl he
have hsum := Fraction.add_le_add hp hlocal
have hbound := Fraction.magnitudes.le_trans htri hsum
simpa only [integerFine, integerCoarse, integerDuration,
  HarmonicBinaryPrefix.schedule_replicate_step] using hbound

def integerErrorBudget (w h : Fraction) (s : Point × Point) : Nat → Fraction
| 0 => Fraction.ofInt 0
| k + 1 => Fraction.add
  (Fraction.mul (kappa w h) (integerErrorBudget w h s k))
  (integerLocalBudget w h s k)

/-- Every finite integer subdivision is controlled by the recursively
computed actual local sources and one-cell amplification. -/
theorem integer_error_le_budget (w h : Fraction) (s : Point × Point) :
  (k : Nat) → Fraction.le
    (stateNorm (stateSub (integerFine w h s k) (integerCoarse w h s k)))
    (integerErrorBudget w h s k)
| 0 => by
  have he := zero_step w s
  have he' : Fraction.equiv
    (stateNorm (stateSub (cell (linearField w) (Fraction.ofInt 0) s) s))
    (stateNorm (stateSub s s)) :=
    stateNorm_equiv (stateSub_congr he
      (⟨Fraction.equiv_refl _, Fraction.equiv_refl _⟩,
        ⟨Fraction.equiv_refl _, Fraction.equiv_refl _⟩))
  have hz := Fraction.equiv_trans
    (stateSub_norm_symm s (cell (linearField w) (Fraction.ofInt 0) s))
    (Fraction.equiv_trans he' (stateSub_self_norm_zero s))
  simpa only [integerFine, integerCoarse, integerDuration,
    integerErrorBudget, List.replicate_zero, schedule] using
    Fraction.le_of_equiv hz
| k + 1 => by
  have hi := integer_error_le_budget w h s k
  have hm := Fraction.mul_le_mul_nonnegative_left hi (kappa w h)
    (kappa_nonnegative w h)
  have hs := Fraction.add_le_add_right hm (integerLocalBudget w h s k)
  exact Fraction.magnitudes.le_trans (integer_error_step w h s k) hs

def quadraticCap (w : Fraction) (s : Point × Point) : Nat → Fraction
| 0 => Fraction.ofInt 0

```

```

| k + 1 => Fraction.add
  (Fraction.mul (kappaCap w) (quadraticCap w s k))
  (Fraction.mul (Fraction.ofInt (k : Int)) (splitFactor w s))

theorem quadraticCap_nonnegative (w : Fraction) (s : Point × Point) :
  (k : Nat) → 0 ≤ (quadraticCap w s k).num
| 0 => by simp [quadraticCap, Fraction.ofInt]
| k + 1 => Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _ (kappaCap_nonnegative w)
    (quadraticCap_nonnegative w s k))
  (Fraction.nonnegative_mul _ _ (Int.ofNat_nonneg k)
    (splitFactor_nonnegative w s))

/-- For each fixed integer `k`, the discrepancy between one `k*h` cell
and `k` actual `h` cells is at most `h^2` times a coefficient independent
of `h`. The stated short-prefix condition is finite and will be discharged
from the global dyadic window before passing to Cauchy values. -/
theorem integer_error_quadratic (w h : Fraction) (s : Point × Point)
  (hh : 0 ≤ h.num) (hb : Fraction.le h.abs (Fraction.ofInt 1)) :
  (k : Nat) →
    (∀ i, i ≤ k → Fraction.le (integerDuration h i).abs (Fraction.ofInt 1)) →
    Fraction.le
      (stateNorm (stateSub (integerFine w h s k) (integerCoarse w h s k)))
      (Fraction.mul (Fraction.mul h h) (quadraticCap w s k))
| 0, _ => by
  have h₀ := integer_error_le_budget w h s 0
  apply Fraction.le_equiv_right h₀
  simp only [integerErrorBudget, quadraticCap, Fraction.equiv,
    Fraction.mul, Fraction.ofInt]
  simp
| k + 1, hshort => by
  have hprev := integer_error_quadratic w h s hh hb k
  (fun i hi => hshort i (by omega))
  have hstep := integer_error_step_quadratic w h s k hh
    (hshort k (by omega)) hb
  have hkappa := kappa_le_cap w h hb
  have h₂ : 0 ≤ (Fraction.mul h h).num :=
    Fraction.nonnegative_mul _ _ hh hh
  have hC : 0 ≤
    (Fraction.mul (Fraction.mul h h) (quadraticCap w s k)).num :=
    Fraction.nonnegative_mul _ _ h₂ (quadraticCap_nonnegative w s k)
  have h₁ := Fraction.mul_le_mul_nonnegative_left hprev (kappa w h)
    (kappa_nonnegative w h)
  have h₂' := Fraction.mul_le_mul_nonnegative hkappa
    (Fraction.mul (Fraction.mul h h) (quadraticCap w s k)) hC
  have h₃ := Fraction.magnitudes.le_trans h₁ h₂'
  have hsum := Fraction.add_le_add_right h₃
    (Fraction.mul
      (Fraction.mul (Fraction.ofInt (k : Int)) (Fraction.mul h h))
      (splitFactor w s))
  have hbound := Fraction.magnitudes.le_trans hstep hsum
  apply Fraction.le_equiv_right hbound
  simp only [quadraticCap, Fraction.equiv, Fraction.mul,
    Fraction.add, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

/-- Lipschitz control of an actual block of `k` end-kick cells. -/
theorem integerFine_perturbation (w h : Fraction)
  (s t : Point × Point) :
  (k : Nat) → Fraction.le
    (stateNorm (stateSub (integerFine w h s k) (integerFine w h t k)))
    (Fraction.mul (fpower (kappa w h) k) (stateNorm (stateSub s t)))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [integerFine, List.replicate_zero, schedule,
    fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
| k + 1 => by
  have hp := cell_perturbation w h (integerFine w h s k)

```



```

    (integerFine w h t k)
    have hi := integerFine_perturbation w h s t k
    have hm := Fraction.mul_le_mul_nonnegative_left hi (kappa w h)
    (kappa_nonnegative w h)
    have hchain := Fraction.magnitudes.le_trans hp hm
    have hfinal := Fraction.le_equiv_right hchain
    (Fraction.equiv_symm (Fraction.mul_assoc
      (kappa w h) (fpower (kappa w h) k)
      (stateNorm (stateSub s t))))
    simp only [integerFine, HarmonicBinaryPrefix.schedule_replicate_step,
      fpower] using hfinal

def fineBlocks (w h : Fraction) (k : Nat) (s : Point × Point) :
  Nat → Point × Point
| 0 => s
| N + 1 => integerFine w h (fineBlocks w h k s N) k

def coarseBlocks (w h : Fraction) (k : Nat) (s : Point × Point) :
  Nat → Point × Point
| 0 => s
| N + 1 => integerCoarse w h (coarseBlocks w h k s N) k

theorem block_error_step (w h : Fraction) (k : Nat)
  (s : Point × Point) (N : Nat)
  (hh : 0 ≤ h.num) (hb : Fraction.le h.abs (Fraction.ofInt 1))
  (hshort : ∀ i, i ≤ k →
    Fraction.le (integerDuration h i).abs (Fraction.ofInt 1)) :
  Fraction.le
    (stateNorm (stateSub
      (fineBlocks w h k s (N + 1)) (coarseBlocks w h k s (N + 1))))
    (Fraction.add
      (Fraction.mul (fpower (kappa w h) k)
        (stateNorm (stateSub
          (fineBlocks w h k s N) (coarseBlocks w h k s N))))
      (Fraction.mul (Fraction.mul h h)
        (quadraticCap w (coarseBlocks w h k s N) k))) := by
  let f := fineBlocks w h k s N
  let c := coarseBlocks w h k s N
  have htri := stateSub_triangle
    (integerFine w h f k) (integerFine w h c k)
    (integerCoarse w h c k)
  have hp := integerFine_perturbation w h f c k
  have hl := integer_error_quadratic w h c hh hb k hshort
  have hbound := Fraction.magnitudes.le_trans htri
    (Fraction.add_le_add hp hl)
  simp only [fineBlocks, coarseBlocks] using hbound

theorem splitFactor_le (w : Fraction) (s t : Point × Point)
  (hst : Fraction.le (stateNorm s) (stateNorm t)) :
  Fraction.le (splitFactor w s) (splitFactor w t) := by
  have h1 := Fraction.mul_le_mul_nonnegative_left hst w.abs
    (Fraction.abs_num_nonnegative w)
  have h2 := Fraction.mul_le_mul_nonnegative_left h1 w.abs
    (Fraction.abs_num_nonnegative w)
  exact Fraction.add_le_add h1 (Fraction.add_le_add h1 h2)

theorem splitFactor_double (w : Fraction) (s t : Point × Point)
  (hst : Fraction.le (stateNorm s)
    (Fraction.mul (Fraction.ofInt 2) (stateNorm t))) :
  Fraction.le (splitFactor w s)
    (Fraction.mul (Fraction.ofInt 2) (splitFactor w t)) := by
  have h1 := Fraction.mul_le_mul_nonnegative_left hst w.abs
    (Fraction.abs_num_nonnegative w)
  have h2 := Fraction.mul_le_mul_nonnegative_left h1 w.abs
    (Fraction.abs_num_nonnegative w)
  have hsum := Fraction.add_le_add h1 (Fraction.add_le_add h1 h2)
  apply Fraction.le_equiv_right hsum
  simp only [splitFactor, Fraction.equiv, Fraction.add, Fraction.mul,
    Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add]

```

```

ac_nf

theorem quadraticCap_double (w : Fraction) (s t : Point × Point)
  (hst : Fraction.le (stateNorm s)
    (Fraction.mul (Fraction.ofInt 2) (stateNorm t))) :
  (k : Nat) → Fraction.le (quadraticCap w s k)
    (Fraction.mul (Fraction.ofInt 2) (quadraticCap w t k))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [quadraticCap, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  simp
| k + 1 => by
  have hi := quadraticCap_double w s t hst k
  have hs := splitFactor_double w s t hst
  have h1 := Fraction.mul_le_mul_nonnegative_left hi (kappaCap w)
    (kappaCap_nonnegative w)
  have h2 := Fraction.mul_le_mul_nonnegative_left hs
    (Fraction.ofInt (k : Int)) (Int.ofNat_nonneg k)
  have hsum := Fraction.add_le_add h1 h2
  apply Fraction.le_equiv_right hsum
  simp only [quadraticCap, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt]
  simp only [Int.add_mul, Int.mul_add]
  ac_nf

theorem coarseBlocks_norm_bound (w h : Fraction) (k : Nat)
  (s : Point × Point) :
  (N : Nat) → Fraction.le (stateNorm (coarseBlocks w h k s N))
    (Fraction.mul (fpower (kappa w (integerDuration h k)) N)
      (stateNorm s))
| 0 => by
  apply Fraction.le_of_equiv
  simp only [coarseBlocks, fpower, Fraction.equiv,
    Fraction.mul, Fraction.ofInt]
  simp
| N + 1 => by
  have hc := cell_bound w (integerDuration h k)
    (coarseBlocks w h k s N)
  have hi := coarseBlocks_norm_bound w h k s N
  have hm := Fraction.mul_le_mul_nonnegative_left hi
    (kappa w (integerDuration h k))
    (kappa_nonnegative w (integerDuration h k))
  have hbound := Fraction.magnitudes.le_trans hc hm
  apply Fraction.le_equiv_right hbound
  simp only [coarseBlocks, integerCoarse, fpower, Fraction.equiv,
    Fraction.mul]
  ac_nf

theorem coarseBlocks_state_le_two (w h : Fraction) (k : Nat)
  (s : Point × Point) (N : Nat)
  (hpower : Fraction.le
    (fpower (kappa w (integerDuration h k)) N) (Fraction.ofInt 2)) :
  (i : Nat) → i ≤ N → Fraction.le
    (stateNorm (coarseBlocks w h k s i))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  intro i hi
  let r := kappa w (integerDuration h k)
  have hpref := fpower_prefix_le r (kappa_nonnegative w _) (one_le_kappa w _)
    i N hi
  have hp := Fraction.magnitudes.le_trans hpref hpower
  have hm := Fraction.mul_le_mul_nonnegative hp (stateNorm s)
    (stateNorm_nonnegative s)
  exact Fraction.magnitudes.le_trans (coarseBlocks_norm_bound w h k s i) hm

def blockSource (w h : Fraction) (k : Nat) (s : Point × Point) : Fraction :=
  Fraction.mul (Fraction.mul h h)
    (Fraction.mul (Fraction.ofInt 2) (quadraticCap w s k))

theorem blockSource_nonnegative (w h : Fraction) (k : Nat)
  (s : Point × Point) (hh : 0 ≤ h.num) :

```

```

    0 ≤ (blockSource w h k s).num :=
      Fraction.nonnegative_mul _ _
        (Fraction.nonnegative_mul _ _ hh hh)
        (Fraction.nonnegative_mul _ _ (by decide)
          (quadraticCap_nonnegative w s k))

/-- Finite accumulation with a measured coarse-state confinement and an
explicit short-prefix premise. No Cauchy or partition-independence field. -/
theorem block_error_le_budget (w h : Fraction) (k : Nat)
  (s : Point × Point) (N : Nat)
  (hh : 0 ≤ h.num) (hb : Fraction.le h.abs (Fraction.ofInt 1))
  (hshort : ∀ j, j ≤ k →
    Fraction.le (integerDuration h j).abs (Fraction.ofInt 1))
  (hcoarsePower : Fraction.le
    (fpower (kappa w (integerDuration h k)) N) (Fraction.ofInt 2)) :
  (i : Nat) → i ≤ N →
    Fraction.le
      (stateNorm (stateSub
        (fineBlocks w h k s i) (coarseBlocks w h k s i)))
      (sourceBudget (fpower (kappa w h) k) (blockSource w h k s) i)
| 0, _ => by
  have hz := stateSub_self_norm_zero s
  exact Fraction.le_of_equiv (by
    simpa only [fineBlocks, coarseBlocks, sourceBudget] using hz)
| i + 1, hi => by
  have hiprev : i ≤ N := by omega
  have hprev := block_error_le_budget w h k s N hh hb hshort
    hcoarsePower i hiprev
  have hstep := block_error_step w h k s i hh hb hshort
  have hstate := coarseBlocks_state_le_two w h k s N hcoarsePower i hiprev
  have hcap := quadraticCap_double w (coarseBlocks w h k s i) s hstate k
  have h₂ : 0 ≤ (Fraction.mul h h).num :=
    Fraction.nonnegative_mul _ _ hh hh
  have hlocal := Fraction.mul_le_mul_nonnegative_left hcap
    (Fraction.mul h h) h₂
  have h₁ := Fraction.mul_le_mul_nonnegative_left hprev
    (fpower (kappa w h) k)
    (fpower_nonnegative _ (kappa_nonnegative w h) k)
  have hsum := Fraction.add_le_add h₁ hlocal
  have hbound := Fraction.magnitudes.le_trans hstep hsum
  simpa only [sourceBudget, blockSource] using hbound

def FullSmallTime (w d : Fraction) (N : Nat) : Prop :=
  Fraction.le
    (Fraction.mul (fullTime d N)
      (Fraction.add (Fraction.ofInt 1) w.abs)) halfThreshold

theorem kappa_duration_congr (w a b : Fraction)
  (hab : Fraction.equiv a b) :
  Fraction.equiv (kappa w a) (kappa w b) := by
  unfold kappa
  have ha := Fraction.abs_equiv hab
  exact Fraction.mul_equiv
    (Fraction.add_equiv (Fraction.equiv_refl _) ha)
    (Fraction.add_equiv (Fraction.equiv_refl _)
      (Fraction.mul_equiv ha (Fraction.equiv_refl _)))

theorem full_power_le_two (w d : Fraction) (N : Nat)
  (hd : 0 ≤ d.num) (hs : FullSmallTime w d N) :
  Fraction.le (fpower (kappa w d) N) (Fraction.ofInt 2) := by
  have htime := Fraction.mul_equiv (half_totalTime d N)
    (Fraction.equiv_refl (Fraction.add (Fraction.ofInt 1) w.abs))
  have hsmall : SmallTime w d.half N :=
    Fraction.le_equiv_left htime hs
  have hpower := coarse_power_le_two w d.half N hd hsmall
  have he := kappa_duration_congr w
    (Fraction.add d.half d.half) d (Fraction.half_add_self d)
  exact Fraction.le_equiv_left
    (Fraction.equiv_symm (fpower_congr he N)) hpower

```

```

theorem block_power_le_two (w h : Fraction) (k N : Nat)
  (hh : 0 ≤ h.num)
  (hs : FullSmallTime w (integerDuration h k) N) :
  Fraction.le
    (fpower (fpower (kappa w h) k) N) (Fraction.ofInt 2) := by
  have htime := Fraction.mul_equiv (integer_fullTime h k N)
    (Fraction.equiv_refl (Fraction.add (Fraction.ofInt 1) w.abs))
  have hsmall : FullSmallTime w h (k * N) :=
    Fraction.le_equiv_left htime hs
  have hpower := full_power_le_two w h (k * N) hh hsmall
  exact Fraction.le_equiv_left
    (fpower_integer_blocks (kappa w h) k N) hpower

/-- Accumulated comparison of `N` actual coarse cells of duration `k*h`
with `k*N` actual fine cells. The finite short-prefix inequalities ensure
each local split is inside the calibrated unit window. -/
theorem accumulated_integer_error (w h : Fraction) (k N : Nat)
  (s : Point × Point)
  (hh : 0 ≤ h.num) (hb : Fraction.le h.abs (Fraction.ofInt 1))
  (hshort : ∀ j, j ≤ k →
    Fraction.le (integerDuration h j).abs (Fraction.ofInt 1))
  (hs : FullSmallTime w (integerDuration h k) N) :
  Fraction.le
    (stateNorm (stateSub
      (fineBlocks w h k s N) (coarseBlocks w h k s N)))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (Fraction.ofInt (N : Int))
        (blockSource w h k s))) := by
  let r := fpower (kappa w h) k
  let C := blockSource w h k s
  have hd : 0 ≤ (integerDuration h k).num :=
    integerDuration_nonnegative h hh k
  have hcoarse := full_power_le_two w (integerDuration h k) N hd hs
  have hfine := block_power_le_two w h k N hh hs
  have h₀ := block_error_le_budget w h k s N hh hb hshort
  hcoarse N (Nat.le_refl N)
  have hr : 0 ≤ r.num := fpower_nonnegative _ (kappa_nonnegative w h) k
  have hone : Fraction.le (Fraction.ofInt 1) r :=
    one_le_fpower (kappa w h) (kappa_nonnegative w h)
    (one_le_kappa w h) k
  have hC : 0 ≤ C.num := blockSource_nonnegative w h k s hh
  have h₁ := sourceBudget_power r C hr hC hone N
  have hm₁ := Fraction.mul_le_mul_nonnegative hfine C hC
  have hm₂ := Fraction.mul_le_mul_nonnegative_left hm₁
    (Fraction.ofInt (N : Int)) (Int.ofNat_nonneg N)
  have h₁' := Fraction.le_equiv_right h₁
    (Fraction.mul_equiv (Fraction.equiv_refl _)
      (Fraction.mul_comm C (fpower r N)))
  have hbound := Fraction.magnitudes.le_trans h₀
    (Fraction.magnitudes.le_trans h₁' hm₂)
  apply Fraction.le_equiv_right hbound
  simp only [C, r, Fraction.equiv, Fraction.mul, Fraction.ofInt]
  ac_nf

theorem full_window_duration_le_one (w d : Fraction) (N : Nat)
  (hd : 0 ≤ d.num) (hN : 0 < N)
  (hs : FullSmallTime w d N) :
  Fraction.le d (Fraction.ofInt 1) := by
  have hc : Fraction.le (Fraction.ofInt 1)
    (Fraction.ofInt (N : Int)) := by
  unfold Fraction.le Fraction.ofInt
  dsimp
  simp only [Int.mul_one]
  exact Int.ofNat_le.mpr hN
  have hdtim₁ := Fraction.mul_le_mul_nonnegative hc d hd
  have hdtim : Fraction.le d (fullTime d N) := by
  apply Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp) hdtim₁
  have hfactor : Fraction.le (Fraction.ofInt 1)

```

```

    (Fraction.add (Fraction.ofInt 1) w.abs) :=
    Fraction.le_add_nonnegative _ _ (Fraction.abs_num_nonnegative w)
  have htime : 0 ≤ (fullTime d N).num :=
    Int.mul_nonneg (Int.ofNat_nonneg N) hd
  have hfactorTime := Fraction.mul_le_mul_nonnegative_left hfactor
    (fullTime d N) htime
  have htimeWeighted : Fraction.le (fullTime d N)
    (Fraction.mul (fullTime d N)
      (Fraction.add (Fraction.ofInt 1) w.abs)) := by
    have he : Fraction.equiv (fullTime d N)
      (Fraction.mul (fullTime d N) (Fraction.ofInt 1)) := by
      simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    simp
    exact Fraction.le_equiv_left he hfactorTime
  have hhalf : Fraction.le halfThreshold (Fraction.ofInt 1) := by
    unfold halfThreshold Fraction.le Fraction.ofInt
    decide
  exact Fraction.magnitudes.le_trans hdttime
    (Fraction.magnitudes.le_trans htimeWeighted
      (Fraction.magnitudes.le_trans hs hhalf))

theorem integer_window_short (w h : Fraction) (k N : Nat)
  (hh : 0 ≤ h.num) (hk : 0 < k) (hN : 0 < N)
  (hs : FullSmallTime w (integerDuration h k) N) :
  Fraction.le h.abs (Fraction.ofInt 1) ∧
  ∀ j, j ≤ k →
    Fraction.le (integerDuration h j).abs (Fraction.ofInt 1) := by
  let d := integerDuration h k
  have hd : 0 ≤ d.num := integerDuration_nonnegative h hh k
  have hdle := full_window_duration_le_one w d N hd hN hs
  have hfirst := integerDuration_le h hh 1 k hk
  have hfirstEq : Fraction.equiv (integerDuration h 1) h := by
    simp only [integerDuration, Fraction.equiv, Fraction.add, Fraction.ofInt]
    simp
  have hhle : Fraction.le h (Fraction.ofInt 1) :=
    Fraction.magnitudes.le_trans
      (Fraction.le_equiv_left (Fraction.equiv_symm hfirstEq) hfirst) hdle
  constructor
  · exact Fraction.le_equiv_left
    (Fraction.abs_of_nonnegative h hh) hhle
  · intro j hj
    have hprefix := integerDuration_le h hh j k hj
    have hjle := Fraction.magnitudes.le_trans hprefix hdle
    exact Fraction.le_equiv_left
      (Fraction.abs_of_nonnegative (integerDuration h j)
        (integerDuration_nonnegative h hh j)) hjle

/-- The positive-count small-window form needs no separate local
shortness assumptions. -/
theorem accumulated_integer_error_positive (w h : Fraction) (k N : Nat)
  (s : Point × Point)
  (hh : 0 ≤ h.num) (hk : 0 < k) (hN : 0 < N)
  (hs : FullSmallTime w (integerDuration h k) N) :
  Fraction.le
    (stateNorm (stateSub
      (fineBlocks w h k s N) (coarseBlocks w h k s N)))
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (Fraction.ofInt (N : Int))
        (blockSource w h k s))) := by
  obtain (hb, hshort) := integer_window_short w h k N hh hk hN hs
  exact accumulated_integer_error w h k N s hh hb hshort hs

theorem integerFine_eq_run (w h : Fraction) (s : Point × Point) :
  (n : Nat) → integerFine w h s n = BoundedIteration.run (linearField w) h s n
| 0 => rfl
| n+1 => by
  simp only [integerFine, schedule_replicate_step, BoundedIteration.run]
  rw [← integerFine, integerFine_eq_run w h s n]
  rfl

```

```

theorem fineBlocks_eq_schedule (w h : Fraction) (k : Nat) (s : Point × Point) :
  (N : Nat) → fineBlocks w h k s N = integerFine w h s (k*N)
| 0 => rfl
| N+1 => by
  rw [fineBlocks, fineBlocks_eq_schedule w h k s N]
  simp only [integerFine_eq_run]
  rw [← BoundedIteration.run_add]
  congr 1

theorem coarseBlocks_eq_schedule (w h : Fraction) (k : Nat) (s : Point × Point) :
  (N : Nat) → coarseBlocks w h k s N = integerFine w (integerDuration h k) s N
| 0 => rfl
| N+1 => by
  rw [coarseBlocks, coarseBlocks_eq_schedule w h k s N]
  simp only [integerCoarse, integerFine, schedule_replicate_step]

```

```

end NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision

```

NewtonLimitDynamics/Polygon/HarmonicDyadicAgreement.lean, 16 theorems, 224 lines

```

import BarrowLib.Polygon.FiniteAddress
import NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision
import NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement
import BarrowLib.Polygon.IntegerTime

/-! E/G agreement at every finite binary dyadic time, from an actual
integer-subdivision comparison whose error decreases geometrically. -/

namespace NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement
open NewtonLimitDynamics TimeSubdivision CentralSchedule HarmonicStability
open HarmonicDyadic HarmonicBinaryPrefix HarmonicComparison PointBounds
open HarmonicTimeComparison HarmonicTimeRealization HarmonicAccumulation
open IntegerSchedule FiniteRecurrence BinaryTime CauchyValues HarmonicIntegerSubdivision

theorem timeApprox_small (b : Nat → Bool) (w T : Fraction) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  DyadicSmallTime w (timeApprox b T m) := by
  have ht := prefix_elapsed_le_time b T m hT
  have hp := Fraction.le_equiv_left (Fraction.equiv_symm (prefix_elapsed b T m)) ht
  exact Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative hp (Fraction.add (Fraction.ofInt 1) w.abs)
      (Fraction.nonnegative_add _ _ (by decide) (Fraction.abs_num_nonnegative w))) hs

theorem dyadic_integer_small (b : Nat → Bool) (w T : Fraction) (m j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  FullSmallTime w (integerDuration (duration T (m+j)) (ticks b m)) (blocks j) := by
  have hd := Fraction.mul_equiv (dyadic_fullTime b T m j)
  (Fraction.equiv_refl (Fraction.add (Fraction.ofInt 1) w.abs))
  exact Fraction.le_equiv_left hd (timeApprox_small b w T m hT hs)

def dyadicIntegerCoefficient (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat) : Fraction :=
  Fraction.mul (Fraction.ofInt 4)
  (Fraction.mul (Fraction.mul (duration T m) (duration T m))
    (quadraticCap w s (ticks b m)))

theorem dyadicIntegerCoefficient_nonnegative (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat) (hT : 0 ≤ T.num) :
  0 ≤ (dyadicIntegerCoefficient b w T s m).num :=
  Fraction.nonnegative_mul _ _ (by decide)
  (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hT hT)
    (quadraticCap_nonnegative w s (ticks b m)))

theorem dyadic_integer_error_cap (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m j : Nat) :
  Fraction.equiv
    (Fraction.mul (Fraction.ofInt 2)
      (Fraction.mul (Fraction.ofInt (blocks j:Int))
        (blockSource w (duration T (m+j)) (ticks b m) s)))
    (duration (dyadicIntegerCoefficient b w T s m) j) := by

```

```

have hb : (blocks j:Int)=(2:Int)^j := by
  simp only [blocks,Int.natCast_pow]
  rfl
simp only [blockSource,dyadicIntegerCoefficient,duration,hb,
  Fraction.equiv,Fraction.mul,Fraction.ofInt,Int.natCast_pow,
  FiniteGrowth.denominator_power_add]
simp only [show (4:Int)=2*2 by rfl]
ac_nf

theorem dyadic_fine_prefix (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m j : Nat) (hz : ∀ i, m ≤ i → b i = false) :
  fineBlocks w (duration T (m+j)) (ticks b m) s (blocks j) =
    prefixState b w T s (m+j) := by
  rw [fineBlocks_eq_schedule]
  simp only [integerFine,prefixState,ticks_zero_tail b m hz j]

theorem dyadic_coarse_endpoint (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m j : Nat) :
  stateEquiv
    (coarseBlocks w (duration T (m+j)) (ticks b m) s (blocks j))
    (endpoint w (timeApprox b T m) s j) := by
  rw [coarseBlocks_eq_schedule]
  exact schedule_replicate_congr w _ _ (dyadic_integer_duration b T m j)
    (blocks j) s s ((Fraction.equiv_refl _,Fraction.equiv_refl _),
      (Fraction.equiv_refl _,Fraction.equiv_refl _))

theorem finite_tail_level_error (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m j : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hk : 0 < ticks b m) (hz : ∀ i, m ≤ i → b i = false) :
  Fraction.le
    (distance (endpoint w (timeApprox b T m) s j) (prefixState b w T s (m+j)))
    (duration (dyadicIntegerCoefficient b w T s m) j) := by
  have hi := accumulated_integer_error_positive w (duration T (m+j))
    (ticks b m) (blocks j) s hT hk (Nat.pow_pos (by decide))
    (dyadic_integer_small b w T m j hT hs)
  rw [dyadic_fine_prefix b w T s m j hz] at hi
  have he : Fraction.equiv
    (distance (prefixState b w T s (m+j))
      (coarseBlocks w (duration T (m+j)) (ticks b m) s (blocks j)))
    (distance (prefixState b w T s (m+j)) (endpoint w (timeApprox b T m) s j)) :=
    stateNorm_equiv (stateSub_congr
      ((Fraction.equiv_refl _,Fraction.equiv_refl _),
        (Fraction.equiv_refl _,Fraction.equiv_refl _))
      (dyadic_coarse_endpoint b w T s m j))
  exact Fraction.le_equiv_left (stateSub_norm_symm _ _)
    (Fraction.le_equiv_right
      (Fraction.le_equiv_left (Fraction.equiv_symm he) hi)
      (dyadic_integer_error_cap b w T s m j))

theorem finite_tail_names (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hz : ∀ i, m ≤ i → b i = false) :
  NameEquiv
    (endpointName w (timeApprox b T m) s (timeApprox_nonnegative b T m hT)
      (timeApprox_small b w T m hT hs)) (prefixName b w T s hT hs) := by
  by_cases hk : ticks b m = 0
  · have ht : (timeApprox b T m).num = 0 := by simp only [timeApprox,Fraction.mul,
    Fraction.ofInt,hk,Int.natCast_zero,Int.zero_mul]
    intro eps heps
    refine ⟨m, ?_⟩
    intro j hj
    have he := zero_time_endpoint w (timeApprox b T m) s j ht
    have hindex : m+(j-m)=j := by omega
    have hp : prefixState b w T s j = s := by
      rw [← hindex]
      simp only [prefixState,ticks_zero_tail b m hz (j-m),hk,Nat.zero_mul,
        List.replicate_zero,schedule]
    have heq : Fraction.equiv

```

```

      (distance (endpoint w (timeApprox b T m) s j) (prefixState b w T s j))
      (distance s s) := by
    rw [hp]
    exact stateNorm_equiv (stateSub_congr he
      ((Fraction.equiv_refl _, Fraction.equiv_refl _),
      (Fraction.equiv_refl _, Fraction.equiv_refl _)))
    exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv heq)
      (distance_self_lt s eps heps)
  · have hkpos : 0 < ticks b m := by omega
    intro eps heps
    obtain (N,hN) := duration_eventually_small (dyadicIntegerCoefficient b w T s m)
      eps.half.half (dyadicIntegerCoefficient_nonnegative b w T s m hT) heps
    obtain (K,hK) := (prefixName b w T s hT hs).cauchy eps.half.half heps
    refine (N+K, ?_)
    intro j hj
    have herror := Fraction.magnitudes.lt_of_le_lt
      (finite_tail_level_error b w T s m j hT hs hkpos hz) (hN j (by omega))
    have hshift := hK (m+j) j (by omega) (by omega)
    have htri := stateSub_triangle (endpoint w (timeApprox b T m) s j)
      (prefixState b w T s (m+j)) (prefixState b w T s j)
    have hsum := Fraction.add_le_add (Fraction.magnitudes.lt_implies_le herror)
      (Fraction.magnitudes.lt_implies_le hshift)
    have hbound := Fraction.le_equiv_right (Fraction.magnitudes.le_trans htri hsum)
      (Fraction.half_add_self eps.half)
    exact Fraction.magnitudes.lt_of_le_lt hbound (Fraction.half_lt eps heps)

/-- Every finite binary dyadic time has the same completed state under the
endpoint and one-global-family prefix constructions. -/
theorem finite_tail_value (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hz : ∀ i, m ≤ i → b i = false) :
  CauchyValues.timeValue w s
    (timeApprox b T m, timeApprox_nonnegative b T m hT, timeApprox_small b w T m hT hs) =
    gammaValue w T s hT hs (Quotient.mk _ b) :=
  Quotient.sound (finite_tail_names b w T s m hT hs hz)

def dyadicTime (w T : Fraction) (k m : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (_hk : k < blocks m) : ShortRationalTime w :=
  (timeApprox (finiteAddress m k) T m,
  timeApprox_nonnegative (finiteAddress m k) T m hT,
  timeApprox_small (finiteAddress m k) w T m hT hs)

theorem dyadicTime_exact (w T : Fraction) (k m : Nat) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (hk : k < blocks m) :
  (dyadicTime w T k m hT hs hk).val =
  Fraction.mul (Fraction.ofInt (k:Int)) (duration T m) := by
  simp only [dyadicTime, timeApprox, finiteAddress_ticks m k hk]

/-- Every numerator below 2^m has an explicitly constructed address;
agreement does not assume a representation exists. The full endpoint uses
full_window_value. -/
theorem dyadic_value (w T : Fraction) (s : Point × Point) (k m : Nat)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (hk : k < blocks m) :
  CauchyValues.timeValue w s (dyadicTime w T k m hT hs hk) =
  gammaValue w T s hT hs (Quotient.mk _ (finiteAddress m k)) :=
  finite_tail_value (finiteAddress m k) w T s m hT hs (finiteAddress_tail m k)

theorem timeValue_equiv_parameter (w : Fraction) (s : Point × Point)
  (t u : ShortRationalTime w) (ht : Fraction.equiv t.val u.val) :
  CauchyValues.timeValue w s t = CauchyValues.timeValue w s u := by
  apply Quotient.sound
  apply nameEquiv_of_levelwise_stateEquiv
  intro j
  exact schedule_replicate_congr w _ _ (duration_congr ht j) (blocks j) s s
    ((Fraction.equiv_refl _, Fraction.equiv_refl _),
    (Fraction.equiv_refl _, Fraction.equiv_refl _))

theorem full_dyadic_value_at_represented_time (w T : Fraction) (s : Point × Point)
  (m : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)

```



```

    (t : ShortRationalTime w)
    (ht : Fraction.equiv t.val
      (Fraction.mul (Fraction.ofInt (blocks m: Int)) (duration T m))) :
    CauchyValues.timeValue w s t = gammaValue w T s hT hs (rightTime T ht) :=
    (timeValue_equiv_parameter w s t (T, hT, hs)
      (Fraction.equiv_trans ht (blocks_duration T m))).trans
    (full_window_value w T s hT hs)

theorem dyadic_value_at_represented_time (w T : Fraction) (s : Point × Point)
  (k m : Nat) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (hk : k < blocks m) (t : ShortRationalTime w)
  (ht : Fraction.equiv t.val
    (Fraction.mul (Fraction.ofInt (k: Int)) (duration T m))) :
  CauchyValues.timeValue w s t =
    gammaValue w T s hT hs (Quotient.mk _ (finiteAddress m k)) := by
  have he : Fraction.equiv t.val (dyadicTime w T k m hT hs hk).val := by
    rw [dyadicTime_exact w T k m hT hs hk]
    exact ht
  exact (timeValue_equiv_parameter w s t (dyadicTime w T k m hT hs hk) he).trans
    (dyadic_value w T s k m hT hs hk)

/-- A three-tick time has equal completed values even though the earlier
exact control proves its finite endpoint and prefix schedules unequal. -/
theorem three_tick_value_agreement (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  CauchyValues.timeValue w s (dyadicTime w T 3 2 hT hs (by decide)) =
    gammaValue w T s hT hs (Quotient.mk _ (finiteAddress 2 3)) :=
  dyadic_value w T s 3 2 hT hs (by decide)

theorem three_tick_address_control :
  ticks (finiteAddress 2 3) 2 = 3 ∧
  finiteAddress 2 3 2 = false ∧ finiteAddress 2 3 5 = false := by decide

```

end NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement

NewtonLimitDynamics/Polygon/CalibratedForces.lean, 4 theorems, 82 lines

```

import NewtonLimitDynamics.Polygon.ForceClasses
import BarrowLib.Polygon.TimeCalibration
import BarrowLib.Polygon.EquivalentDuration

/-- Modern calibrated instances of the general finite force estimates.
The free calibration fixes no physical action constant and adds no historical
premise. General convergence, confinement and restart remain separate. -/

namespace NewtonLimitDynamics.Polygon.CalibratedForces
open NewtonLimitDynamics
open TimeSubdivision PointBounds CentralSchedule HarmonicStability ForceClasses

/-- Actual two-precision sampled polygons retain the weighted sampling budget. -/
theorem sampled_calibrated_discrepancy (o : Oracle) (tau h L : Fraction)
  (ht : 0 < tau.num) (hL : LipschitzOn o L)
  (i j : Nat) (hij : i ≤ j) (s : Point × Point) (n : Nat)
  (hR : ∀ k, k < n →
    o.region (BoundedIteration.run (o.sample i) h s (k+1)).1 ∧
    o.region (BoundedIteration.run (o.sample j) h s (k+1)).1)
  (hs : TimeCalibration.Window tau h L ht n) :
  Fraction.le
    (TimeCalibration.distance tau
      (schedule (o.sample i) (List.replicate n h) s)
      (schedule (o.sample j) (List.replicate n h) s))
    (Fraction.mul (Fraction.ofInt (2 * (n : Int)))
      (Fraction.mul (Fraction.mul tau h.abs)
        (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i)))) := by
  rw [← run_eq_schedule, ← run_eq_schedule]
  apply EquivalentDuration.run_uniform_error_at tau ht (o.sample i) (o.sample j)
  h h L _ s (Fraction.equiv_refl _) hL.1
  (Fraction.nonnegative_add _ _
    (Fraction.nonnegative_add _ _ (o.error_nonnegative i) (o.error_nonnegative i))
    (o.error_nonnegative i))
  n hs

```

```

intro k hk
exact samples_comparison_contract o L hL i j hij _ _ (hR k hk).1 (hR k hk).2

/-- Harmonic motion is an instance of the calibrated general estimate. -/
theorem harmonic_calibrated_cell (tau w h : Fraction) (ht : 0 < tau.num)
(s t : Point × Point) :
  Fraction.le
    (TimeCalibration.distance tau (cell (linearField w) h s) (cell (linearField w) h t))
    (Fraction.add
      (Fraction.mul (TimeCalibration.amplification tau h w.abs ht)
        (TimeCalibration.distance tau s t))
      (Fraction.mul (Fraction.mul tau h.abs) (Fraction.ofInt 0))) :=
  TimeCalibration.cell_amplification tau ht
    (Fraction.abs_num_nonnegative w) (harmonic_comparison_contract w)

/-- The centre-at-infinity instance uses L=0. -/
theorem parallel_calibrated_cell (tau h : Fraction) (ht : 0 < tau.num)
(a : Point) (s t : Point × Point) :
  Fraction.le
    (TimeCalibration.distance tau (cell (fun _ => a) h s) (cell (fun _ => a) h t))
    (Fraction.add
      (Fraction.mul (TimeCalibration.amplification tau h (Fraction.ofInt 0) ht)
        (TimeCalibration.distance tau s t))
      (Fraction.mul (Fraction.mul tau h.abs) (Fraction.ofInt 0))) := by
  have hcontract : FiniteEstimates.comparisonContract (fun _ => a) (fun _ => a)
    (Fraction.ofInt 0) (Fraction.ofInt 0) := by
    intro p q
    exact Fraction.le_equiv_right
      (Fraction.le_of_equiv (FiniteEstimates.pointDistance_self_zero a))
      (by simp [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt])
  exact TimeCalibration.cell_amplification tau ht (fun _ => a) (fun _ => a) h
    (Fraction.ofInt 0) (Fraction.ofInt 0) s t (by decide) hcontract

set_option maxHeartbeats 500000 in
/-- Exact finite harmonic mechanics commute with a positive change of time unit. -/
theorem harmonic_cell_rescale (c w h : Fraction) (hc : 0 < c.num)
(s : Point × Point) :
  stateEquiv
    (TimeCalibration.rescaleState c hc (cell (linearField w) h s))
    (cell (linearField (TimeCalibration.rescaleConstant c w hc))
      (Fraction.mul c h) (TimeCalibration.rescaleState c hc s)) := by
  constructor <=> constructor <=>
  simp only [stateEquiv, pointEquiv, cell, linearField, negF, pointAdd, pointScale,
    TimeCalibration.rescaleState, TimeCalibration.rescaleConstant, TimeCalibration.inverse,
    Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add,
    Int.neg_mul, Int.mul_neg] <=> ac_nf

end NewtonLimitDynamics.Polygon.CalibratedForces

NewtonLimitDynamics/Polygon/GeneralForcePrecision.lean, 6 theorems, 62 lines

import NewtonLimitDynamics.Polygon.ForceClasses
import BarrowLib.Polygon.GeometricTail

/-- A monotone mesh-precision choice extracted from an oracle's force error. -/

namespace NewtonLimitDynamics.Polygon.GeneralForcePrecision

open NewtonLimitDynamics
open ForceClasses

theorem target_positive (E0 : Fraction) (hE : 0 < E0.num) (j : Nat) :
  0 < (GeometricTail.tailCap E0 j).num := hE

noncomputable def threshold (o : Oracle) (E0 : Fraction)
(hE : 0 < E0.num) (j : Nat) : Nat :=
  Classical.choose (o.error_vanishes (GeometricTail.tailCap E0 j)
    (target_positive E0 hE j))

theorem threshold_error (o : Oracle) (E0 : Fraction)
(hE : 0 < E0.num) (j n : Nat)

```

```

    (hn : threshold o E0 hE j ≤ n) :
    Fraction.lt (o.error n) (GeometricTail.tailCap E0 j) :=
    Classical.choose_spec (o.error_vanishes (GeometricTail.tailCap E0 j)
    (target_positive E0 hE j)) n hn

noncomputable def precision (o : Oracle) (E0 : Fraction)
  (hE : 0 < E0.num) : Nat → Nat
| 0 => threshold o E0 hE 0
| j + 1 => max (precision o E0 hE j) (threshold o E0 hE (j + 1))

theorem precision_successor (o : Oracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) :
  precision o E0 hE j ≤ precision o E0 hE (j + 1) :=
  Nat.le_max_left _ _

theorem precision_monotone (o : Oracle) (E0 : Fraction)
  (hE : 0 < E0.num) {i j : Nat} (hi : i ≤ j) :
  precision o E0 hE i ≤ precision o E0 hE j := by
  induction j with
  | zero =>
    have hi : i = 0 := by omega
    subst i
    exact Nat.le_refl _
  | succ j ih =>
    by_cases he : i = j + 1
    · subst i
      exact Nat.le_refl _
    · exact Nat.le_trans (ih (by omega)) (precision_successor o E0 hE j)

theorem precision_threshold (o : Oracle) (E0 : Fraction)
  (hE : 0 < E0.num) :
  (j : Nat) → threshold o E0 hE j ≤ precision o E0 hE j
| 0 => Nat.le_refl _
| _ + 1 => Nat.le_max_right _ _

theorem precision_error (o : Oracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) :
  Fraction.lt (o.error (precision o E0 hE j))
  (GeometricTail.tailCap E0 j) :=
  threshold_error o E0 hE j _ (precision_threshold o E0 hE j)

end NewtonLimitDynamics.Polygon.GeneralForcePrecision

```

NewtonLimitDynamics/Polygon/GeneralForceEndpoint.lean, 23 theorems, 429 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePrecision
import BarrowLib.Polygon.CalibratedRefinement
import NewtonLimitDynamics.Polygon.RegionConfinement

/-! Construct fixed-time motion values from actual sampled central polygons.
Lipschitz comparison and sample bounds hold only on a certified band. The
partial-time invariant derives actual/coarse and both shadow membership
before force sampling; no whole-plane or supplied confinement trace is used. No motion-Cauchy,
trajectory, partition-independence or derivative premise is supplied. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceEndpoint
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic ForceClasses CauchyValues
open GeneralForcePrecision

noncomputable def field (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) : Point → Point :=
  o.sample (precision o.toOracle E0 hE j)

noncomputable def endpoint (o : CentralOracle) (E0 T : Fraction)
  (hE : 0 < E0.num) (s : Point × Point) (j : Nat) : Point × Point :=
  BoundedIteration.run (field o E0 hE j) (duration T j) s (blocks j)

structure Conditions (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) where
  calibration_positive : 0 < tau.num

```

```

lipschitz : LipschitzOn o.toOracle L
window : Fraction.le
  (Fraction.mul T (TimeCalibration.rate tau L calibration_positive)) (1,2,by decide)
inner_radius : Fraction
outer_radius : Fraction
frame : RegionConfinement.Frame o.region T B inner_radius outer_radius s
samples_on_band : ∀ j p, RegionConfinement.Band inner_radius outer_radius p →
  Fraction.le (pointNorm (field o E0 hE j p)) B

namespace Conditions
variable {o : CentralOracle} {E0 T tau L B : Fraction} {s : Point × Point} {hE : 0 < E0.num}

def time_nonnegative (d : Conditions o E0 T tau L B s hE) : 0 ≤ T.num :=
  d.frame.time_nonnegative

def bound_nonnegative (d : Conditions o E0 T tau L B s hE) : 0 ≤ B.num :=
  d.frame.bound_nonnegative

theorem run_band (d : Conditions o E0 T tau L B s hE)
  (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  RegionConfinement.Band d.inner_radius d.outer_radius
  (BoundedIteration.run (field o E0 hE j) h s n).1 :=
  RegionConfinement.run_band o.region T B _ _ s d.frame _
  (sample_central o _) (d.samples_on_band j) h hh n hn

theorem run_region (d : Conditions o E0 T tau L B s hE)
  (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  o.region (BoundedIteration.run (field o E0 hE j) h s n).1 :=
  d.frame.contains_band _ (d.run_band j h hh n hn)

theorem actual_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) :
  BoundedIteration.BoundedSamples (field o E0 hE j) (duration T j) s B (blocks j) :=
  RegionConfinement.run_bounded_samples o.region T B _ _ s d.frame _
  (sample_central o _) (d.samples_on_band j) (duration T j) d.time_nonnegative (blocks j)
  (Fraction.le_of_equiv (blocks_duration T j))

theorem coarse_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) :
  BoundedIteration.BoundedSamples (field o E0 hE j)
  (Fraction.add (duration T (j+1)) (duration T (j+1))) s B (blocks j) :=
  RegionConfinement.run_bounded_samples o.region T B _ _ s d.frame _
  (sample_central o _) (d.samples_on_band j)
  (Fraction.add (duration T (j+1)) (duration T (j+1)))
  (Fraction.nonnegative_add _ _ d.time_nonnegative d.time_nonnegative) (blocks j)
  (Fraction.le_of_equiv (coarse_time T j))

theorem shadow_samples (d : Conditions o E0 T tau L B s hE)
  (j k : Nat) (hk : k < blocks j) : Fraction.le
  (pointNorm (field o E0 hE j)
    (FiniteEstimates.cell (field o E0 hE j) (duration T (j+1))
      (FiniteAccumulation.coarseAt (field o E0 hE j) (duration T (j+1)) s k)).1)) B :=
  d.samples_on_band j _ (RegionConfinement.shadow_bands o.region T B _ _ s d.frame _
    (sample_central o _) (d.samples_on_band j) j k hk).1

end Conditions

noncomputable def sampleError (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) : Fraction :=
  let e := o.error (precision o.toOracle E0 hE j)
  Fraction.add (Fraction.add e e) e

theorem sampleError_nonnegative (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) : 0 ≤ (sampleError o E0 hE j).num :=
  Fraction.nonnegative_add _ _
  (Fraction.nonnegative_add _ _ (o.error_nonnegative _) (o.error_nonnegative _))
  (o.error_nonnegative _)

theorem cross_contract (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)

```

```

    (j : Nat) (p q : Point) (hp : o.region p) (hq : o.region q) :
    Fraction.le (FiniteEstimates.pointDistance (field o E0 hE (j+1) p) (field o E0 hE j q))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) (sampleError o E0 hE j)) := by
have hc := samples_comparison_contract o.toOracle L d.lipschitz
    (precision o.toOracle E0 hE j) (precision o.toOracle E0 hE (j+1))
    (precision_successor o.toOracle E0 hE j) q p hq hp
exact Fraction.le_equiv_right
    (Fraction.le_equiv_left (FiniteEstimates.pointDistance_symm _ _) hc)
    (Fraction.add_equiv (Fraction.mul_equiv_left L (FiniteEstimates.pointDistance_symm _ _))
    (Fraction.equiv_refl _))

theorem local_contract (o : CentralOracle) (E0 T tau L B : Fraction)
    (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
    (j : Nat) (p q : Point) (hp : o.region p) (hq : o.region q) :
    Fraction.le (FiniteEstimates.pointDistance (field o E0 hE j p) (field o E0 hE j q))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) (sampleError o E0 hE j)) :=
samples_comparison_contract o.toOracle L d.lipschitz _ _ (Nat.le_refl _) p q hp hq

theorem coarse_velocity (o : CentralOracle) (E0 T tau L B : Fraction)
    (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
    (j k : Nat) (hk : k < blocks j) :
    Fraction.le
        (pointNorm (FiniteAccumulation.coarseAt (field o E0 hE j) (duration T (j+1)) s k).2)
        (velocityCap T B s) := by
rw [CalibratedRefinement.coarseAt_eq_run]
have hh : 0 ≤ (Fraction.add (duration T (j+1)) (duration T (j+1))).num :=
    Fraction.nonnegative_add _ _ d.time_nonnegative d.time_nonnegative
have hb : BoundedIteration.BoundedSamples (field o E0 hE j)
    (Fraction.add (duration T (j+1)) (duration T (j+1))) s B k :=
    fun i hi => d.coarse_samples j i (by omega)
have ht := Fraction.magnitudes.le_trans
    (BoundedIteration.time_monotone _ hh k (blocks j) (Nat.le_of_lt hk))
    (Fraction.le_of_equiv (coarse_time T j))
exact BoundedIteration.velocity_bound_at_time _ _ s B T hh d.bound_nonnegative k hb ht

theorem fine_window (o : CentralOracle) (E0 T tau L B : Fraction)
    (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
    (j : Nat) :
    TimeCalibration.Window tau (duration T (j+1)) L d.calibration_positive (2*blocks j) := by
have ht := Fraction.equiv_trans
    (Fraction.mul_equiv (Fraction.equiv_refl _)
    (Fraction.abs_of_nonnegative (duration T (j+1)) d.time_nonnegative)) (fine_time T j)
exact TimeCalibration.window_of_elapsed _ _ T _ d.calibration_positive d.lipschitz.1 _
    (Fraction.le_of_equiv ht) d.window

theorem coarse_window (o : CentralOracle) (E0 T tau L B : Fraction)
    (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
    (j : Nat) :
    TimeCalibration.Window tau
        (Fraction.add (duration T (j+1)) (duration T (j+1))) L
        d.calibration_positive (blocks j) := by
have hh : 0 ≤ (Fraction.add (duration T (j+1)) (duration T (j+1))).num :=
    Fraction.nonnegative_add _ _ d.time_nonnegative d.time_nonnegative
have ht := Fraction.equiv_trans
    (Fraction.mul_equiv (Fraction.equiv_refl _) (Fraction.abs_of_nonnegative _ hh)) (coarse_time T j)
exact TimeCalibration.window_of_elapsed _ _ T _ d.calibration_positive d.lipschitz.1 _
    (Fraction.le_of_equiv ht) d.window

/-- Actual prefix comparison with a full-window budget, including representation error. -/
theorem paired_finite_bound (o : CentralOracle) (E0 T tau L B : Fraction)
    (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
    (j n : Nat) (hn : n ≤ blocks j) :
    Fraction.le (TimeCalibration.distance tau
        (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
        (BoundedIteration.run (field o E0 hE j) (duration T j) s n))
    (Fraction.add
        (Fraction.mul (Fraction.ofInt (2*(blocks j : Int)))
        (CalibratedRefinement.blockSource tau (duration T (j+1)) L
        (sampleError o E0 hE j) B (velocityCap T B s) d.calibration_positive))
        (Fraction.mul (Fraction.ofInt (2*(blocks j : Int))))

```

```

(Fraction.mul
  (Fraction.mul tau
    (Fraction.add (duration T (j+1)) (duration T (j+1))).abs)
    (sampleError o E0 hE j)))) := by
have h1 := RegionConfinement.sampled_refinement_bound o T B d.inner_radius d.outer_radius
  s d.frame L tau d.lipschitz d.calibration_positive
  (precision o.toOracle E0 hE j) (precision o.toOracle E0 hE (j+1))
  (precision_successor o.toOracle E0 hE j) (d.samples_on_band j) (d.samples_on_band (j+1))
  j n hn (TimeCalibration.window_mono _ _ d.calibration_positive d.lipschitz.1
    (2*n) (2*blocks j) (by omega) (fine_window o E0 T tau L B s hE d j))
have hfull : 0 ≤ (Fraction.add (duration T (j+1)) (duration T (j+1))).num :=
  Fraction.nonnegative_add _ _ d.time_nonnegative d.time_nonnegative
have hdttime := Fraction.magnitudes.le_trans
  (BoundedIteration.time_monotone _ hfull n (blocks j) hn)
  (Fraction.le_of_equiv (coarse_time T j))
have hetime := Fraction.magnitudes.le_trans
  (BoundedIteration.time_monotone (duration T j) d.time_nonnegative n (blocks j) hn)
  (Fraction.le_of_equiv (blocks_duration T j))
have h2 := RegionConfinement.sampled_equivalent_duration_bound o T B d.inner_radius d.outer_radius
  s d.frame L tau d.lipschitz d.calibration_positive
  (precision o.toOracle E0 hE j) (precision o.toOracle E0 hE j) (Nat.le_refl _)
  (d.samples_on_band j) (d.samples_on_band j)
  (Fraction.add (duration T (j+1)) (duration T (j+1))) (duration T j)
  (Fraction.equiv_symm (duration_halving T j)) hfull d.time_nonnegative n hdttime hetime
  (TimeCalibration.window_mono _ _ d.calibration_positive d.lipschitz.1
    n (blocks j) hn (coarse_window o E0 T tau L B s hE d j))
have htri := TimeCalibration.distance_triangle tau d.calibration_positive
  (FiniteAccumulation.fineAt (field o E0 hE (j+1)) (duration T (j+1)) s n)
  (FiniteAccumulation.coarseAt (field o E0 hE j) (duration T (j+1)) s n)
  (BoundedIteration.run (field o E0 hE j) (duration T j) s n)
have hb := Fraction.magnitudes.le_trans htri (Fraction.add_le_add h1
  (by simp only [CalibratedRefinement.coarseAt_eq_run] using h2))
rw [CalibratedRefinement.fineAt_eq_run] at hb
have hc : Fraction.le (Fraction.ofInt (2*(n : Int)))
  (Fraction.ofInt (2*(blocks j : Int))) := by
  simp only [Fraction.le, Fraction.ofInt, Int.mul_one] using
  (show 2*(n : Int) ≤ 2*(blocks j : Int) by omega)
have hs := CalibratedRefinement.blockSource_nonnegative tau (duration T (j+1)) L
  (sampleError o E0 hE j) B (velocityCap T B s) d.calibration_positive d.lipschitz.1
  (sampleError_nonnegative o E0 hE j) d.bound_nonnegative
  (velocityCap_nonnegative T B s d.time_nonnegative d.bound_nonnegative)
have he : 0 ≤ (Fraction.mul
  (Fraction.mul tau (Fraction.add (duration T (j+1)) (duration T (j+1))).abs)
  (sampleError o E0 hE j)).num := Fraction.nonnegative_mul _ _
  (Fraction.nonnegative_mul _ _ (Int.le_of_lt d.calibration_positive)
    (Fraction.abs_num_nonnegative _)) (sampleError_nonnegative o E0 hE j)
exact Fraction.magnitudes.le_trans hb (Fraction.add_le_add
  (Fraction.mul_le_mul_nonnegative hc _ hs) (Fraction.mul_le_mul_nonnegative hc _ he))

```

```

theorem adjacent_finite_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j : Nat) :
  Fraction.le (TimeCalibration.distance tau
    (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j))
    (Fraction.add
      (Fraction.mul (Fraction.ofInt (2*(blocks j : Int)))
        (CalibratedRefinement.blockSource tau (duration T (j+1)) L
          (sampleError o E0 hE j) B (velocityCap T B s) d.calibration_positive))
      (Fraction.mul (Fraction.ofInt (2*(blocks j : Int)))
        (Fraction.mul
          (Fraction.mul tau
            (Fraction.add (duration T (j+1)) (duration T (j+1))).abs)
            (sampleError o E0 hE j)))) := by
have hb := paired_finite_bound o E0 T tau L B s hE d j (blocks j) (Nat.le_refl _)
have hn : 2*blocks j = blocks (j+1) := by rw [blocks_succ]; omega
rw [hn] at hb
exact hb

```

```

/-- The same mesh estimate holds uniformly at every paired prefix. -/
theorem paired_mesh_bound (o : CentralOracle) (E0 T tau L B : Fraction)

```

```

(s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
(j n : Nat) (hn : n ≤ blocks j) :
  Fraction.le (TimeCalibration.distance tau
    (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
    (BoundedIteration.run (field o E0 hE j) (duration T j) s n))
    (Fraction.add
      (Fraction.mul (Fraction.mul T (duration T (j+1)))
        (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)))
      (Fraction.mul (Fraction.ofInt 7)
        (Fraction.mul (Fraction.mul tau T) (sampleError o E0 hE j)))) := by
let h := duration T (j+1)
let E := sampleError o E0 hE j
let V := velocityCap T B s
have hh : 0 ≤ h.num := d.time_nonnegative
have hfull : 0 ≤ (Fraction.add h h).num := Fraction.nonnegative_add _ _ hh hh
have hHT : Fraction.le h.abs T := by
  rw [Fraction.abs_eq_of_nonnegative h hh]
  exact duration_le_window T d.time_nonnegative (j+1)
have hsmall1 : TimeCalibration.Window tau h L d.calibration_positive 1 := by
  apply TimeCalibration.window_of_elapsed _ _ T _ d.calibration_positive d.lipschitz.1 1 _ d.window
  exact Fraction.le_equiv_left (by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one, Int.natCast_one]) hHT
have hpow := TimeCalibration.amplification_power_le_two tau h L
  d.calibration_positive d.lipschitz.1 1 hsmall1
have hK : Fraction.le (TimeCalibration.amplification tau h L d.calibration_positive)
  (Fraction.ofInt 2) := Fraction.le_equiv_left (by
  simp only [HarmonicAccumulation.fpower, Fraction.equiv, Fraction.mul, Fraction.ofInt,
    Int.one_mul, Int.mul_one]) hpow
have hsrc := CalibratedRefinement.blockSource_bound tau d.calibration_positive h T L E B V
  d.lipschitz.1 (sampleError_nonnegative o E0 hE j) d.bound_nonnegative hHT hK
have hm := Fraction.mul_le_mul_nonnegative_left hsrc
  (Fraction.ofInt (2*(blocks j : Int))) (by dsimp [Fraction.ofInt]; omega)
have hb := Fraction.magnitudes.le_trans (paired_finite_bound o E0 T tau L B s hE d j n hn)
  (Fraction.add_le_add_right hm
    (Fraction.mul (Fraction.ofInt (2*(blocks j : Int)))
      (Fraction.mul (Fraction.mul tau (Fraction.add h h).abs) E)))
rw [Fraction.abs_eq_of_nonnegative h hh, Fraction.abs_eq_of_nonnegative (Fraction.add h h) hfull] at hb
apply Fraction.le_equiv_right hb
simp only [h, E, V, blocks, duration, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
  Int.natCast_pow, Int.pow_succ, show (7 : Int) = 5+1+1 by rfl,
  Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
ac_nf

theorem adjacent_mesh_bound (o : CentralOracle) (E0 T tau L B : Fraction)
(s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
(j : Nat) :
  Fraction.le (TimeCalibration.distance tau
    (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j))
    (Fraction.add
      (Fraction.mul (Fraction.mul T (duration T (j+1)))
        (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)))
      (Fraction.mul (Fraction.ofInt 7)
        (Fraction.mul (Fraction.mul tau T) (sampleError o E0 hE j)))) := by
have hb := paired_mesh_bound o E0 T tau L B s hE d j (blocks j) (Nat.le_refl _)
have hn : 2*blocks j = blocks (j+1) := by rw [blocks_succ]; omega
rw [hn] at hb
exact hb

/-- A dimensioned acceleration precision scale remains explicit. -/
def weightedCoefficient (E0 T tau L B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add
    (Fraction.mul (Fraction.mul T T)
      (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)))
    (Fraction.mul (Fraction.ofInt 42) (Fraction.mul (Fraction.mul tau T) E0))

theorem weightedCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
(hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num)
(hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (weightedCoefficient E0 T tau L B s).num := by
have hV := velocityCap_nonnegative T B s hT hB

```

```

have hC : 0 ≤ (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s)).num :=
  Fraction.nonnegative_add _ _ hB
  (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) hL)
    (Fraction.nonnegative_add _ _ hV (Fraction.nonnegative_mul _ _ hT hB)))
exact Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hT hT) hC)
  (Fraction.nonnegative_mul _ _ (by decide)
    (Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) hT)
      (Int.le_of_lt hE)))

/-- Geometric force precision and actual mesh consistency give a derived adjacent tail. -/
theorem paired_weighted_tail (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j n : Nat) (hn : n ≤ blocks j) :
  Fraction.le (TimeCalibration.distance tau
    (BoundedIteration.run (field o E0 hE (j+1)) (duration T (j+1)) s (2*n))
    (BoundedIteration.run (field o E0 hE j) (duration T j) s n))
    (GeometricTail.tailCap (weightedCoefficient E0 T tau L B s) (j+1)) := by
  have he0 := Fraction.magnitudes.lt_implies_le (precision_error o.toOracle E0 hE j)
  have he := Fraction.add_le_add (Fraction.add_le_add he0 he0) he0
  have he3 : Fraction.le (sampleError o E0 hE j)
    (Fraction.mul (Fraction.ofInt 3) (GeometricTail.tailCap E0 j)) := by
    apply Fraction.le_equiv_right he
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      show (3 : Int) = 1+1+1 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  have hm := Fraction.mul_le_mul_nonnegative_left
    (Fraction.mul_le_mul_nonnegative_left he3 (Fraction.mul tau T)
      (Fraction.nonnegative_mul _ _ (Int.le_of_lt d.calibration_positive) d.time_nonnegative))
    (Fraction.ofInt 7) (by decide)
  have hb := Fraction.magnitudes.le_trans (paired_mesh_bound o E0 T tau L B s hE d j n hn)
    (Fraction.add_le_add_left hm
      (Fraction.mul (Fraction.mul T (duration T (j+1)))
        (CalibratedRefinement.consistencyCoefficient tau T L B (velocityCap T B s))))
  apply Fraction.le_equiv_right hb
  simp only [weightedCoefficient, GeometricTail.tailCap, duration, Fraction.equiv,
    Fraction.add, Fraction.mul, Fraction.ofInt, Int.pow_succ,
    show (42 : Int) = 7*3*2 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf

theorem adjacent_weighted_tail (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j : Nat) :
  Fraction.le (TimeCalibration.distance tau
    (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j))
    (GeometricTail.tailCap (weightedCoefficient E0 T tau L B s) (j+1)) := by
  have hb := paired_weighted_tail o E0 T tau L B s hE d j (blocks j) (Nat.le_refl _)
  have hn : 2*blocks j = blocks (j+1) := by rw [blocks_succ]; omega
  rw [hn] at hb
  exact hb

noncomputable def coefficient (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) (TimeCalibration.inverse tau ht))
    (weightedCoefficient E0 T tau L B s)

theorem coefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (coefficient E0 T tau L B s ht).num :=
  Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos))
    (weightedCoefficient_nonnegative E0 T tau L B s hE hT ht hL hB)

theorem adjacent_tail (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j : Nat) :
  Fraction.le (FiniteEstimates.stateDistance
    (endpoint o E0 T hE s (j+1)) (endpoint o E0 T hE s j))
    (GeometricTail.tailCap (coefficient E0 T tau L B s d.calibration_positive) (j+1)) := by

```



```

have hC : 0 ≤ (Fraction.add (Fraction.ofInt 1)
  (TimeCalibration.inverse tau d.calibration_positive)).num :=
  Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)
have hb := Fraction.magnitudes.le_trans
  (TimeCalibration.uncalibrated_le_distance tau d.calibration_positive _ _)
  (Fraction.mul_le_mul_nonnegative_left (adjacent_weighted_tail o E0 T tau L B s hE d j) _ hC)
apply Fraction.le_equiv_right hb
simp only [coefficient, GeometricTail.tailCap, Fraction.equiv, Fraction.mul]
ac_nf

/-- The desired Cauchy proof is constructed from the actual adjacent recurrence. -/
noncomputable def endpointName (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE) :
  EndpointCauchyName where
approx := endpoint o E0 T hE s
cauchy := by
  intro eps heps
  let A := coefficient E0 T tau L B s d.calibration_positive
  have hA := coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative
  d.calibration_positive d.lipschitz.1 d.bound_nonnegative
  refine (GeometricTail.modulus A eps, ?_)
  intro m n hm hn
  exact Fraction.magnitudes.lt_of_le_lt
    (GeometricTail.two_sided _ A hA (adjacent_tail o E0 T tau L B s hE d) _ m n hm hn)
    (GeometricTail.doubleTail_lt_tolerance A eps hA heps)

noncomputable def endpointValue (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE) : Value :=
  realize (endpointName o E0 T tau L B s hE d)

theorem approximants_converge (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ n, N ≤ n → Within (embed (endpoint o E0 T hE s n))
    (endpointValue o E0 T tau L B s hE d) eps :=
  constant_approximants_converge _ eps heps

theorem zero_time_endpoint (o : CentralOracle) (E0 T : Fraction)
  (hE : 0 < E0.num) (s : Point × Point) (hT : T.num = 0) (j : Nat) :
  stateEquiv (endpoint o E0 T hE s j) s :=
  BoundedIteration.zero_duration_run (field o E0 hE j) (duration T j) hT s (blocks j)

theorem zero_time_value (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (hT : T.num = 0) : endpointValue o E0 T tau L B s hE d = embed s := by
  apply Quotient.sound
  intro eps heps
  refine (0, fun j _ => ?_)
  have hz := (distance_zero_iff_stateEquiv _ _).mpr (zero_time_endpoint o E0 T hE s hT j)
  exact Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv hz)
    ((Fraction.positive_iff_zero_lt eps).mp heps)

end NewtonLimitDynamics.Polygon.GeneralForceEndpoint

```

NewtonLimitDynamics/Polygon/HarmonicGeneralEndpoint.lean, 12 theorems, 176 lines

```
import NewtonLimitDynamics.Polygon.GeneralForceEndpoint
```

```

/-! The harmonic family is an actual instance of the general sampled endpoint
construction. Its finite acceleration bounds are derived on the short family,
not postulated globally. The old endpoint names and values remain available. -/

```

```

namespace NewtonLimitDynamics.Polygon.HarmonicGeneralEndpoint
open NewtonLimitDynamics
open TimeSubdivision PointBounds ForceClasses HarmonicStability HarmonicDyadic

```

```

/-- Precision selection has no effect on a zero-error exact law. -/
theorem harmonic_field (w E0 : Fraction) (hw : 0 ≤ w.num) (hE : 0 < E0.num) (j : Nat) :
  GeneralForceEndpoint.field (harmonicOracle w hw) E0 hE j = linearField w := rfl

```

```

def harmonicBound (w : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul w.abs (Fraction.mul (Fraction.ofInt 4) (stateNorm s))

theorem harmonicBound_nonnegative (w : Fraction) (s : Point × Point) :
  0 ≤ (harmonicBound w s).num := Fraction.nonnegative_mul _ _
  (Fraction.abs_num_nonnegative w)
  (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s))

theorem linear_sample_norm (w : Fraction) (p : Point) :
  Fraction.equiv (pointNorm (linearField w p)) (Fraction.mul w.abs (pointNorm p)) :=
  Fraction.equiv_trans (pointNorm_scale (negF w) p)
  (Fraction.mul_equiv (Fraction.abs_neg w) (Fraction.equiv_refl _))

theorem linear_sample_bound (w : Fraction) (s : Point × Point) (p : Point)
  (hp : Fraction.le (pointNorm p) (Fraction.mul (Fraction.ofInt 4) (stateNorm s))) :
  Fraction.le (pointNorm (linearField w p)) (harmonicBound w s) :=
  Fraction.le_equiv_left (linear_sample_norm w p)
  (Fraction.mul_le_mul_nonnegative_left hp w.abs (Fraction.abs_num_nonnegative w))

theorem full_run_state_bound (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (j k : Nat) (hk : k ≤ blocks j) :
  Fraction.le
    (stateNorm (BoundedIteration.run (linearField w)
      (Fraction.add (duration T (j+1)) (duration T (j+1))) s k))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  rw [run_eq_schedule, HarmonicAccumulation.coarseAt_schedule]
  exact HarmonicUniform.coarse_state_le_two w (duration T (j+1)) s k hT
  (HarmonicUniform.smallTime_prefix w (duration T (j+1)) k (blocks j) hT hk
    (dyadic_smallTime w T j hs))

theorem two_to_four (s : Point × Point) :
  Fraction.le (Fraction.mul (Fraction.ofInt 2) (stateNorm s))
  (Fraction.mul (Fraction.ofInt 4) (stateNorm s)) :=
  Fraction.mul_le_mul_nonnegative (by unfold Fraction.le Fraction.ofInt; decide) (stateNorm s)
  (stateNorm_nonnegative s)

theorem time_le_one (w T : Fraction) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le T (Fraction.ofInt 1) := by
  have hone := Fraction.le_add_nonnegative (Fraction.ofInt 1) w.abs
  (Fraction.abs_num_nonnegative w)
  have hm := Fraction.mul_le_mul_nonnegative_left hone T hT
  have he : Fraction.equiv (Fraction.mul T (Fraction.ofInt 1)) T := by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one]
  have ht := Fraction.magnitudes.le_trans
    (Fraction.le_equiv_left (Fraction.equiv_symm he) hm) hs
  exact Fraction.magnitudes.le_trans ht (by
    change Fraction.le ((1,2,by decide) : Fraction) (Fraction.ofInt 1)
    unfold Fraction.le Fraction.ofInt
    decide)

theorem shadow_position_bound (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (j k : Nat) (hk : k < blocks j) :
  Fraction.le
    (pointNorm (FiniteEstimates.cell (linearField w) (duration T (j+1))
      (FiniteAccumulation.coarseAt (linearField w) (duration T (j+1)) s k)).1)
    (Fraction.mul (Fraction.ofInt 4) (stateNorm s)) := by
  let t := FiniteAccumulation.coarseAt (linearField w) (duration T (j+1)) s k
  have ht : Fraction.le (stateNorm t) (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
    dsimp only [t]
    rw [CalibratedRefinement.coarseAt_eq_run]
    exact full_run_state_bound w T s hT hs j k (Nat.le_of_lt hk)
  have hp := Fraction.magnitudes.le_trans (point_le_state t) ht
  have hv := Fraction.magnitudes.le_trans (velocity_le_state t) ht
  have hh : Fraction.le (duration T (j+1)).abs (Fraction.ofInt 1) := by
    rw [Fraction.abs_eq_of_nonnegative (duration T (j+1)) hT]
    exact Fraction.magnitudes.le_trans (GeneralForceEndpoint.duration_le_window T hT (j+1)) (time_le_one
      (w T hT hs))
  have hmv := Fraction.magnitudes.le_trans
    (Fraction.mul_le_mul_nonnegative_left hv (duration T (j+1)).abs (Fraction.abs_num_nonnegative _))
    (Fraction.mul_le_mul_nonnegative hh (Fraction.mul (Fraction.ofInt 2) (stateNorm s)))

```

```

    (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s)))
have hc := Fraction.magnitudes.le_trans (FiniteEstimates.cell_position_growth (linearField w) (duration
  ↪ T (j+1)) t)
  (Fraction.add_le_add hp hmv)
apply Fraction.le_equiv_right hc
simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
  show (4 : Int) = 2+2 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
ac_nf

/-- The retained harmonic window supplies a finite regional ball budget. -/
theorem harmonic_band_budget (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  Fraction.le (Fraction.add (pointNorm s.1)
    (Fraction.mul T (GeneralForceEndpoint.velocityCap T (harmonicBound w s) s)))
    (Fraction.mul (Fraction.ofInt 4) (stateNorm s)) := by
have hwl := Fraction.le_equiv_right
  (Fraction.le_add_nonnegative w.abs (Fraction.ofInt 1) (by decide))
  (Fraction.add_comm _ _)
have hTw := Fraction.magnitudes.le_trans
  (Fraction.mul_le_mul_nonnegative_left hwl T hT) hs
have hc : 0 ≤ (Fraction.mul (Fraction.ofInt 4) (stateNorm s)).num :=
  Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s)
have hTB := Fraction.le_equiv_right
  (Fraction.le_equiv_left (Fraction.equiv_symm (Fraction.mul_assoc _ _ _))
    (Fraction.mul_le_mul_nonnegative hTw _ hc))
  (show Fraction.equiv (Fraction.mul ((1,2,by decide) : Fraction)
    (Fraction.mul (Fraction.ofInt 4) (stateNorm s)))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) by
    simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt]
    ac_nf)
have hV : Fraction.le (GeneralForceEndpoint.velocityCap T (harmonicBound w s) s)
  (Fraction.mul (Fraction.ofInt 3) (stateNorm s)) :=
  Fraction.le_equiv_right (Fraction.add_le_add (velocity_le_state s) hTB)
  (by simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    show (3 : Int)=1+2 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]; ac_nf)
have hTV := Fraction.magnitudes.le_trans
  (Fraction.mul_le_mul_nonnegative_left hV T hT)
  (Fraction.mul_le_mul_nonnegative (time_le_one w T hT hs)
    (Fraction.mul (Fraction.ofInt 3) (stateNorm s))
    (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s)))
apply Fraction.le_equiv_right (Fraction.add_le_add (point_le_state s) hTV)
simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
  show (4 : Int)=1+3 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
ac_nf

/-- Every force-data premise of the general construction is derived for the old harmonic window. -/
def conditions (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  GeneralForceEndpoint.Conditions (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs (harmonicBound w
    ↪ s) s hE where
calibration_positive := by decide
lipschitz := harmonic_lipschitz_on w hw
window := by
  apply Fraction.le_equiv_left (b := Fraction.mul T (Fraction.add (Fraction.ofInt 1) w.abs)) _ hs
  apply Fraction.mul_equiv (Fraction.equiv_refl _)
  simp only [TimeCalibration.rate, TimeCalibration.inverse, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one]
inner_radius := Fraction.ofInt 0
outer_radius := Fraction.mul (Fraction.ofInt 4) (stateNorm s)
frame := RegionConfinement.ball_frame _ T (harmonicBound w s) _ s hT
  (harmonicBound_nonnegative w s) (harmonic_band_budget w T s hT hs)
  (fun _ => True.intro)
samples_on_band := by
  intro j p hp
  exact linear_sample_bound w s p hp.2

theorem harmonic_endpoint_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (j : Nat) :
  GeneralForceEndpoint.endpoint (harmonicOracle w hw) E0 T hE s j = HarmonicDyadic.endpoint w T s j :=
  ↪ by

```

```

rw [GeneralForceEndpoint.endpoint,harmonic_field,run_eq_schedule]
rfl

theorem harmonic_name_equiv (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  CauchyValues.NameEquiv
    (GeneralForceEndpoint.endpointName (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
      (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs))
    (HarmonicDyadic.endpointName w T s hT hs) := by
  intro eps heps
  refine {0, fun j _ => ?_}
  change Fraction.lt (CauchyValues.distance (GeneralForceEndpoint.endpoint (harmonicOracle w hw) E0 T hE
    ↪ s j)
    (HarmonicDyadic.endpoint w T s j)) eps
  rw [harmonic_endpoint_eq]
  exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (FiniteEstimates.stateDistance_self_zero _))
    ((Fraction.positive_iff_zero_lt eps).mp heps)

theorem harmonic_value_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  GeneralForceEndpoint.endpointValue (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
    (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) =
  CauchyValues.realize (HarmonicDyadic.endpointName w T s hT hs) :=
  Quotient.sound (harmonic_name_equiv w E0 T s hw hE hT hs)

end NewtonLimitDynamics.Polygon.HarmonicGeneralEndpoint

NewtonLimitDynamics/Polygon/GeneralForcePrefix.lean, 13 theorems, 200 lines

import NewtonLimitDynamics.Polygon.GeneralForceEndpoint
import BarrowLib.Polygon.BinaryTime

/-! Actual prefixes of one dyadic sampled central-force family. The Cauchy
estimate is derived from paired refinement and the actual optional cell. Force
bounds are on the represented runs, and no motion-name premise is supplied. -/

namespace NewtonLimitDynamics.Polygon.GeneralForcePrefix
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix ForceClasses CauchyValues
open GeneralForceEndpoint

structure Conditions (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  extends GeneralForceEndpoint.Conditions o E0 T tau L B s hE where
namespace Conditions
variable {o : CentralOracle} {E0 T tau L B : Fraction} {s : Point × Point} {hE : 0 < E0.num}

theorem actual_samples (d : Conditions o E0 T tau L B s hE) (j : Nat) :
  BoundedIteration.BoundedSamples (field o E0 hE j) (duration T j) s B (blocks j) :=
  d.toConditions.actual_samples j

end Conditions

noncomputable def countState (o : CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (j n : Nat) : Point × Point :=
  BoundedIteration.run (field o E0 hE j) (duration T j) s n

noncomputable def prefixState (b : Nat → Bool) (o : CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (j : Nat) : Point × Point :=
  countState o E0 T s hE j (ticks b j)

theorem count_time_le (T : Fraction) (hT : 0 ≤ T.num) (j n : Nat) (hn : n ≤ blocks j) :
  Fraction.le (BoundedIteration.time (duration T j) n) T :=
  Fraction.magnitudes.le_trans
    (BoundedIteration.time_monotone (duration T j) hT n (blocks j) hn)
    (Fraction.le_of_equiv (blocks_duration T j))

theorem count_velocity (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j n : Nat) (hn : n ≤ blocks j) :

```

```

    Fraction.le (pointNorm (countState o E0 T s hE j n).2) (velocityCap T B s) :=
    BoundedIteration.velocity_bound_at_time (field o E0 hE j) (duration T j) s B T
    d.time_nonnegative d.bound_nonnegative
    n (fun i hi => d.actual_samples j i (by omega))
    (count_time_le T d.time_nonnegative j n hn)

theorem count_region (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j n : Nat) (hn : n ≤ blocks j) : o.region (countState o E0 T s hE j n).1 :=
  d.toConditions.run_region j (duration T j) d.time_nonnegative n
  (count_time_le T d.time_nonnegative j n hn)

/-- Every restarted comparison uses two certified points of the original run. -/
theorem restarted_comparison (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j n k : Nat) (hn : n+k ≤ blocks j) (i : Nat) (hi : i < k) :
  Fraction.le (FiniteEstimates.pointDistance
    (field o E0 hE j (BoundedIteration.run (field o E0 hE j) (duration T j)
      (countState o E0 T s hE j n) (i+1)).1)
    (field o E0 hE j (countState o E0 T s hE j n).1))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance
      (BoundedIteration.run (field o E0 hE j) (duration T j)
        (countState o E0 T s hE j n) (i+1)).1
      (countState o E0 T s hE j n).1)) (sampleError o E0 hE j)) := by
  apply local_contract o E0 T tau L B s hE d.toConditions j _ _ _
  (count_region o E0 T tau L B s hE d j n (by omega))
  change o.region (BoundedIteration.run (field o E0 hE j) (duration T j)
    (BoundedIteration.run (field o E0 hE j) (duration T j) s n) (i+1)).1
  rw [← BoundedIteration.run_add]
  exact count_region o E0 T tau L B s hE d j (n+(i+1)) (by omega)

def speedCap (T tau B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (velocityCap T B s) (Fraction.mul tau B)

theorem speedCap_nonnegative (T tau B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hB : 0 ≤ B.num) :
  0 ≤ (speedCap T tau B s).num :=
  Fraction.nonnegative_add _ _ (velocityCap_nonnegative T B s hT hB)
  (Fraction.nonnegative_mul _ _ (Int.le_of_lt ht) hB)

/-- Every increment samples the actual arrival of the represented family. -/
theorem count_step_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j n : Nat) (hn : n < blocks j) :
  Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j (n+1))
    (countState o E0 T s hE j n))
    (Fraction.mul (duration T j) (speedCap T tau B s)) := by
  have hb := TimeCalibration.cell_increment_bound tau (duration T j) B
  (velocityCap T B s) (field o E0 hE j) (countState o E0 T s hE j n)
  (Int.le_of_lt d.calibration_positive) (count_velocity o E0 T tau L B s hE d j n (Nat.le_of_lt hn))
  (d.actual_samples j n hn)
  rw [Fraction.abs_eq_of_nonnegative (duration T j) d.time_nonnegative] at hb
  exact hb

theorem prefix_next (b : Nat → Bool) (o : CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (j : Nat) :
  prefixState b o E0 T s hE (j+1) =
  if b j then countState o E0 T s hE (j+1) (2*ticks b j+1)
  else countState o E0 T s hE (j+1) (2*ticks b j) := by
  simp only [prefixState, ticks, bit]
  split <;> rfl

def weightedCoefficient (E0 T tau L B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (GeneralForceEndpoint.weightedCoefficient E0 T tau L B s)
  (Fraction.mul T (speedCap T tau B s))

theorem weightedCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (weightedCoefficient E0 T tau L B s).num :=

```

```

Fraction.nonnegative_add _ _
  (GeneralForceEndpoint.weightedCoefficient_nonnegative E0 T tau L B s hE hT ht hL hB)
  (Fraction.nonnegative_mul _ _ hT (speedCap_nonnegative T tau B s hT ht hB))

theorem adjacent_weighted_tail (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j : Nat) :
  Fraction.le (TimeCalibration.distance tau (prefixState b o E0 T s hE (j+1))
    (prefixState b o E0 T s hE j))
    (GeometricTail.tailCap (weightedCoefficient E0 T tau L B s) (j+1)) := by
  have href := GeneralForceEndpoint.paired_weighted_tail o E0 T tau L B s hE d.toConditions
    j (ticks b j) (ticks_le_blocks b j)
  have hop := count_step_bound o E0 T tau L B s hE d (j+1) (2*ticks b j) (by
    have hn := ticks_lt_blocks b j
    rw [blocks_succ]
    omega)
  have he : Fraction.equiv (Fraction.mul (duration T (j+1)) (speedCap T tau B s))
    (GeometricTail.tailCap (Fraction.mul T (speedCap T tau B s)) (j+1)) := by
    simp only [duration, GeometricTail.tailCap, Fraction.equiv, Fraction.mul]
    ac_nf
  rw [prefix_next]
  by_cases hb : b j
  · rw [if_pos hb]
    have ht := TimeCalibration.distance_triangle tau d.calibration_positive
      (countState o E0 T s hE (j+1) (2*ticks b j+1))
      (countState o E0 T s hE (j+1) (2*ticks b j)) (prefixState b o E0 T s hE j)
    exact Fraction.le_equiv_right
      (Fraction.magnitudes.le_trans ht
        (Fraction.add_le_add (Fraction.le_equiv_right hop he) href))
      (Fraction.equiv_trans (Fraction.add_comm _ _) (GeometricTail.tail_add _ _ (j+1)))
  · rw [if_neg hb]
    have hnon : 0 ≤ (GeometricTail.tailCap (Fraction.mul T (speedCap T tau B s)) (j+1)).num :=
      Fraction.nonnegative_mul _ _ d.time_nonnegative
        (speedCap_nonnegative T tau B s d.time_nonnegative d.calibration_positive d.bound_nonnegative)
    exact Fraction.le_equiv_right
      (Fraction.magnitudes.le_trans href (Fraction.le_add_nonnegative _ _ hnon))
      (GeometricTail.tail_add _ _ (j+1))

noncomputable def coefficient (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) (TimeCalibration.inverse tau ht))
    (weightedCoefficient E0 T tau L B s)

theorem coefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num)
  (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (coefficient E0 T tau L B s ht).num :=
  Fraction.nonnegative_mul _ _
    (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos))
    (weightedCoefficient_nonnegative E0 T tau L B s hE hT ht hL hB)

theorem adjacent_tail (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE)
  (j : Nat) :
  Fraction.le (FiniteEstimates.stateDistance (prefixState b o E0 T s hE (j+1))
    (prefixState b o E0 T s hE j))
    (GeometricTail.tailCap (coefficient E0 T tau L B s d.calibration_positive) (j+1)) := by
  have hC : 0 ≤ (Fraction.add (Fraction.ofInt 1)
    (TimeCalibration.inverse tau d.calibration_positive)).num :=
    Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)
  have hb := Fraction.magnitudes.le_trans
    (TimeCalibration.uncalibrated_le_distance tau d.calibration_positive _ _)
    (Fraction.mul_le_mul_nonnegative_left
      (adjacent_weighted_tail b o E0 T tau L B s hE d j) _ hC)
  apply Fraction.le_equiv_right hb
  simp only [coefficient, GeometricTail.tailCap, Fraction.equiv, Fraction.mul]
  ac_nf

noncomputable def prefixName (b : Nat → Bool) (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Conditions o E0 T tau L B s hE) :

```

```

    EndpointCauchyName where
approx := prefixState b o E0 T s hE
cauchy := by
  intro eps heps
  let A := coefficient E0 T tau L B s d.calibration_positive
  have hA := coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative
  d.calibration_positive d.lipschitz.1 d.bound_nonnegative
  refine (GeometricTail.modulus A eps,?)
  intro m n hm hn
  exact Fraction.magnitudes.lt_of_le_lt
    (GeometricTail.two_sided _ A hA (adjacent_tail b o E0 T tau L B s hE d) _ m n hm hn)
    (GeometricTail.doubleTail_lt_tolerance A eps hA heps)

theorem zero_time_prefix (b : Nat → Bool) (o : CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (hT : T.num = 0) (j : Nat) :
  stateEquiv (prefixState b o E0 T s hE j) s :=
  BoundedIteration.zero_duration_run (field o E0 hE j) (duration T j) hT s (ticks b j)

```

end NewtonLimitDynamics.Polygon.GeneralForcePrefix

NewtonLimitDynamics/Polygon/GeneralForceGrowth.lean, 6 theorems, 136 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePrefix
import BarrowLib.Polygon.CalibratedGrowth

/-- Derive the motion-sample bounds on a finite ball from regional linear
force growth and the calibrated window. The ball budget is closed before
sampling, using only initial-state magnitudes and force data. Actual and
shadow confinement then follow from the shared partial-time invariant.
Singular laws use the annular constructor rather than growth at the origin. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceGrowth
open NewtonLimitDynamics TimeSubdivision PointBounds HarmonicDyadic ForceClasses
open FiniteEstimates
open GeneralForceEndpoint GeneralForcePrecision

def shadowCap (E0 T tau : Fraction) (s : Point × Point) (ht : 0 < tau.num) : Fraction :=
  CalibratedGrowth.driftCap tau T (Fraction.mul (Fraction.ofInt 2) E0) s ht

def forceCap (E0 T tau L : Fraction) (s : Point × Point) (ht : 0 < tau.num) : Fraction :=
  Fraction.add (Fraction.mul L (shadowCap E0 T tau s ht)) (Fraction.mul (Fraction.ofInt 2) E0)

structure Data (o : CentralOracle) (E0 T tau L : Fraction) (s : Point × Point) where
  time_nonnegative : 0 ≤ T.num
  calibration_positive : 0 < tau.num
  lipschitz : LipschitzOn o.toOracle L
  window : Fraction.le
    (Fraction.mul T (TimeCalibration.rate tau L calibration_positive)) (1,2,by decide)
  ball_contained : ∀ p, Fraction.le (pointNorm p) (shadowCap E0 T tau s calibration_positive) →
    o.region p

theorem shadowCap_nonnegative (E0 T tau : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num) :
  0 ≤ (shadowCap E0 T tau s ht).num := by
  have hM := CalibratedGrowth.cap_nonnegative tau T (Fraction.mul (Fraction.ofInt 2) E0) s
  (Int.le_of_lt ht) hT (Fraction.nonnegative_mul _ _ (by decide) (Int.le_of_lt hE))
  exact Fraction.nonnegative_add _ _ hM
  (Fraction.nonnegative_mul _ _ hT
    (Fraction.nonnegative_mul _ _ hM (Int.le_of_lt tau.den_pos)))

/-- Chosen force precision gives a growth estimate only at regional points. -/
theorem field_growth (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s)
  (j : Nat) (p : Point) (hp : o.region p) :
  Fraction.le (pointNorm (field o E0 hE j p))
    (Fraction.add (Fraction.mul L (pointNorm p)) (Fraction.mul (Fraction.ofInt 2) E0)) := by
  have hzero : o.region (Fraction.ofInt 0, Fraction.ofInt 0) := d.ball_contained _ (by
    simp [Fraction.le, pointNorm, Fraction.add, Fraction.abs, Fraction.ofInt] using
      (shadowCap_nonnegative E0 T tau s hE d.time_nonnegative d.calibration_positive))
  have he := Fraction.magnitudes.lt_implies_le (precision_error o.toOracle E0 hE j)
  have ht : Fraction.le (GeometricTail.tailCap E0 j) E0 :=

```

```

    duration_le_window E0 (Int.le_of_lt hE) j
  have herr := Fraction.add_le_add (Fraction.magnitudes.le_trans he ht)
    (Fraction.magnitudes.le_trans he ht)
  have htwo : Fraction.equiv (Fraction.add E0 E0) (Fraction.mul (Fraction.ofInt 2) E0) := by
    simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt]
    simp only [show (2: Int) = 1 + 1 by rfl, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
    ac_nf
  exact Fraction.magnitudes.le_trans
    (sample_linear_growth o L d.lipschitz hzero _ p hp)
    (Fraction.add_le_add_left (Fraction.le_equiv_right herr htwo) _)

theorem forceCap_nonnegative (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s) :
  0 ≤ (forceCap E0 T tau L s d.calibration_positive).num :=
  Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ d.lipschitz.1
      (shadowCap_nonnegative E0 T tau s hE d.time_nonnegative d.calibration_positive))
    (Fraction.nonnegative_mul _ _ (by decide) (Int.le_of_lt hE))

/-- The outer budget follows from the existing calibrated window. No
membership of an iterate or shadow is included in Data. -/
def region_frame (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s) :
  RegionConfinement.Frame o.region T (forceCap E0 T tau L s d.calibration_positive)
    (Fraction.ofInt 0) (shadowCap E0 T tau s d.calibration_positive) s :=
  RegionConfinement.ball_frame o.region T _ _ s d.time_nonnegative
    (forceCap_nonnegative o E0 T tau L s hE d)
    (CalibratedGrowth.driftCap_budget tau T L (Fraction.mul (Fraction.ofInt 2) E0) s
      d.calibration_positive d.time_nonnegative d.lipschitz.1
      (Fraction.nonnegative_mul _ _ (by decide) (Int.le_of_lt hE)) d.window)
    d.ball_contained

theorem field_bound_on_ball (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s)
  (j : Nat) (p : Point)
  (hp : RegionConfinement.Band (Fraction.ofInt 0)
    (shadowCap E0 T tau s d.calibration_positive) p) :
  Fraction.le (pointNorm (field o E0 hE j p)) (forceCap E0 T tau L s d.calibration_positive) :=
  Fraction.magnitudes.le_trans
    (field_growth o E0 T tau L s hE d j p (d.ball_contained p hp.2))
    (Fraction.add_le_add_right (Fraction.mul_le_mul_nonnegative_left hp.2 L d.lipschitz.1) _)

/-- Actual iterates are bounded independently of mesh or level precision. -/
theorem run_state_bound (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s)
  (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  Fraction.le (TimeCalibration.norm tau (BoundedIteration.run (field o E0 hE j) h s n))
    (CalibratedGrowth.cap tau T (Fraction.mul (Fraction.ofInt 2) E0) s) := by
  have he := Fraction.le_equiv_left
    (Fraction.mul_equiv (Fraction.equiv_refl (Fraction.ofInt (n: Int)))
      (Fraction.abs_of_nonnegative h hh)) hn
  apply CalibratedGrowth.run_norm_le_cap_at tau d.calibration_positive _ h T L _ s
    d.lipschitz.1 (Fraction.nonnegative_mul _ _ (by decide) (Int.le_of_lt hE)) n _ he d.window
  intro k hk
  have hb := RegionConfinement.run_band o.region T _ _ _ s
    (region_frame o E0 T tau L s hE d) _ (sample_central o _)
    (field_bound_on_ball o E0 T tau L s hE d j) h hh (k+1)
    (Fraction.magnitudes.le_trans (BoundedIteration.time_monotone h hh (k+1) n (by omega)) hn)
  exact field_growth o E0 T tau L s hE d j _ (d.ball_contained _ hb.2)

/-- One finite argument covers actual and doubled-half meshes. -/
theorem bounded_samples (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s)
  (j : Nat) (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  BoundedIteration.BoundedSamples (field o E0 hE j) h s
    (forceCap E0 T tau L s d.calibration_positive) n :=
  RegionConfinement.run_bounded_samples o.region T _ _ _ s
    (region_frame o E0 T tau L s hE d) _ (sample_central o _)

```



```

    (field_bound_on_ball o E0 T tau L s hE d j) h hh n hn

/-- The existing regional construction needs only the ball force contract;
all actual, coarse and both shadow certificates are derived internally. -/
def conditions (o : CentralOracle) (E0 T tau L : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : Data o E0 T tau L s) :
  GeneralForcePrefix.Conditions o E0 T tau L
    (forceCap E0 T tau L s d.calibration_positive) s hE where
  calibration_positive := d.calibration_positive
  lipschitz := d.lipschitz
  window := d.window
  inner_radius := Fraction.ofInt 0
  outer_radius := shadowCap E0 T tau s d.calibration_positive
  frame := region_frame o E0 T tau L s hE d
  samples_on_band := field_bound_on_ball o E0 T tau L s hE d

end NewtonLimitDynamics.Polygon.GeneralForceGrowth

NewtonLimitDynamics/Polygon/RegionConfinement.lean, 16 theorems, 419 lines

import NewtonLimitDynamics.Polygon.GeneralForcePrecision
import BarrowLib.Polygon.CalibratedRefinement
import BarrowLib.Polygon.TriangleBounds

/--! Finite regional confinement before force sampling. One partial-time
invariant controls actual runs and restarted shadow cells. The only force
bound is on the named region; no whole-plane regularity, supplied motion or
completed quantity is used. Kepler sampling and localization of the existing
completion are separate remaining parts of handoff A.6. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceEndpoint
open NewtonLimitDynamics TimeSubdivision PointBounds HarmonicDyadic

def velocityCap (T B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.add (pointNorm s.2) (Fraction.mul T B)

theorem velocityCap_nonnegative (T B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hB : 0 ≤ B.num) : 0 ≤ (velocityCap T B s).num :=
  Fraction.nonnegative_add _ _ (pointNorm_nonnegative _)
  (Fraction.nonnegative_mul _ _ hT hB)

theorem duration_le_window (T : Fraction) (hT : 0 ≤ T.num) (j : Nat) :
  Fraction.le (duration T j) T := by
  have hp : (1 : Int) ≤ (2 : Int)^j := by
  have hj := two_pow_ge_succ j
  have hn : (0 : Int) ≤ (j : Int) := Int.ofNat_nonneg j
  omega
  have hm := Int.mul_le_mul_of_nonneg_left hp
  (Int.mul_nonneg hT (Int.le_of_lt T.den_pos))
  simpa only [duration, Fraction.le, Int.mul_one, Int.mul_assoc] using hm

theorem fine_time (T : Fraction) (j : Nat) :
  Fraction.equiv
    (BoundedIteration.time (duration T (j+1)) (2*blocks j)) T := by
  have hn : 2*blocks j = blocks (j+1) := by rw [blocks_succ]; omega
  rw [hn]
  exact blocks_duration T (j+1)

theorem coarse_time (T : Fraction) (j : Nat) :
  Fraction.equiv
    (BoundedIteration.time
      (Fraction.add (duration T (j+1)) (duration T (j+1))) (blocks j)) T :=
  Fraction.equiv_trans
    (Fraction.mul_equiv (Fraction.equiv_refl _) (Fraction.equiv_symm (duration_halving T j)))
    (blocks_duration T j)

end NewtonLimitDynamics.Polygon.GeneralForceEndpoint

namespace NewtonLimitDynamics.Polygon.RegionConfinement
open NewtonLimitDynamics TimeSubdivision PointBounds ForceClasses
open GeneralForceEndpoint

```

```

def Band (r R : Fraction) (p : Point) : Prop :=
  Fraction.le r (pointNorm p) ∧ Fraction.le (pointNorm p) R

/-- Geometric and small-time data, without a confinement field. For a ball,
choose r=0; an annulus uses the conserved areal product and positive speed. -/
structure Frame (region : Point → Prop) (T B r R : Fraction) (s0 : Point × Point) where
  time_nonnegative : 0 ≤ T.num
  bound_nonnegative : 0 ≤ B.num
  inner_zero_or_speed_positive : r.num = 0 ∨ 0 < (velocityCap T B s0).num
  areal_bound : Fraction.le (Fraction.mul r (velocityCap T B s0)) (CentralSchedule.momentum s0).abs
  outer_bound : Fraction.le
    (Fraction.add (pointNorm s0.1) (Fraction.mul T (velocityCap T B s0))) R
  contains_band : ∀ p, Band r R p → region p

structure Invariant (T B : Fraction) (s0 : Point × Point) (t : Fraction) (q : Point × Point) where
  velocity_bound : Fraction.le (pointNorm q.2)
    (Fraction.add (pointNorm s0.2) (Fraction.mul t B))
  position_bound : Fraction.le (pointNorm q.1)
    (Fraction.add (pointNorm s0.1) (Fraction.mul t (velocityCap T B s0)))
  areal_product : Fraction.equiv (CentralSchedule.momentum q) (CentralSchedule.momentum s0)

/-- The regular ball case permits zero areal product and zero speed. -/
def ball_frame (region : Point → Prop) (T B R : Fraction) (s0 : Point × Point)
  (hT : 0 ≤ T.num) (hB : 0 ≤ B.num)
  (ho : Fraction.le (Fraction.add (pointNorm s0.1) (Fraction.mul T (velocityCap T B s0))) R)
  (hR : ∀ p, Fraction.le (pointNorm p) R → region p) :
  Frame region T B (Fraction.ofInt 0) R s0 where
  time_nonnegative := hT
  bound_nonnegative := hB
  inner_zero_or_speed_positive := Or.inl rfl
  outer_bound := ho
  contains_band := fun p hp => hR p hp.2
  areal_bound := by
    unfold Fraction.le Fraction.mul Fraction.ofInt
    simp only [Int.zero_mul, Int.one_mul]
    exact Int.mul_nonneg (Fraction.abs_num_nonnegative _) (Int.le_of_lt (velocityCap T B s0).den_pos)

theorem initial_invariant (T B : Fraction) (s0 : Point × Point) :
  Invariant T B s0 (Fraction.ofInt 0) s0 := by
  refine {?, ?, Fraction.equiv_refl _} <=> apply Fraction.le_of_equiv <=>
  simp only [Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.zero_mul, Int.mul_zero, Int.one_mul, Int.mul_one, Int.add_zero] <=> ac_nf

/-- Determinant conservation supplies the lower radius; no division by a
possibly zero speed is made in the ball case. -/
theorem band_of_bounds (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (p v : Point) (hp : Fraction.le (pointNorm p) R)
  (hv : Fraction.le (pointNorm v) (velocityCap T B s0))
  (hm : Fraction.equiv (det p v) (CentralSchedule.momentum s0)) : Band r R p := by
  refine {?, hp}
  rcases d.inner_zero_or_speed_positive with hz | hv
  · unfold Fraction.le
    simp only [hz, Int.zero_mul]
    exact Int.mul_nonneg (pointNorm_nonnegative p) (Int.le_of_lt r.den_pos)
  · exact TriangleBounds.radius_lower_of_areal_bound p v r hv hv
    (Fraction.le_equiv_right d.areal_bound (Fraction.equiv_symm (Fraction.abs_equiv hm)))

theorem invariant_band (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (t : Fraction) (q : Point × Point) (ht : Fraction.le t T)
  (hq : Invariant T B s0 t q) : Band r R q.1 := by
  have hp := Fraction.magnitudes.le_trans hq.position_bound
  (Fraction.add_le_add_left
    (Fraction.mul_le_mul_nonnegative ht (velocityCap T B s0)
      (velocityCap_nonnegative T B s0 d.time_nonnegative d.bound_nonnegative)) _)
  have hv := Fraction.magnitudes.le_trans hq.velocity_bound
  (Fraction.add_le_add_left (Fraction.mul_le_mul_nonnegative ht B d.bound_nonnegative) _)
  exact band_of_bounds region T B r R s0 d q.1 q.2

```

```

(Fraction.magnitudes.le_trans hp d.outer_bound) hv hq.areal_product

/-- The drift arrives inside the region before its force value is evaluated. -/
theorem arrival_band (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (t h : Fraction) (q : Point × Point)
  (hh : 0 ≤ h.num) (ht : Fraction.le (Fraction.add t h) T)
  (hq : Invariant T B s0 t q) : Band r R (FiniteEstimates.cell a h q).1 := by
have hprior := Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative t h hh) ht
have hv := Fraction.magnitudes.le_trans hq.velocity_bound
(Fraction.add_le_add_left (Fraction.mul_le_mul_nonnegative hprior B d.bound_nonnegative) _)
have hpos := Fraction.le_equiv_right (FiniteEstimates.cell_position_growth a h q)
(Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv
(Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl _)))
have hp := Fraction.magnitudes.le_trans hpos
(Fraction.add_le_add hq.position_bound (Fraction.mul_le_mul_nonnegative_left hv h hh))
have he : Fraction.equiv
(Fraction.add (Fraction.add (pointNorm s0.1) (Fraction.mul t (velocityCap T B s0)))
(Fraction.mul h (velocityCap T B s0)))
(Fraction.add (pointNorm s0.1) (Fraction.mul (Fraction.add t h) (velocityCap T B s0))) := by
simp only [Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
ac_nf
have hp' := Fraction.magnitudes.le_trans (Fraction.le_equiv_right hp he)
(Fraction.add_le_add_left
(Fraction.mul_le_mul_nonnegative ht (velocityCap T B s0)
(velocityCap_nonnegative T B s0 d.time_nonnegative d.bound_nonnegative)) _)
have hm := Fraction.equiv_trans (CentralSchedule.det_drift q.1 q.2 h) hq.areal_product
exact band_of_bounds region T B r R s0 d _ q.2
(Fraction.magnitudes.le_trans hp' d.outer_bound) hv hm

/-- One invariant step. The region bound is used only after arrival_band. -/
theorem advance (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (hc : CentralSchedule.central a)
  (hb : ∀ p, Band r R p → Fraction.le (pointNorm (a p)) B)
  (t h : Fraction) (q : Point × Point) (hh : 0 ≤ h.num)
  (ht : Fraction.le (Fraction.add t h) T) (hq : Invariant T B s0 t q) :
Invariant T B s0 (Fraction.add t h) (FiniteEstimates.cell a h q) := by
have ha := hb _ (arrival_band region T B r R s0 d a t h q hh ht hq)
have hv := Fraction.le_equiv_right (FiniteEstimates.cell_velocity_growth a h B q ha)
(Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv
(Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl _)))
have hv' := Fraction.magnitudes.le_trans hv
(Fraction.add_le_add_right hq.velocity_bound (Fraction.mul h B))
have hvq : Fraction.le (pointNorm q.2) (velocityCap T B s0) :=
Fraction.magnitudes.le_trans hq.velocity_bound
(Fraction.add_le_add_left (Fraction.mul_le_mul_nonnegative
(Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative t h hh) ht)
B d.bound_nonnegative) _)
have hp := Fraction.le_equiv_right (FiniteEstimates.cell_position_growth a h q)
(Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv
(Fraction.abs_of_nonnegative h hh) (Fraction.equiv_refl _)))
have hp' := Fraction.magnitudes.le_trans hp
(Fraction.add_le_add hq.position_bound (Fraction.mul_le_mul_nonnegative_left hvq h hh))
refine {_, _, Fraction.equiv_trans (CentralSchedule.cell_momentum a hc h q) hq.areal_product}
· apply Fraction.le_equiv_right hv'
simp only [Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
ac_nf
· apply Fraction.le_equiv_right hp'
simp only [Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
ac_nf

/-- All finite mesh levels inherit the regional bound without BoundedSamples
as an input. The force need not be continuous for this confinement result. -/
theorem run_invariant (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (hc : CentralSchedule.central a)
  (hb : ∀ p, Band r R p → Fraction.le (pointNorm (a p)) B)
  (h : Fraction) (hh : 0 ≤ h.num) :
(n : Nat) → Fraction.le (BoundedIteration.time h n) T →

```

```

Invariant T B s0 (BoundedIteration.time h n) (BoundedIteration.run a h s0 n)
| 0, _ => by
  have hi := initial_invariant T B s0
  have he : Fraction.equiv (Fraction.ofInt 0) (BoundedIteration.time h 0) := by
    simp only [BoundedIteration.time, Fraction.equiv, Fraction.ofInt, Fraction.mul,
      Int.natCast_zero, Int.zero_mul, Int.mul_zero]
  exact (Fraction.le_equiv_right hi.velocity_bound
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv he (Fraction.equiv_refl _))),
    Fraction.le_equiv_right hi.position_bound
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv he (Fraction.equiv_refl _))),
    hi.areal_product)
| n+1, ht => by
  have he : Fraction.equiv (Fraction.add (BoundedIteration.time h n) h)
    (BoundedIteration.time h (n+1)) := by
    simp only [BoundedIteration.time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
      Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
  ac_nf
  have hi := run_invariant region T B r R s0 d a hc hb h hh n
    (Fraction.magnitudes.le_trans (BoundedIteration.time_monotone h hh n (n+1) (by omega)) ht)
  have hs := advance region T B r R s0 d a hc hb _ h _ hh
    (Fraction.le_equiv_left he ht) hi
  refine (??, ??, hs.areal_product)
  · exact Fraction.le_equiv_right hs.velocity_bound
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv he (Fraction.equiv_refl _)))
  · exact Fraction.le_equiv_right hs.position_bound
    (Fraction.add_equiv (Fraction.equiv_refl _) (Fraction.mul_equiv he (Fraction.equiv_refl _)))

theorem run_band (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (hc : CentralSchedule.central a)
  (hb : ∀ p, Band r R p → Fraction.le (pointNorm (a p)) B)
  (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  Band r R (BoundedIteration.run a h s0 n).1 :=
invariant_band region T B r R s0 d _ _ hn
  (run_invariant region T B r R s0 d a hc hb h hh n hn)

/-- Bounds at every actual sample are a conclusion of confinement, never
the induction hypothesis used to establish it. -/
theorem run_bounded_samples (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (hc : CentralSchedule.central a)
  (hb : ∀ p, Band r R p → Fraction.le (pointNorm (a p)) B)
  (h : Fraction) (hh : 0 ≤ h.num) (n : Nat)
  (hn : Fraction.le (BoundedIteration.time h n) T) :
  BoundedIteration.BoundedSamples a h s0 B n := by
intro i hi
exact hb_ (run_band region T B r R s0 d a hc hb h hh (i+1)
  (Fraction.magnitudes.le_trans (BoundedIteration.time_monotone h hh (i+1) n (by omega)) hn))

/-- Both coarse-field shadow arrivals are confined. Their partial-time speed
budgets account for the first shadow kick before the second arrival. -/
theorem shadow_bands (region : Point → Prop) (T B r R : Fraction)
  (s0 : Point × Point) (d : Frame region T B r R s0)
  (a : Point → Point) (hc : CentralSchedule.central a)
  (hb : ∀ p, Band r R p → Fraction.le (pointNorm (a p)) B)
  (m k : Nat) (hk : k < HarmonicDyadic.blocks m) :
  Band r R (FiniteEstimates.cell a (HarmonicDyadic.duration T (m+1))
    (FiniteAccumulation.coarseAt a (HarmonicDyadic.duration T (m+1)) s0 k)).1 ∧
  Band r R (FiniteEstimates.twoHalf a (HarmonicDyadic.duration T (m+1))
    (FiniteAccumulation.coarseAt a (HarmonicDyadic.duration T (m+1)) s0 k)).1 := by
let h := HarmonicDyadic.duration T (m+1)
let t := BoundedIteration.time (Fraction.add h h) k
let q := FiniteAccumulation.coarseAt a h s0 k
have hh : 0 ≤ h.num := d.time_nonnegative
have hfull : 0 ≤ (Fraction.add h h).num := Fraction.nonnegative_add _ _ hh hh
have hend := Fraction.magnitudes.le_trans
  (BoundedIteration.time_monotone (Fraction.add h h) hfull (k+1)
    (HarmonicDyadic.blocks m) (by omega)) (Fraction.le_of_equiv (coarse_time T m))
have he : Fraction.equiv (Fraction.add (Fraction.add t h) h)

```

```

    (BoundedIteration.time (Fraction.add h h) (k+1)) := by
    simp only [t, BoundedIteration.time, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt,
    Int.natCast_add, Int.natCast_one, Int.add_mul, Int.mul_add, Int.one_mul, Int.mul_one]
    ac_nf
    have hsecond := Fraction.le_equiv_left he hend
    have hfirst := Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative (Fraction.add t h) h hh)
    ↪ hsecond
    have hprior := Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative t h hh) hfirst
    have hq : Invariant T B s0 t q := by
    dsimp only [q, t]
    rw [CalibratedRefinement.coarseAt_eq_run]
    exact run_invariant region T B r R s0 d a hc hb (Fraction.add h h) hfull k hprior
    have h1 := advance region T B r R s0 d a hc hb t h q hh hfirst hq
    have h2 := advance region T B r R s0 d a hc hb (Fraction.add t h) h
    (FiniteEstimates.cell a h q) hh hsecond h1
    exact {invariant_band region T B r R s0 d __ hfirst h1,
    invariant_band region T B r R s0 d __ hsecond h2}

/-- Regional one-cell comparison: certify both drift arrivals first, then
use local Lipschitz comparison and the calibrated pointwise primitive. -/
theorem sampled_cell_comparison (o : CentralOracle) (T B r R : Fraction)
(s0 : Point × Point) (d : Frame o.region T B r R s0)
(L tau : Fraction) (hL : Lipschitz0n o.toOracle L) (htau : 0 < tau.num)
(i j : Nat) (hij : i ≤ j) (t u h : Fraction) (q z : Point × Point)
(hh : 0 ≤ h.num) (ht : Fraction.le (Fraction.add t h) T)
(hu : Fraction.le (Fraction.add u h) T)
(hq : Invariant T B s0 t q) (hz : Invariant T B s0 u z) :
Fraction.le (TimeCalibration.distance tau
(FiniteEstimates.cell (o.sample i) h q) (FiniteEstimates.cell (o.sample j) h z))
(Fraction.add
(Fraction.mul (TimeCalibration.amplification tau h L htau)
(TimeCalibration.distance tau q z))
(Fraction.mul (Fraction.mul tau h. abs)
(Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i)))) := by
have hp := d.contains_band _ (arrival_band o.region T B r R s0 d
(o.sample i) t h q hh ht hq)
have hr := d.contains_band _ (arrival_band o.region T B r R s0 d
(o.sample j) u h z hh hu hz)
exact TimeCalibration.cell_amplification_at tau htau (o.sample i) (o.sample j)
h L _ q z hL.1 (samples_comparison_contract o.toOracle L hL i j hij __ hp hr)

/-- Actual paired refinement on a certified band. Both fine arrivals, both
coarse-field shadow arrivals and the full coarse arrival are certified before
any of the four local force comparisons is used. -/
theorem sampled_refinement_bound (o : CentralOracle) (T B r R : Fraction)
(s0 : Point × Point) (d : Frame o.region T B r R s0)
(L tau : Fraction) (hL : Lipschitz0n o.toOracle L) (htau : 0 < tau.num)
(i j : Nat) (hij : i ≤ j)
(hbi : ∀ p, Band r R p → Fraction.le (pointNorm (o.sample i p)) B)
(hbj : ∀ p, Band r R p → Fraction.le (pointNorm (o.sample j p)) B) (m n : Nat) (hn : n ≤
↪ HarmonicDyadic.blocks m)
(hs : TimeCalibration.Window tau (HarmonicDyadic.duration T (m+1)) L htau (2*n)) :
Fraction.le (TimeCalibration.distance tau
(FiniteAccumulation.fineAt (o.sample j) (HarmonicDyadic.duration T (m+1)) s0 n)
(FiniteAccumulation.coarseAt (o.sample i) (HarmonicDyadic.duration T (m+1)) s0 n))
(Fraction.mul (Fraction.ofInt (2*(n : Int)))
(CalibratedRefinement.blockSource tau (HarmonicDyadic.duration T (m+1)) L
(Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i))
B (velocityCap T B s0) htau)) := by
let h := HarmonicDyadic.duration T (m+1)
let E := Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i)
have hh : 0 ≤ h.num := d.time_nonnegative
have hfull : 0 ≤ (Fraction.add h h).num := Fraction.nonnegative_add __ hh hh
have hf : ∀ k, k ≤ 2*HarmonicDyadic.blocks m →
Band r R (BoundedIteration.run (o.sample j) h s0 k).1 := by
intro k hk
exact run_band o.region T B r R s0 d _ (sample_central o j) hbh h hh k
(Fraction.magnitudes.le_trans
(BoundedIteration.time_monotone h hh k (2*HarmonicDyadic.blocks m) hk)
(Fraction.le_of_equiv (fine_time T m)))

```

```

have hct :  $\forall k, k \leq \text{HarmonicDyadic.blocks } m \rightarrow$ 
  Fraction.le (BoundedIteration.time (Fraction.add h h) k) T := by
    intro k hk
    exact Fraction.magnitudes.le_trans
      (BoundedIteration.time_monotone _ hfull k (HarmonicDyadic.blocks m) hk)
      (Fraction.le_of_equiv (coarse_time T m))
have hc :  $\forall k, k \leq \text{HarmonicDyadic.blocks } m \rightarrow$ 
  Band r R (BoundedIteration.run (o.sample i) (Fraction.add h h) s0 k).1 := by
    intro k hk
    exact run_band o.region T B r R s0 d _ (sample_central o i) hbi
      _ hfull k (hct k hk)
have hcross :  $\forall p q, o.\text{region } p \rightarrow o.\text{region } q \rightarrow$ 
  Fraction.le (FiniteEstimates.pointDistance (o.sample j p) (o.sample i q))
    (Fraction.add (Fraction.mul L (FiniteEstimates.pointDistance p q)) E) := by
    intro p q hp hq
    have he := samples_comparison_contract o.toOracle L hL i j hij q p hq hp
    exact Fraction.le_equiv_right
      (Fraction.le_equiv_left (FiniteEstimates.pointDistance_symm _ _) he)
      (Fraction.add_equiv
        (Fraction.mul_equiv_left L (FiniteEstimates.pointDistance_symm _ _))
        (Fraction.equiv_refl _))
apply CalibratedRefinement.actual_uniform_error_at tau htau (o.sample j) (o.sample i)
  h L E B (velocityCap T B s0) s0 hL.1
  (Fraction.nonnegative_add
    (Fraction.nonnegative_add _ _ (o.error_nonnegative i) (o.error_nonnegative i))
    (o.error_nonnegative i)) d.bound_nonnegative
  (velocityCap_nonnegative T B s0 d.time_nonnegative d.bound_nonnegative) n hs
· intro k hk
  have hkm :  $k < \text{HarmonicDyadic.blocks } m := \text{by } \omega$ 
  obtain (hb1,hb2) := shadow_bands o.region T B r R s0 d _
    (sample_central o i) hbi m k hkm
  have ha1 : Band r R (FiniteEstimates.cell (o.sample j) h
    (FiniteAccumulation.fineAt (o.sample j) h s0 k)).1 := by
    simpa only [CalibratedRefinement.fineAt_eq_run,BoundedIteration.run] using
      hf (2*k+1) (by  $\omega$ )
  have ha2 : Band r R (FiniteEstimates.twoHalf (o.sample j) h
    (FiniteAccumulation.fineAt (o.sample j) h s0 k)).1 := by
    simpa only [CalibratedRefinement.fineAt_eq_run,FiniteEstimates.twoHalf,
      BoundedIteration.run] using hf (2*k+2) (by  $\omega$ )
  have hc1 : Band r R (FiniteEstimates.oneFull (o.sample i) h
    (FiniteAccumulation.coarseAt (o.sample i) h s0 k)).1 := by
    simpa only [CalibratedRefinement.coarseAt_eq_run,FiniteEstimates.oneFull,
      BoundedIteration.run] using hc (k+1) (by  $\omega$ )
  exact (hcross _ _ (d.contains_band _ ha1) (d.contains_band _ hb1),
    hcross _ _ (d.contains_band _ ha2) (d.contains_band _ hb2),
    samples_comparison_contract o.toOracle L hL i i (Nat.le_refl _) _ _
      (d.contains_band _ hb1) (d.contains_band _ hc1),
    samples_comparison_contract o.toOracle L hL i i (Nat.le_refl _) _ _
      (d.contains_band _ hb2) (d.contains_band _ hc1))
· intro k hk
  exact hbi _ (shadow_bands o.region T B r R s0 d _
    (sample_central o i) hbi m k (by  $\omega$ )).1
· intro k hk
  have hv := (run_invariant o.region T B r R s0 d _ (sample_central o i) hbi
    (Fraction.add h h) hfull k (hct k (by  $\omega$ ))).velocity_bound
  simpa only [CalibratedRefinement.coarseAt_eq_run] using
    Fraction.magnitudes.le_trans hv (Fraction.add_le_add_left
      (Fraction.mul_le_mul_nonnegative (hct k (by  $\omega$ )) B d.bound_nonnegative) _)

/-- Equivalent represented durations retain their own actual force samples.
Membership of both arrival families is derived from the same regional frame. -/
theorem sampled_equivalent_duration_bound (o : CentralOracle) (T B r R : Fraction)
  (s0 : Point  $\times$  Point) (f : Frame o.region T B r R s0)
  (L tau : Fraction) (hL : LipschitzOn o.toOracle L) (htau :  $0 < \text{tau.num}$ )
  (i j : Nat) (hij :  $i \leq j$ )
  (hbi :  $\forall p, \text{Band } r R p \rightarrow \text{Fraction.le } (\text{pointNorm } (o.sample i p)) B$ )
  (hbj :  $\forall p, \text{Band } r R p \rightarrow \text{Fraction.le } (\text{pointNorm } (o.sample j p)) B$ ) (d e : Fraction) (hde :
     $\hookleftarrow \text{Fraction.equiv } d e$ )
  (hd :  $0 \leq d.\text{num}$ ) (he :  $0 \leq e.\text{num}$ ) (n : Nat)
  (hdt : Fraction.le (BoundedIteration.time d n) T)

```

```

(het : Fraction.le (BoundedIteration.time e n) T)
(hs : TimeCalibration.Window tau d L htau n) :
Fraction.le (TimeCalibration.distance tau
  (BoundedIteration.run (o.sample i) d s0 n) (BoundedIteration.run (o.sample j) e s0 n))
  (Fraction.mul (Fraction.ofInt (2*(n : Int)))
    (Fraction.mul (Fraction.mul tau d.abs)
      (Fraction.add (Fraction.add (o.error i) (o.error i)) (o.error i)))) := by
apply EquivalentDuration.run_uniform_error_at tau htau (o.sample i) (o.sample j)
d e L _ s0 hde hL.1
(Fraction.nonnegative_add _ _
  (Fraction.nonnegative_add _ _ (o.error_nonnegative i) (o.error_nonnegative i))
  (o.error_nonnegative i)) n hs
intro k hk
have hp := run_band o.region T B r R s0 f _ (sample_central o i) hbi d hd (k+1)
(Fraction.magnitudes.le_trans (BoundedIteration.time_monotone d hd (k+1) n (by omega)) hdt)
have hq := run_band o.region T B r R s0 f _ (sample_central o j) hbje he (k+1)
(Fraction.magnitudes.le_trans (BoundedIteration.time_monotone e he (k+1) n (by omega)) het)
exact samples_comparison_contract o.toOracle L hL i j hij _ _
(f.contains_band _ hp) (f.contains_band _ hq)
end NewtonLimitDynamics.Polygon.RegionConfinement

```

NewtonLimitDynamics/Polygon/GeneralForceTime.lean, 21 theorems, 308 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePrefix
import BarrowLib.Polygon.BinaryEndpoints
import BarrowLib.Polygon.FiniteSequenceGap
import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.TailValues
import BarrowLib.Polygon.SampledValues
import BarrowLib.Polygon.CompletionGeometry

/-! A local general sampled central-force time map, constructed from actual
prefixes. Same-grid drift/kick bounds prove representative invariance before
the time quotient is lifted. This is a binary-time domain, not a supplied
real trajectory or a derivative/ODE identification. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceTime
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicTimeComparison
open GeneralForceEndpoint GeneralForcePrefix CauchyValues BinaryTime
open HarmonicTimeRealization

noncomputable def stateTimeFactor (T tau B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction :=
  Fraction.mul (Fraction.add (Fraction.ofInt 1) (TimeCalibration.inverse tau ht))
  (speedCap T tau B s)

theorem stateTimeFactor_nonnegative (T tau B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hB : 0 ≤ B.num) :
  0 ≤ (stateTimeFactor T tau B s ht).num :=
  Fraction.nonnegative_mul _ _
  (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos))
  (speedCap_nonnegative T tau B s hT ht hB)

theorem count_gap (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (j n k : Nat) (hnk : n+k ≤ blocks j) :
  Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j (n+k))
    (countState o E0 T s hE j n))
    (Fraction.mul (Fraction.ofInt (k : Int))
      (Fraction.mul (duration T j) (speedCap T tau B s))) :=
  FiniteSequenceGap.finite_gap (TimeCalibration.distance tau) (TimeCalibration.distance_self_zero tau)
  (TimeCalibration.distance_triangle tau d.calibration_positive)
  (countState o E0 T s hE j) (blocks j) (Fraction.mul (duration T j) (speedCap T tau B s))
  (fun i hi => count_step_bound o E0 T tau L B s hE d j i hi) n k hnk

theorem count_ordered_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (j n k : Nat) (hnk : n+k ≤ blocks j) :
  Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j (n+k))

```

```

(countState o E0 T s hE j n))
(Fraction.mul (durationDifference (countTime T j n) (countTime T j (n+k))))).abs
(speedCap T tau B s)) := by
apply Fraction.le_equiv_right (count_gap o E0 T tau L B s hE d j n k hnk)
exact Fraction.equiv_trans (Fraction.equiv_symm (Fraction.mul_assoc _ _ _))
(Fraction.mul_equiv (Fraction.equiv_symm (countTime_abs_difference T j n k d.time_nonnegative))
(Fraction.equiv_refl _))

theorem count_same_grid_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
(s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
(j m n : Nat) (hm : m ≤ blocks j) (hn : n ≤ blocks j) :
Fraction.le (TimeCalibration.distance tau (countState o E0 T s hE j m)
(countState o E0 T s hE j n))
(Fraction.mul (durationDifference (countTime T j n) (countTime T j m)).abs
(speedCap T tau B s)) := by
rcases Nat.le_total n m with hnm | hmn
· have he : n+(m-n)=m := by omega
have hb := count_ordered_bound o E0 T tau L B s hE d j n (m-n) (by omega)
simpa only [he] using hb
· have he : m+(n-m)=n := by omega
have hb := count_ordered_bound o E0 T tau L B s hE d j m (n-m) (by omega)
rw [he] at hb
exact Fraction.le_equiv_right
(Fraction.le_equiv_left (TimeCalibration.distance_symm tau _ _) hb)
(Fraction.mul_equiv (durationDifference_abs_symm _ _) (Fraction.equiv_refl _))

/-- Actual same-family prefixes are Lipschitz in their actual grid times. -/
theorem prefix_time_bound (b c : Nat → Bool) (o : ForceClasses.CentralOracle)
(E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
(d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
Fraction.le (distance (prefixState b o E0 T s hE j) (prefixState c o E0 T s hE j))
(Fraction.mul (distance (timeState b T j) (timeState c T j))
(stateTimeFactor T tau B s d.calibration_positive)) := by
let C := Fraction.ofInt 1) (TimeCalibration.inverse tau d.calibration_positive)
have hC : 0 ≤ C.num := Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)
have hb := Fraction.magnitudes.le_trans (TimeCalibration.uncalibrated_le_distance tau
↪ d.calibration_positive _ _)
(Fraction.mul_le_mul_nonnegative_left
(count_same_grid_bound o E0 T tau L B s hE d j (ticks b j) (ticks c j)
(ticks_le_blocks b j) (ticks_le_blocks c j)) C hC)
apply Fraction.le_equiv_right hb
exact Fraction.equiv_trans (by
simp only [C, stateTimeFactor, countTime, timeApprox, Fraction.equiv, Fraction.mul]
ac_nf) (Fraction.mul_equiv
(Fraction.equiv_symm (timeState_distance b c T j)) (Fraction.equiv_refl _))

theorem address_state_equiv (b c : Nat → Bool) (o : ForceClasses.CentralOracle)
(E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
(d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
(hbc : AddressEquiv T d.time_nonnegative b c) :
NameEquiv (prefixName b o E0 T tau L B s hE d) (prefixName c o E0 T tau L B s hE d) := by
intro eps heps
let C := stateTimeFactor T tau B s d.calibration_positive
have hC := stateTimeFactor_nonnegative T tau B s d.time_nonnegative d.calibration_positive
↪ d.bound_nonnegative
obtain (N,hN) := hbc (factorDelta C eps hC) (factorDelta_positive C eps hC heps)
refine (N,fun j hj => ?_)
exact Fraction.magnitudes.lt_of_le_lt (prefix_time_bound b c o E0 T tau L B s hE d j)
(factor_control C eps (distance (timeState b T j) (timeState c T j)) hC
(stateNorm_nonnegative _) (hN j hj))

noncomputable def gammaValue (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
(s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) :
BinaryTime T d.time_nonnegative → Value :=
Quotient.lift (fun b => realize (prefixName b o E0 T tau L B s hE d))
(fun b c hbc => Quotient.sound (address_state_equiv b c o E0 T tau L B s hE d hbc))

/-- The inner and outer radii hold at every actual prefix vertex. -/
theorem prefix_band (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
(s : Point × Point) (hE : 0 < E0.num)

```



```

    (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
    RegionConfinement.Band d.inner_radius d.outer_radius (prefixState b o E0 T s hE j).1 :=
    d.toConditions.run_band j (duration T j) d.time_nonnegative (ticks b j)
    (count_time_le T d.time_nonnegative j (ticks b j) (ticks_le_blocks b j))

/-- Actual prefix membership is derived from the central confinement frame. -/
theorem prefix_region (b : Nat → Bool) (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
  o.region (prefixState b o E0 T s hE j).1 :=
  d.frame.contains_band _ (prefix_band b o E0 T tau L B s hE d j)

/-- The constructed curve has a regional representative at every time. -/
theorem gamma_admissible (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (x : BinaryTime T d.time_nonnegative) :
  SampledValues.Admissible (fun q => o.region q.1) (gammaValue o E0 T tau L B s hE d x) := by
  induction x using Quotient.inductionOn with
  | _ b => exact (prefixName b o E0 T tau L B s hE d, rfl,
    prefix_region b o E0 T tau L B s hE d)

/-- Every constructed curve position lies in the closed coordinate band.
The lower radius is stated by exclusion of smaller closed balls, without a
new completed magnitude or a supplied confinement hypothesis. -/
theorem gamma_band (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (x : BinaryTime T d.time_nonnegative) :
  Within (PositionValues.positionValue (gammaValue o E0 T tau L B s hE d x))
    (embed (PositionValues.zeroPoint, PositionValues.zeroPoint)) d.outer_radius ∧
  ∀ D, Within (PositionValues.positionValue (gammaValue o E0 T tau L B s hE d x))
    (embed (PositionValues.zeroPoint, PositionValues.zeroPoint)) D →
    Fraction.le d.inner_radius D := by
  induction x using Quotient.inductionOn with
  | _ b =>
    exact CompletionGeometry.position_band_realize
      (prefixName b o E0 T tau L B s hE d) d.inner_radius d.outer_radius
      (prefix_band b o E0 T tau L B s hE d)

/-- Changing the calibration or verified bounds keeps the same actual family. -/
theorem gamma_conditions_independent (o : ForceClasses.CentralOracle)
  (E0 T tau L B tau' L' B' : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (e : GeneralForcePrefix.Conditions o E0 T tau' L' B' s hE)
  (x : BinaryTime T d.time_nonnegative) :
  gammaValue o E0 T tau L B s hE d x = gammaValue o E0 T tau' L' B' s hE e x := by
  induction x using Quotient.inductionOn with
  | _ b =>
    apply Quotient.sound
    intro eps heps
    refine {0, fun j _ => ?_}
    exact distance_self_lt (prefixState b o E0 T s hE j) eps heps

theorem gammaValue_address (b : Nat → Bool) (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) :
  gammaValue o E0 T tau L B s hE d (Quotient.mk _ b) =
  realize (prefixName b o E0 T tau L B s hE d) := rfl

/-- A uniform-in-address geometric bound on actual prefix vertices. -/
theorem prefix_value_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Within (embed (prefixState b o E0 T s hE m))
    (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
    (GeometricTail.tailCap (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) m) :=
  TailValues.approximant_bound (prefixName b o E0 T tau L B s hE d)
  (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive)
  (GeneralForcePrefix.coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative)

```

```

    d.calibration_positive d.lipschitz.1 d.bound_nonnegative)
  (GeneralForcePrefix.adjacent_tail b o E0 T tau L B s hE d) m

theorem prefix_uniform_convergence (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (embed (prefixState b o E0 T s hE m))
      (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b)) eps := by
  have hA := GeneralForcePrefix.coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative
    d.calibration_positive d.lipschitz.1 d.bound_nonnegative
  obtain ⟨N, hN⟩ := duration_eventually_small
    (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) eps hA heps
  refine ⟨N, fun m hm b => ?_⟩
  exact within_mono _ _ _ (Fraction.magnitudes.lt_implies_le (hN m hm))
    (prefix_value_bound b o E0 T tau L B s hE d m)

theorem gamma_within (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (x y : BinaryTime T d.time_nonnegative) (R : Fraction)
  (hxy : TimeWithin T d.time_nonnegative x y R) :
  Within (gammaValue o E0 T tau L B s hE d x) (gammaValue o E0 T tau L B s hE d y)
    (Fraction.mul R (stateTimeFactor T tau B s d.calibration_positive)) := by
  induction x using Quotient.inductionOn with
  | _ b =>
    induction y using Quotient.inductionOn with
    | _ c =>
      change NameBound (prefixName b o E0 T tau L B s hE d)
        (prefixName c o E0 T tau L B s hE d)
        (Fraction.mul R (stateTimeFactor T tau B s d.calibration_positive))
      exact nameBound_scale (prefixName b o E0 T tau L B s hE d)
        (prefixName c o E0 T tau L B s hE d)
        (BinaryTime.timeName b T d.time_nonnegative) (BinaryTime.timeName c T d.time_nonnegative)
        (stateTimeFactor T tau B s d.calibration_positive) R
        (stateTimeFactor_nonnegative T tau B s d.time_nonnegative d.calibration_positive
          ↪ d.bound_nonnegative)
        (fun j => prefix_time_bound b c o E0 T tau L B s hE d j) hxy

noncomputable def timeTolerance (T tau B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hB : 0 ≤ B.num) (eps : Fraction) : Fraction :=
  factorDelta (stateTimeFactor T tau B s ht) eps.half (stateTimeFactor_nonnegative T tau B s ht ht hB)

theorem timeTolerance_positive (T tau B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hB : 0 ≤ B.num) (eps : Fraction) (heps : 0 < eps.num) :
  0 < (timeTolerance T tau B s ht ht hB eps).num := factorDelta_positive _ _ _ heps

theorem gamma_uniform_continuity (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (eps : Fraction) (heps : 0 < eps.num) (x y : BinaryTime T d.time_nonnegative)
  (hxy : TimeWithin T d.time_nonnegative x y
    (timeTolerance T tau B s d.time_nonnegative d.calibration_positive d.bound_nonnegative eps)) :
  Within (gammaValue o E0 T tau L B s hE d x) (gammaValue o E0 T tau L B s hE d y) eps.half :=
  within_mono _ _ _ (factor_delta_weak _ _
    (stateTimeFactor_nonnegative T tau B s d.time_nonnegative d.calibration_positive d.bound_nonnegative)
    (by simp only [Fraction.half] using Int.le_of_lt heps))
    (gamma_within o E0 T tau L B s hE d x y _ hxy)

noncomputable def gammaPosition (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (x : BinaryTime T d.time_nonnegative) : PositionValues.PositionValue :=
  PositionValues.asPosition (gammaValue o E0 T tau L B s hE d x)

theorem gammaPosition_within (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (x y : BinaryTime T d.time_nonnegative) (R : Fraction)
  (hxy : TimeWithin T d.time_nonnegative x y R) :
  Within (gammaPosition o E0 T tau L B s hE d x).val
    (gammaPosition o E0 T tau L B s hE d y).val
    (Fraction.mul R (stateTimeFactor T tau B s d.calibration_positive)) :=
  PositionValues.positionValue_within _ _ _ (gamma_within o E0 T tau L B s hE d x y R hxy)

```

```

theorem left_endpoint_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) :
  gammaValue o E0 T tau L B s hE d (leftTime T d.time_nonnegative) = embed s := by
  apply Quotient.sound
  intro eps heps
  refine (0, fun j _ => ?_)
  change Fraction.lt (distance (prefixState leftAddress o E0 T s hE j) s) eps
  change Fraction.lt (distance
    (BoundedIteration.run (field o E0 hE j) (duration T j) s (ticks leftAddress j)) s) eps
  rw [show ticks leftAddress j = 0 from all_zero_ticks j]
  exact distance_self_lt s eps heps

theorem zero_time_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (hT : T.num = 0) (x : BinaryTime T d.time_nonnegative) :
  gammaValue o E0 T tau L B s hE d x = embed s := by
  induction x using Quotient.inductionOn with
  | _ b =>
    apply Quotient.sound
    intro eps heps
    refine (0, fun j _ => ?_)
    exact Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv ((distance_zero_iff_stateEquiv _ _).mpr
        (zero_time_prefix b o E0 T s hE hT j))) ((Fraction.positive_iff_zero_lt eps).mp heps)

/-- General E/G agreement at the end of the constructed window. -/
theorem right_endpoint_value (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) :
  gammaValue o E0 T tau L B s hE d (rightTime T d.time_nonnegative) =
  endpointValue o E0 T tau L B s hE d.toConditions := by
  apply Quotient.sound
  intro eps heps
  let C := stateTimeFactor T tau B s d.calibration_positive
  have hC := stateTimeFactor_nonnegative T tau B s d.time_nonnegative d.calibration_positive
  ↪ d.bound_nonnegative
  obtain (N, hN) := duration_eventually_small T (factorDelta C eps hC)
  d.time_nonnegative (factorDelta_positive C eps hC heps)
  refine (N, fun j hj => ?_)
  have hop := count_step_bound o E0 T tau L B s hE d j (ticks rightAddress j) (ticks_lt_blocks
    ↪ rightAddress j)
  rw [right_ticks] at hop
  have hcal := Fraction.le_equiv_left (TimeCalibration.distance_symm tau _ _) hop
  have hgauge := Fraction.magnitudes.le_trans
    (TimeCalibration.uncalibrated_le_distance tau d.calibration_positive _ _)
    (Fraction.mul_le_mul_nonnegative_left hcal
      (Fraction.add (Fraction.ofInt 1) (TimeCalibration.inverse tau d.calibration_positive))
      (Fraction.nonnegative_add _ _ (by decide) (Int.le_of_lt tau.den_pos)))
  have hb := Fraction.le_equiv_right hgauge (by
    change Fraction.equiv _ (Fraction.mul (duration T j) C)
    simp only [C, stateTimeFactor, Fraction.equiv, Fraction.mul]
    ac_nf)
  exact Fraction.magnitudes.lt_of_le_lt hb
    (factor_control C eps (duration T j) hC d.time_nonnegative (hN j hj))
end NewtonLimitDynamics.Polygon.GeneralForceTime

```

NewtonLimitDynamics/Polygon/HarmonicGeneralTime.lean, 5 theorems, 65 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceTime
import NewtonLimitDynamics.Polygon.HarmonicGeneralEndpoint
import NewtonLimitDynamics.Polygon.HarmonicTimeRealization
import NewtonLimitDynamics.Polygon.PositionValues

```

```

/-! Exact harmonic instance of the constructed general time map. All actual
sample bounds are derived on the retained short window. The old harmonic
declarations remain available, and no new historical edge is asserted. -/

```

```

namespace NewtonLimitDynamics.Polygon.HarmonicGeneralTime
open NewtonLimitDynamics

```

```

open TimeSubdivision PointBounds ForceClasses HarmonicStability HarmonicDyadic
open HarmonicGeneralEndpoint

theorem actual_run_state_bound (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (j k : Nat) (hk : k ≤ blocks j) :
  Fraction.le (stateNorm (BoundedIteration.run (linearField w) (duration T j) s k))
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s)) := by
  rw [run_eq_schedule]
  exact HarmonicTimeRealization.countState_le_two w T s j k hT hs hk

def conditions (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  GeneralForcePrefix.Conditions (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
  (harmonicBound w s) s hE where
  toConditions := HarmonicGeneralEndpoint.conditions w E0 T s hw hE hT hs
theorem harmonic_prefix_eq (b : Nat → Bool) (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (j : Nat) :
  GeneralForcePrefix.prefixState b (harmonicOracle w hw) E0 T s hE j =
  HarmonicBinaryPrefix.prefixState b w T s j := by
  rw [GeneralForcePrefix.prefixState, GeneralForcePrefix.countState, harmonic_field, run_eq_schedule]
  rfl

theorem harmonic_name_equiv (b : Nat → Bool) (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) :
  CauchyValues.NameEquiv
    (GeneralForcePrefix.prefixName b (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
      (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs))
    (HarmonicBinaryPrefix.prefixName b w T s hT hs) := by
  intro eps heps
  refine {0, fun j _ => ?_}
  change Fraction.lt (CauchyValues.distance
    (GeneralForcePrefix.prefixState b (harmonicOracle w hw) E0 T s hE j)
    (HarmonicBinaryPrefix.prefixState b w T s j)) eps
  rw [harmonic_prefix_eq]
  exact CauchyValues.distance_self_lt _ eps heps

theorem harmonic_value_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (x : BinaryTime.BinaryTime T hT) :
  GeneralForceTime.gammaValue (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
  (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) x =
  HarmonicTimeRealization.gammaValue w T s hT hs x := by
  induction x using Quotient.inductionOn with
  | _ b => exact Quotient.sound (harmonic_name_equiv b w E0 T s hw hE hT hs)

theorem harmonic_position_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (x : BinaryTime.BinaryTime T hT) :
  GeneralForceTime.gammaPosition (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
  (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) x =
  PositionValues.gammaPosition w T s hT hs x := by
  exact congrArg PositionValues.asPosition (harmonic_value_eq w E0 T s hw hE hT hs x)

end NewtonLimitDynamics.Polygon.HarmonicGeneralTime

```

NewtonLimitDynamics/Polygon/GeneralForceSecants.lean, 9 theorems, 168 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceTime
import BarrowLib.Polygon.KinematicEstimates
import BarrowLib.Polygon.DyadicNodes

```

```

/-! Position secants of the actual constructed central-force map converge
to its constructed velocity along the bracketing dyadic cells. The finite
remainder is derived and transferred through Cauchy values. This does not
assert an unrestricted derivative or identify acceleration with force. -/

```

```

namespace NewtonLimitDynamics.Polygon.GeneralForceSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds ForceClasses HarmonicDyadic HarmonicBinaryPrefix
open CauchyValues BinaryTime PositionValues SecantValues DyadicNodes HarmonicTimeRealization
open GeneralForceEndpoint GeneralForcePrefix GeneralForceTime

```

```

noncomputable def nodeName (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  EndpointCauchyName :=
  shiftedName (if k=blocks m then endpointName o E0 T tau L B s hE d.toConditions
    else prefixName (finiteAddress m k) o E0 T tau L B s hE d) m

theorem node_approx (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (m k : Nat) (hk : k≤blocks m) (j : Nat) :
  (nodeName o E0 T tau L B s hE d m k).approx j =
  GeneralForcePrefix.countState o E0 T s hE (m+j) (k*blocks j) := by
  by_cases he : k=blocks m
  · rw [nodeName,if_pos he]
    change GeneralForcePrefix.countState o E0 T s hE (m+j) (blocks (m+j)) = _
    rw [blocks_add,he]
  · rw [nodeName,if_neg he]
    change GeneralForcePrefix.countState o E0 T s hE (m+j)
      (ticks (finiteAddress m k) (m+j)) = _
    rw [finiteAddress_later_ticks m k j (by omega)]

theorem node_value (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  realize (nodeName o E0 T tau L B s hE d m k) =
  GeneralForceTime.gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k) := by
  by_cases he : k=blocks m
  · rw [nodeName,if_pos he,shiftedValue,nodeTime,if_pos he]
    exact (GeneralForceTime.right_endpoint_value o E0 T tau L B s hE d).symm
  · rw [nodeName,if_neg he,shiftedValue,nodeTime,if_neg he]
    rfl

theorem node_region (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (m k j : Nat) : o.region ((nodeName o E0 T tau L B s hE d m k).approx j).1 := by
  by_cases he : k=blocks m
  · simp only [nodeName,if_pos he,shiftedName]
    exact d.toConditions.run_region (m+j) (duration T (m+j)) d.time_nonnegative
      (blocks (m+j)) (Fraction.le_of_equiv (blocks_duration T (m+j)))
  · simp only [nodeName,if_neg he,shiftedName]
    exact prefix_region (finiteAddress m k) o E0 T tau L B s hE d (m+j)

theorem node_admissible (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  SampledValues.Admissible (fun q => o.region q.1) (realize (nodeName o E0 T tau L B s hE d m k)) :=
  SampledValues.admissible_realize _ _ (node_region o E0 T tau L B s hE d m k)

/-- A secant of completed curve values, divided by the actual coarse cell time. -/
noncomputable def cellSecant (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (hT : 0 < T.num) (m k : Nat) : Value :=
  secantValue (TimeCalibration.inverse (duration T m) hT)
    (GeneralForceTime.gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1)))
    (GeneralForceTime.gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))

/-- Actual finite drift remainders pass to the secant of the completed curve. -/
theorem cell_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (hT : 0 < T.num) (m k : Nat) (hk : k+1≤blocks m) :
  Within (cellSecant o E0 T tau L B s hE d hT m k)
    (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE d
      (nodeTime T d.time_nonnegative m k))) (Fraction.mul (duration T m) B) := by
  rw [cellSecant,← node_value o E0 T tau L B s hE d m (k+1),
    ← node_value o E0 T tau L B s hE d m k]
  change NameBound
    (secantName (TimeCalibration.inverse (duration T m) hT)
      (nodeName o E0 T tau L B s hE d m (k+1)) (nodeName o E0 T tau L B s hE d m k))
    (mapName velocityState velocity_nonexpansive (nodeName o E0 T tau L B s hE d m k)) _
  apply nameBound_of_eventual_le _ _ 0
  intro j _
  change Fraction.le

```

```

    (distance (secantState (TimeCalibration.inverse (duration T m) hT)
      ((nodeName o E0 T tau L B s hE d m (k+1)).approx j)
      ((nodeName o E0 T tau L B s hE d m k).approx j))
      (velocityState ((nodeName o E0 T tau L B s hE d m k).approx j))) _
  rw [node_approx o E0 T tau L B s hE d m (k+1) hk j,
    node_approx o E0 T tau L B s hE d m k (by omega) j]
  let a := field o E0 hE (m+j)
  let h := duration T (m+j)
  let n := k*blocks j
  let q := GeneralForcePrefix.countState o E0 T s hE (m+j) n
  have hn : n+blocks j ≤ blocks (m+j) := by
    rw [blocks_add]
    have hm := Nat.mul_le_mul_right (blocks j) hk
    simp only [n, Nat.add_mul, Nat.one_mul] using hm
  have hb := BoundedIteration.boundedSamples_restart a h s B (blocks (m+j)) n (blocks j)
  hn (d.actual_samples (m+j))
  have hp : 0 < (blocks j : Int) := Int.ofNat_pos.mpr (by unfold blocks; exact Nat.pow_pos (by decide))
  have ht : 0 < (BoundedIteration.time h (blocks j)).num := Int.mul_pos hp hT
  have hr := KinematicEstimates.position_secant_equivalent_time a h q B (duration T m)
    (Int.le_of_lt hT) d.bound_nonnegative (blocks j) ht hT
    (Fraction.equiv_symm (grid_time T m j)) hb
  have he : n+blocks j=(k+1)*blocks j := by simp only [n, Nat.add_mul, Nat.one_mul]
  change Fraction.le
    (FiniteEstimates.pointDistance
      (pointScale (TimeCalibration.inverse (duration T m) hT)
        (pointSub (BoundedIteration.run a h (BoundedIteration.run a h s n) (blocks j)).1
          (BoundedIteration.run a h s n).1)) (BoundedIteration.run a h s n).2) _ at hr
  rw [← BoundedIteration.run_add, he] at hr
  exact Fraction.le_equiv_left (pointState_distance _ _) hr

noncomputable def rateCoefficient (T tau B : Fraction) (s : Point × Point) (ht : 0 < tau.num) : Fraction
  ↪ :=
  Fraction.add B (GeneralForceTime.stateTimeFactor T tau B s ht)

theorem rateCoefficient_nonnegative (T tau B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hB : 0 ≤ B.num) :
  0 ≤ (rateCoefficient T tau B s ht).num :=
  Fraction.nonnegative_add _ _ hB (GeneralForceTime.stateTimeFactor_nonnegative T tau B s hT ht hB)

/-- This secant uses both actual completed cell endpoints, including the right edge. -/
theorem bracketing_secant_bound (b : Nat → Bool) (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m : Nat) :
  Within (cellSecant o E0 T tau L B s hE d hT m (ticks b m))
    (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b)))
    (Fraction.mul (duration T m) (rateCoefficient T tau B s d.calibration_positive)) := by
  have hs := cell_secant_bound o E0 T tau L B s hE d hT m (ticks b m) (by
    have hn := ticks_lt_blocks b m
    omega)
  have hv := velocityValue_within _ _ _ (GeneralForceTime.gamma_within o E0 T tau L B s hE d
    (nodeTime T d.time_nonnegative m (ticks b m)) (Quotient.mk _ b) (duration T m)
    (truncation_time_within b T d.time_nonnegative m))
  have hb := within_triangle _ _ _ _ hs hv
  exact within_mono _ _ _ _ (Fraction.le_of_equiv (Fraction.equiv_symm (Fraction.mul_add _ _ _))) hb

/-- Constructed velocity is the limit of the completed bracketing position secants. -/
theorem dyadic_velocity_uniform_identification (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (cellSecant o E0 T tau L B s hE d hT m (ticks b m))
      (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))) eps := by
  let C := rateCoefficient T tau B s d.calibration_positive
  have hC := rateCoefficient_nonnegative T tau B s d.time_nonnegative d.calibration_positive
  ↪ d.bound_nonnegative
  have hTC : 0 ≤ (Fraction.mul T C).num := Fraction.nonnegative_mul _ _ d.time_nonnegative hC
  obtain (N, hN) := duration_eventually_small (Fraction.mul T C) eps hTC heps
  refine (N, fun m hm b => ?_)
  have he : Fraction.equiv (Fraction.mul (duration T m) C) (duration (Fraction.mul T C) m) := by

```

```

    simp only [duration, Fraction.equiv, Fraction.mul]
    ac_nf
    exact within_mono _ _ _ (Fraction.le_equiv_left he (Fraction.magnitudes.lt_implies_le (hN m hm)))
      (bracketing_secant_bound b o E0 T tau L B s hE d hT m)

theorem dyadic_velocity_identification (b : Nat → Bool) (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m →
    Within (cellSecant o E0 T tau L B s hE d hT m (ticks b m))
      (velocityValue (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))) eps := by
  obtain ⟨N, hN⟩ := dyadic_velocity_uniform_identification o E0 T tau L B s hE d hT eps heps
  exact ⟨N, fun m hm => hN m hm b⟩

```

end NewtonLimitDynamics.Polygon.GeneralForceSecants

NewtonLimitDynamics/Polygon/HarmonicSecants.lean, 2 theorems, 43 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceSecants
import NewtonLimitDynamics.Polygon.HarmonicGeneralTime

/-! The retained harmonic curve's position-secants result is a corollary
of the actual general central-force construction. No derivative primitive
or new historical dependency is used. -/

namespace NewtonLimitDynamics.Polygon.HarmonicSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicStability ForceClasses
open CauchyValues SecantValues DyadicNodes

def cellSecant (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) : Value :=
  secantValue (TimeCalibration.inverse (duration T m) ht)
    (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1)))
    (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))

theorem cellSecant_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (ht : 0 < T.num) (m k : Nat) :
  GeneralForceSecants.cellSecant (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s hE
    (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht m k =
  cellSecant w T s hT hs ht m k := by
  rw [GeneralForceSecants.cellSecant, cellSecant,
    HarmonicGeneralTime.harmonic_value_eq, HarmonicGeneralTime.harmonic_value_eq]

theorem velocity_secants_converge (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (cellSecant w T s hT hs ht m (ticks b m))
      (velocityValue (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps := by
  obtain ⟨N, hN⟩ := GeneralForceSecants.dyadic_velocity_uniform_identification (harmonicOracle w hw)
    E0 T (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s hE
    (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht eps heps
  refine ⟨N, fun m hm b => ?_⟩
  have h := hN m hm b
  rw [cellSecant_eq, HarmonicGeneralTime.harmonic_value_eq] at h
  exact h

```

end NewtonLimitDynamics.Polygon.HarmonicSecants

NewtonLimitDynamics/Polygon/CompletedForce.lean, 12 theorems, 223 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceTime
import BarrowLib.Polygon.SampledValues

/-! Extend the actual sampled central force to completed positions with a
certified regional Cauchy representative. No whole-plane domain is required.
The same mesh precision used by the motion is retained. Error exhaustion and

```

Lipschitz comparisons construct the extension and its bounds; no desired acceleration value or curve force equation is supplied. -/

```

namespace NewtonLimitDynamics.Polygon.CompletedForce
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic CauchyValues
open ForceClasses GeneralForcePrecision GeneralForceEndpoint HarmonicTimeRealization

def errorCoefficient (E0 : Fraction) : Fraction :=
  Fraction.add (Fraction.add E0 E0) E0

theorem precisionError_le_sampleError (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) :
  Fraction.le (o.error (precision o.toOracle E0 hE j)) (sampleError o E0 hE j) :=
  Fraction.magnitudes.le_trans (Fraction.le_add_nonnegative _ _ (o.error_nonnegative _))
  (Fraction.le_add_nonnegative _ _ (o.error_nonnegative _))

theorem sampleError_tail (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (j : Nat) :
  Fraction.le (sampleError o E0 hE j) (duration (errorCoefficient E0) j) := by
  have he := Fraction.magnitudes.lt_implies_le (precision_error o.toOracle E0 hE j)
  have ht := Fraction.add_le_add (Fraction.add_le_add he he) he
  apply Fraction.le_equiv_right ht
  simp only [errorCoefficient, GeometricTail.tailCap, duration, Fraction.equiv,
    Fraction.add, Int.add_mul, Int.mul_add]
  ac_nf

theorem sampleError_vanishes (o : CentralOracle) (E0 : Fraction)
  (hE : 0 < E0.num) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j, N ≤ j → Fraction.lt (sampleError o E0 hE j) eps := by
  have hC : 0 ≤ (errorCoefficient E0).num := Fraction.nonnegative_add _ _
  (Fraction.nonnegative_add _ _ (Int.le_of_lt hE) (Int.le_of_lt hE)) (Int.le_of_lt hE)
  obtain (N, hN) := duration_eventually_small (errorCoefficient E0) eps hC heps
  exact (N, fun j hj => Fraction.magnitudes.lt_of_le_lt
    (sampleError_tail o E0 hE j) (hN j hj))

noncomputable def family (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) : SampledValues.Family where
  sample := fun j s => accelerationState (field o E0 hE j s.1)
  region := fun s => o.region s.1
  coefficient := L
  coefficient_nonnegative := hL.1
  error := sampleError o E0 hE
  error_nonnegative := sampleError_nonnegative o E0 hE
  error_vanishes := sampleError_vanishes o E0 hE
  ordered_bound := by
    intro i j hij s t hs ht
    have hc := samples_comparison_contract o.toOracle L hL
      (precision o.toOracle E0 hE i) (precision o.toOracle E0 hE j)
      (precision_monotone o.toOracle E0 hE hij) s.1 t.1 hs ht
    have hb := Fraction.le_equiv_left (acceleration_distance _ _) hc
    have hp := Fraction.mul_le_mul_nonnegative_left
      (point_le_state (stateSub s t)) L hL.1
    have hs := Fraction.magnitudes.le_trans hb (Fraction.add_le_add_right hp _)
    exact Fraction.le_equiv_right hs
      (Fraction.add_equiv (Fraction.mul_comm _ _) (Fraction.equiv_refl _))

noncomputable def forceName (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (a : EndpointCauchyName)
  (ha : ∀ n, o.region (a.approx n).1) : EndpointCauchyName :=
  SampledValues.sampledName (family o E0 L hE hL) a ha

noncomputable def forceValue (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (x : Value)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x) : Value :=
  SampledValues.sampledValue (family o E0 L hE hL) x hx

theorem forceValue_realize (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (a : EndpointCauchyName)
  (ha : ∀ n, o.region (a.approx n).1)

```



```

    (hx : SampledValues.Admissible (fun q => o.region q.1) (realize a)) :
    forceValue o E0 L hE hL (realize a) hx = realize (forceName o E0 L hE hL a ha) :=
    SampledValues.sampledValue_realize (family o E0 L hE hL) a ha hx

theorem forceValue_within (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (x y : Value)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x)
  (hy : SampledValues.Admissible (fun q => o.region q.1) y)
  (R : Fraction) (hxy : Within x y R) :
  Within (forceValue o E0 L hE hL x hx) (forceValue o E0 L hE hL y hy)
  (Fraction.mul R L) :=
  SampledValues.sampledValue_within (family o E0 L hE hL) x y hx hy R hxy

theorem position_admissible (o : CentralOracle) (x : Value)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x) :
  SampledValues.Admissible (fun q => o.region q.1) (PositionValues.positionValue x) := by
  obtain (a,rfl,ha) := hx
  exact (PositionValues.mapName PositionValues.positionState PositionValues.position_nonexpansive a,
    rfl,ha)

theorem force_input_position (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (x : Value)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x) :
  forceValue o E0 L hE hL (PositionValues.positionValue x) (position_admissible o x hx) =
  forceValue o E0 L hE hL x hx := by
  obtain (a,rfl,ha) := hx
  change forceValue o E0 L hE hL
  (realize (PositionValues.mapName PositionValues.positionState PositionValues.position_nonexpansive
    ↪ a)) _ = _
  rw [forceValue_realize o E0 L hE hL
    (PositionValues.mapName PositionValues.positionState PositionValues.position_nonexpansive a)
    (fun n => ha n),forceValue_realize o E0 L hE hL a ha]
  rfl

theorem force_output_position (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (x : Value)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x) :
  PositionValues.positionValue (forceValue o E0 L hE hL x hx) =
  forceValue o E0 L hE hL x hx := by
  obtain (a,rfl,ha) := hx
  rw [forceValue_realize o E0 L hE hL a ha]
  rfl

noncomputable def acceleration (o : CentralOracle) (E0 L : Fraction) (hE : 0 < E0.num)
  (hL : LipschitzOn o.toOracle L) (x : PositionValues.PositionValue)
  (hx : SampledValues.Admissible (fun q => o.region q.1) x.val) : PositionValues.PositionValue :=
  (forceValue o E0 L hE hL x.val hx,force_output_position o E0 L hE hL x.val hx)

/-- Agreement with the retained force value at every rational point. The mesh
precision may remain constant; no premise that precision(j) >= j is used. -/
theorem force_rational_agreement (o : CentralOracle) (E0 L : Fraction)
  (hE : 0 < E0.num) (hL : LipschitzOn o.toOracle L)
  (s : Point × Point) (hs : o.region s.1) :
  forceValue o E0 L hE hL (embed s) (SampledValues.admissible_embed _ s hs) =
  accelerationValue o.toOracle s.1 hs := by
  change forceValue o E0 L hE hL (realize (constantName s)) _ = accelerationValue o.toOracle s.1 hs
  rw [forceValue_realize o E0 L hE hL (constantName s) (fun _ => hs)]
  apply Quotient.sound
  intro eps heps
  obtain (N,hN) := o.error_vanishes eps heps
  obtain (M,hM) := sampleError_vanishes o E0 hE eps heps
  refine (max N M,fun j hj => ?_)
  let q := precision o.toOracle E0 hE j
  by_cases hq : q ≤ j
  · have hc := o.coherent q j hq s.1 hs
    have he := precisionError_le_sampleError o E0 hE j
    exact Fraction.magnitudes.lt_of_le_lt (Fraction.magnitudes.le_trans hc he) (hM j (by omega))
  · exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_equiv_left (stateSub_norm_symm _))
    (o.coherent j q (by omega) s.1 hs)) (hN j (by omega))

```

```

/-- Precision scales and valid Lipschitz bounds do not change the completed
force. Independence of the motion from these choices is a separate theorem. -/
theorem force_precision_independent (o : CentralOracle) (E0 E1 L L' : Fraction)
  (hE : 0 < E0.num) (hE' : 0 < E1.num)
  (hL : LipschitzOn o.toOracle L) (hL' : LipschitzOn o.toOracle L')
  (x : Value) (hx : SampledValues.Admissible (fun q => o.region q.1) x) :
  forceValue o E0 L hE hL x hx = forceValue o E1 L' hE' hL' x hx := by
  obtain (a,rfl,ha) := hx
  rw [forceValue_realize o E0 L hE hL a ha,forceValue_realize o E1 L' hE' hL' a ha]
  apply Quotient.sound
  intro eps heps
  obtain (N,hN) := sampleError_vanishes o E0 hE eps heps
  obtain (M,hM) := sampleError_vanishes o E1 hE' eps heps
  refine (max N M,fun j hj => ?_)
  let q := precision o.toOracle E0 hE j
  let q' := precision o.toOracle E1 hE' j
  by_cases hq : q ≤ q'
  · exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.magnitudes.le_trans (o.coherent q q' hq (a.approx j).1 (ha j))
    (precisionError_le_sampleError o E0 hE j)) (hN j (by omega))
  · exact Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_equiv_left (stateSub_norm_symm _ _))
    (Fraction.magnitudes.le_trans
    (o.coherent q' q (by omega) (a.approx j).1 (ha j))
    (precisionError_le_sampleError o E1 hE' j))) (hM j (by omega))

def curveErrorCoefficient (A E0 L : Fraction) : Fraction :=
  Fraction.add (Fraction.mul A L) (errorCoefficient E0)

theorem prefix_force_bound (b : Nat → Bool) (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (j : Nat) :
  Within (embed (accelerationState (field o E0 hE j)
    (GeneralForcePrefix.prefixState b o E0 T s hE j).1)))
    (forceValue o E0 L hE d.lipschitz
    (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
    (GeneralForceTime.gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)))
    (duration (curveErrorCoefficient
    (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) E0 L) j) := by
  let A := GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive
  have hb := SampledValues.sampled_approximant_bound (family o E0 L hE d.lipschitz)
    (GeneralForcePrefix.prefixName b o E0 T tau L B s hE d)
    (GeneralForceTime.prefix_region b o E0 T tau L B s hE d)
    (GeneralForceTime.gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)) j (GeometricTail.tailCap
    ↪ A j)
  (GeneralForceTime.prefix_value_bound b o E0 T tau L B s hE d j)
  apply within_mono _ _ _ ?_ hb
  have ht := Fraction.add_le_add_left (sampleError_tail o E0 hE j)
    (Fraction.mul (GeometricTail.tailCap A j) L)
  apply Fraction.le_equiv_right ht
  simp only [A,curveErrorCoefficient,GeometricTail.tailCap,duration,Fraction.equiv,
    Fraction.add,Fraction.mul,Int.add_mul,Int.mul_add]
  ac_nf

/-- Uniform convergence of actual polygon force samples to the force at the
constructed completed positions. This is not yet the acceleration equation. -/
theorem prefix_force_uniform_convergence (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j, N ≤ j → ∀ b : Nat → Bool,
    Within (embed (accelerationState (field o E0 hE j)
    (GeneralForcePrefix.prefixState b o E0 T s hE j).1)))
    (forceValue o E0 L hE d.lipschitz
    (GeneralForceTime.gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
    (GeneralForceTime.gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) eps := by
  let A := GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive
  have hA := GeneralForcePrefix.coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative
    d.calibration_positive d.lipschitz.1 d.bound_nonnegative

```

```

have hC : 0 ≤ (curveErrorCoefficient A E0 L).num :=
  Fraction.nonnegative_add _ _ (Fraction.nonnegative_mul _ _ hA d.lipschitz.1)
  (Fraction.nonnegative_add _ _
    (Fraction.nonnegative_add _ _ (Int.le_of_lt hE) (Int.le_of_lt hE)) (Int.le_of_lt hE))
obtain (N,hN) := duration_eventually_small (curveErrorCoefficient A E0 L) eps hC heps
exact (N,fun j hj b => within_mono
  (Fraction.magnitudes.lt_implies_le (hN j hj)) (prefix_force_bound b o E0 T tau L B s hE d j))
end NewtonLimitDynamics.Polygon.CompletedForce

```

NewtonLimitDynamics/Polygon/HarmonicCompletedForce.lean, 2 theorems, 47 lines

```

import NewtonLimitDynamics.Polygon.CompletedForce
import NewtonLimitDynamics.Polygon.HarmonicGeneralTime
import BarrowLib.Polygon.SecantValues

/-! The general completed force specializes to the retained linear central
law at all completed positions, and its actual polygon force samples converge
uniformly along the retained harmonic curve. -/

namespace NewtonLimitDynamics.Polygon.HarmonicCompletedForce
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicComparison HarmonicDyadic HarmonicBinaryPrefix
open HarmonicStability ForceClasses CauchyValues SecantValues PositionValues

def linearValue (w : Fraction) (x : Value) : Value :=
  secantValue (negF w) x (embed (zeroPoint,zeroPoint))

theorem completed_linear_force (w E0 : Fraction) (hw : 0 ≤ w.num) (hE : 0 < E0.num)
  (hL : LipschitzOn (harmonicOracle w hw).toOracle w.abs)
  (x : Value) (hx : SampledValues.Admissible (fun q => (harmonicOracle w hw).region q.1) x) :
  CompletedForce.forceValue (harmonicOracle w hw) E0 w.abs hE
  hL x hx = linearValue w x := by
obtain (a,rfl,ha) := hx
rw [CompletedForce.forceValue_realize (harmonicOracle w hw) E0 w.abs hE hL a ha]
.
  apply Quotient.sound
  apply nameEquiv_of_levelwise_stateEquiv
  intro j
  exact {pointScale_congr (negF w) (pointEquiv_symm (pointSub_zero (a.approx j)).1)),
    (Fraction.equiv_refl _,Fraction.equiv_refl _)}

theorem retained_curve_force_samples_converge (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ j, N ≤ j → ∀ b : Nat → Bool,
    Within (embed (accelerationState (linearField w (prefixState b w T s j)).1)))
      (linearValue w (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps := by
obtain (N,hN) := CompletedForce.prefix_force_uniform_convergence (harmonicOracle w hw)
  E0 T (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s hE
  (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) eps heps
refine (N,fun j hj b => ?_)
have h := hN j hj b
rw [completed_linear_force] at h
rw [HarmonicGeneralTime.harmonic_value_eq,
  HarmonicGeneralTime.harmonic_prefix_eq,HarmonicGeneralEndpoint.harmonic_field] at h
exact h
end NewtonLimitDynamics.Polygon.HarmonicCompletedForce

```

NewtonLimitDynamics/Polygon/GeneralForceAccelerationSecants.lean, 5 theorems, 158 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceSecants
import NewtonLimitDynamics.Polygon.CompletedForce
import BarrowLib.Polygon.AccelerationEstimates

/-! Completed dyadic velocity secants converge to the completed force at the
constructed positions. Actual finite velocity remainders and force sampling
error derive the estimate; no acceleration equation is a premise. The result

```

```

is a bracketing dyadic secant criterion, not unrestricted differentiation. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceAccelerationSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds ForceClasses HarmonicDyadic HarmonicBinaryPrefix
open CauchyValues BinaryTime PositionValues SecantValues DyadicNodes HarmonicTimeRealization
open GeneralForceEndpoint GeneralForcePrefix GeneralForceTime GeneralForceSecants

/-- Replace a curve input by its certified node representative as a whole
force value, so dependent region certificates are transported together. -/
theorem node_force_value (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat) :
  CompletedForce.forceValue o E0 L hE d.lipschitz
    (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
    (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)) =
  CompletedForce.forceValue o E0 L hE d.lipschitz
    (realize (nodeName o E0 T tau L B s hE d m k))
    (node_admissible o E0 T tau L B s hE d m k) :=
  SampledValues.sampledValue_congr (CompletedForce.family o E0 L hE d.lipschitz) _ _
    (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
    (node_admissible o E0 T tau L B s hE d m k)
    (node_value o E0 T tau L B s hE d m k).symm

noncomputable def cellAccelerationSecant (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) : Value :=
  secantValue (TimeCalibration.inverse (duration T m) hT)
    (velocityValue (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1))))
    (velocityValue (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))

theorem cell_acceleration_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) :
  Within (cellAccelerationSecant o E0 T tau L B s hE d hT m k)
    (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
      ↪ d.time_nonnegative m k))
      (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))
    (Fraction.mul L (Fraction.mul (duration T m) (velocityCap T B s))) := by
  rw [node_force_value o E0 T tau L B s hE d m k, cellAccelerationSecant, ← node_value o E0 T tau L B s hE
    ↪ d m (k+1),
    ← node_value o E0 T tau L B s hE d m k]
  let f := CompletedForce.family o E0 L hE d.lipschitz
  change Within
    (realize (secantName (TimeCalibration.inverse (duration T m) hT)
      (mapName velocityState velocity_nonexpansive (nodeName o E0 T tau L B s hE d m (k+1)))
      (mapName velocityState velocity_nonexpansive (nodeName o E0 T tau L B s hE d m k))))
    (SampledValues.sampledValue f (realize (nodeName o E0 T tau L B s hE d m k))
      (node_admissible o E0 T tau L B s hE d m k)) _
  rw [← SampledValues.sampledValue_offset f _ m,
    SampledValues.sampledValue_realize (SampledValues.offsetFamily f m)
      (nodeName o E0 T tau L B s hE d m k) (node_region o E0 T tau L B s hE d m k)]
  change NameBound _ _ _
  apply SampledValues.nameBound_of_vanishing_error _ _ _ (fun j => sampleError o E0 hE (m+j))
  · intro eps heps
    obtain (N, hN) := CompletedForce.sampleError_vanishes o E0 hE eps heps
    exact {N, fun j hj => hN (m+j)} (by omega)
  · intro j
    change Fraction.le
      (distance (secantState (TimeCalibration.inverse (duration T m) hT)
        (velocityState ((nodeName o E0 T tau L B s hE d m (k+1)).approx j))
        (velocityState ((nodeName o E0 T tau L B s hE d m k).approx j))
        (accelerationState (field o E0 hE (m+j))
          ((nodeName o E0 T tau L B s hE d m k).approx j).1))) _
  rw [node_approx o E0 T tau L B s hE d m (k+1) hk j,
    node_approx o E0 T tau L B s hE d m k (by omega) j]
  let a := field o E0 hE (m+j)
  let h := duration T (m+j)
  let n := k*blocks j

```

```

let q := GeneralForcePrefix.countState o E0 T s hE (m+j) n
have hn : n+blocks j ≤ blocks (m+j) := by
  rw [blocks_add]
  have hm := Nat.mul_le_mul_right (blocks j) hk
  simp only [n, Nat.add_mul, Nat.one_mul] using hm
have hp : 0 < (blocks j : Int) := Int.ofNat_pos.mpr (by unfold blocks; exact Nat.pow_pos (by decide))
have ht : 0 < (BoundedIteration.time h (blocks j)).num := Int.mul_pos hp hT
have hv : ∀ i, i < blocks j → Fraction.le
  (pointNorm (BoundedIteration.run a h q i).2) (velocityCap T B s) := by
  intro i hi
  change Fraction.le
  (pointNorm (BoundedIteration.run a h (BoundedIteration.run a h s n) i).2) _
  rw [← BoundedIteration.run_add]
  exact GeneralForcePrefix.count_velocity o E0 T tau L B s hE d (m+j) (n+i) (by omega)
have hr := AccelerationEstimates.acceleration_secant_equivalent_time_at a h q L
  (sampleError o E0 hE (m+j)) (velocityCap T B s) (duration T m)
  (Int.le_of_lt hT) d.lipschitz.1
  (velocityCap_nonnegative T B s d.time_nonnegative d.bound_nonnegative)
  (blocks j) ht
  (fun i hi => restarted_comparison o E0 T tau L B s hE d (m+j) n (blocks j) hn i hi) hT
  ↪ (Fraction.equiv_symm (grid_time T m j)) hv
have he : n+blocks j=(k+1)*blocks j := by simp only [n, Nat.add_mul, Nat.one_mul]
change Fraction.le (FiniteEstimates.pointDistance
  (pointScale (TimeCalibration.inverse (duration T m) hT)
    (pointSub (BoundedIteration.run a h (BoundedIteration.run a h s n) (blocks j)).2
      (BoundedIteration.run a h s n).2)) (a (BoundedIteration.run a h s n).1)) _ at hr
  rw [← BoundedIteration.run_add, he] at hr
  exact Fraction.le_equiv_left (pointState_distance _ _) hr

noncomputable def accelerationCoefficient (T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction :=
  Fraction.mul L (Fraction.add (velocityCap T B s) (stateTimeFactor T tau B s ht))

theorem accelerationCoefficient_nonnegative (T tau L B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (accelerationCoefficient T tau L B s ht).num :=
  Fraction.nonnegative_mul _ _ hL (Fraction.nonnegative_add _ _
    (velocityCap_nonnegative T B s hT hB) (stateTimeFactor_nonnegative T tau B s hT ht hB))

theorem bracketing_acceleration_secant_bound (b : Nat → Bool) (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m : Nat) :
  Within (cellAccelerationSecant o E0 T tau L B s hE d hT m (ticks b m))
    (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
      (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)))
    (Fraction.mul (duration T m) (accelerationCoefficient T tau L B s d.calibration_positive)) := by
  have hs := cell_acceleration_secant_bound o E0 T tau L B s hE d hT m (ticks b m)
  (by have hn := ticks_lt_blocks b m; omega)
  have hf := CompletedForce.forceValue_within o E0 L hE d.lipschitz _ _
    (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (ticks b m)))
    (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)) _
    (gamma_within o E0 T tau L B s hE d
      (nodeTime T d.time_nonnegative m (ticks b m)) (Quotient.mk _ b) (duration T m)
      (truncation_time_within b T d.time_nonnegative m))
  have hb := within_triangle _ _ _ _ hs hf
  apply within_mono _ _ _ _ ?_ hb
  apply Fraction.le_of_equiv
  simp only [accelerationCoefficient, Fraction.equiv, Fraction.mul, Fraction.add, Int.add_mul, Int.mul_add]
  ac_nf

/-- The force at the constructed position is the uniform limit of completed
bracketing dyadic velocity secants. No acceleration is supplied as data. -/
theorem dyadic_acceleration_uniform_identification (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (cellAccelerationSecant o E0 T tau L B s hE d hT m (ticks b m))
      (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _
        ↪ b))

```

```

      (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) eps := by
let C := accelerationCoefficient T tau L B s d.calibration_positive
have hC := accelerationCoefficient_nonnegative T tau L B s d.time_nonnegative
d.calibration_positive d.lipschitz.1 d.bound_nonnegative
obtain (N,hN) := duration_eventually_small (Fraction.mul T C) eps
(Fraction.nonnegative_mul _ _ d.time_nonnegative hC) heps
refine (N,fun m hm b => ?_)
have he : Fraction.equiv (Fraction.mul (duration T m) C) (duration (Fraction.mul T C) m) := by
  simp only [duration,Fraction.equiv,Fraction.mul]
ac_nf
exact within_mono _ _ _ _
(Fraction.le_equiv_left he (Fraction.magnitudes.lt_implies_le (hN m hm)))
(bracketing_acceleration_secant_bound b o E0 T tau L B s hE d hT m)

end NewtonLimitDynamics.Polygon.GeneralForceAccelerationSecants

```

NewtonLimitDynamics/Polygon/HarmonicAccelerationSecants.lean, 2 theorems, 45 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceAccelerationSecants
import NewtonLimitDynamics.Polygon.HarmonicCompletedForce

/-! The retained harmonic curve's completed velocity secants converge to
its completed linear central force, as an instance of the general result. -/

namespace NewtonLimitDynamics.Polygon.HarmonicAccelerationSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicStability ForceClasses
open CauchyValues SecantValues DyadicNodes

def cellAccelerationSecant (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (ht : 0 < T.num) (m k : Nat) : Value :=
  secantValue (TimeCalibration.inverse (duration T m) ht)
  (velocityValue (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1))))
  (velocityValue (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k)))

theorem cellAccelerationSecant_eq (w E0 T : Fraction) (s : Point × Point) (hw : 0 ≤ w.num)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (ht : 0 < T.num) (m k : Nat) :
  GeneralForceAccelerationSecants.cellAccelerationSecant (harmonicOracle w hw)
  E0 T (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s hE
  (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht m k =
  cellAccelerationSecant w T s hT hs ht m k := by
rw [GeneralForceAccelerationSecants.cellAccelerationSecant,cellAccelerationSecant,
  HarmonicGeneralTime.harmonic_value_eq,HarmonicGeneralTime.harmonic_value_eq]

theorem acceleration_secants_converge (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (ht : 0 < T.num) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
  Within (cellAccelerationSecant w T s hT hs ht m (ticks b m))
  (HarmonicCompletedForce.linearValue w
    (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps := by
obtain (N,hN) := GeneralForceAccelerationSecants.dyadic_acceleration_uniform_identification
  (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
  (HarmonicGeneralEndpoint.harmonicBound w s) s hE
  (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht eps heps
refine (N,fun m hm b => ?_)
have h := hN m hm b
rw [cellAccelerationSecant_eq,HarmonicCompletedForce.completed_linear_force,
  HarmonicGeneralTime.harmonic_value_eq] at h
exact h

end NewtonLimitDynamics.Polygon.HarmonicAccelerationSecants

```

NewtonLimitDynamics/Polygon/GeneralForcePolygonCurve.lean, 8 theorems, 107 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceTime
import BarrowLib.Polygon.PolygonValues

/-! An actual coarse polygon quotient for the general sampled central-force
construction. Shared finite-vertex geometry proves all aliases; the actual

```

```

velocity cap and prefix tail give a uniform whole-edge curve comparison. -/

namespace NewtonLimitDynamics.Polygon.GeneralForcePolygonCurve
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix CauchyValues BinaryTime
open PositionValues HarmonicTimeRealization GeneralForceEndpoint GeneralForcePrefix GeneralForceTime

noncomputable def vertices (o : ForceClasses.CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (m : Nat) : PolygonValues.VertexChain T m where
  state := countState o E0 T s hE m
  join := fun _ => (Fraction.equiv_refl _, Fraction.equiv_refl _)

noncomputable def polygonMap (o : ForceClasses.CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (m : Nat) :
  BinaryTime T hT → PositionValue := PolygonValues.polygonMap T hT m (vertices o E0 T s hE m)

noncomputable def edgeCoefficient (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction :=
  Fraction.add (Fraction.mul T (velocityCap T B s))
  (GeneralForcePrefix.coefficient E0 T tau L B s ht)

theorem edgeCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (edgeCoefficient E0 T tau L B s ht).num :=
  Fraction.nonnegative_add _
  (Fraction.nonnegative_mul _ _ hT (velocityCap_nonnegative T B s hT hB))
  (GeneralForcePrefix.coefficient_nonnegative E0 T tau L B s hE hT ht hL hB)

theorem edgeRadius_geometric (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) (m : Nat) :
  Fraction.equiv
  (Fraction.add (Fraction.mul (duration T m) (velocityCap T B s))
  (GeometricTail.tailCap (GeneralForcePrefix.coefficient E0 T tau L B s ht) m))
  (duration (edgeCoefficient E0 T tau L B s ht) m) := by
  simp only [edgeCoefficient, GeometricTail.tailCap, duration, Fraction.equiv, Fraction.add,
  Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

theorem polygon_vertex_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Within (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)).val
  (positionValue (embed (prefixState b o E0 T s hE m)))
  (Fraction.mul (duration T m) (velocityCap T B s)) :=
  PolygonValues.polygon_vertex_bound b T d.time_nonnegative m (vertices o E0 T s hE m) _
  (count_velocity o E0 T tau L B s hE d m (ticks b m) (ticks_le_blocks b m))

/-- Every point of every coarse edge is controlled against the constructed
curve, with an explicit geometric mesh bound. No area is supplied. -/
theorem whole_edge_bound (b : Nat → Bool) (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Within (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)).val
  (gammaPosition o E0 T tau L B s hE d (Quotient.mk _ b)).val
  (duration (edgeCoefficient E0 T tau L B s d.calibration_positive) m) := by
  have hp := polygon_vertex_bound b o E0 T tau L B s hE d m
  have hg := positionValue_within _ _ _ (prefix_value_bound b o E0 T tau L B s hE d m)
  have hb := within_triangle _ _ _ _ hp hg
  exact within_mono _ _ _ _
  (Fraction.le_of_equiv (edgeRadius_geometric E0 T tau L B s d.calibration_positive m)) hb

theorem polygonMap_whole_edge_bound (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (t : BinaryTime T d.time_nonnegative) :
  Within (polygonMap o E0 T s hE d.time_nonnegative m t).val
  (gammaPosition o E0 T tau L B s hE d t).val
  (duration (edgeCoefficient E0 T tau L B s d.calibration_positive) m) := by
  induction t using Quotient.inductionOn with
  | _ b => exact whole_edge_bound b o E0 T tau L B s hE d m

```

```

theorem polygonMap_uniform_convergence (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ t : BinaryTime T d.time_nonnegative,
    Within (polygonMap o E0 T s hE d.time_nonnegative m t).val
      (gammaPosition o E0 T tau L B s hE d t).val eps := by
  obtain ⟨N, hN⟩ := duration_eventually_small
    (edgeCoefficient E0 T tau L B s d.calibration_positive) eps
    (edgeCoefficient_nonnegative E0 T tau L B s hE d.time_nonnegative d.calibration_positive
      d.lipschitz.1 d.bound_nonnegative) heps
  exact ⟨N, fun m hm t => within_mono _ _ _ (Fraction.magnitudes.lt_implies_le (hN m hm))
    (polygonMap_whole_edge_bound o E0 T tau L B s hE d m t)⟩

theorem polygonMap_left (o : ForceClasses.CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (m : Nat) :
  polygonMap o E0 T s hE hT m (leftTime T hT) = embedPosition s.1 :=
  PolygonValues.polygonMap_left T hT m (vertices o E0 T s hE m)

theorem shared_initial_endpoint (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  polygonMap o E0 T s hE d.time_nonnegative m (leftTime T d.time_nonnegative) =
    gammaPosition o E0 T tau L B s hE d (leftTime T d.time_nonnegative) := by
  rw [polygonMap_left]
  apply Subtype.ext
  change positionValue (embed (s.1, zeroPoint)) = positionValue
    (gammaValue o E0 T tau L B s hE d (leftTime T d.time_nonnegative))
  rw [GeneralForceTime.left_endpoint_value]
  rfl

end NewtonLimitDynamics.Polygon.GeneralForcePolygonCurve

```

NewtonLimitDynamics/Polygon/HarmonicGeneralPolygon.lean, 1 theorem, 41 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePolygonCurve
import NewtonLimitDynamics.Polygon.HarmonicPolygonCurve
import NewtonLimitDynamics.Polygon.HarmonicGeneralEndpoint

/-! Exact equality of the general sampled polygon instance and the retained
harmonic polygon map. It requires no curve or small-window premise. -/

namespace NewtonLimitDynamics.Polygon.HarmonicGeneralPolygon
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicStability ForceClasses
open CauchyValues PositionValues

theorem harmonic_polygonMap_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (m : Nat)
  (t : BinaryTime.BinaryTime T hT) :
  GeneralForcePolygonCurve.polygonMap (harmonicOracle w hw) E0 T s hE hT m t =
    HarmonicPolygonCurve.polygonMap w T s hT m t := by
  induction t using Quotient.inductionOn with
  | _ b =>
    apply Subtype.ext
    change positionValue (realize (PolygonValues.polygonName b T hT m
      (GeneralForcePolygonCurve.vertices (harmonicOracle w hw) E0 T s hE m))) =
      positionValue (realize (HarmonicPolygonCurve.polygonName b w T s hT m))
    apply congrArg positionValue
    apply Quotient.sound
    apply nameEquiv_of_levelwise_stateEquiv
    intro j
    have hp : GeneralForcePrefix.countState (harmonicOracle w hw) E0 T s hE m (ticks b m) =
      prefixState b w T s m := by
      rw [GeneralForcePrefix.countState, HarmonicGeneralEndpoint.harmonic_field, run_eq_schedule]
      rfl
    change stateEquiv
    (AffineValues.affineState
      (GeneralForcePrefix.countState (harmonicOracle w hw) E0 T s hE m (ticks b m)).1
      (GeneralForcePrefix.countState (harmonicOracle w hw) E0 T s hE m (ticks b m)).2 _)
    (AffineValues.affineState (prefixState b w T s m).1 (prefixState b w T s m).2 _))

```



```

    rw [hp]
    exact ((Fraction.equiv_refl _, Fraction.equiv_refl _),
           (Fraction.equiv_refl _, Fraction.equiv_refl _))
end NewtonLimitDynamics.Polygon.HarmonicGeneralPolygon

NewtonLimitDynamics/Polygon/HarmonicPathRegion.lean, 16 theorems, 182 lines

import NewtonLimitDynamics.Polygon.HarmonicPolygonCurve
import BarrowLib.Polygon.SquareOuterContent
import BarrowLib.Polygon.MatchedRegion

/-- The actual nonnegative matched region between the constructed harmonic
curve and the coarse polygon. Each cell uses simultaneous polygon/curve
positions and all rational convex connectors, then closure in the completed
plane. The union counts overlapping and crossing lobes once, without signed
cancellation. The final endpoint connector is included in the last cell. -/
namespace NewtonLimitDynamics.Polygon.HarmonicPathRegion
open NewtonLimitDynamics
open TimeSubdivision PointBounds CentralSchedule HarmonicStability
open HarmonicDyadic HarmonicBinaryPrefix HarmonicTimeRealization
open BinaryTime CauchyValues PositionValues HarmonicPolygonCurve
open ConvexCover ConvexValues CompletionGeometry SquareOuterContent

/-- The actual level-m coarse vertex at the start of cell k. -/
def cellStart (w T : Fraction) (s : Point × Point) (m k : Nat) : Point :=
  (schedule (linearField w) (List.replicate k (duration T m)) s).1

/-- Simultaneous positions, using quotient maps rather than a supplied curve. -/
def cellPatch (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (m k : Nat) (x : PositionValue) : Prop :=
  MatchedRegion.cellPatch T hT (polygonMap w T s hT m)
  (gammaPosition w T s hT hs) m k x

/-- Cell closures give all limit connector points, including their endpoints.
The finite union is an unsigned point set, not a determinant sum. -/
def Region (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (m : Nat) (x : PositionValue) : Prop :=
  MatchedRegion.Region T hT (polygonMap w T s hT m) (gammaPosition w T s hT hs) m x

theorem edgeRadius_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (m : Nat) : 0 ≤ (edgeRadius w T s m).num :=
  Fraction.nonnegative_add _ _
  (Fraction.nonnegative_mul _ _ hT
   (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s)))
  (HarmonicBinaryPrefix.coefficient_nonnegative w T s hT)

def coverRadius (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (m : Nat) : NonnegativeRadius :=
  (edgeRadius w T s m, edgeRadius_nonnegative w T s hT m)

theorem simultaneous_endpoints_square (b : Nat → Bool) (w T : Fraction)
  (s : Point × Point) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
  CoordinateSquare (cellStart w T s m (ticks b m)) (coverRadius w T s hT m)
  (polygonMap w T s hT m (Quotient.mk _ b)) ∧
  CoordinateSquare (cellStart w T s m (ticks b m)) (coverRadius w T s hT m)
  (gammaPosition w T s hT hs (Quotient.mk _ b)) := by
  have hD : 0 ≤ (Fraction.mul (duration T m)
    (Fraction.mul (Fraction.ofInt 2) (stateNorm s))).num :=
    Fraction.nonnegative_mul _ _ hT
    (Fraction.nonnegative_mul _ _ (by decide) (stateNorm_nonnegative s))
  have hE : 0 ≤ (HarmonicBinaryPrefix.tailCap w T s m).num :=
    HarmonicBinaryPrefix.coefficient_nonnegative w T s hT
  have hpoly := polygon_vertex_bound b w T s hT hs m
  have hcurve := within_symm _ _ _ (positionValue_within _ _ _
    (binaryValue_prefix_bound b w T s hT hs m))
  have hp := within_mono _ _ _ (Fraction.le_add_nonnegative _ _ hE) hpoly
  have hg := within_mono _ _ _
    (Fraction.le_equiv_right (Fraction.le_add_nonnegative _ _ hD)
     (Fraction.add_comm _ _)) hcurve
  exact (square_of_ball_bound _ _ _ hp, square_of_ball_bound _ _ _ hg)

```

```

theorem cellPatch_square (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m k : Nat)
  (x : PositionValue) (hx : cellPatch w T s hT hs m k x) :
  CoordinateSquare (cellStart w T s m k) (coverRadius w T s hT m) x := by
  exact MatchedRegion.cellPatch_square T hT _ _ m (cellStart w T s m)
  (coverRadius w T s hT m) (fun b => simultaneous_endpoints_square b w T s hT hs m) k x hx

theorem closed_cell_square (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m k : Nat)
  (x : PositionValue) (hx : Closure (cellPatch w T s hT hs m k) x) :
  CoordinateSquare (cellStart w T s m k) (coverRadius w T s hT m) x :=
  MatchedRegion.closed_cell_square T hT _ _ m (cellStart w T s m)
  (coverRadius w T s hT m) (fun b => simultaneous_endpoints_square b w T s hT hs m) k x hx

/-- One derived coordinate square per actual coarse cell. -/
def actualCover (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
  (hs : DyadicSmallTime w T) (m : Nat) : Cover (Region w T s hT hs m) :=
  MatchedRegion.actualCover T hT _ _ m (cellStart w T s m) (coverRadius w T s hT m)
  (fun b => simultaneous_endpoints_square b w T s hT hs m)

theorem connector_in_region (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) :
  Region w T s hT hs m (convexPosition a ha
    (polygonMap w T s hT m t) (gammaPosition w T s hT hs t)) :=
  MatchedRegion.connector_in_region T hT _ _ m t a ha

/-- Reversing a connector changes no unsigned region point. -/
theorem reversed_connector_in_region (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (t : BinaryTime T hT) (a : Fraction) (ha : UnitInterval a) :
  Region w T s hT hs m (convexPosition a ha
    (gammaPosition w T s hT hs t) (polygonMap w T s hT m t)) :=
  MatchedRegion.reversed_connector_in_region T hT _ _ m t a ha

theorem polygon_in_region (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (t : BinaryTime T hT) : Region w T s hT hs m (polygonMap w T s hT m t) :=
  MatchedRegion.polygon_in_region T hT _ _ m t

theorem curve_in_region (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (t : BinaryTime T hT) : Region w T s hT hs m (gammaPosition w T s hT hs t) :=
  MatchedRegion.curve_in_region T hT _ _ m t

theorem shared_initial_endpoint (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
  polygonMap w T s hT m (leftTime T hT) = gammaPosition w T s hT hs (leftTime T hT) :=
  (polygonMap_left w T s hT m).trans (gammaPosition_left w T s hT hs).symm

/-- The last-cell connector is explicitly present even when endpoints differ. -/
theorem final_connector_in_region (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (a : Fraction) (ha : UnitInterval a) :
  Region w T s hT hs m (convexPosition a ha
    (polygonMap w T s hT m (rightTime T hT))
    (asPosition (endpointValue w T s hT hs))) := by
  have h := connector_in_region w T s hT hs m (rightTime T hT) a ha
  rwa [gammaPosition_right] at h

def budgetCoefficient (w T : Fraction) (s : Point × Point) : Fraction :=
  squareArea (edgeCoefficient w T s)

theorem budgetCoefficient_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) : 0 ≤ (budgetCoefficient w T s).num :=
  squareArea_nonnegative _ (edgeCoefficient_nonnegative w T s hT)

/-- The actual region cover has budget  $4C^2/2^m$ , with C the edge coefficient. -/
theorem actual_budget_geometric (w T : Fraction) (s : Point × Point)

```

```

    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
    Fraction.equiv (actualCover w T s hT hs m).budget
    (duration (budgetCoefficient w T s) m) := by
    exact MatchedRegion.uniform_budget_geometric (cellStart w T s m)
    (coverRadius w T s hT m) m (edgeCoefficient w T s) (edgeRadius_geometric w T s m)

/-- Polygon/constructed-curve outer content, represented exactly by its lower
cut. The definition quantifies over all finite square covers of the actual set. -/
def D_mesh (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num)
    (hs : DyadicSmallTime w T) (m : Nat) : Fraction → Prop :=
    LowerContent (Region w T s hT hs m)

theorem D_mesh_nonnegative (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
    D_mesh w T s hT hs m (Fraction.ofInt 0) := content_zero_lower _

theorem D_mesh_bound (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
    (q : Fraction) (hq : D_mesh w T s hT hs m q) :
    Fraction.le q (duration (budgetCoefficient w T s) m) :=
    Fraction.le_equiv_right (hq (actualCover w T s hT hs m))
    (actual_budget_geometric w T s hT hs m)

/-- Nonnegative outer contents tend to zero: every positive rational test bound
fails to be a lower bound from one explicit mesh onward. -/
theorem D_mesh_tends_zero (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
    (eps : Fraction) (heps : 0 < eps.num) :
    ∃ N, ∀ m, N ≤ m → ∀ q, D_mesh w T s hT hs m q → Fraction.lt q eps := by
    obtain ⟨N, hN⟩ := duration_eventually_small (budgetCoefficient w T s) eps
    (budgetCoefficient_nonnegative w T s hT) heps
    exact ⟨N, fun m hm q hq => Fraction.magnitudes.lt_of_le_lt
        (D_mesh_bound w T s hT hs m q hq) (hN m hm)⟩

theorem D_mesh_zero_window (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
    (hz : T.num = 0) (q : Fraction) :
    D_mesh w T s hT hs m q ↔ Fraction.le q (Fraction.ofInt 0) := by
    constructor
    · intro hq
      have hb := D_mesh_bound w T s hT hs m q hq
      apply Fraction.le_equiv_right hb
      simp [budgetCoefficient, SquareOuterContent.squareArea, edgeCoefficient,
          HarmonicBinaryPrefix.coefficient, duration, Fraction.equiv, Fraction.add,
          Fraction.mul, Fraction.ofInt, hz]
    · intro hq
      exact content_downward _ _ _ hq (content_zero_lower _)

end NewtonLimitDynamics.Polygon.HarmonicPathRegion

```

NewtonLimitDynamics/Polygon/HarmonicPathContent.lean, 6 theorems, 61 lines

```

import NewtonLimitDynamics.Polygon.HarmonicPathRegion
import BarrowLib.Polygon.SquareContentValues

/--! Cauchy scalar outer content of the actual constructed harmonic matched
region. The all-cover infimum is represented exactly; the scalar is independent
of the initial covering budget. Ordinary Euclidean area and P5 are separate. -/
namespace NewtonLimitDynamics.Polygon.HarmonicPathContent
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic HarmonicTimeRealization CauchyValues PositionValues BinaryTime
open ScalarOrder SquareOuterContent SquareContentValues HarmonicPolygonCurve HarmonicPathRegion

noncomputable def D_meshValue (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) : ScalarValue :=
    contentValue (Region w T s hT hs m) (actualCover w T s hT hs m)

/-- The constructed scalar has precisely the already proved all-cover lower cut. -/
theorem D_meshValue_lower_cut (w T : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) (q : Fraction) :
    Below q (D_meshValue w T s hT hs m).val ↔ D_mesh w T s hT hs m q :=

```

```

    contentValue_lower_cut _ _ q

theorem D_meshValue_nonnegative (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
  Below (Fraction.ofInt 0) (D_meshValue w T s hT hs m).val :=
  contentValue_nonnegative _ _

/-- This is a scalar bound, derived from containment of the actual region. -/
theorem D_meshValue_budget_bound (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
  Within (D_meshValue w T s hT hs m).val (embed (scalarState (Fraction.ofInt 0)))
    (duration (budgetCoefficient w T s) m) :=
  within_mono _ _ _ (Fraction.le_of_equiv (actual_budget_geometric w T s hT hs m))
    (contentValue_within_zero _ _)

theorem D_meshValue_tends_zero (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (eps : Fraction)
  (heps : 0 < eps.num) :
  ∃ N, ∀ m, N ≤ m →
    Within (D_meshValue w T s hT hs m).val (embed (scalarState (Fraction.ofInt 0))) eps := by
  obtain ⟨N, hN⟩ := duration_eventually_small (budgetCoefficient w T s) eps
  (budgetCoefficient_nonnegative w T s hT) heps
  exact ⟨N, fun m hm => within_mono
    (Fraction.magnitudes.lt_implies_le (hN m hm)) (D_meshValue_budget_bound w T s hT hs m)⟩

theorem D_meshValue_independent_cover (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat)
  (c : Cover (Region w T s hT hs m)) :
  D_meshValue w T s hT hs m = contentValue (Region w T s hT hs m) c :=
  contentValue_independent_cover _ _ c

theorem D_meshValue_zero_window (w T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) (hz : T.num = 0) :
  (D_meshValue w T s hT hs m).val = embed (scalarState (Fraction.ofInt 0)) := by
  apply (within_zero_iff _ _).mp
  apply within_mono _ _ _ (D_meshValue_budget_bound w T s hT hs m)
  apply Fraction.le_of_equiv
  simp [budgetCoefficient, SquareOuterContent.squareArea, edgeCoefficient,
    HarmonicBinaryPrefix.coefficient, duration, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt, hz]

```

end NewtonLimitDynamics.Polygon.HarmonicPathContent

NewtonLimitDynamics/Polygon/GeneralForcePathRegion.lean, 10 theorems, 163 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePolygonCurve
import BarrowLib.Polygon.MatchedRegion

```

```

/-! The actual unsigned region between the sampled central-force polygon and
its constructed curve. The shared matched-region geometry retains all cell
closures and the final connector. Covers are derived from actual vertex and
prefix bounds; no area or enclosure of a supplied curve is a premise. -/

```

```

namespace NewtonLimitDynamics.Polygon.GeneralForcePathRegion
open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicBinaryPrefix HarmonicTimeRealization BinaryTime
open CauchyValues PositionValues ConvexCover ConvexValues CompletionGeometry SquareOuterContent
open GeneralForceEndpoint GeneralForcePrefix GeneralForceTime GeneralForcePolygonCurve

```

```

noncomputable def cellStart (o : ForceClasses.CentralOracle) (E0 T : Fraction)
  (s : Point × Point) (hE : 0 < E0.num) (m k : Nat) : Point :=
  (countState o E0 T s hE m k).1

```

```

def cellPatch (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m k : Nat)
  (x : PositionValue) : Prop :=
  MatchedRegion.cellPatch T d.time_nonnegative
    (polygonMap o E0 T s hE d.time_nonnegative m)
    (gammaPosition o E0 T tau L B s hE d) m k x

```

```

def Region (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (x : PositionValue) : Prop :=
  MatchedRegion.Region T d.time_nonnegative
  (polygonMap o E0 T s hE d.time_nonnegative m)
  (gammaPosition o E0 T tau L B s hE d) m x

noncomputable def coverRadius (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : NonnegativeRadius :=
  (duration (edgeCoefficient E0 T tau L B s d.calibration_positive) m,
   edgeCoefficient_nonnegative E0 T tau L B s hE d.time_nonnegative d.calibration_positive
    d.lipschitz.1 d.bound_nonnegative)

theorem simultaneous_endpoints_square (b : Nat → Bool) (o : ForceClasses.CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  CoordinateSquare (cellStart o E0 T s hE m (ticks b m)) (coverRadius o E0 T tau L B s hE d m)
    (polygonMap o E0 T s hE d.time_nonnegative m (Quotient.mk _ b)) ∧
  CoordinateSquare (cellStart o E0 T s hE m (ticks b m)) (coverRadius o E0 T tau L B s hE d m)
    (gammaPosition o E0 T tau L B s hE d (Quotient.mk _ b)) := by
  have hD : 0 ≤ (Fraction.mul (duration T m) (velocityCap T B s)).num :=
    Fraction.nonnegative_mul _ _ d.time_nonnegative
    (velocityCap_nonnegative T B s d.time_nonnegative d.bound_nonnegative)
  have hE' : 0 ≤ (GeometricTail.tailCap
    (GeneralForcePrefix.coefficient E0 T tau L B s d.calibration_positive) m).num :=
    GeneralForcePrefix.coefficient_nonnegative E0 T tau L B s hE d.time_nonnegative
    d.calibration_positive d.lipschitz.1 d.bound_nonnegative
  have hpoly := polygon_vertex_bound b o E0 T tau L B s hE d m
  have hcurve := within_symm _ _ (positionValue_within _ _
    (prefix_value_bound b o E0 T tau L B s hE d m))
  have hr := edgeRadius_geometric E0 T tau L B s d.calibration_positive m
  have hp := within_mono _ _ _
    (Fraction.le_equiv_right (Fraction.le_add_nonnegative _ _ hE') hr) hpoly
  have hg := within_mono _ _ _
    (Fraction.le_equiv_right
      (Fraction.le_equiv_right (Fraction.le_add_nonnegative _ _ hD) (Fraction.add_comm _ _)) hr) hcurve
  exact (square_of_ball_bound _ _ hp, square_of_ball_bound _ _ hg)

/-- One proved square per actual coarse cell, covering its closure. -/
noncomputable def actualCover (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Cover (Region o E0 T tau L B s hE d m) :=
  MatchedRegion.actualCover T d.time_nonnegative _ _ m (cellStart o E0 T s hE m)
  (coverRadius o E0 T tau L B s hE d m)
  (fun b => simultaneous_endpoints_square b o E0 T tau L B s hE d m)

theorem connector_in_region (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (t : BinaryTime T d.time_nonnegative) (a : Fraction) (ha : UnitInterval a) :
  Region o E0 T tau L B s hE d m (convexPosition a ha
    (polygonMap o E0 T s hE d.time_nonnegative m t) (gammaPosition o E0 T tau L B s hE d t)) :=
  MatchedRegion.connector_in_region T d.time_nonnegative _ _ m t a ha

/-- The final endpoint may differ from its coarse vertex, so its connector is
retained explicitly rather than assumed to vanish. -/
theorem final_connector_in_region (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (a : Fraction) (ha : UnitInterval a) :
  Region o E0 T tau L B s hE d m (convexPosition a ha
    (polygonMap o E0 T s hE d.time_nonnegative m (rightTime T d.time_nonnegative))
    (asPosition (endpointValue o E0 T tau L B s hE d.toConditions))) := by
  have h := connector_in_region o E0 T tau L B s hE d m (rightTime T d.time_nonnegative) a ha
  change Region o E0 T tau L B s hE d m (convexPosition a ha
    (asPosition (gammaValue o E0 T tau L B s hE d (rightTime T d.time_nonnegative)))) at h
  rwa [GeneralForceTime.right_endpoint_value] at h

```

```

noncomputable def budgetCoefficient (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) : Fraction := squareArea (edgeCoefficient E0 T tau L B s ht)

theorem budgetCoefficient_nonnegative (E0 T tau L B : Fraction) (s : Point × Point)
  (hE : 0 < E0.num) (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (budgetCoefficient E0 T tau L B s ht).num :=
  squareArea_nonnegative _ (edgeCoefficient_nonnegative E0 T tau L B s hE hT ht hL hB)

theorem actual_budget_geometric (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Fraction.equiv (actualCover o E0 T tau L B s hE d m).budget
    (duration (budgetCoefficient E0 T tau L B s d.calibration_positive) m) :=
  MatchedRegion.uniform_budget_geometric (cellStart o E0 T s hE m)
    (coverRadius o E0 T tau L B s hE d m) m
    (edgeCoefficient E0 T tau L B s d.calibration_positive) (Fraction.equiv_refl _)

def D_mesh (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : Fraction → Prop :=
  LowerContent (Region o E0 T tau L B s hE d m)

theorem D_mesh_nonnegative (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  D_mesh o E0 T tau L B s hE d m (Fraction.ofInt 0) := content_zero_lower _

theorem D_mesh_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (q : Fraction) (hq : D_mesh o E0 T tau L B s hE d m q) :
  Fraction.le q (duration (budgetCoefficient E0 T tau L B s d.calibration_positive) m) :=
  Fraction.le_equiv_right (hq (actualCover o E0 T tau L B s hE d m))
    (actual_budget_geometric o E0 T tau L B s hE d m)

theorem D_mesh_tends_zero (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N, ∀ m, N ≤ m → ∀ q, D_mesh o E0 T tau L B s hE d m q → Fraction.lt q eps := by
  obtain ⟨N, hN⟩ := duration_eventually_small
    (budgetCoefficient E0 T tau L B s d.calibration_positive) eps
    (budgetCoefficient_nonnegative E0 T tau L B s hE d.time_nonnegative d.calibration_positive
      d.lipschitz.1 d.bound_nonnegative) heps
  exact ⟨N, fun m hm q hq => Fraction.magnitudes.lt_of_le_lt
    (D_mesh_bound o E0 T tau L B s hE d m q hq) (hN m hm)⟩

theorem budget_zero_window (E0 T tau L B : Fraction) (s : Point × Point)
  (ht : 0 < tau.num) (m : Nat) (hz : T.num=0) :
  Fraction.equiv (duration (budgetCoefficient E0 T tau L B s ht) m) (Fraction.ofInt 0) := by
  simp [budgetCoefficient, squareArea, edgeCoefficient, GeneralForcePrefix.coefficient,
    GeneralForcePrefix.weightedCoefficient, GeneralForceEndpoint.weightedCoefficient,
    duration, Fraction.equiv, Fraction.add, Fraction.mul, Fraction.ofInt, hz]

theorem D_mesh_zero_window (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (hz : T.num=0) (q : Fraction) :
  D_mesh o E0 T tau L B s hE d m q ↔ Fraction.le q (Fraction.ofInt 0) := by
  constructor
  · intro hq
    exact Fraction.le_equiv_right (D_mesh_bound o E0 T tau L B s hE d m q hq)
      (budget_zero_window E0 T tau L B s d.calibration_positive m hz)
  · intro hq
    exact content_downward _ _ hq (content_zero_lower _)

end NewtonLimitDynamics.Polygon.GeneralForcePathRegion

```

NewtonLimitDynamics/Polygon/GeneralForcePathContent.lean, 6 theorems, 66 lines

import NewtonLimitDynamics.Polygon.GeneralForcePathRegion

```

import BarrowLib.Polygon.SquareContentValues

/-! Canonical Cauchy scalar outer content of the actual general matched
polygon/curve region. The all-cover cut, enclosure and geometric decay are
derived; the choice of initial cover affects no completed content value. -/

namespace NewtonLimitDynamics.Polygon.GeneralForcePathContent
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic HarmonicTimeRealization CauchyValues PositionValues BinaryTime
open ScalarOrder SquareOuterContent SquareContentValues GeneralForcePathRegion

noncomputable def D_meshValue (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) : ScalarValue :=
  contentValue (Region o E0 T tau L B s hE d m) (actualCover o E0 T tau L B s hE d m)

theorem D_meshValue_lower_cut (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (q : Fraction) :
  Below q (D_meshValue o E0 T tau L B s hE d m).val ↔ D_mesh o E0 T tau L B s hE d m q :=
  contentValue_lower_cut _ _ q

theorem D_meshValue_nonnegative (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Below (Fraction.ofInt 0) (D_meshValue o E0 T tau L B s hE d m).val :=
  contentValue_nonnegative _ _

theorem D_meshValue_budget_bound (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) :
  Within (D_meshValue o E0 T tau L B s hE d m).val (embed (scalarState (Fraction.ofInt 0)))
  (duration (budgetCoefficient E0 T tau L B s d.calibration_positive) m) :=
  within_mono _ _ _ (Fraction.le_of_equiv (actual_budget_geometric o E0 T tau L B s hE d m))
  (contentValue_within_zero _ _)

theorem D_meshValue_tends_zero (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N, ∀ m, N ≤ m →
    Within (D_meshValue o E0 T tau L B s hE d m).val (embed (scalarState (Fraction.ofInt 0))) eps := by
  obtain {N,hN} := duration_eventually_small
    (budgetCoefficient E0 T tau L B s d.calibration_positive) eps
    (budgetCoefficient_nonnegative E0 T tau L B s hE d.time_nonnegative d.calibration_positive
     d.lipschitz.1 d.bound_nonnegative) heps
  exact {N,fun m hm => within_mono _ _ _
    (Fraction.magnitudes.lt_implies_le (hN m hm)) (D_meshValue_budget_bound o E0 T tau L B s hE d m))

theorem D_meshValue_independent_cover (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat)
  (c : Cover (Region o E0 T tau L B s hE d m)) :
  D_meshValue o E0 T tau L B s hE d m = contentValue (Region o E0 T tau L B s hE d m) c :=
  contentValue_independent_cover _ _ c

theorem D_meshValue_zero_window (o : ForceClasses.CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (m : Nat) (hz : T.num=0) :
  (D_meshValue o E0 T tau L B s hE d m).val = embed (scalarState (Fraction.ofInt 0)) := by
  apply (within_zero_iff _ _).mp
  exact within_mono _ _ _
    (Fraction.le_of_equiv (budget_zero_window E0 T tau L B s d.calibration_positive m hz))
    (D_meshValue_budget_bound o E0 T tau L B s hE d m)

end NewtonLimitDynamics.Polygon.GeneralForcePathContent

```

NewtonLimitDynamics/Polygon/HarmonicGeneralPathContent.lean, 2 theorems, 34 lines

```

import NewtonLimitDynamics.Polygon.GeneralForcePathContent
import NewtonLimitDynamics.Polygon.HarmonicPathContent
import NewtonLimitDynamics.Polygon.HarmonicGeneralPolygon

```

```

import NewtonLimitDynamics.Polygon.HarmonicGeneralTime

/-! The actual general harmonic matched region equals the retained region.
Their scalar contents agree despite different proved cover coefficients. -/

namespace NewtonLimitDynamics.Polygon.HarmonicGeneralPathContent
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic ForceClasses HarmonicGeneralEndpoint HarmonicGeneralTime
open PositionValues SquareContentValues

theorem harmonic_region_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (m : Nat) (x : PositionValue) :
  GeneralForcePathRegion.Region (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
    (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) m x ↔
    HarmonicPathRegion.Region w T s hT hs m x := by
  have hp := funext (fun t : BinaryTime.BinaryTime T hT =>
    HarmonicGeneralPolygon.harmonic_polygonMap_eq w E0 T s hw hE hT m t)
  have hg := funext (fun t : BinaryTime.BinaryTime T hT =>
    harmonic_position_eq w E0 T s hw hE hT hs t)
  change MatchedRegion.Region T hT _ _ m x ↔ MatchedRegion.Region T hT _ _ m x
  rw [hp, hg]

theorem harmonic_content_eq (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (m : Nat) :
  GeneralForcePathContent.D_meshValue (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
    (harmonicBound w s) s hE (conditions w E0 T s hw hE hT hs) m =
  HarmonicPathContent.D_meshValue w T s hT hs m :=
  contentValue_region_congr _ _ (harmonic_region_eq w E0 T s hw hE hT hs m) _ _

end NewtonLimitDynamics.Polygon.HarmonicGeneralPathContent

NewtonLimitDynamics/Polygon/ParallelQuadraticEndpoint.lean, 4 theorems, 83 lines

import BarrowLib.Polygon.QuadraticEstimates
import BarrowLib.Polygon.CauchyValues
import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.GeometricTail

/-! Actual centre-at-infinity polygon endpoints converge to the explicit
quadratic state at every nonnegative rational time. Cauchy names are derived
from the finite half-mesh error, with no supplied curve or small window. -/

namespace NewtonLimitDynamics.Polygon.ParallelQuadraticEndpoint
open NewtonLimitDynamics
open TimeSubdivision PointBounds FiniteEstimates BoundedIteration KinematicEstimates QuadraticEstimates
open HarmonicDyadic HarmonicTimeRealization CauchyValues PositionValues

def endpoint (a : Point) (T : Fraction) (s : Point × Point) (j : Nat) : Point × Point :=
  run (fun _ => a) (duration T j) s (blocks j)

def quadraticState (a : Point) (T : Fraction) (s : Point × Point) : Point × Point :=
  (quadraticPosition T s a, pointAdd s.2 (pointScale T a))

def coefficient (a : Point) (T : Fraction) : Fraction :=
  Fraction.mul (Fraction.mul T T).half (pointNorm a)

theorem coefficient_nonnegative (a : Point) (T : Fraction) (hT : 0 ≤ T.num) :
  0 ≤ (coefficient a T).num :=
  Fraction.nonnegative_mul _ _ (Fraction.nonnegative_mul _ _ hT hT) (pointNorm_nonnegative a)

theorem full_elapsed (T : Fraction) (j : Nat) :
  Fraction.equiv (time (duration T j) (blocks j)) T := by
  simp [time, duration, blocks, Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.natCast_pow]
  ac_nf

/-- The exact finite error has only the position half-mesh term. -/
theorem endpoint_error (a : Point) (T : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (j : Nat) :
  Fraction.equiv (distance (endpoint a T s j) (quadraticState a T s))
    (duration (coefficient a T) j) := by

```



```

have hc := constant_run_formula a (duration T j) s (blocks j)
have he := full_elapsed T j
have hp := Fraction.equiv_trans
  (pointDistance_equiv hc.1 (pointEquiv_symm (quadratic_time_congr _ T s a he)))
  (half_mesh_bias (duration T j) s a hT (blocks j))
have hv : pointEquiv (endpoint a T s j).2 (quadraticState a T s).2 :=
  pointEquiv_trans hc.2 (pointAdd_congr (Fraction.equiv_refl _, Fraction.equiv_refl _)
    (pointScale_ratio_congr he (Fraction.equiv_refl _, Fraction.equiv_refl _)))
have hzero := Fraction.equiv_trans
  (pointDistance_equiv hv (Fraction.equiv_refl _, Fraction.equiv_refl _))
  (pointDistance_self_zero (quadraticState a T s).2)
apply Fraction.equiv_trans (Fraction.add_equiv hp hzero)
apply Fraction.equiv_trans (Fraction.add_zero _)
apply Fraction.equiv_trans
  (Fraction.mul_equiv (RationalIntervals.half_equiv
    (Fraction.mul_equiv he (Fraction.equiv_refl _))) (Fraction.equiv_refl _))
simp only [coefficient, duration, Fraction.equiv, Fraction.mul, Fraction.half]
ac_nf

def endpointName (a : Point) (T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) :
  EndpointCauchyName where
  approx := endpoint a T s
  cauchy := by
    intro eps heps
    obtain (N, hN) := duration_eventually_small (coefficient a T) eps.half
    (coefficient_nonnegative a T hT) heps
    refine (N, fun m n hn => ?_)
    have ha := Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv (endpoint_error a T s hT m)) (hN m
      ↪ hn)
    have hb := Fraction.magnitudes.lt_of_le_lt
      (Fraction.le_of_equiv (Fraction.equiv_trans (stateDistance_symm _ _) (endpoint_error a T s hT n)))
      (hN n hn)
    exact Fraction.magnitudes.lt_of_le_lt (stateDistance_triangle _ (quadraticState a T s) _)
      (Fraction.magnitudes.lt_of_lt_le (Fraction.add_lt_add ha hb) (Fraction.le_of_equiv
        ↪ (Fraction.half_add_self eps)))

def endpointValue (a : Point) (T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) : Value :=
  realize (endpointName a T s hT)

theorem endpointValue_eq (a : Point) (T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) :
  endpointValue a T s hT = embed (quadraticState a T s) := by
  apply Quotient.sound
  intro eps heps
  obtain (N, hN) := duration_eventually_small (coefficient a T) eps
  (coefficient_nonnegative a T hT) heps
  exact (N, fun n hn => Fraction.magnitudes.lt_of_le_lt
    (Fraction.le_of_equiv (endpoint_error a T s hT n)) (hN n hn))

end NewtonLimitDynamics.Polygon.ParallelQuadraticEndpoint

```

NewtonLimitDynamics/Diagnostic/QuadraticEndpointPotential.lean, 4 theorems, 68 lines

```

import NewtonLimitDynamics.Diagnostic.DeflectionPotential
import NewtonLimitDynamics.Polygon.ParallelQuadraticEndpoint

/-! The half-coefficient deflection triangle and Galilean potential relation
at actual constructed rational endpoint values. The completed endpoint is
derived from the constant-force polygons. This is not D_mesh or a universal
action scale, and no general curved potential expansion is assumed. -/

namespace NewtonLimitDynamics.Diagnostic.QuadraticEndpointPotential
open NewtonLimitDynamics
open Polygon.TimeSubdivision Polygon.HarmonicStability Polygon.TriangleBounds
open Polygon.QuadraticEstimates Polygon.PositionValues Polygon.ParallelQuadraticEndpoint
open DeflectionPotential

theorem quadratic_deflection_triangle (h : Fraction) (B v a : Point) :
  Fraction.equiv (triangleTwice B (inertial h B v) (quadraticPosition h (B, v) a))
  (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v a)).half := by
  simp only [triangleTwice, inertial, quadraticPosition, det, pointSub, pointNeg,

```

```

    pointAdd,pointScale,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.half,
    Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg]
ac_nf <=> omega

theorem galilean_quadratic_potential_step (m g h : Fraction) (B v : Point) :
  Fraction.equiv
    (potentialStep (linearPotential m g) (inertial h B v)
      (quadraticPosition h (B,v) (fallAcceleration g)))
    (negF (Fraction.mul (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h))).half := by
simp only [potentialStep,linearPotential,inertial,quadraticPosition,fallAcceleration,
  negF,pointAdd,pointScale,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.half,
  Fraction.ofInt,Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,
  Int.zero_mul,Int.mul_zero,Int.mul_one,Int.one_mul]
ac_nf <=> omega

/-- Both triangle and potential drop acquire the same half, so the motion-
dependent time ratio is unchanged from the finite kick comparison. -/
theorem galilean_quadratic_cross_relation (m g h : Fraction) (B v : Point) :
  Fraction.equiv
    (Fraction.mul (Fraction.mul m g)
      (triangleTwice B (inertial h B v) (quadraticPosition h (B,v) (fallAcceleration g))))
    (Fraction.mul (Fraction.mul h v.1)
      (potentialStep (linearPotential m g) (inertial h B v)
        (quadraticPosition h (B,v) (fallAcceleration g)))) := by
simp only [potentialStep,linearPotential,triangleTwice,inertial,quadraticPosition,
  fallAcceleration,det,negF,pointSub,pointNeg,pointAdd,pointScale,
  Fraction.equiv,Fraction.add,Fraction.mul,Fraction.half,Fraction.ofInt,
  Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg,
  Int.zero_mul,Int.mul_zero,Int.mul_one,Int.one_mul]
ac_nf <=> omega

/-- The point in the potential/triangle formulas is the actual completed
endpoint, rather than a supplied quadratic curve point. -/
theorem galilean_constructed_endpoint_relation (m g h : Fraction) (B v : Point)
  (hh : 0 ≤ h.num) :
  ∃ C : Point,
    asPosition (endpointValue (fallAcceleration g) h (B,v) hh)=embedPosition C ∧
    Fraction.equiv (triangleTwice B (inertial h B v) C)
      (Fraction.mul (Fraction.mul (Fraction.mul h h) h) (det v (fallAcceleration g))).half ∧
    Fraction.equiv (potentialStep (linearPotential m g) (inertial h B v) C)
      (negF (Fraction.mul (Fraction.mul m (Fraction.mul g g)) (Fraction.mul h h))).half ∧
    Fraction.equiv (Fraction.mul (Fraction.mul m g) (triangleTwice B (inertial h B v) C))
      (Fraction.mul (Fraction.mul h v.1) (potentialStep (linearPotential m g) (inertial h B v) C)) := by
refine (quadraticPosition h (B,v) (fallAcceleration g),?,
  quadratic_deflection_triangle h B v (fallAcceleration g),
  galilean_quadratic_potential_step m g h B v,galilean_quadratic_cross_relation m g h B v)
rw [endpointValue_eq]
rfl

```

end NewtonLimitDynamics.Diagnostic.QuadraticEndpointPotential

NewtonLimitDynamics/Polygon/GeneralForceQuadraticSecants.lean, 5 theorems, 185 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceAccelerationSecants
import BarrowLib.Polygon.QuadraticSecants

/--! The normalized departure from the constructed tangent over a dyadic cell
converges to the completed force. Finite quadratic remainders and proved
Cauchy-name boundedness remove the actual half-mesh and sampling errors.
No Taylor expansion, derivative or desired second-order equation is supplied. -/

namespace NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants
open NewtonLimitDynamics
open TimeSubdivision PointBounds ForceClasses HarmonicDyadic HarmonicBinaryPrefix
open CauchyValues BinaryTime PositionValues SecantValues DyadicNodes HarmonicTimeRealization
open GeneralForceEndpoint GeneralForcePrefix GeneralForceTime GeneralForceSecants

noncomputable def cellSecondSecant (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m k : Nat) : Value :=
  QuadraticSecants.secondValue (duration T m) hT

```

```

(gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
(gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1)))

/-- The actual finite node approximants retain their sample and half-mesh
errors. Both the completed second-order bridge and potential calculations use
this estimate, rather than repeating the restarted-run proof. -/
theorem node_second_sample_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) (j : Nat) :
  Fraction.le
    (FiniteEstimates.pointDistance (QuadraticSecants.secondState (duration T m) hT
      ((nodeName o E0 T tau L B s hE d m k).approx j)
      ((nodeName o E0 T tau L B s hE d m (k+1)).approx j)).1
      (field o E0 hE (m+j) ((nodeName o E0 T tau L B s hE d m k).approx j).1))
    (Fraction.add
      (Fraction.mul (Fraction.ofInt 2) (Fraction.mul L (Fraction.mul (duration T m) (velocityCap T B
        ↪ s))))
      (Fraction.add (Fraction.mul (sampleError o E0 hE (m+j)) (Fraction.ofInt 2))
        (Fraction.mul (duration (Fraction.ofInt 1) j)
          (pointNorm (field o E0 hE (m+j) ((nodeName o E0 T tau L B s hE d m k).approx j).1)))))) := by
  rw [node_approx o E0 T tau L B s hE d m k (by omega) j,
    node_approx o E0 T tau L B s hE d m (k+1) hk j]
  let a := field o E0 hE (m+j)
  let h := duration T (m+j)
  let n := k*blocks j
  let q := GeneralForcePrefix.countState o E0 T s hE (m+j) n
  have hn : n+blocks j ≤ blocks (m+j) := by
    rw [blocks_add]
    have hm := Nat.mul_le_mul_right (blocks j) hk
    simpa only [n, Nat.add_mul, Nat.one_mul] using hm
  have hp : 0 < (blocks j : Int) := Int.ofNat_pos.mpr (by unfold blocks; exact Nat.pow_pos (by decide))
  have ht : 0 < (BoundedIteration.time h (blocks j)).num := Int.mul_pos hp hT
  have hv : ∀ i, i < blocks j → Fraction.le
    (pointNorm (BoundedIteration.run a h q i).2) (velocityCap T B s) := by
    intro i hi
    change Fraction.le (pointNorm (BoundedIteration.run a h (BoundedIteration.run a h s n) i).2) _
    rw [← BoundedIteration.run_add]
    exact GeneralForcePrefix.count_velocity o E0 T tau L B s hE d (m+j) (n+i) (by omega)
  have hr := QuadraticSecants.finite_second_equivalent_time_at a h q L (sampleError o E0 hE (m+j))
    (velocityCap T B s) (duration T m) (Int.le_of_lt hT) d.lipschitz.1
    (sampleError_nonnegative o E0 hE (m+j))
    (velocityCap_nonnegative T B s d.time_nonnegative d.bound_nonnegative)
    (blocks j) hT
    (fun i hi => restarted_comparison o E0 T tau L B s hE d (m+j) n (blocks j) hn i hi) hT
    (Fraction.equiv_symm (grid_time T m j)) hv
  have he : n+blocks j = (k+1)*blocks j := by simp only [n, Nat.add_mul, Nat.one_mul]
  change Fraction.le (FiniteEstimates.pointDistance
    (QuadraticSecants.secondState (duration T m) hT (BoundedIteration.run a h s n)
      (BoundedIteration.run a h (BoundedIteration.run a h s n) (blocks j)).1
      (a (BoundedIteration.run a h s n).1)) _ at hr
  rw [← BoundedIteration.run_add, he] at hr
  have hbias : Fraction.equiv (Fraction.mul h (TimeCalibration.inverse (duration T m) hT))
    (duration (Fraction.ofInt 1) j) := by
    simp only [h, duration, TimeCalibration.inverse, Fraction.equiv, Fraction.mul, Fraction.ofInt,
      FiniteGrowth.denominator_power_add]
  ac_nf
  apply Fraction.le_equiv_right hr
  apply Fraction.equiv_trans (Fraction.add_equiv (Fraction.equiv_refl _)
    (Fraction.mul_equiv hbias (Fraction.equiv_refl _)))
  simp only
  ↪ [AccelerationEstimates.source, Fraction.equiv, Fraction.add, Fraction.mul, Int.add_mul, Int.mul_add]
  ac_nf

theorem cell_second_secant_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) :
  Within (cellSecondSecant o E0 T tau L B s hE d hT m k)
    (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
      ↪ d.time_nonnegative m k))

```

```

    (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))
    (Fraction.mul (Fraction.ofInt 2) (Fraction.mul L (Fraction.mul (duration T m) (velocityCap T B
    ↪ s)))) := by
rw [GeneralForceAccelerationSecants.node_force_value o E0 T tau L B s hE d m k,
    cellSecondSecant, QuadraticSecants.secondValue,
    ↪ node_value o E0 T tau L B s hE d m k, ↪ node_value o E0 T tau L B s hE d m (k+1)]
let f := CompletedForce.family o E0 L hE d.lipschitz
change Within (realize (secantName (Fraction.mul (Fraction.ofInt 2) (TimeCalibration.inverse (duration
    ↪ T m) hT))
    (secantName (TimeCalibration.inverse (duration T m) hT)
    (nodeName o E0 T tau L B s hE d m (k+1)) (nodeName o E0 T tau L B s hE d m k))
    (mapName velocityState velocity_nonexpansive (nodeName o E0 T tau L B s hE d m k))))
    (SampledValues.sampledValue f (realize (nodeName o E0 T tau L B s hE d m k))
    (node_admissible o E0 T tau L B s hE d m k)) _
rw [↪ SampledValues.sampledValue_offset f _ _ m,
    SampledValues.sampledValue_realize (SampledValues.offsetFamily f m)
    (nodeName o E0 T tau L B s hE d m k) (node_region o E0 T tau L B s hE d m k)]
let g := SampledValues.sampledName (SampledValues.offsetFamily f m) (nodeName o E0 T tau L B s hE d m k)
    (node_region o E0 T tau L B s hE d m k)
change NameBound _ g _
apply SampledValues.nameBound_of_vanishing_error _ _ _ (fun j =>
    Fraction.add (Fraction.mul (sampleError o E0 hE (m+j)) (Fraction.ofInt 2))
    (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (g.approx j).1))))
· intro eps heps
let delta := factorDelta (Fraction.ofInt 2) eps.half (by decide)
have hd : 0 < delta.num := factorDelta_positive _ _ _ heps
obtain (N, hN) := CompletedForce.sampleError_vanishes o E0 hE delta hd
obtain (M, hM) := mesh_position_product_vanishes g eps.half heps
refine (max N M, fun j hj => ?_)
have h1 := factor_control (Fraction.ofInt 2) eps.half (sampleError o E0 hE (m+j))
    (by decide) (sampleError_nonnegative o E0 hE (m+j)) (hN (m+j) (by omega))
exact CauchyValues.lt_equiv_right (Fraction.add_lt_add h1 (hM j (by omega))) (Fraction.half_add_self
    ↪ eps)
· intro j
change Fraction.le
    (distance (QuadraticSecants.secondState (duration T m) hT
    ((nodeName o E0 T tau L B s hE d m k).approx j)
    ((nodeName o E0 T tau L B s hE d m (k+1)).approx j))
    (accelerationState (field o E0 hE (m+j)) ((nodeName o E0 T tau L B s hE d m k).approx j).1)))
    (Fraction.add _ (Fraction.add (Fraction.mul (sampleError o E0 hE (m+j)) (Fraction.ofInt 2))
    (Fraction.mul (duration (Fraction.ofInt 1) j)
    (pointNorm (field o E0 hE (m+j)) ((nodeName o E0 T tau L B s hE d m k).approx j).1))))
    exact Fraction.le_equiv_left (pointState_distance _ _)
    (node_second_sample_bound o E0 T tau L B s hE d hT m k hk j)

noncomputable def secondCoefficient (T tau L B : Fraction) (s : Point × Point)
    (ht : 0 < tau.num) : Fraction :=
    Fraction.mul L (Fraction.add (Fraction.mul (Fraction.ofInt 2) (velocityCap T B s))
    (stateTimeFactor T tau B s ht))

theorem secondCoefficient_nonnegative (T tau L B : Fraction) (s : Point × Point)
    (hT : 0 ≤ T.num) (ht : 0 < tau.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
    0 ≤ (secondCoefficient T tau L B s ht).num :=
    Fraction.nonnegative_mul _ _ hL (Fraction.nonnegative_add _ _
    (Fraction.nonnegative_mul _ _ (by decide) (velocityCap_nonnegative T B s hT hB))
    (stateTimeFactor_nonnegative T tau B s hT ht hB))

theorem bracketing_second_secant_bound (b : Nat → Bool) (o : CentralOracle)
    (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
    (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num) (m : Nat) :
    Within (cellSecondSecant o E0 T tau L B s hE d hT m (ticks b m))
    (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _ b))
    (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)))
    (Fraction.mul (duration T m) (secondCoefficient T tau L B s d.calibration_positive)) := by
have hs := cell_second_secant_bound o E0 T tau L B s hE d hT m (ticks b m)
    (by have hn := ticks_lt_blocks b m; omega)
have hf := CompletedForce.forceValue_within o E0 L hE d.lipschitz _ _
    (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (ticks b m)))
    (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b)) _
    (gamma_within o E0 T tau L B s hE d

```

```

      (nodeTime T d.time_nonnegative m (ticks b m)) (Quotient.mk _ b) (duration T m)
      (truncation_time_within b T d.time_nonnegative m))
have hb := within_triangle _ _ _ _ _ hs hf
apply within_mono _ _ _ _ ?_ hb
apply Fraction.le_of_equiv
simp only [secondCoefficient, Fraction.equiv, Fraction.mul, Fraction.add, Int.add_mul, Int.mul_add]
ac_nf

/-- The half-coefficient position departure is derived on the actual
completed curve at every bracketing dyadic cell, including the final boundary. -/
theorem dyadic_second_uniform_identification (o : CentralOracle)
  (E0 T tau L B : Fraction) (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (cellSecondSecant o E0 T tau L B s hE d hT m (ticks b m))
      (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (Quotient.mk _
        ↪ b)))
      (gamma_admissible o E0 T tau L B s hE d (Quotient.mk _ b))) eps := by
let C := secondCoefficient T tau L B s d.calibration_positive
have hC := secondCoefficient_nonnegative T tau L B s d.time_nonnegative
  d.calibration_positive d.lipschitz.1 d.bound_nonnegative
obtain ⟨N, hN⟩ := duration_eventually_small (Fraction.mul T C) eps
  (Fraction.nonnegative_mul _ _ d.time_nonnegative hC) heps
have he : ∀ m, Fraction.equiv (Fraction.mul (duration T m) C) (duration (Fraction.mul T C) m) := by
  intro m
  simp only [duration, Fraction.equiv, Fraction.mul]
  ac_nf
exact ⟨N, fun m hm b => within_mono _ _ _ _
  (Fraction.le_equiv_left (he m) (Fraction.magnitudes.lt_implies_le (hN m hm)))
  (bracketing_second_secant_bound b o E0 T tau L B s hE d hT m)⟩

```

end NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants

NewtonLimitDynamics/Polygon/HarmonicQuadraticSecants.lean, 1 theorem, 31 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants
import NewtonLimitDynamics.Polygon.HarmonicCompletedForce

```

```

/-! The retained harmonic curve inherits the constructed normalized
second-order position departure criterion, with all sample bounds derived. -/

```

```

namespace NewtonLimitDynamics.Polygon.HarmonicQuadraticSecants
open NewtonLimitDynamics
open TimeSubdivision HarmonicDyadic HarmonicBinaryPrefix ForceClasses CauchyValues DyadicNodes

```

```

theorem second_secants_converge (w E0 T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hE : 0 < E0.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T)
  (ht : 0 < T.num) (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ b : Nat → Bool,
    Within (QuadraticSecants.secondValue (duration T m) ht
      (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (ticks b m)))
      (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (ticks b m+1))))
      (HarmonicCompletedForce.linearValue w
        (HarmonicTimeRealization.gammaValue w T s hT hs (Quotient.mk _ b))) eps := by
obtain ⟨N, hN⟩ := GeneralForceQuadraticSecants.dyadic_second_uniform_identification
  (harmonicOracle w hw) E0 T (Fraction.ofInt 1) w.abs
  (HarmonicGeneralEndpoint.harmonicBound w s) s hE
  (HarmonicGeneralTime.conditions w E0 T s hw hE hT hs) ht eps heps
refine ⟨N, fun m hm b => ?_⟩
have h := hN m hm b
rw [GeneralForceQuadraticSecants.cellSecondSecant, HarmonicGeneralTime.harmonic_value_eq,
  HarmonicGeneralTime.harmonic_value_eq, HarmonicCompletedForce.completed_linear_force,
  HarmonicGeneralTime.harmonic_value_eq] at h
exact h

```

end NewtonLimitDynamics.Polygon.HarmonicQuadraticSecants

NewtonLimitDynamics/Diagnostic/ConstructedHarmonicPotential.lean, 9 theorems, 264 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants
import NewtonLimitDynamics.Polygon.HarmonicCompletedForce
import BarrowLib.Polygon.QuadraticPotentialValues

/--! The harmonic polynomial potential evaluated at the actual constructed
curve and its tangent continuation. Its normalized per-cell drop tends to
-m*dot(a,a)/2. Finite second-order and half-mesh errors are retained until
proved vanishing. This law test does not construct general radial potentials
or identify the deflection triangle with D_mesh. -/

namespace NewtonLimitDynamics.Diagnostic.ConstructedHarmonicPotential
open NewtonLimitDynamics
open Polygon
open TimeSubdivision PointBounds FiniteEstimates HarmonicStability HarmonicDyadic
open HarmonicTimeComparison HarmonicComparison CauchyValues PositionValues SecantValues
open PairingValues QuadraticPotentialValues DyadicNodes BinaryTime HarmonicTimeRealization
open ForceClasses GeneralForceEndpoint GeneralForceSecants

/-- Finite work identifies the polynomial with the linear central force:
the residual after -mass*a(p) dot (q-p) is the quadratic displacement term. -/
theorem finite_work_remainder (mass w : Fraction) (p q : Point) :
  Fraction.equiv
    (Fraction.add
      (durationDifference (quadratic (Fraction.mul mass w).half p)
        (quadratic (Fraction.mul mass w).half q))
      (Fraction.mul mass (dot (linearField w p) (pointSub q p))))
    (Fraction.mul (Fraction.mul mass w).half (dot (pointSub q p) (pointSub q p))) := by
  simp only [quadratic,durationDifference,linearField,negF,dot,pointSub,pointNeg,
    pointAdd,pointScale,Fraction.equiv,Fraction.add,Fraction.mul,Fraction.half,
    Int.add_mul,Int.mul_add,Int.neg_mul,Int.mul_neg]
  simp only [show (2 : Int)=1+1 by rfl,Int.add_mul,Int.mul_add,Int.one_mul,Int.mul_one]
  ac_nf <;> omega

private structure Frame where
  P : Fraction
  V : Fraction
  A : Fraction
  U : Fraction
  Z : Fraction

private def frame (w T : Fraction) (s : Point × Point) : Frame :=
  let P := Fraction.mul (Fraction.ofInt 2) (stateNorm s)
  let V := velocityCap T (HarmonicGeneralEndpoint.harmonicBound w s) s
  let A := Fraction.mul w.abs P
  let U := Fraction.mul (Fraction.ofInt 2) (Fraction.mul w.abs V)
  (P,V,A,U,Fraction.add A (Fraction.add (Fraction.mul T U) A))

private theorem frame_nonnegative (w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) :
  0 ≤ (frame w T s).P.num ∧ 0 ≤ (frame w T s).V.num ∧ 0 ≤ (frame w T s).A.num ∧
  0 ≤ (frame w T s).U.num ∧ 0 ≤ (frame w T s).Z.num := by
  have hP := Fraction.nonnegative_mul (Fraction.ofInt 2) (stateNorm s) (by decide) (stateNorm_nonnegative
    ↪ s)
  have hV := velocityCap_nonnegative T (HarmonicGeneralEndpoint.harmonicBound w s) s hT
    (HarmonicGeneralEndpoint.harmonicBound_nonnegative w s)
  have hA := Fraction.nonnegative_mul w.abs _ (Fraction.abs_num_nonnegative w) hP
  have hU := Fraction.nonnegative_mul (Fraction.ofInt 2) _ (by decide)
    (Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative w) hV)
  exact {hP,hV,hA,hU,Fraction.nonnegative_add _ _ hA
    (Fraction.nonnegative_add _ _ (Fraction.nonnegative_mul _ _ hT hU) hA)}

def coefficient (mass w T : Fraction) (s : Point × Point) : Fraction :=
  let f := frame w T s
  remainderCoefficient (Fraction.mul mass w).half T f.P f.V f.Z f.U

theorem coefficient_nonnegative (mass w T : Fraction) (s : Point × Point) (hT : 0 ≤ T.num) :
  0 ≤ (coefficient mass w T s).num := by
  obtain {hP,hV,hU,hZ} := frame_nonnegative w T s hT
  exact remainderCoefficient_nonnegative _ _ _ _ hT hP hV hZ hU

/-- The Euclidean squared acceleration is the dot product, rather than the

```

```

coordinate l1 gauge used to bound errors. -/
def forceEnergyValue (mass : Fraction) (a : Value) : Value :=
  secantValue (negF mass.half) (pairingValue dotForm a a) (embed (zeroPoint,zeroPoint))

private def workName (mass w : Fraction) (a : EndpointCauchyName) : EndpointCauchyName :=
  secantName (Fraction.mul mass w).half
  (pairingName dotForm (secantName (negF w) a (constantName (zeroPoint,zeroPoint))) a)
  (constantName (zeroPoint,zeroPoint))

private theorem work_approximant (mass w : Fraction) (a : EndpointCauchyName) (j : Nat) :
  stateEquiv ((workName mass w a).approx j)
  (scalarState (Fraction.mul (Fraction.mul mass w).half
    (dot (linearField w (a.approx j).1) (a.approx j).1))) := by
  let g := secantName (negF w) a (constantName (zeroPoint,zeroPoint))
  have he := scaled_pairing_approximant dotForm (Fraction.mul mass w).half g a j
  have hg : pointEquiv (g.approx j).1 (linearField w (a.approx j).1) :=
    pointScale_congr (negF w) (pointSub_zero _)
  exact ((Fraction.equiv_trans he.1.1 (Fraction.mul_equiv_left _
    (dot_congr hg (Fraction.equiv_refl _, Fraction.equiv_refl _))), he.1.2), he.2)

theorem force_work_eq_energy (mass w : Fraction) (x : Value) :
  secantValue (Fraction.mul mass w).half
  (pairingValue dotForm (HarmonicCompletedForce.linearValue w x) x) (embed (zeroPoint,zeroPoint)) =
  forceEnergyValue mass (HarmonicCompletedForce.linearValue w x) := by
  induction x using Quotient.inductionOn with
  | _ a =>
    apply Quotient.sound
    apply nameEquiv_of_levelwise_stateEquiv
    intro j
    have hzero : zeroPoint=(Fraction.ofInt 0,Fraction.ofInt 0) := rfl
    have hscalar : ∀ q, scalarState q=((q,Fraction.ofInt 0),(Fraction.ofInt 0,Fraction.ofInt 0)) := fun _
      => rfl
    constructor
    · constructor <=>
      simp only [forceEnergyValue,HarmonicCompletedForce.linearValue,secantName,secantState,
        pointState,pairingName,pairingState,dotForm,constantName,hscalar,hzero,dot,
        pointEquiv,pointSub,pointNeg,pointAdd,pointScale,negF,Fraction.equiv,Fraction.add,
        Fraction.mul,Fraction.half,Fraction.ofInt,Int.add_mul,Int.mul_add,Int.neg_mul,
        Int.mul_neg,Int.one_mul,Int.mul_one,Int.zero_mul,Int.mul_zero,Int.add_zero,Int.zero_add,
        Int.neg_zero] <=> ac_nf <=> omega
    · exact (Fraction.equiv_refl _, Fraction.equiv_refl _)

private theorem node_frame_bounds (w T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) (j : Nat) :
  let d := HarmonicGeneralTime.conditions w (Fraction.ofInt 1) T s hw (by decide) hT hs
  let a := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m k
  let b := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m (k+1)
  let z := (QuadraticSecants.secondState (duration T m) ht (a.approx j) (b.approx j)).1
  let f := frame w T s
  Fraction.le (pointNorm (a.approx j).1) f.P ∧
  Fraction.le (pointNorm (a.approx j).2) f.V ∧
  Fraction.le (pointNorm z) f.Z ∧
  Fraction.le (pointDistance z (linearField w (a.approx j).1))
    (Fraction.add (Fraction.mul (duration T m) f.U)
      (Fraction.mul (duration (Fraction.ofInt 1) j) f.A)) := by
  let d := HarmonicGeneralTime.conditions w (Fraction.ofInt 1) T s hw (by decide) hT hs
  let a := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m k
  let b := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m (k+1)
  let f := frame w T s
  let z := (QuadraticSecants.secondState (duration T m) ht (a.approx j) (b.approx j)).1
  have hn : k*blocks j ≤ blocks (m+j) := by
    rw [blocks_add]
    exact Nat.mul_le_mul_right (blocks j) (by omega)
  have ha := node_approx (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m k (by omega) j

```

```

have hp : Fraction.le (pointNorm (a.approx j).1) f.P := by
  rw [ha]
  exact Fraction.magnitudes.le_trans (point_le_state _)
    (HarmonicGeneralTime.actual_run_state_bound w T s hT hs (m+j) (k*blocks j) hn)
have hv : Fraction.le (pointNorm (a.approx j).2) f.V := by
  rw [ha]
  exact GeneralForcePrefix.count_velocity _ _ _ _ _ d _ _ hn
have hf : Fraction.le (pointNorm (linearField w (a.approx j).1)) f.A :=
  Fraction.le_equiv_left (HarmonicGeneralEndpoint.linear_sample_norm _ _)
    (Fraction.mul_le_mul_nonnegative_left hp w.abs (Fraction.abs_num_nonnegative w))
have hb := GeneralForceQuadraticSecants.node_second_sample_bound (harmonicOracle w hw)
  (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s
  (by decide) d ht m k hk j
have he : Fraction.equiv
  (Fraction.add
    (Fraction.mul (Fraction.ofInt 2) (Fraction.mul w.abs (Fraction.mul (duration T m) f.V)))
    (Fraction.add (Fraction.mul (sampleError (harmonicOracle w hw) (Fraction.ofInt 1) (by decide)
      ↪ (m+j))
      (Fraction.ofInt 2))
      (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (linearField w (a.approx j).1))))))
    (Fraction.add (Fraction.mul (duration T m) f.U)
      (Fraction.mul (duration (Fraction.ofInt 1) j) (pointNorm (linearField w (a.approx j).1)))) := by
  simp only [f, frame, sampleError, harmonicOracle, exactOracle, Fraction.equiv, Fraction.add,
    Fraction.mul, Fraction.ofInt, Int.zero_mul, Int.mul_zero, Int.add_zero, Int.zero_add,
    Int.one_mul, Int.mul_one, Int.add_mul, Int.mul_add]
ac_nf
have hd : Fraction.le (pointDistance z (linearField w (a.approx j).1))
  (Fraction.add (Fraction.mul (duration T m) f.U)
    (Fraction.add (Fraction.mul T f.U) f.A) :=
  Fraction.magnitudes.le_trans (Fraction.le_equiv_right hb he)
    (Fraction.add_le_add_left
      (Fraction.mul_le_mul_nonnegative_left hf (duration (Fraction.ofInt 1) j) (by change (0 : Int) ≤ 1;
        ↪ omega)) _)
obtain (_,_,hA,hU,_) := frame_nonnegative w T s hT
have hsmall := duration_le_window T hT m
have hunit := duration_le_window (Fraction.ofInt 1) (by decide) j
have hd' : Fraction.le (pointDistance z (linearField w (a.approx j).1))
  (Fraction.add (Fraction.mul T f.U) f.A) := Fraction.magnitudes.le_trans hd (Fraction.add_le_add
    (Fraction.mul_le_mul_nonnegative hsmall f.U hU)
    (Fraction.le_equiv_right (Fraction.mul_le_mul_nonnegative hunit f.A hA)
      (by simp only [Fraction.equiv, Fraction.mul, Fraction.ofInt, Int.one_mul, Int.mul_one])))
have hz := Fraction.magnitudes.le_trans (pointNorm_le_distance_add z (linearField w (a.approx j).1))
  (Fraction.add_le_add hd' hf)
refine {hp,hv,Fraction.le_equiv_right hz ?,hd}
simp only [f, frame, Fraction.equiv, Fraction.add, Int.add_mul, Int.mul_add]
ac_nf

/-- The potential difference is evaluated on the actual constructed endpoint
and the tangent continuation. Its normalized drop has an explicit O(H) bound. -/
theorem normalized_potential_cell_bound (mass w T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) :
  Within
    (normalizedStepValue (Fraction.mul mass w).half (duration T m) ht
      (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))
      (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1))))
    (forceEnergyValue mass (HarmonicCompletedForce.linearValue w
      (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))))
    (Fraction.mul (duration T m) (coefficient mass w T s)) := by
  let d := HarmonicGeneralTime.conditions w (Fraction.ofInt 1) T s hw (by decide) hT hs
  let a := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m k
  let b := nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T (Fraction.ofInt 1) w.abs
    (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m (k+1)
  have hval (i : Nat) : realize (nodeName (harmonicOracle w hw) (Fraction.ofInt 1) T
    (Fraction.ofInt 1) w.abs (HarmonicGeneralEndpoint.harmonicBound w s) s (by decide) d m i) =
    HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m i) := by
    rw [node_value, HarmonicGeneralTime.harmonic_value_eq]
  rw [← hval k, ← hval (k+1), ← force_work_eq_energy]
  change NameBound (normalizedStepName (Fraction.mul mass w).half (duration T m) ht a b)

```



```

    (workName mass w a) _
  let f := frame w T s
  let Q := roundingCoefficient (Fraction.mul mass w).half f.P f.A
  obtain (hP,hV,hA,hU,hZ) := frame_nonnegative w T s hT
  have hQ : 0 ≤ Q.num := Fraction.nonnegative_mul _ _ (Fraction.abs_num_nonnegative _
    (Fraction.nonnegative_mul _ _ hP hA))
  apply SampledValues.nameBound_of_vanishing_error _ _ _ (fun j => Fraction.mul (duration (Fraction.ofInt
    ↪ 1) j) Q)
  · intro eps heps
    obtain (N,hN) := duration_eventually_small Q eps hQ heps
    refine (N,fun j hj => Fraction.magnitudes.lt_of_le_lt (Fraction.le_of_equiv ?_) (hN j hj))
    simp only [duration,Fraction.equiv,Fraction.mul,Fraction.ofInt,Int.one_mul,Int.mul_one]
    ac_nf
  · intro j
    obtain (hp,hv,hz,hd) := node_frame_bounds w T s hw hT hs ht m k hk j
    have hb := uniform_frame_bound (Fraction.mul mass w).half (duration T m) T f.P f.V f.Z f.U
      (duration (Fraction.ofInt 1) j) f.A ht (a.approx j) (b.approx j)
      (linearField w (a.approx j).1) hP hV hZ (duration_le_window T hT m) hp hv hz hd
    have he := Fraction.equiv_trans (stateNorm_equiv (HarmonicDyadic.stateSub_congr
      (normalized_step_approximant (Fraction.mul mass w).half (duration T m) ht a b j)
      (work_approximant mass w a j))) (scalarState_distance _ _)
    exact Fraction.le_equiv_left he hb

/-- Uniform leading-order potential drop on all constructed dyadic cells,
including the last cell: Delta V/H² tends to -mass*dot(a left,a left)/2. -/
theorem normalized_potential_steps_converge (mass w T : Fraction) (s : Point × Point)
  (hw : 0 ≤ w.num) (hT : 0 ≤ T.num) (hs : DyadicSmallTime w T) (ht : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ k, k+1 ≤ blocks m →
    Within
      (normalizedStepValue (Fraction.mul mass w).half (duration T m) ht
        (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k))
        (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m (k+1))))
      (forceEnergyValue mass (HarmonicCompletedForce.linearValue w
        (HarmonicTimeRealization.gammaValue w T s hT hs (nodeTime T hT m k)))) eps := by
  have hC := coefficient_nonnegative mass w T s hT
  obtain (N,hN) := duration_eventually_small (Fraction.mul T (coefficient mass w T s)) eps
    (Fraction.nonnegative_mul _ _ hT hC) heps
  refine (N,fun m hm k hk => within_mono _ _ _ _ ?_)
    (normalized_potential_cell_bound mass w T s hw hT hs ht m k hk)
  apply Fraction.le_equiv_left (b := duration (Fraction.mul T (coefficient mass w T s)) m)
    _ (Fraction.magnitudes.lt_implies_le (hN m hm))
  simp only [duration,Fraction.equiv,Fraction.mul]
  ac_nf

/-- A finite quadratic prediction still has a nonzero potential remainder.
This rational polynomial control is not an independently computed orbit. -/
theorem quadratic_prediction_control :
  let h : Fraction := (1,2,by decide)
  let p : Point := (Fraction.ofInt 1,Fraction.ofInt 0)
  let v : Point := (Fraction.ofInt 0,Fraction.ofInt 1)
  let c : Fraction := (1,2,by decide)
  let q := QuadraticEstimates.quadraticPosition h (p,v) (linearField (Fraction.ofInt 1) p)
  Fraction.equiv (durationDifference (quadratic c (pointAdd p (pointScale h v))) (quadratic c q))
    ((-15,128,by decide) : Fraction) ∧
  - Fraction.equiv (durationDifference (quadratic c (pointAdd p (pointScale h v))) (quadratic c q))
    ((-1,8,by decide) : Fraction) := by decide

```

end NewtonLimitDynamics.Diagnostic.ConstructedHarmonicPotential

NewtonLimitDynamics/Polygon/GeneralForceTangentTriangle.lean, 3 theorems, 117 lines

```

import NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants
import BarrowLib.Polygon.TangentTriangleValues

```

```

/--! Leading signed doubled tangent-deflection triangle on the actual
constructed sampled Lipschitz central-force curve. Completed determinants
transfer the proved second-order position bound, without assuming an area
expansion. The triangle is distinct from the matched region and its D_mesh. -/

```

namespace NewtonLimitDynamics.Polygon.GeneralForceTangentTriangle

```

open NewtonLimitDynamics
open TimeSubdivision PointBounds HarmonicDyadic HarmonicTimeRealization
open ForceClasses GeneralForceEndpoint GeneralForcePrefix GeneralForceTime GeneralForceSecants
open CauchyValues PositionValues SecantValues PairingValues TangentTriangleValues DyadicNodes

def coefficient (T L B : Fraction) (s : Point × Point) : Fraction :=
  Fraction.mul L (Fraction.mul (velocityCap T B s) (velocityCap T B s))

theorem coefficient_nonnegative (T L B : Fraction) (s : Point × Point)
  (hT : 0 ≤ T.num) (hL : 0 ≤ L.num) (hB : 0 ≤ B.num) :
  0 ≤ (coefficient T L B s).num :=
  Fraction.nonnegative_mul _ _ hL (Fraction.nonnegative_mul _ _
    (velocityCap_nonnegative T B s hT hB) (velocityCap_nonnegative T B s hT hB))

/-- Actual completed triangle/H³ differs from det(v_left,a_left)/2 by
at most H*L*V², including the cell ending at the full endpoint. -/
theorem cell_triangle_bound (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (m k : Nat) (hk : k+1 ≤ blocks m) :
  Within
    (normalizedTriangleValue (duration T m) hT
      (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
      (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1))))
    (secantValue (Fraction.ofInt 1).half
      (pairingValue detForm
        (velocityValue (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
          (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
            ↪ d.time_nonnegative m k))
            (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))))
          (embed (zeroPoint,zeroPoint))))
      (Fraction.mul (duration T m) (coefficient T L B s)) := by
  rw [normalized_triangle_identity]
  have hs := GeneralForceQuadraticSecants.cell_second_secant_bound o E0 T tau L B s hE d hT m k hk
  rw [GeneralForceQuadraticSecants.cellSecondSecant] at hs
  rw [GeneralForceAccelerationSecants.node_force_value o E0 T tau L B s hE d m k] at hs
  rw [← node_value o E0 T tau L B s hE d m k,
    ← node_value o E0 T tau L B s hE d m (k+1)] at hs
  rw [CompletedForce.forceValue_realize o E0 L hE d.lipschitz
    (nodeName o E0 T tau L B s hE d m k) (node_region o E0 T tau L B s hE d m k)] at hs
  let a := nodeName o E0 T tau L B s hE d m k
  let b := nodeName o E0 T tau L B s hE d m (k+1)
  let v := mapName velocityState velocity_nonexpansive a
  let z := secantName (Fraction.mul (Fraction.ofInt 2) (TimeCalibration.inverse (duration T m) hT))
    (secantName (TimeCalibration.inverse (duration T m) hT) b a) v
  let f := CompletedForce.forceName o E0 L hE d.lipschitz a (node_region o E0 T tau L B s hE d m k)
  let V := velocityCap T B s
  let R := Fraction.mul (Fraction.ofInt 2) (Fraction.mul L (Fraction.mul (duration T m) V))
  have hV := velocityCap_nonnegative T B s d.time_nonnegative d.bound_nonnegative
  have hv : ∀ j : Nat, 0 ≤ j → Fraction.le (pointNorm (v.approx j).1) V := by
    intro j
    change Fraction.le (pointNorm (a.approx j).2) V
    rw [node_approx o E0 T tau L B s hE d m k (by omega) j]
    apply GeneralForcePrefix.count_velocity o E0 T tau L B s hE d (m+j) (k*blocks j)
    rw [blocks_add]
    exact Nat.mul_le_mul_right (blocks j) (by omega)
  have hz : NameBound z f R := hs
  have hp := pairing_name_bound_right detForm v z f V R hV 0 hv hz
  let q := (Fraction.ofInt 1).half
  have hq : 0 ≤ q.num := by decide
  have hb := nameBound_scale
    (secantName q (pairingName detForm v z) (constantName (zeroPoint,zeroPoint)))
    (secantName q (pairingName detForm v f) (constantName (zeroPoint,zeroPoint)))
    (pairingName detForm v z) (pairingName detForm v f) q.abs (Fraction.mul R V)
    (Fraction.abs_num_nonnegative q) (fun j => by
      have hlevel := secant_distance_bound q
        ((pairingName detForm v z).approx j) (zeroPoint,zeroPoint)
        ((pairingName detForm v f).approx j) (zeroPoint,zeroPoint)
      apply Fraction.le_equiv_right hlevel
      apply Fraction.equiv_trans (Fraction.mul_equiv (Fraction.equiv_refl _))

```

```

      (Fraction.add_equiv (Fraction.equiv_refl _) (FiniteEstimates.stateDistance_self_zero _)))
    simp only [Fraction.equiv, Fraction.mul, Fraction.add, Fraction.ofInt,
      Int.zero_mul, Int.mul_zero, Int.zero_add, Int.add_zero, Int.one_mul, Int.mul_one]
    ac_nf) hp
  apply nameBound_mono _ _ _ (Fraction.le_of_equiv ?_) hb
  rw [Fraction.abs_eq_of_nonnegative q hq]
  simp only [R, V, q, coefficient, Fraction.equiv, Fraction.mul, Fraction.half, Fraction.ofInt,
    Int.mul_one, Int.one_mul]
  ac_nf

/-- Uniform leading signed doubled triangle on all constructed dyadic cells.
Unsigned triangle area has half the absolute doubled-area value; lobe and
matched-region area require their own geometric identifications. -/
theorem normalized_triangles_converge (o : CentralOracle) (E0 T tau L B : Fraction)
  (s : Point × Point) (hE : 0 < E0.num)
  (d : GeneralForcePrefix.Conditions o E0 T tau L B s hE) (hT : 0 < T.num)
  (eps : Fraction) (heps : 0 < eps.num) :
  ∃ N : Nat, ∀ m, N ≤ m → ∀ k, k+1 ≤ blocks m →
    Within
      (normalizedTriangleValue (duration T m) hT
        (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))
        (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m (k+1))))
      (secantValue (Fraction.ofInt 1).half
        (pairingValue detForm
          (velocityValue (gammaValue o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k)))
          (CompletedForce.forceValue o E0 L hE d.lipschitz (gammaValue o E0 T tau L B s hE d (nodeTime T
            ↪ d.time_nonnegative m k))
            (gamma_admissible o E0 T tau L B s hE d (nodeTime T d.time_nonnegative m k))))
          (embed (zeroPoint, zeroPoint))) eps := by
  have hC := coefficient_nonnegative T L B s d.time_nonnegative d.lipschitz.1 d.bound_nonnegative
  obtain (N, hN) := duration_eventually_small (Fraction.mul T (coefficient T L B s)) eps
  (Fraction.nonnegative_mul _ _ d.time_nonnegative hC) heps
  refine (N, fun m hm k hk => within_mono _ _ _ ?_
    (cell_triangle_bound o E0 T tau L B s hE d hT m k hk))
  apply Fraction.le_equiv_left (b := duration (Fraction.mul T (coefficient T L B s)) m)
  _ (Fraction.magnitudes.lt_implies_le (hN m hm))
  simp only [duration, Fraction.equiv, Fraction.mul]
  ac_nf

end NewtonLimitDynamics.Polygon.GeneralForceTangentTriangle

```

BarrowLib.lean, 0 theorems, 55 lines

```

import BarrowLib.Common.Quadratic
import BarrowLib.Common.RationalMagnitudes
import BarrowLib.Common.FiniteGrowth
import BarrowLib.Polygon.PointAlgebra
import BarrowLib.Polygon.PointBounds
import BarrowLib.Polygon.FiniteEstimates
import BarrowLib.Polygon.FiniteAccumulation
import BarrowLib.Polygon.BoundedIteration
import BarrowLib.Polygon.TriangleBounds
import BarrowLib.Polygon.ConvexCover
import BarrowLib.Polygon.DyadicArithmetic
import BarrowLib.Polygon.FinitePower
import BarrowLib.Polygon.StateDistance
import BarrowLib.Polygon.EndpointCauchyName
import BarrowLib.Polygon.CauchyValues
import BarrowLib.Polygon.GeometricTail
import BarrowLib.Polygon.BinaryTime
import BarrowLib.Polygon.PositionValues
import BarrowLib.Polygon.ScaledTolerance
import BarrowLib.Polygon.AffineValues
import BarrowLib.Polygon.IntegerRefinement
import BarrowLib.Polygon.BinaryCells
import BarrowLib.Polygon.AffineBoundary
import BarrowLib.Polygon.IntegerSchedule
import BarrowLib.Polygon.FiniteRecurrence
import BarrowLib.Polygon.IntegerTime
import BarrowLib.Polygon.FiniteAddress
import BarrowLib.Polygon.FiniteFactorProducts

```

```

import BarrowLib.Polygon.TimeCalibration
import BarrowLib.Polygon.CalibratedGrowth
import BarrowLib.Polygon.EquivalentDuration
import BarrowLib.Polygon.CalibratedRefinement
import BarrowLib.Polygon.ConvexValues
import BarrowLib.Polygon.CompletionGeometry
import BarrowLib.Polygon.SquareOuterContent
import BarrowLib.Common.RationalExhaustion
import BarrowLib.Polygon.RationalIntervals
import BarrowLib.Polygon.ScalarOrder
import BarrowLib.Polygon.BoundedCuts
import BarrowLib.Polygon.SquareContentValues
import BarrowLib.Polygon.FiniteSequenceGap
import BarrowLib.Polygon.BinaryEndpoints
import BarrowLib.Polygon.TailValues
import BarrowLib.Polygon.KinematicEstimates
import BarrowLib.Polygon.SecantValues
import BarrowLib.Polygon.DyadicNodes
import BarrowLib.Polygon.SampledValues
import BarrowLib.Polygon.AccelerationEstimates
import BarrowLib.Polygon.PolygonValues
import BarrowLib.Polygon.MatchedRegion
import BarrowLib.Polygon.QuadraticEstimates
import BarrowLib.Polygon.QuadraticSecants
import BarrowLib.Polygon.PairingValues
import BarrowLib.Polygon.QuadraticPotentialValues
import BarrowLib.Polygon.TangentTriangleValues

```

NewtonLimitDynamics.lean, 0 theorems, 86 lines

```

import NewtonLimitDynamics.DeMotu1684.QuadraticDeflection
import NewtonLimitDynamics.Principia1687.LemmaX
import NewtonLimitDynamics.Principia1713.LemmaX
import NewtonLimitDynamics.Common.RationalMagnitudes
import NewtonLimitDynamics.Common.FiniteGrowth
import BarrowLib
import NewtonLimitDynamics.Principia1713.ForceComparison
import NewtonLimitDynamics.Principia1687.ConstructedRatio
import NewtonLimitDynamics.Polygon.Finite
import NewtonLimitDynamics.DeMotu1684.AreaLaw
import NewtonLimitDynamics.Principia1687.PropositionI
import NewtonLimitDynamics.Principia1713.PropositionI
import NewtonLimitDynamics.Polygon.Enclosure
import NewtonLimitDynamics.Polygon.Contact
import NewtonLimitDynamics.Polygon.RefinementStrip
import NewtonLimitDynamics.Polygon.PathDefect
import NewtonLimitDynamics.Polygon.TimeSubdivision
import NewtonLimitDynamics.Polygon.PartitionControl
import NewtonLimitDynamics.Polygon.ZeroForce
import NewtonLimitDynamics.Polygon.InertialControl
import NewtonLimitDynamics.Polygon.InertialDefect
import NewtonLimitDynamics.Polygon.PartialCell
import NewtonLimitDynamics.Polygon.UniformRefinement
import NewtonLimitDynamics.Polygon.PartitionComparison
import NewtonLimitDynamics.Polygon.Converse
import NewtonLimitDynamics.Polygon.RelativeMotion
import NewtonLimitDynamics.Polygon.StripArea
import NewtonLimitDynamics.Polygon.CentralSchedule
import NewtonLimitDynamics.Polygon.ForceClasses
import NewtonLimitDynamics.Polygon.HarmonicStability
import NewtonLimitDynamics.Polygon.HarmonicRefinement
import NewtonLimitDynamics.Polygon.PointBounds
import NewtonLimitDynamics.Polygon.TriangleBounds
import NewtonLimitDynamics.Polygon.HarmonicComparison
import NewtonLimitDynamics.Polygon.HarmonicAccumulation
import NewtonLimitDynamics.Polygon.HarmonicUniform
import NewtonLimitDynamics.Polygon.ConvexCover
import NewtonLimitDynamics.Polygon.HarmonicCover
import NewtonLimitDynamics.Polygon.HarmonicDyadic
import NewtonLimitDynamics.Polygon.HarmonicTimeComparison
import NewtonLimitDynamics.Polygon.HarmonicBinaryPrefix

```

```

import NewtonLimitDynamics.Polygon.CauchyValues
import NewtonLimitDynamics.Polygon.BinaryTime
import NewtonLimitDynamics.Polygon.HarmonicTimeRealization
import NewtonLimitDynamics.Polygon.PositionValues
import NewtonLimitDynamics.Polygon.MonotoneEnclosure
import NewtonLimitDynamics.Contact.Bounds
import NewtonLimitDynamics.Contact.FiniteSums
import NewtonLimitDynamics.Comparison.Routes
import NewtonLimitDynamics.Comparison.CircleCompare
import NewtonLimitDynamics.Contact.AreaCoefficient
import NewtonLimitDynamics.Diagnostic.InverseCubeAreal
import NewtonLimitDynamics.Diagnostic.PhaseArea
import NewtonLimitDynamics.Diagnostic.DeflectionPotential
import NewtonLimitDynamics.Polygon.HarmonicConstructionAgreement
import NewtonLimitDynamics.Polygon.HarmonicPolygonCurve
import NewtonLimitDynamics.Polygon.HarmonicIntegerSubdivision
import NewtonLimitDynamics.Polygon.HarmonicDyadicAgreement
import NewtonLimitDynamics.Polygon.CalibratedForces
import NewtonLimitDynamics.Polygon.GeneralForcePrecision
import NewtonLimitDynamics.Polygon.GeneralForceEndpoint
import NewtonLimitDynamics.Polygon.HarmonicGeneralEndpoint
import NewtonLimitDynamics.Polygon.GeneralForcePrefix
import NewtonLimitDynamics.Polygon.GeneralForceGrowth
import NewtonLimitDynamics.Polygon.RegionConfinement
import NewtonLimitDynamics.Polygon.GeneralForceTime
import NewtonLimitDynamics.Polygon.HarmonicGeneralTime
import NewtonLimitDynamics.Polygon.GeneralForceSecants
import NewtonLimitDynamics.Polygon.HarmonicSecants
import NewtonLimitDynamics.Polygon.CompletedForce
import NewtonLimitDynamics.Polygon.HarmonicCompletedForce
import NewtonLimitDynamics.Polygon.GeneralForceAccelerationSecants
import NewtonLimitDynamics.Polygon.HarmonicAccelerationSecants
import NewtonLimitDynamics.Polygon.GeneralForcePolygonCurve
import NewtonLimitDynamics.Polygon.HarmonicGeneralPolygon
import NewtonLimitDynamics.Polygon.HarmonicPathRegion
import NewtonLimitDynamics.Polygon.HarmonicPathContent
import NewtonLimitDynamics.Polygon.GeneralForcePathRegion
import NewtonLimitDynamics.Polygon.GeneralForcePathContent
import NewtonLimitDynamics.Polygon.HarmonicGeneralPathContent
import NewtonLimitDynamics.Polygon.ParallelQuadraticEndpoint
import NewtonLimitDynamics.Diagnostic.QuadraticEndpointPotential
import NewtonLimitDynamics.Polygon.GeneralForceQuadraticSecants
import NewtonLimitDynamics.Polygon.HarmonicQuadraticSecants
import NewtonLimitDynamics.Diagnostic.ConstructedHarmonicPotential
import NewtonLimitDynamics.Polygon.GeneralForceTangentTriangle

```